基于集成学习与条件 Copula 的啤酒风味预测及受约束风格迁移

摘 要

针对啤酒化学测量存在缺失、成分相关性强及部分风格样本较少的问题，研究感官预测、关键化合物分析和风格迁移。按啤酒编号核对附件后，采用 250 款啤酒、45 项感官属性；第一、二问使用 231 维化学输入，第三问使用 69 项校准量与 157 项相对量。现有感官标签为标准化连续值，无法从附件恢复原始 0—6 评分，故在给定尺度上建模和评价。

第一问采用训练折中位数补值、缺失指示与多输出 ExtraTrees，五折折外 MAE 为 0.3409、RMSE 为 0.4862、宏平均 R^2为 0.2146，RMSE 较均值基线降低 15.36%；额外随机隐藏 20%测试特征后，RMSE 增加 2.67%。第二问复核附件相关矩阵后采用 Spearman 相关，在绝对相关阈值 0.7 下得到 231 个节点、109 条边，并为 45 项感官属性选出 135 个 Top-3 关联位置。200 次 Bootstrap 中仅 14 个位置的入选频率达到 0.8，去风格均值后 21 项感官的 Top-3 完全变化。

第三问比较收缩 Copula、局部边缘 Copula 和平滑混合，通过嵌套验证选择生成方法。Kriek、Lambic 的能量差异较基准分别降低 20.5%、23.4%，Tripel 略有变差；三种目标风格共生成 900 条化学谱并预测感官画像。Pils/Lager 向 Tripel 迁移时，同时限制四项缺陷浓度和 Stale hops 预测增量。固定树、中央候选池的余量优先方案有 10/11 款通过模型检查，所选谱目标距离缩短中位数为 57.74%；几何优先的探索性方案为 70.08%。由于 Stale hops 跨风格预测仍弱，上述结果有待新样本检测和盲品验证。

关键词：啤酒风味；多输出预测；关联网络；条件 Copula；受约束迁移

目 录

1 问题重述.....1
2 数据核验与统一处理.....1
3 问题分析与技术路线.....2
4 模型假设与符号.....3
5 缺失条件下的感官预测.....4
5.1 预测目标与评分量尺.....4
5.2 折内缺失处理与多输出 ExtraTrees.....5
5.3 折外验证与模型比较.....5
5.4 缺失压力与极端外推诊断.....6
5.5 逐维表现与应用边界.....7
5.6 补充对照与未见风格验证.....7
6 化合物关联网络与关键候选.....8
6.1 Spearman 关联网络.....8
6.2 感官 Top-3 候选与重抽样稳定性.....9
6.3 方法与风格敏感性.....10
6.4 化学依据与解释边界.....11
6.5 折外置换与预测依赖.....12
7 风格条件生成与受约束迁移.....13
7.1 混合化学表示与高斯秩映射.....13
7.2 共享信息的条件 Copula 基准.....14
7.3 局部边缘 Copula 与平滑混合.....14
7.4 嵌套选择、生成质量与感官画像.....14
7.5 距离、支持域与中央候选池.....16
7.6 化学阈值与最低改善要求.....17
7.7 Stale hops 预测及 beer 81 误差诊断.....17
7.8 配对预测筛选与留出模型复核.....18
7.9 两种排序的迁移结果.....18
7.10 方案选择与后续验证.....19
8 结果检验与模型评价.....20
9 结论与建议.....21
使用声明.....21
参考文献.....21
附录 A 感官字段及逐维精度.....23
附录 B 完整 Top 3 候选.....25
附录 C 逐源迁移检查.....45
附录 D 软件环境与运行方法.....48
附录 E 完整计算机源程序.....50

1 问题重述

啤酒的香气与风味由多类化合物共同形成，原料、发酵和陈化过程都会改变其组成。化学成分与感官强度并非一一对应，混合物作用及风格差异增加了预测和解释的难度。本题以原研究的商用啤酒化学测量、训练品评及相关数据[1]为基础，要求完成评分预测、关键化合物分析和风格生成。

第一问保留化学测量存在缺失的样本，预测 Aroma 与 Flavour 属性。题目要求 0—6 评分，而实际监督标签已标准化，需先核实评分量尺。第二问利用化合物相关矩阵构建网络，为每项感官属性选择 Top-3 候选，并检验其稳定性和化学依据。

第三问生成 Tripel、Kriek 和 Lambic 的化学谱及感官画像，再将 Pils/Lager 向 Tripel 迁移，限制 DMS、Diacetyl、4VG、trans-2-nonenal 和 Stale hops 等缺陷恶化。评价包括生成分布质量、目标风格接近度和逐款约束检查。

2 数据核验与统一处理

2.1 样本范围与感官目标

对附件逐表读取并按 beer 核对，附件 1、4、7 均包含 250 个唯一啤酒编号，覆盖 22 个风格。附件 1 除三个身份或类别字段外有 231 个数值特征；附件 4 有 50 项感官属性，依据附件 8 的名称字典，选取 Aroma 15 项与 Taste 30 项，共 45 项作为本题目标，排除 alcohol、aftertaste、body、co2 和 overall。Tripel、Kriek、Lambic 和 Pils/Lager 分别有 28、10、11 和 11 款。题面背景的 226 款与实际文件不一致，本文以可一一核对的 250 款为计算样本，不按文字数量截断。

表 2-1 主要附件的实际范围及用途

附件	实际数据范围	本文用途
1.csv	250 款 × 231 个数值特征	第一问输入与第二问候选分析
2.csv	231 × 231 相关矩阵	化合物关联网络与数值复核
3.xlsx	12 张评分卡及汇总公式	确定评分规则与名称映射
4.csv	250 款 × 50 个感官属性	筛选 45 维连续监督目标
5.csv 与 6.csv	50 × 50 及 44 × 50	描述性矩阵核验，不作折外筛选先验
7.csv	250 款 × 186 个描述词变量	辅助诊断，未作为生成谱的预测输入
8.xlsx	校准量、阈值及名称等工作表	字段、单位与化学阈值核实

按 beer 进行一对一连接。附件 4、7 的条码均只剩 96 个不同值，不能作为主键；编号和条码也不作为模型输入。附件 3 的 12 张评分卡均未填写，汇总公式的缓存零值不算实测评分。

2.2 缺失值、尺度与异常值

附件 1 有 73 个化学缺失单元，占 250×231 的 0.1264%，分布于 9 列、63 款酒（25.2% 样本），最高单列缺失率为 10.8%。保留全部 250 款，仅用训练数据估计补值参数。45 项感官目标均无缺失。化学表包含对数或相对测量及理化量，负值不能一律删除，各列也不能统一取指数还原浓度。

与附件 8 校准量对照，37 个基础字段在有效比较范围内符合以 10 为底的对数关系，零值曾以最小正值的一半替代；其余重叠 GC-MS 字段不满足统一直接 log10 关系。第一、二问保留给定数值表示；第三问删除五项汇总或派生量，并以 69 项匹配校准字段替换，形成 69 维校准量与 157 维相对量的 226 维混合表示。校准数据中的 3 个负值转为缺失，混合表共保留 72 个缺失单元。相对量没有统一浓度单位，化学阈值只在核实过的校准字段上执行。

极端观测通过定位和模型对照处理。beer 156 的输入极值影响线性外推，beer 81 的高 Stale hops 评分难以由化学近邻解释，均未发现足以删除记录的证据。原始附件保留不变，处理过程和结果来源另存核验记录。

2.3 相关矩阵及信息边界

对附件 2 逐对复算 Spearman 相关，最大绝对差为 6.66×10^{-16}；与 Pearson 的差异明显。因此按实物矩阵采用 Spearman，而不沿用题面对 Pearson 的文字称谓。附件 5、6 也可由对应样本的 Spearman 相关复现。附件 6 实际仅 44 个化学指标，不代替完整化学数据。

附件 2、5、6 均由本批样本导出，用于描述性分析；预测交叉验证前不使用这些矩阵或附件 8 的 GBM 重要性、SHAP 筛选变量。论坛表缺少评论全文、时间和批次信息，无法补充真实品评标签。

3 问题分析与技术路线

三问使用同一套样本键和字段字典，评价对象各有不同：第一问检验真实啤酒的折外预测误差；第二问分析本批样本的关联与风格混杂；第三问检验生成分布，再搜索满足约束的迁移谱。

预测采用多输出树，并与均值和岭回归比较；解释采用可直接复算的秩相关；生成采用经验边缘与收缩依赖结构。迁移先检查化学浓度与训练支持范围，再检查预测缺陷变化，比较目标接近度和保护约束之间的取舍。

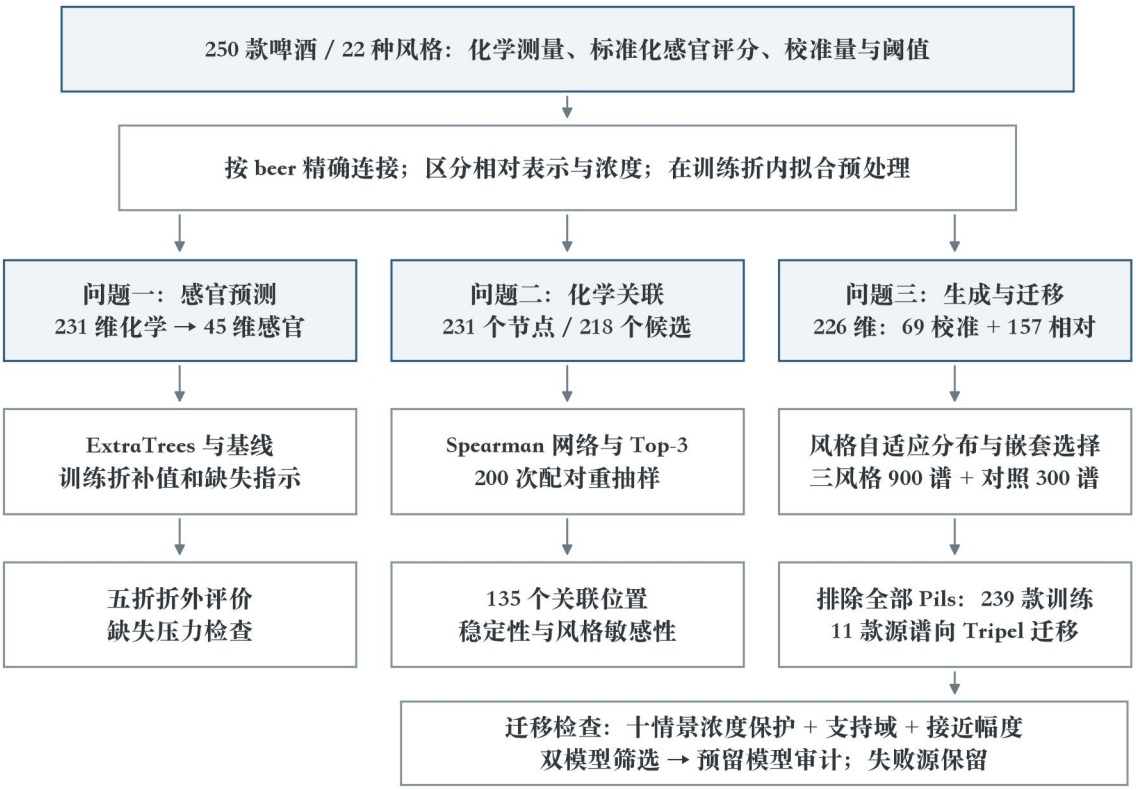


图 1 三问的数据处理、模型与验证流程。第一问使用 231 维原表示；第三问使用 69 维校准量与 157 维相对量组成的 226 维混合表示。迁移训练排除全部 11 款 Pils。

第一问采用随机五折；生成方法在训练集内选择，再由外层留出数据评价；Stale hops 按非 Pils 风格分组验证。Pils 标签已多次用于开发诊断，不能再视为独立测试集。迁移失败的源酒始终保留在通过率分母中。

4 模型假设与符号

4.1 建模假设及影响

假设一，现有风格混合中的啤酒可作为随机五折的评价单位，训练与测试按编号隔离。不同批次或全新风格的预测能力需另行验证。

假设二，附件的 45 维连续感官值在本批样本间可比。预测沿用该标准化尺度；缺少逐人评分及转换参数时，不推定其与原始 0—6 分存在已知映射。

假设三，经验边缘和收缩秩相关可近似小样本风格分布，通过留出分布距离、零频率、方差和近邻指标检验；生产可行性另需实测。

假设四，附件阈值可作为化学缺陷的参考。用阈值均值、中位数及扰动情景检查敏感性；Stale hops 没有单一化学对应项，另用感官预测约束。

4.2 主要符号

表 4-1 主要符号及含义

符号	含义	尺度或范围
n，p，K	样本数、化学维数、感官维数	n=250；p=231 或 226；K=45
x_i，y_i，s_i	化学向量、感官向量、风格标签	化学量按字段；感官为标准化值
T，m_j,T，d_{ij}	训练集、训练中位数、缺失指示	$d_{ij}\in\{0,1\}$
ρ_{jl}，τ，π_{jk}	秩相关、网络阈值、入选频率	$\tau\in\{0.6,0.7,0.8\}$；$\pi\in[0,1]$
F_j，Φ，z(x)	经验边缘、标准正态 CDF、高斯秩向量	无量纲
μ_s，Σ_s，R_s	风格秩中心、协方差、相关矩阵	秩空间参数
c_d，T_d，e_d	缺陷浓度、感官阈值、超阈程度	浓度与阈值单位一致；e_d 无量纲
d(x)，$r_i(x)$，$G_i(x)$	目标距离、修改幅度、距离改善率	秩空间距离；G 为相对改善
f_b，Δ_b，$Q_{0.95}$	第 b 个代理、配对预测差、95%分位数	标准化感官尺度

5 缺失条件下的感官预测

5.1 预测目标与评分量尺

以 250 款酒的 231 维化学表示预测 15 项 Aroma 和 30 项 Taste 属性。按 beer 连接化学与感官表，编号、条码及风格标签不作为输入。保留全部样本，在训练管线中处理化学缺失值。

部分描述符在原评分卡上以勾选记录，选中后继承所属类别的总体强度；经标准化和汇总，附件标签均为连续值，因此采用回归。

题目要求预测 0—6 等级，但附件 4 提供的是标准化连续值。原研究先按品评员中心化、除以标准差，再汇总[1]。设 s_{itk}为啤酒 i、品评员 t、属性 k 的原评分，T_i为参与该酒品评的人员集合，则

$$y_{ik}=\frac{1}{|T_i|}\sum_{t\in T_i}\frac{s_{itk}-\mu_{tk}}{\sigma_{tk}}. \qquad (5\text{-}1)$$

附件未提供逐人评分、μ_{tk}、σ_{tk}及完整参与关系，附件 3 的评分输入也均为空。若在允许范围内令 $s'=a_ks+b_k$、$\mu'=a_k\mu+b_k$、$\sigma'=a_k\sigma$，$a_k>0$，式（5-1）不变，原量尺的原点与单位却可改变。仅凭最终均值、标准差或[0,6]范围不能确定逆变换。因此本问输出可核查的 45 维连续预测，并在附件尺度计算误差；原始 0—6 等级尚未完成。可信校准需要与

本批 beer 编号对应的真实评分，或逐人评分、转换参数和参与记录，不能用分位分级或线性拉伸代替。

5.2 折内缺失处理与多输出 ExtraTrees

设外层训练集为 T，d_{ij}=1 表示第 j 个化学值缺失，d_{ij}=0 表示已观测，$m_{j,T}$ 为训练集该列已观测值的中位数。采用中位数补值并附加缺失指示，使模型区分实测中位水平与缺失后填入的中位水平：

$$\tilde{x}_{ij} = x_{ij}\ (d_{ij} = 0),\ \tilde{x}_{ij} = m_{j,T}\ (d_{ij} = 1). \tag{5-2}$$

仅对训练时出现缺失的列 J_T 附加指示，训练中全空的列按 0 保留。测试时使用训练所得中位数和指示列，不重新拟合；树模型不再标准化化学特征。

补值保留对极端观测的稳健性，缺失指示保留检测缺失的信息。两者均在划分后估计。

采用多输出极端随机树集成 ExtraTrees[2]。每个节点随机抽取特征和候选阈值，以子节点损失之和最小为准划分；各输出共享树结构，叶节点保存目标均值。对节点样本集 A，45 维联合平方误差定义为

$$\mathcal{L}(A) = \sum_{i\in A}\sum_{k=1}^{45}(y_{ik} - \bar{y}_{Ak})^2. \tag{5-3}$$

$$\hat{y}_k(x) = \frac{1}{B}\sum_{b=1}^{B}\frac{1}{|L_b(x)|}\sum_{i\in L_b(x)} y_{ik},\ B = 300. \tag{5-4}$$

其中 L_b(x)为样本 x 在第 b 棵树中的叶节点。固定 300 棵树、叶节点至少 3 个样本，每次划分随机使用 70%的输入特征。叶均值和集成平均可减少极端线性外推。

45 个输出共享划分，不逐目标调参。汇总后的目标方差并不完全相同，因此联合训练的权重与逐维等权评价不同，需同时报告总体和逐维误差。

5.3 折外验证与模型比较

固定随机种子 20261023，将 250 款酒随机分为五折，每次 200 款训练、50 款测试，合并每款酒仅在未参与训练时得到的折外预测。均值基线只使用训练折目标均值。普通岭回归[3]在补值后标准化输入与输出，并在各外层训练集内用四折选择 $\alpha\in\{0.1,1,10,100,1000,10^4,10^5,10^6\}$；补值和标准化在每个内层训练折重新拟合。ExtraTrees 参数固定，不参加上述内层搜索。

$$\mathrm{RMSE} = \sqrt{\frac{1}{250\times 45}\sum_{i=1}^{250}\sum_{k=1}^{45}(y_{ik} - \hat{y}_{ik}^{\mathrm{OOF}})^2}. \tag{5-5}$$

$$R^2_{\mathrm{macro}} = \frac{1}{45}\sum_{k=1}^{45}\left[1 - \frac{\sum_i (y_{ik} - \hat{y}_{ik}^{\mathrm{OOF}})^2}{\sum_i (y_{ik} - \bar{y}_k)^2}\right]. \tag{5-6}$$

表 5-1 45 维标准化感官预测的折外指标

模型	MAE	RMSE	宏平均 R^2
训练折均值	0.409634	0.574456	-0.007574
普通岭回归	0.406092	0.589966	-0.065285
ExtraTrees	0.340851	0.486220	0.214638
1%/99%裁剪岭回归	0.352513	0.499179	0.185784

ExtraTrees 的 RMSE 较均值降低 15.36%，五个测试折均改善，降幅 11.60%—17.91%。普通岭回归 MAE 略低，但 RMSE 高于均值 2.70%，受少数大误差影响。训练 1%—99%分位裁剪后的岭回归 RMSE 为 0.499179，仍高于 ExtraTrees，故采用后者。裁剪对照在检查原五折误差后提出，复用了该划分，属于探索性比较。

MAE 反映平均绝对偏差，RMSE 更重视大误差，宏平均 R^2使 45 个目标等权。负 R^2表示折外误差大于该目标相对全样本均值的平方离差，不代表分类准确率。

5.4 缺失压力与极端外推诊断

在每折已观测测试特征中额外随机隐藏 10%或 20%，训练模型保持不变。两种遮蔽使用同组随机数，10%掩码是 20%掩码的子集，模型间使用相同掩码。结果见表 5-2。

表 5-2 新增随机缺失下的 RMSE

模型	无新增缺失	隐藏 10%	隐藏 20%	20%相对变化
普通岭回归	0.589966	0.590951	0.564464	-4.32%
ExtraTrees	0.486220	0.490987	0.499218	+2.67%
裁剪岭回归	0.499179	0.507932	0.514142	+3.00%

ExtraTrees 在 20%情景下 RMSE 增加 2.67%，裁剪岭回归增加 3.00%。普通岭回归的下降主要来自 beer 156：其 X2.methyltetrahydrofuran.3.one 为 12596.43287，对应训练折最大值仅 31.25344；acetic 预测为 12.00039，实测为−0.28048，该酒 45 维平均平方误差为 8.651434。

20%掩码恰好隐藏该极端单元格，补值后该酒误差降至 0.721929，其他 249 款的合计误差反而增加。ExtraTrees 和裁剪岭回归对此酒的误差为 0.597805、0.635978。该结果反映极端外推，不能解释为缺失有益；没有证据认定记录错误，仍保留该酒。

裁剪边界在每个内层训练折估计。beer 156 所在外层折中，该特征训练 99%分位数为 19.311161，故能缓和外推；但裁剪也可能压缩真实稀有成分。一次随机遮蔽未覆盖检出限删失或成组仪器故障。

5.5 逐维表现与应用边界

ExtraTrees 有 44 项 R^2为正、29 项超过 0.1；Taste 和 Aroma 宏平均 R^2分别为 0.235783、0.172350。acidity、lactic、acetic、bitternes 的 R^2为 0.708844、0.651877、0.646041、0.553414；A_esters_fruity、stale_hops、dms 仅为−0.010524、0.018091、0.038774，逐维差异明显。

A_esters_fruity 和 stale_hops 的折外 Spearman 相关也仅为 0.015752、0.153261。现有数据不能分离标签稀疏、风格差异与化学覆盖不足的影响，这些弱维度不宜承担强感官保证。

裁剪岭回归在 16 项上优于 ExtraTrees，但不按同批折外结果逐维拼接赢家，以免抬高评价。最终用 250 款拟合 ExtraTrees 供新样本预测，训练内拟合值不计入精度。上述结果适用于现有风格混合，未证明对全新风格的预测能力。附件制作者已做的全局标准化无法撤销，评价以该表示为条件。

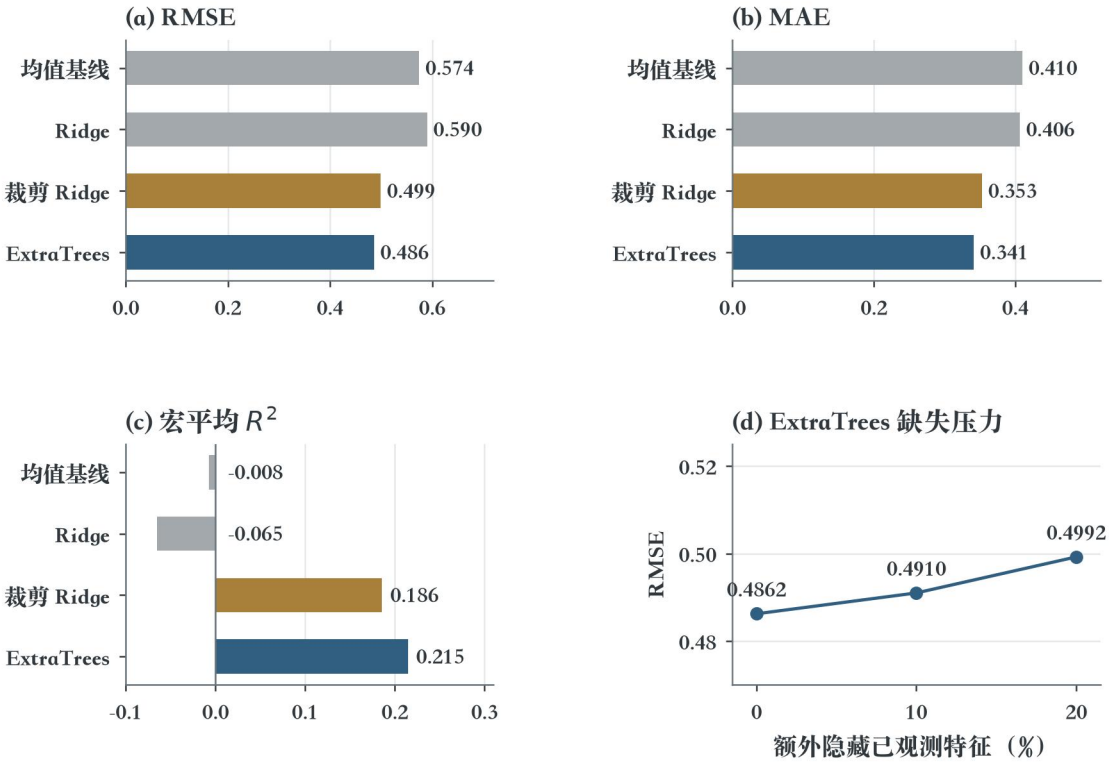


图 2 第一问四种方法的五折折外误差、宏平均 R^2及 ExtraTrees 缺失压力结果。裁剪 Ridge 为稳健性对照；额外隐藏 20%已观测特征后，ExtraTrees 的 RMSE 由 0.4862 增至 0.4992。

5.6 补充对照与未见风格验证

主结果完成后增加固定参数三折对照，用于检查模型家族及部署范围；这是同一批 250 款酒上的补充分析，不作为独立外部测试，也不替换表 5-1 的五折指标。四种模型使用相同随机划分，所有补值均只在训练折估计。ExtraTrees 沿用 300 树、叶节点至少 3 样本及 0.7 特征比例；GBR 逐目标拟合 100 棵深度 2 的树，学习率 0.05、叶节点至少

5 样本；HGB 逐目标迭代 100 次，学习率 0.05、最多 7 叶、叶节点至少 10 样本、正则系数 1。未据此调参或逐目标拼接赢家。

表 5-3 固定参数补充对照与整风格留出

验证	模型	RMSE	宏平均 R^2
随机三折	训练折均值	0.5743	-0.0067
随机三折	ExtraTrees	0.4900	0.2043
随机三折	逐目标 GBR	0.4948	0.1786
随机三折	逐目标 HGB	0.4951	0.1775
整风格三折	训练折均值	0.5878	-0.0461
整风格三折	ExtraTrees	0.5084	0.1516

整风格划分由 GroupKFold 按类别样本数平衡，三个测试折为 83、83、84 款，训练与测试风格互斥。与训练样本量接近的随机三折相比，ExtraTrees 的 RMSE 增加 3.76%。这反映未见风格预测更困难，不能将随机划分下的精度直接推广至新风格。固定参数提升树没有整体优势，故保留 ExtraTrees 主方案。

对同一随机三折模型再施加 20 组配对测试掩码：10%掩码的 RMSE 均值为 0.4956、掩码间标准差为 0.0013；20%掩码的 RMSE 均值为 0.5037、掩码间标准差为 0.0022。这里的波动仅来自测试掩码，不含重新分折或重训的不确定性，也不模拟检出限删失和整组仪器失效。

6 化合物关联网络与关键候选

本问分析全部 250 款啤酒，用化合物共变网络观察冗余和共同来源，再按化合物—感官边际关联提出候选。网络结构、候选稳定性和化学气味证据分别报告，相关清单不作为因果效应或独立测试性能。

6.1 Spearman 关联网络

逐项核对附件 2 与附件 1 重算矩阵后，二者 Spearman 相关最大绝对差为 6.66×10^{-16}，说明附件 2 对应秩相关。设两个化学变量在共同非缺失样本中的平均秩向量为 R_j、R_l，则相关系数及无向邻接矩阵定义为

$$\rho_{jl} = \mathrm{corr}(R_j, R_l),\ a_{jl} = \mathbf{1}(|\rho_{jl}| \geq \tau),\ j \neq l. \qquad (6\text{-}1)$$

并列值取平均秩，缺失值按变量对删除。主阈值τ=0.7 时保留 231 个节点、109 条无向边，边均为正相关，150 个节点孤立。阈值改为 0.6 或 0.8 时，边数为 266 或 39，孤立节点为 101 或 194。节点度数及加权度数反映共变邻居数量与强度；孤立不表示没有气味作用。

全图保留颜色、pH 等综合参数以描述样本结构；Top-3 则排除这类非单一化合物指标。τ只控制图的稀疏程度，不是显著性或气味阈值。

同来源成分可能随原料、发酵和风格共同变化，网络连边不能解释为化学反应或感官贡献。全样本网络只用于本问描述；若用于预测筛选，须在训练折内重建。

6.2 感官 Top-3 候选与重抽样稳定性

从 231 项中排除五项派生汇总量及颜色、pH 等共 13 项综合参数，保留 218 项候选，对 45 个感官目标计算 9810 个关联。每项目标按$|\rho_{jk}|$降序取前三项，保留正负号，并列按附件顺序处理。135 个候选位置涉及 60 种化合物，不以后验文献解释或预计算 GBM/SHAP 改选。

化学—感官相关也使用共同观测样本，附表给出有效样本数。负相关仅表示化学值的秩与感官评分秩反向，不等于抑制作用。相对峰面积及变换值不能直接与浓度阈值比较。

为判断排名对样本扰动的依赖，固定种子进行 B=200 次全样本行 Bootstrap，每次重新计算相关并选择前三项。设$S_k^{(b)}$为第 b 次的入选集合，频率定义为

$$\pi_{jk} = \frac{1}{B}\sum_{b=1}^{B} \mathbf{1}\left(j \in S_k^{(b)}\right). \qquad (6\text{-}2)$$

135 个位置的平均入选频率为 0.455，中位数 0.400，14 个达到 0.8。频率的最大蒙特卡洛标准误约 0.035，小差异不宜过度解释。全样本行重采样没有固定风格比例，也不提供筛选后的显著性保证。

另在 22 种风格内分别有放回抽取原样本数，固定混合比例重做 200 次 Bootstrap。原 135 项的平均频率由 0.455 变为 0.484，达到 0.8 的位置由 14 变为 19。乙酸乙酯对应感官的频率为 0.985，4VG 为 0.890。该补充不改全样本相关前三；固定风格比例仍保留风格之间的差异，不能视为已消除混杂。200 次频率的小幅差别亦受蒙特卡洛误差影响。

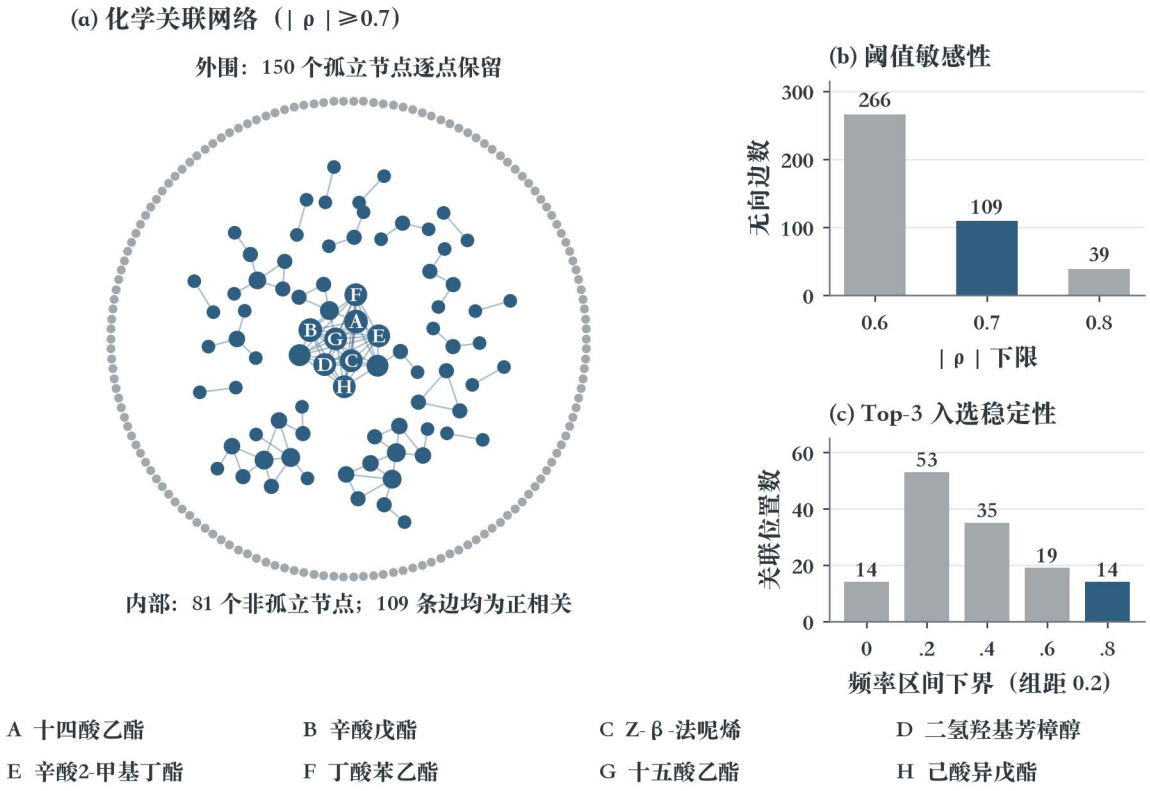


图 3 化学关联网络及稳定性。(a) 实际|Spearman ρ|≥0.7 网络，保留全部 231 节点和 109 条正相关边；81 个非孤立节点采用固定种子 20261023 的分量内力导向布局及分量打包，并进行显示避碰，150 个孤立节点在外围环逐点保留，节点面积随度增加。A—H 为度最高八个节点，图内列简名；布局距离不表示化学浓度或因果关系。(b) 三个阈值下的边数。(c) 135 个 Top-3 位置的 200 次配对重抽样入选频率，其中 14 个达到 0.8；频率区间左闭右开，最后一组包含 1。

6.3 方法与风格敏感性

以 Pearson 替代 Spearman，并对化学与感官变量同时减去所属风格均值后重新计算秩相关。令 s(i)为啤酒 i 的风格，该检查为

$$\tilde{x}_{ij} = x_{ij} - \bar{x}_{s(i),j},\ \tilde{y}_{ik} = y_{ik} - \bar{y}_{s(i),k}. \tag{6-3}$$

用 Jaccard 衡量两份 Top-3 集合的一致性：

$$J(S,T) = \frac{|S \cap T|}{|S \cup T|}. \tag{6-4}$$

Spearman 与 Pearson 的平均 Jaccard 为 0.222；全样本与去风格均值后的平均为 0.160，21 项感官的两份清单完全不重合。候选显著依赖关联定义和风格构成。去均值只移除当前表示下的风格均值，不能控制风格内工艺、品牌等混杂。

22 种风格的样本量介于 3 至 31 款，去均值后的剩余变异也不相同。因此保留敏感性分析中的变化，不据预期机制替换原排名；Bootstrap 频率只说明本批样本构成下的排名稳定性。

表 6-1 既有 Top-3 代表候选的稳定性与证据判断

感官—化合物	全样本ρ	入选频率	去风格ρ	证据判断
酯味指甲油(F)—乙酸乙酯	0.503	0.980	0.270	气味相符[1][4]
酯香香蕉(A)—乙酸异戊酯	0.283	0.650	0.300	气味相符[5]
酒花柑橘香(A)—香茅醇	0.370	0.590	0.281	混合物协同[6]
4VG 感官—4-乙烯基愈创木酚	0.473	0.875	0.428	气味相符[5]
苦味—辛酸乙酯	0.593	0.985	0.465	同源间接关联[1]
麦芽总体味(F)—乙醇	0.472	0.960	-0.011	风格共变标记
酯香香蕉(A)—正己醇	0.340	0.945	0.237	山楂样背景[7]
麦芽焦糖味(F)—乙醛	0.433	0.970	0.142	生青味标准[7]
丁香味—柠檬烯	0.342	0.915	0.340	柑橘背景[8]
橙味—樟脑	0.367	0.865	0.312	花香协同背景[9]
DMS 感官—松油烯-4-醇	-0.196	0.190	-0.126	目标机制未验证
陈旧酒花—2-十一醇	0.300	0.645	0.129	目标机制未验证

6.4 化学依据与解释边界

表 6-1 中，乙酸乙酯的果香/溶剂样气味[1][4]、乙酸异戊酯的香蕉气味及 4VG 的丁香气味[5]与相应描述一致。香茅醇的依据来自与香叶醇、芳樟醇共同存在时的柑橘协同[6]，不能把单变量相关换算为独立贡献。辛酸乙酯—苦味虽有 0.985 的入选频率，原研究也只提出复杂感官联系的可能解释[1]；该研究与附件共享样本，不算外部重复验证。

高入选频率也可能对应不匹配的气味。独立鲜啤 GC-O/AEDA 研究把正己醇描述为山楂样，并以乙醛作为生青味训练标准[7]，没有证明二者分别产生香蕉酯香和麦芽焦糖味。Amarillo 酿造对照中的柠檬烯等成分随柑橘/花香共同改变[8]；芫荽啤酒实验报告樟脑在过量芳樟醇下增强花香[9]。这些结果补充了候选背景，仍不能解释本题柠檬烯—丁香味或樟脑—橙味的直接作用。

对未入选 Top-3 的已知缺陷化合物另按名称列出对照，结果见表 6-2，原候选清单保持不变。DMS 和双乙酰分别具有卷心菜样与黄油样气味；啤酒加标试验还发现乙酸乙酯、乙酸异戊酯和芳樟醇对其感知有不同方向、浓度相关的影响[4]。这些试验说明基质交互确实存在，但不能据此断言它就是本样本弱相关的原因。

表 6-2 已知缺陷化合物的名称对照（不改变 Top-3）

感官—化合物	n/候选秩	全样本ρ	去风格ρ	入选频率	气味依据与样本结果
DMS 感官—DMS	250/85	0.088	0.032	0.005	气味已知[4]；本样本弱相关
双乙酰感官—双乙酰	241/80	-0.101	-0.049	0.000	气味已知[4]；本样本弱相关
4VG 感官—4VG	250/1	0.473	0.428	0.875	气味相符[5]；入选稳定
t2n 感官—反式-2-壬烯醛	250/16	-0.151	-0.0005	0.050	纸板味依据[10]；本样本反向

表 6-2 沿用 218 项候选中的绝对相关排名和同一 Bootstrap 结果。双乙酰有 241 对有效观测，其余三项为 250 对；DMS、双乙酰的相关区间跨零，t2n 对应化合物的负相关在去风格后接近零。因此，气味已知不保证商业样本中的单调相关，也不能用文献强行修正数据符号。反式-2-壬烯醛的老化纸板味有同位素研究支持[10]，但它不等于 Stale hops。附件未单列游离异戊酸，不能把异戊酸乙酯当作同一种缺陷物；陈旧酒花 Top-3 的直接机制仍未证实。

铁—金属味也需限定证据口径：二价铁盐溶液的人体感官试验观察到持续金属感[11]，但盐种、浓度及基质不同于本题的啤酒总铁测量，故只列一般背景，不移用试验浓度作为啤酒阈值。

附录 B-2 逐项给出 135 个候选位置的化学名称、附件来源类别、单位、阈值、证据说明与引用。10 项有相符的直接气味依据，36 项仅有一般背景，29 项属间接关联，60 项仍未验证；一般背景不表示相应目标机制已经成立。阈值缺失处不补造，外部试验阈值不覆盖附件值。后续宜优先验证兼具排名稳定、风格内同向和匹配气味依据的候选，采用加标、重组或省略试验检验其实际贡献。

6.5 折外置换与预测依赖

为区分边际关联与模型使用的信息，按第一问原五折划分和固定 ExtraTrees 参数重拟合，本次折外 RMSE 为 0.486219；主表原保存值为 0.486220。在每个验证折中分别置换一个输入变量，连同其缺失指示一起移动，其他输入和模型固定；每折重复 5 次。记 f_fk 为第 f 折对第 k 目标的预测器，V_f 为验证样本集合、n_f 为其大小，π为该折随机排列，置换引起的误差变化为

$$\Delta_{jk}^{(f,r)} = \frac{1}{n_f}\sum_{i\in V_f}[(y_{ik} - f_{fk}(x_i^{\pi_{j,r}}))^2 - (y_{ik} - f_{fk}(x_i))^2]. \qquad (6\text{-}5)$$

先对每折 5 次取均值，再按验证样本数合并；负值保留，不截为零。表 6-3 给出代表物，标准差按五个折均值计算，正值折数只作稳定性描述，不作为显著性检验。对全部 231 项输入计算后，仅在与相关分析相同的 218 项化学候选中排序，完整 45 项结果见表 B-3。

表 6-3 代表候选的折外置换结果

感官—化合物	置换名次	平均ΔMSE	折间标准差	正值折数
酯味指甲油(F)—乙酸乙酯	2	0.00235	0.00204	4/5
4VG 感官—4VG	1	0.00624	0.00299	5/5
麦芽总体味(F)—乙醇	5	0.00248	0.00121	5/5
陈旧酒花—乙酸乙酯	1	0.00084	0.00074	4/5

置换前三与相关前三的平均 Jaccard 为 0.080，30/45 个目标没有共同候选。两种排名回答不同问题，故并列保留。4VG 在边际相关、风格内相关、气味依据及预测依赖上均有支持；乙酸乙酯亦被预测器使用。麦芽总体—乙醇虽有预测依赖，风格内相关却近零，仍可能只是风格或共同工艺的标记。陈旧酒花的置换首项不在原相关前三中，其误差增量也小，不能据此提出加料因果方案。

相关输入可能相互替代，逐项置换还会打破真实化学组合。为检查这一问题，每个训练折独立重建|Spearman|≥0.7 网络，并将连通分量整体置换；五折分别形成 165、167、166、169、167 个分量。成员和逐折误差随附件保存，不将成员不同的分量混作同一对象求平均。这些分析提供模型依赖证据，均不识别因果效应。第一问使用 231 维附件表示，第三问改用 69 维校准量与 157 维相对量并另训代理；本问候选没有直接作为第三问的生成约束，三问的联系应限于证据核对和缺陷变量解释。

7 风格条件生成与受约束迁移

第三问先学习目标风格的化学联合分布，再将 Pils/Lager 向 Tripel 迁移。生成检验分布贴近程度和新组合，迁移则在化学与感官约束下比较可行候选，两者分别评价。

7.1 混合化学表示与高斯秩映射

依据附件及原研究的数据框架[1]，删除五项汇总或派生变量，保留 226 维输入：69 维以附件 8 的校准量替换，157 维沿用附件 1 的相对表示，按 beer 连接。校准负值转为缺失，相对量负值保留。填补与经验分布只在当前训练集估计；迁移训练排除 11 款 Pils，保留 239 款。

采用 Gaussian Copula 分离边缘分布与依赖结构[12]。对第 j 维的不同观测值 v，记小于 v 的样本数为 $N_{(<v)}$、等于 v 的样本数为 $N_{(=v)}$，用中点经验概率处理零值和并列值：

$$\widehat{F}_j^{\mathrm{mid}}(v)=\frac{N_{<v}+N_{=v}/2}{n}. \tag{7-1}$$

$$z_j=\Phi^{-1}(\widehat{F}_j^{\mathrm{mid}}(x_j)),\ x_j^*=\widehat{F}_j^{-1}(\Phi(z_j^*)). \tag{7-2}$$

训练值使用中点概率，中间输入按线性插值评分，概率截在[0.5/n,1−0.5/n]。逆变换按经验 CDF 阶梯取值，生成新的联合组合，但单列取值仍来自训练观测。

7.2 共享信息的条件 Copula 基准

小风格样本不足以稳定估计 226 维协方差。基准模型共用全局经验边缘，将风格中心向全局中心收缩，局部协方差向合并风格内协方差收缩。协方差采用 Ledoit–Wolf 估计并加入小对角项。对 n_s款风格样本，

$$a_s = \frac{n_s}{n_s+5},\ b_s = \frac{n_s}{n_s+20},\ \tilde{\mu}_s = a_s\bar{z}_s + (1-a_s)\bar{z}. \tag{7-3}$$

$$\widetilde{\Sigma}_s = b_s\widehat{\Sigma}_s + (1-b_s)\widehat{\Sigma}_{\text{within}},\ z^* \sim N(\tilde{\mu}_s, \widetilde{\Sigma}_s). \tag{7-4}$$

合并协方差由各风格减去自身中心后的残差估计。收缩降低估计波动，也可能减弱 Kriek、Lambic 特征，故再比较局部边缘方法。

7.3 局部边缘 Copula 与平滑混合

各风格分别建立经验边缘和高斯秩。缺失用该风格训练中位数填补，整列缺失时回退全局训练中位数。正则协方差经对角标准化得到相关矩阵，再混合局部 R_s与合并局部秩所得 R 共享。记风格秩样本为 z_{si}，

$$\widetilde{R}_s = \frac{n_s}{n_s+10}R_s + \frac{10}{n_s+10}R_{\text{shared}},\varepsilon \sim N(0, \widetilde{R}_s). \tag{7-5}$$

$$z^*_{\text{copula}} = \varepsilon,\ z^*_{\text{mixture}} = z_{sI} + 0.45\varepsilon,\ I \sim \text{Unif}\{1, \ldots, n_s\}. \tag{7-6}$$

两者均用风格经验逆 CDF 还原。局部 Copula 使用单位边际高斯噪声，平滑混合在随机训练秩中心附近加固定强度扰动。前者在期望上保留经验零频率，后者须另行检查；两者都不能补足未观测尾部。

7.4 嵌套选择、生成质量与感官画像

采用两次三折风格分层外层验证，各训练集内部以两折选择三个候选。评价坐标统一由当前训练集基准 Copula 及五维 PCA 确定，再除以训练主成分标准差。每种方法生成 300 条，重复两个模拟种子。主指标为真实留出谱 u 与生成谱 v 的经验能量差异：

$$\hat{\mathcal{E}}(U,V) = 2A_{UV} - A_{UU} - A_{VV},\ A_{UV} = \frac{1}{ml}\sum_{i=1}^{m}\sum_{k=1}^{l}\| u_i - v_k\|_2. \tag{7-7}$$

A_{uu}、A_{vv}为组内全部有序对距离均值，包含零对角项；式（7-7）为未开平方的 V 统计量，越小越好。内层每折须满足：补缺后整行复制率、仅比较原有观测格的复制率均为零，生成与真实留出谱的最近邻距离中位数比≥0.35，最近训练行相等特征比例中位数≤0.95。通过后按平均能量选择，外层评价完整选择流程。

表 7-1 风格生成的外层能量差异与全量拟合方法

风格	基准 Copula	嵌套选择器	风格重采样	全量拟合方法
Tripel	0.2866	0.2939	0.3218	收缩 Copula
Kriek	0.9528	0.7575	0.7434	局部边缘 Copula
Lambic	0.8701	0.6669	0.6304	平滑混合
Pils/Lager（对照）	0.8098	0.5001	0.5736	局部边缘 Copula

Kriek、Lambic 的能量差异较基准降低 20.5%、23.4%，Tripel 增加约 2.5%。前两者仍略逊于直接重采样。全量拟合后，三种指定风格各生成 300 条，另生成 300 条 Pils 对照；1200 条均无完整训练行复制。

按训练四分位距归一的边缘 Wasserstein 距离，其跨特征中位数在 Kriek 由 0.5345 降至 0.4701，Lambic 由 0.4913 降至 0.4114。不过 Kriek 的跨特征均值由 94.16 升至 153.19，反映极端变量影响；相关差异也未同步改善，故不能认定联合分布全面优于基准。重复划分复用同一批酒，折间波动不作独立样本置信区间。

生成谱的 44 项感官由 226 维全量多输出树预测，Stale hops 改用 239 款非 Pils 训练的固定单目标树。报告均值及 5%、95%分位数，表示生成谱之间的预测差异，均保留附件标准化尺度。

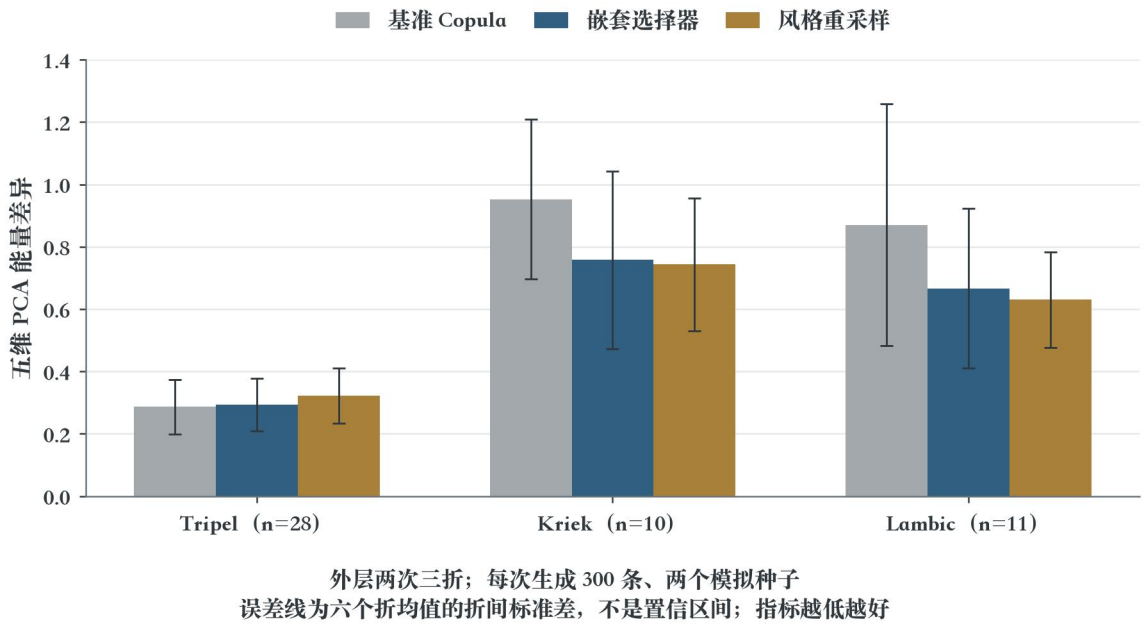
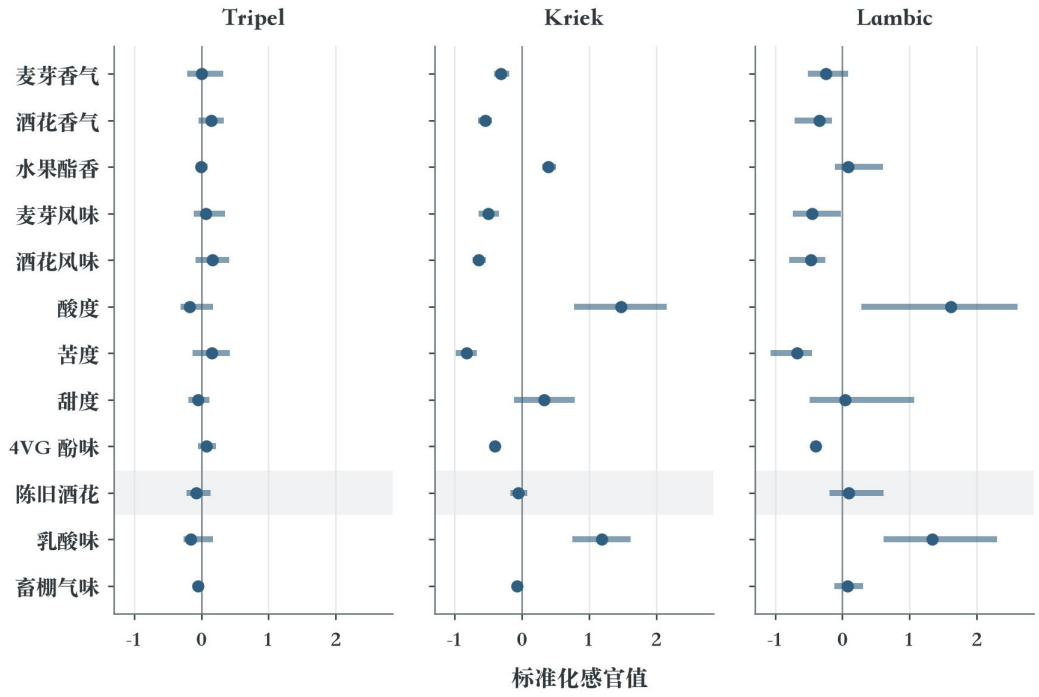


图 4 三种风格的生成能量差异。柱为外层六折均值，误差线为折间标准差，非置信区间；每折先平均两个模拟种子。



圆点：生成谱预测均值；线段：5%—95% 分位范围（每风格 300 条）

该范围描述生成分布，不是预测置信区间，也不是原始 0—6 分

灰行陈旧酒花使用非 Pils 单目标代理；其余维度使用全量多输出代理

图 5 三种风格的 12 维代表感官画像。点为预测均值，线为每风格 300 条生成谱的 5%—95%范围，并非预测置信区间。Stale hops 用非 Pils 单目标树，其余维度用 226 维多输出代理；均为标准化尺度。

7.5 距离、支持域与中央候选池

迁移沿用 239 款训练酒拟合的高斯秩变换和 Tripel 几何，使各方案使用同一距离。记 z(x)为秩映射，目标距离 d 与改动 r 为

$$d(x) = \sqrt{\frac{(z(x)-\tilde{\mu}_T)^{\mathrm{T}}\tilde{\Sigma}_T^{-1}(z(x)-\tilde{\mu}_T)}{p}},\ p = 226. \tag{7-8}$$

$$r_i(x) = \frac{\|z(x)-z(x_i)\|_2}{\sqrt{p}},\ G_i(x) = 1 - \frac{d(x)}{d(x_i)}. \tag{7-9}$$

候选须在非 Pils 训练逐维范围内，且到真实 Tripel 的最近邻秩空间 RMS 距离不超过 Tripel 内部留一最近邻距离的 95%分位数；另要求 $r_i \geq 0.05$。该支持范围为经验规则，没有卡方概率标定。

将已保存的前 500 条 Tripel 生成谱映射至秩空间，按 $c \in \{0.2,0.4,0.6\}$向收缩中心移动并逆变换，得到 1500 条中央目标谱。与源谱按$\gamma \in \{0.25,0.5,0.75,1\}$插值，另加四条朝真实 Tripel 中位谱插值的候选：

$$z_{kc}^{\text{central}} = \tilde{\mu}_T + c\{z(t_k) - \tilde{\mu}_T\},\ t_{kc}^{\text{central}} = \hat{F}^{-1}\{\Phi(z_{kc}^{\text{central}})\}. \tag{7-10}$$

$$x'_{ikc\gamma} = (1-\gamma)x_i + \gamma\, t_{kc}^{\text{central}}. \tag{7-11}$$

中央池含 1500×4+4=6004 个候选，与原 1500 条目标谱的 6004 池等量比较；另设原 500 条谱的 2004 池。中央目标谱均唯一，投影后仍逐源检查重复和支持范围，不能把候选数当作独立观测数。

投影只截断四项缺陷浓度，使其不低于零且不超过情景交集上界，其余维度保留插值结果。截断后重新计算距离、支持范围和感官预测，以检查联合结构是否仍合理。

7.6 化学阈值与最低改善要求

DMS、双乙酰、4VG 分别采用附件阈值均值和中位数，各乘 0.8、0.9、1.0、1.1、1.2，形成十个情景。各情景下均要求超阈程度不增加：

$$\max(0, x_d^{'} - T_{ds}) \le \max(0, x_{id} - T_{ds}) \ (\forall\, s). \qquad (7\text{-}12)$$

$$0 \le x_d^{'} \le \min_s \max(T_{ds}, x_{id}) = \max\{\min_s T_{ds}, x_{id}\}. \qquad (7\text{-}13)$$

最严格阈值为 DMS 13.8 μg/L、双乙酰 0.072 mg/L、4VG 0.16 mg/L。源浓度已超阈时不得增加，未超阈时允许在阈值内变化。附件未给 trans-2-nonenal 阈值，故直接要求其浓度不增加。四项源值均采用观测校准量。

以每款源酒原已保存迁移谱的改善幅度 G_i锚为参照，要求保留至少 80%，且绝对改善不低于 10%：

$$G_i(x') \ge \max\{0.10, 0.80 G_i^{\text{anchor}}\}. \qquad (7\text{-}14)$$

敏感性比较另设保留 100%原改善及绝对 10%、30%、50%、60%下限。两种预测模型、三个候选池、六种下限、两种排序共 72 组，均以 11 款源酒为通过率分母。

7.7 Stale hops 预测及 beer 81 误差诊断

Stale hops 仅由化学输入预测。非 Pils 训练内按风格留出误差和响应稳定性选择 Ridge；四外层分组验证评价整个选择规则，RMSE 为 0.4200，同折多输出树为 0.4280。Pils 留出诊断中 Ridge 误差为 0.5614；实际用于迁移的固定单目标树、旧多输出树误差分别为 0.559206、0.526276。本轮另以相同参数、不同种子重拟合两种树作为选型对照，误差为 0.5429、0.5307，这两组数对应不同模型实例。尾部加权备选为 0.5157，但未通过非 Pils 精度门槛，Pils 的 R^2也仅为−0.0007，故仍作备选诊断。

先排除精度未优于均值或预测近常数的配置，再在一标准误差容忍范围内比较训练响应稳定性。辅助感官方向和尾部权重均在训练折估计，推理仍只输入化学量。该容忍范围用于选型，不解释为参数置信区间。

beer 81 实测 Stale hops 为 1.6182，居 250 款第 5 高，239 款训练酒中仅 4 款达到此水平。其旧模型 25 近邻评分范围为[−0.3668,1.0643]，非负加权均值无法预测到该高值。核查未见身份错配、缺失或非法校准负值；Ridge 仍预测−0.0979，说明尾部信息及化学相似性尚不足以解释陈旧感。保留该酒，不改标签。Pils 标签已多次查看，此处属于开发诊断。

表 7-2 五项缺陷的变量及保护范围

缺陷	表示与单位	附件阈值	约束	解释边界
DMS	校准量，μg/L	均值 17.25；中位 19	十情景下超阈程度不增加	给定情景的化学保护
Diacetyl	校准量，mg/L	均值 0.29925；中位 0.09	同上	给定情景的化学保护
4VG	校准量，mg/L	均值 0.20633；中位 0.20	同上，依题面按缺陷处理	给定情景的化学保护
trans-2-nonenal	校准量，mg/L	附件空缺	候选浓度不大于源浓度	不保证低于感知阈值
Stale hops	标准化感官预测	无单一化学行及阈值	中心预测与配对差 Q95 不增加，预留模型复核	代理检查，需实际品评验证

7.8 配对预测筛选与留出模型复核

分别用 Ridge 和固定单目标树预测 Stale hops，并保留旧多输出树作交叉检查。候选和源谱交由同一个 bootstrap 模型预测，形成配对差。要求当前 40 个筛选模型、旧 30 个筛选模型的差值 95%分位数均≤0，两个中心预测也均不增加：

$$\Delta_b^{(a)}(x', x_i) = f_b^{(a)}(x') - f_b^{(a)}(x_i),\ a = \text{new, old}. \tag{7-15}$$

$$Q_{0.95}\{\Delta_b^{(\text{new})}\} \le 0,\ Q_{0.95}\{\Delta_b^{(\text{old})}\} \le 0. \tag{7-16}$$

$$f_0^{(a)}(x') - f_0^{(a)}(x_i) \le 0,\ a = \text{new, old}. \tag{7-17}$$

另要求配对响应非零，池内平均响应标准差>10^{-6}，以排除近常数预测。旧中心为 45 输出树，旧 bootstrap 模型为单目标 120 树 ExtraTrees。选谱后，用预留当前 20 个模型和新种子旧协议 20 个模型复核；固定树另生成 20 个复核模型。失败后不重选谱。旧历史 10 个模型仅作对照。模型均共享 239 款训练数据，分位数不是实际感官的 95%保证。

7.9 两种排序的迁移结果

记两组筛选差值分位数为 q 新、q 旧。余量优先先最小化较差分位数，完全相等时再比较几何目标 J；几何优先直接最小化 J。以 h_i为源谱到真实 Tripel 秩均值的 RMS 距离，

$$J_i(x') = \frac{d(x')}{d(x_i)} + 0.2\frac{r_i(x')}{h_i}. \tag{7-18}$$

$$x^*_{i,\text{margin}} = \text{arglexmin}_{x' \in \mathcal{C}_i}(\max(q_{\text{new}}, q_{\text{old}}), J_i(x')). \tag{7-19}$$

$$x^*_{i,\text{geometry}} = \text{argmin}_{x' \in \mathcal{C}_i} J_i(x'). \tag{7-20}$$

表 7-3 保留原改善幅度至少 80%时的迁移结果

预测模型/候选池/排序	选出数	复核通过数	所选谱改善中位数	通过谱改善中位数
稳定 Ridge/中央 6004/余量优先	5/11	5/11	53.17%	53.17%
固定树/原 6004/余量优先	9/11	9/11	54.38%	54.38%
固定树/中央 6004/余量优先	11/11	10/11	57.74%	58.44%
固定树/中央 6004/几何优先	11/11	10/11	70.08%	70.08%

主比较采用固定树、中央 6004 池、余量优先。相对原 6004 池，通过数从 9 增至 10，所选谱改善中位数从 54.38%增至 57.74%；后者包含复核失败的谱，通过谱中位数为 58.44%。beer 96 在两种排序下均因当前复核分位数+0.0210 失败。

两种预测模型使用相同源酒、候选数、化学情景及接近下限，可行集合的差别来自模型响应。候选存在和最终通过分开报告；固定树通过数较高，并不说明其真实陈旧感预测更准确。

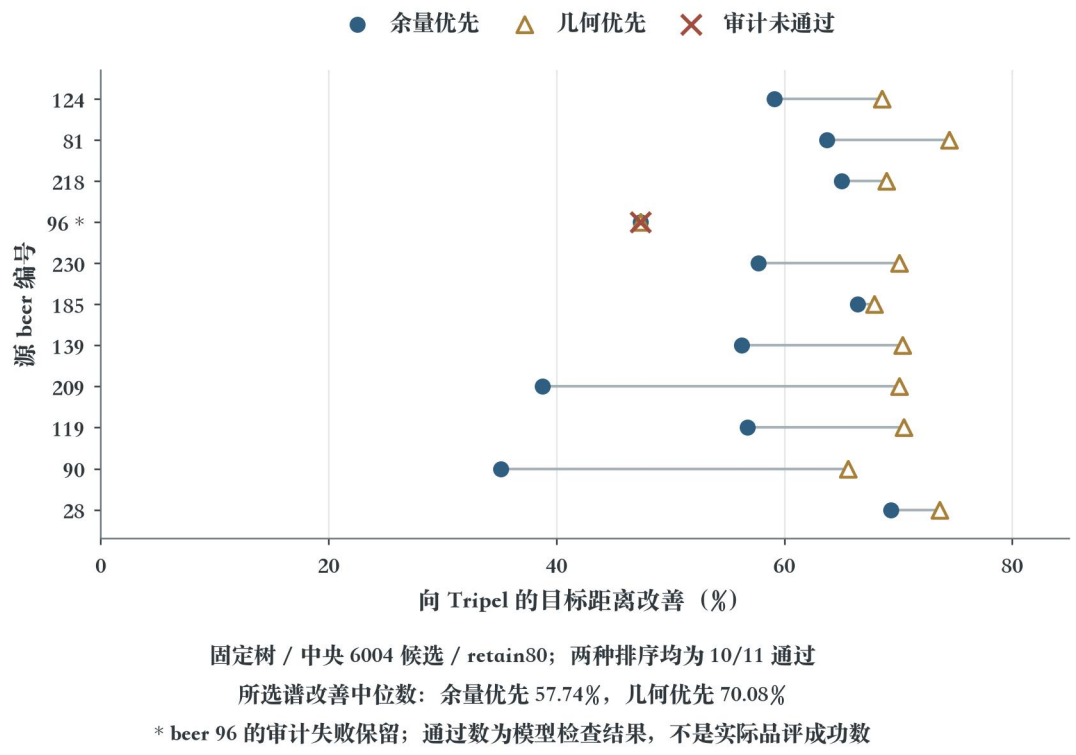


图 6 固定树、中央 6004 候选池与 retain80 条件下，11 款 Pils 源的所选谱目标距离改善。余量优先和几何优先均通过 10/11 款，beer 96 的审计失败保留；改善中位数分别为 57.74%与 70.08%，包括已选但失败的谱。几何优先为探索性应用示例，模型检查通过不代表实际品评成功。

7.10 方案选择与后续验证

固定树的几何优先方案同样有 10/11 款通过，改善中位数为 70.08%，可作应用示例。该方案在查看 72 组结果后提出，属于探索性选择；最初 Ridge 方案仍为 5/11。有限候选搜索不代表连续空间的全局最优。

化学保护仅覆盖给定十个阈值情景。迁移结果为化学目标谱和预测画像，10/11 是模型检查通过数；实际品评效果及酿造可行性须通过新批次检测和盲品检验。

8 结果检验与模型评价

8.1 误差与稳定性证据的适用范围

第一问 RMSE 较均值降低 15.36%，新增 20%随机缺失后增加 2.67%，但逐维性能差异明显。Stale hops 及部分酯香描述符仍接近均值预测，不能由酸度等强项推及所有感官。评价仅使用一个随机五折种子，且以附件已有的数据处理为条件。

第二问的网络边数随阈值由 266 降至 109 再降至 39；Top-3 平均 Bootstrap 入选频率为 0.455，说明不少化合物可相互替代。去风格均值后 21 项感官的 Top-3 完全变化，表明全局相关受风格差异影响。统计稳定性与直接气味依据需分别判断。

第三问 Kriek、Lambic 的能量差异下降，但部分指标仍逊于直接重采样。生成方法在贴近分布与形成新组合之间存在取舍。感官画像的分位区间描述生成谱之间的差异，不是新酒预测误差的置信区间。

8.2 阈值与迁移策略敏感性

化学约束取阈值均值、中位数及各自±20%扰动的十个情景交集。浓度保护通过并不代表 Stale hops 感官检查通过，两者分别报告。

72 组迁移方案比较两种预测模型、三个候选池、六种接近下限和两种排序。主比较保持预测模型、检查规则、下限与候选数相同；几何优先结果作为探索性方案展示。逐源结果保留无候选的酒及 beer 96 失败。

固定树与稳定 Ridge 在相同候选空间分别有 10/11 和 5/11 款通过，说明可行方案依赖预测模型。Ridge 在非 Pils 分组验证中略好，却未改善 Pils 误差。预留重抽样模型的复核仍来自同一批训练酒，无法替代独立品评。

8.3 方法优点与主要不足

本方法统一核对身份、字段、量尺与浓度单位，补值和选型限定在训练数据内。预测、关联、生成分别设置对照，迁移保留候选数、约束和失败记录，结果可以按样本追溯。

主要不足是原始 0—6 评分无法校准，小风格样本偏少，且多数化学候选尚缺直接机制证据。Stale hops 跨风格预测尤其薄弱；附件没有储存、批次及重复品评条件，无法据此解释误差来源。

9 结论与建议

基于 250 款啤酒，本文完成 45 维标准化感官预测、化合物关联分析和风格条件生成。ExtraTrees 五折 RMSE 为 0.4862；$|\rho|\geq 0.7$ 时网络有 109 条边，报告 135 个 Top-3 位置及其稳定性。三种指定风格各生成 300 条化学谱，并给出感官预测画像。

Pils/Lager 向 Tripel 迁移时，固定树、中央候选池和余量优先方案有 10/11 款通过模型检查，所选谱目标距离缩短中位数为 57.74%；几何优先的探索性结果为 70.08%。这些结果支持进一步筛选化学目标谱，尚需理化检测和盲品验证。

后续应补充逐人原始评分及标准化参数，并对陈旧感高的酒收集储存时间、温度、包装、批次和重复品评记录。化学候选可通过加标、重组或省略试验检验；迁移方案应在新批次样本上重新测试。

使用声明

本研究使用 OpenAI Codex 辅助核对数据、编写与排查 Python 程序、复核计算结果、制作图表，以及修订论文文字和排版。论文中的数值以附件数据和程序运行结果为依据，原始数据未由生成内容替代；作者对提交内容及结论负责。

参考文献

[1] Michiel Schreurs, Supinya Piampongsant, Miguel Roncoroni, et al. Predicting and improving complex beer flavor through machine learning[J]. Nature Communications, 15: 2368, 2024. DOI: 10.1038/s41467-024-46346-0.

[2] Pierre Geurts, Damien Ernst, Louis Wehenkel. Extremely randomized trees[J]. Machine Learning, 63: 3–42, 2006. DOI: 10.1007/s10994-006-6226-1.

[3] Arthur E. Hoerl, Robert W. Kennard. Ridge Regression: Biased Estimation for Nonorthogonal Problems[J]. Technometrics, 12(1): 55–67, 1970. DOI: 10.1080/00401706.1970.10488634.

[4] Stefan Hanke, V. Ditz, M. Herrmann, et al. Influence of Ethyl Acetate, Isoamyl Acetate and Linalool on off-flavour Perception in Beer[J]. BrewingScience, 63(7/8): 94–99, 2010. https://brewingscience.de/index.php/brewingscience/article/view/456，访问日期 2026-10-23。

[5] Magdalena Müller, Martina Gastl, Thomas Becker. Key constituents, flavour profiles and specific sensory evaluation of wheat style non-alcoholic beers depending on their production method[J]. Journal of the Institute of Brewing, 127(3): 262–272, 2021. DOI: 10.1002/jib.663.

[6] Kiyoshi Takoi, Yutaka Itoga, Koichiro Koie, et al. The Contribution of Geraniol Metabolism to the Citrus Flavour of Beer: Synergy of Geraniol and β-Citronellol Under Coexistence with Excess Linalool[J]. Journal of the Institute of Brewing, 116(3): 251–260, 2010. DOI: 10.1002/j.2050-0416.2010.tb00428.x.

[7] Yu Zhang, Sinuo Li, Qi Meng, et al. Characterization of Key Odor-Active Compounds in Draft Beers for the Chinese Market Using Molecular Sensory Science Approaches[J]. Molecules, 29(11): 2537, 2024. DOI: 10.3390/molecules29112537.

[8] Ann Van Holle, Anita Van Landschoot, Isabel Roldán-Ruiz, et al. The brewing value of Amarillo hops (Humulus lupulus L.) grown in northwestern USA: A preliminary study of terroir significance[J]. Journal of the Institute of Brewing, 123(3): 312–318, 2017. DOI: 10.1002/jib.433.

[9] Kiyoshi Takoi, Haruna Kumamaru, Atsushi Tanigawa. Study on Characteristic Aroma in Special Beers Brewed with Coriander (Coriandrum sativum L.) Seeds: Profiling of Flavor Compounds Derived from Coriander Seeds in Different Growing Areas[J]. ACS Food Science & Technology, 4(10): 2439–2447, 2024. DOI: 10.1021/acsfoodscitech.4c00579.

[10] Sophie Noël, Catherine Liégeois, Guillaume Lermusieau, et al. Release of Deuterated Nonenal during Beer Aging from Labeled Precursors Synthesized in the Boiling Kettle[J]. Journal of Agricultural and Food Chemistry, 47(10): 4323–4326, 1999. DOI: 10.1021/jf990211r.

[11] Heidi Hai-Ling Yang, Harry T. Lawless. Time–intensity characteristics of iron compounds[J]. Food Quality and Preference, 17(5): 337–343, 2006. DOI: 10.1016/j.foodqual.2005.04.002.

[12] Han Liu, John Lafferty, Larry Wasserman. The Nonparanormal: Semiparametric Estimation of High Dimensional Undirected Graphs[J]. Journal of Machine Learning Research, 10: 2295–2328, 2009. https://www.jmlr.org/papers/v10/liu09a.html，访问日期 2026-10-23。

附录 A 感官字段及逐维精度

表 A-1 列出第一问 ExtraTrees 的全部 45 维折外误差。A_前缀表示香气，F_前缀为相应风味；其余字段按附件字典保留。所有值均为附件标准化尺度，完整 250×45 折外预测随电子附录保存。

表 A-1 全部感官维度的五折折外指标

目标字段	MAE	RMSE	R^2	Spearman
A_malt_all	0.4278	0.5503	0.4083	0.6341
A_malt_grain	0.3387	0.4497	0.0286	0.1604
A_malt_bread	0.3432	0.4939	0.1042	0.3733
A_malt_cara	0.3791	0.5131	0.3298	0.5875
A_malt_burn	0.3328	0.5678	0.3011	0.3431
A_hops_all	0.3943	0.5200	0.3773	0.6145
A_hops_citrus	0.3286	0.4374	0.1892	0.4410
A_hops_tropical	0.3423	0.5678	0.1331	0.3059
A_hops_noble	0.3459	0.4530	0.2832	0.5371
A_hops_woody	0.2653	0.3569	0.0640	0.2015
A_esters_all	0.4397	0.5517	0.1592	0.3962
A_esters_ethac	0.3752	0.4667	0.1207	0.3230
A_esters_isoaa	0.4169	0.5298	0.0298	0.1885
A_esters_flower	0.3128	0.4088	0.0674	0.2009
A_esters_fruity	0.3577	0.5062	-0.0105	0.0158
F_malt_all	0.3868	0.4900	0.4896	0.6906
F_malt_grain	0.3370	0.4385	0.0451	0.2367
F_malt_bread	0.3316	0.4334	0.2065	0.5242
F_malt_cara	0.3515	0.4731	0.3133	0.5737
F_malt_burn	0.3369	0.5217	0.4152	0.6102
F_hops_all	0.3944	0.4926	0.4943	0.7074
F_hops_citrus	0.3012	0.3931	0.2292	0.4205
F_hops_tropical	0.3062	0.4699	0.1467	0.3606
F_hops_noble	0.3717	0.4903	0.3564	0.5989
F_hops_woody	0.3080	0.4302	0.1411	0.4202
F_esters_all	0.4027	0.5124	0.2280	0.4983
F_esters_ethac	0.3354	0.4415	0.2069	0.4940
F_esters_isoaa	0.3967	0.5458	0.0712	0.3302
F_esters_flower	0.2904	0.4123	0.0116	0.1955

目标字段	MAE	RMSE	R^2	Spearman
F_esters_fruity	0.3370	0.4856	0.0732	0.2557
acidity	0.3025	0.4623	0.7088	0.4999
bitternes	0.3709	0.4875	0.5534	0.7504
sweetness	0.4324	0.5849	0.2583	0.5144
X4vg	0.3964	0.5929	0.1689	0.5123
diacetyl	0.3674	0.4928	0.0883	0.2703
dms	0.3708	0.4896	0.0388	0.2161
metallic	0.3604	0.4602	0.1028	0.3283
stale_hops	0.3062	0.4367	0.0181	0.1533
t2n	0.2960	0.3793	0.0989	0.2151
orange	0.3411	0.6428	0.1122	0.2533
coriander	0.2411	0.4295	0.0961	0.1420
clove	0.2521	0.5369	0.0749	0.3109
lactic	0.2093	0.4329	0.6519	0.4526
acetic	0.1939	0.4441	0.6460	0.3503
barnyard	0.3099	0.4420	0.0278	0.1741

附录 B 完整 Top 3 候选

表 B-1 沿用绝对 Spearman 相关排名，不按文献证据后验改选。每项依次给出附件字段名、带符号相关ρ与 Bootstrap 入选频率π。化学名称、来源、单位、阈值及逐行证据等级见表 B-2 及电子附件 chemical_evidence_final.csv；135 个位置涉及 60 种不同化合物。

表 B-1 45 项感官的全部 Top-3 关联候选

感官字段	第 1 项	第 2 项	第 3 项
A_malt_all	acetaldehyde ρ=0.361，π=0.565	methylpyrazine ρ=0.356，π=0.510	ethanol..v.v. ρ=0.335，π=0.325
A_malt_grain	terpinen.4.ol ρ=-0.265，π=0.620	beta.phellandrene ρ=-0.248，π=0.400	isoborneol ρ=-0.225，π=0.235
A_malt_bread	glycerol ρ=0.267，π=0.310	ethanol..v.v. ρ=0.260，π=0.260	terpinen.4.ol ρ=-0.250，π=0.270
A_malt_cara	acetaldehyde ρ=0.350，π=0.520	X2.ethyl.1.hexanol ρ=-0.317，π=0.375	ethanol..v.v. ρ=0.315，π=0.295
A_malt_burn	isobutanol ρ=0.249，π=0.350	DES ρ=0.243，π=0.320	ethanol..v.v. ρ=0.239，π=0.310
A_hops_all	ethyl_octanoate ρ=0.464，π=0.785	X.E..2.nonenal ρ=0.432，π=0.440	ethyl_decanoate ρ=0.428，π=0.410
A_hops_citrus	citronellol ρ=0.370，π=0.590	humulene.epoxide.II ρ=0.357，π=0.515	linalool ρ=0.327，π=0.265
A_hops_tropical	DMS ρ=0.314，π=0.720	X3.furanmethanol ρ=-0.259，π=0.280	methyl.geranate ρ=0.256，π=0.255
A_hops_noble	ethyl_octanoate ρ=0.482，π=0.925	ethyl_decanoate ρ=0.469，π=0.900	X.E..2.nonenal ρ=0.409，π=0.340
A_hops_woody	trans.dihydro.alpha.terpinyl.acetate ρ=0.279，π=0.255	alpha.calacorene ρ=0.273，π=0.175	ethyl.tetradecanoate ρ=0.267，π=0.165
A_esters_all	n.hexanol ρ=0.357，π=0.675	acetaldehyde ρ=0.353，π=0.710	ethyl_acetate ρ=0.321，π=0.435
A_esters_ethac	ethyl_acetate ρ=0.356，π=0.585	acetaldehyde ρ=0.335，π=0.400	ethanol..v.v. ρ=0.333，π=0.325
A_esters_isoaa	n.hexanol ρ=0.340，π=0.945	isopentyl_acetate ρ=0.283，π=0.650	isoamyl_alcohol ρ=0.223，π=0.145
A_esters_flower	beta.phellandrene ρ=0.196，π=0.280	isobutyl_acetate ρ=0.192，π=0.210	X1.alpha.terpineol ρ=0.184，π=0.155
A_esters_fruity	beta.phellandrene ρ=0.232，π=0.485	eugenol ρ=0.193，π=0.210	lactic_acid.mg.L. ρ=0.186，π=0.180
F_malt_all	ethanol..v.v. ρ=0.472，π=0.960	acetaldehyde ρ=0.471，π=0.935	MeSH ρ=0.356，π=0.190

感官字段	第 1 项	第 2 项	第 3 项
F_malt_grain	isoborneol ρ=-0.276，π=0.540	terpinen.4.ol.acetate ρ=-0.256，π=0.295	X.E..beta.ionone ρ=-0.249，π=0.320
F_malt_bread	acetaldehyde ρ=0.378，π=0.590	ethanol..v.v. ρ=0.372，π=0.480	glycerol ρ=0.367，π=0.445
F_malt_cara	acetaldehyde ρ=0.433，π=0.970	ethanol..v.v. ρ=0.387，π=0.795	ethyl.butyrate ρ=0.309，π=0.190
F_malt_burn	ethanol..v.v. ρ=0.431，π=0.900	MeSH ρ=0.403，π=0.690	glycerol ρ=0.364，π=0.320
F_hops_all	ethyl_octanoate ρ=0.501，π=0.745	X.E..2.nonenal ρ=0.456，π=0.315	ethyl_decanoate ρ=0.453，π=0.295
F_hops_citrus	humulene.epoxide.II ρ=0.285，π=0.355	ethyl.furoate ρ=-0.275，π=0.365	furfural ρ=-0.274，π=0.285
F_hops_tropical	geraniol ρ=0.285，π=0.445	hexyl.acetate ρ=0.281，π=0.420	DMS ρ=0.275，π=0.450
F_hops_noble	ethyl_octanoate ρ=0.505，π=0.970	ethyl_decanoate ρ=0.458，π=0.600	ethanol..v.v. ρ=0.434，π=0.335
F_hops_woody	ethyl.tetradecanoate ρ=0.346，π=0.460	ethyl_decanoate ρ=0.342，π=0.390	humulene.epoxide.II ρ=0.326，π=0.180
F_esters_all	acetaldehyde ρ=0.404，π=0.780	ethyl_acetate ρ=0.380，π=0.675	n.hexanol ρ=0.376，π=0.465
F_esters_ethac	ethyl_acetate ρ=0.503，π=0.980	ethanol..v.v. ρ=0.466，π=0.885	glycerol ρ=0.443，π=0.630
F_esters_isoaa	n.hexanol ρ=0.305，π=0.675	isopentyl_acetate ρ=0.274，π=0.465	acetaldehyde ρ=0.244，π=0.235
F_esters_flower	beta.phellandrene ρ=0.234，π=0.550	isopentyl.butanoate ρ=-0.201，π=0.280	terpinolene ρ=0.177，π=0.150
F_esters_fruity	DES ρ=-0.292，π=0.740	acetic_acid.g.L. ρ=0.244，π=0.325	humulene.epoxide.II ρ=-0.234，π=0.235
acidity	prenyl.isobutyrate ρ=0.435，π=0.565	ethyl.phenylacetate ρ=0.430，π=0.470	perillyl.acetate ρ=0.417，π=0.330
bitternes	ethyl_octanoate ρ=0.593，π=0.985	ethyl_decanoate ρ=0.559，π=0.720	MeSH ρ=0.511，π=0.285
sweetness	ethyl_decanoate ρ=-0.391，π=0.570	pentyl.octanoate ρ=-0.363，π=0.490	iron ρ=0.359，π=0.465
X4vg	X4VG ρ=0.473，π=0.875	delta.terpinyl.acetate ρ=0.399，π=0.450	neryl.formate ρ=-0.383，π=0.285
diacetyl	eugenol ρ=-0.323，π=0.690	ethyl_decanoate ρ=-0.283，π=0.410	alpha.calacorene ρ=-0.275，π=0.285
dms	terpinen.4.ol ρ=-0.196，π=0.190	E.nerolidol ρ=0.194，π=0.200	car.3.en.2.one ρ=0.186，π=0.180

感官字段	第 1 项	第 2 项	第 3 项
metallic	ethyl_isovalerate ρ=-0.250，π=0.350	iron ρ=0.244，π=0.305	X6.methyl.5.hepten.2.one ρ=-0.237，π=0.210
stale_hops	X2.undecanol ρ=0.300，π=0.645	alloaromadendrene.epoxide ρ=0.278，π=0.445	neryl.formate ρ=0.270，π=0.265
t2n	beta.phellandrene ρ=-0.214，π=0.270	terpinen.4.ol ρ=-0.204，π=0.160	geraniol ρ=-0.201，π=0.250
orange	camphor ρ=0.367，π=0.865	limonene ρ=0.296，π=0.380	ethanol..v.v. ρ=-0.292，π=0.315
coriander	X2.methyl.benzofuran ρ=0.238，π=0.240	E.ocimenone ρ=0.236，π=0.205	terpinen.4.ol ρ=0.235，π=0.190
clove	limonene ρ=0.342，π=0.915	eucalyptol ρ=0.236，π=0.285	camphor ρ=0.206，π=0.115
lactic	lactic_acid.mg.L. ρ=0.448，π=0.550	perillyl.acetate ρ=0.443，π=0.525	prenyl.isobutyrate ρ=0.429，π=0.360
acetic	prenyl.isobutyrate ρ=0.406，π=0.685	ethyl_isovalerate ρ=0.391，π=0.470	ethyl.phenylacetate ρ=0.385，π=0.370
barnyard	X2.undecanol ρ=0.285，π=0.545	neryl.formate ρ=0.280，π=0.375	alloaromadendrene.epoxide ρ=0.266，π=0.340

表 B-2 给出每个候选的化学名称、附件来源、量纲及文献支持范围。阈值只用于对应的校准浓度；空缺阈值不补造。一般背景与间接关联均不视为目标感官机制已经证实。

表 B-2 完整候选的化学信息及证据

感官与名次	化合物	来源与量纲	依据及适用范围
A_esters_all 第 1 项	正己醇 n.hexanol	酵母；mg/L 均值未给出；中位未给出	一般背景 [7] 鲜啤 GC-O/AEDA 检出有气味活性的正己醇，嗅闻描述为山楂样。 可补充果香背景，未检验本行香蕉香或酯类评分的直接来源。
A_esters_all 第 2 项	乙醛 acetaldehyde	酵母；mg/L 均值 50；中位 50	一般背景 [7] 鲜啤感官实验以乙醛作生青味训练标准。该训练标准未验证乙醛对本行麦芽、焦糖或酯香描述的直接作用。
A_esters_all 第 3 项	乙酸乙酯 ethyl_acetate	酵母；mg/L 均值 24；中位 23	直接气味依据 [1];[4] 乙酸乙酯具有果香/溶剂样气味；啤酒加标亦显示其改变部分缺陷感知。 气味与酯类/指甲油样描述相符，但不验证本样本相关系数或单独剂量效应。
A_esters_ethac 第 1 项	乙酸乙酯 ethyl_acetate	酵母；mg/L 均值 24；中位 23	直接气味依据 [1];[4] 乙酸乙酯具有果香/溶剂样气味；啤酒加标亦显示其改变部分缺陷感知。 气味与酯类/指甲油样描述相符，但不验证本样本相关系数或单独剂量效应。

感官与名次	化合物	来源与量纲	依据及适用范围
A_esters_ethac 第 2 项	乙醛 acetaldehyde	酵母；mg/L 均值 50；中位 50	一般背景 [7] 鲜啤感官实验以乙醛作生青味训练标准。该训练标准未验证乙醛对本行麦芽、焦糖或酯香描述的直接作用。
A_esters_ethac 第 3 项	乙醇 ethanol..v.v.	酵母；v/v 均值 1.75726；中位 1.7372	间接关联 [1] 原研究将酯类及多种醇类作为共同发酵来源讨论；可形成工艺/风格共变标记。间接共变解释，不证明本行麦芽或酯类具体气味由该物质直接产生。
A_esters_flower 第 1 项	Beta-phellandrene beta.phellandrene	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
A_esters_flower 第 2 项	乙酸异丁酯 isobutyl_acetate	酵母；mg/L 均值 1；中位 1	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
A_esters_flower 第 3 项	α-松油醇 X1.alpha.terpineol	酒花；mg/L 均值 1.5；中位 1.5	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
A_esters_fruity 第 1 项	Beta-phellandrene beta.phellandrene	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
A_esters_fruity 第 2 项	丁香酚 eugenol	香辛料；mg/L 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
A_esters_fruity 第 3 项	乳酸 lactic_acid.mg.L.	野生菌群；mg/L 均值 416.667；中位 400	一般背景 [1] 原研究说明乳酸贡献酸啤酒的酸味。支持酸感官背景；非本行果香等其他描述的直接来源证明。
A_esters_isoaa 第 1 项	正己醇 n.hexanol	酵母；mg/L 均值未给出；中位未给出	一般背景 [7] 鲜啤 GC-O/AEDA 检出有气味活性的正己醇，嗅闻描述为山楂样。 可补充果香背景，未检验本行香蕉香或酯类评分的直接来源。
A_esters_isoaa 第 2 项	乙酸异戊酯 isopentyl_acetate	酵母；mg/L 均值 0.603333；中位 0.6	直接气味依据 [5] GC-O/MS 将乙酸异戊酯的气味标为香蕉。 相同气味描述的定性依据，非本样本关联的干预验证。

感官与名次	化合物	来源与量纲	依据及适用范围
A_esters_isoaa 第 3 项	异戊醇 isoamyl_alcohol	酵母；mg/L 均值 40.015；中位 45	间接关联 [1] 原研究将酯类及多种醇类作为共同发酵来源讨论；可形成工艺/风格共变标记。 间接共变解释，不证明本行麦芽或酯类具体气味由该物质直接产生。
A_hops_all 第 1 项	辛酸乙酯 ethyl_octanoate	酵母；mg/L 均值 0.7；中位 0.75	间接关联 [1];[5] GC-O/MS 报告果香/胶样气味；原研究报告同酒花感知和苦味的意外关联。 原研究与本题共享数据，非独立重复；未证明该酯直接产生酒花香或苦味。
A_hops_all 第 2 项	反式-2-壬烯醛 X.E..2.nonenal	其他；mg/L 均值未给出；中位未给出	一般背景 [10] 同位素研究支持老化期间反式-2-壬烯醛释放与纸板味相关。 本行目标为酒花感官，不能由纸板味证据解释其正相关，也不能与 Stale hops 等同。
A_hops_all 第 3 项	癸酸乙酯 ethyl_decanoate	酵母；mg/L 均值 1.035；中位 1.035	间接关联 [1] 原研究报告癸酸乙酯同其他发酵产物共变，并讨论同酒花感知和苦味的关联。 同源数据的背景说明，非独立的直接气味/抑制机制验证。
A_hops_citrus 第 1 项	香茅醇 citronellol	酒花；mg/L 均值 0.0245；中位 0.0245	直接气味依据 [6] 香叶醇、香茅醇与芳樟醇在啤酒中具有果香/柑橘协同；芳樟醇亦有 GC-O 花香/柑橘依据。 三组分交互依赖基质；不支持其他目标（如 t2n）的直接抑制机制。
A_hops_citrus 第 2 项	Humulene epoxide II humulene.epoxide.II	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
A_hops_citrus 第 3 项	芳樟醇 linalool	酒花；mg/L 均值 0.027285；中位 0.0045	直接气味依据 [6];[5] 香叶醇、香茅醇与芳樟醇在啤酒中具有果香/柑橘协同；芳樟醇亦有 GC-O 花香/柑橘依据。 三组分交互依赖基质；不支持其他目标（如 t2n）的直接抑制机制。
A_hops_noble 第 1 项	辛酸乙酯 ethyl_octanoate	酵母；mg/L 均值 0.7；中位 0.75	间接关联 [1];[5] GC-O/MS 报告果香/胶样气味；原研究报告同酒花感知和苦味的意外关联。 原研究与本题共享数据，非独立重复；未证明该酯直接产生酒花香或苦味。

感官与名次	化合物	来源与量纲	依据及适用范围
A_hops_noble 第 2 项	癸酸乙酯 ethyl_decanoate	酵母；mg/L 均值 1.035；中位 1.035	间接关联 [1] 原研究报告癸酸乙酯同其他发酵产物共变，并讨论同酒花感知和苦味的关联。 同源数据的背景说明，非独立的直接气味/抑制机制验证。
A_hops_noble 第 3 项	反式-2-壬烯醛 X.E..2.nonenal	其他；mg/L 均值未给出；中位未给出	一般背景 [10] 同位素研究支持老化期间反式-2-壬烯醛释放与纸板味相关。 本行目标为酒花感官，不能由纸板味证据解释其正相关，也不能与 Stale hops 等同。
A_hops_tropical 第 1 项	二甲基硫醚 DMS	麦芽；µg/L 均值 17.25；中位 19	一般背景 [4] DMS 具有卷心菜样气味，其感知受乙酸酯和芳樟醇改变。 没有直接验证 DMS 导致本行热带酒花感官。
A_hops_tropical 第 2 项	3-furanmethanol X3.furanmethanol	麦芽；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
A_hops_tropical 第 3 项	Methyl geranate methyl.geranate	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
A_hops_woody 第 1 项	Trans-dihydro-α-terpinyl acetate trans.dihydro.alpha.terpinyl.acetate	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
A_hops_woody 第 2 项	α-calacorene alpha.calacorene	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
A_hops_woody 第 3 项	Ethyl tetradecanoate ethyl.tetradecanoate	酵母；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
A_malt_all 第 1 项	乙醛 acetaldehyde	酵母；mg/L 均值 50；中位 50	一般背景 [7] 鲜啤感官实验以乙醛作生青味训练标准。 该训练标准未验证乙醛对本行麦芽、焦糖或酯香描述的直接作用。
A_malt_all 第 2 项	Methylpyrazine methylpyrazine	麦芽；mg/L 均值 1；中位 1	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。

感官与名次	化合物	来源与量纲	依据及适用范围
A_malt_all 第 3 项	乙醇 ethanol..v.v.	酵母；v/v 均值 1.75726；中位 1.7372	间接关联 [1] 原研究将酯类及多种醇类作为共同发酵来源讨论；可形成工艺/风格共变标记。 间接共变解释，不证明本行麦芽或酯类具体气味由该物质直接产生。
A_malt_bread 第 1 项	甘油 glycerol	酵母；g/L 均值 4.43333；中位 2	间接关联 [1] 原研究将酯类及多种醇类作为共同发酵来源讨论；可形成工艺/风格共变标记。 间接共变解释，不证明本行麦芽或酯类具体气味由该物质直接产生。
A_malt_bread 第 2 项	乙醇 ethanol..v.v.	酵母；v/v 均值 1.75726；中位 1.7372	间接关联 [1] 原研究将酯类及多种醇类作为共同发酵来源讨论；可形成工艺/风格共变标记。 间接共变解释，不证明本行麦芽或酯类具体气味由该物质直接产生。
A_malt_bread 第 3 项	松油烯-4-醇 terpinen.4.ol	香辛料；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
A_malt_burn 第 1 项	异丁醇 isobutanol	酵母；mg/L 均值 80；中位 100	间接关联 [1] 原研究将酯类及多种醇类作为共同发酵来源讨论；可形成工艺/风格共变标记。 间接共变解释，不证明本行麦芽或酯类具体气味由该物质直接产生。
A_malt_burn 第 2 项	二乙基硫醚 DES	酵母；μg/L 均值 1.2；中位 1.2	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
A_malt_burn 第 3 项	乙醇 ethanol..v.v.	酵母；v/v 均值 1.75726；中位 1.7372	间接关联 [1] 原研究将酯类及多种醇类作为共同发酵来源讨论；可形成工艺/风格共变标记。 间接共变解释，不证明本行麦芽或酯类具体气味由该物质直接产生。
A_malt_cara 第 1 项	乙醛 acetaldehyde	酵母；mg/L 均值 50；中位 50	一般背景 [7] 鲜啤感官实验以乙醛作生青味训练标准。 该训练标准未验证乙醛对本行麦芽、焦糖或酯香描述的直接作用。
A_malt_cara 第 2 项	2-乙基-1-己醇 X2.ethyl.1.hexanol	酵母；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。

感官与名次	化合物	来源与量纲	依据及适用范围
A_malt_cara 第 3 项	乙醇 ethanol..v.v.	酵母；v/v 均值 1.75726；中位 1.7372	间接关联 [1] 原研究将酯类及多种醇类作为共同发酵来源讨论；可形成工艺/风格共变标记。 间接共变解释，不证明本行麦芽或酯类具体气味由该物质直接产生。
A_malt_grain 第 1 项	松油烯-4-醇 terpinen.4.ol	香辛料；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
A_malt_grain 第 2 项	Beta-phellandrene beta.phellandrene	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
A_malt_grain 第 3 项	异龙脑 isoborneol	其他；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
F_esters_all 第 1 项	乙醛 acetaldehyde	酵母；mg/L 均值 50；中位 50	一般背景 [7] 鲜啤感官实验以乙醛作生青味训练标准。该训练标准未验证乙醛对本行麦芽、焦糖或酯香描述的直接作用。
F_esters_all 第 2 项	乙酸乙酯 ethyl_acetate	酵母；mg/L 均值 24；中位 23	直接气味依据 [1];[4] 乙酸乙酯具有果香/溶剂样气味；啤酒加标亦显示其改变部分缺陷感知。 气味与酯类/指甲油样描述相符，但不验证本样本相关系数或单独剂量效应。
F_esters_all 第 3 项	正己醇 n.hexanol	酵母；mg/L 均值未给出；中位未给出	一般背景 [7] 鲜啤 GC-O/AEDA 检出有气味活性的正己醇，嗅闻描述为山楂样。 可补充果香背景，未检验本行香蕉香或酯类评分的直接来源。
F_esters_ethac 第 1 项	乙酸乙酯 ethyl_acetate	酵母；mg/L 均值 24；中位 23	直接气味依据 [1];[4] 乙酸乙酯具有果香/溶剂样气味；啤酒加标亦显示其改变部分缺陷感知。 气味与酯类/指甲油样描述相符，但不验证本样本相关系数或单独剂量效应。
F_esters_ethac 第 2 项	乙醇 ethanol..v.v.	酵母；v/v 均值 1.75726；中位 1.7372	间接关联 [1] 原研究将酯类及多种醇类作为共同发酵来源讨论；可形成工艺/风格共变标记。 间接共变解释，不证明本行麦芽或酯类具体气味由该物质直接产生。

感官与名次	化合物	来源与量纲	依据及适用范围
F_esters_ethac 第 3 项	甘油 glycerol	酵母；g/L 均值 4.43333；中位 2	间接关联 [1] 原研究将酯类及多种醇类作为共同发酵来源讨论；可形成工艺/风格共变标记。 间接共变解释，不证明本行麦芽或酯类具体气味由该物质直接产生。
F_esters_flower 第 1 项	Beta-phellandrene beta.phellandrene	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
F_esters_flower 第 2 项	Isopentyl butanoate isopentyl.butanoate	酵母；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
F_esters_flower 第 3 项	Terpinolene terpinolene	香辛料；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
F_esters_fruity 第 1 项	二乙基硫醚 DES	酵母；μg/L 均值 1.2；中位 1.2	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
F_esters_fruity 第 2 项	乙酸 acetic_acid.g.L.	野生菌群；g/L 均值 0.14025；中位 0.145	一般背景 [5] 啤酒 GC-O/MS 报告乙酸的酸气味。 不验证本行酯类果味的直接产生机制。
F_esters_fruity 第 3 项	Humulene epoxide II humulene.epoxide.II	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
F_esters_isoaa 第 1 项	正己醇 n.hexanol	酵母；mg/L 均值未给出；中位未给出	一般背景 [7] 鲜啤 GC-O/AEDA 检出有气味活性的正己醇，嗅闻描述为山楂样。 可补充果香背景，未检验本行香蕉香或酯类评分的直接来源。
F_esters_isoaa 第 2 项	乙酸异戊酯 isopentyl_acetate	酵母；mg/L 均值 0.603333；中位 0.6	直接气味依据 [5] GC-O/MS 将乙酸异戊酯的气味标为香蕉。 相同气味描述的定性依据，非本样本关联的干预验证。
F_esters_isoaa 第 3 项	乙醛 acetaldehyde	酵母；mg/L 均值 50；中位 50	一般背景 [7] 鲜啤感官实验以乙醛作生青味训练标准。 该训练标准未验证乙醛对本行麦芽、焦糖或酯香描述的直接作用。

感官与名次	化合物	来源与量纲	依据及适用范围
F_hops_all 第 1 项	辛酸乙酯 ethyl_octanoate	酵母；mg/L 均值 0.7；中位 0.75	间接关联 [1];[5] GC-O/MS 报告果香/胶样气味；原研究报告同酒花感知和苦味的意外关联。 原研究与本题共享数据，非独立重复；未证明该酯直接产生酒花香或苦味。
F_hops_all 第 2 项	反式-2-壬烯醛 X.E..2.nonenal	其他；mg/L 均值未给出；中位未给出	一般背景 [10] 同位素研究支持老化期间反式-2-壬烯醛释放与纸板味相关。 本行目标为酒花感官，不能由纸板味证据解释其正相关，也不能与 Stale hops 等同。
F_hops_all 第 3 项	癸酸乙酯 ethyl_decanoate	酵母；mg/L 均值 1.035；中位 1.035	间接关联 [1] 原研究报告癸酸乙酯同其他发酵产物共变，并讨论同酒花感知和苦味的关联。 同源数据的背景说明，非独立的直接气味/抑制机制验证。
F_hops_citrus 第 1 项	Humulene epoxide II humulene.epoxide.II	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
F_hops_citrus 第 2 项	呋喃甲酸乙酯 ethyl.furoate	麦芽；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
F_hops_citrus 第 3 项	糠醛 furfural	麦芽；mg/L 均值 60；中位 37.5	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
F_hops_noble 第 1 项	辛酸乙酯 ethyl_octanoate	酵母；mg/L 均值 0.7；中位 0.75	间接关联 [1];[5] GC-O/MS 报告果香/胶样气味；原研究报告同酒花感知和苦味的意外关联。 原研究与本题共享数据，非独立重复；未证明该酯直接产生酒花香或苦味。
F_hops_noble 第 2 项	癸酸乙酯 ethyl_decanoate	酵母；mg/L 均值 1.035；中位 1.035	间接关联 [1] 原研究报告癸酸乙酯同其他发酵产物共变，并讨论同酒花感知和苦味的关联。 同源数据的背景说明，非独立的直接气味/抑制机制验证。
F_hops_noble 第 3 项	乙醇 ethanol..v.v.	酵母；v/v 均值 1.75726；中位 1.7372	间接关联 [1] 原研究将酯类及多种醇类作为共同发酵来源讨论；可形成工艺/风格共变标记。 间接共变解释，不证明本行麦芽或酯类具体气味由该物质直接产生。

感官与名次	化合物	来源与量纲	依据及适用范围
F_hops_tropical 第 1 项	香叶醇 geraniol	酒花；mg/L 均值 0.0315；中位 0.023	一般背景 [6] 香叶醇、香茅醇与芳樟醇在啤酒中具有果香/柑橘协同；芳樟醇亦有 GC-O 花香/柑橘依据。 三组分交互依赖基质；不支持其他目标（如 t2n）的直接抑制机制。
F_hops_tropical 第 2 项	乙酸己酯 hexyl.acetate	酵母；mg/L 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
F_hops_tropical 第 3 项	二甲基硫醚 DMS	麦芽；μg/L 均值 17.25；中位 19	一般背景 [4] DMS 具有卷心菜样气味，其感知受乙酸酯和芳樟醇改变。 没有直接验证 DMS 导致本行热带酒花感官。
F_hops_woody 第 1 项	Ethyl tetradecanoate ethyl.tetradecanoate	酵母；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
F_hops_woody 第 2 项	癸酸乙酯 ethyl_decanoate	酵母；mg/L 均值 1.035；中位 1.035	间接关联 [1] 原研究报告癸酸乙酯同其他发酵产物共变，并讨论同酒花感知和苦味的关联。 同源数据的背景说明，非独立的直接气味/抑制机制验证。
F_hops_woody 第 3 项	Humulene epoxide II humulene.epoxide.II	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
F_malt_all 第 1 项	乙醇 ethanol..v.v.	酵母；v/v 均值 1.75726；中位 1.7372	间接关联 [1] 原研究将酯类及多种醇类作为共同发酵来源讨论；可形成工艺/风格共变标记。 间接共变解释，不证明本行麦芽或酯类具体气味由该物质直接产生。
F_malt_all 第 2 项	乙醛 acetaldehyde	酵母；mg/L 均值 50；中位 50	一般背景 [7] 鲜啤感官实验以乙醛作生青味训练标准。 该训练标准未验证乙醛对本行麦芽、焦糖或酯香描述的直接作用。
F_malt_all 第 3 项	甲硫醇 MeSH	酵母；μg/L 均值未给出；中位未给出	一般背景 [1] 原研究描述甲硫醇在高浓度下呈硫臭、腐烂卷心菜样气味。 未证明本行麦芽或苦味由甲硫醇直接产生；来源与本题共享数据。

感官与名次	化合物	来源与量纲	依据及适用范围
F_malt_bread 第 1 项	乙醛 acetaldehyde	酵母；mg/L 均值 50；中位 50	一般背景 [7] 鲜啤感官实验以乙醛作生青味训练标准。该训练标准未验证乙醛对本行麦芽、焦糖或酯香描述的直接作用。
F_malt_bread 第 2 项	乙醇 ethanol..v.v.	酵母；v/v 均值 1.75726；中位 1.7372	间接关联 [1] 原研究将酯类及多种醇类作为共同发酵来源讨论；可形成工艺/风格共变标记。间接共变解释，不证明本行麦芽或酯类具体气味由该物质直接产生。
F_malt_bread 第 3 项	甘油 glycerol	酵母；g/L 均值 4.43333；中位 2	间接关联 [1] 原研究将酯类及多种醇类作为共同发酵来源讨论；可形成工艺/风格共变标记。间接共变解释，不证明本行麦芽或酯类具体气味由该物质直接产生。
F_malt_burn 第 1 项	乙醇 ethanol..v.v.	酵母；v/v 均值 1.75726；中位 1.7372	间接关联 [1] 原研究将酯类及多种醇类作为共同发酵来源讨论；可形成工艺/风格共变标记。间接共变解释，不证明本行麦芽或酯类具体气味由该物质直接产生。
F_malt_burn 第 2 项	甲硫醇 MeSH	酵母；μg/L 均值未给出；中位未给出	一般背景 [1] 原研究描述甲硫醇在高浓度下呈硫臭、腐烂卷心菜样气味。 未证明本行麦芽或苦味由甲硫醇直接产生；来源与本题共享数据。
F_malt_burn 第 3 项	甘油 glycerol	酵母；g/L 均值 4.43333；中位 2	间接关联 [1] 原研究将酯类及多种醇类作为共同发酵来源讨论；可形成工艺/风格共变标记。间接共变解释，不证明本行麦芽或酯类具体气味由该物质直接产生。
F_malt_cara 第 1 项	乙醛 acetaldehyde	酵母；mg/L 均值 50；中位 50	一般背景 [7] 鲜啤感官实验以乙醛作生青味训练标准。该训练标准未验证乙醛对本行麦芽、焦糖或酯香描述的直接作用。
F_malt_cara 第 2 项	乙醇 ethanol..v.v.	酵母；v/v 均值 1.75726；中位 1.7372	间接关联 [1] 原研究将酯类及多种醇类作为共同发酵来源讨论；可形成工艺/风格共变标记。间接共变解释，不证明本行麦芽或酯类具体气味由该物质直接产生。
F_malt_cara 第 3 项	丁酸乙酯 ethyl.butyrate	酵母；mg/L 均值 0.45；中位 0.45	一般背景 [5] GC-O/MS 对 3-甲基丁酸乙酯及丁酸乙酯报告果香/甜果香描述。 不验证本行醋酸、金属或麦芽焦糖感官的直接作用。

感官与名次	化合物	来源与量纲	依据及适用范围
F_malt_grain 第 1 项	异龙脑 isoborneol	其他；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
F_malt_grain 第 2 项	Terpinen-4-ol acetate terpinen.4.ol.acetate	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
F_malt_grain 第 3 项	反式-β-紫罗兰酮 X.E..beta.ionone	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
X4vg 第 1 项	4-乙烯基愈创木酚 X4VG	酵母；mg/L 均值 0.206333；中位 0.2	直接气味依据 [5] 啤酒 GC-O/MS 识别 4-乙烯基愈创木酚为丁香样气味。 定性气味依据，不代表预测关系或统一风格阈值已验证。
X4vg 第 2 项	Terpiryl acetate delta.terpinyl.acetate	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
X4vg 第 3 项	Neryl formate neryl.formate	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
acetic 第 1 项	Prenyl isobutyrate prenyl.isobutyrate	酵母；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
acetic 第 2 项	异戊酸乙酯 ethyl_isovalerate	其他；mg/L 均值 0.1；中位 0.1	一般背景 [5] GC-O/MS 对 3-甲基丁酸乙酯及丁酸乙酯报告果香/甜果香描述。 不验证本行醋酸、金属或麦芽焦糖感官的直接作用。
acetic 第 3 项	苯乙酸乙酯 ethyl.phenylacetate	酵母；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
acidity 第 1 项	Prenyl isobutyrate prenyl.isobutyrate	酵母；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。

感官与名次	化合物	来源与量纲	依据及适用范围
acidity 第 2 项	苯乙酸乙酯 ethyl.phenylacetate	酵母；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
acidity 第 3 项	Perillyl acetate perillyl.acetate	香辛料；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
barnyard 第 1 项	2-十一醇 X2.undecanol	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
barnyard 第 2 项	Neryl formate neryl.formate	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
barnyard 第 3 项	Allo-aromadendrene epoxide alloaromadendrene.epoxide	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
bitternes 第 1 项	辛酸乙酯 ethyl_octanoate	酵母；mg/L 均值 0.7；中位 0.75	间接关联 [1];[5] GC-O/MS 报告果香/胶样气味；原研究报告同酒花感知和苦味的意外关联。 原研究与本题共享数据，非独立重复；未证明该酯直接产生酒花香或苦味。
bitternes 第 2 项	癸酸乙酯 ethyl_decanoate	酵母；mg/L 均值 1.035；中位 1.035	间接关联 [1] 原研究报告癸酸乙酯同其他发酵产物共变，并讨论同酒花感知和苦味的关联。 同源数据的背景说明，非独立的直接气味/抑制机制验证。
bitternes 第 3 项	甲硫醇 MeSH	酵母；μg/L 均值未给出；中位未给出	一般背景 [1] 原研究描述甲硫醇在高浓度下呈硫臭、腐烂卷心菜样气味。 未证明本行麦芽或苦味由甲硫醇直接产生；来源与本题共享数据。
clove 第 1 项	柠檬烯 limonene	酒花；未给出 均值未给出；中位未给出	一般背景 [8] Amarillo 单酒花酿造对照中，柠檬烯等萜类与柑橘/花香谱共同变化。 非柠檬烯单体加标，不支持丁香机制，也未隔离本行橙味效应。
clove 第 2 项	桉叶油醇 eucalyptol	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。

感官与名次	化合物	来源与量纲	依据及适用范围
clove 第 3 项	樟脑 camphor	香辛料；未给出 均值未给出；中位未给出	一般背景 [9] 芫荽啤酒实验检出樟脑，并报告其与过量芳樟醇下的花香协同。 混合物花香不等于单体橙味或丁香味；本行关联仍未获直接验证。
coriander 第 1 项	2-methyl-benzofuran X2.methyl.benzofuran	麦芽；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
coriander 第 2 项	Ocimenone E.ocimenone	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
coriander 第 3 项	松油烯-4-醇 terpinen.4.ol	香辛料；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
diacetyl 第 1 项	丁香酚 eugenol	香辛料；mg/L 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
diacetyl 第 2 项	癸酸乙酯 ethyl_decanoate	酵母；mg/L 均值 1.035；中位 1.035	一般背景 [1] 原研究报告癸酸乙酯同其他发酵产物共变，并讨论同酒花感知和苦味的关联。 同源数据的背景说明，非独立的直接气味/抑制机制验证。
diacetyl 第 3 项	α-calacorene alpha.calacorene	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
dms 第 1 项	松油烯-4-醇 terpinen.4.ol	香辛料；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
dms 第 2 项	反式橙花叔醇 E.nerolidol	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
dms 第 3 项	Car-3-en-2-one car.3.en.2.one	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。

感官与名次	化合物	来源与量纲	依据及适用范围
lactic 第 1 项	乳酸 lactic_acid.mg.L.	野生菌群；mg/L 均值 416.667；中位 400	直接气味依据 [1] 原研究说明乳酸贡献酸啤酒的酸味。 支持酸感官背景；非本行果香等其他描述的直接来源证明。
lactic 第 2 项	Perillyl acetate perillyl.acetate	香辛料；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
lactic 第 3 项	Prenyl isobutyrate prenyl.isobutyrate	酵母；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
metallic 第 1 项	异戊酸乙酯 ethyl_isovalerate	其他；mg/L 均值 0.1；中位 0.1	一般背景 [5] GC-O/MS 对 3-甲基丁酸乙酯及丁酸乙酯报告果香/甜果香描述。 不验证本行醋酸、金属或麦芽焦糖感官的直接作用。
metallic 第 2 项	铁（总量字段） iron	其他；mg/L 均值 0.1；中位 0.1	一般背景 [11] 二价铁盐溶液的时间—强度实验观察到持续金属感。 盐种、价态、基质与啤酒总铁不同；只作背景，不外推阈值或甜味机制。
metallic 第 3 项	Methyl heptenone X6.methyl.5.hepten.2.one	酵母；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
orange 第 1 项	樟脑 camphor	香辛料；未给出 均值未给出；中位未给出	一般背景 [9] 芫荽啤酒实验检出樟脑，并报告其与过量芳樟醇下的花香协同。 混合物花香不等于单体橙味或丁香味；本行关联仍未获直接验证。
orange 第 2 项	柠檬烯 limonene	酒花；未给出 均值未给出；中位未给出	一般背景 [8] Amarillo 单酒花酿造对照中，柠檬烯等萜类与柑橘/花香谱共同变化。 非柠檬烯单体加标，不支持丁香机制，也未隔离本行橙味效应。
orange 第 3 项	乙醇 ethanol..v.v.	酵母；v/v 均值 1.75726；中位 1.7372	间接关联 [1] 原研究将酯类及多种醇类作为共同发酵来源讨论；可形成工艺/风格共变标记。 间接共变解释，不证明本行麦芽或酯类具体气味由该物质直接产生。
stale_hops 第 1 项	2-十一醇 X2.undecanol	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。

感官与名次	化合物	来源与量纲	依据及适用范围
stale_hops 第 2 项	Allo-aromadendrene epoxide alloaromadendrene.epoxide	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
stale_hops 第 3 项	Neryl formate neryl.formate	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
sweetness 第 1 项	癸酸乙酯 ethyl_decanoate	酵母；mg/L 均值 1.035；中位 1.035	一般背景 [1] 原研究报告癸酸乙酯同其他发酵产物共变，并讨论同酒花感知和苦味的关联。 同源数据的背景说明，非独立的直接气味/抑制机制验证。
sweetness 第 2 项	Pentyl octanoate pentyl.octanoate	酵母；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
sweetness 第 3 项	铁（总量字段） iron	其他；mg/L 均值 0.1；中位 0.1	一般背景 [11] 二价铁盐溶液的时间—强度实验观察到持续金属感。 盐种、价态、基质与啤酒总铁不同；只作背景，不外推阈值或甜味机制。
t2n 第 1 项	Beta-phellandrene beta.phellandrene	酒花；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
t2n 第 2 项	松油烯-4-醇 terpinen.4.ol	香辛料；未给出 均值未给出；中位未给出	尚未证实 本次核实文献未提供该化合物—感官项的直接证据。 仅保留统计候选，不作直接机制解释。
t2n 第 3 项	香叶醇 geraniol	酒花；mg/L 均值 0.0315；中位 0.023	一般背景 [6] 香叶醇、香茅醇与芳樟醇在啤酒中具有果香/柑橘协同；芳樟醇亦有 GC-O 花香/柑橘依据。 三组分交互依赖基质；不支持其他目标（如 t2n）的直接抑制机制。

表 B-3 给出 218 项化学候选中的预测依赖前三，各项标出平均ΔMSE 与正值折数。J 为与原边际相关前三的 Jaccard；原表 B-1 保持不变。完整 231 项置换结果、五折相关模块及分层 Bootstrap 结果随支撑材料保存。

表 B-3 全部感官的折外置换候选

感官	置换第 1 项	第 2 项	第 3 项	J
A_malt_all	lactic_acid.mg.L. Δ=0.0103；4/5	X4.ethyl.phenol Δ=0.00403；4/5	beta.phellandrene Δ=0.00341；5/5	0.00
A_malt_grain	cis.dihydro.beta.terpineol Δ=0.00156；5/5	ethyl_isovalerate Δ=0.00101；4/5	CS2 Δ=0.000621；4/5	0.00
A_malt_bread	lactic_acid.mg.L. Δ=0.00263；4/5	E.cinnamaldehyde Δ=0.00115；4/5	ethyl_acetate Δ=0.00101；3/5	0.00
A_malt_cara	lactic_acid.mg.L. Δ=0.00581；4/5	ethyl_isovalerate Δ=0.00329；4/5	X4.ethyl.phenol Δ=0.00199；4/5	0.00
A_malt_burn	lactic_acid.mg.L. Δ=0.00845；5/5	alpha.muurolol Δ=0.00216；4/5	ethyl.furoate Δ=0.00192；4/5	0.00
A_hops_all	ethyl_isovalerate Δ=0.0221；5/5	lactic_acid.mg.L. Δ=0.0117；5/5	ethyl_octanoate Δ=0.00838；5/5	0.20
A_hops_citrus	ethyl_isovalerate Δ=0.0044；4/5	lactic_acid.mg.L. Δ=0.00143；3/5	cis.dihydro.beta.terpineol Δ=0.00132；4/5	0.00
A_hops_tropical	ethyl_isovalerate Δ=0.0123；5/5	lactic_acid.mg.L. Δ=0.0093；4/5	isopentyl.butanoate Δ=0.00354；5/5	0.00
A_hops_noble	ethyl_isovalerate Δ=0.0103；5/5	ethyl_octanoate Δ=0.00527；5/5	ethyl_decanoate Δ=0.00378；5/5	0.50
A_hops_woody	ethyl_isovalerate Δ=0.00252；5/5	cis.dihydro.beta.terpineol Δ=0.000737；2/5	X1.octen.3.ol Δ=0.000352；5/5	0.00
A_esters_all	ethyl_acetate Δ=0.00411；5/5	ethyl_decanoate Δ=0.00157；4/5	ethyl_octanoate Δ=0.00146；4/5	0.20
A_esters_ethac	ethyl_acetate Δ=0.00124；4/5	X2E.6Z.farnesal Δ=0.000951；4/5	glycerol Δ=0.000707；5/5	0.20
A_esters_isoaa	beta.phellandrene Δ=0.00188；4/5	ethyl_acetate Δ=0.00178；5/5	ethyl.hexadecanoate Δ=0.00133；3/5	0.00
A_esters_flower	beta.phellandrene Δ=0.00192；5/5	X4.ethyl.guaiacol Δ=0.00115；4/5	ethyl_isovalerate Δ=0.00103；3/5	0.20
A_esters_fruity	beta.phellandrene Δ=0.00227；5/5	ethyl_isovalerate Δ=0.000667；3/5	X.E..alpha.ionone Δ=0.000477；3/5	0.20
F_malt_all	lactic_acid.mg.L. Δ=0.0179；5/5	X4.ethyl.phenol Δ=0.00538；4/5	X4.ethyl.guaiacol Δ=0.00395；4/5	0.00
F_malt_grain	cis.dihydro.beta.terpineol Δ=0.00202；3/5	ethyl_isovalerate Δ=0.00104；3/5	geraniol Δ=0.000468；5/5	0.00
F_malt_bread	lactic_acid.mg.L. Δ=0.00422；5/5	glycerol Δ=0.00137；5/5	cis.dihydro.beta.terpineol Δ=0.00136；5/5	0.20
F_malt_cara	lactic_acid.mg.L. Δ=0.00818；5/5	ethyl_isovalerate Δ=0.00362；5/5	X4.ethyl.phenol Δ=0.00193；4/5	0.00
F_malt_burn	lactic_acid.mg.L. Δ=0.0175；5/5	ethyl.furoate Δ=0.00485；5/5	cis.dihydro.beta.terpineol Δ=0.00425；3/5	0.00

感官	置换第 1 项	第 2 项	第 3 项	J
F_hops_all	ethyl_isovalerate Δ=0.0283；5/5	lactic_acid.mg.L. Δ=0.0184；5/5	cis.dihydro.beta.terpineol Δ=0.0145；5/5	0.00
F_hops_citrus	ethyl_isovalerate Δ=0.0034；4/5	lactic_acid.mg.L. Δ=0.00303；4/5	cis.dihydro.beta.terpineol Δ=0.00213；4/5	0.00
F_hops_tropical	ethyl_isovalerate Δ=0.00692；5/5	lactic_acid.mg.L. Δ=0.00635；5/5	cis.dihydro.beta.terpineol Δ=0.00188；3/5	0.00
F_hops_noble	ethyl_isovalerate Δ=0.0153；5/5	cis.dihydro.beta.terpineol Δ=0.00812；4/5	lactic_acid.mg.L. Δ=0.00683；5/5	0.00
F_hops_woody	ethyl_isovalerate Δ=0.00753；5/5	lactic_acid.mg.L. Δ=0.00204；3/5	methyl.geranate Δ=0.000755；3/5	0.00
F_esters_all	ethyl_acetate Δ=0.00521；5/5	ethyl_isovalerate Δ=0.00392；3/5	ethanol..v.v. Δ=0.00329；4/5	0.20
F_esters_ethac	ethanol..v.v. Δ=0.00247；4/5	ethyl_acetate Δ=0.00235；4/5	ethyl_octanoate Δ=0.00204；5/5	0.50
F_esters_isoaa	lactic_acid.mg.L. Δ=0.00221；4/5	beta.phellandrene Δ=0.0022；4/5	ethyl_acetate Δ=0.00208；5/5	0.00
F_esters_flower	beta.phellandrene Δ=0.0016；5/5	E.cinnamaldehyde Δ=0.00155；4/5	X4.ethyl.guaiacol Δ=0.000996；4/5	0.20
F_esters_fruity	MeSH Δ=0.00118；4/5	X.E..alpha.ionone Δ=0.00085；3/5	ethyl_acetate Δ=0.000756；5/5	0.00
acidity	lactic_acid.mg.L. Δ=0.134；5/5	X4.ethyl.phenol Δ=0.0115；3/5	X4.ethyl.guaiacol Δ=0.0115；4/5	0.00
bitternes	lactic_acid.mg.L. Δ=0.0273；5/5	cis.dihydro.beta.terpineol Δ=0.0208；5/5	ethyl_isovalerate Δ=0.0137；5/5	0.00
sweetness	E.cinnamaldehyde Δ=0.00832；5/5	ethyl_isovalerate Δ=0.00532；4/5	ethyl_octanoate Δ=0.00482；4/5	0.00
X4vg	X4VG Δ=0.00624；5/5	lactic_acid.mg.L. Δ=0.00556；4/5	ethyl_isovalerate Δ=0.00383；4/5	0.20
diacetyl	E.cinnamaldehyde Δ=0.00323；4/5	lactic_acid.mg.L. Δ=0.00298；4/5	ethyl_isovalerate Δ=0.00249；5/5	0.00
dms	cis.dihydro.beta.terpineol Δ=0.000687；4/5	methyl.geranate Δ=0.000541；5/5	X2.butyloctanoate Δ=0.000523；5/5	0.00
metallic	ethyl_isovalerate Δ=0.0079；5/5	lactic_acid.mg.L. Δ=0.00266；4/5	ethyl_octanoate Δ=0.00175；4/5	0.20
stale_hops	ethyl_acetate Δ=0.000843；4/5	heptyl.acetate Δ=0.000664；3/5	beta.phellandrene Δ=0.000553；3/5	0.00
t2n	ethyl_isovalerate Δ=0.00217；4/5	cis.dihydro.beta.terpineol Δ=0.000953；4/5	ethyl_acetate Δ=0.000943；4/5	0.00
orange	linalool Δ=0.00574；5/5	ethyl_isovalerate Δ=0.0056；3/5	ethyl..E..linalool Δ=0.0052；4/5	0.00

感官	置换第 1 项	第 2 项	第 3 项	J
coriander	ethyl_isovalerate Δ=0.00478；3/5	geraniol Δ=0.00176；4/5	X1.alpha.terpineol Δ=0.00152；4/5	0.00
clove	cis.dihydro.beta.terpineol Δ=0.00462；3/5	limonene Δ=0.00354；5/5	alpha.muurolol Δ=0.00242；4/5	0.20
lactic	lactic_acid.mg.L. Δ=0.0868；5/5	ethyl_isovalerate Δ=0.00547；3/5	E.cinnamaldehyde Δ=0.00503；3/5	0.20
acetic	lactic_acid.mg.L. Δ=0.0891；5/5	ethyl_isovalerate Δ=0.00927；4/5	X4.ethyl.phenol Δ=0.00865；4/5	0.20
barnyard	ethyl_acetate Δ=0.00096；4/5	heptyl.acetate Δ=0.000697；3/5	beta.phellandrene Δ=0.000601；3/5	0.00

附录 C 逐源迁移检查

表 C-1 采用固定树、中央 6004 池、retain80、余量优先口径。两个 Q95 为预留当前模型及新种子旧协议模型的配对预测差分位数；任一大于零均记为失败。改善率已包含所有选中谱，不删除失败的 beer 96。完整 72 点策略网格及十情景化学检查随电子附录保存。

表 C-1 全部 11 款源的余量优先迁移检查

beer	距离改善	保留锚幅度	当前 Q95	旧协议 Q95	审计结果
124	59.14%	235.85%	-0.04880	-0.03263	通过
81	63.76%	90.04%	-0.08334	-0.15091	通过
218	65.02%	101.31%	-0.20229	-0.10714	通过
96	47.35%	163.70%	+0.02100	-0.04175	未通过
230	57.74%	87.86%	-0.07386	-0.09737	通过
185	66.46%	275.68%	-0.00883	-0.05151	通过
139	56.25%	85.21%	-0.05956	-0.06710	通过
209	38.78%	177.04%	-0.04141	-0.06655	通过
119	56.75%	86.27%	-0.10962	-0.14841	通过
90	35.11%	166.18%	-0.03585	-0.03919	通过
28	69.35%	101.80%	-0.29602	-0.32845	通过

表 C-2 全部 72 组迁移策略结果

模型	候选集	接近下限	排序	选出	通过	改善中位数
固定树	收缩 6004	≥10%	几何	11/11	10/11	70.08%
固定树	收缩 6004	≥10%	余量	11/11	10/11	56.75%
固定树	收缩 6004	≥30%	几何	11/11	10/11	70.08%
固定树	收缩 6004	≥30%	余量	11/11	10/11	56.75%
固定树	收缩 6004	≥50%	几何	10/11	10/11	70.08%
固定树	收缩 6004	≥50%	余量	10/11	10/11	57.95%
固定树	收缩 6004	≥60%	几何	10/11	10/11	70.08%
固定树	收缩 6004	≥60%	余量	10/11	10/11	66.75%
固定树	收缩 6004	保留 100%	几何	11/11	10/11	70.08%
固定树	收缩 6004	保留 100%	余量	11/11	10/11	66.46%
固定树	收缩 6004	保留 80%	几何	11/11	10/11	70.08%
固定树	收缩 6004	保留 80%	余量	11/11	10/11	57.74%
固定树	原 2004	≥10%	几何	9/11	7/11	64.18%

模型	候选集	接近下限	排序	选出	通过	改善中位数
固定树	原 2004	≥10%	余量	9/11	7/11	21.90%
固定树	原 2004	≥30%	几何	6/11	6/11	65.75%
固定树	原 2004	≥30%	余量	6/11	6/11	55.18%
固定树	原 2004	≥50%	几何	6/11	6/11	65.75%
固定树	原 2004	≥50%	余量	6/11	6/11	60.88%
固定树	原 2004	≥60%	几何	5/11	5/11	65.78%
固定树	原 2004	≥60%	余量	5/11	5/11	65.78%
固定树	原 2004	保留 100%	几何	8/11	6/11	64.95%
固定树	原 2004	保留 100%	余量	8/11	6/11	64.95%
固定树	原 2004	保留 80%	几何	9/11	7/11	64.18%
固定树	原 2004	保留 80%	余量	9/11	7/11	54.38%
固定树	原 6004	≥10%	几何	9/11	7/11	64.18%
固定树	原 6004	≥10%	余量	9/11	8/11	20.53%
固定树	原 6004	≥30%	几何	7/11	7/11	65.72%
固定树	原 6004	≥30%	余量	7/11	7/11	54.38%
固定树	原 6004	≥50%	几何	6/11	6/11	65.75%
固定树	原 6004	≥50%	余量	6/11	6/11	60.88%
固定树	原 6004	≥60%	几何	5/11	5/11	65.78%
固定树	原 6004	≥60%	余量	5/11	5/11	65.78%
固定树	原 6004	保留 100%	几何	8/11	6/11	64.95%
固定树	原 6004	保留 100%	余量	8/11	8/11	64.95%
固定树	原 6004	保留 80%	几何	9/11	7/11	64.18%
固定树	原 6004	保留 80%	余量	9/11	9/11	54.38%
岭回归	收缩 6004	≥10%	几何	7/11	5/11	57.13%
岭回归	收缩 6004	≥10%	余量	7/11	7/11	32.63%
岭回归	收缩 6004	≥30%	几何	7/11	5/11	57.13%
岭回归	收缩 6004	≥30%	余量	7/11	6/11	32.64%
岭回归	收缩 6004	≥50%	几何	5/11	4/11	57.70%
岭回归	收缩 6004	≥50%	余量	5/11	5/11	53.17%
岭回归	收缩 6004	≥60%	几何	2/11	2/11	70.10%
岭回归	收缩 6004	≥60%	余量	2/11	2/11	63.12%
岭回归	收缩 6004	保留 100%	几何	3/11	2/11	66.55%
岭回归	收缩 6004	保留 100%	余量	3/11	3/11	66.13%

模型	候选集	接近下限	排序	选出	通过	改善中位数
岭回归	收缩 6004	保留 80%	几何	5/11	3/11	57.70%
岭回归	收缩 6004	保留 80%	余量	5/11	5/11	53.17%
岭回归	原 2004	≥10%	几何	8/11	6/11	26.85%
岭回归	原 2004	≥10%	余量	8/11	7/11	15.93%
岭回归	原 2004	≥30%	几何	2/11	2/11	49.14%
岭回归	原 2004	≥30%	余量	2/11	2/11	49.82%
岭回归	原 2004	≥50%	几何	1/11	1/11	68.12%
岭回归	原 2004	≥50%	余量	1/11	1/11	68.12%
岭回归	原 2004	≥60%	几何	1/11	1/11	68.12%
岭回归	原 2004	≥60%	余量	1/11	1/11	68.12%
岭回归	原 2004	保留 100%	几何	3/11	2/11	27.22%
岭回归	原 2004	保留 100%	余量	3/11	2/11	27.22%
岭回归	原 2004	保留 80%	几何	3/11	2/11	27.22%
岭回归	原 2004	保留 80%	余量	3/11	1/11	27.22%
岭回归	原 6004	≥10%	几何	9/11	8/11	28.55%
岭回归	原 6004	≥10%	余量	9/11	7/11	18.21%
岭回归	原 6004	≥30%	几何	4/11	4/11	31.09%
岭回归	原 6004	≥30%	余量	4/11	4/11	30.75%
岭回归	原 6004	≥50%	几何	1/11	1/11	68.12%
岭回归	原 6004	≥50%	余量	1/11	1/11	68.12%
岭回归	原 6004	≥60%	几何	1/11	1/11	68.12%
岭回归	原 6004	≥60%	余量	1/11	1/11	68.12%
岭回归	原 6004	保留 100%	几何	3/11	2/11	29.23%
岭回归	原 6004	保留 100%	余量	3/11	2/11	27.22%
岭回归	原 6004	保留 80%	几何	4/11	3/11	25.18%
岭回归	原 6004	保留 80%	余量	4/11	3/11	22.28%

改善中位数按全部选中谱计算，包括复核未通过者；无候选时记为横线。每组分母均为 11，不以选出的条数代替源样本总数。逐化合物十情景检查及全部数值文件随支撑材料提交。

附录 D 软件环境与运行方法

模型原计算环境为 Python 3.13，精确依赖列于表 D-1。程序采用 UTF-8 文本、正斜杠相对路径和固定换行规则，支持 Windows。原始赛题附件与 solution_a 文件夹并列，保持官方目录名；数据不重复打包。所有计算源码及论文构建程序全文列于附录 E。

表 D-1 计算依赖版本

软件包	版本
cloudpickle	3.1.2
contourpy	1.4.0
cycler	0.12.1
et_xmlfile	2.0.0
fonttools	4.67.0
joblib	1.6.0
kiwisolver	1.5.1
matplotlib	3.11.2
narwhals	2.27.1
numpy	2.5.4
openpyxl	3.1.5
packaging	26.3
pandas	3.1.0
pillow	13.0.0
pyparsing	3.3.3
pypdf	6.21.2
python-dateutil	3.0.0
scikit-learn	1.9.1
scipy	1.18.1
six	1.17.0
threadpoolctl	3.7.0
tzdata	2026.5

在 Windows PowerShell 中进入解压后的根目录，建立独立环境并安装依赖：

```
py -3.13 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r solution_a/requirements-lock.txt
```

对包含完整 outputs 的仓库，默认只核查现有结果，不重新训练或改写冻结记录：

```
.\.venv\Scripts\python.exe -X utf8 solution_a/run_all.py
```

支撑材料中的 paper_results 和 supplement_results 为引用结果副本，不作为训练输出目录。首次完整重建在新解压目录执行：

```
.\.venv\Scripts\python.exe -X utf8 solution_a/run_all.py --rebuild
```

重建须保留全部原始附件，依次执行数据核验、预测、关联分析、风格生成、迁移及补充验证。程序对非空历史或补充输出拒绝覆盖；重新试验应另建副本。补充验证源码与结果位于 supplement 目录，run_supplement.py 可单独核查。计算耗时随硬件变化。不同操作系统、数学库及并行实现可能出现小幅浮点差，须结合模型参数及逐项指标判断。

论文构建另需 python-docx、lxml、reportlab 和 matplotlib。Windows 中由 Word 兼容 COM 接口导出正文 PDF，再拼接文字型源码附录。完整依赖、命令及文件校验方法见支撑材料 README.txt。

附录 E 完整计算机源程序

本附录按文件给出计算、校验及论文构建所用源程序。源代码以可选取的文本排印，左侧为原文件行号；长行折行处不另编号。运行时使用支撑包中的同名文件，避免 PDF 阅读器复制时丢失缩进。文件清单、源码全文和支撑包采用同一份程序目录。

表 E-1 源程序目录

文件（相对 solution_a）	行数
paper/build_paper.py	404
paper/export_pdf.ps1	26
paper/make_core.py	101
paper/make_figures.py	468
paper/native_math.py	168
paper/package_support.py	86
paper/source_appendix.py	145
paper/submission_content.py	96
paper/supplement_sections.py	115
paper/verify_paper.py	173
run_all.py	52
run_q3.py	10
run_q3_domain.py	38
run_q3_optimized.py	37
run_q3_refined.py	40
run_supplement.py	30
src/01_audit.py	467
src/02_q1_baseline.py	173
src/02b_q1_robust.py	93
src/03_q2_network.py	246
src/04_stage_summary.py	131
src/05_verify_outputs.py	111
src/06_q3_sensory.py	230
src/07_q3_generation.py	419
src/08_q3_transfer.py	320
src/09_q3_report.py	271
src/10_q3_verify.py	543
src/11_q3_generation_optimized.py	288
src/12_q3_stale_optimized.py	241

文件（相对 solution_a）	行数
src/12b_q3_stale_fixed.py	217
src/13_q3_transfer_optimized.py	335
src/14_q3_optimization_report.py	293
src/15_q3_optimized_verify.py	732
src/16_q3_stale_domain.py	323
src/17_q3_transfer_domain.py	337
src/18_q3_domain_report.py	513
src/19_q3_domain_verify.py	566
src/20_q3_transfer_consensus.py	21
src/21_q3_beer81_audit.py	453
src/22_q3_stale_refined.py	406
src/23_q3_transfer_balanced.py	328
src/24_q3_refinement_report.py	188
src/25_q3_refined_verify.py	728
src/common.py	73
src/platform_selfcheck.py	118
src/q3_copula.py	152
src/q3_data.py	122
src/q3_generator_optimized.py	222
src/q3_stale_domain.py	158
src/q3_stale_optimized.py	180
src/q3_stale_refined.py	205
supplement/q1_additional_checks.py	67
supplement/q1_comparison.py	151
supplement/q2_permutation.py	189
supplement/q2_stratified_bootstrap.py	191
supplement/verify_supplement.py	155

paper/build_paper.py

SHA-256: 38f824678d238151e03f78adb133996e26e3714879387c101d8311a789096403

```
"""Assemble the editable submission paper from verified section and result files."""
from copy import deepcopy
import hashlib
import json
from pathlib import Path
import re
from zipfile import ZipFile

import pandas as pd
from docx import Document
from docx.enum.style import WD_STYLE_TYPE
from docx.enum.table import WD_ALIGN_VERTICAL, WD_TABLE_ALIGNMENT
from docx.enum.text import WD_ALIGN_PARAGRAPH, WD_BREAK, WD_TAB_ALIGNMENT, WD_TAB_LEADER
from docx.oxml import OxmlElement
from docx.oxml.ns import qn
from docx.shared import Cm, Pt, RGBColor

from native_math import to_omml

HERE = Path(__file__).resolve().parent
ROOT = HERE.parent
REPO = ROOT.parent
REFERENCE = next((REPO / "2026年第七届MathorCup数学应用挑战赛—大数据竞赛初赛赛题").glob("*论文模板.docx"))
FINAL = HERE / "啤酒风味建模提交稿.docx"
(HERE/'qa').mkdir(parents=True,exist_ok=True)
from submission_content import add_submission_appendices
from supplement_sections import add_main_sections

core = json.loads((HERE / "sections/core.json").read_text(encoding="utf-8"))
figs = json.loads((HERE / "figure_manifest.json").read_text(encoding="utf-8"))["figures"]
FIGURES = {Path(x["relativepath"]).stem: x for x in figs}


def paragraph(text):
    return {"type": "paragraph", "text": text}


def heading(text, level=1):
    return {"type": "heading", "level": level, "text": text}


def table(caption, headers, rows, widths=None, compact=False):
    return {"type": "table", "caption": caption, "headers": headers,
            "rows": rows, "widths": widths, "compact": compact}


q1, q2, q3 = [json.loads((HERE / f"sections/q{i}.json").read_text(encoding="utf-8"))["blocks"] for i in [1, 2, 3]]
q1, q2, q3 = add_main_sections(q1, q2, q3, ROOT)
q1.insert(next(i for i, b in enumerate(q1) if b.get("text", "").startswith("5.6 ")), {"type": "figure", "key": "figure2_q1"})
# Place figures close to the results they support.
q2.insert(next(i for i, b in enumerate(q2) if b.get("text", "").startswith("6.3 ")), {"type": "figure", "key": "figure3_q2"})
q3.insert(next(i for i, b in enumerate(q3) if b.get("text", "").startswith("7.5 ")), {"type": "figure", "key": "figure4_generation"})
```

paper/build_paper.py（续）

```
q3.insert(next(i for i, b in enumerate(q3) if b.get("text", "").startswith("7.5 ")), {"type": "figure", "key": "figure5_profiles"})
q3.insert(next(i for i, b in enumerate(q3) if b.get("text", "").startswith("7.10 ")), {"type": "figure", "key": "figure6_transfer"})
blocks = deepcopy(core["opening"] + q1 + q2 + q3 + core["ending"])

refs = json.loads((HERE / "chemical_references.json").read_text(encoding="utf-8"))["references"]
refs.extend([
    {"number": 2, "authors": ["Pierre Geurts", "Damien Ernst", "Louis Wehenkel"],
     "title": "Extremely randomized trees", "journal": "Machine Learning", "year": 2006,
     "volume": "63", "pages": "3–42", "doi": "10.1007/s10994-006-6226-1",
     "url": "https://doi.org/10.1007/s10994-006-6226-1"},
    {"number": 3, "authors": ["Arthur E. Hoerl", "Robert W. Kennard"],
     "title": "Ridge Regression: Biased Estimation for Nonorthogonal Problems", "journal": "Technometrics",
     "year": 1970, "volume": "12", "issue": "1", "pages": "55–67", "doi": "10.1080/00401706.1970.10488634",
     "url": "https://www.stat.cmu.edu/technometrics/70-79/VOL-12-01/v1201055.pdf"},
    {"number": 4, "authors": ["Han Liu", "John Lafferty", "Larry Wasserman"],
     "title": "The Nonparanormal: Semiparametric Estimation of High Dimensional Undirected Graphs",
     "journal": "Journal of Machine Learning Research", "year": 2009, "volume": "10", "pages": "2295–2328",
     "doi": None, "url": "https://www.jmlr.org/papers/v10/liu09a.html"},
])

# Reference numbers follow first appearance, including table cells.
order = []
def find_refs(value):
    if isinstance(value, str):
        for number in re.findall(r"\[(\d+)\]", value):
            if int(number) not in order:
                order.append(int(number))
    elif isinstance(value, dict):
        for val in value.values(): find_refs(val)
    elif isinstance(value, list):
        for val in value: find_refs(val)
find_refs(blocks)
mapping = {old: i+1 for i, old in enumerate(order)}
def remap(value):
    if isinstance(value, str):
        return re.sub(r"\[(\d+)\]", lambda m: f"[{mapping[int(m[1])]}]", value)
    if isinstance(value, list): return [remap(v) for v in value]
    if isinstance(value, dict): return {k: remap(v) for k, v in value.items()}
    return value
blocks = remap(blocks)
reference_lookup = {r["number"]: r for r in refs}
references = [dict(reference_lookup[old], original_number=old, number=mapping[old]) for old in order]
(HERE / "references.json").write_text(json.dumps(references, ensure_ascii=False, indent=2)+"\n", encoding="utf-8")
chemical_meta=remap(json.loads((HERE/"chemical_references.json").read_text(encoding="utf-8")))
for ref in chemical_meta['references']:
    ref['original_number']=ref['number']
    ref['number']=mapping[ref['number']]
(HERE/'chemical_references_final.json').write_text(json.dumps(chemical_meta,ensure_ascii=False,indent=2)+"\n", encoding="utf-8")

```

paper/build_paper.py（续）

```
def format_reference(r):
    authors = ", ".join(r["authors"][:3]) + (", et al." if len(r["authors"])>3 else ".")
    vol = r.get("volume", "") + (f"({r['issue']})" if r.get("issue") else "")
    source = f"{r['journal']}, {vol}: {r['pages']}, {r['year']}."
    link = f"DOI: {r['doi']}." if r.get("doi") else f"{r['url']}，访问日期2026-10-23。"
    return f"[{r['number']}] {authors} {r['title']}[J]. {source} {link}"

blocks += [heading("使用声明"), paragraph((ROOT / "docs/use_statement.md").read_text(encoding="utf-8").strip().splitlines()[-1]), heading("参考文献")]
blocks += [{"type": "reference", "text": format_reference(r)} for r in references]
appendix_start = len(blocks)
blocks += [heading("附录A 感官字段及逐维精度"), paragraph("表A-1列出第一问ExtraTrees的全部45维折外误差。A_前缀表示香气，F_前缀为相应风味；其余字段按附件字典保留。所有值均为附件标准化尺度，完整250×45折外预测随电子附录保存。")]
metrics = pd.read_csv(ROOT / "outputs/q1/metrics_by_target.csv")
metrics = metrics.loc[metrics.model.eq("extra_trees")]
blocks.append(table("表A-1 全部感官维度的五折折外指标", ["目标字段", "MAE", "RMSE", "R²", "Spearman"],
                    [[r.target, f"{r.mae:.4f}", f"{r.rmse:.4f}", f"{r.macro_r2:.4f}", f"{r.spearman:.4f}"] for r in metrics.itertuples()],
                    [6.0, 2.5, 2.5, 2.5, 2.5], True))
blocks += [heading("附录B 完整Top 3候选"), paragraph("表B-1沿用绝对Spearman相关排名，不按文献证据后验改选。每项依次给出附件字段名、带符号相关ρ与Bootstrap入选频率π。化学名称、来源、单位、阈值及逐行证据等级见表B-2及电子附件chemical_evidence_final.csv；135个位置涉及60种不同化合物。")]
top = pd.read_csv(ROOT / "outputs/q2/top3_compounds.csv")
rows = []
for target in metrics.target:
    selected = top.loc[top.sensory_target.eq(target)].sort_values("rank")
    assert len(selected)==3
    rows.append([target] + [f"{r.compound}\nρ={r.spearman_rho:.3f}，π={r.bootstrap_top3_frequency:.3f}" for r in selected.itertuples()])
blocks.append(table("表B-1 45项感官的全部Top-3关联候选", ["感官字段", "第1项", "第2项", "第3项"], rows, [3.1, 4.3, 4.3, 4.3], True))
evidence = pd.read_csv(HERE / "chemical_evidence.csv").fillna("")
for col in [c for c in evidence.columns if "ref" in c]:
    evidence[col] = evidence[col].map(lambda x: remap(x) if isinstance(x, str) else x)
for col in evidence.columns:
    evidence[col]=evidence[col].map(lambda x: x.replace('chemical_references.json','chemical_references_final.json') if isinstance(x,str) else x)
evidence.to_csv(HERE / "chemical_evidence_final.csv", index=False, encoding="utf-8-sig", lineterminator="\n")
blocks += [heading("附录C 逐源迁移检查"), paragraph("表C-1采用固定树、中央6004池、retain80、余量优先口径。两个Q95为预留当前模型及新种子旧协议模型的配对预测差分位数；任一大于零均记为失败。改善率已包含所有选中谱，不删除失败的beer 96。完整72点策略网格及十情景化学检查随电子附录保存。")]
transfer = pd.read_csv(ROOT / "outputs/q3_refined/transfer/transfer_summary.csv")
selected = transfer.loc[transfer.arm.eq("fixed") & transfer.pool.eq("contracted_6004") & transfer.bound.eq("retain80") & transfer.ranking.eq("margin")]
blocks.append(table("表C-1 全部11款源的余量优先迁移检查", ["beer", "距离改善", "保留锚幅度", "当前Q95", "旧协议Q95", "审计结果"],
                    [[str(r.source_beer), f"{r.target_improvement:.2%}", f"{r.retained_anchor_gain:.2%}", f"{r.new_audit_q95:+.5f}", f"{r.old_fresh_audit_q95:+.5f}", "通过" if r.fresh_consistency_passed else "未通过"] for r in selected.itertuples()],
                    [1.3, 2.5, 3.2, 3.0, 3.0, 2.5], True))
blocks = add_submission_appendices(blocks, ROOT, HERE, mapping)

# Save a structured editable source with every table included.
manuscript = dict(title=core["title"], abstract=core["abstract"], keywords=core["keywords"], blocks=blocks)
```

paper/build_paper.py（续）

```
(HERE / "manuscript.json").write_text(json.dumps(manuscript, ensure_ascii=False, indent=2)+"\n", encoding="utf-8")

# Document the source-derived page system before editing its working copy.
with ZipFile(REFERENCE) as z:
    inventory = {name: hashlib.sha256(z.read(name)).hexdigest() for name in z.namelist() if not name.endswith("/")}
template_sha = hashlib.sha256(REFERENCE.read_bytes()).hexdigest()
(HERE / "qa/template_inventory.json").write_text(json.dumps(inventory, indent=2)+"\n", encoding="utf-8")
(HERE / "qa/artifact.md").write_text(f"""# Paper template execution contract
Reference: {REFERENCE}\nSHA-256: {template_sha}\nReference render: qa/template/page-1.png through page-3.png.
The retained source has three portrait A4 sections, 2.00025 cm top/bottom and 2.50119 cm side margins, footer distance 1.74978 cm.
Source roles: centered 14 pt title and contents heading; 12 pt abstract/body; Chinese serif body and Times New Roman Latin text. Existing style and opaque package parts are retained.
Editable: main body placeholder paragraphs and identity table; section header/footer text; styles used for manuscript headings and tables; document properties; settings; image relationships. Replace all placeholder content, retain the three-section abstract/contents/body system, and restart body page numbers at one.
Preserve-only: font table, theme and customXml parts; inventory in template_inventory.json. The original template is never changed.
Source template is sparse and has no completed mathematical/table/figure content. Added components use 12 pt body with 1.25 line spacing, 14/12 pt headings, 9.5/10 pt tables, native OMML equations and 300 dpi figures. Table cells wrap and repeat headers. No fixed row heights.
The identity placeholders and source page decorations are removed to satisfy anonymous paper and no-header rules. Chinese font uses the installed Windows SimSun/SimHei and equations use Cambria Math. The source document grid is disabled for content paragraphs because its interaction with the CJK font doubled line spacing; the specified 1.25 body spacing is used instead, with unchanged page geometry.
Title/abstract occupy section one, static linked contents section two, body and appendices section three. Contents page caches are computed from the rendered draft and PAGEREF fields are marked for Word refresh. Word-compatible COM updates the displayed fields during PDF export without saving over the constructed DOCX.
Slots: source body paragraphs 3/4/5/22 are title/abstract/keywords; 23–35 are contents placeholders and break; 36 onward is body. These slots may expand. All source placeholder and table text is intentionally removed.
Fidelity gates: template hash, A4 geometry, three sections, anonymous headers/footers, no placeholders, intact preserve-only parts, every final page visually checked, formulas and references reconciled.
""", encoding="utf-8")

doc = Document(REFERENCE)
sections = [deepcopy(s._sectPr) for s in doc.sections]
for spec in sections:
    grid=spec.find(qn("w:docGrid"))
    if grid is not None: spec.remove(grid)
body = doc._element.body
for element in list(body): body.remove(element)
body.append(sections[-1])

def style(name, size, bold=False, family="SimSun"):
    if name not in doc.styles:
        doc.styles.add_style(name, WD_STYLE_TYPE.PARAGRAPH)
    s = doc.styles[name]
    s.font.name = "Times New Roman"
    s.font.size = Pt(size)
    s.font.bold = bold
```

paper/build_paper.py（续）

```
    s.font.color.rgb = RGBColor(0, 0, 0)
    fonts = s.element.get_or_add_rPr().get_or_add_rFonts()
    fonts.set(qn("w:eastAsia"), family)
    fonts.set(qn("w:ascii"), "Times New Roman")
    fonts.set(qn("w:hAnsi"), "Times New Roman")
    for key in ["asciiTheme", "hAnsiTheme", "eastAsiaTheme"]:
        fonts.attrib.pop(qn("w:"+key), None)
    pf = s.paragraph_format
    pf.line_spacing = 1.25
    pf.space_before = Pt(0)
    pf.space_after = Pt(4)
    pf.widow_control = True
    snap=OxmlElement("w:snapToGrid");snap.set(qn("w:val"),"0");s.element.get_or_add_pPr().append(snap)
    return s

normal = style("Normal", 12)
normal.paragraph_format.first_line_indent = Pt(24)
normal.paragraph_format.alignment = WD_ALIGN_PARAGRAPH.JUSTIFY
style("Title", 14, True).paragraph_format.alignment = WD_ALIGN_PARAGRAPH.CENTER
for level, size in [(1, 14), (2, 12), (3, 12)]:
    s=style(f"Heading {level}", size, True, "SimHei")
    s.paragraph_format.first_line_indent = Pt(0)
    s.paragraph_format.space_before = Pt(12 if level==1 else 7)
    s.paragraph_format.space_after = Pt(6)
    s.paragraph_format.keep_with_next = True
    s.paragraph_format.keep_together = True
    outline=OxmlElement("w:outlineLvl");outline.set(qn("w:val"), str(level-1));s.element.get_or_add_pPr().append(outline)
for name, size in [("Caption",10.5),("TableText",10),("ReferenceText",10.5),("ContentsText",10.5)]:
    s=style(name,size)
    s.paragraph_format.first_line_indent=Pt(0)
style("Equation",11).paragraph_format.first_line_indent=Pt(0)

def add_text(text, sty=None):
    x=doc.add_paragraph(style=sty)
    x.add_run(text)
    return x

def section_break(spec):
    x=doc.add_paragraph()
    x.paragraph_format.space_after=Pt(0)
    x.paragraph_format.line_spacing=1
    x.paragraph_format.first_line_indent=Pt(0)
    x._p.get_or_add_pPr().append(deepcopy(spec))
    return x

def bookmark(x, name, number):
    start=OxmlElement("w:bookmarkStart");start.set(qn("w:id"),str(number));start.set(qn("w:name"),name)
    end=OxmlElement("w:bookmarkEnd");end.set(qn("w:id"),str(number))
    x._p.insert(0,start);x._p.append(end)

def field(x, instruction, display):
    node=OxmlElement("w:fldSimple");node.set(qn("w:instr"),instruction)
    r=OxmlElement("w:r");t=OxmlElement("w:t");t.text=str(display);r.append(t);node.append(r);x._p.append(node)

title=add_text(core["title"],"Title")
title.paragraph_format.space_after=Pt(12)
```

paper/build_paper.py（续）

```
title.paragraph_format.first_line_indent=Pt(0)
abstract_heading=add_text("摘 要","Heading 2")
abstract_heading.alignment=WD_ALIGN_PARAGRAPH.CENTER
for text in core["abstract"]: add_text(text)
keywords=add_text("关键词："+core["keywords"])
keywords.paragraph_format.first_line_indent=Pt(0)
keywords._p.get_or_add_pPr().append(deepcopy(sections[0]))

toc_title=add_text("目 录","Title")
toc_title.paragraph_format.space_after=Pt(15)
headings=[]
for i,b in enumerate(blocks):
    if b["type"]=="heading":
        text=b["text"]
        if b["level"]==1 or (b["level"]==2 and re.match(r"[567]\.\d+ ",text)):
            headings.append((i,b,f"sec_{i}"))
page_map_path=HERE/"qa/page_map.json"
page_map=json.loads(page_map_path.read_text(encoding="utf-8")) if page_map_path.exists() else {}
for i,b,name in headings:
    x=add_text("", "ContentsText")
    x.paragraph_format.line_spacing=Pt(13)
    x.paragraph_format.space_after=Pt(0)
    x.paragraph_format.left_indent=Cm(.45 if b["level"]==2 else 0)
    x.paragraph_format.tab_stops.add_tab_stop(Cm(15.8), WD_TAB_ALIGNMENT.RIGHT, WD_TAB_LEADER.DOTS)
    link=OxmlElement("w:hyperlink");link.set(qn("w:anchor"),name)
    r=OxmlElement("w:r");t=OxmlElement("w:t");t.text=b["text"];r.append(t);link.append(r);x._p.append(link)
    x.add_run("\t")
    field(x,f"PAGEREF {name} \\h",page_map.get(b["text"],""))
section_break(sections[1])

def add_table(b):
    cap=add_text(b["caption"],"Caption")
    cap.alignment=WD_ALIGN_PARAGRAPH.CENTER
    cap.paragraph_format.keep_with_next=True
    widths=b.get("widths")
    if not widths:
        n=len(b["headers"])
        widths=([6]+[10/(n-1)]*(n-1)) if n>2 else [4.5,11.5]
    tbl=doc.add_table(rows=1, cols=len(b["headers"]))
    tbl.alignment=WD_TABLE_ALIGNMENT.CENTER
    tbl.autofit=False
    for col,w in zip(tbl.columns,widths): col.width=Cm(w)
    allrows=[b["headers"]]+b["rows"]
    for n,values in enumerate(allrows):
        row=tbl.rows[0] if n==0 else tbl.add_row()
        trpr=row._tr.get_or_add_trPr()
        cant=OxmlElement("w:cantSplit");trpr.append(cant)
        if n==0:
            repeat=OxmlElement("w:tblHeader");trpr.append(repeat)
        for j,(cell,value) in enumerate(zip(row.cells,values)):
            cell.width=Cm(widths[j]);cell.vertical_alignment=WD_ALIGN_VERTICAL.CENTER
            pr=cell._tc.get_or_add_tcPr()
            shade=OxmlElement("w:shd");shade.set(qn("w:fill"),"EDEFF2" if n==0 else "FFFFFF");pr.append(shade)
            borders=OxmlElement("w:tcBorders")
            for side in ["top","left","bottom","right"]:
                edge=OxmlElement("w:"+side);edge.set(qn("w:val"),"single");edge.set(qn("w:sz"),"4");edge.set(qn("w:color"),"D9D9D9");borders.append(edge)
```

paper/build_paper.py（续）

```
            pr.append(borders)
            margins=OxmlElement("w:tcMar")
            for side,amount in [("top",65),("bottom",65),("left",80),("right",80)]:
                v=OxmlElement("w:"+side);v.set(qn("w:w"),str(amount));v.set(qn("w:type"),"dxa");margins.append(v)
            pr.append(margins)
            x=cell.paragraphs[0];x.style=doc.styles["TableText"]
            x.paragraph_format.line_spacing=1.12;x.paragraph_format.space_after=Pt(0)
            x.paragraph_format.keep_with_next=n==0 or (len(b["rows"])<=4 and n<len(b["rows"]))
            val=str(value)
            x.alignment=WD_ALIGN_PARAGRAPH.CENTER if (j>0 and len(val)<20) or n==0 else WD_ALIGN_PARAGRAPH.LEFT
            r=x.add_run(val);r.font.size=Pt(9.5 if b.get("compact") else 10)
            r.bold=n==0
    # Keep the gap after the table small and explicit.
    x=doc.add_paragraph();x.paragraph_format.space_after=Pt(3);x.paragraph_format.line_spacing=.3
    x.paragraph_format.first_line_indent=Pt(0)

bookmark_records=[]
for index,b in enumerate(blocks):
    kind=b["type"]
    if kind=="heading":
        x=add_text(b["text"],f"Heading {b['level']}")
        if b['text'].startswith('附录'): x.paragraph_format.page_break_before=True
        bookmark(x,f"sec_{index}",1000+index)
        bookmark_records.append({"text":b["text"],"name":f"sec_{index}","level":b["level"]})
    elif kind in ["paragraph","reference"]:
        x=add_text(b["text"],"ReferenceText" if kind=="reference" else None)
        if kind=="reference":
            x.paragraph_format.left_indent=Cm(.65);x.paragraph_format.first_line_indent=Cm(-.65)
    elif kind=="equation":
        x=doc.add_paragraph(style="Equation")
        x.paragraph_format.keep_together=True
        x.paragraph_format.space_before=Pt(3);x.paragraph_format.space_after=Pt(5)
        x.paragraph_format.tab_stops.add_tab_stop(Cm(7.8),WD_TAB_ALIGNMENT.CENTER)
        x.paragraph_format.tab_stops.add_tab_stop(Cm(15.8),WD_TAB_ALIGNMENT.RIGHT)
        x.add_run("\t");x._p.append(to_omml(b["latex"]));x.add_run("\t（"+b["label"]+"）")
    elif kind=="code":
        x=doc.add_paragraph()
        x.paragraph_format.first_line_indent=Pt(0)
        x.paragraph_format.line_spacing=Pt(12)
        x.paragraph_format.keep_together=True
        r=x.add_run(b["text"])
        r.font.name="Consolas";r.font.size=Pt(9)
    elif kind=="table": add_table(b)
    elif kind=="figure":
        fig=FIGURES[b["key"]]
        x=doc.add_paragraph();x.alignment=WD_ALIGN_PARAGRAPH.CENTER
        x.paragraph_format.first_line_indent=Pt(0);x.paragraph_format.keep_with_next=True
        x.paragraph_format.line_spacing=1
        x.paragraph_format.space_before=Pt(0);x.paragraph_format.space_after=Pt(3)
        x.add_run().add_picture(str(HERE/fig["relativepath"]),width=Cm(15 if b['key']=='figure5_profiles' else 16))
        caption=fig["caption"].replace("预先列定的", "选定的")
        if b['key']=='figure4_generation':
```

```
            caption='图4 三种风格的生成能量差异。柱为外层六折均值，误差线为折间标准差，非置信区间；每折先平均两个模拟种子。'
        if b['key']=='figure5_profiles':
            caption='图5 三种风格的12维代表感官画像。点为预测均值，线为每风格300条生成谱的5%–95%范围，并非预测置信区间。Stale hops用非Pils单目标树，其余维度用226维多输出代理；均为标准化尺度。'
        x=add_text(caption,"Caption");x.alignment=WD_ALIGN_PARAGRAPH.JUSTIFY
        x.paragraph_format.keep_together=True
    else: raise ValueError(kind)

# Clear source header/footer placeholders and use centered body-only numbering.
for i,section in enumerate(doc.sections):
    section.header.is_linked_to_previous=False
    section.footer.is_linked_to_previous=False
    for part in [section.header,section.footer]:
        for node in list(part._element): part._element.remove(node)
        part.add_paragraph()
    if i==2:
        x=section.footer.paragraphs[0];x.alignment=WD_ALIGN_PARAGRAPH.CENTER
        x.paragraph_format.first_line_indent=Pt(0)
        field(x,"PAGE",1)
    for hdr in section._sectPr.findall(qn("w:headerReference")):
        section._sectPr.remove(hdr)
    pn=section._sectPr.find(qn("w:pgNumType"))
    if i==2:
        if pn is None: pn=OxmlElement("w:pgNumType");section._sectPr.append(pn)
        pn.set(qn("w:start"),"1")
settings=doc.settings.element
update=settings.find(qn("w:updateFields"))
if update is None: update=OxmlElement("w:updateFields");settings.append(update)
update.set(qn("w:val"),"true")
mathpr=settings.find(qn("m:mathPr"))
if mathpr is None: mathpr=OxmlElement("m:mathPr");settings.append(mathpr)
font=mathpr.find(qn("m:mathFont"))
if font is None: font=OxmlElement("m:mathFont");mathpr.append(font)
font.set(qn("m:val"),"Cambria Math")
doc.core_properties.title=core["title"]
doc.core_properties.author=""
doc.core_properties.last_modified_by=""
doc.core_properties.comments=""
doc.core_properties.subject="啤酒风味建模"
doc.core_properties.keywords=core["keywords"]
doc.save(FINAL)

# python-docx normalizes relationship XML even for untouched custom parts.
# Restore explicitly preserve-only package members from the retained template.
preserve=[name for name in inventory if name.startswith('customXml/') or name.startswith('word/theme/') or name=='word/fontTable.xml']
temporary=FINAL.with_suffix('.tmp.docx')
with ZipFile(REFERENCE) as original, ZipFile(FINAL) as built, ZipFile(temporary,'w') as out:
    for info in built.infolist():
        data=original.read(info.filename) if info.filename in preserve else built.read(info.filename)
        out.writestr(info,data)
temporary.replace(FINAL)

assert hashlib.sha256(REFERENCE.read_bytes()).hexdigest()==template_sha
with ZipFile(FINAL) as z:
    for name in preserve:
        assert hashlib.sha256(z.read(name)).hexdigest()==inventory[name],name
```

paper/build_paper.py（续）

```
    xml=z.read('word/document.xml').decode()
    assert len(re.findall(r'<w:sectPr(?:\s|>)',xml))==3
    assert xml.count('<m:oMath>')==sum(b['type']=='equation' for b in blocks)
    for old in ['论文题目','摘要正文','这里开始论文正文','MCB260XXXX','键入章级']:
        assert old not in xml,old
(HERE/"qa/bookmarks.json").write_text(json.dumps(bookmark_records,ensure_ascii=False,indent=2)+"\n", encoding="utf-8")
print(f"Created {FINAL.name}: {len(blocks)} blocks, {sum(b['type']=='equation' for b in blocks)} native equations, {len(references)} references.")

```

paper/export_pdf.ps1

SHA-256: bb2fefe160eb6e17e3d59fa7774f2a0c5bdb887362b0c13ef719019e6214f603

```
param(
    [string]$InputDocx = (Join-Path $PSScriptRoot '啤酒风味建模提交稿.docx'),
    [string]$OutputPdf = (Join-Path $PSScriptRoot 'qa/main.pdf')
)
$ErrorActionPreference = 'Stop'
$inputPath = (Resolve-Path -LiteralPath $InputDocx).Path
$outputPath = [IO.Path]::GetFullPath($OutputPdf)
[IO.Directory]::CreateDirectory([IO.Path]::GetDirectoryName($outputPath)) | Out-Null
$word = $null
$document = $null
try {
    $word = New-Object -ComObject Word.Application
    $word.Visible = $false
    $word.DisplayAlerts = 0
    $document = $word.Documents.Open($inputPath, $false, $true)
    $document.Repaginate()
    $document.Fields.Update() | Out-Null
    $document.Repaginate()
    $document.Fields.Update() | Out-Null
    $document.ExportAsFixedFormat($outputPath, 17)
    if (-not (Test-Path -LiteralPath $outputPath)) { throw 'PDF export did not create a file.' }
    Get-Item -LiteralPath $outputPath | Select-Object FullName, Length
} finally {
    if ($null -ne $document) { try { $document.Close(0) } catch { Write-Verbose $_ } }
    if ($null -ne $word) { try { $word.Quit() } catch { Write-Verbose $_ } }
}
```

paper/make_core.py

SHA-256: 203d7fb18a1417246c2f11df20e4db4e58cb6bb3093c0ac289be5161bcc448f9

```
"""Build manuscript framing from the adopted experimental scope."""
import json
from pathlib import Path

HERE = Path(__file__).resolve().parent


def h(text, level=1):
    return {"type": "heading", "level": level, "text": text}


def p(text):
    return {"type": "paragraph", "text": text}


def table(caption, headers, rows):
    return {"type": "table", "caption": caption, "headers": headers, "rows": rows}


opening = [
    h("1 问题重述"),
    p("啤酒的香气与风味由多类化合物共同形成，原料、发酵和陈化过程都会改变其组成。化学成分与感官强度并非一一对应，混合物作用及风格差异增加了预测和解释的难度。本题以原研究的商用啤酒化学测量、训练品评及相关数据[1]为基础，要求完成评分预测、关键化合物分析和风格生成。"),
    p("第一问保留化学测量存在缺失的样本，预测Aroma与Flavour属性。题目要求0–6评分，而实际监督标签已标准化，需先核实评分量尺。第二问利用化合物相关矩阵构建网络，为每项感官属性选择Top-3候选，并检验其稳定性和化学依据。"),
    p("第三问生成Tripel、Kriek和Lambic的化学谱及感官画像，再将Pils/Lager向Tripel迁移，限制DMS、Diacetyl、4VG、trans-2-nonenal和Stale hops等缺陷恶化。评价包括生成分布质量、目标风格接近度和逐款约束检查。"),
    h("2 数据核验与统一处理"),
    h("2.1 样本范围与感官目标", 2),
    p("对附件逐表读取并按beer核对，附件1、4、7均包含250个唯一啤酒编号，覆盖22个风格。附件1除三个身份或类别字段外有231个数值特征；附件4有50项感官属性，依据附件8的名称字典，选取Aroma 15项与Taste 30项，共45项作为本题目标，排除alcohol、aftertaste、body、co2和overall。Tripel、Kriek、Lambic和Pils/Lager分别有28、10、11和11款。题面背景的226款与实际文件不一致，本文以可一一核对的250款为计算样本，不按文字数量截断。"),
    table("表2-1 主要附件的实际范围及用途", ["附件", "实际数据范围", "本文用途"], [
        ["1.csv", "250款 × 231个数值特征", "第一问输入与第二问候选分析"],
        ["2.csv", "231 × 231相关矩阵", "化合物关联网络与数值复核"],
        ["3.xlsx", "12张评分卡及汇总公式", "确定评分规则与名称映射"],
        ["4.csv", "250款 × 50个感官属性", "筛选45维连续监督目标"],
        ["5.csv与6.csv", "50 × 50及44 × 50", "描述性矩阵核验，不作折外筛选先验"],
        ["7.csv", "250款 × 186个描述词变量", "辅助诊断，未作为生成谱的预测输入"],
        ["8.xlsx", "校准量、阈值及名称等工作表", "字段、单位与化学阈值核实"],
    ]),
    p("按beer进行一对一连接。附件4、7的条码均只剩96个不同值，不能作为主键；编号和条码也不作为模型输入。附件3的12张评分卡均未填写，汇总公式的缓存零值不算实测评分。"),
    h("2.2 缺失值、尺度与异常值", 2),
    p("附件1有73个化学缺失单元，占250×231的0.1264%，分布于9列、63款酒（25.2%样本），最高单列缺失率为10.8%。保留全部250款，仅用训练数据估计补值参数。45项感官目标均无缺失。化学表包含对数或相对测量及理化量，负值不能一律删除，各列也不能统一取指数还原浓度。"),
    p("与附件8校准量对照，37个基础字段在有效比较范围内符合以10为底的对数关系，零值曾以最小正值的一半替代；其余重叠GC-MS字段不满足统一直接log10关系。第一、二问保留给定数值表示；第三问删除五项汇总或派生量，并以69项匹配校准字段替换，形成69维校准量与157维相对量的226维混合表示。校准数据中的3个负值转为缺失，混合表共保留72个缺失单元。相对量没有统一浓度单位，化学阈值只在核实过的校准字段上执行。"),
    p("极端观测通过定位和模型对照处理。beer 156的输入极值影响线性外推，beer 81的高Stale hops评分难以由化学近邻解释，均未发现足以删除记录的证据。原始附件保留不变，处理过程和结果来源另存
```

```
    核验记录。"),
    h("2.3 相关矩阵及信息边界", 2),
    p("对附件2逐对复算Spearman相关，最大绝对差为6.66×10⁻¹⁶；与Pearson的差异明显。因此按实物矩阵采用Spearman，而不沿用题面对Pearson的文字称谓。附件5、6也可由对应样本的Spearman相关复现。附件6实际仅44个化学指标，不代替完整化学数据。"),
    p("附件2、5、6均由本批样本导出，用于描述性分析；预测交叉验证前不使用这些矩阵或附件8的GBM重要性、SHAP筛选变量。论坛表缺少评论全文、时间和批次信息，无法补充真实品评标签。"),
    h("3 问题分析与技术路线"),
    p("三问使用同一套样本键和字段字典，评价对象各有不同：第一问检验真实啤酒的折外预测误差；第二问分析本批样本的关联与风格混杂；第三问检验生成分布，再搜索满足约束的迁移谱。"),
    p("预测采用多输出树，并与均值和岭回归比较；解释采用可直接复算的秩相关；生成采用经验边缘与收缩依赖结构。迁移先检查化学浓度与训练支持范围，再检查预测缺陷变化，比较目标接近度和保护约束之间的取舍。"),
    {"type": "figure", "key": "figure1_workflow"},
    p("第一问采用随机五折；生成方法在训练集内选择，再由外层留出数据评价；Stale hops按非Pils风格分组验证。Pils标签已多次用于开发诊断，不能再视为独立测试集。迁移失败的源酒始终保留在通过率分母中。"),
    h("4 模型假设与符号"),
    h("4.1 建模假设及影响", 2),
    p("假设一，现有风格混合中的啤酒可作为随机五折的评价单位，训练与测试按编号隔离。不同批次或全新风格的预测能力需另行验证。"),
    p("假设二，附件的45维连续感官值在本批样本间可比。预测沿用该标准化尺度；缺少逐人评分及转换参数时，不推定其与原始0–6分存在已知映射。"),
    p("假设三，经验边缘和收缩秩相关可近似小样本风格分布，通过留出分布距离、零频率、方差和近邻指标检验；生产可行性另需实测。"),
    p("假设四，附件阈值可作为化学缺陷的参考。用阈值均值、中位数及扰动情景检查敏感性；Stale hops没有单一化学对应项，另用感官预测约束。"),
    h("4.2 主要符号", 2),
    table("表4-1 主要符号及含义", ["符号", "含义", "尺度或范围"], [
        ["n，p，K", "样本数、化学维数、感官维数", "n=250；p=231或226；K=45"],
        ["$x_i$，$y_i$，$s_i$", "化学向量、感官向量、风格标签", "化学量按字段；感官为标准化值"],
        ["T，$m_j$,T，$d_{ij}$", "训练集、训练中位数、缺失指示", "$d_{ij}\in\{0,1\}$"],
        ["$\rho_{jl}$，τ，$\pi_{jk}$", "秩相关、网络阈值、入选频率", "$\tau\in\{0.6,0.7,0.8\}$；$\pi\in[0,1]$"],
        ["$F_j$，Φ，z(x)", "经验边缘、标准正态CDF、高斯秩向量", "无量纲"],
        ["$\mu_s$，$\Sigma_s$，$R_s$", "风格秩中心、协方差、相关矩阵", "秩空间参数"],
        ["c_d，T_d，e_d", "缺陷浓度、感官阈值、超阈程度", "浓度与阈值单位一致；e_d无量纲"],
        ["d(x)，$r_i$(x)，$G_i$(x)", "目标距离、修改幅度、距离改善率", "秩空间距离；G为相对改善"],
        ["f_b，Δ_b，$Q_{0.95}$", "第b个代理、配对预测差、95%分位数", "标准化感官尺度"],
    ]),
]

ending = [
    h("8 结果检验与模型评价"),
    h("8.1 误差与稳定性证据的适用范围", 2),
    p("第一问RMSE较均值降低15.36%，新增20%随机缺失后增加2.67%，但逐维性能差异明显。Stale hops及部分酯香描述符仍接近均值预测，不能由酸度等强项推及所有感官。评价仅使用一个随机五折种子，且以附件已有的数据处理为条件。"),
    p("第二问的网络边数随阈值由266降至109再降至39；Top-3平均Bootstrap入选频率为0.455，说明不少化合物可相互替代。去风格均值后21项感官的Top-3完全变化，表明全局相关受风格差异影响。统计稳定性与直接气味依据需分别判断。"),
    p("第三问Kriek、Lambic的能量差异下降，但部分指标仍逊于直接重采样。生成方法在贴近分布与形成新组合之间存在取舍。感官画像的分位区间描述生成谱之间的差异，不是新酒预测误差的置信区间。"),
    h("8.2 阈值与迁移策略敏感性", 2),
    p("化学约束取阈值均值、中位数及各自±20%扰动的十个情景交集。浓度保护通过并不代表Stale hops感官检查通过，两者分别报告。"),
    p("72组迁移方案比较两种预测模型、三个候选池、六种接近下限和两种排序。主比较保持预测模型、检查规则、下限与候选数相同；几何优先结果作为探索性方案展示。逐源结果保留无候选的酒及beer 96失败。"),
    p("固定树与稳定Ridge在相同候选空间分别有10/11和5/11款通过，说明可行方案依赖预测模型。Ridge在非Pils分组验证中略好，却未改善Pils误差。预留重抽样模型的复核仍来自同一批训练酒，无法替代
```

paper/make_core.py（续）

```
    独立品评。"),
    h("8.3 方法优点与主要不足", 2),
    p("本方法统一核对身份、字段、量尺与浓度单位，补值和选型限定在训练数据内。预测、关联、生成分别设置对照，迁移保留候选数、约束和失败记录，结果可以按样本追溯。"),
    p("主要不足是原始0–6评分无法校准，小风格样本偏少，且多数化学候选尚缺直接机制证据。Stale hops跨风格预测尤其薄弱；附件没有储存、批次及重复品评条件，无法据此解释误差来源。"),
    h("9 结论与建议"),
    p("基于250款啤酒，本文完成45维标准化感官预测、化合物关联分析和风格条件生成。ExtraTrees五折RMSE为0.4862；|ρ|≥0.7时网络有109条边，报告135个Top-3位置及其稳定性。三种指定风格各生成300条化学谱，并给出感官预测画像。"),
    p("Pils/Lager向Tripel迁移时，固定树、中央候选池和余量优先方案有10/11款通过模型检查，所选谱目标距离缩短中位数为57.74%；几何优先的探索性结果为70.08%。这些结果支持进一步筛选化学目标谱，尚需理化检测和盲品验证。"),
    p("后续应补充逐人原始评分及标准化参数，并对陈旧感高的酒收集储存时间、温度、包装、批次和重复品评记录。化学候选可通过加标、重组或省略试验检验；迁移方案应在新批次样本上重新测试。"),
]

abstract = [
    "针对啤酒化学测量存在缺失、成分相关性强及部分风格样本较少的问题，研究感官预测、关键化合物分析和风格迁移。按啤酒编号核对附件后，采用250款啤酒、45项感官属性；第一、二问使用231维化学输入，第三问使用69项校准量与157项相对量。现有感官标签为标准化连续值，无法从附件恢复原始0–6评分，故在给定尺度上建模和评价。",
    "第一问采用训练折中位数补值、缺失指示与多输出ExtraTrees，五折折外MAE为0.3409、RMSE为0.4862、宏平均R²为0.2146，RMSE较均值基线降低15.36%；额外随机隐藏20%测试特征后，RMSE增加2.67%。第二问复核附件相关矩阵后采用Spearman相关，在绝对相关阈值0.7下得到231个节点、109条边，并为45项感官属性选出135个Top-3关联位置。200次Bootstrap中仅14个位置的入选频率达到0.8，去风格均值后21项感官的Top-3完全变化。",
    "第三问比较收缩Copula、局部边缘Copula和平滑混合，通过嵌套验证选择生成方法。Kriek、Lambic的能量差异较基准分别降低20.5%、23.4%，Tripel略有变差；三种目标风格共生成900条化学谱并预测感官画像。Pils/Lager向Tripel迁移时，同时限制四项缺陷浓度和Stale hops预测增量。固定树、中央候选池的余量优先方案有10/11款通过模型检查，所选谱目标距离缩短中位数为57.74%；几何优先的探索性方案为70.08%。由于Stale hops跨风格预测仍弱，上述结果有待新样本检测和盲品验证。",
]

payload = {"title": "基于集成学习与条件Copula的啤酒风味预测及受约束风格迁移",
           "abstract": abstract,
           "keywords": "啤酒风味；多输出预测；关联网络；条件Copula；受约束迁移",
           "opening": opening, "ending": ending}
(HERE / "sections").mkdir(parents=True, exist_ok=True)
(HERE / "sections/core.json").write_text(json.dumps(payload, ensure_ascii=False, indent=2)+"\n", encoding="utf-8", newline="\n")

```

paper/make_figures.py

SHA-256: 9afa35400e3fb8c506fd2bd9652115f0fa7f6def743d654aa9afce88c368314e

```
"""Render source-backed, 16 cm research figures without changing experiment outputs.""
"
from __future__ import annotations

import json
import os
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager
from matplotlib.font_manager import FontProperties
from matplotlib.patches import FancyArrowPatch, Rectangle
from matplotlib.text import Text
import numpy as np
import pandas as pd


PAPER = Path(__file__).resolve().parent
SOLUTION = PAPER.parent
OUTPUT = SOLUTION / "outputs"
FIGURES = PAPER / "figures"
font_candidates = [os.environ.get("MATHORCUP_FONT", ""),
                   "C:/Windows/Fonts/simsun.ttc",
                   "/System/Library/Fonts/Supplemental/Songti.ttc",
                   "/usr/share/fonts/opentype/noto/NotoSerifCJK-Regular.ttc"]
CJK_FONT = next((Path(f) for f in font_candidates if f and Path(f).is_file()), None)
if CJK_FONT is None:
    raise FileNotFoundError("Set MATHORCUP_FONT to an installed Chinese font file.")
font_manager.fontManager.addfont(str(CJK_FONT))
FONT = FontProperties(fname=str(CJK_FONT))
BLUE = "#315F82"
GOLD = "#A98039"
GREY = "#A3A8AC"
DARK = "#30383D"
LIGHT = "#E5E8EA"
FAIL = "#A14D40"
WIDTH_CM = 16.0
plt.rcParams.update({
    "font.family": FONT.get_name(), "font.size": 9,
    "axes.labelsize": 9, "axes.titlesize": 10,
    "xtick.labelsize": 8.5, "ytick.labelsize": 8.5,
    "legend.fontsize": 8.5, "axes.unicode_minus": False,
    "axes.edgecolor": "#69737A", "axes.linewidth": 0.6,
    "text.color": DARK, "axes.labelcolor": DARK,
    "xtick.color": DARK, "ytick.color": DARK,
    "svg.fonttype": "path", "savefig.facecolor": "white",
    "figure.facecolor": "white", "axes.spines.top": False,
    "axes.spines.right": False,
})


def read_csv(relative: str) -> pd.DataFrame:
    return pd.read_csv(OUTPUT / relative)


def new_figure(height_cm: float):
```

paper/make_figures.py（续）

```
    return plt.figure(figsize=(WIDTH_CM / 2.54, height_cm / 2.54), dpi=150)


def save(figure, name: str):
    # Apply an explicit font file to every text object, including tick labels.
    for text in figure.findobj(match=Text):
        text.set_fontproperties(FontProperties(
            fname=str(CJK_FONT), size=text.get_fontsize(), weight=text.get_fontweight(
))
    figure.savefig(FIGURES / f"{name}.png", dpi=300)
    figure.savefig(FIGURES / f"{name}.svg")
    plt.close(figure)


def grid(axis, direction="x"):
    axis.set_axisbelow(True)
    axis.grid(axis=direction, color=LIGHT, linewidth=0.6)
    axis.tick_params(length=2.5, width=0.6)


def figure1():
    fig = new_figure(11.2)
    ax = fig.add_axes([0.025, 0.025, 0.95, 0.95])
    ax.set(xlim=(0, 1), ylim=(0, 1))
    ax.axis("off")

    def box(x, y, w, h, text, highlighted=False):
        ax.add_patch(Rectangle((x, y), w, h,
                               facecolor="#F0F4F7" if highlighted else "white",
                               edgecolor=BLUE if highlighted else "#89959D", linewidth=
0.8))
        ax.text(x + w / 2, y + h / 2, text, ha="center", va="center", fontsize=9,
                linespacing=1.35)

    def arrow(start, end):
        ax.add_patch(FancyArrowPatch(start, end, arrowstyle="-|>", mutation_scale=8,
                                     linewidth=0.7, color="#6C7A84"))

    box(0.06, 0.864, 0.88, 0.105,
        "250 款啤酒 / 22 种风格：化学测量、标准化感官评分、校准量与阈值", True)
    box(0.11, 0.717, 0.78, 0.105,
        "按 beer 精确连接；区分相对表示与浓度；在训练折内拟合预处理")
    arrow((0.5, 0.864), (0.5, 0.823))
    centers = [0.163, 0.5, 0.837]
    xs = [0.01, 0.347, 0.684]
    top = ["问题一：感官预测\n231 维化学 → 45 维感官",
           "问题二：化学关联\n231 个节点 / 218 个候选",
           "问题三：生成与迁移\n226 维：69 校准 + 157 相对"]
    mid = ["ExtraTrees 与基线\n训练折补值和缺失指示",
           "Spearman 网络与 Top-3\n200 次配对重抽样",
           "风格自适应分布与嵌套选择\n三风格 900 谱 + 对照 300 谱"]
    bottom = ["五折折外评价\n缺失压力检查",
              "135 个关联位置\n稳定性与风格敏感性",
              "排除全部 Pils：239 款训练\n11 款源谱向 Tripel 迁移"]
    for i, x in enumerate(xs):
        arrow((centers[i], 0.717), (centers[i], 0.657))
        box(x, 0.53, 0.306, 0.125, top[i], True)
        box(x, 0.34, 0.306, 0.13, mid[i])
        box(x, 0.15, 0.306, 0.13, bottom[i])
        arrow((centers[i], 0.53), (centers[i], 0.471))
        arrow((centers[i], 0.34), (centers[i], 0.281))
```

paper/make_figures.py（续）

```
    box(0.355, 0.004, 0.632, 0.103,
        "迁移检查：十情景浓度保护 + 支持域 + 接近幅度\n双模型筛选 → 预留模型审计；失败源保留")
    arrow((0.837, 0.15), (0.837, 0.108))
    save(fig, "figure1_workflow")


def figure2():
    base = read_csv("q1/metrics_overall.csv").set_index("model")
    robust = read_csv("q1_robust/metrics_overall.csv").set_index("model")
    table = pd.concat([base.loc[["mean", "ridge"]], robust.loc[["ridge_winsor"]],
                       base.loc[["extra_trees"]]])
    fig = new_figure(11.2)
    axes = fig.subplots(2, 2)
    fig.subplots_adjust(left=0.17, right=0.975, top=0.93, bottom=0.17,
                        hspace=0.68, wspace=0.65)
    labels = ["均值基线", "Ridge", "裁剪 Ridge", "ExtraTrees"]
    colors = [GREY, GREY, GOLD, BLUE]
    for ax, metric, title in zip(axes.flat[:3], ["rmse", "mae", "macro_r2"],
                                 ["(a) RMSE", "(b) MAE", "(c) 宏平均 $R^2$"]):
        values = table[metric].to_numpy()
        y = np.arange(4)
        ax.barh(y, values, height=0.56, color=colors)
        ax.set_yticks(y, labels)
        ax.invert_yaxis()
        ax.set_title(title, loc="left", pad=5)
        if metric == "macro_r2":
            ax.set_xlim(-0.1, 0.31)
            ax.axvline(0, color="#69737A", linewidth=0.65)
        else:
            ax.set_xlim(0, 0.72 if metric == "rmse" else 0.52)
        for pos, value in zip(y, values):
            label_x = 0.014 if value < 0 else value + (0.012 if metric == "rmse" else 0.009)
            ax.text(label_x, pos,
                    f"{value:.3f}", va="center", fontsize=8.5)
        grid(ax)
    ax = axes[1, 1]
    rates = np.array([0, 10, 20])
    values = np.array([base.loc["extra_trees", "rmse"],
                       base.loc["extra_trees_mask10", "rmse"],
                       base.loc["extra_trees_mask20", "rmse"]])
    ax.plot(rates, values, "o-", color=BLUE, markersize=4, linewidth=1.2)
    ax.set_title("(d) ExtraTrees 缺失压力", loc="left", pad=5)
    ax.set(xlim=(-1.5, 21.5), ylim=(0.47, 0.525), xticks=rates,
           xlabel="额外隐藏已观测特征（%）", ylabel="RMSE")
    for rate, value in zip(rates, values):
        ax.annotate(f"{value:.4f}", (rate, value), xytext=(0, 6),
                    textcoords="offset points", ha="center", fontsize=8.5)
    grid(ax, "y")
    fig.text(0.5, 0.025,
             f"五折折外评价：20% 缺失压力下 RMSE 上升 {(values[2]/values[0]-1)*100:.2f}%",
             ha="center", fontsize=8.5)
    save(fig, "figure2_q1")


NETWORK_SHORT_NAMES = {
    "ethyl.tetradecanoate": "十四酸乙酯", "pentyl.octanoate": "辛酸戊酯",
    "X1.2.dihydro.8.hydroxy..2E..linalool": "二氢羟基芳樟醇",
    "X2.methylbutyl.octanoate": "辛酸2-甲基丁酯",
```

paper/make_figures.py（续）

```
    "X2.phenyl.ethyl.butyrate": "丁酸苯乙酯",
    "X.Z..beta.farnesene": "Z-β-法呢烯",
    "ethyl.pentadecanoate": "十五酸乙酯", "isopentyl.hexanoate": "己酸异戊酯",
}


def spring_layout(adjacency, seed=20261023):
    """Force layout within one connected component."""
    count = len(adjacency)
    pos = np.random.default_rng(seed).normal(size=(count, 2))
    pos /= max(np.linalg.norm(pos, axis=1).max(), 1e-8)
    degree = (adjacency > 0).sum(axis=1)
    repulsion = 1 + 0.12 * (degree[:, None] + degree[None, :])
    scale = np.sqrt(1 / count)
    for step in range(550):
        delta = pos[:, None, :] - pos[None, :, :]
        distance = np.maximum(np.linalg.norm(delta, axis=2), 1e-4)
        forces = ((repulsion * scale**2 / distance**2
                   - adjacency * distance / scale)[:, :, None] * delta).sum(axis=1)
        forces -= 0.06 * pos
        length = np.maximum(np.linalg.norm(forces, axis=1), 1e-8)
        temperature = 0.075 * (1 - step / 550) + 0.001
        pos += forces * (np.minimum(length, temperature) / length)[:, None]
        pos -= pos.mean(axis=0)
    return 0.76 * pos / np.linalg.norm(pos, axis=1).max()


def network_layout(adjacency, seed=20261023):
    """Pack connected-component force layouts without shrinking their edges away."""
    seen, components = set(), []
    for node in range(len(adjacency)):
        if node in seen:
            continue
        pending, component = [node], []
        while pending:
            current = pending.pop()
            if current in seen:
                continue
            seen.add(current)
            component.append(current)
            pending.extend(np.flatnonzero(adjacency[current] > 0).tolist())
        components.append(sorted(component))
    components.sort(key=lambda component: (-len(component), component[0]))
    candidates = np.array(np.meshgrid(np.arange(-0.9, 0.901, 0.02),
                                      np.arange(-0.9, 0.901, 0.02))).reshape(2, -1).T
    order = np.argsort(np.linalg.norm(candidates, axis=1), kind="stable")
    candidates = candidates[order]
    positions = np.empty((len(adjacency), 2))
    circles = []
    radius_by_count = {15: 0.33, 11: 0.235, 6: 0.16, 4: 0.125, 3: 0.105, 2: 0.078}
    for number, component in enumerate(components):
        radius = radius_by_count[len(component)]
        valid = np.linalg.norm(candidates, axis=1) + radius <= 0.90
        for center, old_radius in circles:
            valid &= np.linalg.norm(candidates - center, axis=1) >= radius + old_radiu
s + 0.022
        if not valid.any():
            raise ValueError("No room to display all connected components")
        center = candidates[np.flatnonzero(valid)[0]]
        local = radius * spring_layout(adjacency[np.ix_(component, component)], seed +
 number) / 0.76
```

paper/make_figures.py（续）

```
        # Dense correlated hubs otherwise overlap after component normalization.
        # This collision pass changes display coordinates only.
        for _ in range(260):
            for i in range(len(component)):
                for j in range(i):
                    delta = local[i] - local[j]
                    distance = np.linalg.norm(delta)
                    if distance < 0.115:
                        direction = delta / max(distance, 1e-8)
                        shift = 0.51 * (0.115 - distance) * direction
                        local[i] += shift
                        local[j] -= shift
            norm = np.linalg.norm(local, axis=1)
            local *= np.minimum(1, radius / np.maximum(norm, 1e-8))[:, None]
        positions[component] = center + local
        circles.append((center, radius))
    return positions


def figure3():
    summary = json.loads((OUTPUT / "q2/summary.json").read_text(encoding="utf-8"))
    top3 = read_csv("q2/top3_compounds.csv")
    nodes = read_csv("q2/network_nodes.csv")
    edges = read_csv("q2/network_edges_rho_0.7.csv")
    active = nodes[nodes["degree_0.7"].gt(0)].copy().set_index("field")
    isolated = nodes[nodes["degree_0.7"].eq(0)]
    assert len(nodes) == 231 and len(active) == 81 and len(isolated) == 150
    assert len(edges) == 109 and edges.sign.eq(1).all()
    lookup = {field: i for i, field in enumerate(active.index)}
    adjacency = np.zeros((len(active), len(active)))
    for row in edges.itertuples():
        i, j = lookup[row.source], lookup[row.target]
        adjacency[i, j] = adjacency[j, i] = row.absolute_weight
    np.testing.assert_array_equal((adjacency > 0).sum(axis=1), active["degree_0.7"])
    positions = network_layout(adjacency)
    fig = new_figure(11.8)
    network = fig.add_axes([0.015, 0.28, 0.605, 0.65])
    network.set(xlim=(-1.19, 1.19), ylim=(-1.27, 1.22), aspect="equal")
    network.axis("off")
    network.set_title("(a) 化学关联网络（|ρ|≥0.7）", loc="left", fontsize=9.5, pad=10)
    for row in edges.itertuples():
        pair = positions[[lookup[row.source], lookup[row.target]]]
        network.plot(pair[:, 0], pair[:, 1], color=BLUE, alpha=0.40, linewidth=0.6, zo
rder=1)
    network.scatter(positions[:, 0], positions[:, 1], color=BLUE,
                    s=12 + 5 * active["degree_0.7"].to_numpy(), zorder=2)
    angle = np.linspace(0, 2 * np.pi, len(isolated), endpoint=False)
    network.scatter(1.02 * np.cos(angle), 1.02 * np.sin(angle),
                    s=5.5, color=GREY, zorder=2)
    network.text(0, 1.13, "外围：150 个孤立节点逐点保留", ha="center", fontsize=8.5)
    network.text(0, -1.17, "内部：81 个非孤立节点；109 条边均为正相关", ha="center", font
size=8.5)
    ranked = active.reset_index().sort_values(["degree_0.7", "field"],
                                              ascending=[False, True]).head(8)
    for number, row in enumerate(ranked.itertuples()):
        label = chr(ord("A") + number)
        x, y = positions[lookup[row.field]]
        network.text(x, y, label, ha="center", va="center", fontsize=8,
                     color="white", zorder=3)
        fig.text(0.035 + (number % 4) * 0.245, 0.165 - (number // 4) * 0.047,
                 f"{label}  {NETWORK_SHORT_NAMES[row.field]}", ha="left", fontsize=8.3
```

```
    )

    threshold_ax = fig.add_axes([0.715, 0.66, 0.26, 0.24])
    thresholds = [row["absolute_rho_threshold"] for row in summary["networks"]]
    edge_counts = [row["edges"] for row in summary["networks"]]
    threshold_ax.bar(range(3), edge_counts, color=[GREY, BLUE, GREY], width=0.55)
    threshold_ax.set(xticks=range(3), xticklabels=[f"{v:.1f}" for v in thresholds],
                     ylim=(0, 315), xlabel="|ρ| 下限", ylabel="无向边数")
    threshold_ax.set_title("(b) 阈值敏感性", loc="left", fontsize=9.5)
    for x, value in enumerate(edge_counts):
        threshold_ax.text(x, value + 9, str(value), ha="center", fontsize=8.5)
    grid(threshold_ax, "y")
    stability_ax = fig.add_axes([0.715, 0.295, 0.26, 0.205])
    frequencies = top3.bootstrap_top3_frequency.to_numpy()
    bins = np.array([0, 0.2, 0.4, 0.6, 0.8, 1.00001])
    count, _ = np.histogram(frequencies, bins=bins)
    assert len(frequencies) == 135 and int((frequencies >= 0.8).sum()) == 14
    stability_ax.bar(np.arange(5), count, color=[GREY] * 4 + [BLUE], width=0.75)
    stability_ax.set(xticks=np.arange(5), xticklabels=["0", ".2", ".4", ".6", ".8"],
                     xlabel="频率区间下界（组距 0.2）", ylabel="关联位置数",
                     ylim=(0, max(count) + 12))
    stability_ax.set_title("(c) Top-3 入选稳定性", loc="left", fontsize=9.5)
    for x, value in enumerate(count):
        stability_ax.text(x, value + 1.5, str(value), ha="center", fontsize=8.5)
    grid(stability_ax, "y")
    fig.text(0.5, 0.065, "A–H 为度最高的八个节点；节点面积随度增加；布局位置不代表浓度", ha="center", fontsize=8.5)
    fig.text(0.5, 0.02, "200 次配对重抽样；14/135 个关联位置的入选频率达到 0.8", ha="center", fontsize=8.5)
    save(fig, "figure3_q2")


def figure4():
    summary = read_csv("q3_optimized/generation/cv_summary.csv")
    styles = ["Tripel", "Kriek", "Lambic"]
    methods = ["legacy_copula", "nested_selected", "style_bootstrap"]
    names = ["基准 Copula", "嵌套选择器", "风格重采样"]
    fig = new_figure(8.6)
    ax = fig.add_axes([0.105, 0.21, 0.87, 0.67])
    width = 0.23
    for k, (method, name, color) in enumerate(zip(methods, names, [GREY, BLUE, GOLD])):
        part = summary[summary.method.eq(method)].set_index("style").loc[styles]
        ax.bar(np.arange(3) + (k - 1) * width, part.energy_pca5_mean, width=width,
               color=color, label=name, yerr=part.energy_pca5_std,
               error_kw={"ecolor": DARK, "capsize": 2.5, "elinewidth": 0.7, "capthick": 0.7})
    ax.set(xticks=np.arange(3), xticklabels=["Tripel（n=28）", "Kriek（n=10）", "Lambic（n=11）"],
           ylabel="五维 PCA 能量差异", ylim=(0, 1.4))
    grid(ax, "y")
    fig.legend(loc="upper center", bbox_to_anchor=(0.54, 0.99), ncol=3, frameon=False,
               handlelength=1.5, columnspacing=1.2)
    fig.text(0.5, 0.07, "外层两次三折；每次生成 300 条、两个模拟种子", ha="center", fontsize=8.5)
    fig.text(0.5, 0.025, "误差线为六个折均值的折间标准差，不是置信区间；指标越低越好", ha="center", fontsize=8.5)
    save(fig, "figure4_generation")


PROFILE_TARGETS = [
```

paper/make_figures.py（续）

```
    ("A_malt_all", "麦芽香气"), ("A_hops_all", "酒花香气"),
    ("A_esters_fruity", "水果酯香"), ("F_malt_all", "麦芽风味"),
    ("F_hops_all", "酒花风味"), ("acidity", "酸度"),
    ("bitternes", "苦度"), ("sweetness", "甜度"),
    ("X4vg", "4VG 酚味"), ("stale_hops", "陈旧酒花"),
    ("lactic", "乳酸味"), ("barnyard", "畜棚气味"),
]


def figure5():
    summary = read_csv("q3_optimized/generated_sensory_summary.csv")
    fig = new_figure(12.7)
    axes = fig.subplots(1, 3, sharex=True, sharey=True)
    fig.subplots_adjust(left=0.17, right=0.98, top=0.90, bottom=0.27, wspace=0.15)
    keys, labels = zip(*PROFILE_TARGETS)
    y = np.arange(len(keys))
    for ax, style in zip(axes, ["Tripel", "Kriek", "Lambic"]):
        part = summary[summary["style"].eq(style)].set_index("target").loc[list(keys)]
        assert np.all(part.predicted_p05 <= part.predicted_mean)
        assert np.all(part.predicted_mean <= part.predicted_p95)
        ax.axvline(0, color="#7C858C", linewidth=0.6)
        ax.axhspan(8.55, 9.45, color="#F0F2F3", zorder=-1)
        ax.hlines(y, part.predicted_p05, part.predicted_p95, color=BLUE, alpha=0.6, li
newidth=2.4)
        ax.scatter(part.predicted_mean, y, color=BLUE, s=15, zorder=3)
        ax.set_title(style, pad=8)
        ax.set(xlim=(-1.3, 2.85), xticks=[-1, 0, 1, 2], yticks=y, yticklabels=labels,
               ylim=(11.6, -0.6))
        ax.tick_params(axis="y", length=0)
        grid(ax)
    fig.text(0.58, 0.19, "标准化感官值", ha="center", fontsize=9)
    fig.text(0.5, 0.12, "圆点：生成谱预测均值；线段：5%–95% 分位范围（每风格 300 条）", ha
="center", fontsize=8.5)
    fig.text(0.5, 0.075, "该范围描述生成分布，不是预测置信区间，也不是原始 0–6 分", ha="ce
nter", fontsize=8.5)
    fig.text(0.5, 0.025, "灰行陈旧酒花使用非 Pils 单目标代理；其余维度使用全量多输出代理",
ha="center", fontsize=8.5)
    save(fig, "figure5_profiles")


def figure6():
    summary = read_csv("q3_refined/transfer/transfer_summary.csv")
    part = summary[summary.arm.eq("fixed") & summary.pool.eq("contracted_6004")
                   & summary.bound.eq("retain80")]
    source_order = [124, 81, 218, 96, 230, 185, 139, 209, 119, 90, 28]
    margin = part[part.ranking.eq("margin")].set_index("source_beer").loc[source_order
]
    geometry = part[part.ranking.eq("geometry")].set_index("source_beer").loc[source_o
rder]
    assert len(margin) == len(geometry) == 11
    assert margin.fresh_consistency_passed.sum() == geometry.fresh_consistency_passed.
sum() == 10
    assert margin.loc[~margin.fresh_consistency_passed].index.tolist() == [96]
    assert geometry.loc[~geometry.fresh_consistency_passed].index.tolist() == [96]
    fig = new_figure(10.3)
    ax = fig.add_axes([0.16, 0.28, 0.80, 0.60])
    y = np.arange(11)
    m = margin.target_improvement.to_numpy() * 100
    g = geometry.target_improvement.to_numpy() * 100
    ax.hlines(y, m, g, color="#A7B1B8", linewidth=1)
    ax.scatter(m, y, color=BLUE, marker="o", s=25, label="余量优先", zorder=3)
```

paper/make_figures.py（续）

```
    ax.scatter(g, y, edgecolor=GOLD, facecolor="white", marker="^", linewidth=1.1,
               s=35, label="几何优先", zorder=3)
    failed_y = source_order.index(96)
    ax.scatter([m[failed_y]], [failed_y], color=FAIL, marker="x", s=55, linewidth=1.4,
               label="审计未通过", zorder=4)
    ax.set(yticks=y, yticklabels=[str(beer) + (" *" if beer == 96 else "") for beer in source_order],
           ylim=(10.7, -0.7), xlim=(0, 85), xticks=[0, 20, 40, 60, 80],
           xlabel="向 Tripel 的目标距离改善（%）", ylabel="源 beer 编号")
    grid(ax)
    ax.legend(loc="upper center", bbox_to_anchor=(0.5, 1.17), ncol=3, frameon=False,
              handletextpad=0.4, columnspacing=1.0)
    fig.text(0.5, 0.12, "固定树 / 中央 6004 候选 / retain80；两种排序均为 10/11 通过", ha="center", fontsize=8.5)
    fig.text(0.5, 0.07, "所选谱改善中位数：余量优先 57.74%，几何优先 70.08%", ha="center", fontsize=8.5)
    fig.text(0.5, 0.02, "* beer 96 的审计失败保留；通过数为模型检查结果，不是实际品评成功数", ha="center", fontsize=8.5)
    save(fig, "figure6_transfer")


def main(only=None):
    FIGURES.mkdir(parents=True, exist_ok=True)
    for number, draw in enumerate([figure1, figure2, figure3, figure4, figure5, figure6], 1):
        if only is None or number == only:
            draw()
    descriptions = [
        ("figure1_workflow", "图1 三问的数据处理、模型与验证流程。第一问使用231维原表示；第三问使用69维校准量与157维相对量组成的226维混合表示。迁移训练排除全部11款Pils。", 11.2,
         ["outputs/q1/summary.json", "outputs/q2/summary.json", "outputs/q3_optimized/generation/summary.json", "outputs/q3_refined/stale/summary.json", "src/q3_data.py"]),
        ("figure2_q1", "图2 第一问四种方法的五折折外误差、宏平均R²及ExtraTrees缺失压力结果。裁剪Ridge为稳健性对照；额外隐藏20%已观测特征后，ExtraTrees的RMSE由0.4862增至0.4992。", 11.2,
         ["outputs/q1/metrics_overall.csv", "outputs/q1/missingness_stress.csv", "outputs/q1_robust/metrics_overall.csv"]),
        ("figure3_q2", "图3 化学关联网络及稳定性。(a) 实际|Spearman ρ|≥0.7网络，保留全部231节点和109条正相关边；81个非孤立节点采用固定种子20261023的分量内力导向布局及分量打包，并进行显示避碰，150个孤立节点在外围环逐点保留，节点面积随度增加。A–H为度最高八个节点，图内列简名；布局距离不表示化学浓度或因果关系。(b) 三个阈值下的边数。(c) 135个Top-3位置的200次配对重抽样入选频率，其中14个达到0.8；频率区间左闭右开，最后一组包含1。", 11.8,
         ["outputs/q2/network_nodes.csv", "outputs/q2/network_edges_rho_0.7.csv", "outputs/q2/summary.json", "outputs/q2/top3_compounds.csv"]),
        ("figure4_generation", "图4 三种目标风格的生成能量差异，比较基准Copula、完整嵌套选择器及重采样对照。柱为外层六个折均值的平均值，误差线为其标准差；每折先平均两个模拟种子。折间标准差不是置信区间，真实样本数列于横轴。", 8.6,
         ["outputs/q3_optimized/generation/cv_summary.csv", "outputs/q3_optimized/generation/cv_fold_means.csv", "outputs/q2/summary.json"]),
        ("figure5_profiles", "图5 三种风格在列示的12个代表感官维度上的生成画像。圆点为每风格300条谱的预测均值，线段为5%–95%生成分布范围，三个面板使用共同横轴。陈旧酒花来自239款非Pils训练的固定单目标树，其余维度来自226维全量多输出代理；区间不是校准预测置信区间，数值不是原始0–6分。", 12.7,
         ["outputs/q3_optimized/generated_sensory_summary.csv", "outputs/q3_optimized/generation/summary.json", "outputs/q3_optimized/stale_fixed/summary.json"]),
        ("figure6_transfer", "图6 固定树、中央6004候选池与retain80条件下，11款Pils源的所选谱目标距离改善。余量优先和几何优先均通过10/11款，beer 96的审计失败保留；改善中位数分别为57.74%与70.08%，包括已选但失败的谱。几何优先为探索性应用示例，模型检查通过不代表实际品评成功。", 10.3,
         ["outputs/q3_refined/transfer/transfer_summary.csv", "outputs/q3_refined/transfer/frontier.csv"]),
```

paper/make_figures.py（续）

```
    ]
    manifest = {
        "renderer": "matplotlib", "font_file": str(CJK_FONT), "dpi": 300,
        "widthcm": WIDTH_CM, "source_root": "solution_a", "sources_mutated": False,
        "figures": [{"id": index, "caption": caption,
                     "relativepath": f"figures/{name}.png",
                     "svg_relativepath": f"figures/{name}.svg",
                     "widthcm": WIDTH_CM, "heightcm": height,
                     "sources": sources}
                    for index, (name, caption, height, sources) in enumerate(descripti
ons, 1)],
        "profile_targets": [{"target": key, "label": label} for key, label in PROFILE_
TARGETS],
        "verification": {
            "figure3_nodes": 231, "figure3_edges": 109,
            "figure3_active_nodes": 81, "figure3_isolated_nodes": 150,
            "figure3_all_edges_positive": True, "figure3_layout_seed": 20261023,
            "figure3_top3_positions": 135, "figure3_frequency_at_least_0_8": 14,
            "figure6_sources": 11, "figure6_passed_each_ranking": 10,
            "figure6_failed_source_each_ranking": 96,
        },
    }
    for figure in manifest["figures"]:
        assert all((SOLUTION / source).is_file() for source in figure["sources"])
    (PAPER / "figure_manifest.json").write_text(
        json.dumps(manifest, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
    print(f"Saved {'six figure pairs' if only is None else 'figure ' + str(only)} and
figure_manifest.json in {PAPER}")


if __name__ == "__main__":
    import argparse

    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--only", type=int, choices=range(1, 7), help="Render one figu
re only")
    main(parser.parse_args().only)
```

paper/native_math.py

SHA-256: 03fa40fc1d8212dba0473ef822c83aecb962d75147777ce020585d550df879b9

```
"""Small, deliberately bounded TeX-to-OMML path for this manuscript."""
import re
from lxml import etree

M = "http://schemas.openxmlformats.org/officeDocument/2006/math"
W = "http://schemas.openxmlformats.org/wordprocessingml/2006/main"


def el(tag, *children, **attrs):
    node = etree.Element(f"{{{M}}}{tag}")
    for key, value in attrs.items():
        node.set(f"{{{M}}}{key}", str(value))
    node.extend(children)
    return node


def run(text, plain=False):
    node = el("r")
    if plain:
        node.append(el("rPr", el("sty", val="p")))
    pr = etree.SubElement(node, f"{{{W}}}rPr")
    fonts = etree.SubElement(pr, f"{{{W}}}rFonts")
    for key in ["ascii", "hAnsi", "eastAsia"]:
        fonts.set(f"{{{W}}}{key}", "Cambria Math")
    size = etree.SubElement(pr, f"{{{W}}}sz")
    size.set(f"{{{W}}}val", "22")
    t = etree.SubElement(node, f"{{{M}}}t")
    t.set("{http://www.w3.org/XML/1998/namespace}space", "preserve")
    t.text = text
    return node


SYMBOLS = dict(alpha="α", beta="β", gamma="γ", delta="δ", epsilon="ε", varepsilon="ε",
               lambda_="λ", mu="μ", pi="π", rho="ρ", sigma="σ", tau="τ", kappa="κ",
               Phi="Φ", Sigma="Σ", Delta="Δ", Omega="Ω", ell="ℓ", in_="∈",
               notin="∉", neq="≠", ne="≠", leq="≤", le="≤", geq="≥", ge="≥",
               times="×", cdot="·", cap="∩", cup="∪", forall="∀", prime="′",
               ldots="…", dots="…", lVert="‖", rVert="‖", vert="|", Vert="‖",
               to="→", rightarrow="→", approx="≈", sim="∼", infinity="∞", infty="∞",
               sum="∑", prod="∏", partial="∂", pm="±", mid="|", mathbfone="1")
SYMBOLS["lambda"] = SYMBOLS.pop("lambda_")
SYMBOLS["in"] = SYMBOLS.pop("in_")


class Parser:
    def __init__(self, source):
        source = re.sub(r"\\{2,}", r"\\", source)
        self.tokens = re.findall(r"\\[A-Za-z]+|\\.|.", source)
        self.i = 0
        self.unknown = []

    def take(self):
        value = self.tokens[self.i]
        self.i += 1
        return value

    def argument(self):
        while self.i < len(self.tokens) and self.tokens[self.i].isspace():
            self.i += 1
```

paper/native_math.py（续）

```
        if self.i >= len(self.tokens):
            return []
        if self.tokens[self.i] == "{":
            self.i += 1
            return self.sequence("}")
        return self.atom()

    def atom(self):
        tok = self.take()
        if tok == "{":
            return self.sequence("}")
        if not tok.startswith("\\"):
            return [run(tok)]
        cmd = tok[1:]
        if cmd in ["left", "right", "big", "Big", "bigg", "Bigg"]:
            return []
        if cmd in [",", ";", "quad", "qquad", " ", "!"]:
            return [run(" " if cmd != "!" else "")]
        if cmd in ["{", "}", "_", "%", "|", "[", "]"]:
            return [run(cmd)]
        if cmd in ["frac", "dfrac", "tfrac"]:
            return [el("f", el("num", *self.argument()), el("den", *self.argument()))]
        if cmd == "sqrt":
            return [el("rad", el("radPr", el("degHide", val="1")), el("deg"), el("e", *self.argument()))]
        if cmd in ["hat", "widehat", "tilde", "widetilde", "bar", "overline"]:
            chars = {"hat": "^", "widehat": "^", "tilde": "~", "widetilde": "~", "bar": "¯", "overline": "¯"}
            return [el("acc", el("accPr", el("chr", val=chars[cmd])), el("e", *self.argument()))]
        if cmd in ["mathrm", "operatorname", "text", "mathsf", "mathbf", "boldsymbol", "mathcal", "mathbb"]:
            nodes = self.argument()
            for node in nodes:
                for r in [node] if node.tag == f"{{{M}}}r" else node.iter(f"{{{M}}}r"):
                    pr = r.find(f"{{{M}}}rPr")
                    if pr is None:
                        pr = el("rPr")
                        r.insert(0, pr)
                    pr.append(el("sty", val="b" if cmd in ["mathbf", "boldsymbol"] else "p"))
                    if cmd == "mathcal":
                        pr.append(el("scr", val="script"))
            return [el("box", el("e", *nodes))]
        if cmd in ["max", "min", "arg", "log", "exp", "sin", "cos", "rank", "corr", "argmin"]:
            return [run(cmd, plain=True)]
        if cmd in SYMBOLS:
            return [run(SYMBOLS[cmd])]
        self.unknown.append(cmd)
        return [run(cmd, plain=True)]

    def sequence(self, until=None):
        nodes = []
        while self.i < len(self.tokens):
            tok = self.tokens[self.i]
            if tok == until:
                self.i += 1
                break
            if tok in ["_", "^"]:
```

paper/native_math.py（续）

```
                self.i += 1
                sub = self.argument() if tok == "_" else None
                sup = self.argument() if tok == "^" else None
                if self.i < len(self.tokens) and self.tokens[self.i] in ["_", "^"]:
                    other = self.take()
                    if other == "_":
                        sub = self.argument()
                    else:
                        sup = self.argument()
                base = nodes.pop() if nodes else run("")
                text = "".join(base.itertext())
                if text in ["∑", "∏"]:
                    props = el("naryPr", el("chr", val=text), el("limLoc", val="undOvr
"))
                    if sub is None:
                        props.append(el("subHide", val="1"))
                    if sup is None:
                        props.append(el("supHide", val="1"))
                    nodes.append(el("nary", props, el("sub", *(sub or [])), el("sup",
*(sup or [])), el("e")))
                elif sub is not None and sup is not None:
                    nodes.append(el("sSubSup", el("e", base), el("sub", *sub), el("sup
", *sup)))
                elif sub is not None:
                    nodes.append(el("sSub", el("e", base), el("sub", *sub)))
                else:
                    nodes.append(el("sSup", el("e", base), el("sup", *(sup or []))))
            else:
                nodes.extend(self.atom())
        return nodes


def to_omml(source):
    parser = Parser(source)
    result = el("oMath", *parser.sequence())
    # An OMML n-ary operator must own a non-empty expression; otherwise Word
    # displays an editing placeholder square. Attach its following atom.
    for parent in list(result.iter())[::-1]:
        for child in list(parent)[::-1]:
            if child.tag != f"{{{M}}}nary":
                continue
            expression=child.find(f"{{{M}}}e")
            if len(expression):
                continue
            following=child.getnext()
            while following is not None and not "".join(following.itertext()).strip():
                next_node=following.getnext()
                parent.remove(following)
                following=next_node
            if following is not None:
                parent.remove(following)
                expression.append(following)
            else:
                expression.append(run("1"))
    if parser.unknown:
        raise ValueError(f"Unsupported mathematical command(s): {parser.unknown}")
    return result

```

paper/package_support.py

SHA-256: d815cec1cd6d2890c3434716c1f9e4b753f82033147262dde2776ffcedec5630

```
"""Package complete source programs and saved numerical evidence."""
import hashlib
import json
from pathlib import Path
from zipfile import ZIP_DEFLATED, ZipFile
from submission_content import source_files

HERE = Path(__file__).resolve().parent
ROOT = HERE.parent
OUT = ROOT / 'submission'
OUT.mkdir(exist_ok=True)
entries = {}
for path in source_files(ROOT):
    entries['solution_a/' + path.relative_to(ROOT).as_posix()] = path
for name in ['requirements-lock.txt', 'final_model_selection.json', 'README.md']:
    entries['solution_a/' + name] = ROOT / name
for path in (ROOT / 'docs').glob('*.md'):
    entries['solution_a/docs/' + path.name] = path
for pattern in ['*.json', '*.csv', '*.txt', 'sections/*.json', 'figures/*.png', 'figures/*.svg']:
    for path in HERE.glob(pattern):
        entries['solution_a/paper/' + path.relative_to(HERE).as_posix()] = path
for path in sorted((ROOT / 'outputs').rglob('*')):
    if path.is_file() and path.suffix in ['.csv', '.json']:
        entries['paper_results/' + path.relative_to(ROOT / 'outputs').as_posix()] = path
for path in sorted((ROOT / 'supplement/results').rglob('*')):
    if path.is_file() and path.suffix in ['.csv', '.json', '.npz']:
        entries['supplement_results/' + path.relative_to(ROOT / 'supplement/results').as_posix()] = path
entries['solution_a/supplement/README.txt'] = ROOT / 'supplement/README.txt'
entries['verification/supplement.json'] = ROOT / 'supplement/verification.json'
attributes = ROOT.parent / '.gitattributes'
if attributes.exists():
    entries['.gitattributes'] = attributes
for path in (ROOT / 'review/verification').rglob('*.json'):
    entries['verification/' + path.relative_to(ROOT / 'review/verification').as_posix()] = path
for name in ['source_inventory.json', 'paper_verification.json', 'visual_review.json']:
    path = HERE / 'qa' / name
    if path.exists():
        entries['verification/' + name] = path

guide = '''A题论文配套程序与数值结果

1. PDF附录E已印出全部源程序；本包保留可直接运行的文件及缩进。
2. solution_a/src/为主分析程序，run_all.py为统一入口，supplement/为补充分析，paper/为论文结构化源稿、表图和构建检查程序。
3. paper_results/及supplement_results/保存论文引用结果，不是运行输出目录。本包不含原始赛题附件、模型权重及大型候选缓存。
4. 原始赛题目录与solution_a并列，保持官方目录名。计算使用Python 3.13及requirements-lock.txt。
   Windows PowerShell在解压根目录运行：
   py -3.13 -m venv .venv
   .\\.venv\\Scripts\\python.exe -m pip install -r solution_a/requirements-lock.txt
   .\\.venv\\Scripts\\python.exe -X utf8 solution_a/run_all.py --rebuild
   全部模型、候选缓存、outputs与supplement/results由源码重建。--rebuild拒绝覆盖非空输出目录。
```

paper/package_support.py（续）

```
5. 对重建完成的完整目录，去掉--rebuild即可只读验证。报告写入solution_a/review/verification/。
6. 重建论文需原赛题Word模板、Windows中文字体和Word兼容COM接口（本次使用WPS）：
   python -m pip install -r solution_a/paper/requirements-paper.txt
   python solution_a/paper/make_figures.py
   python solution_a/paper/build_paper.py
   powershell -ExecutionPolicy Bypass -File solution_a/paper/export_pdf.ps1
   python solution_a/paper/source_appendix.py
   python solution_a/paper/package_support.py
   paper脚本读取outputs；欲保留论文原始数值，可在单独的论文构建副本中将paper_results复制为solution_a/outputs，
   将supplement_results复制为solution_a/supplement/results，
   仅运行上述paper构建脚本，不对该不完整副本运行run_all.py。
   verify_paper.py须在完整重建目录运行，并先用Poppler的pdftoppm命令把qa/main.pdf渲染为qa/render/page-*.png。
7. MANIFEST.json记录包内文件SHA-256。verification内为原始冻结结果的只读验证和独立重建检查记录。
8. 原评分量尺不能从现有附件唯一恢复，文中标准化预测不冒充0–6评分；化学相关不等于感官因果，迁移结果尚未经实际品评。
'''
manifest = {'scope': 'Complete printed source, paper evidence and saved numerical results; original attachments excluded.',
            'program_files': len(source_files(ROOT)),
            'files': [{'path': name, 'bytes': p.stat().st_size,
                       'sha256': hashlib.sha256(p.read_bytes()).hexdigest()} for name, p in sorted(entries.items())]}
out = OUT / 'A2600730附件.zip'
with ZipFile(out, 'w', compression=ZIP_DEFLATED, compresslevel=7) as archive:
    archive.writestr('README.txt', guide)
    archive.writestr('MANIFEST.json', json.dumps(manifest, ensure_ascii=False, indent=2) + '\n')
    for name, path in sorted(entries.items()):
        archive.write(path, name)
with ZipFile(out) as archive:
    assert archive.testzip() is None
    for folder in ['q1', 'q1_robust', 'q2', 'q3_optimized', 'q3_refined']:
        assert any(name.startswith('paper_results/' + folder + '/') for name in archive.namelist()), folder
    for row in manifest['files']:
        assert hashlib.sha256(archive.read(row['path'])).hexdigest() == row['sha256']
    inventory = json.loads((HERE / 'qa/source_inventory.json').read_text(encoding='utf-8'))
    for row in inventory['files']:
        assert hashlib.sha256(archive.read('solution_a/' + row['path'])).hexdigest() == row['sha256'], row['path']
(HERE / 'qa/support_manifest.json').write_text(json.dumps(manifest, ensure_ascii=False, indent=2) + '\n', encoding='utf-8')
print(f'{out.name}: {len(entries) + 2} files, {manifest["program_files"]} programs, {out.stat().st_size / 2**20:.2f} MiB')
```

paper/source_appendix.py

SHA-256: c03d5ebd4bbf07609a8f9a77ef73999cbcb591100b9209165fd23b26155d93ca

```
"""Print every source line and combine it with the main paper PDF."""
import hashlib
import json
from pathlib import Path
import re

from pypdf import PdfReader, PdfWriter
from pypdf.annotations import Link
from pypdf.generic import NameObject
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.pdfgen import canvas
from reportlab.lib.pagesizes import A4
from reportlab.lib.units import mm

from submission_content import source_files

HERE = Path(__file__).resolve().parent
ROOT = HERE.parent
OUT = ROOT / "submission"
QA = HERE / "qa"
OUT.mkdir(exist_ok=True)
QA.mkdir(exist_ok=True)


def build():
    main = PdfReader(QA / "main.pdf")
    pdfmetrics.registerFont(TTFont("CodeMono", "C:/Windows/Fonts/consola.ttf"))
    pdfmetrics.registerFont(TTFont("CodeCJK", "C:/Windows/Fonts/simsun.ttc", subfontIn
dex=0))
    pdfmetrics.registerFont(TTFont("CodeMath", "C:/Windows/Fonts/cambria.ttc", subfont
Index=1))
    width, height = A4
    left, right, top, bottom = 25 * mm, width - 25 * mm, height - 20 * mm, 22 * mm
    code_x = left + 27
    font_size, leading = 9, 11.5
    path = QA / "source.pdf"
    c = canvas.Canvas(str(path), pagesize=A4, pageCompression=1)
    c.setTitle("完整计算机源程序")
    c.setAuthor("")
    page = 0
    y = top
    records = []
    geometry = []

    def font(char):
        for face in ("CodeMono", "CodeCJK", "CodeMath"):
            if ord(char) in pdfmetrics.getFont(face).face.charToGlyph:
                return face
        raise ValueError(f"No embedded font covers U+{ord(char):04X}")

    def draw(text, x, baseline, size=font_size):
        runs = []
        for char in text:
            if runs and runs[-1][0] == font(char):
                runs[-1][1] += char
            else:
                runs.append([font(char), char])
        for face, run in runs:
```

paper/source_appendix.py（续）

```
            c.setFont(face, size)
            c.drawString(x, baseline, run)
            x += pdfmetrics.stringWidth(run, face, size)
        assert x <= right + 0.1, (page, text, x, right)
        geometry.append({"page": page, "x1": round(x, 2), "y": round(baseline, 2)})

    def new_page(name, continuation=False):
        nonlocal page, y
        if page:
            c.showPage()
        page += 1
        c.setFont("CodeMono", 10)
        c.drawCentredString(width / 2, 17.5 * mm, str(len(main.pages) - 2 + page))
        y = top - 10
        draw(name + ("（续）" if continuation else ""), left, y, 10)
        y -= 22

    for p in source_files(ROOT):
        relative = p.relative_to(ROOT).as_posix()
        content = p.read_text(encoding="utf-8-sig")
        lines = content.splitlines()
        new_page(relative)
        start = page
        sha = hashlib.sha256(p.read_bytes()).hexdigest()
        draw("SHA-256: " + sha, left, y, 8)
        y -= 18
        for number, line in enumerate(lines, 1):
            line = line.expandtabs(4)
            chunks, current, occupied = [], "", 0.0
            for char in line:
                advance = pdfmetrics.stringWidth(char, font(char), font_size)
                if occupied + advance > right - code_x:
                    chunks.append(current)
                    current, occupied = "", 0.0
                current += char
                occupied += advance
            chunks.append(current)
            for j, chunk in enumerate(chunks):
                if y < bottom:
                    new_page(relative, True)
                if j == 0:
                    c.setFont("CodeMono", 7)
                    c.setFillGray(.45)
                    c.drawRightString(code_x - 7, y, str(number))
                    c.setFillGray(0)
                draw(chunk, code_x, y)
                y -= leading
        records.append({"path": relative, "lines": len(lines), "sha256": sha,
                        "first_pdf_page": len(main.pages) + start,
                        "last_pdf_page": len(main.pages) + page})
    c.save()
    writer = PdfWriter(clone_from=QA / "main.pdf")
    # Rebuild internal contents links: COM exporters may misread PAGEREF switches.
    contents = writer.pages[1]
    if "/Annots" in contents:
        del contents[NameObject("/Annots")]
    blocks = json.loads((HERE / "manuscript.json").read_text(encoding="utf-8"))["block
s"]
    headings = [b for b in blocks if b["type"] == "heading" and
                (b["level"] == 1 or (b["level"] == 2 and re.match(r"[567]\.\d+ ", b["t
ext"])))]
```

paper/source_appendix.py（续）

```
    normalized = [re.sub(r"\s+", "", p.extract_text()) for p in main.pages]
    toc_links = []
    for i, b in enumerate(headings):
        text = b["text"]
        target = next(j for j in range(2, len(normalized)) if re.sub(r"\s+", "", text) in normalized[j])
        y0 = 734.55 - i * 13
        annotation = writer.add_annotation(1, Link(rect=(70.9, y0, 524.5, y0 + 13), target_page_index=target))
        annotation["/Dest"][0] = writer.pages[target].indirect_reference
        writer.add_outline_item(text, target)
        toc_links.append({"text": text, "target_pdf_page": target + 1})
    writer.append(path)
    for r in records:
        writer.add_outline_item(r["path"], r["first_pdf_page"] - 1)
    writer.add_metadata({"/Title": "啤酒风味预测、关键化合物关联与风格迁移", "/Author": ""})
    final = OUT / "A2600730.pdf"
    with final.open("wb") as f:
        writer.write(f)
    manifest = {"main_pages": len(main.pages), "source_pages": page,
                "total_pages": len(main.pages) + page, "files": records,
                "source_lines": sum(r["lines"] for r in records), "contents_links": toc_links,
                "layout": {"font_pt": font_size, "leading_pt": leading,
                           "all_lines_within_margins": True}}
    (QA / "source_inventory.json").write_text(json.dumps(manifest, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
    print(json.dumps({k: v for k, v in manifest.items() if k not in ("files", "contents_links")}, ensure_ascii=False))
    print(final)


if __name__ == "__main__":
    build()
```

paper/submission_content.py

SHA-256: 13763a78fd4f910da449e4c25a0facaf6ab6f6dbf529f6a0846578437a848fe0

```
"""Tables and reproduction instructions included in the submitted paper."""
from pathlib import Path
import re

import pandas as pd


def source_files(root):
    """Use one inventory for the printed appendix and the source archive."""
    paths = list(root.glob("*.py"))
    paths += list((root / "src").glob("*.py"))
    paths += list((root / "paper").glob("*.py"))
    paths += list((root / "paper").glob("*.ps1"))
    paths += list((root / "tests").glob("*.py"))
    paths += list((root / "supplement").glob("*.py"))
    return sorted(set(paths), key=lambda p: p.relative_to(root).as_posix())


def add_submission_appendices(blocks, root, paper, mapping):
    def paragraph(text):
        return {"type": "paragraph", "text": text}

    def heading(text, level=1):
        return {"type": "heading", "level": level, "text": text}

    def table(caption, headers, rows, widths):
        return {"type": "table", "caption": caption, "headers": headers,
                "rows": rows, "widths": widths, "compact": True}

    evidence = pd.read_csv(paper / "chemical_evidence_final.csv").fillna("")
    labels = {"direct": "直接气味依据", "general": "一般背景",
              "indirect": "间接关联", "unverified": "尚未证实"}
    rows = []
    for r in evidence.to_dict("records"):
        chemical = str(r.get("chemical_name_display") or r["chemical_name"])
        source = str(r.get("origin_display") or r["origin_paper"])
        unit = str(r.get("unit_display") or r["unit"] or "相对量")
        threshold = str(r.get("threshold_display") or "附件未给匹配阈值")
        claim = str(r.get("evidence_claim") or r.get("evidence_support_note", ""))
        boundary = str(r.get("evidence_boundary") or "相关系数不构成因果证据。")
        refs = str(r.get("source_ref", ""))
        rows.append([f"{r['sensory_target']}\n第{int(r['rank'])}项",
                     f"{chemical}\n{r['compound']}",
                     f"{source}；{unit}\n{threshold}",
                     f"{labels[r['evidence_category']]} {refs}\n{claim}\n{boundary}"])
    extra = [paragraph("表B-2给出每个候选的化学名称、附件来源、量纲及文献支持范围。"
                       "阈值只用于对应的校准浓度；空缺阈值不补造。一般背景与间接关联均不视为目标感官机制已经证实。"),
             table("表B-2 完整候选的化学信息及证据", ["感官与名次", "化合物", "来源与量纲", "依据及适用范围"],
                   rows, [2.5, 3.8, 3.5, 6.2])]
    idx = next(i for i, b in enumerate(blocks) if b.get("text", "").startswith("附录C"))
    blocks[idx:idx] = extra
    from supplement_sections import permutation_appendix
    idx = next(i for i, b in enumerate(blocks) if b.get("text", "").startswith("附录C"))
    blocks[idx:idx] = permutation_appendix(root)

```

paper/submission_content.py（续）

```
    frontier = pd.read_csv(root / "outputs/q3_refined/transfer/frontier.csv")
    pool = {"legacy_2004": "原2004", "legacy_6004": "原6004", "contracted_6004": "收缩6004"}
    bound = {"retain80": "保留80%", "retain100": "保留100%", "absolute10": "≥10%",
             "absolute30": "≥30%", "absolute50": "≥50%", "absolute60": "≥60%"}
    gridrows = []
    for r in frontier.itertuples():
        value = "—" if pd.isna(r.median_improvement_selected) else f"{r.median_improvement_selected:.2%}"
        gridrows.append(["固定树" if r.arm == "fixed" else "岭回归", pool.get(r.pool, r.pool),
                         bound.get(r.bound, r.bound), "余量" if r.ranking == "margin" else "几何",
                         f"{r.selected}/11", f"{r.fresh_consistency_passed}/11", value])
    blocks += [table("表C-2 全部72组迁移策略结果", ["模型", "候选集", "接近下限", "排序", "选出", "通过", "改善中位数"],
                     gridrows, [1.6, 2.2, 2.1, 1.4, 1.5, 1.5, 3.7]),
               paragraph("改善中位数按全部选中谱计算，包括复核未通过者；无候选时记为横线。"
                         "每组分母均为11，不以选出的条数代替源样本总数。逐化合物十情景检查及全部数值文件随支撑材料提交。")]
    blocks += [heading("附录D 软件环境与运行方法"),
               paragraph("模型原计算环境为Python 3.13，精确依赖列于表D-1。程序采用UTF-8文本、"
                         "正斜杠相对路径和固定换行规则，支持Windows。原始赛题附件与solution_a文件夹并列，"
                         "保持官方目录名；数据不重复打包。所有计算源码及论文构建程序全文列于附录E。"),
               table("表D-1 计算依赖版本", ["软件包", "版本"],
                     [line.split("==", 1) for line in (root / "requirements-lock.txt").read_text(encoding="utf-8").splitlines()
                      if "==" in line], [8, 8]),
               paragraph("在Windows PowerShell中进入解压后的根目录，建立独立环境并安装依赖："),
               {"type": "code", "text": "py -3.13 -m venv .venv\n.\\.venv\\Scripts\\python.exe -m pip install -r solution_a/requirements-lock.txt"},
               paragraph("对包含完整outputs的仓库，默认只核查现有结果，不重新训练或改写冻结记录："),
               {"type": "code", "text": ".\\.venv\\Scripts\\python.exe -X utf8 solution_a/run_all.py"},
               paragraph("支撑材料中的paper_results和supplement_results为引用结果副本，不作为训练输出目录。首次完整重建在新解压目录执行："),
               {"type": "code", "text": ".\\.venv\\Scripts\\python.exe -X utf8 solution_a/run_all.py --rebuild"},
               paragraph("重建须保留全部原始附件，依次执行数据核验、预测、关联分析、风格生成、迁移及补充验证。"
                         "程序对非空历史或补充输出拒绝覆盖；重新试验应另建副本。补充验证源码与结果位于supplement目录，run_supplement.py可单独核查。计算耗时随硬件变化。"
                         "不同操作系统、数学库及并行实现可能出现小幅浮点差，须结合模型参数及逐项指标判断。"),
               paragraph("论文构建另需python-docx、lxml、reportlab和matplotlib。Windows中由Word兼容COM接口导出正文PDF，"
                         "再拼接文字型源码附录。完整依赖、命令及文件校验方法见支撑材料README.txt。"),
               heading("附录E 完整计算机源程序"),
               paragraph("本附录按文件给出计算、校验及论文构建所用源程序。源代码以可选取的文本排印，"
                         "左侧为原文件行号；长行折行处不另编号。运行时使用支撑包中的同名文件，"
                         "避免PDF阅读器复制时丢失缩进。文件清单、源码全文和支撑包采用同一份程序目录。")]
```

paper/submission_content.py（续）

```
    files = source_files(root)
    blocks.append(table("表E-1 源程序目录", ["文件（相对solution_a）", "行数"],
                        [[p.relative_to(root).as_posix(), str(len(p.read_text(encoding="utf-8-sig").splitlines()))]
                         for p in files], [13.5, 2.5]))
    return blocks
```

paper/supplement_sections.py

SHA-256: 921d0df962703e9f8152abb9f713e998701ade9bdb7fbd24c810db82281e27b1

```
"""Insert supplementary results without changing the primary result tables."""
import json
import pandas as pd


def paragraph(text):
    return {"type": "paragraph", "text": text}


def table(caption, headers, rows, widths):
    return {"type": "table", "caption": caption, "headers": headers,
            "rows": rows, "widths": widths, "compact": True}


def add_main_sections(q1, q2, q3, root):
    results = root / "supplement/results"
    metric = pd.read_csv(results / "q1/metrics_overall.csv")
    grouped = pd.read_csv(results / "q1/balanced_group_metrics_overall.csv")
    names = {"mean": "训练折均值", "extra_trees": "ExtraTrees",
             "gbr_per_target": "逐目标GBR", "hgb_per_target": "逐目标HGB"}
    rows = []
    for r in metric.loc[metric.protocol.eq("random_3fold")].itertuples():
        rows.append(["随机三折", names[r.model], f"{r.rmse:.4f}", f"{r.macro_r2:.4f}"]
)
    for r in grouped.itertuples():
        rows.append(["整风格三折", names[r.model], f"{r.rmse:.4f}", f"{r.macro_r2:.4f}"
])
    random_rmse = metric.loc[metric.protocol.eq("random_3fold") & metric.model.eq("ext
ra_trees"), "rmse"].iloc[0]
    grouped_rmse = grouped.loc[grouped.model.eq("extra_trees"), "rmse"].iloc[0]
    pressure = metric.loc[metric.protocol.eq("random_3fold_test_mask")]
    stress_text = []
    for rate in [.1, .2]:
        values = pressure.loc[pressure.rate.eq(rate), "rmse"]
        stress_text.append(f"{rate:.0%}掩码的RMSE均值为{values.mean():.4f}、掩码间标准差
为{values.std(ddof=1):.4f}")
    q1 += [{"type": "heading", "level": 2, "text": "5.6 补充对照与未见风格验证"},
           paragraph("主结果完成后增加固定参数三折对照，用于检查模型家族及部署范围；这是同一
批250款酒上的补充分析，"
                     "不作为独立外部测试，也不替换表5-1的五折指标。四种模型使用相同随机划分
，所有补值均只在训练折估计。"
                     "ExtraTrees沿用300树、叶节点至少3样本及0.7特征比例；GBR逐目标拟合100
棵深度2的树，学习率0.05、"
                     "叶节点至少5样本；HGB逐目标迭代100次，学习率0.05、最多7叶、叶节点至少1
0样本、正则系数1。未据此调参或逐目标拼接赢家。"),
           table("表5-3 固定参数补充对照与整风格留出", ["验证", "模型", "RMSE", "宏平均R²"
], rows, [4, 6, 3, 3]),
           paragraph(f"整风格划分由GroupKFold按类别样本数平衡，三个测试折为83、83、84款，
训练与测试风格互斥。"
                     f"与训练样本量接近的随机三折相比，ExtraTrees的RMSE增加{grouped_rmse/
random_rmse-1:.2%}。"
                     "这反映未见风格预测更困难，不能将随机划分下的精度直接推广至新风格。固定
参数提升树没有整体优势，故保留ExtraTrees主方案。"),
           paragraph("对同一随机三折模型再施加20组配对测试掩码：" + "；".join(stress_text)
 + "。"
                     "这里的波动仅来自测试掩码，不含重新分折或重训的不确定性，也不模拟检出限
删失和整组仪器失效。")]

    bootstrap = json.loads((results / "q2_stratified/summary.json").read_text(encoding
```

```
="utf-8"))
    q2.insert(next(i for i, b in enumerate(q2) if b.get("text", "").startswith("6.3 ")),
              paragraph(f"另在22种风格内分别有放回抽取原样本数，固定混合比例重做200次Bootstrap。原135项的平均频率由"
                        f"{bootstrap['original_top3_mean_frequency_global']:.3f}变为{bootstrap['original_top3_mean_frequency_stratified']:.3f}，"
                        f"达到0.8的位置由14变为{bootstrap['original_top3_stratified_frequency_ge_0p8']}。"
                        "乙酸乙酯对应感官的频率为0.985，4VG为0.890。该补充不改全样本相关前三；"
                        "固定风格比例仍保留风格之间的差异，不能视为已消除混杂。200次频率的小幅差别亦受蒙特卡洛误差影响。"))
    permutation = json.loads((results / "q2_permutation/summary.json").read_text(encoding="utf-8"))
    importance = pd.read_csv(results / "q2_permutation/importance.csv")
    case_rows = []
    cases = [("酯味指甲油(F)", "F_esters_ethac", "ethyl_acetate", "乙酸乙酯"),
             ("4VG感官", "X4vg", "X4VG", "4VG"),
             ("麦芽总体味(F)", "F_malt_all", "ethanol..v.v.", "乙醇"),
             ("陈旧酒花", "stale_hops", "ethyl_acetate", "乙酸乙酯")]
    for label, target, feature, name in cases:
        r = importance.loc[importance.sensory_target.eq(target) & importance.feature.eq(feature)].iloc[0]
        case_rows.append([label + "—" + name, str(int(r.candidate_rank)), f"{r.delta_mse:.5f}",
                          f"{r.fold_sd:.5f}", f"{int(r.positive_folds)}/5"])
    q2 += [{"type": "heading", "level": 2, "text": "6.5 折外置换与预测依赖"},
           paragraph("为区分边际关联与模型使用的信息，按第一问原五折划分和固定ExtraTrees参数重拟合，"
                     f"本次折外RMSE为{permutation['rmse']:.6f}；主表原保存值为{permutation['frozen_oof_rmse']:.6f}。"
                     "在每个验证折中分别置换一个输入变量，连同其缺失指示一起移动，其他输入和模型固定；每折重复5次。"
                     "记f_fk为第f折对第k目标的预测器，V_f为验证样本集合、n_f为其大小，π为该折随机排列，置换引起的误差变化为"),
           {"type": "equation", "label": "6-5", "latex":
            "\\Delta_{jk}^{(f,r)}=\\frac{1}{n_f}\\sum_{i\\in V_f}[(y_{ik}-f_{fk}(x_i^{\\pi_{j,r}}))^2-(y_{ik}-f_{fk}(x_i))^2]."},
           paragraph("先对每折5次取均值，再按验证样本数合并；负值保留，不截为零。表6-3给出代表物，"
                     "标准差按五个折均值计算，正值折数只作稳定性描述，不作为显著性检验。"
                     "对全部231项输入计算后，仅在与相关分析相同的218项化学候选中排序，完整45项结果见表B-3。"),
           table("表6-3 代表候选的折外置换结果", ["感官—化合物", "置换名次", "平均ΔMSE", "折间标准差", "正值折数"],
                 case_rows, [5.6, 2, 2.8, 3.1, 2.5]),
           paragraph(f"置换前三与相关前三的平均Jaccard为{permutation['mean_jaccard_with_spearman']:.3f}，"
                     f"{permutation['targets_with_no_top3_overlap']}/45个目标没有共同候选。两种排名回答不同问题，故并列保留。"
                     "4VG在边际相关、风格内相关、气味依据及预测依赖上均有支持；乙酸乙酯亦被预测器使用。"
                     "麦芽总体—乙醇虽有预测依赖，风格内相关却近零，仍可能只是风格或共同工艺的标记。"
                     "陈旧酒花的置换首项不在原相关前三中，其误差增量也小，不能据此提出加料因果方案。"),
           paragraph("相关输入可能相互替代，逐项置换还会打破真实化学组合。为检查这一问题，"
                     "每个训练折独立重建|Spearman|≥0.7网络，并将连通分量整体置换；五折分别形成"
                     + "、".join(map(str, permutation['modules_per_fold'])) + "个分量。
```

```
    "
                        "成员和逐折误差随附件保存，不将成员不同的分量混作同一对象求平均。"
                        "这些分析提供模型依赖证据，均不识别因果效应。第一问使用231维附件表示，第三问改用69维校准量与157维相对量并另训代理；"
                        "本问候选没有直接作为第三问的生成约束，三问的联系应限于证据核对和缺陷变量解释。")]

    for b in q3:
        if "caption" in b:
            b["caption"] = b["caption"].replace("表7-2", "表7-3")
        if "text" in b:
            b["text"] = b["text"].replace("表7-2", "表7-3")
    defect_rows = [
        ["DMS", "校准量，μg/L", "均值17.25；中位19", "十情景下超阈程度不增加", "给定情景的化学保护"],
        ["Diacetyl", "校准量，mg/L", "均值0.29925；中位0.09", "同上", "给定情景的化学保护"],
        ["4VG", "校准量，mg/L", "均值0.20633；中位0.20", "同上，依题面按缺陷处理", "给定情景的化学保护"],
        ["trans-2-nonenal", "校准量，mg/L", "附件空缺", "候选浓度不大于源浓度", "不保证低于感知阈值"],
        ["Stale hops", "标准化感官预测", "无单一化学行及阈值", "中心预测与配对差Q95不增加，预留模型复核", "代理检查，需实际品评验证"]]
    q3.insert(next(i for i, b in enumerate(q3) if b.get("text", "").startswith("7.8 ")),
              table("表7-2 五项缺陷的变量及保护范围", ["缺陷", "表示与单位", "附件阈值", "约束", "解释边界"],
                    defect_rows, [2.5, 2.8, 3.2, 4.2, 3.3]))
    return q1, q2, q3


def permutation_appendix(root):
    folder = root / "supplement/results/q2_permutation"
    top = pd.read_csv(folder / "top3.csv")
    comparison = pd.read_csv(folder / "top3_comparison.csv").set_index("sensory_target")
    rows = []
    for target in comparison.index:
        selected = top.loc[top.sensory_target.eq(target)].sort_values("candidate_rank")
        rows.append([target] + [f"{r.feature}\nΔ={r.delta_mse:.3g}；{r.positive_folds}/5" for r in selected.itertuples()]
                    + [f"{comparison.loc[target, 'jaccard']:.2f}"])
    return [paragraph("表B-3给出218项化学候选中的预测依赖前三，各项标出平均ΔMSE与正值折数。"
                      "J为与原边际相关前三的Jaccard；原表B-1保持不变。完整231项置换结果、五折相关模块及分层Bootstrap结果随支撑材料保存。"),
            table("表B-3 全部感官的折外置换候选", ["感官", "置换第1项", "第2项", "第3项", "J"], rows, [2.8, 4, 4, 4, 1.2])]
```

paper/verify_paper.py

SHA-256: e165fe39d6a5e582729eb1465250363521e0dc9a28284bc8d2df2f4fb9c6de35

```
"""Verify saved manuscript numbers, references, pagination and source integrity."""
import argparse
import hashlib
import json
from pathlib import Path
import re
import sys
from zipfile import ZipFile

from docx import Document
from lxml import etree
import pandas as pd
from pypdf import PdfReader

HERE=Path(__file__).resolve().parent
ROOT=HERE.parent
REPO=ROOT.parent
parser=argparse.ArgumentParser()
parser.add_argument('--render-dir', default=str(HERE/'qa/render'))
parser.add_argument('--write-page-map',action='store_true')
args=parser.parse_args()
render=Path(args.render_dir)
pdf=HERE/'qa/main.pdf'
docx=HERE/'啤酒风味建模提交稿.docx'
source=json.loads((HERE/'manuscript.json').read_text(encoding="utf-8"))
equations=sum(b['type']=='equation' for b in source['blocks'])
refs=json.loads((HERE/'references.json').read_text(encoding="utf-8"))

def norm(text): return re.sub(r'\s+','',text)
read=PdfReader(pdf)
texts=[page.extract_text() for page in read.pages]
compact=[norm(t) for t in texts]
assert '关键词' in texts[0] and '摘要' in compact[0]
assert '目录' in compact[1]
assert '1问题重述' in compact[2]
appendix=next(i for i,t in enumerate(compact[2:],2) if '附录A感官字段及逐维精度' in t)
body_pages=appendix-2
assert body_pages<=30,body_pages
assert len(list(render.glob('page-*.png')))==len(read.pages)
page_map={}
for mark in json.loads((HERE/'qa/bookmarks.json').read_text(encoding="utf-8")):
    matches=[i for i,t in enumerate(compact[2:],2) if norm(mark['text']) in t]
    assert matches,mark['text']
    page_map[mark['text']]=matches[0]-1
if args.write_page_map:
    (HERE/'qa/page_map.json').write_text(json.dumps(page_map,ensure_ascii=False,indent
=2)+'\n', encoding='utf-8')

doc=Document(docx)
assert len(doc.sections)==3
for section in doc.sections:
    assert abs(section.page_width.cm-21)<.01 and abs(section.page_height.cm-29.7)<.01
assert not doc.core_properties.author and not doc.core_properties.last_modified_by
ns={'w':'http://schemas.openxmlformats.org/wordprocessingml/2006/main',
    'm':'http://schemas.openxmlformats.org/officeDocument/2006/math'}
with ZipFile(docx) as z:
    tree=etree.fromstring(z.read('word/document.xml'))
    assert len(tree.xpath('//m:oMath',namespaces=ns))==equations
    assert len(tree.xpath('//w:drawing',namespaces=ns))==6
```

paper/verify_paper.py（续）

```
    assert not tree.xpath('//m:nary/m:e[not(*)]',namespaces=ns)
    media={hashlib.sha256(z.read(n)).hexdigest() for n in z.namelist() if n.startswith('word/media/')}
    for image in (HERE/'figures').glob('*.png'):
        assert hashlib.sha256(image.read_bytes()).hexdigest() in media,image.name

blocks=source['blocks']
assert sum(b['type']=='equation' for b in blocks)==equations
assert sum(b['type']=='figure' for b in blocks)==6
citations=[]
for b in blocks:
    if b['type']=='reference':continue
    content=json.dumps(b,ensure_ascii=False)
    for n in re.findall(r'\[(\d+)\]',content):
        if int(n) not in citations:citations.append(int(n))
assert citations==list(range(1,len(refs)+1)),citations
assert [r['number'] for r in refs]==list(range(1,len(refs)+1))
table_lookup={b['caption']:b for b in blocks if b['type']=='table'}
q1=next(b for cap,b in table_lookup.items() if cap.startswith('表A-1'))
assert len(q1['rows'])==45
metrics=pd.read_csv(ROOT/'outputs/q1/metrics_by_target.csv').query("model=='extra_trees'")
assert q1['rows']==[[r.target,f'{r.mae:.4f}',f'{r.rmse:.4f}',f'{r.macro_r2:.4f}',f'{r.spearman:.4f}'] for r in metrics.itertuples()]
top=next(b for cap,b in table_lookup.items() if cap.startswith('表B-1'))
assert len(top['rows'])==45
evidence=pd.read_csv(HERE/'chemical_evidence_final.csv')
original=pd.read_csv(ROOT/'outputs/q2/top3_compounds.csv')
keys=['sensory_target','rank','compound']
assert evidence[keys].equals(original[keys])
dictionary=json.loads((HERE/'chemical_references_final.json').read_text(encoding="utf-8"))
assert set(dictionary['evidence_counts'])=={'direct','general','indirect','unverified'}
for r in dictionary['references']:
    assert next(x for x in refs if x['number']==r['number'])['title']==r['title']
transfer=next(b for cap,b in table_lookup.items() if cap.startswith('表C-1'))
assert len(transfer['rows'])==11
summary=pd.read_csv(ROOT/'outputs/q3_refined/transfer/transfer_summary.csv')
chosen=summary.loc[summary.arm.eq('fixed') & summary.pool.eq('contracted_6004') & summary.bound.eq('retain80') & summary.ranking.eq('margin')]
assert int(chosen.fresh_consistency_passed.sum())==10
assert chosen.loc[~chosen.fresh_consistency_passed,'source_beer'].tolist()==[96]
assert abs(chosen.target_improvement.median()-.5774027685828544)<1e-9

assert len(next(b for cap,b in table_lookup.items() if cap.startswith('表B-2'))['rows'])==135
assert len(next(b for cap,b in table_lookup.items() if cap.startswith('表C-2'))['rows'])==72
assert len(next(b for cap,b in table_lookup.items() if cap.startswith('表B-3'))['rows'])==45
supplement=json.loads((ROOT/'supplement/verification.json').read_text(encoding='utf-8'))
assert supplement['status']=='PASS'
# Independently reconcile the supplementary tables with saved result rows.
supp=ROOT/'supplement/results'
model_names={'mean':'训练折均值','extra_trees':'ExtraTrees',
             'gbr_per_target':'逐目标GBR','hgb_per_target':'逐目标HGB'}
random_metrics=pd.read_csv(supp/'q1/metrics_overall.csv').query("protocol=='random_3fold'")
group_metrics=pd.read_csv(supp/'q1/balanced_group_metrics_overall.csv')
```

paper/verify_paper.py（续）

```
expected_models=[]
for label,frame in [('随机三折',random_metrics),('整风格三折',group_metrics)]:
    expected_models += [[label,model_names[r.model],f'{r.rmse:.4f}',f'{r.macro_r2:.4f}'] for r in frame.itertuples()]
assert next(b for cap,b in table_lookup.items() if cap.startswith('表5-3'))['rows']==expected_models
importance=pd.read_csv(supp/'q2_permutation/importance.csv')
expected_cases=[]
for label,target,feature in [('酯味指甲油(F)—乙酸乙酯','F_esters_ethac','ethyl_acetate'),
                             ('4VG感官—4VG','X4vg','X4VG'),
                             ('麦芽总体味(F)—乙醇','F_malt_all','ethanol..v.v.'),
                             ('陈旧酒花—乙酸乙酯','stale_hops','ethyl_acetate')]:
    r=importance.loc[importance.sensory_target.eq(target)&importance.feature.eq(feature)].iloc[0]
    expected_cases.append([label,str(int(r.candidate_rank)),f'{r.delta_mse:.5f}',f'{r.fold_sd:.5f}',f'{int(r.positive_folds)}/5'])
assert next(b for cap,b in table_lookup.items() if cap.startswith('表6-3'))['rows']==expected_cases
perm_top=pd.read_csv(supp/'q2_permutation/top3.csv')
perm_comparison=pd.read_csv(supp/'q2_permutation/top3_comparison.csv').set_index('sensory_target')
expected_perm=[]
for target in perm_comparison.index:
    selected=perm_top.loc[perm_top.sensory_target.eq(target)].sort_values('candidate_rank')
    expected_perm.append([target]+[f'{r.feature}\nΔ={r.delta_mse:.3g}；{r.positive_folds}/5' for r in selected.itertuples()]
                         +[f"{perm_comparison.loc[target,'jaccard']:.2f}"])
assert next(b for cap,b in table_lookup.items() if cap.startswith('表B-3'))['rows']==expected_perm
defects=next(b for cap,b in table_lookup.items() if cap.startswith('表7-2'))['rows']
assert len(defects)==5 and [r[0] for r in defects]==['DMS','Diacetyl','4VG','trans-2-nonenal','Stale hops']
assert defects[3][2]=='附件空缺' and '代理检查' in defects[4][-1]
from submission_content import source_files
inventory=json.loads((HERE/'qa/source_inventory.json').read_text(encoding='utf-8'))
assert [p.relative_to(ROOT).as_posix() for p in source_files(ROOT)]==[r['path'] for r in inventory['files']]
for row in inventory['files']:
    assert hashlib.sha256((ROOT/row['path']).read_bytes()).hexdigest()==row['sha256'], row['path']
final=PdfReader(ROOT/'submission/A2600730.pdf')
assert len(final.pages)==inventory['total_pages']
assert inventory['main_pages']==len(read.pages)
assert not final.metadata.get('/Author')
for row in inventory['files']:
    recovered=[]
    def source_text(text, cm, tm, font, size):
        if abs(size-9)<.01 and tm[4]>=97.8:
            recovered.append(text)
    for page in final.pages[row['first_pdf_page']-1:row['last_pdf_page']]:
        page.extract_text(visitor_text=source_text)
    expected=(ROOT/row['path']).read_text(encoding='utf-8-sig')
    assert norm(''.join(recovered))==norm(expected), ('printed_source_text',row['path'])
assert len(final.pages[1]['/Annots'])==len(inventory['contents_links'])
for annot,link in zip(final.pages[1]['/Annots'],inventory['contents_links']):
    dest=annot.get_object()['/Dest']
    assert final.get_page_number(dest[0].get_object())==link['target_pdf_page']-1

```

paper/verify_paper.py（续）

```
# This is read-only: never rerun experiment writers for paper formatting.
sys.path.insert(0,str(ROOT))
from run_q3_refined import preserve_refined
preserve_refined()
raw=REPO/'2026年第七届MathorCup数学应用挑战赛—大数据竞赛初赛赛题'/'2026年MathorCup大数据挑战赛-赛道A初赛'/'附件'
for r in pd.read_csv(ROOT/'outputs/audit/source_files.csv').itertuples():
    assert hashlib.sha256((raw/r.source).read_bytes()).hexdigest()==r.sha256,r.source

report={'status':'PASS','main_pdf_pages':len(read.pages),'final_pdf_pages':len(final.pages),'printed_source_files':len(inventory['files']),'printed_source_lines':inventory['source_lines'],'front_matter_pages':2,'body_including_references_pages':body_pages,
        'appendix_pages':len(read.pages)-appendix,'native_equations':equations,'figures':6,'references':len(refs),
        'q1_target_rows':45,'top3_positions':135,'transfer_sources':11,'raw_source_hashes_unchanged':9,
        'frozen_refined_protection_passed':True,'source_refs_reconciled':True,
        'supplementary_tables_reconciled':True,
        'visual_review':'Recorded separately in qa/visual_review.json after inspection of page PNGs.',
        'artifact_sha256':{'docx':hashlib.sha256(docx.read_bytes()).hexdigest(),'main_pdf':hashlib.sha256(pdf.read_bytes()).hexdigest(),'final_pdf':hashlib.sha256((ROOT/'submission/A2600730.pdf').read_bytes()).hexdigest()}}
(HERE/'qa/paper_verification.json').write_text(json.dumps(report,ensure_ascii=False,indent=2)+'\n', encoding='utf-8')
print(json.dumps(report,ensure_ascii=False,indent=2))

```

run_all.py

SHA-256: e8fef69f625466acc22b552ec490bc6c5bda3836423deca6cc993a21704c57fd

```
"""Check saved experiments, or explicitly rebuild them in a clean copy."""
import argparse
import os
from pathlib import Path
import subprocess
import sys

HERE = Path(__file__).resolve().parent


def run(script, *arguments):
    env = os.environ.copy()
    env["PYTHONUTF8"] = "1"
    env["PYTHONIOENCODING"] = "utf-8"
    path = HERE / (script if script.startswith("run_") else "src/" + script)
    print(f"Running {script}", flush=True)
    subprocess.run([sys.executable, "-X", "utf8", str(path), *map(str, arguments)],
                   check=True, env=env)


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--rebuild", action="store_true",
                        help="Train all experiments; requires an absent or empty outputs directory.")
    parser.add_argument("--report-dir", type=Path, default=HERE / "review/verification",
                        help="Reports from saved-result checks (outside frozen experiment folders).")
    args = parser.parse_args()
    outputs = HERE / "outputs"
    if args.rebuild:
        supplement = HERE / "supplement/results"
        if (outputs.exists() and any(outputs.iterdir())) or (supplement.exists() and any(supplement.iterdir())):
            parser.error("--rebuild requires a clean copy with no existing outputs; saved results were left unchanged.")
        for script in ["01_audit.py", "02_q1_baseline.py", "02b_q1_robust.py",
                       "03_q2_network.py", "04_stage_summary.py"]:
            run(script)
        run("05_verify_outputs.py", "--output", outputs / "verification.json")
        for script in ["run_q3.py", "run_q3_optimized.py", "run_q3_domain.py", "run_q3_refined.py"]:
            run(script)
        run("run_supplement.py", "--rebuild")
    else:
        if not (outputs / "q3_refined/verification.json").is_file():
            parser.error("Saved experiments are incomplete; use --rebuild in a clean copy with the original attachments.")
        if args.report_dir.resolve().is_relative_to(outputs.resolve()):
            parser.error("--report-dir must be outside outputs to preserve the cited experiments.")
        run("platform_selfcheck.py")
        run("05_verify_outputs.py", "--output", args.report_dir / "q1_q2.json")
        run("25_q3_refined_verify.py", "--output", args.report_dir / "q3_refined.json")
        run("run_supplement.py")


```

run_all.py（续）

```
if __name__ == "__main__":
    main()
```

run_q3.py

SHA-256: 943fd76e3cc904583cf6ba12f559700aa052be0a6ac698e9467b80eac2f78d13

```
"""Reproduce the conditional generation and constrained transfer experiments."""
from pathlib import Path
import subprocess
import sys

HERE = Path(__file__).resolve().parent
for script in ["06_q3_sensory.py", "07_q3_generation.py", "08_q3_transfer.py",
               "09_q3_report.py", "10_q3_verify.py"]:
    print(f"Running {script}", flush=True)
    subprocess.run([sys.executable, "-X", "utf8", str(HERE / "src" / script)], check=T
rue)
```

run_q3_domain.py

SHA-256: 6a8ebd9be8e7a3e9e47ca878daa3d5f25e7dccd5f64d092f6efc8d32ba38a3c4

```
"""Reproduce style-held-out prediction and joint committee transfer checks."""
import hashlib
import json
from pathlib import Path
import subprocess
import sys

from run_q3_optimized import preserve_baseline

HERE = Path(__file__).resolve().parent
PREVIOUS = HERE / "outputs/q3_optimized"
SNAPSHOT = HERE / "outputs/q3_domain/previous_hashes.json"


def preserve_previous():
    preserve_baseline()
    if not (PREVIOUS / "verification.json").exists():
        raise RuntimeError("Run run_q3_optimized.py before the style-held-out experiment.")
    files = [{"path":path.relative_to(PREVIOUS).as_posix(),"bytes":path.stat().st_size,
              "sha256":hashlib.sha256(path.read_bytes()).hexdigest()}
             for path in sorted(PREVIOUS.rglob("*")) if path.is_file()]
    files.sort(key=lambda row: row["path"])
    if SNAPSHOT.exists():
        if files != json.loads(SNAPSHOT.read_text(encoding="utf-8"))["files"]:
            raise RuntimeError("The frozen optimization has changed; restore it or use a separate experiment copy.")
    else:
        SNAPSHOT.parent.mkdir(parents=True,exist_ok=True)
        SNAPSHOT.write_text(json.dumps({"baseline_directory":"solution_a/outputs/q3_optimized",
            "hash_algorithm":"sha256","file_count":len(files),"files":files},
            ensure_ascii=False,indent=2)+"\n",encoding="utf-8", newline="\n")


if __name__ == "__main__":
    preserve_previous()
    for script in ["16_q3_stale_domain.py","17_q3_transfer_domain.py","20_q3_transfer_consensus.py",
                   "18_q3_domain_report.py","19_q3_domain_verify.py"]:
        print(f"Running {script}",flush=True)
        subprocess.run([sys.executable, "-X", "utf8", str(HERE/"src"/script)],check=True)
```

run_q3_optimized.py

SHA-256: a50fb056c91889f873e63b720329ace2a32d8d8e5dcc55f962c1e778b8e1ab7f

```
"""Reproduce the optimization while preserving the frozen first-round baseline."""
import hashlib
import json
from pathlib import Path
import subprocess
import sys

HERE = Path(__file__).resolve().parent
BASELINE = HERE / "outputs/q3"
SNAPSHOT = HERE / "outputs/q3_optimized/baseline_hashes.json"


def preserve_baseline():
    paths = sorted(path for path in BASELINE.rglob("*") if path.is_file())
    if not (BASELINE / "verification.json").exists():
        raise RuntimeError("Run the original third-question experiment before the optimization.")
    rows = [{"path": path.relative_to(BASELINE).as_posix(), "bytes": path.stat().st_size,
             "sha256": hashlib.sha256(path.read_bytes()).hexdigest()} for path in paths]
    rows.sort(key=lambda row: row["path"])
    if SNAPSHOT.exists():
        expected = json.loads(SNAPSHOT.read_text(encoding="utf-8"))
        if rows != expected["files"]:
            raise RuntimeError("The frozen baseline has changed. Restore it or run a separate experiment copy.")
    else:
        SNAPSHOT.parent.mkdir(parents=True, exist_ok=True)
        record = {"baseline_directory": "solution_a/outputs/q3", "hash_algorithm": "sha256",
                  "file_count": len(rows), "files": rows}
        SNAPSHOT.write_text(json.dumps(record, ensure_ascii=False, indent=2) + "\n", encoding="utf-8", newline="\n")


if __name__ == "__main__":
    preserve_baseline()
    for script in ["11_q3_generation_optimized.py", "12_q3_stale_optimized.py",
                   "12b_q3_stale_fixed.py", "13_q3_transfer_optimized.py",
                   "14_q3_optimization_report.py", "15_q3_optimized_verify.py"]:
        print(f"Running {script}", flush=True)
        subprocess.run([sys.executable, "-X", "utf8", str(HERE / "src" / script)], check=True)
```

run_q3_refined.py

SHA-256: e5fde50e937cde776f4f878aa08b680d22ca9950d8f6dc004e37920c3a335176

```
"""Reproduce beer81 diagnosis and balanced transfer without changing prior runs."""
import hashlib
import json
from pathlib import Path
import subprocess
import sys

from run_q3_domain import preserve_previous

HERE=Path(__file__).resolve().parent
PRIOR=HERE/"outputs/q3_domain"
SNAPSHOT=HERE/"outputs/q3_refined/previous_hashes.json"


def preserve_refined():
    preserve_previous()
    if not (PRIOR/"verification.json").exists():
        raise RuntimeError("Run run_q3_domain.py before the refinement experiment.")
    files=[{"path":path.relative_to(PRIOR).as_posix(),"bytes":path.stat().st_size,
            "sha256":hashlib.sha256(path.read_bytes()).hexdigest()}
           for path in sorted(PRIOR.rglob("*")) if path.is_file()]
    files.sort(key=lambda row: row["path"])
    if SNAPSHOT.exists():
        if files!=json.loads(SNAPSHOT.read_text(encoding="utf-8"))["files"]:
            raise RuntimeError("The frozen domain experiment changed; restore it or use a separate copy.")
    else:
        SNAPSHOT.parent.mkdir(parents=True,exist_ok=True)
        SNAPSHOT.write_text(json.dumps({"baseline_directory":"solution_a/outputs/q3_domain",
            "hash_algorithm":"sha256","file_count":len(files),"files":files},ensure_ascii=False,indent=2)+"\n",encoding="utf-8", newline="\n")


if __name__=="__main__":
    preserve_refined()
    for name in ["21_q3_beer81_audit.py","22_q3_stale_refined.py","23_q3_transfer_balanced.py",
                 "24_q3_refinement_report.py","25_q3_refined_verify.py"]:
        print(f"Running {name}",flush=True)
        command = [sys.executable, "-X", "utf8", str(HERE/"src"/name)]
        if name == "25_q3_refined_verify.py":
            command += ["--output", str(HERE/"outputs/q3_refined/verification.json")]
        subprocess.run(command, check=True)
```

run_supplement.py

SHA-256: 1b88d48389db8b27fbbd2ecc6e5d64fcf388d28a68018932c7a369e1d9a273b8

```
"""Build or check the supplementary post-hoc analyses in their own directory."""
import argparse
from pathlib import Path
import subprocess
import sys

HERE = Path(__file__).resolve().parent


def run(name):
    subprocess.run([sys.executable, "-X", "utf8", str(HERE / "supplement" / name)], check=True)


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--rebuild", action="store_true")
    args = parser.parse_args()
    folder = HERE / "supplement/results"
    if args.rebuild:
        if folder.exists() and any(folder.iterdir()):
            parser.error("Supplementary results already exist; rebuild in a clean copy.")
        for name in ["q1_comparison.py", "q1_additional_checks.py", "q2_stratified_bootstrap.py", "q2_permutation.py"]:
            run(name)
    elif not (folder / "q2_permutation/summary.json").exists():
        parser.error("Supplementary results are missing; run --rebuild after the main experiments.")
    run("verify_supplement.py")


if __name__ == "__main__":
    main()
```

src/01_audit.py

SHA-256: 09175215dfb7fb5e6aa55931b9ecc98b35ff878e009b6f2aa0893b3824a34321

```
#!/usr/bin/env python3
"""Read-only, reproducible audit of the eight Track A attachments.

No original file is modified; no network request is made. Findings have not
been independently verified by a human. Uses pandas/numpy and read-only
openpyxl; does not require scipy, and does not author any workbook.
"""

from __future__ import annotations

import argparse
import hashlib
import json
import platform
from pathlib import Path

import numpy as np
import pandas as pd
import openpyxl


ATTRIBUTION = "本地附件结构、数值一致性与来源完整性审计。"
KEYS = ["beer", "beer_id", "tasting_category_fine"]


def normalize_label(value: object) -> str:
    return " ".join(str(value).replace("_x000D_", "").split())


def number(value: object) -> float | None:
    return float(value) if pd.notna(value) and np.isfinite(float(value)) else None


def sha256(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()


def profile(frame: pd.DataFrame, source: str) -> list[dict]:
    records = []
    for column in frame.columns:
        series = frame[column]
        numeric = pd.api.types.is_numeric_dtype(series)
        row = {
            "source": source, "column": str(column), "dtype": str(series.dtype),
            "rows": len(series), "missing": int(series.isna().sum()),
            "missing_rate": float(series.isna().mean()) if len(series) else None,
            "unique_nonmissing": int(series.nunique(dropna=True)),
            "all_missing": bool(series.isna().all()),
            "constant_nonmissing": bool(series.nunique(dropna=True) == 1),
        }
        if numeric:
            finite = series[np.isfinite(series)]
            q1, q3 = finite.quantile([0.25, 0.75]) if len(finite) else (np.nan, np.nan
)
            iqr = q3 - q1
            row.update({
                "min": number(finite.min()), "q01": number(finite.quantile(0.01)),
                "q25": number(q1), "median": number(finite.median()),
                "q75": number(q3), "q99": number(finite.quantile(0.99)),
```

src/01_audit.py（续）

```
                "max": number(finite.max()), "mean": number(finite.mean()),
                "std_ddof1": number(finite.std(ddof=1)),
                "negative_count": int((series < 0).sum()),
                "zero_count": int((series == 0).sum()),
                "infinite_count": int(np.isinf(series).sum()),
                "tukey_outside_1_5_iqr": int(((finite < q1 - 1.5 * iqr) | (finite > q3 + 1.5 * iqr)).sum()),
            })
        records.append(row)
    return records


def table_summary(frame: pd.DataFrame) -> dict:
    numeric = frame.select_dtypes(include=np.number)
    return {
        "rows": len(frame), "columns": len(frame.columns),
        "column_names": list(map(str, frame.columns)),
        "missing_cells": int(frame.isna().sum().sum()),
        "exact_duplicate_rows": int(frame.duplicated().sum()),
        "numeric_columns": len(numeric.columns),
        "infinite_cells": int(np.isinf(numeric).sum().sum()),
    }


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--data-dir", type=Path)
    parser.add_argument("--output-dir", type=Path)
    args = parser.parse_args()
    project = Path(__file__).resolve().parents[2]
    if args.data_dir:
        data_dir = args.data_dir.resolve()
    else:
        candidates = list(project.glob("**/*赛道A初赛/附件"))
        if len(candidates) != 1:
            raise RuntimeError("Expected exactly one Track A attachment directory; use --data-dir.")
        data_dir = candidates[0]
    output = args.output_dir or project / "solution_a/outputs/audit"
    output.mkdir(parents=True, exist_ok=True)
    report_path = project / "solution_a/docs/data_audit.md"
    report_path.parent.mkdir(parents=True, exist_ok=True)

    paths = [data_dir / f"{i}.{'xlsx' if i in (3, 8) else 'csv'}" for i in range(1, 9)]
    paths.append(data_dir / "数据说明.txt")
    hashes_before = {path.name: sha256(path) for path in paths}
    csv = {i: pd.read_csv(data_dir / f"{i}.csv") for i in (1, 2, 4, 5, 6, 7)}
    xlsx = {i: pd.read_excel(data_dir / f"{i}.xlsx", sheet_name=None) for i in (3, 8)}
    frames = {f"{i}.csv": frame for i, frame in csv.items()}
    for i, sheets in xlsx.items():
        frames.update({f"{i}.xlsx::{name}": frame for name, frame in sheets.items()})
    records = [row for source, frame in frames.items() for row in profile(frame, source)]
    columns = pd.DataFrame(records)
    columns.to_csv(output / "column_profile.csv", index=False, lineterminator="\n")
    columns.loc[columns.missing > 0].to_csv(output / "missingness.csv", index=False, lineterminator="\n")
    pd.DataFrame([{"source": path.name, "bytes": path.stat().st_size,
                   "sha256": hashes_before[path.name]} for path in paths]).to_csv(output / "source_files.csv", index=False, lineterminator="\n")
```

src/01_audit.py（续）

```
114
115     chemical = csv[1].set_index("beer")
116     sensory = csv[4].set_index("beer")
117     comments = csv[7].set_index("beer")
118     calibrated = xlsx[8]["Calibrated chemical dataset"].set_index("beer")
119     prices = xlsx[8]["Beer Prices"].set_index("beer")
120     shap = xlsx[8]["GBM SHAP Values"]
121     concentrations = shap.loc[shap.data == "concentration"].set_index("beer")
122     shap_values = shap.loc[shap.data == "SHAP"].set_index("beer")
123     keyed = {
124         "1.csv": chemical, "4.csv": sensory, "7.csv": comments,
125         "8.xlsx::Calibrated chemical dataset": calibrated,
126         "8.xlsx::Beer Prices": prices,
127         "8.xlsx::GBM SHAP Values::concentration": concentrations,
128         "8.xlsx::GBM SHAP Values::SHAP": shap_values,
129     }
130     checks, id_discrepancies = [], []
131     for source, frame in keyed.items():
132         common = chemical.index.intersection(frame.index)
133         aligned = frame.reindex(chemical.index)
134         row = {
135             "source": source, "rows": len(frame), "unique_beer": int(frame.index.nuniq
    ue()),
136             "duplicate_beer_rows": int(frame.index.duplicated(keep=False).sum()),
137             "missing_beer_rows": int(frame.index.isna().sum()),
138             "unique_beer_id": int(frame.beer_id.nunique()),
139             "duplicate_beer_id_rows": int(frame.beer_id.duplicated(keep=False).sum()),
140             "beer_overlap_with_1": len(common),
141             "beer_missing_from_1": len(frame.index.difference(chemical.index)),
142             "beer_missing_in_source": len(chemical.index.difference(frame.index)),
143             "same_order_as_1": frame.index.equals(chemical.index),
144             "beer_id_mismatches_with_1": int((aligned.beer_id != chemical.beer_id).sum
    ()),
145             "category_mismatches_with_1": int((aligned.tasting_category_fine != chemic
    al.tasting_category_fine).sum())
146                 if "tasting_category_fine" in aligned else None,
147         }
148         checks.append(row)
149         for beer in common:
150             if frame.at[beer, "beer_id"] != chemical.at[beer, "beer_id"]:
151                 id_discrepancies.append({"source": source, "beer": beer,
152                     "beer_id_in_1": str(chemical.at[beer, "beer_id"]),
153                     "beer_id_in_source": str(frame.at[beer, "beer_id"])})
154     pd.DataFrame(checks).to_csv(output / "key_checks.csv", index=False, lineterminator
    ="\n")
155     pd.DataFrame(id_discrepancies).to_csv(output / "beer_id_discrepancies.csv", index=
    False, lineterminator="\n")
156
157     # Chemical labels follow 2.csv's order. The complete Spearman reconciliation
158     # below checks this positional correspondence, rather than assuming it.
159     m2, m5, m6 = [csv[i].set_index(csv[i].columns[0]) for i in (2, 5, 6)]
160     chem_columns = list(csv[1].columns[3:])
161     chem_map = dict(zip(m2.columns, chem_columns, strict=True))
162     names = xlsx[8]["Sensory names"]
163     panel_names = names.loc[names.dataset == "Trained panel"]
164     panel_metadata = panel_names.set_index("R_name")
165     selected_45 = panel_names.loc[panel_names.Category_Paper.isin(["Aroma", "Taste"]),
     "R_name"].tolist()
166     sensory_lookup = {normalize_label(row.CleanName_PanelFeedback): row.R_name
167                       for row in panel_names.itertuples(index=False)}
168     sen_map = {name: sensory_lookup[normalize_label(name)] for name in m5.columns}
```

src/01_audit.py（续）

```
    thresholds = xlsx[8]["Sensory thresholds"].set_index("field_name_R")
    feature_map = []
    for display, field in chem_map.items():
        feature_map.append({"domain": "chemical", "field": field, "matrix_label": display,
            "source": "1.csv; 2.csv; 8.xlsx::Sensory thresholds",
            "map_basis": "2.csv order; all Spearman matrix entries reproduced",
            "metadata_name": thresholds.at[field, "chemical_name"],
            "unit": thresholds.at[field, "unit"],
            "in_cross_matrix_6": display in m6.index,
            "in_calibrated_dataset": field in calibrated.columns})
    for display, field in sen_map.items():
        feature_map.append({"domain": "sensory", "field": field, "matrix_label": display,
            "source": "4.csv; 5.csv; 8.xlsx::Sensory names",
            "map_basis": "Sensory names: R_name <-> CleanName_PanelFeedback",
            "category_paper": panel_metadata.at[field, "Category_Paper"],
            "selected_aroma_taste_45": field in selected_45})
    pd.DataFrame(feature_map).to_csv(output / "feature_name_map.csv", index=False, lineterminator="\n")

    reconcile = []
    matrices = {}
    for name, given in [("2.csv", m2), ("5.csv", m5), ("6.csv", m6)]:
        values = given.to_numpy(dtype=float)
        square = values.shape[0] == values.shape[1]
        matrices[name] = {
            "numeric_shape": list(values.shape), "row_labels_unique": bool(given.index.is_unique),
            "column_labels_unique": bool(given.columns.is_unique),
            "min": float(values.min()), "max": float(values.max()),
            "symmetry_max_error": float(np.max(np.abs(values - values.T))) if square else None,
            "diagonal_max_error_from_1": float(np.max(np.abs(np.diag(values) - 1))) if square else None,
        }
        for method in ("pearson", "spearman"):
            if name == "2.csv":
                recalculated = chemical[[chem_map[x] for x in given.columns]].corr(method=method).to_numpy()
                observations = chemical[[chem_map[x] for x in given.columns]].notna().astype(int)
                n_pairs = observations.T @ observations
            elif name == "5.csv":
                recalculated = sensory[[sen_map[x] for x in given.columns]].corr(method=method).to_numpy()
                observations = sensory[[sen_map[x] for x in given.columns]].notna().astype(int)
                n_pairs = observations.T @ observations
            else:
                left = chemical[[chem_map[x] for x in given.index]]
                right = sensory[[sen_map[x] for x in given.columns]]
                combined = pd.concat([left, right], axis=1)
                recalculated = combined.corr(method=method).to_numpy()[:len(left.columns), len(left.columns):]
                n_pairs = left.notna().astype(int).T @ right.notna().astype(int)
            error = np.abs(recalculated - values)
            reconcile.append({"matrix": name, "method": method, "entries": values.size,
                "pair_n_min": int(n_pairs.min().min()), "pair_n_max": int(n_pairs.max().max()),
```

src/01_audit.py（续）

```
                "max_abs_error": float(np.max(error)), "mean_abs_error": float(np.mean(error)),
                "entries_with_error_gt_1e_10": int((error > 1e-10).sum()),
                "matches_at_1e_10": bool(np.all(error <= 1e-10))})
    pd.DataFrame(reconcile).to_csv(output / "matrix_reconciliation.csv", index=False, lineterminator="\n")

    concentration_error = chemical[chem_columns] - concentrations.loc[chemical.index, chem_columns]
    calibration_check = []
    for field in calibrated.columns[2:]:
        comparison = pd.concat([chemical[field], calibrated[field]], axis=1)
        raw = calibrated[field]
        min_positive_half = raw.loc[raw > 0].min() / 2
        replaced = raw.where(raw != 0, min_positive_half)
        log10_values = pd.Series(np.nan, index=raw.index)
        valid_log10 = replaced > 0
        log10_values.loc[valid_log10] = np.log10(replaced.loc[valid_log10])
        log10_error = (chemical[field] - log10_values).abs()
        calibration_check.append({"field": field, "n_pairwise_complete": int(comparison.notna().all(axis=1).sum()),
            "spearman_with_1": float(comparison.corr(method="spearman").iloc[0, 1]),
            "missing_1": int(chemical[field].isna().sum()), "missing_calibrated": int(calibrated[field].isna().sum()),
            "negative_calibrated": int((calibrated[field] < 0).sum()),
            "unit_metadata": thresholds.at[field, "unit"],
            "transformation_group": "first_37_basic" if field in chem_columns[:37] else "remaining_33_GCMS_overlap",
            "calibrated_zero_count": int((raw == 0).sum()),
            "zero_replacement_min_positive_half": number(min_positive_half),
            "n_log10_comparable": int(log10_error.notna().sum()),
            "log10_max_abs_error": number(log10_error.max()),
            "log10_matches_at_1e_7": bool(log10_error.max() < 1e-7),
        })
    pd.DataFrame(calibration_check).to_csv(output / "calibrated_comparison.csv", index=False, lineterminator="\n")

    negative_calibrated = []
    for field in calibrated.columns[2:]:
        for beer, value in calibrated[field][calibrated[field] < 0].items():
            negative_calibrated.append({"beer": beer, "field": field, "value": value,
                "unit_metadata": thresholds.at[field, "unit"]})
    pd.DataFrame(negative_calibrated).to_csv(output / "negative_calibrated_values.csv", index=False, lineterminator="\n")
    extreme = []
    for source, frame in [("1.csv", chemical), ("4.csv", sensory), ("7.csv", comments)]:
        for field in frame.columns[2:]:
            valid = frame[field].dropna()
            if not len(valid):
                continue
            q1, q3 = valid.quantile([0.25, 0.75])
            iqr = q3 - q1
            for beer, value in valid.loc[(valid < q1 - 3 * iqr) | (valid > q3 + 3 * iqr)].items():
                extreme.append({"source": source, "beer": beer, "field": field, "value": value,
                    "q25": q1, "q75": q3, "rule": "outside [q25-3IQR,q75+3IQR]; flag only, not error verdict"})
    pd.DataFrame(extreme).to_csv(output / "extreme_value_flags.csv", index=False, lineterminator="\n")
```

src/01_audit.py（续）

```
265
266     # Inspect formula-bearing forms read-only, so cached zeros are not mistaken
267     # for observations or 12 additional beers.
268     workbook3 = openpyxl.load_workbook(data_dir / "3.xlsx", read_only=True, data_only=
    False)
269     template_formulas = {sheet.title: sum(cell.data_type == "f" for row in sheet for c
    ell in row)
270                          for sheet in workbook3.worksheets}
271     workbook3.close()
272     price_cols = [col for col in prices if col.startswith("Price_")]
273     prices[price_cols].notna().sum().rename("available_beers").to_csv(output / "price_
    availability.csv", lineterminator="\n")
274     category_counts = chemical.tasting_category_fine.value_counts().rename_axis("style
    ").reset_index(name="beers")
275     category_counts.to_csv(output / "style_counts.csv", index=False, lineterminator="\
    n")
276     category_map = pd.concat([chemical.tasting_category_fine.rename("category_1"),
277                               prices.tasting_category.rename("category_prices")], axis=
    1)
278     category_map.groupby(["category_1", "category_prices"]).size().reset_index(name="b
    eers").to_csv(
279         output / "price_category_map.csv", index=False, lineterminator="\n")
280     missing_primary = {str(i): {name: int(count) for name, count in csv[i].isna().sum(
    ).items() if count}
281                        for i in (1, 4, 7)}
282     assertions = {
283         "all_1_4_7_beer_unique_and_same_set": all(frame.index.is_unique and set(frame.
    index) == set(chemical.index)
284                                                   for frame in (chemical, sensory, co
    mments)),
285         "all_three_matrices_reproduce_spearman": all(row["matches_at_1e_10"] for row i
    n reconcile if row["method"] == "spearman"),
286         "concentration_block_matches_1_values_and_missingness": bool(np.nanmax(np.abs(
    concentration_error.to_numpy())) < 1e-10
287             and chemical[chem_columns].isna().equals(concentrations.loc[chemical.index
    , chem_columns].isna())),
288         "source_hashes_unchanged": hashes_before == {path.name: sha256(path) for path
    in paths},
289         "first_37_basic_match_log10_on_valid_pairs": all(row["log10_matches_at_1e_7"]
290             for row in calibration_check if row["transformation_group"] == "first_37_b
    asic"),
291         "aroma_taste_metadata_select_exactly_45": len(selected_45) == 45 and set(selec
    ted_45).issubset(sensory.columns),
292     }
293     basic_log10 = [row for row in calibration_check if row["transformation_group"] ==
    "first_37_basic"]
294     summary = {
295         "attribution": ATTRIBUTION,
296         "audit_date": "2026-10-23", "source_directory": data_dir.relative_to(project).
    as_posix() if data_dir.is_relative_to(project) else str(data_dir),
297         "environment": {"python": platform.python_version(), "pandas": pd.__version__,
     "numpy": np.__version__, "openpyxl": openpyxl.__version__},
298         "source_sha256": hashes_before, "tables": {name: table_summary(frame) for name
    , frame in frames.items()},
299         "documentation_discrepancies": [
300             {"location": "题面第1页", "claim": "226种不同啤酒", "observed": "附件1/4/7各
    250个唯一beer"},
301             {"location": "题面第3页附件2说明", "claim": "皮尔逊相关系数", "observed": "附
    件2/5/6精确复现Spearman"},
302             {"location": "题面附件6说明", "claim": "46×50", "observed": "44×50数值矩阵"}
    ,
```

src/01_audit.py（续）

```
            {"location": "题面附件7说明", "claim": "250×187", "observed": "250×186数值字段，另3列标识/类别"},
        ],
        "primary_missingness": missing_primary, "key_checks": checks, "matrix_shapes": matrices,
        "matrix_reconciliation": reconcile, "assertions": assertions,
        "feature_counts": {"chemical": len(chem_columns), "panel": len(sensory.columns) - 2,
                           "panel_aroma_taste_45": len(selected_45),
                           "review_derived": len(comments.columns) - 2,
                           "review_usable_nonempty": int(comments.iloc[:, 2:].notna().any().sum()),
                           "calibrated_chemical": len(calibrated.columns) - 2},
        "template_formula_counts": template_formulas,
        "supplementary": {
            "shap_data_counts": shap.data.value_counts().to_dict(),
            "concentration_block_max_abs_difference_from_1": float(np.nanmax(np.abs(concentration_error.to_numpy()))),
            "calibrated_missing_cells": int(calibrated.iloc[:, 2:].isna().sum().sum()),
            "calibrated_negative_cells": len(negative_calibrated),
            "calibrated_rank_rho_min": min(row["spearman_with_1"] for row in calibration_check),
            "basic_log10_fields": len(basic_log10),
            "basic_log10_comparable_values": sum(row["n_log10_comparable"] for row in basic_log10),
            "basic_log10_max_abs_error": max(row["log10_max_abs_error"] for row in basic_log10),
            "basic_log10_zero_replacement_count": sum(row["calibrated_zero_count"] for row in basic_log10),
            "basic_log10_excluded_negative_values": sum(row["negative_calibrated"] for row in basic_log10),
            "basic_log10_excluded_calibrated_missing_values": sum(row["missing_calibrated"] for row in basic_log10),
            "remaining_33_not_direct_log10": all(not row["log10_matches_at_1e_7"]
                for row in calibration_check if row["transformation_group"] == "remaining_33_GCMS_overlap"),
            "prices_all_missing_beers": int(prices[price_cols].isna().all(axis=1).sum()),
            "price_missing_cells": int(prices[price_cols].isna().sum().sum()),
        },
        "limitations": ["No raw review text, panel individual ratings, measurement dates or preprocessing recipe supplied.",
                        "No independent human verification; no source data edited or uploaded.",
                        "Outlier flags are descriptive; they do not prove measurement errors.",
                        "Provided MDI/SHAP and full-sample matrices must not be used as supervised-selection inputs before CV splitting."],
    }
    (output / "audit_summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2, allow_nan=False) + "\n", encoding="utf-8", newline="\n")
    report_path.write_text(make_report(summary, csv, xlsx, reconcile, checks, negative_calibrated), encoding="utf-8", newline="\n")
    if not all(assertions.values()):
        raise RuntimeError(f"Audit invariants failed: {assertions}")
    print(json.dumps({"status": "completed", "report": str(report_path), "output_directory": str(output),
                      "assertions": assertions}, ensure_ascii=False, indent=2))


```

src/01_audit.py（续）

```
def make_report(summary: dict, csv: dict, xlsx: dict, reconcile: list, checks: list, negatives: list) -> str:
    rows = []
    for i in (1, 2, 4, 5, 6, 7):
        d = csv[i]
        meaning = {1: "250啤酒×231化学特征，另3列标识/类别", 2: "231×231化学相关矩阵，另1列行标签",
                   4: "250啤酒×50品评特征，另3列标识/类别", 5: "50×50品评相关矩阵，另1列行标签",
                   6: "44×50化学–品评相关子矩阵，另1列行标签",
                   7: "250啤酒×186评论派生特征，另3列标识/类别"}[i]
        rows.append(f"| {i}.csv | {len(d)}×{len(d.columns)} | {int(d.isna().sum().sum())} | {meaning} |")
    sheet_rows = [f"| {name} | {len(frame)}×{len(frame.columns)} | {int(frame.isna().sum().sum())} |"
                  for name, frame in xlsx[8].items()]
    corr_rows = [f"| {row['matrix']} | {row['method']} | {row['pair_n_min']}–{row['pair_n_max']} | {row['max_abs_error']:.6g} | {row['mean_abs_error']:.6g} | {row['matches_at_1e_10']} |"
                 for row in reconcile]
    key4 = next(row for row in checks if row["source"] == "4.csv")
    missing_rows = [f"| {name} | {count} | {count / 250:.1%} |" for name, count in summary["primary_missingness"]["1"].items()]
    neg_rows = [f"| {row['beer']} | {row['field']} | {row['value']:.8g} | {row['unit_metadata'] if pd.notna(row['unit_metadata']) else '未提供'} |" for row in negatives]
    supplementary = summary["supplementary"]
    return f"""# A题附件数据审计

审计日期：2026-10-23。{ATTRIBUTION}

本次仅检查附件本身的可用性、关联键、缺失、数值尺度和相关矩阵口径。原始文件保持不变；未联网、未上传数据。可复现脚本为 `solution_a/src/01_audit.py`，机器证据保存在 `solution_a/outputs/audit/`。脚本执行成功并不等同于人工审核。

## 1. 可用于建模的实际数据

原始附件1、4、7各有 **250个唯一啤酒样本**；可以通过 `beer` 一对一合并。化学变量231列，感官变量50列，论坛评论派生变量186列（其中3列全空，非空变量183列）。不得未经依据将250样本裁剪为226样本。题面/说明的文字口径应与这些实测结构分开记录，需参赛队确认官方是否另有筛选规则。

其中，附件8 `Sensory names` 标为Aroma的15项和Taste的30项恰好组成题目要求的45维；排除的是 `alcohol, aftertaste, body, co2, overall`。完整50字段与45维建模目标须分别统计，字段选择见 `feature_name_map.csv` 的 `selected_aroma_taste_45`。感官量尺和模板细节参见同目录 `sensory_mapping_review.md`；本审计不把附件4的连续值解释成原始0–6分。

题面第1页写“测量了226种不同啤酒的化学特性”，附件数据说明则写250个啤酒样品；本次附件实测支持250。题面对附件6写46×50、附件7写250×187，实测分别为44×50、250×186个数值字段。论文应逐项披露这些冲突，并说明采用实际附件范围的理由，不应静默改写题意。

| 文件 | 读取尺寸（含表头对应列） | 缺失单元格 | 实际含义 |
| --- | --- | ---: | --- |
{chr(10).join(rows)}

CSV均为UTF-8可读取，三个样本级CSV没有完全重复记录；数值列未检测到正负无穷。`beer` 覆盖1–250；样本顺序并非编号顺序，因此应显式按键关联。类别共22种，最小类别Faro为3例，最大类别Blond为31例，分组验证和类别比较须考虑样本不平衡。

附件3含12张 `beer_1`–`beer_12` 品评模板，每张读取为100×33，以及12×52的 `summary`。`summary` 中除编号外612个值全为缓存零，且对应612个公式；`beer_1` 中有54个公式。它是品评表设计，**不是12条额外观测**，也不是缺失的逐人评分记录。表头出现的 `A_ester_fruity`、`F_ester_fruity`、`4vg` 等与附件4名称不完全一致，不能无检查拼接。
```

```
381
382 ## 2. 关联键：必须使用beer
383
384 **高风险、证据充分：附件4和7的 `beer_id` 已损失精度。** 两表各仅有{key4['unique_beer_id']}
    个不同条码，{key4['duplicate_beer_id_rows']}行处于重复条码组；分别有{key4['beer_id_mismat
    ches_with_1']} / 250条记录与附件1的条码不一致。例如beer=10：附件1为5410228202929，附件4/7
    为5410230000000。按条码关联会丢行或产生多对多扩张。
385
386 附件1/4/7的 `beer` 集合、行序及类别逐条一致。附件8的标定数据、SHAP的两个区块、价格表均覆盖相
    同250个 `beer`；其条码与附件1一致。推荐保留附件1的原始条码作为展示信息，合并时使用 `validate
    ='one_to_one'`；不要把已截断条码当作独立产品。价格表按beer排序，与附件1顺序不同。价格类别有
    `wit`→`Wheat`、`bruin`→`Brown`、`oud bruin`→`Flanders old brown` 等别名，映射见 `price_
    category_map.csv`。
387
388 证据：`key_checks.csv`、`beer_id_discrepancies.csv`。条码应按字符串保存；已经丢失的末位不
    能靠格式化恢复。
389
390 ## 3. 2/5/6的相关系数实测复核
391
392 **三张给定矩阵均精确复现Spearman，不能称为Pearson。** 复算按共有 `beer` 对齐、逐变量配对删
    除缺失；Spearman使用平均秩处理并列值，比较容差为1e-10。
393
394 题面第3页附件2说明写“单元格为皮尔逊相关系数”，与附件数据说明的Spearman表述矛盾；以下数值复核
    支持后者。
395
396 | 矩阵 | 复算方法 | 每对样本数 | 最大绝对误差 | 平均绝对误差 | 容差内一致 |
397 | --- | --- | ---: | ---: | ---: | --- |
398 {chr(10).join(corr_rows)}
399
400 化学名称从附件2顺序映射到附件1，并由整张矩阵精确复现交叉验证；231个字段在附件8 `Sensory thre
    sholds.field_name_R` 中均可找到。感官名称按附件8 `Sensory names` 的 `R_name` 与 `CleanNa
    me_PanelFeedback` 显式匹配。**附件4尾部的 `aftertaste, body, co2` 与附件5的 `Carbonation
    , Aftertaste, Body` 顺序不同**；按位置直接对齐会制造错误差异。
401
402 附件2、5的行列标签唯一、矩阵对称且对角为1。附件6仅包含44个化学指标，不代表全231个化学指标。建
    模及机制分析如需其余指标，应从样本级数据重新计算。任何基于相关性进行的特征筛选都应在训练折内重
    算；本节全样本复核只用于审计，不应充当验证折未知信息。
403
404 证据：`matrix_reconciliation.csv`、`feature_name_map.csv`；JSON同时记录矩阵形状、范围、对
    称误差。
405
406 ## 4. 缺失与数值异常
407
408 附件1共73个缺失，占250×231数值单元的0.1264%，分布于以下9列。缺失比例不高，但不能无条件删去所
    有含缺失样本；插补参数须仅在训练折拟合。
409
410 | 化学字段 | 缺失数 | 样本缺失率 |
411 | --- | ---: | ---: |
412 {chr(10).join(missing_rows)}
413
414 附件4无缺失。附件7中 `palate_acetaldehyde`、`palate_acetate`、`palate_dentist` 各250个缺
    失；其余183列无缺失。建议在派生数据中记录并剔除这3列，保留原始附件。
415
416 **中风险、尺度不统一的证据充分：不能假定附件1全部已做标准z变换。** 存在负值且各列样本标准差从
    约0.003123到796.633，均值也不统一为零。`X2.methyltetrahydrofuran.3.one` 最大12596.43287
    （beer=156），`gamma.nonalactone` 最大7256.135876（beer=181），`X.E..beta.damascenone`
    最大5991.012523（beer=151）。附件8的concentration区块复现同样值，因此这不是本次CSV解析偶然
    错误；仍不能仅据此判定测量错误。主模型应比较训练折内标准化/稳健缩放等处理；不要把这些数直接解
    释为mg/L，也不要无依据删除极端样本。已核实的37列log10关系和其余字段的限制见第6节。
417
418 `column_profile.csv` 给出全字段缺失、分位数、均值、标准差、负值、零值、IQR标记。`extreme_va
```

```
    lue_flags.csv` 使用3IQR规则提供定位；IQR=0或稀疏词汇会产生很多标记，**标记不等于异常测量**
    。没有时间列、采集日期和历史版本，本审计不对时效性、批次漂移或随时间变化作推断。
419
420 ## 5. 附件7的含义边界
421
422 附件7是按 `aroma_`、`palate_`、`taste_` 三个前缀组织、每组62词的数值表。附件8 `Sensory nam
    es.dataset` 中把相应描述词来源标为Ratebeer。没有评论正文、用户、逐条评分、评论日期或评论数，
    也没有 `ave_score_5` 字段（虽在名称字典中出现）。因此可把它作为论坛描述词派生特征，不能伪称已
    做原始评论分词、情感分析，亦不能把该表任何一列未经定义当成消费者总体评分。现有文件没有提供这些
    数值的完整归一化/变换说明。
423
424 ## 6. 附件8的补充信息及使用限制
425
426 | 工作表 | 尺寸 | 缺失单元格 |
427 | --- | --- | ---: |
428 {chr(10).join(sheet_rows)}
429
430 - `Sensory thresholds` 是261条化学字段元数据，包含单位、阈值、来源等；部分文本有编码显示问题
    ，阈值和单位需逐项核对后才能用于配方约束。字段元数据不能支持把全表采用同一个逆变换。
431 - `Sensory names` 有101条名称/类别记录，包含50个trained panel指标及Ratebeer字段；字典中出
    现不代表样本表实际存在。
432 - `Calibrated chemical dataset` 是250样本×70个标定数值，另3列标识/类别；有{supplementary[
    'calibrated_missing_cells']}个缺失。与附件1共有字段的Spearman最小约{supplementary['calib
    rated_rank_rho_min']:.6f}，但并非全为1；37个基础理化字段与33个GCMS重叠字段需分别处理，不能
    全表取指数。若要估计现实添加量，应优先核对该页单位和阈值，再定义处理方案。
433 - `GBM importance MDI` 是已有模型的231项重要性，第一列是重要性数值、第二列是字段名；`GBM SH
    AP Values` 的500行是250条SHAP与250条concentration，不是500种啤酒。后者与附件1数值最大差{su
    pplementary['concentration_block_max_abs_difference_from_1']:.3g}。这些是既有模型派生结
    果，训练数据和建模过程未完整给出；不能当成新的独立样本或在交叉验证之前据此筛选特征。
434 - `Beer Prices` 有7个渠道价格列，共{supplementary['price_missing_cells']}个价格缺失；{su
    pplementary['prices_all_missing_beers']}种啤酒所有渠道价格均缺失。不同国家、渠道、计价单位
    和日期未在该页完整说明，不能直接平均作为原料成本或跨币种价格目标。`Sheet2` 为空表。
435
436 **37个基础字段的底数已由本地数值复核为10。** 附件1从 `acetaldehyde` 到 `kcalperc` 的前37个
    数值字段满足 `x=log10(c)`；校准值为0时先用该列最小正值的一半替代。共有{supplementary['basic
    _log10_comparable_values']}个可比较值，包含{supplementary['basic_log10_zero_replacement
    _count']}个零值替代，最大绝对误差{supplementary['basic_log10_max_abs_error']:.6g}。此范围
    明确排除了{supplementary['basic_log10_excluded_calibrated_missing_values']}个校准缺失和{
    supplementary['basic_log10_excluded_negative_values']}个负值，不能宣称它们也通过直接log1
    0验证。其余33个GCMS重叠字段全部不满足同一直接log10关系；不得对231列统一使用 `exp(x)` 或 `10
    **x` 后计算浓度/OAV。已知37列中，逆变换只能还原零值的替代量，也无法据此恢复真实检出量。逐列证
    据见 `calibrated_comparison.csv`。
437
438 标定数据还存在{len(negatives)}个负值，建议保留并核查检测/标定背景，勿擅自视为真实负浓度用于配
    方约束：
439
440 | beer | 字段 | 标定值 | 元数据单位 |
441 | ---: | --- | ---: | --- |
442 {chr(10).join(neg_rows)}
443
444 ## 7. 建模交接与未决事项
445
446 1. 以250个 `beer` 为样本集，保留附件1的231个化学变量和附件4的50个品评字段。45维感官画像明确
    按元数据Aroma15项+Taste30项选定，不能把完整50字段与45维目标混为一谈；把 `beer`、`beer_id`
    排除出预测特征。是否加入类别由研究问题和验证方案决定。
447 2. 模型评价采用严格训练/测试隔离，插补、缩放、降维、特征筛选均在训练折内完成；不要用附件2/5/6
    的全样本相关性或附件8的MDI/SHAP提前筛选后再报告无偏验证。
448 3. 可复现附件相关矩阵采用Spearman；如果问题明确要求Pearson，另外重算Pearson并明确二者差别。
    所有字段名和关联键均显式匹配。
449 4. 论坛数据只支持描述词数值特征分析；缺少原始评论和消费者整体评分时，应明确任务可识别边界。
450 5. 需要参赛队核对题面样本数冲突、预处理说明、标定单位、负浓度、官方勘误；现阶段不把待核实信息
```

写成事实。

8. 复现与证据清单

在仓库根目录执行：

```bash
/Users/Admin1/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/bin/python3 solution_a/src/01_audit.py
```

也可用安装了pandas、numpy、openpyxl的Python执行同一脚本。可选 `--data-dir` 和 `--output-dir` 修改输入及审计证据目录；脚本不修改原始附件。环境版本保存在 `audit_summary.json`，SHA-256保存在 `source_files.csv`；本次执行前后哈希一致。

证据文件：`audit_summary.json`（总体结构与自动校验）、`source_files.csv`（输入哈希）、`column_profile.csv`（逐列画像）、`missingness.csv`、`key_checks.csv`、`beer_id_discrepancies.csv`、`feature_name_map.csv`、`matrix_reconciliation.csv`、`calibrated_comparison.csv`、`negative_calibrated_values.csv`、`extreme_value_flags.csv`、`style_counts.csv`、`price_availability.csv`、`price_category_map.csv`。所有结论来自本地附件；没有外部文献补充或人工审核声明。
"""

if __name__ == "__main__":
    main()

src/02_q1_baseline.py

SHA-256: 6357ece28220bd3567803a214e4b83d84c8192713622bbf0500ab58e6b0f94ab

```
"""Leakage-controlled prediction of 45 *standardized* sensory dimensions.

This is a starting baseline, NOT a recovered 0-6 sensory score. The attachment
does not provide the per-panelist inverse calibration needed for that recovery.
"""
import json
from pathlib import Path
import platform

import joblib
import numpy as np
import pandas as pd
import sklearn
from scipy.stats import spearmanr
from sklearn.compose import TransformedTargetRegressor
from sklearn.dummy import DummyRegressor
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.impute import SimpleImputer
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import GridSearchCV, KFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from threadpoolctl import threadpool_limits

from common import SEED, SOLUTION, load_xy

OUT = SOLUTION / "outputs" / "q1"
ALPHAS = [0.1, 1.0, 10.0, 100.0, 1000.0, 10000.0, 100000.0, 1000000.0]


def ridge_pipeline():
    return Pipeline([
        ("imputer", SimpleImputer(strategy="median", add_indicator=True,
                                  keep_empty_features=True)),
        ("scale", StandardScaler()),
        ("model", TransformedTargetRegressor(
            regressor=Ridge(solver="svd"), transformer=StandardScaler())),
    ])


def make_search(seed):
    # Scaling/imputation, including y scaling, are refit in EVERY inner fold.
    return GridSearchCV(
        ridge_pipeline(), {"model__regressor__alpha": ALPHAS},
        cv=KFold(n_splits=4, shuffle=True, random_state=seed),
        scoring="neg_mean_squared_error", n_jobs=1, refit=True,
        error_score="raise",
    )


def tree_pipeline(seed):
    return Pipeline([
        ("imputer", SimpleImputer(strategy="median", add_indicator=True,
                                  keep_empty_features=True)),
        ("model", ExtraTreesRegressor(
            n_estimators=300, min_samples_leaf=3, max_features=0.7,
            random_state=seed, n_jobs=1)),
    ])
```

src/02_q1_baseline.py（续）

```
60
61
62 def scores(y, pred):
63     return {
64         "mae": float(mean_absolute_error(y, pred)),
65         "rmse": float(np.sqrt(mean_squared_error(y, pred))),
66         "macro_r2": float(r2_score(y, pred, multioutput="uniform_average")),
67     }
68
69
70 def main():
71     OUT.mkdir(parents=True, exist_ok=True)
72     meta, x, y = load_xy()
73     targets = y.columns.tolist()
74     # Prespecified nested five-fold baseline. IDs and styles are not predictors.
75     outer = KFold(n_splits=5, shuffle=True, random_state=SEED)
76     predictions = {name: np.full(y.shape, np.nan) for name in
77                    ("mean", "ridge", "extra_trees", "ridge_mask10", "ridge_mask20",
78                     "extra_trees_mask10", "extra_trees_mask20")}
79     fold_ids = np.zeros(len(x), dtype=int)
80     folds = []
81     stress_counts = []
82     for fold, (train, test) in enumerate(outer.split(x), start=1):
83         fold_ids[test] = fold
84         xtr, xte, ytr, yte = x.iloc[train], x.iloc[test], y.iloc[train], y.iloc[test]
85         search = make_search(SEED + fold)
86         search.fit(xtr, ytr)
87         candidates = {
88             "ridge": search.best_estimator_,
89             "mean": DummyRegressor(strategy="mean").fit(xtr, ytr),
90             # A fixed nonlinear comparison, not tuned on outer test outcomes.
91             "extra_trees": tree_pipeline(SEED + fold).fit(xtr, ytr),
92         }
93         for name, model in candidates.items():
94             pred = model.predict(xte)
95             predictions[name][test] = pred
96             folds.append({"fold": fold, "model": name, "train_n": len(train),
97                           "test_n": len(test), **scores(yte, pred),
98                           "ridge_alpha": search.best_params_["model__regressor__alpha"
   ]
99                           if name == "ridge" else None})
100         # Deployment stress: randomly hide observed TEST covariates only.
101         # Identical random uniforms make 10% masks a subset of the 20% masks.
102         uniforms = np.random.default_rng(SEED + 100 + fold).random(xte.shape)
103         for rate, suffix in [(0.1, "mask10"), (0.2, "mask20")]:
104             mask = (uniforms < rate) & xte.notna().to_numpy()
105             hidden = xte.mask(mask)
106             stress_counts.append({"fold": fold, "rate": rate,
107                                   "additional_hidden": int(mask.sum()),
108                                   "original_observed": int(xte.notna().sum().sum())})
109             for base in ("ridge", "extra_trees"):
110                 name = f"{base}_{suffix}"
111                 pred = candidates[base].predict(hidden)
112                 predictions[name][test] = pred
113                 folds.append({"fold": fold, "model": name, "train_n": len(train),
114                               "test_n": len(test), **scores(yte, pred),
115                               "ridge_alpha": search.best_params_["model__regressor__al
   pha"]
116                               if base == "ridge" else None})
117         print(f"Fold {fold}/5: alpha={search.best_params_['model__regressor__alpha']},
    "
```

src/02_q1_baseline.py（续）

```
118               f"ridge RMSE={scores(yte, predictions['ridge'][test])['rmse']:.4f}", flu
    sh=True)
119
120     metric_rows, target_rows = [], []
121     truth = y.to_numpy()
122     for name, pred in predictions.items():
123         if not np.isfinite(pred).all():
124             raise AssertionError(f"Incomplete predictions: {name}")
125         metric_rows.append({"model": name, **scores(truth, pred)})
126         for j, target in enumerate(targets):
127             target_rows.append({"model": name, "target": target,
128                                 **scores(truth[:, j], pred[:, j]),
129                                 "spearman": float(spearmanr(truth[:, j], pred[:, j]).s
    tatistic)})
130         table = pd.concat([meta, pd.DataFrame({"fold": fold_ids}),
131                            pd.DataFrame(pred, columns=targets)], axis=1)
132         table.to_csv(OUT / f"oof_{name}.csv", index=False, lineterminator="\n")
133     pd.DataFrame(metric_rows).to_csv(OUT / "metrics_overall.csv", index=False, lineter
    minator="\n")
134     pd.DataFrame(target_rows).to_csv(OUT / "metrics_by_target.csv", index=False, linet
    erminator="\n")
135     pd.DataFrame(folds).to_csv(OUT / "metrics_by_fold.csv", index=False, lineterminato
    r="\n")
136     pd.DataFrame(stress_counts).to_csv(OUT / "missingness_stress.csv", index=False, li
    neterminator="\n")
137
138     # Full-data model is for later synthesis/inference; fitted values are not a test.
139     final = make_search(SEED + 99)
140     final.fit(x, y)
141     joblib.dump({"pipeline": final.best_estimator_, "features": x.columns.tolist(),
142                  "targets": targets, "target_scale": "attachment_4_standardized"},
143                 OUT / "ridge_model.joblib")
144     fitted = final.predict(x)
145     pd.concat([meta, pd.DataFrame(fitted, columns=targets)], axis=1).to_csv(
146         OUT / "fitted_standardized_NOT_validation.csv", index=False, lineterminator="\
    n")
147     pd.concat([meta, y], axis=1).to_csv(OUT / "observed_standardized.csv", index=False
    , lineterminator="\n")
148     tree = tree_pipeline(SEED + 99).fit(x, y)
149     joblib.dump({"pipeline": tree, "features": x.columns.tolist(), "targets": targets,
150                  "target_scale": "attachment_4_standardized"}, OUT / "extra_trees_mode
    l.joblib")
151     summary = {
152         "seed": SEED, "samples": len(x), "features": x.shape[1], "targets": targets,
153         "validation": "5 outer shuffled folds, 4 inner folds; one seed; no style/ID co
    variates",
154         "unit": "attachment 4 standardized sensory values; NOT original 0-6 scores",
155         "final_ridge_alpha": final.best_params_["model__regressor__alpha"],
156         "alpha_grid": ALPHAS, "metrics": metric_rows,
157         "limitations": [
158             "Original 0-6 scale is not identifiable from blank cards and panelist-norm
    alized means.",
159             "Random CV estimates new beers in the existing style mixture, not unseen-s
    tyle transfer.",
160             "Global preprocessing used by attachment creators cannot be undone; metric
    s are conditional on supplied data.",
161             "Missingness stress is synthetic random masking, not evidence about detect
    ion-limit censoring.",
162             "One split seed is a baseline, not completed robustness or uncertainty val
    idation.",
163         ],
```

src/02_q1_baseline.py（续）

```
        "versions": {"python": platform.python_version(), "numpy": np.__version__,
                     "pandas": pd.__version__, "sklearn": sklearn.__version__},
    }
    (OUT / "summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8", newline="\n")
    print(pd.DataFrame(metric_rows).to_string(index=False), flush=True)


if __name__ == "__main__":
    with threadpool_limits(limits=1):
        main()
```

src/02b_q1_robust.py

SHA-256: e7ea1f17106fdeabf52087e9532421927210e3ebbcb05c1dcf13b61bb14679f1

```
"""Exploratory robust Ridge follow-up after diagnosing extreme covariates.

Clipping bounds, imputation, X/Y scaling and alpha selection all stay inside
the training folds. The original split is reused: this is a robustness check,
not an untouched confirmatory test. No samples or targets are deleted.
"""
import importlib.util
import json
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from scipy.stats import spearmanr
from sklearn.model_selection import GridSearchCV, KFold
from sklearn.pipeline import Pipeline
from threadpoolctl import threadpool_limits

from common import QuantileClipper, SEED, SOLUTION, load_xy

spec = importlib.util.spec_from_file_location("baseline", Path(__file__).with_name("02_q1_baseline.py"))
baseline = importlib.util.module_from_spec(spec)
spec.loader.exec_module(baseline)
OUT = SOLUTION / "outputs" / "q1_robust"


def search(seed):
    plain = baseline.ridge_pipeline()
    model = Pipeline([("clip", QuantileClipper()), *plain.steps])
    return GridSearchCV(
        model, {"model__regressor__alpha": baseline.ALPHAS},
        cv=KFold(n_splits=4, shuffle=True, random_state=seed),
        scoring="neg_mean_squared_error", n_jobs=1, error_score="raise",
    )


def main():
    OUT.mkdir(parents=True, exist_ok=True)
    meta, x, y = load_xy()
    pred = {name: np.full(y.shape, np.nan) for name in
            ("ridge_winsor", "ridge_winsor_mask10", "ridge_winsor_mask20")}
    fold_ids = np.zeros(len(x), dtype=int)
    fold_metrics = []
    clipping = []
    for fold, (train, test) in enumerate(KFold(5, shuffle=True, random_state=SEED).split(x), 1):
        fold_ids[test] = fold
        fitted = search(SEED + fold).fit(x.iloc[train], y.iloc[train])
        alpha = fitted.best_params_["model__regressor__alpha"]
        xte = x.iloc[test]
        bounds = fitted.best_estimator_.named_steps["clip"].bounds_
        counts = ((xte.to_numpy() < bounds[0]) | (xte.to_numpy() > bounds[1])).sum(axis=0)
        clipping.extend({"fold": fold, "feature": feature, "test_values_clipped": int(n),
                         "train_lower_01": float(bounds[0, j]), "train_upper_99": float(bounds[1, j])}
                        for j, (feature, n) in enumerate(zip(x.columns, counts)))
```

src/02b_q1_robust.py（续）

```
        uniforms = np.random.default_rng(SEED + 100 + fold).random(xte.shape)
        for rate, name in [(0, "ridge_winsor"), (.1, "ridge_winsor_mask10"), (.2, "ridge_winsor_mask20")]:
            test_input = xte.mask((uniforms < rate) & xte.notna().to_numpy())
            pred[name][test] = fitted.predict(test_input)
            fold_metrics.append({"fold": fold, "model": name, "ridge_alpha": alpha,
                                 **baseline.scores(y.iloc[test], pred[name][test])})
        print(f"Robust fold {fold}/5: alpha={alpha}", flush=True)
    metrics, by_target = [], []
    for name, values in pred.items():
        assert np.isfinite(values).all()
        metrics.append({"model": name, **baseline.scores(y, values)})
        for j, target in enumerate(y.columns):
            by_target.append({"model": name, "target": target,
                              **baseline.scores(y.iloc[:, j], values[:, j]),
                              "spearman": float(spearmanr(y.iloc[:, j], values[:, j]).statistic)})
        pd.concat([meta, pd.DataFrame({"fold": fold_ids}), pd.DataFrame(values, columns=y.columns)], axis=1).to_csv(
            OUT / f"oof_{name}.csv", index=False, lineterminator="\n")
    pd.DataFrame(metrics).to_csv(OUT / "metrics_overall.csv", index=False, lineterminator="\n")
    pd.DataFrame(by_target).to_csv(OUT / "metrics_by_target.csv", index=False, lineterminator="\n")
    pd.DataFrame(fold_metrics).to_csv(OUT / "metrics_by_fold.csv", index=False, lineterminator="\n")
    pd.DataFrame(clipping).to_csv(OUT / "clipping_bounds.csv", index=False, lineterminator="\n")
    final = search(SEED + 99).fit(x, y)
    joblib.dump({"pipeline": final.best_estimator_, "features": x.columns.tolist(),
                 "targets": y.columns.tolist(), "target_scale": "attachment_4_standardized"},
                OUT / "ridge_winsor_model.joblib")
    summary = {
        "purpose": "Exploratory robustness follow-up; original five folds reused after anomaly diagnosis",
        "seed": SEED, "train_quantiles": [.01, .99], "samples_retained": len(x),
        "metrics": metrics, "final_alpha": final.best_params_["model__regressor__alpha"],
        "limitations": ["Not a fresh holdout test", "Fixed 1%/99% clipping can suppress genuine extremes",
                        "Original 0-6 scores still not identified", "One split seed only"],
    }
    (OUT / "summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8", newline="\n")
    print(pd.DataFrame(metrics).to_string(index=False), flush=True)


if __name__ == "__main__":
    with threadpool_limits(limits=1):
        main()
```

src/03_q2_network.py

SHA-256: 56e7858449d5aa25efd2ce581a083d6b477711991dd7f78a3c9552849cc5603b

```python
#!/usr/bin/env python3
"""Q2 descriptive association baseline.

Run from any directory with the repository .venv Python. Data transformations,
selection rules, and validation scope are recorded in outputs/q2/README.md.
"""
from __future__ import annotations

import argparse
import json
from pathlib import Path

import numpy as np
import pandas as pd
from scipy.stats import pearsonr, rankdata, spearmanr


ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "2026年第七届MathorCup数学应用挑战赛—大数据竞赛初赛赛题" / "2026年MathorCup大数据挑战赛-赛道A初赛" / "附件"
OUTPUT = ROOT / "solution_a" / "outputs" / "q2"
EXCLUDE_X = {
    "kcalperc", "esters_sum", "aroma_hops_sum", "acids_sum", "sulfur_sum",
    "bitterness", "color.EBC.", "pH", "protein.g.L.", "betaglucan.mg.L.",
    "GFS", "unfermentables", "CO2.PSI.",
}
EXCLUDE_Y = {"alcohol", "aftertaste", "body", "co2", "overall"}


def unit_columns(a: np.ndarray) -> np.ndarray:
    centered = a - a.mean(axis=0)
    denominator = np.sqrt(np.sum(centered * centered, axis=0))
    return np.divide(centered, denominator, out=np.zeros_like(centered), where=denominator > 0)


def cross_correlation(x: np.ndarray, y: np.ndarray, method: str) -> np.ndarray:
    """Pairwise-complete correlation, reranking y when an x column is missing.

    Every target must be observed. Ranking each x column independently is not
    sufficient for pairwise Spearman: y must also be reranked on the same rows.
    Constant columns are returned as NaN and cannot enter a Top-3.
    """
    if not np.isfinite(y).all():
        raise ValueError("Targets contain missing/nonfinite values; revise pairwise handling.")
    result = np.full((x.shape[1], y.shape[1]), np.nan)
    complete = np.isfinite(x).all(axis=0)
    xx, yy = x[:, complete], y
    if method == "spearman":
        xx, yy = rankdata(xx, axis=0), rankdata(yy, axis=0)
    result[complete] = unit_columns(xx).T @ unit_columns(yy)
    result[np.flatnonzero(complete)[np.ptp(xx, axis=0) == 0], :] = np.nan
    result[:, np.ptp(yy, axis=0) == 0] = np.nan
    for j in np.flatnonzero(~complete):
        keep = np.isfinite(x[:, j])
        if keep.sum() < 3:
            continue
        xx, yy = x[keep, j:j + 1], y[keep]
```

src/03_q2_network.py（续）

```
        if method == "spearman":
            xx, yy = rankdata(xx, axis=0), rankdata(yy, axis=0)
        if np.ptp(xx) == 0:
            continue
        result[j] = (unit_columns(xx).T @ unit_columns(yy))[0]
        result[j, np.ptp(yy, axis=0) == 0] = np.nan
    return np.clip(result, -1, 1)


def top3(correlation: np.ndarray) -> np.ndarray:
    """Stable input-order tie break; missing/constant correlations rank last."""
    if np.any(np.isfinite(correlation).sum(axis=0) < 3):
        raise ValueError("Fewer than three eligible candidates for a target.")
    return np.argsort(-np.nan_to_num(np.abs(correlation), nan=-1), axis=0, kind="stable")[:3]


def jaccard(a: np.ndarray, b: np.ndarray) -> float:
    aa, bb = set(a.tolist()), set(b.tolist())
    return len(aa & bb) / len(aa | bb)


def save_csv(frame: pd.DataFrame, name: str) -> None:
    frame.to_csv(OUTPUT / name, index=False, encoding="utf-8-sig", float_format="%.10g", lineterminator="\n")


def main(bootstraps: int, seed: int) -> None:
    OUTPUT.mkdir(parents=True, exist_ok=True)
    chemical = pd.read_csv(DATA / "1.csv")
    sensory = pd.read_csv(DATA / "4.csv")
    supplied = pd.read_csv(DATA / "2.csv", index_col=0)
    metadata = pd.read_excel(DATA / "8.xlsx", sheet_name="Sensory thresholds")
    metadata = metadata.dropna(subset=["field_name_R"]).drop_duplicates("field_name_R").set_index("field_name_R")
    features = chemical.columns[3:].tolist()
    targets = [c for c in sensory.columns[3:] if c not in EXCLUDE_Y]
    assert len(features) == 231 and len(targets) == 45
    assert chemical.beer.is_unique and sensory.beer.is_unique
    assert supplied.index.tolist() == supplied.columns.tolist()
    merged = chemical.merge(sensory[["beer", *targets]], on="beer", validate="one_to_one", how="inner")
    assert len(merged) == len(chemical) == len(sensory) == 250
    # Attachments retain the same numeric feature order; verify using every pair.
    actual_spearman = chemical[features].corr(method="spearman").to_numpy()
    actual_pearson = chemical[features].corr(method="pearson").to_numpy()
    r = supplied.to_numpy(dtype=float)
    upper = np.triu_indices(len(features), 1)
    delta_s = np.abs(actual_spearman[upper] - r[upper])
    delta_p = np.abs(actual_pearson[upper] - r[upper])
    assert float(np.nanmax(delta_s)) < 1e-10, "Attachment order/definition changed."
    pretty = dict(zip(features, supplied.index.astype(str)))
    candidates = [c for c in features if c not in EXCLUDE_X]
    x, y = merged[candidates].to_numpy(float), merged[targets].to_numpy(float)
    rho = cross_correlation(x, y, "spearman")
    pearson = cross_correlation(x, y, "pearson")
    # Check the optimized pairwise routine against scipy, including missingness.
    check_columns = np.unique(np.r_[np.arange(5), np.flatnonzero(~np.isfinite(x).all(axis=0))])
    max_check_error = 0.0
    for j in check_columns:
```

src/03_q2_network.py（续）

```
        keep = np.isfinite(x[:, j])
        for t in [0, 20, 44]:
            for matrix, function in [(rho, spearmanr), (pearson, pearsonr)]:
                reference = function(x[keep, j], y[keep, t]).statistic
                max_check_error = max(max_check_error, abs(float(reference - matrix[j, t])))
    assert max_check_error < 1e-12

    node_table = pd.DataFrame({"field": features, "chemical_name": [pretty[f] for f in features], "eligible_top3": [f in candidates for f in features]})
    for col in ["origin_paper", "unit", "calibrated", "off_flavors", "threshold_mean", "threshold_median"]:
        node_table[col] = metadata[col].reindex(features).to_numpy()
    network_summary = []
    for threshold in [0.6, 0.7, 0.8]:
        adjacency = np.abs(r) >= threshold
        np.fill_diagonal(adjacency, False)
        left, right = np.where(np.triu(adjacency, 1))
        weights = r[left, right]
        save_csv(pd.DataFrame({
            "source": np.asarray(features)[left], "target": np.asarray(features)[right],
            "source_name": supplied.index.to_numpy()[left], "target_name": supplied.index.to_numpy()[right],
            "spearman_rho": weights, "absolute_weight": np.abs(weights), "sign": np.sign(weights).astype(int),
        }), f"network_edges_rho_{threshold:.1f}.csv")
        node_table[f"degree_{threshold:.1f}"] = adjacency.sum(axis=1)
        node_table[f"strength_{threshold:.1f}"] = (np.abs(r) * adjacency).sum(axis=1)
        network_summary.append({"absolute_rho_threshold": threshold, "edges": len(weights), "positive_edges": int((weights > 0).sum()), "negative_edges": int((weights < 0).sum()), "isolated_nodes": int((adjacency.sum(axis=1) == 0).sum())})
    save_csv(node_table, "network_nodes.csv")

    initial_top = top3(rho)
    pearson_top = top3(pearson)
    # This is a sensitivity check, not a causal adjustment or formal partial rho.
    within_x = merged[candidates] - merged.groupby("tasting_category_fine")[candidates].transform("mean")
    within_y = merged[targets] - merged.groupby("tasting_category_fine")[targets].transform("mean")
    within_rho = cross_correlation(within_x.to_numpy(float), within_y.to_numpy(float), "spearman")
    within_top = top3(within_rho)
    rng = np.random.default_rng(seed)
    selections = np.zeros_like(rho, dtype=int)
    draws = np.empty((bootstraps, *rho.shape), dtype=float)
    for b in range(bootstraps):
        rows = rng.integers(0, len(merged), len(merged))
        draws[b] = cross_correlation(x[rows], y[rows], "spearman")
        winners = top3(draws[b])
        selections[winners, np.arange(len(targets))] += 1
        if (b + 1) % 50 == 0:
            print(f"bootstrap {b + 1}/{bootstraps}", flush=True)
    frequency = selections / bootstraps
    assert np.all(selections.sum(axis=0) == 3 * bootstraps)
    lower, upper_ci = np.nanquantile(draws, [0.025, 0.975], axis=0)
    valid_draws = np.isfinite(draws).sum(axis=0)
    sign_agreement = ((np.sign(draws) == np.sign(rho)[None]) & np.isfinite(draws)).sum(axis=0) / valid_draws
    global_ranks = np.empty_like(initial_top, shape=rho.shape)
```

src/03_q2_network.py（续）

```
    within_ranks = np.empty_like(global_ranks)
    for t in range(len(targets)):
        global_ranks[np.argsort(-np.abs(rho[:, t]), kind="stable"), t] = np.arange(1, len(candidates) + 1)
        within_ranks[np.argsort(-np.abs(within_rho[:, t]), kind="stable"), t] = np.arange(1, len(candidates) + 1)
    rows = []
    method_rows = []
    for t, target in enumerate(targets):
        method_rows.append({
            "sensory_target": target,
            "spearman_top3": " | ".join(candidates[j] for j in initial_top[:, t]),
            "pearson_top3": " | ".join(candidates[j] for j in pearson_top[:, t]),
            "within_style_top3": " | ".join(candidates[j] for j in within_top[:, t]),
            "jaccard_spearman_pearson": jaccard(initial_top[:, t], pearson_top[:, t]),
            "jaccard_global_within_style": jaccard(initial_top[:, t], within_top[:, t]),
        })
        for j, candidate in enumerate(candidates):
            rows.append({
                "sensory_target": target, "rank": int(global_ranks[j, t]), "compound": candidate,
                "chemical_name": pretty[candidate], "n_observed_pairs": int(np.isfinite(x[:, j]).sum()),
                "spearman_rho": rho[j, t], "pearson_r": pearson[j, t],
                "bootstrap_top3_count": int(selections[j, t]), "bootstrap_top3_frequency": frequency[j, t],
                "bootstrap_rho_lower_95": lower[j, t], "bootstrap_rho_upper_95": upper_ci[j, t],
                "bootstrap_sign_agreement": sign_agreement[j, t], "bootstrap_valid_draws": int(valid_draws[j, t]),
                "within_style_spearman_rho": within_rho[j, t], "within_style_rank": int(within_ranks[j, t]),
                "is_pearson_top3": bool(j in pearson_top[:, t]),
                "is_within_style_top3": bool(j in within_top[:, t]),
            })
    scores = pd.DataFrame(rows).sort_values(["sensory_target", "rank"])
    top_table = scores[scores["rank"] <= 3].copy()
    top_table = top_table.merge(node_table, left_on="compound", right_on="field", how="left", validate="many_to_one", suffixes=("", "_network")).drop(columns=["field", "chemical_name_network"])
    methods = pd.DataFrame(method_rows)
    save_csv(scores, "all_candidate_stability.csv")
    save_csv(top_table, "top3_compounds.csv")
    save_csv(methods, "method_sensitivity.csv")
    summary = {
        "scope": "Full-sample descriptive associations; neither causal evidence nor held-out predictive validation.",
        "n_beers": len(merged), "n_features": len(features), "n_candidate_compounds": len(candidates),
        "n_sensory_targets": len(targets), "bootstrap_draws": bootstraps, "seed": seed,
        "excluded_compounds": sorted(EXCLUDE_X), "excluded_sensory_targets": sorted(EXCLUDE_Y),
        "correlation_audit": {"supplied_vs_spearman_max_abs_error": float(np.nanmax(delta_s)), "supplied_vs_spearman_mean_abs_error": float(np.nanmean(delta_s)), "supplied_vs_pearson_mean_abs_error": float(np.nanmean(delta_p)), "supplied_vs_pearson_max_abs_error": float(np.nanmax(delta_p)), "optimized_routine_vs_scipy_max_abs_error": max_check_error},
        "networks": network_summary,
        "top3_stability": {"mean_selection_frequency": float(top_table.bootstrap_top3_
```

src/03_q2_network.py（续）

```
    frequency.mean()), "median_selection_frequency": float(top_table.bootstrap_top3_frequency.median()), "number_frequency_at_least_0_8": int((top_table.bootstrap_top3_frequency >= .8).sum()), "total_selected": len(top_table), "max_frequency_monte_carlo_standard_error": float(np.sqrt(.25 / bootstraps))},
204         "mean_jaccard_spearman_pearson": float(methods.jaccard_spearman_pearson.mean()),
205         "mean_jaccard_global_within_style": float(methods.jaccard_global_within_style.mean()),
206         "targets_with_no_global_within_overlap": int((methods.jaccard_global_within_style == 0).sum()),
207         "style_counts": {str(k): int(v) for k, v in merged.tasting_category_fine.value_counts().items()},
208         "limitations": ["No causal interpretation", "No per-candidate mechanism literature verification yet", "All-sample network is not an external independent prior", "Global row bootstrap does not fix style composition", "Mean-centered within-style rank correlation is a sensitivity analysis, not full confounding control", "Continuous standardized targets do not supply raw 0-6 calibration", "Graph provides structure/context; this baseline ranks marginal correlations and does not fit a graph-regularized predictive model"],
209     }
210     (OUTPUT / "summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2) + "\n", encoding="utf-8", newline="\n")
211     (OUTPUT / "README.md").write_text(f"""# 第二问首轮关联分析
212
213 本目录由 `solution_a/src/03_q2_network.py` 生成，计算结果与 scipy 成对相关计算进行数值核对。重跑：
214
215 ```bash
216 .venv/bin/python solution_a/src/03_q2_network.py --bootstraps {bootstraps} --seed {seed}
217 ```
218
219 数据为按 beer 一对一连接的 250 款啤酒；全图 231 节点、Top-3 候选 218 项、感官目标 45 项。所有计算在附件既有数值表示上进行，没有统一取指数或把连续标签伪还原为 0–6 分。
220
221 | 文件 | 用途 |
222 |---|---|
223 | network_nodes.csv | 全节点名映射、来源/单位/阈值及各阈值度数、加权度数 |
224 | network_edges_rho_*.csv | 绝对相关阈值 0.6/0.7/0.8 的有符号网络 |
225 | top3_compounds.csv | 每个感官维度按绝对 Spearman 相关选择的 3 个候选 |
226 | all_candidate_stability.csv | 全部 218×45 候选的相关、bootstrap 入选频率、分位区间和风格敏感性 |
227 | method_sensitivity.csv | Pearson 替代和去风格均值后的 Top-3/Jaccard |
228 | summary.json | 样本、阈值、网络、稳定性、数值校验及限制摘要 |
229
230 2.csv 与全样本 Spearman 矩阵最大差 {np.nanmax(delta_s):.3g}，故它是当前样本导出的描述性网络；预测交叉验证时必须仅在训练折重建。全样本图可解释 Top-3 的邻域和相关冗余，但本轮未拟合图正则化模型。阈值 0.6/0.7/0.8 是稀疏化参数，不是显著性阈值。
231
232 Bootstrap 为 {bootstraps} 次全样本行重采样，重新排名，固定种子 {seed}，频率最大蒙特卡洛标准误 {np.sqrt(.25 / bootstraps):.4f}。并列时按附件特征顺序打破。区间为相关系数的描述性 bootstrap 分位数，不是选择后推断保证。去风格均值对 X、Y 都在当前表示上按风格减均值后计算 Spearman，只检查风格间均值混杂。
233
234 本目录结论是**关联候选，不是因果因素**；稳定性不等于化学合理性。`8.xlsx` 元信息仅作附件证据，逐条机制文献验证尚未完成。`5.csv/6.csv` 和预计算 GBM/SHAP 未用于筛选。Top-3 的所有 135 项中，入选频率 ≥0.8 的有 {summary['top3_stability']['number_frequency_at_least_0_8']} 项；Spearman/Pearson 平均 Jaccard 为 {summary['mean_jaccard_spearman_pearson']:.3f}，去风格均值前后为 {summary['mean_jaccard_global_within_style']:.3f}。这些是全样本解释稳定性，不是未见啤酒预测性能。
235 """, encoding="utf-8", newline="\n")
```

src/03_q2_network.py（续）

```
    print(json.dumps(summary, ensure_ascii=False, indent=2))


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--bootstraps", type=int, default=200)
    parser.add_argument("--seed", type=int, default=20261023)
    args = parser.parse_args()
    if args.bootstraps < 1:
        parser.error("--bootstraps must be positive")
    main(args.bootstraps, args.seed)
```

src/04_stage_summary.py

SHA-256: 79ef7338a76e6ec46e81e86782c902bc295b1606754c986c18b36d76e3ded45d

```
"""Render inspectable first-round research figures and a generated progress note.

All displayed quantitative results are read from saved experiment outputs.
"""
import json

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from common import SOLUTION


def main():
    base = pd.read_csv(SOLUTION / "outputs/q1/metrics_overall.csv").set_index("model")
    robust = pd.read_csv(SOLUTION / "outputs/q1_robust/metrics_overall.csv").set_index("model")
    q2 = json.loads((SOLUTION / "outputs/q2/summary.json").read_text(encoding="utf-8"))
    targets = pd.read_csv(SOLUTION / "outputs/q1/metrics_by_target.csv")
    tree_targets = targets[targets.model == "extra_trees"]
    figures = SOLUTION / "outputs/figures"
    figures.mkdir(parents=True, exist_ok=True)
    plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 10,
                         "axes.spines.top": False, "axes.spines.right": False,
                         "savefig.facecolor": "white", "figure.facecolor": "white"})
    fig, axes = plt.subplots(1, 2, figsize=(11.2, 5.2), gridspec_kw={"width_ratios": [1, 1]})
    labels = ["Mean baseline", "Ridge", "Robust Ridge*", "ExtraTrees"]
    values = [base.loc["mean", "rmse"], base.loc["ridge", "rmse"],
              robust.loc["ridge_winsor", "rmse"], base.loc["extra_trees", "rmse"]]
    axes[0].barh(labels, values, color=["#A3A8AE", "#A3A8AE", "#C98A27", "#2674A4"], height=.6)
    axes[0].invert_yaxis()
    axes[0].set_xlim(0, max(values) * 1.24)
    axes[0].set_xlabel("Pooled out-of-fold RMSE")
    axes[0].set_title("45 sensory dimensions | 250 beers", loc="left", pad=16)
    for i, value in enumerate(values):
        axes[0].text(value + .01, i, f"{value:.4f}", va="center")
    rates = [0, 10, 20]
    tree_values = base.loc[["extra_trees", "extra_trees_mask10", "extra_trees_mask20"], "rmse"].to_numpy()
    robust_values = robust.loc[["ridge_winsor", "ridge_winsor_mask10", "ridge_winsor_mask20"], "rmse"].to_numpy()
    axes[1].plot(rates, tree_values, "o-", color="#2674A4", label="ExtraTrees", linewidth=1.7)
    axes[1].plot(rates, robust_values, "s--", color="#C98A27", label="Robust Ridge*", linewidth=1.7)
    axes[1].axhline(base.loc["mean", "rmse"], color="#666666", linestyle=":", label="Mean baseline")
    axes[1].set_xticks(rates)
    axes[1].set_ylim(0, .65)
    axes[1].set_xlabel("Additional random test-feature masking (%)")
    axes[1].set_ylabel("Pooled out-of-fold RMSE")
    axes[1].set_title("Missing-feature stress experiment", loc="left", pad=16)
    axes[1].legend(frameon=False, fontsize=9, loc="lower right")
    for rate, value in zip(rates, tree_values):
```

src/04_stage_summary.py（续）

```
        axes[1].annotate(f"{value:.4f}", (rate, value), xytext=(0, -17),
                         textcoords="offset points", ha="center", color="#2674A4", fontsize=9)
    for ax in axes:
        ax.grid(axis="x" if ax is axes[0] else "y", color="#E6E8EB", linewidth=.7)
        ax.set_axisbelow(True)
    fig.suptitle("A track: first-round sensory prediction", x=.08, ha="left", fontsize=16, fontweight="bold")
    fig.text(.08, .025,
             "Targets: attachment 4 standardized values, not original 0-6 scores. One fixed five-fold split.\n"
             "*Robust Ridge is a post-diagnosis check using the same folds; bounds are fitted within training folds.",
             fontsize=9, color="#50555B", va="bottom")
    fig.subplots_adjust(left=.16, right=.97, top=.80, bottom=.24, wspace=.44)
    fig.savefig(figures / "q1_baseline_and_missingness.png", dpi=300)
    fig.savefig(figures / "q1_baseline_and_missingness.svg")
    plt.close(fig)

    combined = pd.concat([base.loc[["mean", "ridge", "extra_trees"]], robust.loc[["ridge_winsor"]]])
    rows = "\n".join(f"| {name} | {row.mae:.4f} | {row.rmse:.4f} | {row.macro_r2:.4f} |"
                     for name, row in combined.iterrows())
    gain = (1 - base.loc["extra_trees", "rmse"] / base.loc["mean", "rmse"]) * 100
    stress = (base.loc["extra_trees_mask20", "rmse"] / base.loc["extra_trees", "rmse"] - 1) * 100
    weak = tree_targets[tree_targets.macro_r2 < .05].sort_values("macro_r2")
    weak_text = ", ".join(f"{r.target} ({r.macro_r2:.3f})" for r in weak.itertuples())
    text = f"""# A题首轮阶段结果

更新日期：2026-10-23。本文件保存开题与首轮实验的历史结果；第三问后续实验见 [第三问结果](q3_results.md)。

## 已确认的数据口径

- 250 款啤酒、231 个化学/理化特征，22 个风格；建模以 `beer` 一对一合并。
- 45 维 Aroma/Flavour = Aroma 15 + Taste 30；4.csv 另有 4 个 Palate 和 1 个 Overall 项。
- 3.xlsx 是 12 张空白评分卡。4.csv 是品评员标准化后聚合的连续标签。现有附件及已检查的作者公开包中，尚未识别出恢复真实 0–6 分所需的原始评分/参数。
- 2/5/6.csv 的矩阵均可由 Spearman 重现。全样本相关矩阵不能直接用作预测交叉验证的折外先验。
- 37 个基础指标的 log10 关系已实测；其余 GCMS 字段不能统一逆对数得到浓度。完整证据见 `data_audit.md` 和 `sensory_mapping_review.md`。

## 第一问：连续感官预测基线

所有指标均在附件既有连续尺度上计算。外层 5 折，岭回归内层 4 折选择正则强度；中位数补值、X/Y标准化及稳健裁剪均只拟合训练折。`beer/beer_id/风格` 没有进入预测特征。ExtraTrees 使用固定参数，未用外层结果调参。

| 模型 | MAE | RMSE | 宏平均 R² |
|---|---:|---:|---:|
{rows}

ExtraTrees 的 RMSE 比均值基线降低 **{gain:.2f}%**。附加随机遮蔽20%测试特征时，RMSE 为 **{base.loc['extra_trees_mask20', 'rmse']:.4f}**，较原实验上升 **{stress:.2f}%**。这只是一次随机缺失压力实验，不代表检出限导致的非随机缺失。

![第一问基线与缺失压力实验](../outputs/figures/q1_baseline_and_missingness.png)


```

src/04_stage_summary.py（续）

```
弱预测维度（ExtraTrees R²<0.05）：{weak_text}。尤其 `stale_hops` 的预测能力很弱，第三问不能凭该模型就宣称实际缺陷已经受到可靠控制。

原岭回归被 beer=156 的极端化合物值严重影响；随机遮蔽20%时恰好隐藏该特征，造成误差降低。不得将其解释为“缺失更多提升模型”。稳健岭回归保留全部250款酒，在每个训练折内部按1%/99%分位裁剪后重跑。此实验由诊断触发、复用原划分，属于探索性稳健性检查，仍需新划分/重复验证。

## 第二问：网络、Top-3与稳定性

网络含231节点，|ρ|≥0.6/0.7/0.8分别有266/109/39条边。排除汇总和非单一化合物指标后，218候选×45目标共9,810条关联记录，完成200次配对行bootstrap。

135个Top-3位置中，入选频率≥0.8的仅 **{q2['top3_stability']['number_frequency_at_least_0_8']}** 项；平均频率 **{q2['top3_stability']['mean_selection_frequency']:.3f}**。Spearman/Pearson平均Jaccard为 **{q2['mean_jaccard_spearman_pearson']:.3f}**，去风格均值前后为 **{q2['mean_jaccard_global_within_style']:.3f}**，其中 **{q2['targets_with_no_global_within_overlap']}** 个维度完全没有交集。

这些结果表明风格混杂、相关化合物替代和尺度影响需要进一步处理，不能把当前Top-3直接写成因果因素。下一轮应验证模块代表性、风格内分层bootstrap，并逐条核实气味机制。结果见 `../outputs/q2/top3_compounds.csv` 与 `../outputs/q2/README.md`。

## 第三问：首轮时点的设计方案

首轮拟采用小样本条件收缩Gaussian copula，以目标风格重采样为基线。后续已完成生成与迁移实验，完整结果见 [第三问结果](q3_results.md)。设计要求是对Tripel/Kriek/Lambic做训练折内拟合和留出分布验证，报告边缘距离、联合结构、多样性和最近邻记忆程度。

迁移必须逐项检查DMS/Diacetyl/4VG/trans-2-nonenal/Stale hops。对已知阈值项约束超阈程度不增加；对缺失阈值或未核验变换项另列更强替代约束和不可验证项。Stale hops只有感官代理，当前预测较弱。70个校准字段与其余相对量可以构成混合表示，但必须用同一表示重训感官模型，不能直接塞进本轮模型。详细公式与验证方案见 `q2_q3_design_review.md`。

## 下一轮优先顺序

1. 明确0–6评分的可识别性边界；取得原始评分/校准数据，或有依据地声明相对等级的定义，分别验证连续值与等级误差。
2. 在新划分或重复交叉验证中比较ExtraTrees与稳健线性模型，加入与现有/未见风格相匹配的验证，评估维度级不确定性和弱缺陷目标。
3. 改善第二问的风格混杂与模块解释，完成化学机制文献核验。
4. 实现第三问并报告有效迁移与失败案例，随后整理完整论文和支撑材料。

## 来源与复现

原始附件保持原样，哈希记录在 `../outputs/audit/source_files.csv`。一键复现命令及文件索引见 `../README.md`。官方原研究为 [Schreurs等，Nature Communications 15,2368 (2024)](https://doi.org/10.1038/s41467-024-46346-0)，公开数据见 [Zenodo 10653704](https://zenodo.org/records/10653704)。来源对应核验详情已记录于感官映射报告。
"""
    (SOLUTION / "docs/stage1_results.md").write_text(text, encoding="utf-8", newline="\n")
    print(f"Saved {figures / 'q1_baseline_and_missingness.png'} and docs/stage1_results.md")


if __name__ == "__main__":
    main()
```

src/05_verify_outputs.py

SHA-256: 34ba31074ee8d2133c5d57dda035e9d14ea5726396991e8ca04d99cc96980810

```
"""Check provenance, sample alignment, saved metrics and graph invariants.

These checks assess numerical consistency and saved-output integrity.
"""
import hashlib
import json
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from common import SOLUTION, data_dir, load_xy


def main(output_path=None):
    checks = []
    meta, x, y = load_xy()
    sources = pd.read_csv(SOLUTION / "outputs/audit/source_files.csv")
    for row in sources.itertuples():
        actual = hashlib.sha256((data_dir() / row.source).read_bytes()).hexdigest()
        assert actual == row.sha256, f"Source changed: {row.source}"
    checks.append(f"Original source SHA-256 unchanged: {len(sources)} files")

    reference_fold = None
    models_checked = 0
    for directory in ["q1", "q1_robust"]:
        folder = SOLUTION / "outputs" / directory
        metrics = pd.read_csv(folder / "metrics_overall.csv")
        for row in metrics.itertuples():
            oof = pd.read_csv(folder / f"oof_{row.model}.csv")
            assert oof.beer.is_unique and oof.beer.tolist() == meta.beer.tolist()
            assert (oof.tasting_category_fine == meta.tasting_category_fine).all()
            assert set(oof.fold) == set(range(1, 6))
            if reference_fold is None:
                reference_fold = oof.fold.to_numpy()
            np.testing.assert_array_equal(oof.fold, reference_fold)
            prediction = oof[y.columns].to_numpy()
            assert np.isfinite(prediction).all()
            np.testing.assert_allclose(
                [row.mae, row.rmse, row.macro_r2],
                [mean_absolute_error(y, prediction),
                 np.sqrt(mean_squared_error(y, prediction)),
                 r2_score(y, prediction, multioutput="uniform_average")],
                rtol=1e-10, atol=1e-12,
            )
            models_checked += 1
    checks.append(f"Sample alignment, folds and independently recomputed metrics: {models_checked} OOF tables")

    naive = pd.read_csv(SOLUTION / "outputs/q1/oof_mean.csv")
    for fold in range(1, 6):
        test = reference_fold == fold
        expected = np.tile(y.loc[~test].mean().to_numpy(), (test.sum(), 1))
        np.testing.assert_allclose(naive.loc[test, y.columns], expected, atol=1e-12)
    checks.append("Mean baseline equals the training-fold target mean for every held-out beer")

```

src/05_verify_outputs.py（续）

```
    bounds = pd.read_csv(SOLUTION / "outputs/q1_robust/clipping_bounds.csv")
    for fold in range(1, 6):
        stored = bounds[bounds.fold == fold].set_index("feature").loc[x.columns]
        actual = np.nanquantile(x.loc[reference_fold != fold], [.01, .99], axis=0)
        np.testing.assert_allclose(stored[["train_lower_01", "train_upper_99"]].to_numpy().T,
                                   actual, rtol=1e-10, atol=1e-12)
    checks.append("Saved outer-fold clipping bounds match only the corresponding training rows")

    for filename in ["q1/ridge_model.joblib", "q1/extra_trees_model.joblib",
                     "q1_robust/ridge_winsor_model.joblib"]:
        saved = joblib.load(SOLUTION / "outputs" / filename)
        assert saved["features"] == x.columns.tolist() and saved["targets"] == y.columns.tolist()
        prediction = saved["pipeline"].predict(x.iloc[:2])
        assert prediction.shape == (2, 45) and np.isfinite(prediction).all()
    checks.append("All three saved models reload and predict using the recorded feature/target order")

    q2 = SOLUTION / "outputs/q2"
    all_scores = pd.read_csv(q2 / "all_candidate_stability.csv")
    top = pd.read_csv(q2 / "top3_compounds.csv")
    nodes = pd.read_csv(q2 / "network_nodes.csv")
    summary = json.loads((q2 / "summary.json").read_text(encoding="utf-8"))
    assert len(all_scores) == 218 * 45 and len(top) == 3 * 45 and len(nodes) == 231
    assert not all_scores.duplicated(["sensory_target", "compound"]).any()
    assert (top.groupby("sensory_target").size() == 3).all()
    np.testing.assert_allclose(all_scores.groupby("sensory_target").bootstrap_top3_frequency.sum(), 3)
    assert all_scores.bootstrap_top3_frequency.between(0, 1).all()
    assert (all_scores.bootstrap_valid_draws == summary["bootstrap_draws"]).all()
    for threshold in [.6, .7, .8]:
        edges = pd.read_csv(q2 / f"network_edges_rho_{threshold:.1f}.csv")
        assert not (edges.source == edges.target).any()
        assert not edges.duplicated(["source", "target"]).any()
        assert (edges.absolute_weight >= threshold - 1e-10).all()
        assert int(nodes[f"degree_{threshold:.1f}"].sum()) == 2 * len(edges)
    checks.append("Q2 candidate counts, bootstrap frequencies and undirected graph degree sums reconcile")

    for path in ["docs/stage1_results.md", "docs/data_audit.md", "docs/sensory_mapping_review.md",
                 "docs/q1_validation_review.md", "docs/q2_q3_design_review.md",
                 "outputs/figures/q1_baseline_and_missingness.png", "README.md"]:
        assert (SOLUTION / path).is_file(), f"Missing deliverable: {path}"
    checks.append("Stage notes, independent review and chart files exist")
    result = {"status": "passed", "checks": checks,
              "scope": "Saved-output integrity and numerical consistency; not proof of model adequacy"}
    if output_path is not None:
        output_path = Path(output_path)
        output_path.parent.mkdir(parents=True, exist_ok=True)
        output_path.write_text(json.dumps(result, ensure_ascii=False, indent=2) + "\n", encoding="utf-8", newline="\n")
    print(json.dumps(result, ensure_ascii=False, indent=2))


if __name__ == "__main__":
    import argparse
    parser = argparse.ArgumentParser(description=__doc__)
```

src/05_verify_outputs.py（续）

```
    parser.add_argument("--output", type=Path, help="Optional report path; existing results are read-only by default.")
    main(parser.parse_args().output)
```

src/06_q3_sensory.py

SHA-256: d4e7e5cc71284e24318acffa44983de164adef4a2c24523606ed20d1dcbc78d9

```
"""Train and validate the hybrid-representation sensory proxy for question 3.

Predictions are attachment-standardized values, not recovered 0--6 scores.
Bootstrap paired differences constrain a fitted proxy, not actual taste.
"""
from __future__ import annotations

import json
import platform
from time import perf_counter

import joblib
import numpy as np
import pandas as pd
import sklearn
from scipy.stats import spearmanr
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import KFold
from sklearn.pipeline import Pipeline
from threadpoolctl import threadpool_limits

from common import SEED, SOLUTION
from q3_data import DEFECTS, load_q3_data

OUT = SOLUTION / "outputs" / "q3" / "sensory"
N_BOOTSTRAP = 40
BOOTSTRAP_TREES = 120


def tree_pipeline(seed: int, n_estimators: int = 300) -> Pipeline:
    return Pipeline([
        ("imputer", SimpleImputer(strategy="median", add_indicator=True,
                                  keep_empty_features=True)),
        ("model", ExtraTreesRegressor(
            n_estimators=n_estimators, min_samples_leaf=3, max_features=0.7,
            random_state=seed, n_jobs=1)),
    ])


def scores(y, pred) -> dict:
    return {
        "mae": float(mean_absolute_error(y, pred)),
        "rmse": float(np.sqrt(mean_squared_error(y, pred))),
        "macro_r2": float(r2_score(y, pred, multioutput="uniform_average")),
    }


def main() -> None:
    started = perf_counter()
    OUT.mkdir(parents=True, exist_ok=True)
    meta, x, y, manifest = load_q3_data()
    # Input artifacts preserve missingness; imputation is fitted by each model.
    meta.to_csv(OUT / "meta.csv", index=False, lineterminator="\n")
    x.to_csv(OUT / "hybrid_features.csv", index=False, lineterminator="\n")
    y.to_csv(OUT / "y45.csv", index=False, lineterminator="\n")
    manifest.to_csv(OUT / "feature_manifest.csv", index=False, lineterminator="\n")
    (OUT / "defect_thresholds.json").write_text(
```

src/06_q3_sensory.py（续）

```
        json.dumps(DEFECTS, ensure_ascii=False, indent=2, allow_nan=False), encoding="utf-8", newline="\n")

    predictions = {name: np.full(y.shape, np.nan) for name in ("mean", "extra_trees")}
    fold_ids = np.zeros(len(x), dtype=int)
    fold_metrics = []
    for fold, (train, test) in enumerate(
            KFold(5, shuffle=True, random_state=SEED).split(x), 1):
        fold_ids[test] = fold
        model = tree_pipeline(SEED + fold).fit(x.iloc[train], y.iloc[train])
        predictions["extra_trees"][test] = model.predict(x.iloc[test])
        predictions["mean"][test] = y.iloc[train].mean().to_numpy()
        for name, values in predictions.items():
            fold_metrics.append({"fold": fold, "model": name, "train_n": len(train),
                                 "test_n": len(test), **scores(y.iloc[test], values[test])})
        print(f"Q3 sensory OOF fold {fold}/5: "
              f"RMSE={scores(y.iloc[test], predictions['extra_trees'][test])['rmse']:.6f}", flush=True)

    metric_rows, target_rows = [], []
    for name, values in predictions.items():
        if not np.isfinite(values).all():
            raise AssertionError(f"Incomplete OOF predictions: {name}")
        metric_rows.append({"model": name, **scores(y, values)})
        for j, target in enumerate(y):
            target_rows.append({"model": name, "target": target,
                                **scores(y.iloc[:, j], values[:, j]),
                                "spearman": float(spearmanr(y.iloc[:, j], values[:, j]).statistic)})
        pd.concat([meta, pd.DataFrame({"fold": fold_ids}),
                   pd.DataFrame(values, columns=y.columns)], axis=1).to_csv(
                       OUT / f"oof_{name}.csv", index=False, lineterminator="\n")
    pd.DataFrame(metric_rows).to_csv(OUT / "metrics_overall.csv", index=False, lineterminator="\n")
    pd.DataFrame(target_rows).to_csv(OUT / "metrics_by_target.csv", index=False, lineterminator="\n")
    pd.DataFrame(fold_metrics).to_csv(OUT / "metrics_by_fold.csv", index=False, lineterminator="\n")
    pd.concat([meta, y], axis=1).to_csv(OUT / "observed_standardized.csv", index=False, lineterminator="\n")

    # A full-data inference model is distinct from held-out evaluation above.
    final = tree_pipeline(SEED + 99).fit(x, y)
    artifact = {"pipeline": final, "features": x.columns.tolist(),
                "targets": y.columns.tolist(), "target_scale": "attachment_4_standardized",
                "representation": "69_calibrated_plus_157_relative", "seed": SEED}
    joblib.dump(artifact, OUT / "sensory_model.joblib", compress=3)
    pd.concat([meta, pd.DataFrame(final.predict(x), columns=y.columns)], axis=1).to_csv(
        OUT / "fitted_standardized_NOT_validation.csv", index=False, lineterminator="\n")

    # Migration sources must not appear in their own proxy's training set.
    source_mask = meta["tasting_category_fine"].eq("Pils/Lager").to_numpy()
    transfer_train = np.flatnonzero(~source_mask)
    transfer_test = np.flatnonzero(source_mask)
    if (len(transfer_train), len(transfer_test)) != (239, 11):
        raise ValueError("Expected 239 non-Pils training samples and 11 held-out sources")
```

src/06_q3_sensory.py（续）

```
    transfer_model = tree_pipeline(SEED + 199).fit(x.iloc[transfer_train], y.iloc[tran
sfer_train])
    transfer_artifact = {
        "pipeline": transfer_model, "features": x.columns.tolist(), "targets": y.colum
ns.tolist(),
        "target_scale": "attachment_4_standardized", "representation": "69_calibrated_
plus_157_relative",
        "seed": SEED, "excluded_style": "Pils/Lager", "train_global_indices": transfer
_train,
        "heldout_global_indices": transfer_test, "source_exclusion_reason": "prevent_s
ource_memorization",
    }
    joblib.dump(transfer_artifact, OUT / "transfer_proxy.joblib", compress=3)
    heldout_pred = transfer_model.predict(x.iloc[transfer_test])
    heldout_mean = np.broadcast_to(y.iloc[transfer_train].mean().to_numpy(), heldout_p
red.shape)
    heldout_metrics, heldout_targets = [], []
    for name, values in [("transfer_proxy", heldout_pred), ("non_pils_mean", heldout_m
ean)]:
        heldout_metrics.append({"model": name, "test_n": len(transfer_test),
                                **scores(y.iloc[transfer_test], values)})
        for j, target in enumerate(y):
            heldout_targets.append({"model": name, "target": target,
                                    **scores(y.iloc[transfer_test, j], values[:, j])})
    pd.concat([meta.iloc[transfer_test].reset_index(drop=True),
               pd.DataFrame(heldout_pred, columns=y.columns)], axis=1).to_csv(
                   OUT / "transfer_pils_heldout_predictions.csv", index=False, lineter
minator="\n")
    pd.concat([meta.iloc[transfer_test].reset_index(drop=True),
               y.iloc[transfer_test].reset_index(drop=True)], axis=1).to_csv(
                   OUT / "transfer_pils_heldout_observed.csv", index=False, linetermin
ator="\n")
    pd.DataFrame(heldout_metrics).to_csv(OUT / "transfer_pils_heldout_metrics.csv", in
dex=False, lineterminator="\n")
    pd.DataFrame(heldout_targets).to_csv(OUT / "transfer_pils_heldout_metrics_by_targe
t.csv", index=False, lineterminator="\n")

    # Row-paired bootstrap: one fitted model predicts BOTH source and candidate.
    # Every Pils/Lager source is excluded. Save global resample positions so
    # exclusion and the train/select-audit split can be independently checked.
    rng = np.random.default_rng(SEED + 2000)
    models, indices = [], []
    for b in range(N_BOOTSTRAP):
        index = rng.choice(transfer_train, size=len(transfer_train), replace=True)
        fitted = tree_pipeline(SEED + 3000 + b, BOOTSTRAP_TREES).fit(
            x.iloc[index], y["stale_hops"].iloc[index])
        models.append(fitted)
        indices.append(index)
        if (b + 1) % 10 == 0:
            print(f"Q3 stale_hops bootstrap {b + 1}/{N_BOOTSTRAP}", flush=True)
    bootstrap = {"models": models, "bootstrap_indices": indices,
                 "features": x.columns.tolist(), "target": "stale_hops",
                 "target_scale": "attachment_4_standardized", "seed": SEED,
                 "bootstrap_seed": SEED + 2000, "paired_quantile": 0.95,
                 "pairing": "same_model_source_candidate_difference",
                 "train_global_indices": transfer_train,
                 "excluded_global_indices": transfer_test,
                 "excluded_style": "Pils/Lager",
                 "selection_model_indices": list(range(30)),
                 "audit_model_indices": list(range(30, N_BOOTSTRAP)),
                 "audit_protocol": "screen_with_first_30; audit_last_10_once_without_r
```

src/06_q3_sensory.py（续）

```
    eselection",
159                 "interpretation": "model_proxy_uncertainty_only_not_taste_guarantee"}
160     joblib.dump(bootstrap, OUT / "stale_bootstrap.joblib", compress=3)
161     np.savez_compressed(OUT / "stale_bootstrap_indices.npz", indices=np.asarray(indice
    s),
162                         source_beer=meta["beer"].to_numpy())
163     bootstrap_predictions = np.vstack([model.predict(x) for model in models])
164     pd.concat([meta, pd.DataFrame({
165         "bootstrap_mean": bootstrap_predictions.mean(axis=0),
166         "bootstrap_q025": np.quantile(bootstrap_predictions, 0.025, axis=0),
167         "bootstrap_q975": np.quantile(bootstrap_predictions, 0.975, axis=0),
168         "observed_stale_hops": y["stale_hops"],
169     })], axis=1).to_csv(OUT / "stale_bootstrap_fitted_NOT_validation.csv", index=False
    , lineterminator="\n")
170
171     # Reuse of the established Q1 folds is explicitly checked if available.
172     reference_path = SOLUTION / "outputs" / "q1" / "oof_extra_trees.csv"
173     identical_q1_folds = None
174     if reference_path.exists():
175         reference = pd.read_csv(reference_path)
176         identical_q1_folds = bool(np.array_equal(reference["beer"], meta["beer"])
177                                   and np.array_equal(reference["fold"], fold_ids))
178         if not identical_q1_folds:
179             raise AssertionError("Q3 OOF does not match existing Q1 folds")
180     overall = pd.DataFrame(metric_rows).set_index("model")
181     stale = [row for row in target_rows if row["target"] == "stale_hops"]
182     summary = {
183         "seed": SEED, "samples": len(x), "features": x.shape[1], "targets": y.shape[1]
    ,
184         "representation_counts": manifest["representation"].value_counts().to_dict(),
185         "calibrated_negatives_invalidated": int(manifest["negative_invalid_count"].sum
    ()),
186         "hybrid_missing_values": int(x.isna().sum().sum()),
187         "validation": "fixed 5-fold shuffled KFold shared with Q1; fold-local median i
    mputation",
188         "identical_q1_folds": identical_q1_folds,
189         "parameters": {"n_estimators": 300, "min_samples_leaf": 3, "max_features": 0.7
    ,
190                        "n_jobs": 1, "imputer": "median_with_missing_indicator"},
191         "metrics": metric_rows, "stale_hops_metrics": stale,
192         "rmse_reduction_vs_mean": float(1 - overall.loc["extra_trees", "rmse"] /
193                                         overall.loc["mean", "rmse"]),
194         "bootstrap": {"repeats": N_BOOTSTRAP, "n_estimators": BOOTSTRAP_TREES,
195                       "target": "stale_hops", "row_resampling": "paired X/y iid with r
    eplacement",
196                       "population": "239 non-Pils/Lager samples; all 11 Pils sources e
    xcluded",
197                       "selection_models": 30, "independent_audit_models": 10,
198                       "audit_protocol": "no reselection after audit",
199                       "paired_difference_quantile": 0.95,
200                       "coverage_calibrated": False,
201                       "audit": "stale_bootstrap_indices.npz and saved bootstrap_indice
    s"},
202         "transfer_proxy": {"train_n": len(transfer_train), "heldout_n": len(transfer_t
    est),
203                            "excluded_style": "Pils/Lager", "metrics": heldout_metrics,
204                            "stale_hops_metrics": [row for row in heldout_targets
205                                                   if row["target"] == "stale_hops"],
206                            "limitations": "11 held-out sources in one style; not a cal
    ibrated transfer test"},
207         "defect_thresholds": DEFECTS,
```

src/06_q3_sensory.py（续）

```
        "limitations": [
            "Targets are attachment-standardized sensory values, not original 0-6 scores.",
            "Random folds assess new samples in the observed style mixture, not unseen-style transfer.",
            "Attachment-level preprocessing and calibration cannot be re-estimated within folds.",
            "Optimization candidates may be out of distribution despite passing chemical constraints.",
            "Stale hops no-increase is a model-proxy constraint, not an actual tasting guarantee.",
            "40 iid row bootstrap models give descriptive paired-model uncertainty; the upper 95% "
            "quantile has neither calibrated coverage nor a physical causal interpretation.",
            "Full-data fitted predictions and bootstrap fitted bands are not validation.",
        ],
        "versions": {"python": platform.python_version(), "numpy": np.__version__,
                     "pandas": pd.__version__, "sklearn": sklearn.__version__},
        "elapsed_seconds": perf_counter() - started,
    }
    (OUT / "summary.json").write_text(
        json.dumps(summary, ensure_ascii=False, indent=2, allow_nan=False), encoding="utf-8", newline="\n")
    print(pd.DataFrame(metric_rows).to_string(index=False), flush=True)
    print("stale_hops:", json.dumps(stale, ensure_ascii=False), flush=True)


if __name__ == "__main__":
    with threadpool_limits(limits=1):
        main()
```

src/07_q3_generation.py

SHA-256: 32ad2b7003ef0525a4cdd5d722f3713697d9fa59a3eed5a82555060e80bc166e

```
"""Run Q3 generation using fixed hyperparameters and repeated held-out validation."""
import os

for _thread_variable in ("OPENBLAS_NUM_THREADS", "OMP_NUM_THREADS", "MKL_NUM_THREADS",
                         "VECLIB_MAXIMUM_THREADS", "NUMEXPR_NUM_THREADS"):
    os.environ[_thread_variable] = "1"

import json
from pathlib import Path
import time

import joblib
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist, pdist
from scipy.stats import rankdata, wasserstein_distance
from sklearn.decomposition import PCA
from sklearn.model_selection import StratifiedKFold
from threadpoolctl import threadpool_limits

from common import SEED, SOLUTION
from q3_copula import ConditionalGaussianCopula
from q3_data import load_q3_data


OUT = SOLUTION / "outputs" / "q3" / "generation"
STYLES = ["Tripel", "Kriek", "Lambic", "Pils/Lager"]
MAIN_PARAMS = (5.0, 20.0)
SENSITIVITY_PARAMS = [(2.0, 5.0), MAIN_PARAMS, (10.0, 50.0)]
REPEAT_SEEDS = [SEED, SEED + 1000]
N_GENERATED = 300
MC_REPEATS = 3
METRICS = ["marginal_w1_iqr", "marginal_w1_iqr_median", "energy_pca5", "spearman_mae",
           "generated_nn_style_median", "holdout_nn_style_median", "nn_ratio",
           "copy_rate", "unique_fraction", "variance_ratio_median",
           "variance_coverage_0p5_2", "style_training_range_fraction"]


def safe_style(style):
    return style.replace("/", "-")


def training_scale(values):
    iqr = np.subtract(*np.percentile(values, [75, 25], axis=0))
    span = np.ptp(values, axis=0)
    scale = np.where(iqr > 1e-12, iqr, np.where(span > 1e-12, span, 1.0))
    return scale, np.where(iqr > 1e-12, "iqr", np.where(span > 1e-12, "range", "one"))


def pca_coordinates(model, x_train):
    z = model.transform(x_train)
    pca = PCA(n_components=min(5, len(z) - 1, z.shape[1]), svd_solver="full").fit(z)
    # Each component is measured in its training standard deviation.
    scale = np.maximum(np.sqrt(pca.explained_variance_), 1e-8)
    return pca, scale


def energy_distance(a, b):
    # Empirical V-statistic; biased upward at tiny held-out n, but same real
```

src/07_q3_generation.py（续）

```
    # rows and same generated n are used for every comparison.
    return float(2 * cdist(a, b).mean() - cdist(a, a).mean() - cdist(b, b).mean())


def rank_correlation(values):
    ranks = rankdata(values, axis=0)
    ranks -= ranks.mean(axis=0)
    norms = np.sqrt(np.sum(ranks ** 2, axis=0))
    standardized = np.divide(ranks, norms, out=np.zeros_like(ranks), where=norms > 0)
    return standardized.T @ standardized


def evaluate(model, x_train, style_train, heldout, generated, pca, pc_scale):
    training = model.impute(x_train)
    reference = model.impute(style_train)
    real = model.impute(heldout)
    fake = model.impute(generated)
    scale, _ = training_scale(training)
    marginal = np.array([wasserstein_distance(real[:, j], fake[:, j]) / scale[j]
                         for j in range(real.shape[1])])
    real_z, fake_z = model.transform(heldout), model.transform(generated)
    reference_z = model.transform(style_train)
    real_pc = pca.transform(real_z) / pc_scale
    fake_pc = pca.transform(fake_z) / pc_scale
    nn_generated = cdist(fake_z, reference_z).min(axis=1) / np.sqrt(real.shape[1])
    nn_real = cdist(real_z, reference_z).min(axis=1) / np.sqrt(real.shape[1])
    valid_corr = (np.ptp(real, axis=0) > 1e-12) & (np.ptp(reference, axis=0) > 1e-12)
    if valid_corr.sum() > 1:
        real_corr = rank_correlation(real[:, valid_corr])
        fake_corr = rank_correlation(fake[:, valid_corr])
        upper = np.triu_indices(len(real_corr), k=1)
        corr_error = float(np.mean(np.abs(real_corr[upper] - fake_corr[upper])))
    else:
        corr_error = float("nan")
    reference_var = np.var(reference, axis=0, ddof=1)
    fake_var = np.var(fake, axis=0, ddof=1)
    variable = reference_var > 1e-20
    ratios = fake_var[variable] / reference_var[variable]
    return {
        "marginal_w1_iqr": float(marginal.mean()),
        "marginal_w1_iqr_median": float(np.median(marginal)),
        "energy_pca5": energy_distance(real_pc, fake_pc),
        "spearman_mae": corr_error,
        "correlation_features": int(valid_corr.sum()),
        "generated_nn_style_median": float(np.median(nn_generated)),
        "holdout_nn_style_median": float(np.median(nn_real)),
        "nn_ratio": float(np.median(nn_generated) / max(np.median(nn_real), 1e-12)),
        "copy_rate": float(np.mean(nn_generated < 1e-12)),
        "unique_fraction": float(len(np.unique(fake, axis=0)) / len(fake)),
        "variance_ratio_median": float(np.median(ratios)),
        "variance_coverage_0p5_2": float(np.mean((ratios >= 0.5) & (ratios <= 2))),
        "variance_reference_features": int(variable.sum()),
        "style_training_range_fraction": float(np.mean(
            (fake >= reference.min(axis=0)) & (fake <= reference.max(axis=0)))),
    }, marginal


def summarize_validation(records):
    # First average Monte Carlo replicates so one data split is one unit in
    # the descriptive fold SD. Repeated folds are dependent: this is not a CI.
    fold = records.groupby(["repeat", "fold", "style", "method"], as_index=False)[METR
```

src/07_q3_generation.py（续）

```
    ICS].mean()
    summary = fold.groupby(["style", "method"])[METRICS].agg(["mean", "std"])
    summary.columns = [f"{metric}_{stat}" for metric, stat in summary.columns]
    return fold, summary.reset_index()


def run_validation(x, styles):
    records, sensitivity, assignments, marginal_records, support_records = [], [], [],
    [], []
    for repeat, split_seed in enumerate(REPEAT_SEEDS, start=1):
        splitter = StratifiedKFold(n_splits=3, shuffle=True, random_state=split_seed)
        for fold, (train, test) in enumerate(splitter.split(x, styles), start=1):
            print(f"Generation validation repeat={repeat}, fold={fold}", flush=True)
            x_train = x.iloc[train]
            train_styles = styles.iloc[train]
            x_test = x.iloc[test]
            test_styles = styles.iloc[test]
            assignments.extend({"repeat": repeat, "fold": fold, "row_index": int(i),
                                "role": "test", "style": str(styles.iloc[i])} for i in
    test)
            for mean_strength, cov_strength in SENSITIVITY_PARAMS:
                model = ConditionalGaussianCopula(mean_strength, cov_strength).fit(x_t
    rain, train_styles)
                pca, pc_scale = pca_coordinates(model, x_train)
                main = (mean_strength, cov_strength) == MAIN_PARAMS
                for style_index, style in enumerate(STYLES):
                    style_train = x_train.loc[train_styles == style]
                    heldout = x_test.loc[test_styles == style]
                    if len(heldout) == 0:
                        raise AssertionError(f"Missing evaluation style {style}")
                    if main:
                        heldout_latent = model.transform(heldout)
                        train_latent = model.transform(style_train)
                        near = cdist(heldout_latent, train_latent).min(axis=1) / np.sq
    rt(x.shape[1])
                        latent_distance = model.distance(heldout, style)
                        support_records.extend({"repeat": repeat, "fold": fold,
                            "row_index": int(i), "style": style,
                            "latent_distance": float(distance), "nn_train_style": floa
    t(nn)}
                            for i, distance, nn in zip(heldout.index, latent_distance,
    near))
                    mc_count = MC_REPEATS if main else 1
                    for mc in range(mc_count):
                        sample_seed = split_seed + fold * 10000 + style_index * 100 +
    mc
                        draws = {"conditional_copula": model.sample(style, N_GENERATED
    , sample_seed)}
                        if main:
                            indices = np.random.default_rng(sample_seed).integers(0, l
    en(style_train), N_GENERATED)
                            draws["style_bootstrap"] = pd.DataFrame(
                                model.impute(style_train)[indices], columns=x.columns)
                            draws["unconditional_copula"] = model.sample(None, N_GENER
    ATED, sample_seed)
                        for method, generated in draws.items():
                            metrics, marginal = evaluate(model, x_train, style_train,
    heldout,
                                                         generated, pca, pc_scale)
                            record = {"repeat": repeat, "fold": fold, "mc": mc,
                                      "split_seed": split_seed, "sample_seed": sample_
```

src/07_q3_generation.py（续）

```
    seed,
                                        "style": style, "method": method,
                                        "train_n": len(x_train), "style_train_n": len(st
yle_train),
                                        "holdout_style_n": len(heldout), "generated_n":
len(generated),
                                        "mean_strength": mean_strength, "cov_strength":
cov_strength,
                                        **metrics}
                                if main:
                                    records.append(record)
                                    if mc == 0:
                                        marginal_records.extend({"repeat": repeat, "fold":
 fold,
                                            "style": style, "method": method, "feature": f
eature,
                                            "w1_iqr": float(value)}
                                            for feature, value in zip(x.columns, marginal)
)
                                else:
                                    sensitivity.append(record)
    return (pd.DataFrame(records), pd.DataFrame(sensitivity),
            pd.DataFrame(assignments), pd.DataFrame(marginal_records), pd.DataFrame(su
pport_records))


def support_diagnostics(model, x, labels):
    z = model.transform(x)
    rows, thresholds = [], []
    for style in sorted(labels.unique()):
        indices = np.flatnonzero(np.asarray(labels) == style)
        style_x = x.iloc[indices]
        distances = model.distance(style_x, style)
        pairwise = cdist(z[indices], z[indices]) / np.sqrt(z.shape[1])
        np.fill_diagonal(pairwise, np.inf)
        nn = pairwise.min(axis=1)
        rows.extend({"row_index": int(i), "style": style,
                     "latent_distance": float(distance),
                     "leave_one_out_nn_style": float(near)}
                    for i, distance, near in zip(indices, distances, nn))
        thresholds.append({"style": style, "n": len(indices),
            "latent_distance_q50": float(np.quantile(distances, 0.5)),
            "latent_distance_q95": float(np.quantile(distances, 0.95)),
            "latent_distance_max": float(distances.max()),
            "loo_nn_style_q50": float(np.quantile(nn, 0.5)),
            "loo_nn_style_q95": float(np.quantile(nn, 0.95)),
            "loo_nn_style_max": float(nn.max()),
            "calibration": "full_fit_in_sample_style_distance_and_loo_nn_descriptive"}
)
    return pd.DataFrame(rows), pd.DataFrame(thresholds)


def generated_diagnostics(model, generated, style, x, labels, support, cv_support):
    fake_z, all_z = model.transform(generated), model.transform(x)
    target_z = all_z[np.asarray(labels) == style]
    all_nn = cdist(fake_z, all_z).min(axis=1) / np.sqrt(fake_z.shape[1])
    target_nn = cdist(fake_z, target_z).min(axis=1) / np.sqrt(fake_z.shape[1])
    distances = model.distance(generated, style)
    thresholds = support.set_index("style").loc[style]
    cv_thresholds = cv_support.set_index("style").loc[style]
    return pd.DataFrame({"sample_id": np.arange(1, len(generated) + 1), "style": style
```

```
    ,
222         "latent_distance": distances, "nn_all": all_nn, "nn_target_style": target_nn,
223         "exact_training_copy": all_nn < 1e-12,
224         "within_latent_style_q95": distances <= thresholds.latent_distance_q95,
225         "within_loo_nn_style_q95": target_nn <= thresholds.loo_nn_style_q95,
226         "within_cv_latent_style_q95": distances <= cv_thresholds.latent_distance_q95,
227         "within_cv_nn_style_q95": target_nn <= cv_thresholds.nn_train_style_q95})
228
229
230 def write_report(summary, sensitivity, full_diagnostics, support, elapsed, x, labels):
231     lines = [
232         "# 第三问：风格条件生成运行报告", "",
233         "运行日期：2026-10-23。", "",
234         "## 数据、模型与固定参数", "",
235         f"使用 {len(x)} 款啤酒、{x.shape[1]} 个混合表示特征及 {labels.nunique()} 个风格。
    "
236         "其中 69 个字段使用附件校准值，157 个使用原文件相对表示；去除 5 个派生汇总列。"
237         "负校准值视为缺失，保留真实零，按训练折中位数补缺。原始附件不作修改。", "",
238         "逐列用训练经验分布中秩映射到正态潜在空间。风格均值以 n/(n+5) 向全局均值收缩；"
239         "协方差以 n/(n+20) 混合风格 Ledoit-Wolf 协方差与全体训练样本去风格均值后的 "
240         "Ledoit-Wolf 共享协方差，另加微小对角项以保证正定。固定参数 (5,20) 在查看验证结果前
    声明，"
241         "不以外层验证选择参数。用 (2,5) 与 (10,50) 单独做敏感性比较。", "",
242         "生成逆映射采用离散经验逆分布，单列取值来自补缺后的训练支持；零及检出限在逆分布中拥有
    "
243         "非零宽度的概率区间。条件潜在分布及高斯近似会改变实际生成的零频率，"
244         "不保证它精确等于全局或目标风格的经验零频率；差异另存 `generated_zero_diagnostics.
    csv`。"
245         "新颖性来自不同列的新组合。连续潜在高斯模型仍只是对点质量依赖关系的近似。"
246         "该方法输出化学目标谱，未包含配料、温度、时间或发酵控制，因此不构成可执行酿造配方。",
     "",
247         "## 无泄漏验证与结果", "",
248         "对全部 22 个风格做 2 次独立随机种子的分层 3 折划分，逐折重拟合补缺、经验分布、"
249         "风格中心和协方差；评价 Tripel、Kriek、Lambic、Pils/Lager。每法每折 300 个生成样本
    , "
250         "3 个 Monte Carlo 种子。比较条件 copula、目标风格训练样本 bootstrap、无条件 copula
    。"
251         "下表先平均同折 Monte Carlo 次数，再对 6 个划分取均值；折间标准差见 CSV，"
252         "由于折及重复间数据有重叠，该标准差不是独立实验置信区间。", "",
253         "|风格|方法|平均边缘 W1↓|边缘W1中位数↓|五维能量距离↓|Spearman差↓|复制率|方差覆盖率|
    ", "|---|---|---:|---:|---:|---:|---:|---:|",
254     ]
255     method_names = {"conditional_copula": "条件copula", "style_bootstrap": "风格bootst
    rap",
256                     "unconditional_copula": "无条件copula"}
257     for _, row in summary.iterrows():
258         lines.append(f"|{row['style']}|{method_names[row['method']]}|"
259             f"{row['marginal_w1_iqr_mean']:.4f}|{row['marginal_w1_iqr_median_mean']:.4
    f}|{row['energy_pca5_mean']:.4f}|"
260             f"{row['spearman_mae_mean']:.4f}|{row['copy_rate_mean']:.1%}|"
261             f"{row['variance_coverage_0p5_2_mean']:.1%}|")
262     lines.extend(["", "边缘 W1 以全部训练行各列 IQR 标准化；零 IQR 依次回退训练极差、常数
    1。"
263         "均值保留了所有极端值的影响，另报跨特征中位数。Kriek 的均值尤其受相对特征 "
264         "`X2.methyltetrahydrofuran.3.one` 的极端值支配，因此不能据平均 W1 的单项改善宣称全
    面优越。"
265         "五维能量距离在训练 Gaussian 空间拟合的 PCA 中计算，各轴以训练标准差缩放，"
266         "使用经验 V 统计量。Spearman 差为测试及风格训练中均非恒定的列之间相关系数的平均绝对
    差，"
267         "生成恒定列的相关记 0。方差覆盖率为生成方差/目标风格训练方差落在 [0.5,2] 的特征比例
    , "
```

src/07_q3_generation.py（续）

```
268         "排除目标训练中恒定列；它衡量风格训练方差再现程度，不代表包含真实总体方差的置信覆盖。", ""])
269     for style in STYLES:
270         table = summary.loc[summary["style"] == style].set_index("method")
271         cond = table.loc["conditional_copula"]
272         boot = table.loc["style_bootstrap"]
273         uncond = table.loc["unconditional_copula"]
274         w1change = (cond.marginal_w1_iqr_mean / boot.marginal_w1_iqr_mean - 1) * 100
275         energychange = (cond.energy_pca5_mean / boot.energy_pca5_mean - 1) * 100
276         lines.append(f"- {style}：条件模型相对风格 bootstrap 的边缘 W1 变化 {w1change:+.1f}%，"
277                      f"能量距离变化 {energychange:+.1f}%；相对无条件模型能量距离变化 "
278                      f"{(cond.energy_pca5_mean / uncond.energy_pca5_mean - 1) * 100:+.1f}%。")
279     lines.extend(["", "bootstrap 直接复制风格训练行，复制率 100%，作为高保真但无新颖性的基线："
280         "不得将条件模型在部分指标劣于 bootstrap 的结果删除。300 是每次模拟样本数，"
281         "并没有将 Kriek 的 10 款真实样本扩充成 300 个独立证据；Kriek/Lambic/Pils-Lager "
282         "每折测试仅约 3–4 款，Tripel 每折约 9–10 款，边缘尾部、相关性与能量距离估计均很不稳定。"
283         "尤其相关矩阵有 226 维，其留出比较只作探索性诊断。需要新增真实啤酒及实测品评验证。", "",
284         "## 全量拟合生成及支持域", "",
285         "在全部数据重新拟合后，为 Tripel、Kriek、Lambic、Pils/Lager 各保存 300 行生成谱；"
286         "输出不是折外性能估计。每行另存相对风格中心的正则 Mahalanobis 距离、"
287         "到所有/目标风格训练啤酒的 Gaussian 均方根近邻距离、完全复制标记。", "",
288         "|风格|完全复制率|距离处于全量训练95%参考内|近邻处于LOO95%参考内|距离处于CV留出95%参考内|", "|---|---:|---:|---:|---:|"])
289     for style, rows in full_diagnostics.groupby("style"):
290         lines.append(f"|{style}|{rows.exact_training_copy.mean():.1%}|"
291                      f"{rows.within_latent_style_q95.mean():.1%}|{rows.within_loo_nn_style_q95.mean():.1%}|"
292                      f"{rows.within_cv_latent_style_q95.mean():.1%}|")
293     lines.extend(["", "支持阈值由全量风格样本的拟合距离 95% 分位数及风格内留一近邻距离 95% 分位数构成，"
294         "用于跨风格候选的描述性筛选。它们不是覆盖率保证或独立校准阈值；中心和距离使用全量拟合，"
295         "小样本阈值有明显不确定性，应同时报告候选的连续距离与是否越界。"
296         "本轮全部生成谱都超过全量训练距离95%阈值，反映高维小样本模型的训练内距离明显乐观。"
297         "另存折外真实样本距离和 `cv_support_thresholds.csv`，用作较宽的对照参考；"
298         "它来自不同折模型，将其用于全量模型候选也没有严格覆盖率保证。"
299         "所有阈值版本均保留，避免在看到通过率后隐藏不利诊断。", "",
300         "## 参数敏感性与复现", "",
301         "敏感性仅用每折相同的第一个 Monte Carlo 种子，比对已预先声明的三对固定参数。"
302         "不得把敏感性最好的外层结果重新报告为无偏的调参后精度。", "",
303         "|均值收缩强度|协方差收缩强度|平均边缘 W1|平均能量距离|", "|---:|---:|---:|---:|"])
304     for _, row in sensitivity.iterrows():
305         lines.append(f"|{row.mean_strength:g}|{row.cov_strength:g}|{row.marginal_w1_iqr:.4f}|{row.energy_pca5:.4f}|")
306     lines.extend(["", f"本次运行耗时约 {elapsed:.1f} 秒，线性代数线程数固定为 1。",
307         "", "```bash", ".venv/bin/python solution_a/src/07_q3_generation.py", "```", "",
308         "模型类接口：`ConditionalGaussianCopula(5,20).fit(X, styles)`；"
309         "`sample(style,n,seed)` 返回同列顺序 DataFrame；`transform(X)` 返回 Gaussian 潜在矩阵；"
310         "`inverse_transform(Z)` 返回离散经验逆映射；`distance(X,style)` 返回按特征数归一的正则 "
311         "Mahalanobis 距离。全量模型保存为 `outputs/q3/generation/fitted_model.joblib`。", "",
312         "主要输出：`cv_metrics.csv`、`cv_summary.csv`、`cv_fold_means.csv`、`cv_mc_vari
```

src/07_q3_generation.py（续）

```
    ability.csv`、"
313         "`cv_assignments.csv`、`cv_marginal_w1_by_feature.csv`、`sensitivity_metrics.c
    sv`、"
314         "`sensitivity_summary.csv`、`generated_*.csv`、`generated_diagnostics.csv`、"
315         "`full_style_support_metrics.csv`、`full_style_support_thresholds.csv`、"
316         "`cv_holdout_support_metrics.csv`、`cv_support_thresholds.csv`、`generated_zer
    o_diagnostics.csv` 与 `summary.json`。", ""])
317     mc_variability = pd.read_csv(OUT / "cv_mc_variability.csv")
318     conditional_mc = mc_variability.loc[mc_variability.method == "conditional_copula",
     "energy_pca5_std"]
319     lines.extend(["条件生成能量距离在固定划分内的 3 次 Monte Carlo 标准差："
320                   f"中位数 {conditional_mc.median():.4f}，最大值 {conditional_mc.max():
    .4f}。"
321                   "它仅反映有限生成次数的随机波动，不覆盖真实风格样本少带来的估计误差。", ""
    ])
322     (SOLUTION / "docs" / "q3_generation.md").write_text("\n".join(lines), encoding="ut
    f-8", newline="\n")
323
324
325 def main():
326     start = time.perf_counter()
327     OUT.mkdir(parents=True, exist_ok=True)
328     meta, x, _, _ = load_q3_data()
329     labels = meta.tasting_category_fine.reset_index(drop=True)
330     x = x.reset_index(drop=True)
331     if min(labels.value_counts()) < 3:
332         raise ValueError("Repeated 3-fold validation requires at least 3 rows per styl
    e")
333     records, sensitivity, assignments, marginal, cv_support_rows = run_validation(x, l
    abels)
334     records.to_csv(OUT / "cv_metrics.csv", index=False, lineterminator="\n")
335     sensitivity.to_csv(OUT / "sensitivity_metrics.csv", index=False, lineterminator="\
    n")
336     assignments.insert(3, "beer", assignments.row_index.map(meta.beer))
337     assignments.to_csv(OUT / "cv_assignments.csv", index=False, lineterminator="\n")
338     marginal.to_csv(OUT / "cv_marginal_w1_by_feature.csv", index=False, lineterminator
    ="\n")
339     cv_support_rows.insert(3, "beer", cv_support_rows.row_index.map(meta.beer))
340     cv_support_rows.to_csv(OUT / "cv_holdout_support_metrics.csv", index=False, linete
    rminator="\n")
341     cv_support = cv_support_rows.groupby("style").agg(
342         reference_rows=("row_index", "size"), unique_beers=("beer", "nunique"),
343         latent_distance_q95=("latent_distance", lambda v: v.quantile(0.95)),
344         nn_train_style_q95=("nn_train_style", lambda v: v.quantile(0.95))).reset_index
    ()
345     cv_support["calibration"] = "repeated_3fold_heldout_descriptive_not_formal_fullfit
    _coverage"
346     cv_support.to_csv(OUT / "cv_support_thresholds.csv", index=False, lineterminator="
    \n")
347     mc_variability = records.groupby(["repeat", "fold", "style", "method"])[METRICS].a
    gg(["mean", "std"])
348     mc_variability.columns = [f"{metric}_{stat}" for metric, stat in mc_variability.co
    lumns]
349     mc_variability.reset_index().to_csv(OUT / "cv_mc_variability.csv", index=False, li
    neterminator="\n")
350     folds, summary = summarize_validation(records)
351     folds.to_csv(OUT / "cv_fold_means.csv", index=False, lineterminator="\n")
352     summary.to_csv(OUT / "cv_summary.csv", index=False, lineterminator="\n")
353     sensitivity_all = pd.concat([sensitivity, records.loc[
354         (records.method == "conditional_copula") & (records.mc == 0)]], ignore_index=T
    rue)
```

src/07_q3_generation.py（续）

```
    sensitivity_summary = sensitivity_all.groupby(["mean_strength", "cov_strength"], as_index=False)[
        ["marginal_w1_iqr", "marginal_w1_iqr_median", "energy_pca5", "spearman_mae", "copy_rate",
         "variance_coverage_0p5_2"]].mean()
    sensitivity_summary.to_csv(OUT / "sensitivity_summary.csv", index=False, lineterminator="\n")
    model = ConditionalGaussianCopula(*MAIN_PARAMS).fit(x, labels)
    joblib.dump(model, OUT / "fitted_model.joblib")
    pd.DataFrame(model.fit_diagnostics_).to_csv(OUT / "fit_diagnostics.csv", index=False, lineterminator="\n")
    _, scale_source = training_scale(model.impute(x))
    pd.DataFrame({"feature": x.columns, "scaling_source": scale_source,
                  "imputation_median": model.median_}).to_csv(OUT / "full_preprocessing.csv", index=False, lineterminator="\n")
    support_rows, support = support_diagnostics(model, x, labels)
    support_rows.insert(1, "beer", support_rows.row_index.map(meta.beer))
    support_rows.to_csv(OUT / "full_style_support_metrics.csv", index=False, lineterminator="\n")
    support.to_csv(OUT / "full_style_support_thresholds.csv", index=False, lineterminator="\n")
    full_diagnostics, zero_diagnostics = [], []
    for index, style in enumerate(STYLES):
        generated = model.sample(style, N_GENERATED, SEED + 100000 + index)
        generated.insert(0, "sample_id", [f"{safe_style(style)}_{i:03d}" for i in range(1, N_GENERATED + 1)])
        generated.insert(1, "target_style", style)
        generated.to_csv(OUT / f"generated_{safe_style(style)}.csv", index=False, lineterminator="\n")
        diag = generated_diagnostics(model, generated[x.columns], style, x, labels, support, cv_support)
        diag["sample_id"] = generated.sample_id
        full_diagnostics.append(diag)
        style_x = x.loc[labels == style]
        imputed_full = model.impute(x)
        imputed_style = model.impute(style_x)
        for j, feature in enumerate(x.columns):
            zero_diagnostics.append({"style": style, "feature": feature,
                "full_observed_zero_fraction": float(x[feature].eq(0).sum() / x[feature].notna().sum()),
                "style_observed_zero_fraction": float(style_x[feature].eq(0).sum() / style_x[feature].notna().sum()),
                "full_imputed_zero_fraction": float(np.mean(imputed_full[:, j] == 0)),
                "style_imputed_zero_fraction": float(np.mean(imputed_style[:, j] == 0)),
                "generated_zero_fraction": float(generated[feature].eq(0).mean())})
    full_diagnostics = pd.concat(full_diagnostics, ignore_index=True)
    full_diagnostics.to_csv(OUT / "generated_diagnostics.csv", index=False, lineterminator="\n")
    pd.DataFrame(zero_diagnostics).to_csv(OUT / "generated_zero_diagnostics.csv", index=False, lineterminator="\n")
    elapsed = time.perf_counter() - start
    summary_json = {
        "date": "2026-10-23",
        "n_real": len(x), "n_features": x.shape[1], "n_styles": labels.nunique(),
        "mean_strength": MAIN_PARAMS[0], "cov_strength": MAIN_PARAMS[1],
        "parameter_policy": "prespecified_fixed_main_and_separate_sensitivity_no_outer_selection",
        "cv_repeat_seeds": REPEAT_SEEDS, "cv_folds": 3, "mc_repeats": MC_REPEATS,
        "generated_n_each_draw": N_GENERATED, "validation_records": len(records),
        "validation_test_style_n_min": int(records.holdout_style_n.min()),
```

src/07_q3_generation.py（续）

```
        "validation_test_style_n_max": int(records.holdout_style_n.max()),
        "full_generation_rows": len(full_diagnostics),
        "full_generation_copy_rate": float(full_diagnostics.exact_training_copy.mean()
),
        "full_generation_within_insample_latent_q95": float(full_diagnostics.within_la
tent_style_q95.mean()),
        "full_generation_within_cv_latent_q95": float(full_diagnostics.within_cv_laten
t_style_q95.mean()),
        "minimum_style_cov_eigenvalue": float(min(d["minimum_cov_eigenvalue"] for d in
 model.fit_diagnostics_)),
        "elapsed_seconds": elapsed,
        "limits": ["Tiny style samples; 300 generated rows are Monte Carlo, not indepe
ndent real evidence",
                   "Shared empirical margins and shrinkage may dilute style-specific d
istributions",
                   "Full-fit support thresholds are descriptive, not independently cal
ibrated",
                   "Chemical target spectra are not brewing recipes or sensory panel v
alidation"],
    }
    (OUT / "summary.json").write_text(json.dumps(summary_json, ensure_ascii=False, ind
ent=2), encoding="utf-8", newline="\n")
    write_report(summary, sensitivity_summary, full_diagnostics, support, elapsed, x,
labels)
    print(json.dumps(summary_json, ensure_ascii=False, indent=2), flush=True)


if __name__ == "__main__":
    with threadpool_limits(limits=1):
        main()
```

src/08_q3_transfer.py

SHA-256: fb99a8dff4c78fd0d705507e65f3a64c9855e18a4462dd2ba53a0df578405cd5

```
"""Constrained Pils/Lager -> Tripel target-spectrum search.

Source beers are excluded from fitted copula and sensory proxy training. The
bootstrap quantiles are model diagnostics, not calibrated outcome guarantees.
"""
from __future__ import annotations

import json

import joblib
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist
from threadpoolctl import threadpool_limits

from common import SEED, SOLUTION
from q3_copula import ConditionalGaussianCopula
from q3_data import load_q3_data, read_defect_thresholds

OUT = SOLUTION / "outputs/q3/transfer"
SOURCE_STYLE = "Pils/Lager"
TARGET_STYLE = "Tripel"
N_GENERATED = 500
LAMBDAS = [0.05, 0.2, 1.0]
DEFAULT_LAMBDA = 0.2
MIN_TARGET_IMPROVEMENT = 0.10
MIN_LATENT_RMS_CHANGE = 0.05
SUPPORT_QUANTILE = 0.95
TOL = 1e-10


def paired_deltas(models, candidates, source):
    return np.column_stack([
        np.asarray(model.predict(candidates)).reshape(-1)
        - float(np.asarray(model.predict(source)).reshape(-1)[0])
        for model in models
    ])


def defect_caps(source, defects, threshold_kind):
    caps = {}
    for name, rule in defects.items():
        value = float(source[rule["feature"]].iloc[0])
        threshold = rule[f"threshold_{threshold_kind}"]
        caps[rule["feature"]] = max(value, float(threshold)) if pd.notna(threshold) el
se value
    return caps


def chemical_check(frame, caps):
    good = np.ones(len(frame), dtype=bool)
    for feature, cap in caps.items():
        values = frame[feature].to_numpy(float)
        good &= np.isfinite(values) & (values >= 0) & (values <= cap + TOL)
    return good


def excess(value, threshold):
    return max(value / threshold - 1, 0) if pd.notna(threshold) else np.nan
```

src/08_q3_transfer.py（续）

```
59
60
61 def main():
62     OUT.mkdir(parents=True, exist_ok=True)
63     meta, x, y, manifest = load_q3_data()
64     source_rows = np.flatnonzero(meta.tasting_category_fine.eq(SOURCE_STYLE))
65     training_rows = np.flatnonzero(~meta.tasting_category_fine.eq(SOURCE_STYLE))
66     defects = read_defect_thresholds()
67     proxy = joblib.load(SOLUTION / "outputs/q3/sensory/transfer_proxy.joblib")
68     ensemble = joblib.load(SOLUTION / "outputs/q3/sensory/stale_bootstrap.joblib")
69     assert proxy["features"] == x.columns.tolist() == ensemble["features"]
70     assert proxy["targets"] == y.columns.tolist()
71     assert len(ensemble["models"]) == 40
72     selection_models, audit_models = ensemble["models"][:30], ensemble["models"][30:]
73     for indices in ensemble["bootstrap_indices"]:
74         assert not np.isin(indices, source_rows).any(), "Source beer entered bootstrap
    training"
75
76     # All eleven transfer source rows are kept out of empirical marginals too.
77     copula = ConditionalGaussianCopula(mean_strength=5.0, cov_strength=20.0)
78     copula.fit(x.iloc[training_rows], meta.tasting_category_fine.iloc[training_rows])
79     target_rows = np.flatnonzero(meta.tasting_category_fine.eq(TARGET_STYLE))
80     target = x.iloc[target_rows]
81     target_z = copula.transform(target)
82     p = x.shape[1]
83     target_pairs = cdist(target_z, target_z) / np.sqrt(p)
84     np.fill_diagonal(target_pairs, np.inf)
85     target_loo_nn = target_pairs.min(axis=1)
86     support_limit = float(np.quantile(target_loo_nn, SUPPORT_QUANTILE))
87     # This is an empirical support rule, not a coverage or production guarantee.
88     center = target_z.mean(axis=0)
89     train_min = x.iloc[training_rows].min().to_numpy()
90     train_max = x.iloc[training_rows].max().to_numpy()
91     base_pool = copula.sample(TARGET_STYLE, N_GENERATED, seed=SEED + 301)
92     target_median = target.median().to_numpy()
93     records, candidate_metrics, chemical_records = [], [], []
94     spectra, profiles, changes, paired_audit_records = [], [], [], []
95     source_spectra = pd.concat([meta.iloc[source_rows].reset_index(drop=True),
96                                 x.iloc[source_rows].reset_index(drop=True)], axis=1)
97     source_spectra.to_csv(OUT / "source_spectra.csv", index=False, lineterminator="\n"
    )
98
99     for source_no, row_index in enumerate(source_rows):
100         beer = int(meta.beer.iloc[row_index])
101         raw_source = x.iloc[[row_index]].copy()
102         complete_defects = all(np.isfinite(raw_source[rule["feature"]].iloc[0])
103                                and raw_source[rule["feature"]].iloc[0] >= 0
104                                for rule in defects.values())
105         if not complete_defects:
106             for weight in LAMBDAS:
107                 records.append({"source_beer": beer, "source_style": SOURCE_STYLE,
108                                 "target_style": TARGET_STYLE, "mode": "protected",
109                                 "status": "source_defect_unverifiable", "lambda": weig
    ht,
110                                 "is_default_policy": weight == DEFAULT_LAMBDA,
111                                 "all_model_checks_passed": False})
112             continue
113         # Other missing features may be filled using training medians; four
114         # protected source values above must always be directly observed.
115         source = raw_source.fillna(x.iloc[training_rows].median())
116         source_z = copula.transform(source)
```

src/08_q3_transfer.py（续）

```
        before_distance = float(copula.distance(source, TARGET_STYLE)[0])
        before_nn = float(cdist(source_z, target_z).min() / np.sqrt(p))
        center_scale = max(float(np.linalg.norm(source_z[0] - center) / np.sqrt(p)), 1e-8)
        caps_median = defect_caps(raw_source, defects, "median")
        caps_mean = defect_caps(raw_source, defects, "mean")
        actual_target_feasible = int(chemical_check(target, caps_median).sum())

        # Fixed search design: conditional samples and interpolation towards them.
        # Same pool for all sources and lambdas; no post-result relaxation.
        source_values = source.to_numpy()[0]
        blocks, descriptions = [], []
        for gamma in [0.25, 0.5, 0.75, 1.0]:
            values = (1 - gamma) * source_values + gamma * base_pool.to_numpy()
            blocks.append(values)
            descriptions.extend(("copula_interpolation", gamma, j) for j in range(len(values)))
        for gamma in [0.25, 0.5, 0.75, 1.0]:
            blocks.append(((1 - gamma) * source_values + gamma * target_median)[None])
            descriptions.append(("target_median_interpolation", gamma, -1))
        unprojected = pd.DataFrame(np.vstack(blocks), columns=x.columns)
        projected = unprojected.copy()
        for feature, cap in caps_median.items():
            projected[feature] = projected[feature].clip(lower=0, upper=cap)

        z = copula.transform(projected)
        distances = copula.distance(projected, TARGET_STYLE)
        nn = cdist(z, target_z).min(axis=1) / np.sqrt(p)
        change = np.linalg.norm(z - source_z, axis=1) / np.sqrt(p)
        marginal_support = ((projected.to_numpy() >= train_min - TOL)
                            & (projected.to_numpy() <= train_max + TOL)).all(axis=1)
        support = (nn <= support_limit + TOL) & marginal_support
        improvement = 1 - distances / max(before_distance, 1e-8)
        chemical_ok = chemical_check(projected, caps_median)
        delta_selection = paired_deltas(selection_models, projected, source)
        selection_q95 = np.quantile(delta_selection, .95, axis=1)
        proxy_ok = selection_q95 <= TOL
        nontrivial = change >= MIN_LATENT_RMS_CHANGE
        target_ok = improvement >= MIN_TARGET_IMPROVEMENT
        eligible = support & nontrivial & target_ok & chemical_ok & proxy_ok
        mean_ok = chemical_check(projected, caps_mean)

        unprojected_z = copula.transform(unprojected)
        unprojected_dist = copula.distance(unprojected, TARGET_STYLE)
        unprojected_change = np.linalg.norm(unprojected_z - source_z, axis=1) / np.sqrt(p)
        unprojected_nn = cdist(unprojected_z, target_z).min(axis=1) / np.sqrt(p)
        unconstrained_support = (unprojected_nn <= support_limit + TOL) & (
            (unprojected.to_numpy() >= train_min - TOL)
            & (unprojected.to_numpy() <= train_max + TOL)).all(axis=1)
        baseline_eligible = (unconstrained_support & (unprojected_change >= MIN_LATENT_RMS_CHANGE)
                             & (1 - unprojected_dist / max(before_distance, 1e-8) >= MIN_TARGET_IMPROVEMENT))
        baseline_chemical = chemical_check(unprojected, caps_median)
        base_source_profile = proxy["pipeline"].predict(source)[0]
        for j in range(len(projected)):
            method, gamma, pool_row = descriptions[j]
            candidate_metrics.append({
                "source_beer": beer, "candidate_index": j, "method": method,
                "gamma": gamma, "pool_row": pool_row,
```

src/08_q3_transfer.py（续）

```
                "target_distance": float(distances[j]), "target_improvement": float(improvement[j]),
                "change_latent_rms": float(change[j]), "target_nn_rms": float(nn[j]),
                "marginal_support": bool(marginal_support[j]), "support_ok": bool(support[j]),
                "chemical_median_ok": bool(chemical_ok[j]), "chemical_mean_ok": bool(mean_ok[j]),
                "selection_paired_delta_q95": float(selection_q95[j]), "selection_proxy_ok": bool(proxy_ok[j]),
                "target_improvement_ok": bool(target_ok[j]), "nontrivial": bool(nontrivial[j]),
                "selection_feasible": bool(eligible[j]),
            })

        for weight in LAMBDAS:
            objective = distances / max(before_distance, 1e-8) + weight * change / center_scale
            baseline_objective = unprojected_dist / max(before_distance, 1e-8) + weight * unprojected_change / center_scale
            choices = [("unconstrained_baseline", baseline_eligible, baseline_objective,
                        unprojected, unprojected_dist, unprojected_nn, unprojected_change),
                       ("protected", eligible, objective, projected, distances, nn, change)]
            for mode, allowed, criterion, pool, pool_dist, pool_nn, pool_change in choices:
                label = f"beer_{beer}_{mode}_lambda_{weight:g}"
                common = {"source_beer": beer, "source_style": SOURCE_STYLE, "target_style": TARGET_STYLE,
                          "mode": mode, "lambda": weight, "is_default_policy": weight == DEFAULT_LAMBDA,
                          "n_candidates": len(pool), "n_selection_feasible": int(allowed.sum()),
                          "n_chemical_feasible": int((chemical_ok if mode == "protected" else baseline_chemical).sum()),
                          "n_support_feasible": int((support if mode == "protected" else unconstrained_support).sum()),
                          "n_real_target_chemical_feasible": actual_target_feasible,
                          "target_distance_before": before_distance, "target_nn_before": before_nn,
                          "support_nn_limit": support_limit,
                          "source_missing_nondefect": int(raw_source.isna().sum().sum()),
                          "transfer_id": label}
                if not allowed.any():
                    records.append({**common, "status": "no_candidate_found", "all_model_checks_passed": False})
                    continue
                candidate_index = int(np.flatnonzero(allowed)[np.argmin(criterion[allowed])])
                selected = pool.iloc[[candidate_index]]
                selection_delta = paired_deltas(selection_models, selected, source)[0]
                audit_delta = paired_deltas(audit_models, selected, source)[0]
                sq95 = float(np.quantile(selection_delta, .95))
                aq95 = float(np.quantile(audit_delta, .95))
                chem_median = bool(chemical_check(selected, caps_median)[0])
                chem_mean = bool(chemical_check(selected, caps_mean)[0])
                audit_ok = aq95 <= TOL
                profile_after = proxy["pipeline"].predict(selected)[0]
                # Audit models do not affect candidate selection. A failed audit
```

src/08_q3_transfer.py（续）

```
                # is retained in outputs; candidates are never reselected using it.
                status = ("model_checks_passed" if audit_ok else "resampling_audit_failed") if mode == "protected" else "baseline_only"
                records.append({**common, "candidate_index": candidate_index,
                                "status": status,
                                "all_model_checks_passed": bool(mode == "protected" and audit_ok and chem_median),
                                "target_distance_after": float(pool_dist[candidate_index]),
                                "target_improvement": float(1 - pool_dist[candidate_index] / max(before_distance, 1e-8)),
                                "target_nn_after": float(pool_nn[candidate_index]),
                                "change_latent_rms": float(pool_change[candidate_index]),
                                "objective": float(criterion[candidate_index]),
                                "chemical_median_ok": chem_median, "chemical_mean_ok": chem_mean,
                                "selection_paired_delta_q95": sq95, "audit_paired_delta_q95": aq95,
                                "audit_proxy_ok": audit_ok,
                                "stale_prediction_before": float(base_source_profile[y.columns.get_loc("stale_hops")]),
                                "stale_prediction_after": float(profile_after[y.columns.get_loc("stale_hops")])})
                spectra.append({"transfer_id": label, "source_beer": beer, "mode": mode,
                                "lambda": weight, **selected.iloc[0].to_dict()})
                for phase, profile in [("source", base_source_profile), ("transferred", profile_after)]:
                    profiles.append({"transfer_id": label, "source_beer": beer, "mode": mode,
                                     "lambda": weight, "phase": phase,
                                     **dict(zip(y.columns, map(float, profile)))})
                for k, deltas in [("selection", selection_delta), ("audit", audit_delta)]:
                    paired_audit_records.extend({"transfer_id": label, "ensemble": k,
                                                 "model_index": b, "paired_delta": float(value)}
                                                for b, value in enumerate(deltas))
                for name, rule in defects.items():
                    feature = rule["feature"]
                    before = float(raw_source[feature].iloc[0])
                    after = float(selected[feature].iloc[0])
                    chemical_records.append({"transfer_id": label, "source_beer": beer,
                                             "mode": mode, "lambda": weight, "defect": name,
                                             "feature": feature, "unit": rule["unit"],
                                             "before": before, "after": after,
                                             "threshold_median": rule["threshold_median"],
                                             "threshold_mean": rule["threshold_mean"],
                                             "median_upper_bound": caps_median[feature],
                                             "mean_upper_bound": caps_mean[feature],
                                             "median_excess_before": excess(before, rule["threshold_median"]),
                                             "median_excess_after": excess(after, rule["threshold_median"]),
                                             "mean_excess_before": excess(before, rule["threshold_mean"]),
```

src/08_q3_transfer.py（续）

```
                                            "mean_excess_after": excess(after, rule["
threshold_mean"]),
                                            "concentration_change": after - before})
                for feature in x.columns:
                    before = float(source[feature].iloc[0])
                    after = float(selected[feature].iloc[0])
                    if abs(after - before) > 1e-12:
                        changes.append({"transfer_id": label, "feature": feature,
                                        "before": before, "after": after, "change": af
ter - before})
        print(f"Transfer {source_no + 1}/{len(source_rows)} beer={beer}: "
              f"{eligible.sum()}/{len(eligible)} selection-feasible candidates", flush
=True)

    tables = {
        "transfer_summary.csv": records, "candidate_diagnostics.csv": candidate_metric
s,
        "transferred_spectra.csv": spectra, "sensory_before_after.csv": profiles,
        "defect_checks.csv": chemical_records, "feature_changes.csv": changes,
        "stale_paired_deltas.csv": paired_audit_records,
    }
    for filename, rows in tables.items():
        pd.DataFrame(rows).to_csv(OUT / filename, index=False, lineterminator="\n")
    result = pd.DataFrame(records)
    default = result[result["is_default_policy"].fillna(False) & result["mode"].eq("pr
otected")]
    n_selected = int(default.status.isin(["model_checks_passed", "resampling_audit_fai
led"]).sum())
    n_checked = int(default.all_model_checks_passed.sum())
    mean_sensitivity = default.get("chemical_mean_ok", pd.Series(False, index=default.
index))
    mean_threshold_all_checks = int((default.all_model_checks_passed
                                     & mean_sensitivity.fillna(False)).sum())
    summary = {
        "seed": SEED, "source_style": SOURCE_STYLE, "target_style": TARGET_STYLE,
        "source_beers": list(map(int, meta.beer.iloc[source_rows])), "n_training_beers
": len(training_rows),
        "source_exclusion": "All Pils/Lager rows excluded from fitted copula, sensory
proxy and bootstrap models",
        "n_base_generated": N_GENERATED, "n_candidates_per_source": N_GENERATED * 4 +
4,
        "lambdas": LAMBDAS, "default_lambda": DEFAULT_LAMBDA,
        "minimum_target_improvement": MIN_TARGET_IMPROVEMENT,
        "minimum_latent_rms_change": MIN_LATENT_RMS_CHANGE,
        "support": {"metric": "full-226-dimensional Gaussian-rank RMS nearest Tripel d
istance",
                    "reference": "Tripel training leave-one-out nearest-neighbor dista
nces",
                    "quantile": SUPPORT_QUANTILE, "limit": support_limit,
                    "additional_rule": "All features within non-source training margin
al support"},
        "constraints": defects, "selection_bootstrap_models": len(selection_models),
        "audit_bootstrap_models": len(audit_models),
        "default_policy": {"n_sources": len(source_rows), "n_candidates_selected": n_s
elected,
                           "n_model_checks_passed": n_checked,
                           "n_audit_failed": int(default.status.eq("resampling_audit_f
ailed").sum()),
                           "n_no_candidate": int(default.status.eq("no_candidate_found
").sum()),
                           "n_source_unverifiable": int(default.status.eq("source_defe
```

src/08_q3_transfer.py（续）

```
    ct_unverifiable").sum()),
299                             "n_all_checks_also_pass_mean_thresholds_same_spectra": mean
    _threshold_all_checks},
300         "limitations": [
301             "Model checks passed does not mean verified sensory or brewing success.",
302             "Bootstrap quantiles after candidate search are uncalibrated model diagnos
    tics, not 95% outcome guarantees.",
303             "The 10 audit models share real training data; separate resampling is not
    an independent experiment.",
304             "No audit-based reselection or post-result relaxation of improvement/suppo
    rt thresholds.",
305             "Trans-2-nonenal has no supplied threshold: its concentration may not incr
    ease.",
306             "The 45 sensory outputs remain on attachment 4 standardized scale, not raw
     0-6.",
307             "Hybrid relative features do not imply measured absolute concentrations or
     a realizable brewing recipe.",
308         ],
309     }
310     (OUT / "summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2
    ), encoding="utf-8", newline="\n")
311     joblib.dump({"copula": copula, "features": x.columns.tolist(),
312                  "training_rows": training_rows, "source_rows": source_rows}, OUT / "t
    ransfer_copula.joblib")
313     pd.DataFrame({"beer": meta.beer.iloc[target_rows].to_numpy(), "loo_nn_rms": target
    _loo_nn}).to_csv(
314         OUT / "target_support_reference.csv", index=False, lineterminator="\n")
315     print(json.dumps(summary["default_policy"], ensure_ascii=False, indent=2), flush=T
    rue)
316
317
318 if __name__ == "__main__":
319     with threadpool_limits(limits=1):
320         main()
```

src/09_q3_report.py

SHA-256: 3621f641847b101cc18abd754d6e7ce9ff6bfb9be162ec0572a66060ec4509d6

```
"""Build Q3 sensory profiles, research figures and a source-backed result note."""
import json

import joblib
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from common import SOLUTION
from q3_data import load_q3_data

OUT = SOLUTION / "outputs/q3"
FIGURES = OUT / "figures"
STYLES = ["Tripel", "Kriek", "Lambic", "Pils/Lager"]
COLORS = {"conditional_copula": "#2876A5", "style_bootstrap": "#9B9FA5",
          "unconditional_copula": "#C88B27"}
STATUS_NAMES = {"model_checks_passed": "通过模型检查",
                "resampling_audit_failed": "重抽样审计未通过",
                "no_candidate_found": "未找到合格候选",
                "source_defect_unverifiable": "源缺陷不可核验"}


def main():
    FIGURES.mkdir(parents=True, exist_ok=True)
    meta, x, y, manifest = load_q3_data()
    generation = OUT / "generation"
    cv = pd.read_csv(generation / "cv_summary.csv")
    gs = json.loads((generation / "summary.json").read_text(encoding="utf-8"))
    ss = json.loads((OUT / "sensory/summary.json").read_text(encoding="utf-8"))
    ts = json.loads((OUT / "transfer/summary.json").read_text(encoding="utf-8"))
    transfer = pd.read_csv(OUT / "transfer/transfer_summary.csv")
    default = transfer[(transfer["mode"] == "protected") & transfer.is_default_policy]
.copy()
    default = default.sort_values("source_beer")
    selected = default[default.candidate_index.notna()]
    diagnostics = pd.read_csv(generation / "generated_diagnostics.csv")
    model = joblib.load(OUT / "sensory/sensory_model.joblib")
    assert model["features"] == x.columns.tolist()
    profiles, summary_profiles = [], []
    for style in STYLES:
        frame = pd.read_csv(generation / f"generated_{style.replace('/', '-')}.csv")
        predicted = model["pipeline"].predict(frame[x.columns])
        diag = diagnostics[diagnostics["style"] == style].reset_index(drop=True)
        profile = pd.DataFrame(predicted, columns=y.columns)
        profile.insert(0, "target_style", style)
        profile.insert(0, "sample_id", frame.sample_id)
        profile["within_loo_nn_style_q95"] = diag.within_loo_nn_style_q95.to_numpy()
        profile["within_cv_latent_style_q95"] = diag.within_cv_latent_style_q95.to_num
py()
        profiles.append(profile)
        for j, target in enumerate(y.columns):
            summary_profiles.append({"style": style, "target": target,
                                     "generated_n": len(profile), "predicted_mean": fl
oat(predicted[:, j].mean()),
                                     "predicted_p05": float(np.quantile(predicted[:, j
], .05)),
                                     "predicted_p95": float(np.quantile(predicted[:, j
```

```
    ], .95)),
56                                      "real_observed_mean": float(y.loc[meta.tasting_ca
    tegory_fine.eq(style), target].mean()),
57                                      "scale": "attachment_4_standardized",
58                                      "interval_meaning": "spread_across_generated_spec
    tra_not_confidence_interval"})
59      pd.concat(profiles, ignore_index=True).to_csv(OUT / "generated_sensory_profiles.cs
    v", index=False, lineterminator="\n")
60      profiles_summary = pd.DataFrame(summary_profiles)
61      profiles_summary.to_csv(OUT / "generated_sensory_summary.csv", index=False, linete
    rminator="\n")
62
63      plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 10,
64                           "axes.spines.top": False, "axes.spines.right": False,
65                           "figure.facecolor": "white", "savefig.facecolor": "white"})
66      fig, ax = plt.subplots(figsize=(10.4, 5.4))
67      positions = np.arange(len(STYLES))
68      for offset, method in zip([-.24, 0, .24], COLORS):
69          rows = cv[cv.method.eq(method)].set_index("style").loc[STYLES]
70          ax.bar(positions + offset, rows.energy_pca5_mean, width=.22,
71                 yerr=rows.energy_pca5_std, capsize=3, color=COLORS[method],
72                 label={"conditional_copula": "Conditional copula", "style_bootstrap": "
    Style bootstrap",
73                        "unconditional_copula": "Unconditional copula"}[method],
74                 error_kw={"elinewidth": 1, "ecolor": "#555555"})
75      ax.set_xticks(positions, STYLES)
76      ax.set_ylabel("Five-dimensional energy discrepancy (V-statistic)")
77      ax.set_title("Style generation: held-out distribution comparison", loc="left", fon
    tsize=14, pad=16)
78      ax.grid(axis="y", color="#E5E7EA", linewidth=.7)
79      ax.set_axisbelow(True)
80      ax.legend(frameon=False, ncol=3, loc="upper center", bbox_to_anchor=(.5, 1.015), f
    ontsize=9)
81      fig.text(.11, .025, "Mean and fold SD over 2 repeats x 3 stratified folds; 3 simul
    ation seeds per fold.\n"
82               "Lower is better. Fold SD is descriptive, not a confidence interval; each
     held-out style has 3-10 real beers.",
83               fontsize=9, color="#555B62")
84      fig.subplots_adjust(left=.11, right=.98, top=.83, bottom=.20)
85      fig.savefig(FIGURES / "generation_validation.png", dpi=300)
86      fig.savefig(FIGURES / "generation_validation.svg")
87      plt.close(fig)
88
89      keys = ["A_malt_all", "F_malt_all", "A_hops_all", "F_hops_all", "A_esters_all",
90              "F_esters_all", "acidity", "bitternes", "sweetness"]
91      means, labels = [], []
92      for style in STYLES[:3]:
93          rows = profiles_summary[profiles_summary["style"] == style].set_index("target"
    ).loc[keys]
94          means.extend([rows.real_observed_mean.to_numpy(), rows.predicted_mean.to_numpy
    ()])
95          labels.extend([f"{style}: observed", f"{style}: generated"])
96      values = np.array(means)
97      fig, ax = plt.subplots(figsize=(11.8, 5.4))
98      limit = max(float(np.max(np.abs(values))), .5)
99      im = ax.imshow(values, cmap="RdBu_r", vmin=-limit, vmax=limit, aspect="auto")
100     ax.set_yticks(np.arange(len(labels)), labels)
101     ax.set_xticks(np.arange(len(keys)), ["Malt\naroma", "Malt\nflavour", "Hops\naroma"
    , "Hops\nflavour",
102                                          "Esters\naroma", "Esters\nflavour", "Acidity",
    "Bitterness", "Sweetness"])
```

src/09_q3_report.py（续）

```
    for (i, j), value in np.ndenumerate(values):
        ax.text(j, i, f"{value:.2f}", ha="center", va="center",
                color="white" if abs(value) > limit * .65 else "#25272B", fontsize=9)
    ax.set_title("Relative sensory fingerprints", loc="left", fontsize=14, pad=14)
    fig.colorbar(im, ax=ax, shrink=.85, label="Attachment-standardized sensory value")
    fig.text(.20, .02, "Generated rows use a fitted sensory predictor; 300 spectra per style. Values are not original 0-6 scores.\n"
             "Observed rows summarize the existing panel data; this figure is descriptive rather than validation.", fontsize=9, color="#555B62")
    fig.subplots_adjust(left=.20, right=.96, top=.86, bottom=.20)
    fig.savefig(FIGURES / "generated_sensory_fingerprints.png", dpi=300)
    fig.savefig(FIGURES / "generated_sensory_fingerprints.svg")
    plt.close(fig)

    fig, axes = plt.subplots(1, 2, figsize=(12.0, 6.2), gridspec_kw={"width_ratios": [1.05, 1]})
    for position, row in enumerate(default.itertuples()):
        axes[0].scatter(row.target_distance_before, position, color="#9B9FA5", marker="o", s=32,
                        label="Source" if position == 0 else None, zorder=3)
        if pd.notna(row.candidate_index):
            color = "#2876A5" if row.all_model_checks_passed else "#C88B27"
            axes[0].plot([row.target_distance_after, row.target_distance_before], [position, position],
                         color=color, linewidth=1.5)
            axes[0].scatter(row.target_distance_after, position, color=color, marker="s", s=30, zorder=3)
        else:
            axes[0].annotate("No candidate", (row.target_distance_before, position),
                             xytext=(-8, -11), textcoords="offset points", ha="right", fontsize=8)
    axes[0].set_yticks(range(len(default)), [f"Beer {b}" for b in default.source_beer])
    axes[0].invert_yaxis()
    axes[0].set_xlabel("Distance to Tripel model centre\n(normalized Mahalanobis)")
    axes[0].set_title("Source and selected target spectrum", loc="left", pad=14)
    axes[0].grid(axis="x", color="#E5E7EA", linewidth=.7)
    axes[0].set_axisbelow(True)
    positions = np.arange(len(selected))
    colors = ["#2876A5" if v else "#C88B27" for v in selected.all_model_checks_passed]
    axes[1].barh(positions, selected.audit_paired_delta_q95, color=colors, height=.65)
    axes[1].scatter(selected.selection_paired_delta_q95, positions, color="#25272B", marker="x", s=35,
                    label="Selection-model upper quantile", zorder=3)
    axes[1].axvline(0, color="#555B62", linewidth=1)
    axes[1].set_yticks(positions, [f"Beer {b}" for b in selected.source_beer])
    axes[1].invert_yaxis()
    axes[1].set_xlabel("Stale hops: paired predicted change\n(upper 0.95 quantile)")
    axes[1].set_title("Resampling audit of selected candidates", loc="left", pad=14)
    axes[1].grid(axis="x", color="#E5E7EA", linewidth=.7)
    axes[1].set_axisbelow(True)
    fig.suptitle("Pils/Lager to Tripel | fixed policy lambda = 0.2", x=.09, ha="left", fontsize=15)
    fig.text(.09, .025, "Bars: audit upper quantile; crosses: selection upper quantile. Blue: model checks passed; gold: audit failed.\n"
             "All 11 sources were excluded from training. The weak Stale hops proxy does not establish actual sensory protection.\n"
             "Only 1 of 11 sources also passes the mean-threshold recheck of the same selected spectra; no reselection.",
             fontsize=9, color="#555B62")
    fig.subplots_adjust(left=.09, right=.98, top=.84, bottom=.24, wspace=.42)
```

src/09_q3_report.py（续）

```
    fig.savefig(FIGURES / "transfer_checks.png", dpi=300)
    fig.savefig(FIGURES / "transfer_checks.svg")
    plt.close(fig)

    # Present the exact metric, not its square root, to match the saved runs.
    comparison_lines = []
    for style in STYLES:
        rows = cv[cv["style"] == style].set_index("method")
        main_row, boot_row = rows.loc["conditional_copula"], rows.loc["style_bootstrap"]
        gain = 100 * (1 - main_row.energy_pca5_mean / boot_row.energy_pca5_mean)
        comparison_lines.append(f"| {style} | {main_row.energy_pca5_mean:.4f} | {boot_row.energy_pca5_mean:.4f} | "
                                f"{rows.loc['unconditional_copula'].energy_pca5_mean:.4f} | {gain:+.1f}% | "
                                f"{main_row.marginal_w1_iqr_median_mean:.4f} / {boot_row.marginal_w1_iqr_median_mean:.4f} |")
    transfer_lines = []
    for row in default.itertuples():
        gain = f"{100 * row.target_improvement:.1f}%" if pd.notna(row.candidate_index) else "—"
        audit = f"{row.audit_paired_delta_q95:+.4f}" if pd.notna(row.candidate_index) else "—"
        mean_pass = "是" if row.all_model_checks_passed and row.chemical_mean_ok == True else "否"
        transfer_lines.append(f"| {row.source_beer} | {STATUS_NAMES[row.status]} | {gain} | {audit} | {mean_pass} |")
    defects = ts["constraints"]
    defect_lines = "\n".join(f"| {name} | `{r['feature']}` | {r['unit']} | {r['threshold_median'] if r['threshold_median'] is not None else '缺失'} | "
                             f"{r['threshold_mean'] if r['threshold_mean'] is not None else '缺失'} |"
                             for name, r in defects.items())
    default_counts = ts["default_policy"]
    q_stale = next(r for r in ss["stale_hops_metrics"] if r["model"] == "extra_trees")
    h_stale = next(r for r in ss["transfer_proxy"]["stale_hops_metrics"] if r["model"] == "transfer_proxy")
    passed = default[default.all_model_checks_passed]
    mean_safe = passed[passed.chemical_mean_ok.eq(True)]
    improvement_range = (f"{passed.target_improvement.min() * 100:.1f}%–{passed.target_improvement.max() * 100:.1f}%"
                         if len(passed) else "未找到通过样本")
    report = f"""# 第三问：风格生成与跨风格迁移实验

日期：2026-10-23。全部数字由保存的模型、谱与验证表计算。复现入口为 `../run_q3.py`。

## 1. 运行结果

已生成 Tripel、Kriek、Lambic、Pils/Lager 各300条谱，共 **{gs['full_generation_rows']}** 条，并给出45维连续感官画像。生成谱的69个校准字段均为非负，未出现完整训练行的直接复制；157个未校准字段保留相对量标记。

对11款Pils/Lager源样本进行Tripel方向迁移。默认λ=0.2、中位阈值口径下，**{default_counts['n_candidates_selected']}**款找到筛选合格候选，**{default_counts['n_model_checks_passed']}**款通过全部模型检查，**{default_counts['n_audit_failed']}**款在保留的重抽样模型审计中未通过，**{default_counts['n_no_candidate']}**款未找到候选。同一选定谱改用均值阈值复核时，仅 **{len(mean_safe)}/11** 仍通过全部检查。

这里的“通过”指本次明确定义的数值与模型检查。Stale hops预测较弱，结果需要实际品评检验后才能判断真实缺陷是否得到控制。
```

src/09_q3_report.py（续）

```
## 2. 统一数据表示

使用226维输入：原1.csv的231维去掉5项派生汇总，其中69维替换为8.xlsx按beer对应的校准原值，剩余157维保留原相对表示。3个负校准观测设为缺失，共72个缺失值；补值在每个训练折内部拟合。原始零值保留，未校准的负值保留原含义。每列来源、表示、单位与缺失记录见 `../outputs/q3/sensory/feature_manifest.csv`。

感官预测器按同一226维表示重新训练，五折折外RMSE为 **0.4865**。画像保持附件4的标准化尺度，尚不能解释成原始0–6分。生成谱的5%–95%预测分位数描述300条谱之间的分布范围，不是预测置信区间。

## 3. 风格条件生成

训练经验分布中秩映射到高斯空间；风格均值以n/(n+5)向全局均值收缩，协方差以n/(n+20)混合风格与共享风格内Ledoit–Wolf协方差。所有矩阵加微小对角正则以保证正定。经验逆CDF生成新的联合组合，单列仍取训练支持中的值。零值在逆CDF中有概率区间，但生成零频率不保证与经验频率完全一致，另有逐字段诊断。

固定主参数(5,20)，单独报告(2,5)、(10,50)敏感性；未用外层结果选择主参数。对22风格分层做2次重复×3折验证，每次比较条件生成、目标风格重采样、无条件生成；每法300条、3个模拟种子。每个风格的真实留出样本只有3–10款，模拟300条不能增加真实证据量。分割重叠，折间标准差属于描述性波动。

能量差异指标在训练折拟合的5维PCA标准化空间中计算：

`E = 2 mean||a-b|| - mean||a-a'|| - mean||b-b'||`。

它是包含对角项的经验V统计量，未开平方；越小越好。W1按训练IQR标准化，零IQR回退到训练range，再回退1。跨特征中位数作为稳健补充，完整均值与逐字段结果同时保存。

| 风格 | 条件生成E | 重采样E | 无条件E | 相对重采样改善 | W1跨特征中位数：条件/重采样 |
|---|---|---|---|---|---|
{chr(10).join(comparison_lines)}

条件模型仅在Tripel的此项指标上优于重采样，其他风格未显示同样优势。重采样复制率为1，条件生成复制率为0，体现新颖性与分布保真性的权衡。Kriek的平均W1被极端化合物值支配，因此不能只凭平均W1判断模型优劣。模型适合提供可检查的候选谱，当前证据不足以宣称所有风格都已获得高保真生成。

相对画像中，Kriek与Lambic的酸度明显向中部收缩，反映小样本条件分布收缩和感官预测误差的共同影响；画像不能取代真实品评。

![生成验证](../outputs/q3/figures/generation_validation.png)

训练内Mahalanobis距离参考较乐观，全部生成谱都超过训练内95%距离分位；采用重复留出真实样本的参考范围时结果明显不同。两种标记均保留在generated_diagnostics.csv，属于描述性参考，不构成已校准的可行性保证。未用事后放宽支持规则来改变迁移结果。

![相对感官画像](../outputs/q3/figures/generated_sensory_fingerprints.png)

## 4. 缺陷保护迁移

全部11款源Pils/Lager排除出迁移Copula、45维感官代理和40个Stale hops重抽样模型训练。迁移训练集239款；源的四项缺陷浓度均有原始校准观测。若缺失，流程返回不可核验，不用插补值充当保护依据。

| 缺陷 | 字段 | 单位 | 中位阈值 | 均值阈值 |
|---|---|---|---|---|
{defect_lines}

有阈值项约束`max(c'/T-1,0) <= max(c_source/T-1,0)`，等价为`c' <= max(T,c_source)`。trans-2-nonenal未提供阈值，采用浓度不得增加的替代约束。4VG按题面列出的保护项执行，不据此推论它在所有风格中都属于缺陷。

每源固定500条条件谱×4个插值比例，加4个目标中位谱插值，共2004个候选；投影后重新计算全部距离及
```

预测。通过条件为：四项化学约束、目标距离至少改善10%、全226维高斯秩空间的目标最近邻距离不超过Tripel训练LOO距离95%分位、所有字段处于非源训练边缘范围、改动RMS至少0.05，以及30个选择模型的配对预测差上0.95分位≤0。

在可行集合内最小化`目标距离/源目标距离 + λ × 改动RMS/源到目标中心RMS`。固定λ为0.05、0.2、1.0，默认0.2；后10个重抽样模型只审计已选谱，不参与重选。两组模型共享真实训练数据，其分位数是模型诊断，既不是独立实验，也没有经过95%覆盖校准。

源beer	默认方案状态	目标距离改善	审计配对差Q95	同谱均值阈值也全部通过
{chr(10).join(transfer_lines)}

通过模型检查的源编号为 **{', '.join(map(str, passed.source_beer))}**，目标距离改善范围 **{improvement_range}**。同谱均值阈值也通过的编号为 **{', '.join(map(str, mean_safe.source_beer)) or '无'}**。DMS均值阈值17.25低于中位阈值19，故阈值口径会改变结果，不能省略这项敏感性。

![迁移检查](../outputs/q3/figures/transfer_checks.png)

5. 适用边界

Stale hops五折R²为 **{q_stale['macro_r2']:.4f}**，RMSE **{q_stale['rmse']:.4f}**；整风格留出11款Pils时R²为 **{h_stale['macro_r2']:.4f}**，RMSE **{h_stale['rmse']:.4f}**。这一关键代理预测能力有限，总体45维预测表现不能替代缺陷目标的验证。候选搜索后的模型上分位约束也不能转化为真实风味不恶化的概率保证。

在28款真实Tripel中，对9个源样本没有真实目标酒同时符合四项化学保护上界，说明目标风格与保护条件存在张力。投影生成的是化学目标谱，尚未给出可实现该谱的配料或发酵控制方案。当前验证不包含生产实验、质量守恒或真实迁移后的品评。

6. 文件与复现

- `../outputs/q3/generation/generated_Tripel.csv`、`generated_Kriek.csv`、`generated_Lambic.csv`、`generated_Pils-Lager.csv`：各300条226维谱。
- `../outputs/q3/generated_sensory_profiles.csv`：全部1200条谱的45维画像；`generated_sensory_summary.csv`含均值、分位与真实风格均值。
- `../outputs/q3/transfer/transferred_spectra.csv`：三种λ下的保护方案及未加缺陷保护的基线谱；失败候选仍保留状态，不作为成功样本。
- `../outputs/q3/transfer/defect_checks.csv`、`sensory_before_after.csv`、`feature_changes.csv`：逐项缺陷、感官与特征改动。
- `../outputs/q3/transfer/candidate_diagnostics.csv`、`stale_paired_deltas.csv`：候选筛选与审计原始证据。
- `q3_generation.md`、`q3_representation.md`、`q3_method_review.md`：模型、数据表示及数值审查；`../outputs/q3/verification.json`为程序一致性检查。

在仓库根目录执行：`.venv/bin/python solution_a/run_q3.py`。本轮参数、源排除与候选判据均随代码保存。

数据来源为赛题附件1/4/8。协方差实现参照 [scikit-learn LedoitWolf文档](https://scikit-learn.org/stable/modules/generated/sklearn.covariance.LedoitWolf.html)，边缘距离使用 [SciPy Wasserstein-1实现](https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.wasserstein_distance.html)。风格收缩、迁移约束及候选审计是本题明确写出的建模选择，未声称由上述文档直接保证其效果。
"""
    (SOLUTION / "docs/q3_results.md").write_text(report, encoding="utf-8", newline="\n")
    print(f"Saved {len(pd.concat(profiles))} sensory profiles, three figures, and docs/q3_results.md")

if __name__ == "__main__":

src/09_q3_report.py（续）

```
271      main()
```

src/10_q3_verify.py

SHA-256: f7ef0ef5732cfbfe61c9332d751840568b11c3742400f4e7e74f741e2f5be563

```
"""Recompute question-3 provenance, saved metrics and transfer constraints."""
from __future__ import annotations

import hashlib
import json
from pathlib import Path
from time import perf_counter

import joblib
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist
from scipy.stats import spearmanr
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import KFold, StratifiedKFold
from threadpoolctl import threadpool_limits

from common import KEYS, SEED, SOLUTION, data_dir, load_xy


OUT = SOLUTION / "outputs" / "q3"
DERIVED = {"kcalperc", "esters_sum", "aroma_hops_sum", "acids_sum", "sulfur_sum"}
DEFECT_FIELDS = ("DMS", "X2.3.butanedione", "X4VG", "X.E..2.nonenal")
GENERATION_STYLES = ("Tripel", "Kriek", "Lambic", "Pils/Lager")
TOL = 1e-10


def close(actual, expected, label="numeric comparison", *, atol=1e-11):
    try:
        np.testing.assert_allclose(np.asarray(actual, dtype=float),
                                   np.asarray(expected, dtype=float),
                                   rtol=1e-9, atol=atol, equal_nan=True)
    except AssertionError as error:
        raise AssertionError(label) from error


def read_json(path):
    return json.loads(Path(path).read_text(encoding="utf-8"))


def score(y, prediction):
    return {
        "mae": float(mean_absolute_error(y, prediction)),
        "rmse": float(np.sqrt(mean_squared_error(y, prediction))),
        "macro_r2": float(r2_score(y, prediction, multioutput="uniform_average")),
    }


def same_table(actual, expected, keys, values, label):
    assert not actual.duplicated(keys).any(), f"Duplicate rows: {label}"
    left = actual.set_index(keys).sort_index()
    right = expected.set_index(keys).sort_index()
    assert left.index.equals(right.index), f"Row keys differ: {label}"
    close(left[values], right[values], label)


def check_hashes():
    table = pd.read_csv(SOLUTION / "outputs/audit/source_files.csv")
    assert len(table) == 9 and table.source.is_unique
```

src/10_q3_verify.py（续）

```
    for row in table.itertuples(index=False):
        raw = (data_dir() / row.source).read_bytes()
        assert len(raw) == row.bytes, f"Source byte count changed: {row.source}"
        assert hashlib.sha256(raw).hexdigest() == row.sha256, f"Source changed: {row.source}"
    return {"files": len(table), "hash_algorithm": "sha256", "unchanged": True}


def load_contract():
    meta, original, y = load_xy()
    calibrated = pd.read_excel(data_dir() / "8.xlsx", sheet_name="Calibrated chemical dataset")
    assert calibrated.beer.is_unique and set(calibrated.beer) == set(meta.beer)
    calibrated = calibrated.set_index("beer").loc[meta.beer].reset_index()
    assert calibrated.tasting_category_fine.equals(meta.tasting_category_fine)
    features = [column for column in original if column not in DERIVED]
    calibrated_features = [column for column in calibrated if column not in set(KEYS) | DERIVED]
    assert len(features) == 226 and len(calibrated_features) == 69
    assert set(calibrated_features).issubset(features)
    expected = original[features].copy()
    negative_count = 0
    for feature in calibrated_features:
        values = calibrated[feature].copy()
        negative_count += int(values.lt(0).sum())
        expected[feature] = values.mask(values.lt(0)).to_numpy()
    assert expected.shape == (250, 226) and y.shape == (250, 45)
    folder = OUT / "sensory"
    stored = pd.read_csv(folder / "hybrid_features.csv")
    assert stored.columns.tolist() == features
    close(stored, expected, "Saved hybrid features differ from original attachments")
    saved_meta = pd.read_csv(folder / "meta.csv", dtype={"beer_id": str})
    pd.testing.assert_frame_equal(saved_meta, meta, check_dtype=False)
    close(pd.read_csv(folder / "y45.csv"), y, "Saved targets differ from attachment 4")
    manifest = pd.read_csv(folder / "feature_manifest.csv")
    assert manifest.feature.tolist() == features and manifest.position.tolist() == list(range(226))
    assert manifest.representation.value_counts().to_dict() == {"relative": 157, "calibrated": 69}
    assert set(manifest.loc[manifest.representation.eq("calibrated"), "feature"]) == set(calibrated_features)
    assert not set(features) & DERIVED
    for row in manifest.itertuples(index=False):
        raw = calibrated[row.feature] if row.feature in calibrated_features else original[row.feature]
        invalid = int(raw.lt(0).sum()) if row.feature in calibrated_features else 0
        assert row.original_missing_count == int(raw.isna().sum())
        assert row.original_1csv_missing_count == int(original[row.feature].isna().sum())
        assert row.negative_invalid_count == invalid
        assert row.final_missing_count == int(expected[row.feature].isna().sum())
        assert row.zero_count == int(expected[row.feature].eq(0).sum())
    assert int(manifest.negative_invalid_count.sum()) == negative_count
    assert not np.isinf(expected.to_numpy()).any()
    calibrated_values = expected[calibrated_features].to_numpy()
    assert (calibrated_values[np.isfinite(calibrated_values)] >= 0).all()
    assert stored.isna().equals(expected.isna())
    context = {"meta": meta, "x": expected, "y": y,
               "features": features, "calibrated_features": calibrated_features}
    return context, {"rows": len(expected), "features": len(features),
```

src/10_q3_verify.py（续）

```
                      "calibrated": len(calibrated_features), "relative": 157,
                      "negative_calibrated_invalidated": negative_count,
                      "missing_cells_preserved": int(expected.isna().sum().sum())}


def supported(values, empirical):
    positions = np.searchsorted(empirical, values)
    left = empirical[np.clip(positions - 1, 0, len(empirical) - 1)]
    right = empirical[np.clip(positions, 0, len(empirical) - 1)]
    return np.isclose(values, left, rtol=1e-9, atol=1e-11) | np.isclose(
        values, right, rtol=1e-9, atol=1e-11)


def check_generation(context):
    x, features = context["x"], context["features"]
    folder = OUT / "generation"
    summary = read_json(folder / "summary.json")
    model = joblib.load(folder / "fitted_model.joblib")
    assert model.columns_ == features and model.n_samples_ == len(x)
    close(model.median_, np.nanmedian(x, axis=0), "Full-fit median differs from traini
ng data")
    training = x.fillna(pd.Series(model.median_, index=features)).to_numpy()
    for j in range(len(features)):
        close(model.empirical_values_[j], np.unique(training[:, j]), "Empirical suppor
t differs from training")
    tables, copies, rows = [], 0, 0
    for style in GENERATION_STYLES:
        frame = pd.read_csv(folder / f"generated_{style.replace('/', '-')}.csv")
        assert len(frame) == 300 and frame.sample_id.is_unique
        assert frame.target_style.eq(style).all()
        assert frame.columns.tolist() == ["sample_id", "target_style"] + features
        fake = frame[features].to_numpy()
        assert np.isfinite(fake).all()
        assert frame[context["calibrated_features"]].ge(0).all().all()
        assert (fake >= training.min(axis=0) - TOL).all()
        assert (fake <= training.max(axis=0) + TOL).all()
        for j in range(len(features)):
            assert supported(fake[:, j], model.empirical_values_[j]).all(), f"Outside
empirical support: {style}/{features[j]}"
        copied = np.any(np.all(np.isclose(fake[:, None, :], training[None, :, :],
                                          rtol=1e-10, atol=1e-12), axis=2), axis=1)
        copies += int(copied.sum())
        rows += len(frame)
        assert not copied.any(), f"Whole training-row copy generated for {style}"
        tables.append(frame)
    generated = pd.concat(tables, ignore_index=True)
    assert generated.sample_id.is_unique and rows == summary["full_generation_rows"]
    close(copies / rows, summary["full_generation_copy_rate"], "Generation copy-rate s
ummary")
    diagnostics = pd.read_csv(folder / "generated_diagnostics.csv")
    assert set(diagnostics.sample_id) == set(generated.sample_id)
    assert not diagnostics.exact_training_copy.any()
    context["generated"] = generated
    return {"styles": list(GENERATION_STYLES), "rows_per_style": 300, "rows": rows,
            "whole_training_row_copies": copies, "calibrated_nonnegative": True,
            "marginal_training_support": True, "discrete_empirical_support": True}


def check_generation_cv(context):
    folder = OUT / "generation"
    meta, x = context["meta"], context["x"]
```

src/10_q3_verify.py（续）

```
    summary = read_json(folder / "summary.json")
    assignments = pd.read_csv(folder / "cv_assignments.csv")
    metrics = pd.read_csv(folder / "cv_metrics.csv")
    assert assignments.role.eq("test").all()
    assert not assignments.duplicated(["repeat", "row_index"]).any()
    assert set(assignments.repeat) == {1, 2} and set(assignments.fold) == {1, 2, 3}
    assert len(assignments) == 2 * len(meta)
    assert (assignments.groupby(["repeat", "row_index"]).size() == 1).all()
    assert assignments.beer.tolist() == assignments.row_index.map(meta.beer).tolist()
    assert assignments["style"].tolist() == assignments.row_index.map(meta.tasting_category_fine).tolist()
    methods = {"conditional_copula", "style_bootstrap", "unconditional_copula"}
    keys = ["repeat", "fold", "mc", "style", "method"]
    assert not metrics.duplicated(keys).any()
    assert set(metrics.method) == methods and set(metrics["style"]) == set(GENERATION_STYLES)
    assert set(metrics.mc) == {0, 1, 2}
    assert len(metrics) == 2 * 3 * 3 * len(GENERATION_STYLES) * len(methods)
    assert (metrics.groupby(["repeat", "fold", "style", "method"]).size() == 3).all()
    assert metrics.generated_n.eq(300).all()
    assert metrics.mean_strength.eq(summary["mean_strength"]).all()
    assert metrics.cov_strength.eq(summary["cov_strength"]).all()
    for repeat, seed in enumerate(summary["cv_repeat_seeds"], 1):
        splitter = StratifiedKFold(3, shuffle=True, random_state=seed)
        for fold, (train, test) in enumerate(splitter.split(x, meta.tasting_category_fine), 1):
            stored = assignments[assignments.repeat.eq(repeat) & assignments.fold.eq(fold)]
            assert set(stored.row_index) == set(test)
            group = metrics[metrics.repeat.eq(repeat) & metrics.fold.eq(fold)]
            assert group.train_n.eq(len(train)).all() and group.split_seed.eq(seed).all()
            for style in GENERATION_STYLES:
                part = group[group["style"].eq(style)]
                assert part.style_train_n.eq(meta.tasting_category_fine.iloc[train].eq(style).sum()).all()
                assert part.holdout_style_n.eq(meta.tasting_category_fine.iloc[test].eq(style).sum()).all()
    fold_table = pd.read_csv(folder / "cv_fold_means.csv")
    fold_keys = ["repeat", "fold", "style", "method"]
    value_columns = [column for column in fold_table if column not in fold_keys]
    assert np.isfinite(metrics[value_columns].to_numpy()).all()
    expected_fold = metrics.groupby(fold_keys, as_index=False)[value_columns].mean()
    same_table(fold_table, expected_fold, fold_keys, value_columns, "CV Monte Carlo fold means")
    expected_summary = expected_fold.groupby(["style", "method"])[value_columns].agg(["mean", "std"])
    expected_summary.columns = [f"{metric}_{stat}" for metric, stat in expected_summary.columns]
    expected_summary = expected_summary.reset_index()
    stored_summary = pd.read_csv(folder / "cv_summary.csv")
    same_table(stored_summary, expected_summary, ["style", "method"],
               [column for column in expected_summary if column not in {"style", "method"}],
               "CV six-split summary")
    assert summary["validation_records"] == len(metrics)
    return {"repeats": 2, "folds": 3, "mc_per_method_fold": 3,
            "every_sample_test_once_per_repeat": True, "metric_records": len(metrics),
            "fold_and_overall_summaries_reconciled": True,
            "scope": "Saved split assignments, schema and aggregation; no claim of independent real experiments"}
```

src/10_q3_verify.py（续）

```
218
219
220 def check_sensory(context):
221     meta, x, y = context["meta"], context["x"], context["y"]
222     folder = OUT / "sensory"
223     overall = pd.read_csv(folder / "metrics_overall.csv")
224     by_target = pd.read_csv(folder / "metrics_by_target.csv")
225     by_fold = pd.read_csv(folder / "metrics_by_fold.csv")
226     expected_folds = np.zeros(len(x), dtype=int)
227     for fold, (_, test) in enumerate(KFold(5, shuffle=True, random_state=SEED).split(x
    ), 1):
228         expected_folds[test] = fold
229     targets, folds = [], []
230     for row in overall.itertuples(index=False):
231         frame = pd.read_csv(folder / f"oof_{row.model}.csv", dtype={"beer_id": str})
232         pd.testing.assert_frame_equal(frame[KEYS], meta, check_dtype=False)
233         np.testing.assert_array_equal(frame.fold, expected_folds)
234         predictions = frame[y.columns].to_numpy()
235         assert np.isfinite(predictions).all()
236         values = score(y, predictions)
237         close([row.mae, row.rmse, row.macro_r2], list(values.values()), f"OOF metrics:
     {row.model}")
238         for j, target in enumerate(y):
239             targets.append({"model": row.model, "target": target,
240                             **score(y.iloc[:, j], predictions[:, j]),
241                             "spearman": float(spearmanr(y.iloc[:, j], predictions[:, j
    ]).statistic)})
242         for fold in range(1, 6):
243             test = expected_folds == fold
244             folds.append({"model": row.model, "fold": fold,
245                           **score(y.loc[test], predictions[test])})
246             if row.model == "mean":
247                 close(predictions[test], np.broadcast_to(y.loc[~test].mean(), predicti
    ons[test].shape),
248                       "Mean OOF baseline must use training-fold targets")
249     same_table(by_target, pd.DataFrame(targets), ["model", "target"],
250                ["mae", "rmse", "macro_r2", "spearman"], "Sensory target OOF metrics")
251     same_table(by_fold, pd.DataFrame(folds), ["model", "fold"],
252                ["mae", "rmse", "macro_r2"], "Sensory fold OOF metrics")
253     sources = np.flatnonzero(meta.tasting_category_fine.eq("Pils/Lager"))
254     train = np.flatnonzero(~meta.tasting_category_fine.eq("Pils/Lager"))
255     proxy = joblib.load(folder / "transfer_proxy.joblib")
256     ensemble = joblib.load(folder / "stale_bootstrap.joblib")
257     np.testing.assert_array_equal(proxy["train_global_indices"], train)
258     np.testing.assert_array_equal(proxy["heldout_global_indices"], sources)
259     np.testing.assert_array_equal(ensemble["train_global_indices"], train)
260     np.testing.assert_array_equal(ensemble["excluded_global_indices"], sources)
261     assert len(train) == 239 and len(sources) == 11 and len(ensemble["models"]) == 40
262     assert proxy["features"] == ensemble["features"] == x.columns.tolist()
263     assert proxy["targets"] == y.columns.tolist()
264     indices = np.asarray(ensemble["bootstrap_indices"])
265     assert indices.shape == (40, len(train)) and np.isin(indices, train).all()
266     assert not np.isin(indices, sources).any()
267     assert ensemble["selection_model_indices"] == list(range(30))
268     assert ensemble["audit_model_indices"] == list(range(30, 40))
269     with np.load(folder / "stale_bootstrap_indices.npz") as saved:
270         np.testing.assert_array_equal(saved["indices"], indices)
271         np.testing.assert_array_equal(saved["source_beer"], meta.beer)
272     heldout_prediction = proxy["pipeline"].predict(x.iloc[sources])
273     heldout = pd.read_csv(folder / "transfer_pils_heldout_predictions.csv")
274     np.testing.assert_array_equal(heldout.beer, meta.beer.iloc[sources])
```

src/10_q3_verify.py（续）

```
    close(heldout[y.columns], heldout_prediction, "Transfer proxy saved source predict
ions")
    heldout_metrics = pd.read_csv(folder / "transfer_pils_heldout_metrics.csv").set_in
dex("model")
    for name, prediction in [("transfer_proxy", heldout_prediction),
                             ("non_pils_mean", np.broadcast_to(y.iloc[train].mean(), h
eldout_prediction.shape))]:
        close(heldout_metrics.loc[name, ["mae", "rmse", "macro_r2"]],
              list(score(y.iloc[sources], prediction).values()), "Held-out Pils sensor
y metrics")
    context.update({"source_rows": sources, "transfer_train": train,
                    "proxy": proxy, "ensemble": ensemble})
    stale = pd.DataFrame(targets)
    stale = stale[stale.target.eq("stale_hops")].set_index("model")
    return {"oof_models": len(overall), "oof_folds": 5, "targets": len(y.columns),
            "excluded_sources": len(sources), "transfer_training_rows": len(train),
            "bootstrap_models": len(ensemble["models"]),
            "bootstrap_resample_shape": list(indices.shape),
            "stale_hops_oof_r2": float(stale.loc["extra_trees", "macro_r2"]),
            "stale_hops_oof_rmse": float(stale.loc["extra_trees", "rmse"]),
            "stale_hops_oof_mean_rmse": float(stale.loc["mean", "rmse"])}


def chemical_caps(source, thresholds, kind):
    result = {}
    for feature in DEFECT_FIELDS:
        value = float(source[feature].iloc[0])
        threshold = thresholds.at[feature, f"threshold_{kind}"]
        result[feature] = max(value, float(threshold)) if pd.notna(threshold) else val
ue
    return result


def chemical_flags(frame, caps):
    values = frame[list(caps)].to_numpy()
    limits = np.array(list(caps.values()))
    return np.isfinite(values).all(axis=1) & (values >= 0).all(axis=1) & (values <= li
mits + TOL).all(axis=1)


def check_transfer(context):
    meta, x, y = context["meta"], context["x"], context["y"]
    source_rows, train = context["source_rows"], context["transfer_train"]
    folder = OUT / "transfer"
    summary = read_json(folder / "summary.json")
    saved = joblib.load(folder / "transfer_copula.joblib")
    copula = saved["copula"]
    np.testing.assert_array_equal(saved["training_rows"], train)
    np.testing.assert_array_equal(saved["source_rows"], source_rows)
    assert copula.columns_ == x.columns.tolist() and copula.n_samples_ == len(train)
    close(copula.median_, np.nanmedian(x.iloc[train], axis=0), "Source-excluded copula
 medians")
    thresholds = pd.read_excel(data_dir() / "8.xlsx", sheet_name="Sensory thresholds")
.set_index("field_name_R")
    assert pd.isna(thresholds.at["X.E..2.nonenal", "threshold_median"])
    target_rows = np.flatnonzero(meta.tasting_category_fine.eq(summary["target_style"]
))
    target = x.iloc[target_rows]
    target_z = copula.transform(target)
    dimensions = x.shape[1]
    target_pairs = cdist(target_z, target_z) / np.sqrt(dimensions)
```

src/10_q3_verify.py（续）

```
    np.fill_diagonal(target_pairs, np.inf)
    loo_nn = target_pairs.min(axis=1)
    support_limit = float(np.quantile(loo_nn, summary["support"]["quantile"]))
    close(support_limit, summary["support"]["limit"], "Transfer target support limit")
    support_csv = pd.read_csv(folder / "target_support_reference.csv")
    np.testing.assert_array_equal(support_csv.beer, meta.beer.iloc[target_rows])
    close(support_csv.loo_nn_rms, loo_nn, "Tripel leave-one-out support reference")
    report = pd.read_csv(folder / "transfer_summary.csv")
    spectra = pd.read_csv(folder / "transferred_spectra.csv").set_index("transfer_id")
    candidates = pd.read_csv(folder / "candidate_diagnostics.csv")
    paired_csv = pd.read_csv(folder / "stale_paired_deltas.csv")
    chemicals = pd.read_csv(folder / "defect_checks.csv")
    profiles = pd.read_csv(folder / "sensory_before_after.csv")
    source_spectra = pd.read_csv(folder / "source_spectra.csv")
    np.testing.assert_array_equal(source_spectra.beer, meta.beer.iloc[source_rows])
    close(source_spectra[x.columns], x.iloc[source_rows], "Transfer raw source spectra
")
    assert not report.duplicated(["source_beer", "mode", "lambda"]).any()
    assert spectra.index.is_unique
    selected_report = report[report.candidate_index.notna()].copy()
    assert set(spectra.index) == set(selected_report.transfer_id)
    assert np.isfinite(spectra[x.columns].to_numpy()).all()
    assert spectra[context["calibrated_features"]].ge(0).all().all()
    assert set(report.source_beer) == set(meta.beer.iloc[source_rows])
    assert set(report["lambda"]) == set(summary["lambdas"])
    assert report.is_default_policy.equals(report["lambda"].eq(summary["default_lambda
"]))
    models = context["ensemble"]["models"]
    source_frame = x.iloc[source_rows].fillna(x.iloc[train].median())
    source_predictions = np.vstack([model.predict(source_frame) for model in models])
    selected_predictions = np.vstack([model.predict(spectra[x.columns]) for model in m
odels])
    source_lookup = {int(meta.beer.iloc[index]): position for position, index in enume
rate(source_rows)}
    center = target_z.mean(axis=0)
    training_min, training_max = x.iloc[train].min().to_numpy(), x.iloc[train].max().t
o_numpy()
    max_quantile_error = 0.0
    max_distance_error = 0.0
    max_nn_error = 0.0
    argmin_checks = 0
    for source_beer, group in candidates.groupby("source_beer"):
        assert len(group) == summary["n_candidates_per_source"]
        assert set(group.candidate_index) == set(range(summary["n_candidates_per_sourc
e"]))
        flags = (group.support_ok & group.nontrivial & group.target_improvement_ok
                 & group.chemical_median_ok & group.selection_proxy_ok)
        assert flags.equals(group.selection_feasible)
        assert group.selection_proxy_ok.equals(group.selection_paired_delta_q95.le(TOL
))
        assert group.nontrivial.equals(group.change_latent_rms.ge(summary["minimum_lat
ent_rms_change"]))
        assert group.target_improvement_ok.equals(group.target_improvement.ge(summary[
"minimum_target_improvement"]))
        assert group.support_ok.equals(group.target_nn_rms.le(support_limit + TOL) & g
roup.marginal_support)
    for _, row in selected_report.iterrows():
        selected = spectra.loc[[row.transfer_id], x.columns]
        source_position = source_lookup[int(row.source_beer)]
        source = source_frame.iloc[[source_position]]
        source_z, selected_z = copula.transform(source), copula.transform(selected)
```

src/10_q3_verify.py（续）

```
        original_source = x.iloc[[source_rows[source_position]]]
        assert original_source[list(DEFECT_FIELDS)].notna().all().all()
        caps_median = chemical_caps(original_source, thresholds, "median")
        caps_mean = chemical_caps(original_source, thresholds, "mean")
        chemical_median = bool(chemical_flags(selected, caps_median)[0])
        chemical_mean = bool(chemical_flags(selected, caps_mean)[0])
        assert chemical_median == row.chemical_median_ok and chemical_mean == row.chem
ical_mean_ok
        position = spectra.index.get_loc(row.transfer_id)
        deltas = selected_predictions[:, position] - source_predictions[:, source_posi
tion]
        selection_q95, audit_q95 = float(np.quantile(deltas[:30], .95)), float(np.quan
tile(deltas[30:], .95))
        max_quantile_error = max(max_quantile_error, abs(selection_q95 - row.selection
_paired_delta_q95),
                                 abs(audit_q95 - row.audit_paired_delta_q95))
        close([selection_q95, audit_q95],
              [row.selection_paired_delta_q95, row.audit_paired_delta_q95], "Selected
paired quantiles")
        assert bool(audit_q95 <= TOL) == row.audit_proxy_ok
        for ensemble_name, values in [("selection", deltas[:30]), ("audit", deltas[30:
])]:
            saved_deltas = paired_csv[paired_csv.transfer_id.eq(row.transfer_id)
                                      & paired_csv.ensemble.eq(ensemble_name)].sort_val
ues("model_index")
            np.testing.assert_array_equal(saved_deltas.model_index, np.arange(len(valu
es)))
            close(saved_deltas.paired_delta, values, "Saved per-model paired differenc
es")
        before = float(copula.distance(source, summary["target_style"])[0])
        after = float(copula.distance(selected, summary["target_style"])[0])
        near = float(cdist(selected_z, target_z).min() / np.sqrt(dimensions))
        change = float(np.linalg.norm(selected_z - source_z) / np.sqrt(dimensions))
        improvement = 1 - after / max(before, 1e-8)
        max_distance_error = max(max_distance_error, abs(after - row.target_distance_a
fter))
        max_nn_error = max(max_nn_error, abs(near - row.target_nn_after))
        close([before, after, near, change, improvement],
              [row.target_distance_before, row.target_distance_after, row.target_nn_af
ter,
               row.change_latent_rms, row.target_improvement], "Selected transfer geom
etry")
        assert near <= support_limit + TOL
        assert (selected.to_numpy() >= training_min - TOL).all()
        assert (selected.to_numpy() <= training_max + TOL).all()
        assert improvement >= summary["minimum_target_improvement"]
        assert change >= summary["minimum_latent_rms_change"]
        scale = max(float(np.linalg.norm(source_z[0] - center) / np.sqrt(dimensions)),
 1e-8)
        close(after / max(before, 1e-8) + row["lambda"] * change / scale, row.objectiv
e,
              "Selected fixed objective")
        if row["mode"] == "protected":
            assert chemical_median and selection_q95 <= TOL
            assert bool(audit_q95 <= TOL) == row.all_model_checks_passed
            expected_status = "model_checks_passed" if audit_q95 <= TOL else "resampli
ng_audit_failed"
            assert row.status == expected_status
            feasible = candidates[candidates.source_beer.eq(row.source_beer) & candida
tes.selection_feasible]
            objective = feasible.target_distance / max(before, 1e-8) + row["lambda"] *
```

src/10_q3_verify.py（续）

```
     feasible.change_latent_rms / scale
            expected_index = int(feasible.iloc[np.argmin(objective)].candidate_index)
            assert int(row.candidate_index) == expected_index, "Audit changed the sele
cted candidate"
            argmin_checks += 1
        else:
            assert row.status == "baseline_only" and not row.all_model_checks_passed
        check_rows = chemicals[chemicals.transfer_id.eq(row.transfer_id)].set_index("f
eature")
        assert set(check_rows.index) == set(DEFECT_FIELDS)
        for feature in DEFECT_FIELDS:
            close([check_rows.at[feature, "before"], check_rows.at[feature, "after"],
                   check_rows.at[feature, "median_upper_bound"], check_rows.at[feature
, "mean_upper_bound"]],
                  [original_source[feature].iloc[0], selected[feature].iloc[0],
                   caps_median[feature], caps_mean[feature]], "Defect concentration au
dit")
            for kind in ["median", "mean"]:
                threshold = thresholds.at[feature, f"threshold_{kind}"]
                for phase, value in [("before", original_source[feature].iloc[0]),
                                     ("after", selected[feature].iloc[0])]:
                    excess = max(value / threshold - 1, 0) if pd.notna(threshold) else
 np.nan
                    close(check_rows.at[feature, f"{kind}_excess_{phase}"], excess, "D
efect excess audit")
        for phase, values in [("source", source), ("transferred", selected)]:
            stored_profile = profiles[profiles.transfer_id.eq(row.transfer_id) & profi
les.phase.eq(phase)]
            assert len(stored_profile) == 1
            close(stored_profile[y.columns], context["proxy"]["pipeline"].predict(valu
es),
                  "Transfer relative sensory profile")
    assert len(paired_csv) == len(spectra) * 40 and len(chemicals) == len(spectra) * 4
    assert len(profiles) == len(spectra) * 2
    for _, row in report[report.candidate_index.isna()].iterrows():
        if row.status == "no_candidate_found":
            assert not candidates[candidates.source_beer.eq(row.source_beer)].selectio
n_feasible.any()
        elif row.status == "source_defect_unverifiable":
            original = x.iloc[[source_rows[source_lookup[int(row.source_beer)]]]][list
(DEFECT_FIELDS)]
            assert original.isna().any().any() or original.lt(0).any().any()
        else:
            raise AssertionError(f"Unrecognized missing-candidate status: {row.status}
")
    default = report[report.is_default_policy & report["mode"].eq("protected")]
    default_counts = {
        "n_sources": len(default),
        "n_candidates_selected": int(default.status.isin(["model_checks_passed", "resa
mpling_audit_failed"]).sum()),
        "n_model_checks_passed": int(default.all_model_checks_passed.sum()),
        "n_audit_failed": int(default.status.eq("resampling_audit_failed").sum()),
        "n_no_candidate": int(default.status.eq("no_candidate_found").sum()),
        "n_source_unverifiable": int(default.status.eq("source_defect_unverifiable").s
um()),
        "n_all_checks_also_pass_mean_thresholds_same_spectra": int(
            (default.all_model_checks_passed & default.chemical_mean_ok.fillna(False))
.sum()),
    }
    assert default_counts == summary["default_policy"], "Default transfer summary diff
ers from CSV"
```

src/10_q3_verify.py（续）

```
    assert len(default) == len(source_rows)
    context["default_transfer_counts"] = default_counts
    return {"selected_spectra_checked": len(spectra), "source_training_rows": len(train),
            "protected_fixed_argmin_checks": argmin_checks,
            "selection_models": 30, "audit_models": 10,
            "maximum_paired_quantile_error": max_quantile_error,
            "maximum_target_distance_error": max_distance_error,
            "maximum_full_space_nn_error": max_nn_error,
            "default_policy": default_counts,
            "scope": "Chemical constraints and uncalibrated model diagnostics; no actual taste guarantee"}


def check_report_artifacts(context):
    path = OUT / "generated_sensory_profiles.csv"
    assert path.is_file(), f"Missing deliverable: {path}"
    report = SOLUTION / "docs/q3_results.md"
    assert report.is_file() and report.stat().st_size > 100, f"Missing or empty deliverable: {report}"
    generated, y = context["generated"], context["y"]
    profiles = pd.read_csv(path)
    assert len(profiles) == len(generated) and profiles.sample_id.is_unique
    assert set(profiles.sample_id) == set(generated.sample_id)
    assert set(y.columns).issubset(profiles.columns)
    final = joblib.load(OUT / "sensory/sensory_model.joblib")
    assert final["features"] == context["features"] and final["targets"] == y.columns.tolist()
    expected = final["pipeline"].predict(generated[context["features"]])
    actual = profiles.set_index("sample_id").loc[generated.sample_id, y.columns]
    assert np.isfinite(actual.to_numpy()).all()
    close(actual, expected, "Generated frozen-model sensory profiles")
    return {"generated_sensory_rows": len(profiles), "sensory_dimensions": len(y.columns),
            "frozen_model_predictions_reconciled": True, "results_report_exists": True}


def main():
    start = perf_counter()
    checks = []
    result = {"status": "running", "checks": checks,
              "scope": "Source provenance, saved-output integrity and independently recomputed numerical consistency"}
    active_check = "source_hashes_before"
    try:
        checks.append({"name": active_check, "status": "passed", "details": check_hashes()})
        active_check = "hybrid_data_contract"
        context, details = load_contract()
        checks.append({"name": active_check, "status": "passed", "details": details})
        for name, operation in [("generation_training_support", check_generation),
                                ("generation_repeated_cv", check_generation_cv),
                                ("sensory_oof_and_source_exclusion", check_sensory),
                                ("transfer_constraints_and_selection", check_transfer),
                                ("report_and_generated_sensory", check_report_artifacts)]:
            active_check = name
            details = operation(context)
            checks.append({"name": name, "status": "passed", "details": details})
```

src/10_q3_verify.py（续）

```
            print(f"Verified: {name}", flush=True)
        active_check = "source_hashes_after"
        checks.append({"name": active_check, "status": "passed", "details": check_hashes()})
        result["status"] = "passed"
        result["limitations"] = [
            "Numerical consistency does not establish model adequacy or physical brewing feasibility.",
            "Bootstrap selection and audit groups share real training data and have no calibrated outcome coverage.",
            "Sensory values are relative attachment-standardized predictions, not recovered raw 0-6 panel scores.",
        ]
    except Exception as error:
        result["status"] = "failed"
        result["failed_check"] = active_check
        result["error"] = f"{type(error).__name__}: {error}"
        raise
    finally:
        result["elapsed_seconds"] = perf_counter() - start
        OUT.mkdir(parents=True, exist_ok=True)
        (OUT / "verification.json").write_text(
            json.dumps(result, ensure_ascii=False, indent=2, allow_nan=False) + "\n", encoding="utf-8", newline="\n")
        print(json.dumps(result, ensure_ascii=False, indent=2, allow_nan=False), flush=True)


if __name__ == "__main__":
    with threadpool_limits(limits=1):
        main()
```

src/11_q3_generation_optimized.py

SHA-256: 4ec959714acc717e509e415faa4401c093c7c3b783edf08bf790c8e5bad498fe

```
"""Evaluate nested, style-margin generation against frozen method baselines."""
import os

for _thread_variable in ("OPENBLAS_NUM_THREADS", "OMP_NUM_THREADS", "MKL_NUM_THREADS",
                         "VECLIB_MAXIMUM_THREADS", "NUMEXPR_NUM_THREADS"):
    os.environ[_thread_variable] = "1"

import importlib
import json
import time

import joblib
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist
from sklearn.model_selection import StratifiedKFold
from threadpoolctl import threadpool_limits

from common import SOLUTION
from q3_data import load_q3_data
from q3_generator_optimized import (CANDIDATE_METHODS, EVALUATION_STYLES,
                                    MAX_INNER_MATCH_FRACTION, MIN_INNER_NN_RATIO,
                                    OptimizedStyleGenerator, SMOOTHING_NOISE)


REFERENCE = importlib.import_module("07_q3_generation")
OUT = SOLUTION / "outputs" / "q3_optimized" / "generation"
OUTER_SEEDS = [20261123, 20262123]
N_GENERATED = 300
MC_REPEATS = 2
METHODS = ["nested_selected", *CANDIDATE_METHODS, "style_bootstrap"]
METRICS = [*REFERENCE.METRICS, "zero_rate_mae", "zero_rate_mae_training_zero_features",
           "zero_rate_mae_calibrated", "nearest_equal_feature_fraction",
           "nearest_equal_feature_fraction_p95", "generated_pair_nn_median",
           "observed_training_copy_rate"]


def extra_metrics(reference, train, heldout, generated, calibrated_columns):
    training = reference.impute(train)
    real = reference.impute(heldout)
    fake = reference.impute(generated)
    zero_error = np.abs(np.mean(real == 0, axis=0) - np.mean(fake == 0, axis=0))
    has_training_zero = np.any(training == 0, axis=0)
    calibrated_indices = [generated.columns.get_loc(c) for c in calibrated_columns]
    fake_z, training_z = reference.transform(generated), reference.transform(train)
    distances = cdist(fake_z, training_z) / np.sqrt(fake_z.shape[1])
    closest = distances.argmin(axis=1)
    equal_fraction = np.isclose(fake, training[closest], rtol=1e-10, atol=1e-12).mean(axis=1)
    pairwise = cdist(fake_z, fake_z) / np.sqrt(fake_z.shape[1])
    np.fill_diagonal(pairwise, np.inf)
    observed = train.to_numpy(dtype=float)
    copied_observed = (np.isclose(fake[:, None, :], observed[None, :, :],
                                  rtol=1e-10, atol=1e-12)
                       | np.isnan(observed)[None, :, :]).all(axis=2).any(axis=1)
    return {"zero_rate_mae": float(zero_error.mean()),
            "zero_rate_mae_training_zero_features": float(zero_error[has_training_zero].mean())
```

src/11_q3_generation_optimized.py（续）

```
            if has_training_zero.any() else 0.0,
            "training_zero_features": int(has_training_zero.sum()),
            "zero_rate_mae_calibrated": float(zero_error[calibrated_indices].mean()),
            "nearest_equal_feature_fraction": float(np.median(equal_fraction)),
            "nearest_equal_feature_fraction_p95": float(np.quantile(equal_fraction, 0.95)),
            "observed_training_copy_rate": float(copied_observed.mean()),
            "generated_pair_nn_median": float(np.median(pairwise.min(axis=1)))}


def fold_summary(records):
    folds = records.groupby(["repeat", "fold", "style", "method"], as_index=False)[METRICS].mean()
    summary = folds.groupby(["style", "method"])[METRICS].agg(["mean", "std"])
    summary.columns = [f"{metric}_{stat}" for metric, stat in summary.columns]
    return folds, summary.reset_index()


def nested_validation(meta, x, styles, calibrated):
    records, assignments, selections, inner_details, margins = [], [], [], [], []
    for repeat, seed in enumerate(OUTER_SEEDS, start=1):
        outer = StratifiedKFold(n_splits=3, shuffle=True, random_state=seed)
        for fold, (train, test) in enumerate(outer.split(x, styles), start=1):
            print(f"Optimized generation repeat={repeat}, fold={fold}", flush=True)
            train_x, train_labels = x.iloc[train], styles.iloc[train]
            selection_seed = seed + fold * 100000
            generator = OptimizedStyleGenerator(selection_seed=selection_seed).fit(train_x, train_labels)
            reference = generator.fitted_candidates_["legacy_copula"]
            pca, pc_scale = REFERENCE.pca_coordinates(reference, train_x)
            for row in generator.selection_table_:
                selections.append({"repeat": repeat, "fold": fold, "selection_seed": selection_seed, **row})
            details = generator.inner_scores_.copy()
            details.insert(0, "outer_repeat", repeat)
            details.insert(1, "outer_fold", fold)
            inner_details.append(details)
            assignments.extend({"repeat": repeat, "fold": fold, "row_index": int(i),
                "beer": meta.beer.iloc[i], "style": styles.iloc[i], "role": "test"} for i in test)
            for style_index, style in enumerate(EVALUATION_STYLES):
                style_train = train_x.loc[train_labels == style]
                heldout = x.iloc[test].loc[styles.iloc[test] == style]
                for mc in range(MC_REPEATS):
                    sample_seed = seed + fold * 10000 + style_index * 100 + mc
                    draws = {method: model.sample(style, N_GENERATED, sample_seed)
                             for method, model in generator.fitted_candidates_.items()}
                    draws["nested_selected"] = draws[generator.selected_methods_[style]]
                    bootstrap_indices = np.random.default_rng(sample_seed).integers(0, len(style_train), N_GENERATED)
                    draws["style_bootstrap"] = pd.DataFrame(reference.impute(style_train)[bootstrap_indices], columns=x.columns)
                    for method in METHODS:
                        generated = draws[method]
                        metrics, marginal = REFERENCE.evaluate(reference, train_x, style_train, heldout,
                                                               generated, pca, pc_scale)
                        records.append({"repeat": repeat, "fold": fold, "style": style
```

src/11_q3_generation_optimized.py（续）

```
    ,
107                                 "method": method, "selected_method": generator.selected_me
    thods_[style]
108                                 if method == "nested_selected" else method,
109                                 "outer_seed": seed, "selection_seed": selection_seed,
110                                 "mc": mc, "sample_seed": sample_seed,
111                                 "train_n": len(train_x), "style_train_n": len(style_train)
    ,
112                                 "holdout_style_n": len(heldout), "generated_n": len(genera
    ted),
113                                 **metrics, **extra_metrics(reference, style_train, heldout
    , generated, calibrated)})
114                             if mc == 0 and method in ("nested_selected", "legacy_copula",
    "style_bootstrap"):
115                                 margins.extend({"repeat": repeat, "fold": fold, "style": s
    tyle,
116                                     "method": method, "feature": feature, "w1_iqr": float(
    value)}
117                                     for feature, value in zip(x.columns, marginal))
118     return (pd.DataFrame(records), pd.DataFrame(assignments), pd.DataFrame(selections)
    ,
119             pd.concat(inner_details, ignore_index=True), pd.DataFrame(margins))
120
121
122 def build_report(summary, selections, full_model, full_diag, elapsed):
123     lines = ["# 第三问生成模型优化：局部风格边缘与嵌套选择", "",
124         "运行日期：2026-10-23。使用 250 款啤酒、226 个混合特征（69 个校准字段与 157 个相对
    字段）。"
125         "旧第三问结果完整保留，本轮文件单独位于 `outputs/q3_optimized/generation/`。", ""
    ,
126         "## 优化针对的问题", "",
127         "旧模型对全部风格共用单列经验边缘，并把小风格均值向全局收缩。Kriek、Lambic "
128         "分别只有 10、11 款真实样本，风格差异可能在这两步被稀释。本轮把风格边缘与相关结构分开
    处理，"
129         "保持小风格的观测边缘，仍用收缩相关矩阵共享信息。", "",
130         "在查看本轮外层结果前固定三个候选：", "",
131         "1. `style_margin_copula`：各风格分别拟合经验边缘和中秩正态分数；局部 Ledoit-Wolf
    "
132         "相关以 n/(n+10) 的权重混合全部训练风格正态分数的共享相关。相关矩阵对角归一到 1，"
133         "从零均值 Gaussian 抽样后用风格局部离散经验逆分布输出。",
134         "2. `style_smoothed_mixture`：使用相同边缘和相关，从风格训练行的正态分数中抽一个中
    心，"
135         "加标准差固定为 0.45 的相关 Gaussian 噪声；该噪声强度固定，不搜索近零扰动。",
136         "3. `legacy_copula`：旧条件 copula 固定参数 (5,20)，作为可被内层选择的保守退路。",
     "",
137         "各风格中位数补缺，风格某列全缺失时回退全部训练行中位数；所有统计量均只用当前训练折。
    "
138         "局部逆分布中的零拥有概率区间，Gaussian 候选在标准正态边缘下按期望再现风格训练零频率
    , "
139         "平滑混合候选不保证相同频率。相对字段中的数值零也可能只是原表示的基准，不能全解释为物
    质不存在。", "",
140         "## 训练内选择与外层检验", "",
141         "外层采用预先固定的 20261123、20262123 两个种子的 22 风格分层 3 折划分。每个外层训
    练集"
142         "再按全部训练风格做分层 2 折，分别为四个评价风格按内层平均五维能量距离选方法。"
143         f"候选必须在每个内折都无整行复制、生成/真实留出近邻距离中位数比至少 {MIN_INNER_NN_R
    ATIO:g}，"
144         f"且与最近训练行相等的特征比例中位数不超过 {MAX_INNER_MATCH_FRACTION:.0%}；"
145         "bootstrap 只作基准，不参与选择。复制检查同时比较完整补缺向量，以及仅比较训练行原本
    有观测"
146         "的各列，避免不同中位数补缺掩盖原观测的整行复制。若无方法通过则退回旧模型，并保留失败
```

```
    诊断。", "",
147         "内层和外层中，同一数据划分的所有候选、被选方法及 bootstrap 均使用同一旧 copula 在该训练折"
148         "拟合的全局 rank 变换、五维 PCA 和 PCA 标准差作评价坐标。没有用各候选自己的局部 CDF 或 PCA "
149         "计算可比较能量距离。每个外层方法每风格生成 300 行，使用两个 Monte Carlo 种子；"
150         "先平均同折的 Monte Carlo，再汇总 6 个划分。", "",
151         "本轮重新划分仍使用同一批已经分析过的 250 款啤酒。嵌套选择避免把本轮外层行直接用于选方法，"
152         "但新种子不等于新增独立验证数据；不能把探索后的新划分称为独立确认性检验。", "",
153         "|风格|方法|五维能量距离↓|平均边缘W1↓|边缘W1中位数↓|校准零频差↓|NN距离比|复制率|方差覆盖|",
154         "|---|---|---:|---:|---:|---:|---:|---:|---:|"]
155     names = {"nested_selected": "训练内选择", "legacy_copula": "旧copula", "style_bootstrap": "风格bootstrap",
156              "style_margin_copula": "局部边缘copula", "style_smoothed_mixture": "平滑混合"}
157     for _, row in summary.iterrows():
158         lines.append(f"|{row['style']}|{names[row['method']]}|{row.energy_pca5_mean:.4f}|"
159             f"{row.marginal_w1_iqr_mean:.4f}|{row.marginal_w1_iqr_median_mean:.4f}|"
160             f"{row.zero_rate_mae_calibrated_mean:.4f}|{row.nn_ratio_mean:.3f}|"
161             f"{row.copy_rate_mean:.1%}|{row.variance_coverage_0p5_2_mean:.1%}|")
162     lines.extend(["", "边缘 W1 各列按外层全部训练行 IQR 标准化，零 IQR 回退训练极差、再回退 1。"
163         "原始极端值仍会支配跨列均值，因此同时保留跨列中位数。能量距离使用经验 V 统计量，"
164         "小留出集下偏高；所有方法的留出行和生成样本数相同。方差覆盖为生成/风格训练方差比处于"
165         "[0.5,2] 的列占比，排除训练恒定列，并非总体方差置信覆盖。Spearman 相关差、"
166         "最近训练行特征匹配比例与生成内近邻距离另存 CSV。", ""])
167     for style in EVALUATION_STYLES:
168         rows = summary.loc[summary['style'] == style].set_index('method')
169         chosen, old, bootstrap = rows.loc['nested_selected'], rows.loc['legacy_copula'], rows.loc['style_bootstrap']
170         change = (chosen.energy_pca5_mean / old.energy_pca5_mean - 1) * 100
171         boot_change = (chosen.energy_pca5_mean / bootstrap.energy_pca5_mean - 1) * 100
172         lines.append(f"- {style}：训练内选择相对旧 copula 的能量距离变化 {change:+.1f}%，"
173                      f"相对 bootstrap 变化 {boot_change:+.1f}%；"
174                      f"边缘 W1 中位数由 {old.marginal_w1_iqr_median_mean:.4f} 变为 {chosen.marginal_w1_iqr_median_mean:.4f}。")
175     lines.extend(["", "## 选择稳定性与新颖性", "",
176         "|风格|各外层训练集所选方法次数|全量训练内选择|", "|---|---|---|"])
177     selected_rows = selections.loc[selections.selected]
178     for style in EVALUATION_STYLES:
179         counts = selected_rows.loc[selected_rows['style'] == style, 'method'].value_counts()
180         count_text = "；".join(f"{names[k]} {v}/6" for k, v in counts.items())
181         lines.append(f"|{style}|{count_text}|{names[full_model.selected_methods_[style]]}|")
182     lines.extend(["", "300 行只增加模拟精度，不增加真实风格证据。低复制率也不证明没有记忆训练分布，"
183         "因此同时报告最近训练行距离、相等特征比例、生成内近邻距离和方差覆盖。"
184         "内层新颖性门槛不能保证外层或全量抽样也通过；下面保留实测结果。", "",
185         "|风格|全量所选方法|完全复制率|NN距离中位数|最近行匹配比例中位数|生成内NN中位数|",
186         "|---|---|---:|---:|---:|---:|"])
187     for style, group in full_diag.groupby('style'):
188         lines.append(f"|{style}|{names[full_model.selected_methods_[style]]}|{group.exact_training_copy.mean():.1%}|"
189                      f"{group.nn_target_style.median():.4f}|{group.equal_fraction_nearest_train.median():.3f}|"
190                      f"{group.nn_generated.median():.4f}|")
```

src/11_q3_generation_optimized.py（续）

```
    lines.extend(["", "## 使用与限制", "",
        "全量模型使用同样的训练内选择，保存为 `fullmodel.joblib`，接口为 "
        "`OptimizedStyleGenerator().fit(X,styles)` 及 `sample(style,n,seed)`，返回原 226 列顺序。"
        "另存四风格各 300 行生成谱（含 sample_id、target_style 元信息）、完整内层与外层记录、"
        "所有候选的外层比较，以及生成零频率诊断。", "",
        "全量 250 行模型只用于生成画像。迁移验证应在去除源风格之后重新拟合，不能复用这个全量"
        "生成器；迁移风格距离仍由旧模型定义，保证新旧比较的量尺一致。", "",
        "风格局部经验边缘不外推，也无法补足未观测的尾部；小风格中每列只有数个取值，"
        "相关与零频率估计不稳定。生成谱是化学目标组合，仍需酿造可行性、理化检测和感官实测验证。", "",
        "```bash", ".venv/bin/python solution_a/src/11_q3_generation_optimized.py", "```", "",
        f"运行耗时约 {elapsed:.1f} 秒，线性代数线程固定为 1。", ""])
    (SOLUTION / 'docs' / 'q3_generation_optimized.md').write_text('\n'.join(lines), encoding='utf-8', newline="\n")


def main():
    started = time.perf_counter()
    OUT.mkdir(parents=True, exist_ok=True)
    meta, x, _, manifest = load_q3_data()
    styles = meta.tasting_category_fine
    calibrated = manifest.loc[manifest.representation == 'calibrated', 'feature'].tolist()
    records, assignments, selections, inner_details, margins = nested_validation(meta, x, styles, calibrated)
    records.to_csv(OUT / 'cv_metrics.csv', index=False, lineterminator="\n")
    assignments.to_csv(OUT / 'cv_assignments.csv', index=False, lineterminator="\n")
    selections.to_csv(OUT / 'inner_selection_outer.csv', index=False, lineterminator="\n")
    inner_details.to_csv(OUT / 'inner_scores_outer.csv', index=False, lineterminator="\n")
    margins.to_csv(OUT / 'cv_marginal_w1_by_feature.csv', index=False, lineterminator="\n")
    folds, summary = fold_summary(records)
    folds.to_csv(OUT / 'cv_fold_means.csv', index=False, lineterminator="\n")
    summary.to_csv(OUT / 'cv_summary.csv', index=False, lineterminator="\n")
    mc = records.groupby(['repeat', 'fold', 'style', 'method'])[METRICS].std().reset_index()
    mc.to_csv(OUT / 'cv_monte_carlo_std.csv', index=False, lineterminator="\n")
    print('Fitting final optimized generator on all 250 rows', flush=True)
    full_model = OptimizedStyleGenerator(selection_seed=OUTER_SEEDS[0]).fit(x, styles)
    joblib.dump(full_model, OUT / 'fullmodel.joblib', compress=3)
    full_model.inner_scores_.to_csv(OUT / 'inner_scores_full.csv', index=False, lineterminator="\n")
    pd.DataFrame(full_model.selection_table_).to_csv(OUT / 'inner_selection_full.csv', index=False, lineterminator="\n")
    pd.DataFrame([{'style': style, 'method': method} for style, method in full_model.selected_methods_.items()]).to_csv(
        OUT / 'full_selected_methods.csv', index=False, lineterminator="\n")
    reference = full_model.fitted_candidates_['legacy_copula']
    full_diags, zero_rows = [], []
    for i, style in enumerate(EVALUATION_STYLES):
        generated = full_model.sample(style, N_GENERATED, OUTER_SEEDS[0] + 900000 + i)
        generated.insert(0, 'sample_id', [f"optimized_{REFERENCE.safe_style(style)}_{j:03d}" for j in range(1, N_GENERATED+1)])
        generated.insert(1, 'target_style', style)
        generated.to_csv(OUT / f'generated_{REFERENCE.safe_style(style)}.csv', index=False, lineterminator="\n")
```

src/11_q3_generation_optimized.py（续）

```
        style_x = x.loc[styles == style]
        generated_x = generated[x.columns]
        fake_z, style_z = reference.transform(generated_x), reference.transform(style_
x)
        pairwise = cdist(fake_z, fake_z) / np.sqrt(x.shape[1])
        np.fill_diagonal(pairwise, np.inf)
        target_distances = cdist(fake_z, style_z) / np.sqrt(x.shape[1])
        nn = target_distances.argmin(axis=1)
        equal = np.isclose(reference.impute(generated_x), reference.impute(style_x)[nn
],
                           rtol=1e-10, atol=1e-12).mean(axis=1)
        raw = style_x.to_numpy(dtype=float)
        observed_copy = (np.isclose(generated_x.to_numpy()[:, None, :], raw[None, :, :
],
                                    rtol=1e-10, atol=1e-12)
                         | np.isnan(raw)[None, :, :]).all(axis=2).any(axis=1)
        full_diags.append(pd.DataFrame({'sample_id': generated.sample_id, 'style': sty
le,
            'method': full_model.selected_methods_[style], 'nn_target_style': target_d
istances.min(axis=1),
            'nn_generated': pairwise.min(axis=1), 'equal_fraction_nearest_train': equa
l,
            'latent_style_distance': reference.distance(generated_x, style),
            'observed_training_copy': observed_copy,
            'exact_training_copy': target_distances.min(axis=1) < 1e-12}))
        for feature in x:
            zero_rows.append({'style': style, 'feature': feature,
                'style_observed_zero_fraction': float(style_x[feature].eq(0).sum() / s
tyle_x[feature].notna().sum()),
                'generated_zero_fraction': float(generated[feature].eq(0).mean())})
    full_diag = pd.concat(full_diags, ignore_index=True)
    full_diag.to_csv(OUT / 'generated_diagnostics.csv', index=False, lineterminator="\
n")
    pd.DataFrame(zero_rows).to_csv(OUT / 'generated_zero_diagnostics.csv', index=False
, lineterminator="\n")
    elapsed = time.perf_counter() - started
    result = {'date': '2026-10-23', 'n_real': len(x), 'n_features': x.shape[1],
        'outer_seeds': OUTER_SEEDS, 'outer_folds': 3, 'inner_folds': 2,
        'mc_repeats': MC_REPEATS, 'generated_n_each_draw': N_GENERATED,
        'candidate_methods': list(CANDIDATE_METHODS), 'smoothing_noise': SMOOTHING_NOI
SE,
        'selection_uses_outer_test': False, 'selection_metric': 'shared_training_refer
ence_pca5_energy',
        'novelty_gates': {'minimum_inner_nn_ratio': MIN_INNER_NN_RATIO,
                          'maximum_inner_nearest_equal_fraction': MAX_INNER_MATCH_FRACT
ION,
                          'maximum_inner_copy_rate': 0.0,
                          'maximum_inner_observed_training_copy_rate': 0.0},
        'cv_records': len(records), 'full_generation_rows': len(full_diag),
        'full_selected_methods': {s: full_model.selected_methods_[s] for s in EVALUATI
ON_STYLES},
        'full_generation_copy_rate': float(full_diag.exact_training_copy.mean()),
        'full_generation_observed_copy_rate': float(full_diag.observed_training_copy.m
ean()),
        'elapsed_seconds': elapsed,
        'limits': ['New outer seeds still reuse the previously explored real dataset',
                   'Small style samples and unstable inner selection',
                   'Local-margin methods use style-imputed support; legacy uses global
 training support',
                   'Chemical spectra are not validated brewing recipes']}
    (OUT / 'summary.json').write_text(json.dumps(result, ensure_ascii=False, indent=2)
```

src/11_q3_generation_optimized.py（续）

```
    , encoding='utf-8', newline="\n")
    build_report(summary, selections, full_model, full_diag, elapsed)
    print(json.dumps(result, ensure_ascii=False, indent=2), flush=True)


if __name__ == '__main__':
    with threadpool_limits(limits=1):
        main()
```

src/12_q3_stale_optimized.py

SHA-256: 56b2e4c89b7616ca459a26b21db244597ffd51d1e2993f13e79121d2cbced3ef

```
"""Repeated nested validation and source-excluded stale-hops transfer proxy."""
from __future__ import annotations

import json
import platform
from time import perf_counter

import joblib
import numpy as np
import pandas as pd
import sklearn
from sklearn.model_selection import KFold
from threadpoolctl import threadpool_limits

from common import SOLUTION, data_dir
from q3_data import load_q3_data
from q3_stale_optimized import (
    AUDIT_BOOTSTRAP_SEED, OUTER_SEEDS, SELECTION_BOOTSTRAP_SEED,
    TRANSFER_CV_SEED, VARIANCE_RATIO_MIN, candidate_specs, choose_candidate,
    inner_compare, make_pipeline, predict_1d, raw_tree_pipeline, scores,
)

OUT = SOLUTION / "outputs" / "q3_optimized" / "stale"
TARGET = "stale_hops"


def write_json(path, value):
    path.write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False), encoding="utf-8", newline="\n")


def main():
    started = perf_counter()
    OUT.mkdir(parents=True, exist_ok=True)
    meta, x, y45, manifest = load_q3_data()
    y = y45[TARGET]
    features = x.columns.tolist()
    calibrated = manifest.loc[manifest.representation.eq("calibrated"), "feature"].tolist()
    metadata = pd.read_excel(data_dir() / "8.xlsx", sheet_name="Sensory thresholds")
    aging = [feature for feature in features
             if feature in set(metadata.loc[metadata.aging.notna(), "field_name_R"])]
    specs = candidate_specs()
    spec_map = {spec["name"]: spec for spec in specs}
    families = sorted({spec["family"] for spec in specs})
    write_json(OUT / "run_manifest.json", {
        "target": TARGET, "features": features, "calibrated_features": calibrated,
        "aging_features": aging, "aging_rule": "8.xlsx Sensory thresholds aging nonmissing; exact field match",
        "candidate_specs": specs, "outer_seeds": list(OUTER_SEEDS), "outer_folds": 5,
        "inner_folds": 3, "inner_objective": "OOF RMSE subject to prediction variance ratio >=0.05",
        "nonconstant_variance_ratio_min": VARIANCE_RATIO_MIN,
        "fallback": "lowest RMSE if no nonconstant candidate; flagged unqualified",
        "outer_outcomes_not_used_for_final_config": True,
        "transfer_cv_seed": TRANSFER_CV_SEED,
        "selection_bootstrap_seed": SELECTION_BOOTSTRAP_SEED,
        "audit_bootstrap_seed": AUDIT_BOOTSTRAP_SEED,
        "forbidden_predictors": ["other observed sensory targets", "attachment 7 comme
```

src/12_q3_stale_optimized.py（续）

```
nts", "identity keys", "style"],
    })
    pd.DataFrame({"feature": aging}).to_csv(OUT / "predeclared_aging_features.csv", index=False, lineterminator="\n")
    all_inner, choices, fold_metrics, oof_rows, metrics_rows = [], [], [], [], []
    methods = ["mean", "old_multioutput_extra_trees", "fixed_single_extra_trees", "nested_selected"]
    methods += [f"nested_family_{family}" for family in families]
    for repeat, outer_seed in enumerate(OUTER_SEEDS, 1):
        predictions = {method: np.full(len(x), np.nan) for method in methods}
        fold_ids = np.zeros(len(x), dtype=int)
        for fold, (train, test) in enumerate(KFold(5, shuffle=True, random_state=outer_seed).split(x), 1):
            inner_seed = outer_seed + 100 * fold
            rows, _ = inner_compare(x.iloc[train], y.iloc[train], specs, features, calibrated, aging, inner_seed)
            for row in rows:
                all_inner.append({"repeat": repeat, "outer_seed": outer_seed, "outer_fold": fold,
                                  "inner_seed": inner_seed, **row})
            selected = {"nested_selected": choose_candidate(rows)}
            selected.update({f"nested_family_{family}": choose_candidate(rows, family) for family in families})
            fitted_cache = {}
            for method, row in selected.items():
                candidate = row["candidate"]
                if candidate not in fitted_cache:
                    fitted_cache[candidate] = make_pipeline(
                        spec_map[candidate], features, calibrated, aging, inner_seed + 90).fit(x.iloc[train], y.iloc[train])
                predictions[method][test] = predict_1d(fitted_cache[candidate], x.iloc[test])
                choices.append({"repeat": repeat, "outer_seed": outer_seed, "outer_fold": fold,
                                "method": method, "selected_candidate": candidate,
                                "inner_oof_rmse": row["rmse"],
                                "inner_oof_variance_ratio": row["prediction_variance_ratio"],
                                "nonconstant_qualified": row["nonconstant_qualified"]})
            old = raw_tree_pipeline(inner_seed + 90).fit(x.iloc[train], y45.iloc[train])
            predictions["old_multioutput_extra_trees"][test] = old.predict(x.iloc[test])[:, y45.columns.get_loc(TARGET)]
            single = raw_tree_pipeline(inner_seed + 90).fit(x.iloc[train], y.iloc[train])
            predictions["fixed_single_extra_trees"][test] = predict_1d(single, x.iloc[test])
            predictions["mean"][test] = float(y.iloc[train].mean())
            fold_ids[test] = fold
            for method in methods:
                fold_metrics.append({"repeat": repeat, "outer_seed": outer_seed, "fold": fold,
                                     "method": method, **scores(y.iloc[test], predictions[method][test])})
            pd.DataFrame(all_inner).to_csv(OUT / "nested_inner_cv_results.csv", index=False, lineterminator="\n")
            pd.DataFrame(choices).to_csv(OUT / "nested_selected_configs.csv", index=False, lineterminator="\n")
            print(f"Stale nested repeat {repeat}/2 fold {fold}/5: {selected['nested_selected']['candidate']}; "
```

src/12_q3_stale_optimized.py（续）

```
                  f"outer RMSE={scores(y.iloc[test], predictions['nested_selected'][test])['rmse']:.6f}", flush=True)
        for method, pred in predictions.items():
            if not np.isfinite(pred).all():
                raise AssertionError(f"Incomplete OOF: repeat={repeat}, {method}")
            metrics_rows.append({"scope": "per_seed", "repeat": repeat, "outer_seed": outer_seed,
                                 "method": method, **scores(y, pred)})
            oof_rows.append(pd.concat([
                meta, pd.DataFrame({"repeat": repeat, "outer_seed": outer_seed, "fold": fold_ids,
                                    "method": method, "observed": y, "prediction": pred})], axis=1))
    oof = pd.concat(oof_rows, ignore_index=True)
    for method in methods:
        rows = oof[oof.method.eq(method)]
        metrics_rows.append({"scope": "pooled_repeated_oof", "repeat": 0, "outer_seed": 0,
                             "method": method, **scores(rows.observed, rows.prediction)})
    metrics = pd.DataFrame(metrics_rows)
    oof.to_csv(OUT / "oof_predictions.csv", index=False, lineterminator="\n")
    metrics.to_csv(OUT / "metrics.csv", index=False, lineterminator="\n")
    pd.DataFrame(fold_metrics).to_csv(OUT / "metrics_by_fold.csv", index=False, lineterminator="\n")
    summary_metrics = metrics[metrics.scope.eq("pooled_repeated_oof")].set_index("method")
    mean_rmse = float(summary_metrics.loc["mean", "rmse"])

    # Only these 239 rows participate in transfer configuration selection.
    excluded = np.flatnonzero(meta.tasting_category_fine.eq("Pils/Lager"))
    transfer_train = np.flatnonzero(~meta.tasting_category_fine.eq("Pils/Lager"))
    if (len(transfer_train), len(excluded)) != (239, 11):
        raise ValueError("Expected all 11 Pils/Lager sources held out")
    transfer_rows, transfer_inner_pred = inner_compare(
        x.iloc[transfer_train], y.iloc[transfer_train], specs, features, calibrated, aging, TRANSFER_CV_SEED)
    pd.DataFrame(transfer_rows).to_csv(OUT / "transfer_inner_cv_results.csv", index=False, lineterminator="\n")
    selected = choose_candidate(transfer_rows)
    selected_spec = spec_map[selected["candidate"]]
    transfer_baseline = np.full(len(transfer_train), np.nan)
    for train_local, test_local in KFold(3, shuffle=True, random_state=TRANSFER_CV_SEED).split(transfer_train):
        transfer_baseline[test_local] = y.iloc[transfer_train[train_local]].mean()
    transfer_baseline_rmse = scores(y.iloc[transfer_train], transfer_baseline)["rmse"]
    qualified = bool(selected["nonconstant_qualified"] and selected["rmse"] < transfer_baseline_rmse)
    final = make_pipeline(selected_spec, features, calibrated, aging, TRANSFER_CV_SEED + 90).fit(
        x.iloc[transfer_train], y.iloc[transfer_train])
    artifact = {"pipeline": final, "features": features, "target": TARGET,
                "target_scale": "attachment_4_standardized", "selected_config": selected_spec,
                "qualified_for_proxy": qualified,
                "inner_oof_variance_ratio": selected["prediction_variance_ratio"],
                "inner_oof_rmse": selected["rmse"], "inner_mean_baseline_rmse": transfer_baseline_rmse,
                "train_global_indices": transfer_train, "excluded_global_indices": excluded,
                "excluded_style": "Pils/Lager", "selection_seed": TRANSFER_CV_SEED,
```

src/12_q3_stale_optimized.py（续）

```
                "qualification_scope": "training_only_inner_CV; not holdout or counterfactual validation",
                "qualification_rule": "training-only inner OOF variance ratio >=0.05 and RMSE < inner mean baseline"}
    joblib.dump(artifact, OUT / "transfer_stale_model.joblib", compress=3)
    pd.concat([meta.iloc[transfer_train].reset_index(drop=True), pd.DataFrame({
        "observed": y.iloc[transfer_train].to_numpy(),
        "selected_inner_oof_prediction": transfer_inner_pred[selected["candidate"]],
        "mean_inner_oof_prediction": transfer_baseline})], axis=1).to_csv(
            OUT / "transfer_selected_inner_oof.csv", index=False, lineterminator="\n")

    # Source holdout is evaluated only after configuration selection/fitting.
    heldout = pd.concat([meta.iloc[excluded].reset_index(drop=True), pd.DataFrame({
        "observed": y.iloc[excluded].to_numpy(), "prediction": predict_1d(final, x.iloc[excluded]),
        "mean_baseline_prediction": y.iloc[transfer_train].mean()})], axis=1)
    heldout.to_csv(OUT / "pils_style_holdout_predictions.csv", index=False, lineterminator="\n")
    holdout_metrics = [{"model": "optimized_single_target", **scores(heldout.observed, heldout.prediction)},
                       {"model": "non_pils_mean", **scores(heldout.observed, heldout.mean_baseline_prediction)}]
    old_path = SOLUTION / "outputs" / "q3" / "sensory" / "transfer_proxy.joblib"
    if old_path.exists():
        old = joblib.load(old_path)
        if set(old["train_global_indices"]) & set(excluded):
            raise AssertionError("Old transfer proxy leaks Pils source rows")
        old_pred = old["pipeline"].predict(x.iloc[excluded])[:, old["targets"].index(TARGET)]
        holdout_metrics.append({"model": "old_multioutput_transfer_proxy", **scores(heldout.observed, old_pred)})
    pd.DataFrame(holdout_metrics).to_csv(OUT / "pils_style_holdout_metrics.csv", index=False, lineterminator="\n")

    models, bootstrap_indices, bootstrap_seeds, estimator_seeds = [], [], [], []
    for b in range(60):
        seed = SELECTION_BOOTSTRAP_SEED + b if b < 40 else AUDIT_BOOTSTRAP_SEED + (b - 40)
        index = np.random.default_rng(seed).choice(transfer_train, size=len(transfer_train), replace=True)
        estimator_seed = seed + 10000
        model = make_pipeline(selected_spec, features, calibrated, aging, estimator_seed).fit(x.iloc[index], y.iloc[index])
        models.append(model)
        bootstrap_indices.append(index)
        bootstrap_seeds.append(seed)
        estimator_seeds.append(estimator_seed)
        if (b + 1) % 10 == 0:
            print(f"Stale optimized bootstrap {b + 1}/60", flush=True)
    ensemble = {"models": models, "bootstrap_indices": bootstrap_indices,
                "features": features, "target": TARGET, "target_scale": "attachment_4_standardized",
                "selected_config": selected_spec, "qualified_for_proxy": qualified,
                "qualification_scope": "training_only_inner_CV; not holdout or counterfactual validation",
                "inner_oof_variance_ratio": selected["prediction_variance_ratio"],
                "inner_oof_rmse": selected["rmse"],
                "train_global_indices": transfer_train, "excluded_global_indices": excluded,
                "selection_model_indices": list(range(40)), "audit_model_indices": list(range(40, 60)),
```

src/12_q3_stale_optimized.py（续）

```
                "bootstrap_seeds": bootstrap_seeds, "estimator_seeds": estimator_seeds
,
                "pairing": "same_model_candidate_minus_source", "paired_quantile": 0.9
5,
                "hyperparameter_uncertainty_included": False,
                "audit_protocol": "40 screening models; 20 separate bootstrap draws fo
r one-time audit; no reselection",
                "interpretation": "model proxy bootstrap quantiles; not calibrated con
fidence bounds"}
    joblib.dump(ensemble, OUT / "bootstrap_ensemble.joblib", compress=3)
    np.savez_compressed(OUT / "bootstrap_indices.npz", indices=np.asarray(bootstrap_in
dices),
                        bootstrap_seeds=np.asarray(bootstrap_seeds), estimator_seeds=n
p.asarray(estimator_seeds))
    assert np.asarray(bootstrap_indices).shape == (60, 239)
    assert all(not set(index) & set(excluded) for index in bootstrap_indices)
    assert set(ensemble["selection_model_indices"]).isdisjoint(ensemble["audit_model_i
ndices"])
    assert np.isfinite(predict_1d(final, x)).all()
    selected_metrics = summary_metrics.loc["nested_selected"].to_dict()
    summary = {
        "samples": len(x), "features": len(features), "target": TARGET,
        "outer_seeds": list(OUTER_SEEDS), "outer_folds": 5, "inner_folds": 3,
        "candidate_count": len(specs), "nonconstant_variance_ratio_min": VARIANCE_RATI
O_MIN,
        "main_result": "nested selection algorithm; not post-hoc best outer family",
        "pooled_repeat_metrics": [{"method": method, **summary_metrics.loc[method].to_
dict()} for method in methods],
        "nested_rmse_reduction_vs_mean": 1 - selected_metrics["rmse"] / mean_rmse,
        "transfer_selected_config": selected_spec, "transfer_qualified_for_proxy": qua
lified,
        "transfer_qualification_scope": "training_only_inner_CV; not holdout or counte
rfactual validation",
        "pils_holdout_improves_mean": holdout_metrics[0]["rmse"] < holdout_metrics[1][
"rmse"],
        "transfer_inner_oof_metrics": selected,
        "transfer_inner_mean_baseline_rmse": transfer_baseline_rmse,
        "pils_holdout_metrics": holdout_metrics,
        "bootstrap": {"models": 60, "selection": 40, "audit": 20, "population": "239 n
on-Pils only",
                      "selected_config_fixed": True, "hyperparameter_uncertainty_inclu
ded": False},
        "limitations": [
            "Only chemical features available to generated profiles are used; no other
 sensory labels or comments are predictors.",
            "The old multi-output comparator uses 45 labels only during its training,
never as prediction features.",
            "Repeated OOF rows are correlated repeats of 250 samples, not 500 independ
ent beers.",
            "The candidate grid and variance gate are fixed; family outer metrics do n
ot select the transfer configuration.",
            "All transfer tuning, preprocessing and bootstrap sampling exclude all 11
Pils/Lager sources.",
            "Pils style holdout has 11 samples in one style and is not used to choose
the model.",
            "Bootstrap fixes the training-only selected configuration and does not inc
lude hyperparameter-selection uncertainty.",
            "Disjoint bootstrap draws share the same observed training data; the audit
 is not independent experimental validation.",
            "Passing paired model-proxy differences cannot guarantee improved real sta
le-hops taste or causal chemical effects.",
```

src/12_q3_stale_optimized.py（续）

```
            "Attachment-standardized target values cannot be interpreted as original 0-6 sensory scores.",
        ],
        "verification": {"bootstrap_source_exclusion": True, "selection_audit_disjoint": True,
                         "final_predictions_finite": True, "bootstrap_indices_shape": [60, 239]},
        "versions": {"python": platform.python_version(), "numpy": np.__version__,
                     "pandas": pd.__version__, "sklearn": sklearn.__version__},
        "elapsed_seconds": perf_counter() - started,
    }
    # Pandas integer dtypes in to_dict are normal Python values here.
    write_json(OUT / "summary.json", summary)
    print(summary_metrics[["rmse", "r2", "spearman", "prediction_variance_ratio"]].to_string(), flush=True)
    print("Transfer chosen:", selected_spec, "qualified:", qualified, flush=True)
    print("Pils holdout:", json.dumps(holdout_metrics, ensure_ascii=False), flush=True)


if __name__ == "__main__":
    with threadpool_limits(limits=1):
        main()
```

src/12b_q3_stale_fixed.py

SHA-256: d7202fbaca1f8f67063ae5084a57a8669a170005c9195d9857033ef88b16cbdc

```
"""Fixed single-target forest follow-up on two additional internal splits.

Parameters are not tuned. This follows exploratory algorithm comparisons on
the same 250 samples and is not an independent external confirmation dataset.
"""
from __future__ import annotations

import json
import platform
from time import perf_counter

import joblib
import numpy as np
import pandas as pd
import sklearn
from sklearn.model_selection import KFold
from threadpoolctl import threadpool_limits

from common import SOLUTION
from q3_data import load_q3_data
from q3_stale_optimized import VARIANCE_RATIO_MIN, predict_1d, raw_tree_pipeline, scor
es

OUT = SOLUTION / "outputs" / "q3_optimized" / "stale_fixed"
TARGET = "stale_hops"
SEEDS = (20263123, 20264123)
TRANSFER_CV_SEED = 20266123
SELECTION_BOOTSTRAP_SEED = 20267123
AUDIT_BOOTSTRAP_SEED = 20268123


def write_json(path, value):
    path.write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False),
encoding="utf-8", newline="\n")


def paired_cluster_bootstrap(oof, repeats=2000, seed=20269123):
    """Resample beers, after averaging each beer's repeated squared errors."""
    errors = oof.assign(squared_error=(oof.prediction-oof.observed)**2)
    per_beer = errors.groupby(["beer", "method"], sort=True).squared_error.mean().unst
ack("method")
    fixed = per_beer["fixed_single_extra_trees"].to_numpy()
    old = per_beer["old_multioutput_extra_trees"].to_numpy()
    draws = np.random.default_rng(seed).integers(0, len(per_beer), size=(repeats, len(
per_beer)))
    fixed_rmse, old_rmse = np.sqrt(fixed[draws].mean(axis=1)), np.sqrt(old[draws].mean
(axis=1))
    difference, reduction = fixed_rmse-old_rmse, 1-fixed_rmse/old_rmse
    return {"seed": seed, "repeats": repeats, "independent_resampling_unit": "beer",
        "beers": len(per_beer), "pre_aggregation": "mean squared error within beer and
 method across two repeats",
        "comparison": "fixed single-target ExtraTrees minus old multi-output ExtraTree
s",
        "point_rmse_difference": float(np.sqrt(fixed.mean())-np.sqrt(old.mean())),
        "rmse_difference_percentile95": np.quantile(difference, [.025, .975]).tolist()
,
        "point_relative_rmse_reduction": float(1-np.sqrt(fixed.mean())/np.sqrt(old.mea
n())),
        "relative_rmse_reduction_percentile95": np.quantile(reduction, [.025, .975]).t
```

src/12b_q3_stale_fixed.py（续）

```
olist(),
        "bootstrap_fraction_rmse_difference_below_zero": float(np.mean(difference<0)),
        "interpretation": "sampling uncertainty conditional on saved OOF predictions;
not a selection-adjusted external-validation interval",
        "limitations": ["Same-data algorithm exploration/selection is not resampled or
 accounted for",
                        "Models are not refitted within these 2000 resamples",
                        "Shared training folds create dependencies beyond the repeated
-beer clustering"]}


def main():
    started = perf_counter()
    OUT.mkdir(parents=True, exist_ok=True)
    meta, x, y45, _ = load_q3_data()
    y = y45[TARGET]
    features = x.columns.tolist()
    selected_config = {"name": "fixed_single_extra_trees_300", "family": "extra_trees"
,
                       "feature_set": "all226", "representation": "raw_hybrid",
                       "params": {"n_estimators": 300, "min_samples_leaf": 3,
                                  "max_features": 0.7, "n_jobs": 1},
                       "bootstrap_n_estimators": 120, "tuning": "none"}
    write_json(OUT / "run_manifest.json", {
        "selected_config": selected_config, "outer_seeds": list(SEEDS), "outer_folds":
 5,
        "transfer_cv_seed": TRANSFER_CV_SEED, "selection_bootstrap_seed": SELECTION_BO
OTSTRAP_SEED,
        "audit_bootstrap_seed": AUDIT_BOOTSTRAP_SEED, "features": features,
        "preprocessing": "raw hybrid; training-only median imputation with missing ind
icators",
        "status": "fixed follow-up after exploratory algorithm comparison on the same
data",
        "not_independent_external_confirmation": True, "parameter_tuning": False,
    })
    methods = ["mean", "old_multioutput_extra_trees", "fixed_single_extra_trees"]
    oof_rows, metric_rows, fold_rows = [], [], []
    stale_position = y45.columns.get_loc(TARGET)
    for repeat, seed in enumerate(SEEDS, 1):
        predictions = {name: np.full(len(x), np.nan) for name in methods}
        folds = np.zeros(len(x), dtype=int)
        for fold, (train, test) in enumerate(KFold(5, shuffle=True, random_state=seed)
.split(x), 1):
            model_seed = seed + fold
            old = raw_tree_pipeline(model_seed).fit(x.iloc[train], y45.iloc[train])
            single = raw_tree_pipeline(model_seed).fit(x.iloc[train], y.iloc[train])
            predictions["old_multioutput_extra_trees"][test] = old.predict(x.iloc[test
])[:, stale_position]
            predictions["fixed_single_extra_trees"][test] = predict_1d(single, x.iloc[
test])
            predictions["mean"][test] = y.iloc[train].mean()
            folds[test] = fold
            for method in methods:
                fold_rows.append({"repeat": repeat, "outer_seed": seed, "fold": fold,
                                  "method": method, **scores(y.iloc[test], predictions
[method][test])})
            print(f"Stale fixed repeat {repeat}/2 fold {fold}/5: "
                  f"single RMSE={scores(y.iloc[test], predictions['fixed_single_extra_
trees'][test])['rmse']:.6f}", flush=True)
        for method, pred in predictions.items():
            assert np.isfinite(pred).all()
```

src/12b_q3_stale_fixed.py（续）

```
            metric_rows.append({"scope": "per_seed", "repeat": repeat, "outer_seed": seed,
                                "method": method, **scores(y, pred)})
            oof_rows.append(pd.concat([meta, pd.DataFrame({"repeat": repeat, "outer_seed": seed,
                "fold": folds, "method": method, "observed": y, "prediction": pred})], axis=1))
    oof = pd.concat(oof_rows, ignore_index=True)
    for method in methods:
        group = oof[oof.method.eq(method)]
        metric_rows.append({"scope": "pooled_repeated_oof", "repeat": 0, "outer_seed": 0,
                            "method": method, **scores(group.observed, group.prediction)})
    metrics = pd.DataFrame(metric_rows)
    oof.to_csv(OUT / "oof_predictions.csv", index=False, lineterminator="\n")
    metrics.to_csv(OUT / "metrics.csv", index=False, lineterminator="\n")
    pd.DataFrame(fold_rows).to_csv(OUT / "metrics_by_fold.csv", index=False, lineterminator="\n")
    cluster_result = paired_cluster_bootstrap(oof)
    write_json(OUT / "paired_cluster_bootstrap.json", cluster_result)
    pooled = metrics[metrics.scope.eq("pooled_repeated_oof")].set_index("method")

    excluded = np.flatnonzero(meta.tasting_category_fine.eq("Pils/Lager"))
    train_global = np.flatnonzero(~meta.tasting_category_fine.eq("Pils/Lager"))
    assert len(excluded) == 11 and len(train_global) == 239
    train_oof, mean_oof = np.full(len(train_global), np.nan), np.full(len(train_global), np.nan)
    for fold, (tr, te) in enumerate(KFold(3, shuffle=True, random_state=TRANSFER_CV_SEED).split(train_global), 1):
        model = raw_tree_pipeline(TRANSFER_CV_SEED + fold).fit(x.iloc[train_global[tr]], y.iloc[train_global[tr]])
        train_oof[te] = predict_1d(model, x.iloc[train_global[te]])
        mean_oof[te] = y.iloc[train_global[tr]].mean()
    train_metrics, mean_metrics = scores(y.iloc[train_global], train_oof), scores(y.iloc[train_global], mean_oof)
    qualified = bool(train_metrics["prediction_variance_ratio"] >= VARIANCE_RATIO_MIN
                     and train_metrics["rmse"] < mean_metrics["rmse"])
    final = raw_tree_pipeline(TRANSFER_CV_SEED + 90).fit(x.iloc[train_global], y.iloc[train_global])
    common_metadata = {"features": features, "target": TARGET,
        "target_scale": "attachment_4_standardized", "selected_config": selected_config,
        "qualified_for_proxy": qualified, "inner_oof_variance_ratio": train_metrics["prediction_variance_ratio"],
        "inner_oof_rmse": train_metrics["rmse"], "inner_mean_baseline_rmse": mean_metrics["rmse"],
        "train_global_indices": train_global, "excluded_global_indices": excluded,
        "excluded_style": "Pils/Lager", "qualification_scope": "training_only_fixed_model_CV; not holdout or counterfactual validation",
        "qualification_rule": "training-only OOF variance ratio >=0.05 and RMSE < same-fold mean baseline",
        "parameter_tuning": False}
    artifact = {"pipeline": final, **common_metadata}
    joblib.dump(artifact, OUT / "transfer_stale_model.joblib", compress=3)
    pd.concat([meta.iloc[train_global].reset_index(drop=True), pd.DataFrame({"observed": y.iloc[train_global].to_numpy(),
        "fixed_inner_oof_prediction": train_oof, "mean_inner_oof_prediction": mean_oof})], axis=1).to_csv(
            OUT / "transfer_inner_oof.csv", index=False, lineterminator="\n")
    heldout = pd.concat([meta.iloc[excluded].reset_index(drop=True), pd.DataFrame({
```

src/12b_q3_stale_fixed.py（续）

```
        "observed": y.iloc[excluded].to_numpy(), "prediction": predict_1d(final, x.iloc[excluded]),
        "mean_baseline_prediction": y.iloc[train_global].mean()})], axis=1)
    old_artifact = joblib.load(SOLUTION / "outputs" / "q3" / "sensory" / "transfer_proxy.joblib")
    assert not set(old_artifact["train_global_indices"]) & set(excluded)
    heldout["old_multioutput_prediction"] = old_artifact["pipeline"].predict(x.iloc[excluded])[:, stale_position]
    heldout.to_csv(OUT / "pils_style_holdout_predictions.csv", index=False, lineterminator="\n")
    heldout_metrics = [{"model": name, **scores(heldout.observed, heldout[column])} for name, column in [
        ("fixed_single_extra_trees", "prediction"), ("non_pils_mean", "mean_baseline_prediction"),
        ("old_multioutput_transfer_proxy", "old_multioutput_prediction")]]
    pd.DataFrame(heldout_metrics).to_csv(OUT / "pils_style_holdout_metrics.csv", index=False, lineterminator="\n")

    models, indices, seeds = [], [], []
    for b in range(60):
        seed = SELECTION_BOOTSTRAP_SEED + b if b < 40 else AUDIT_BOOTSTRAP_SEED + b - 40
        index = np.random.default_rng(seed).choice(train_global, size=len(train_global), replace=True)
        model = raw_tree_pipeline(seed + 10000, n_estimators=120).fit(x.iloc[index], y.iloc[index])
        models.append(model)
        indices.append(index)
        seeds.append(seed)
        if (b + 1) % 20 == 0:
            print(f"Stale fixed bootstrap {b + 1}/60", flush=True)
    ensemble = {"models": models, "bootstrap_indices": indices, **common_metadata,
        "bootstrap_seeds": seeds, "estimator_seeds": [seed + 10000 for seed in seeds],
        "selection_model_indices": list(range(40)), "audit_model_indices": list(range(40, 60)),
        "pairing": "same_model_candidate_minus_source", "paired_quantile": 0.95,
        "bootstrap_n_estimators": 120, "final_n_estimators": 300,
        "hyperparameter_uncertainty_included": False,
        "audit_protocol": "40 screening models; 20 separate bootstrap draws for one-time audit; no reselection",
        "interpretation": "fixed single-target forest proxy; quantiles are not calibrated confidence bounds"}
    joblib.dump(ensemble, OUT / "bootstrap_ensemble.joblib", compress=3)
    np.savez_compressed(OUT / "bootstrap_indices.npz", indices=np.asarray(indices), bootstrap_seeds=np.asarray(seeds))
    assert np.asarray(indices).shape == (60, 239)
    assert all(not set(index) & set(excluded) for index in indices)
    assert len(set(seeds)) == 60
    assert np.isfinite(predict_1d(final, x)).all()
    summary = {
        "status": "fixed follow-up after exploratory model comparisons on the same dataset",
        "samples": len(x), "features": len(features), "target": TARGET,
        "outer_seeds": list(SEEDS), "outer_folds": 5, "parameter_tuning": False,
        "selected_config": selected_config,
        "pooled_repeat_metrics": [{"method": method, **pooled.loc[method].to_dict()} for method in methods],
        "rmse_reduction_vs_mean": float(1-pooled.loc['fixed_single_extra_trees', 'rmse']/pooled.loc['mean', 'rmse']),
        "rmse_reduction_vs_old_multioutput": float(1-pooled.loc['fixed_single_extra_trees', 'rmse']/pooled.loc['old_multioutput_extra_trees', 'rmse']),
```

src/12b_q3_stale_fixed.py（续）

```
        "paired_cluster_bootstrap": cluster_result,
        "transfer_qualified_for_proxy": qualified, "transfer_inner_oof_metrics": train_metrics,
        "transfer_inner_mean_metrics": mean_metrics, "transfer_cv_seed": TRANSFER_CV_SEED,
        "transfer_qualification_scope": common_metadata["qualification_scope"],
        "pils_holdout_metrics": heldout_metrics,
        "pils_holdout_improves_mean": heldout_metrics[0]["rmse"] < heldout_metrics[1]["rmse"],
        "pils_holdout_improves_old": heldout_metrics[0]["rmse"] < heldout_metrics[2]["rmse"],
        "bootstrap": {"models": 60, "selection": 40, "audit": 20, "trees_per_model": 120,
                      "final_trees": 300, "population": "239 non-Pils only", "structure_fixed": True},
        "limitations": [
            "Additional split seeds reuse the same 250 samples; the exploratory algorithm choice is not independently externally confirmed.",
            "No parameter tuning or family search is performed in this follow-up.",
            "The 500 repeated OOF predictions do not constitute 500 independent beers.",
            "All transfer preprocessing, fitting and bootstrap sampling exclude every Pils/Lager source.",
            "Source-style holdout is reported after the fixed model is trained and never changes the chosen algorithm.",
            "The 0.05 OOF variance-ratio gate does not prove candidate chemical perturbation responsiveness.",
            "Bootstrap uses 120-tree approximations to the 300-tree final forest, with fixed parameters and no algorithm-selection uncertainty.",
            "Separate screening/audit bootstrap draws share the same observed training population and are not independent experimental validation.",
            "Passing paired proxy quantiles cannot establish real stale-hops taste or causal effects; original 0-6 scores remain unavailable.",
        ],
        "verification": {"bootstrap_indices_shape": [60, 239], "source_exclusion": True,
                         "bootstrap_seeds_unique": True, "predictions_finite": True},
        "versions": {"python": platform.python_version(), "numpy": np.__version__,
                     "pandas": pd.__version__, "sklearn": sklearn.__version__},
        "elapsed_seconds": perf_counter()-started,
    }
    write_json(OUT / "summary.json", summary)
    print(pooled[["rmse", "r2", "spearman", "prediction_std_ratio"]].to_string(), flush=True)
    print("Fixed transfer qualified:", qualified, "holdout:", json.dumps(heldout_metrics, ensure_ascii=False), flush=True)


if __name__ == "__main__":
    with threadpool_limits(limits=1):
        main()
```

src/13_q3_transfer_optimized.py

SHA-256: bdc28b4229fee67fb8e0d5a6ad87cd259c42dbb5081902162dbea3b2eea3a9d0

```
"""Threshold-robust transfer with matched generation budgets and proxy audits."""
from __future__ import annotations

import json

import joblib
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist
from threadpoolctl import threadpool_limits

from common import SEED, SOLUTION
from q3_data import load_q3_data, read_defect_thresholds
from q3_generator_optimized import OptimizedStyleGenerator

OUT = SOLUTION / "outputs/q3_optimized/transfer"
OLD = SOLUTION / "outputs/q3"
ROOT_OUT = SOLUTION / "outputs/q3_optimized"
N_GENERATED = 500
FACTORS = [0.8, 0.9, 1.0, 1.1, 1.2]
LAMBDAS = [0.05, 0.2, 1.0]
DEFAULT_LAMBDA = 0.2
DEFAULT_POOL = "optimized_generator"
TOL = 1e-10


def scenarios(defects):
    return [{"scenario": f"{statistic}_x{factor:.1f}", "statistic": statistic,
             "factor": factor,
             "thresholds": {rule["feature"]: (float(rule[f"threshold_{statistic}"]) * factor
                                              if rule[f"threshold_{statistic}"] is not None else None)
                            for rule in defects.values()}}
            for statistic in ["median", "mean"] for factor in FACTORS]


def robust_caps(source, defects, scenario_list):
    caps = {}
    for rule in defects.values():
        field = rule["feature"]
        observed = float(source[field].iloc[0])
        values = [item["thresholds"][field] for item in scenario_list]
        finite = [value for value in values if value is not None]
        caps[field] = max(observed, min(finite)) if finite else observed
    return caps


def chemical_feasible(frame, source, scenario):
    good = np.ones(len(frame), dtype=bool)
    for field, threshold in scenario["thresholds"].items():
        source_value = float(source[field].iloc[0])
        cap = max(source_value, threshold) if threshold is not None else source_value
        value = frame[field].to_numpy(float)
        good &= np.isfinite(value) & (value >= 0) & (value <= cap + TOL)
    return good


def paired(models, frame, source):
```

src/13_q3_transfer_optimized.py（续）

```
    return np.column_stack([
        np.asarray(model.predict(frame)).reshape(-1)
        - float(np.asarray(model.predict(source)).reshape(-1)[0])
        for model in models
    ])


def build_pool(source, old_draws, new_draws, target_median):
    blocks, descriptions = [], []
    for family, draws in [("old_copula", old_draws), ("optimized", new_draws)]:
        for gamma in [.25, .5, .75, 1.0]:
            values = (1 - gamma) * source.to_numpy()[0] + gamma * draws.to_numpy()
            blocks.append(values)
            descriptions.extend({"family": family, "gamma": gamma, "draw": j}
                                for j in range(len(values)))
    for gamma in [.25, .5, .75, 1.0]:
        blocks.append(((1 - gamma) * source.to_numpy()[0] + gamma * target_median)[None])
        descriptions.append({"family": "shared_target_median", "gamma": gamma, "draw": -1})
    return pd.DataFrame(np.vstack(blocks), columns=source.columns), pd.DataFrame(descriptions)


def main():
    OUT.mkdir(parents=True, exist_ok=True)
    meta, x, y, manifest = load_q3_data()
    defects = read_defect_thresholds()
    scenario_list = scenarios(defects)
    (OUT / "threshold_scenarios.json").write_text(json.dumps(scenario_list, indent=2), encoding="utf-8", newline="\n")
    source_rows = np.flatnonzero(meta.tasting_category_fine.eq("Pils/Lager"))
    train = np.flatnonzero(~meta.tasting_category_fine.eq("Pils/Lager"))
    target_rows = np.flatnonzero(meta.tasting_category_fine.eq("Tripel"))
    # Keep exactly the baseline geometry; changing the yardstick would make
    # an improvement in target distance difficult to interpret.
    geometry = joblib.load(OLD / "transfer/transfer_copula.joblib")
    np.testing.assert_array_equal(geometry["training_rows"], train)
    reference = geometry["copula"]
    old_draws = reference.sample("Tripel", N_GENERATED, seed=SEED + 301)
    generator = OptimizedStyleGenerator().fit(x.iloc[train], meta.tasting_category_fine.iloc[train])
    new_draws = generator.sample("Tripel", N_GENERATED, seed=SEED + 301)
    assert old_draws.columns.tolist() == new_draws.columns.tolist() == x.columns.tolist()
    joblib.dump({"generator": generator, "training_rows": train, "source_rows": source_rows,
                 "features": x.columns.tolist()}, OUT / "transfer_generator.joblib", compress=3)
    new_draws.to_csv(OUT / "optimized_base_draws.csv", index=False, lineterminator="\n")
    old_draws.to_csv(OUT / "old_base_draws.csv", index=False, lineterminator="\n")

    stale_model = joblib.load(ROOT_OUT / "stale_fixed/transfer_stale_model.joblib")
    ensemble = joblib.load(ROOT_OUT / "stale_fixed/bootstrap_ensemble.joblib")
    assert ensemble["features"] == stale_model["features"] == x.columns.tolist()
    assert len(ensemble["models"]) == 60
    np.testing.assert_array_equal(stale_model["train_global_indices"], train)
    for indices in ensemble["bootstrap_indices"]:
        assert not np.isin(indices, source_rows).any()
    selection_models, audit_models = ensemble["models"][:40], ensemble["models"][40:]
```

src/13_q3_transfer_optimized.py（续）

```
    qualified = bool(stale_model["qualified_for_proxy"])
    old_ensemble = joblib.load(OLD / "sensory/stale_bootstrap.joblib")
    profile_model = joblib.load(OLD / "sensory/transfer_proxy.joblib")
    target_z = reference.transform(x.iloc[target_rows])
    p = x.shape[1]
    pairwise = cdist(target_z, target_z) / np.sqrt(p)
    np.fill_diagonal(pairwise, np.inf)
    support_limit = float(np.quantile(pairwise.min(axis=1), .95))
    target_center = target_z.mean(axis=0)
    minimum, maximum = x.iloc[train].min().to_numpy(), x.iloc[train].max().to_numpy()
    medians = x.iloc[train].median()
    target_median = x.iloc[target_rows].median().to_numpy()

    records, diag_rows, spectra, threshold_rows, delta_rows, profiles = [], [], [], [], [], []
    for number, index in enumerate(source_rows, 1):
        beer = int(meta.beer.iloc[index])
        raw_source = x.iloc[[index]]
        complete = all(np.isfinite(raw_source[rule["feature"]].iloc[0])
                       and raw_source[rule["feature"]].iloc[0] >= 0 for rule in defects.values())
        if not complete:
            for pool in ["old_generator", "optimized_generator", "combined_pool"]:
                for weight in LAMBDAS:
                    records.append({"source_beer": beer, "pool": pool, "lambda": weight,
                                    "is_default_policy": pool == DEFAULT_POOL and weight == DEFAULT_LAMBDA,
                                    "status": "source_defect_unverifiable", "all_model_checks_passed": False,
                                    "cross_proxy_checks_passed": False})
            continue
        source = raw_source.fillna(medians)
        values, descriptions = build_pool(source, old_draws, new_draws, target_median)
        caps = robust_caps(raw_source, defects, scenario_list)
        for field, cap in caps.items():
            values[field] = values[field].clip(lower=0, upper=cap)
        z, source_z = reference.transform(values), reference.transform(source)
        distance = reference.distance(values, "Tripel")
        before = float(reference.distance(source, "Tripel")[0])
        nn = cdist(z, target_z).min(axis=1) / np.sqrt(p)
        change = np.linalg.norm(z - source_z, axis=1) / np.sqrt(p)
        center_scale = max(float(np.linalg.norm(source_z[0] - target_center) / np.sqrt(p)), 1e-8)
        target_improvement = 1 - distance / max(before, 1e-8)
        support = (nn <= support_limit + TOL) & (
            (values.to_numpy() >= minimum - TOL) & (values.to_numpy() <= maximum + TOL)).all(axis=1)
        chemical = np.column_stack([chemical_feasible(values, raw_source, scenario)
                                    for scenario in scenario_list]).all(axis=1)
        unique_values, inverse = np.unique(values.to_numpy(), axis=0, return_inverse=True)
        candidate_delta = paired(selection_models, pd.DataFrame(unique_values, columns=x.columns), source)[inverse]
        q95 = np.quantile(candidate_delta, .95, axis=1)
        nonzero_delta = np.any(np.abs(candidate_delta) > 1e-8, axis=1)
        eligible = (support & chemical & (change >= .05) & (target_improvement >= .10)
                    & (q95 <= TOL) & nonzero_delta)
        pool_masks = {
            "old_generator": descriptions.family.isin(["old_copula", "shared_target_median"]).to_numpy(),
```

src/13_q3_transfer_optimized.py（续）

```
            "optimized_generator": descriptions.family.isin(["optimized", "shared_target_median"]).to_numpy(),
            "combined_pool": np.ones(len(values), dtype=bool),
        }
        # Full candidate matrices are reproducible from the two saved base
        # draws, source rows, fixed interpolation fractions and robust caps.
        for j in range(len(values)):
            diag_rows.append({"source_beer": beer, "candidate_index": j,
                              **descriptions.iloc[j].to_dict(), "target_distance": float(distance[j]),
                              "target_improvement": float(target_improvement[j]),
                              "target_nn_rms": float(nn[j]), "change_latent_rms": float(change[j]),
                              "support_ok": bool(support[j]), "all_thresholds_chemical_ok": bool(chemical[j]),
                              "selection_q95": float(q95[j]), "nonzero_paired_signal": bool(nonzero_delta[j]),
                              "selection_feasible": bool(eligible[j])})

        before_profile = profile_model["pipeline"].predict(source)[0]
        before_stale = float(np.asarray(stale_model["pipeline"].predict(source)).reshape(-1)[0])
        audit_cache = {}
        for pool, pool_mask in pool_masks.items():
            variation = float(np.std(candidate_delta[pool_mask].mean(axis=1)))
            zero_fraction = float(np.mean(np.all(np.abs(candidate_delta[pool_mask]) <= 1e-8, axis=1)))
            allowed = eligible & pool_mask
            for weight in LAMBDAS:
                transfer_id = f"beer_{beer}_{pool}_lambda_{weight:g}"
                common = {"transfer_id": transfer_id, "source_beer": beer, "source_style": "Pils/Lager",
                          "target_style": "Tripel", "pool": pool, "lambda": weight,
                          "is_default_policy": pool == DEFAULT_POOL and weight == DEFAULT_LAMBDA,
                          "n_candidates": int(pool_mask.sum()), "n_selection_feasible": int(allowed.sum()),
                          "target_distance_before": before, "support_nn_limit": support_limit,
                          "proxy_qualified": qualified, "candidate_mean_delta_sd": variation,
                          "candidate_zero_paired_fraction": zero_fraction,
                          "all_model_checks_passed": False, "cross_proxy_checks_passed": False}
                if not allowed.any():
                    records.append({**common, "status": "no_candidate_found"})
                    continue
                objective = distance / max(before, 1e-8) + weight * change / center_scale
                chosen = int(np.flatnonzero(allowed)[np.argmin(objective[allowed])])
                selected = values.iloc[[chosen]]
                select_delta = candidate_delta[chosen]
                cache_key = selected.to_numpy().tobytes()
                if cache_key not in audit_cache:
                    audit_cache[cache_key] = {
                        "audit_delta": paired(audit_models, selected, source)[0],
                        "old_delta": paired(old_ensemble["models"], selected, source)[0],
                        "after_stale": float(np.asarray(stale_model["pipeline"].predict(selected)).reshape(-1)[0]),
                        "after_profile": profile_model["pipeline"].predict(selected)[0
```

src/13_q3_transfer_optimized.py（续）

```
    ],
206                     }
207                 cached = audit_cache[cache_key]
208                 audit_delta = cached["audit_delta"]
209                 audit_q95 = float(np.quantile(audit_delta, .95))
210                 audit_ok = audit_q95 <= TOL
211                 # The frozen first-round committee is an architecture check,
212                 # never a way of choosing a different candidate after the audit.
213                 old_delta = cached["old_delta"]
214                 old_selection_q95 = float(np.quantile(old_delta[:30], .95))
215                 old_audit_q95 = float(np.quantile(old_delta[30:], .95))
216                 old_consensus = old_selection_q95 <= TOL and old_audit_q95 <= TOL
217                 informative = variation > 1e-6 and nonzero_delta[chosen]
218                 passed = bool(qualified and informative and audit_ok)
219                 status = ("proxy_not_qualified" if not qualified else "proxy_uninforma
    tive" if not informative
220                           else "model_checks_passed" if audit_ok else "resampling_audi
    t_failed")
221                 after_stale = cached["after_stale"]
222                 records.append({**common, "candidate_index": chosen, "status": status,
223                                 "objective": float(objective[chosen]),
224                                 "target_distance_after": float(distance[chosen]),
225                                 "target_improvement": float(target_improvement[chosen]
    ),
226                                 "target_nn_after": float(nn[chosen]), "change_latent_r
    ms": float(change[chosen]),
227                                 "all_thresholds_chemical_ok": True,
228                                 "selection_q95": float(q95[chosen]), "audit_q95": audi
    t_q95,
229                                 "all_model_checks_passed": passed,
230                                 "old_selection_q95": old_selection_q95, "old_audit_q95
    ": old_audit_q95,
231                                 "old_proxy_consensus": bool(old_consensus),
232                                 "cross_proxy_checks_passed": bool(passed and old_conse
    nsus),
233                                 "stale_before": before_stale, "stale_after": after_sta
    le})
234                 spectra.append({"transfer_id": transfer_id, "source_beer": beer,
235                                 "pool": pool, "lambda": weight, **selected.iloc[0].to_
    dict()})
236                 after_profile = cached["after_profile"]
237                 for phase, profile, stale_value in [("source", before_profile, before_
    stale),
238                                                     ("transferred", after_profile, af
    ter_stale)]:
239                     profile_values = dict(zip(y.columns, map(float, profile)))
240                     profile_values["stale_hops"] = stale_value
241                     profiles.append({"transfer_id": transfer_id, "phase": phase,
242                                      **profile_values})
243                 for group, delta in [("selection", select_delta), ("audit", audit_delt
    a),
244                                      ("old_selection", old_delta[:30]), ("old_audit",
    old_delta[30:])]:
245                     delta_rows.extend({"transfer_id": transfer_id, "ensemble": group,
246                                        "model_index": k, "paired_delta": float(value)}
247                                       for k, value in enumerate(delta))
248                 for scenario in scenario_list:
249                     for name, rule in defects.items():
250                         field = rule["feature"]
251                         threshold = scenario["thresholds"][field]
252                         old_value = float(raw_source[field].iloc[0])
```

src/13_q3_transfer_optimized.py（续）

```
                        new_value = float(selected[field].iloc[0])
                        cap = max(old_value, threshold) if threshold is not None else old_value
                        threshold_rows.append({"transfer_id": transfer_id, "source_beer": beer,
                                               "pool": pool, "lambda": weight,
                                               "scenario": scenario["scenario"], "statistic": scenario["statistic"],
                                               "factor": scenario["factor"], "defect": name,
                                               "feature": field, "unit": rule["unit"], "threshold": threshold,
                                               "source": old_value, "transferred": new_value,
                                               "upper_bound": cap, "robust_upper_bound": caps[field],
                                               "passed": new_value <= cap + TOL})
        print(f"Optimized transfer {number}/11 beer={beer}, eligible={eligible.sum()}/{len(values)}", flush=True)

    frames = {"transfer_summary.csv": records, "candidate_diagnostics.csv": diag_rows,
              "transferred_spectra.csv": spectra, "threshold_sensitivity.csv": threshold_rows,
              "stale_paired_deltas.csv": delta_rows, "sensory_before_after.csv": profiles}
    for name, rows in frames.items():
        pd.DataFrame(rows).to_csv(OUT / name, index=False, lineterminator="\n")

    # Recheck the unchanged first-round selections on the same threshold grid.
    archived = pd.read_csv(OLD / "transfer/transfer_summary.csv")
    archived = archived[archived["mode"].eq("protected") & archived.is_default_policy]
    archived_spectra = pd.read_csv(OLD / "transfer/transferred_spectra.csv").set_index("transfer_id")
    old_checks = []
    for row in archived.itertuples():
        if pd.isna(row.candidate_index):
            continue
        source = x.loc[meta.beer.eq(row.source_beer)]
        selected = archived_spectra.loc[[row.transfer_id], x.columns]
        for scenario in scenario_list:
            old_checks.append({"source_beer": row.source_beer, "transfer_id": row.transfer_id,
                               "scenario": scenario["scenario"], "old_model_checks_passed": row.all_model_checks_passed,
                               "chemical_passed": bool(chemical_feasible(selected, source, scenario)[0])})
    archived_checks = pd.DataFrame(old_checks)
    archived_checks.to_csv(OUT / "archived_threshold_recheck.csv", index=False, lineterminator="\n")
    result = pd.DataFrame(records)
    policy_rows = []
    for (pool, weight), part in result.groupby(["pool", "lambda"]):
        policy_rows.append({"pool": pool, "lambda": float(weight), "sources": len(part),
                            "selected": int(part.candidate_index.notna().sum()) if "candidate_index" in part else 0,
                            "model_checks_passed": int(part.all_model_checks_passed.sum()),
                            "cross_proxy_checks_passed": int(part.cross_proxy_checks_passed.sum()),
                            "audit_failed": int(part.status.eq("resampling_audit_faile
```

src/13_q3_transfer_optimized.py（续）

```
    d").sum()),
295                              "no_candidate": int(part.status.eq("no_candidate_found").s
    um())})
296     pd.DataFrame(policy_rows).to_csv(OUT / "policy_summary.csv", index=False, lineterm
    inator="\n")
297     archived_robust = archived_checks.groupby("source_beer").chemical_passed.all()
298     old_passed_beers = archived.loc[archived.all_model_checks_passed, "source_beer"]
299     old_all_grid = int(archived_robust.reindex(old_passed_beers, fill_value=False).sum
    ())
300     default = result[result.is_default_policy]
301     summary = {
302         "seed": SEED, "source_beers": list(map(int, meta.beer.iloc[source_rows])), "tr
    aining_n": len(train),
303         "geometry": "Frozen first-round 239-source-excluded copula; unchanged full-ran
    k NN support",
304         "threshold_scenarios": scenario_list, "threshold_robustness": "simultaneous in
    tersection of all 10 scenarios",
305         "target_improvement_minimum": .1, "latent_rms_change_minimum": .05,
306         "support_nn_q95": support_limit, "lambdas": LAMBDAS,
307         "default_pool": DEFAULT_POOL, "default_lambda": DEFAULT_LAMBDA,
308         "matched_pool_candidates": N_GENERATED * 4 + 4, "combined_candidates": N_GENER
    ATED * 8 + 4,
309         "base_draws_equal": bool(np.array_equal(old_draws.to_numpy(), new_draws.to_num
    py())),
310         "proxy_experiment": "fixed_single_target_extra_trees_followup",
311         "proxy_qualified": qualified, "selection_models": 40, "audit_models": 20,
312         "default_policy": {"sources": len(default),
313                            "selected": int(default.candidate_index.notna().sum()) if "
    candidate_index" in default else 0,
314                            "model_checks_passed_all_thresholds": int(default.all_model
    _checks_passed.sum()),
315                            "cross_proxy_checks_passed_all_thresholds": int(default.cro
    ss_proxy_checks_passed.sum()),
316                            "audit_failed": int(default.status.eq("resampling_audit_fai
    led").sum()),
317                            "no_candidate": int(default.status.eq("no_candidate_found")
    .sum())},
318         "archived_baseline": {"model_checks_passed_median_only": int(archived.all_mode
    l_checks_passed.sum()),
319                               "model_checks_passed_all_thresholds_same_spectra": old_a
    ll_grid},
320         "limitations": [
321             "All ten chemical thresholds are enforced by construction; robustness outs
    ide this grid is not established.",
322             "Proxy audit quantiles are uncalibrated and do not certify actual tasting
    outcomes.",
323             "New random splits reuse the same 250 beers and are not independent extern
    al validation.",
324             "Proxy family choice is an exploratory follow-up; repeated splits do not r
    eplace an external test set.",
325             "The old committee is a frozen architecture-consistency check, never used
    to reselect candidates.",
326             "Combined-pool gains are reported separately from matched-budget generator
     comparisons.",
327         ],
328     }
329     (OUT / "summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2
    ), encoding="utf-8", newline="\n")
330     print(json.dumps(summary["default_policy"], ensure_ascii=False, indent=2), flush=T
    rue)
331
```

src/13_q3_transfer_optimized.py（续）

```
332
333 if __name__ == "__main__":
334     with threadpool_limits(limits=1):
335         main()
```

src/14_q3_optimization_report.py

SHA-256: 285c5096f0b8863567c4b0d03f5334353c6dbf5c5d8e88c79b844ebf4879340c

```
"""Summarize nested generation, weak-proxy experiments and robust transfer."""
import json

import joblib
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from common import SOLUTION
from q3_data import load_q3_data

OUT = SOLUTION / "outputs/q3_optimized"
OLD = SOLUTION / "outputs/q3"
STYLES = ["Tripel", "Kriek", "Lambic", "Pils/Lager"]


def read_json(path):
    return json.loads(path.read_text(encoding="utf-8"))


def main():
    meta, x, y, manifest = load_q3_data()
    figures = OUT / "figures"
    figures.mkdir(parents=True, exist_ok=True)
    generation = read_json(OUT / "generation/summary.json")
    search = read_json(OUT / "stale/summary.json")
    fixed = read_json(OUT / "stale_fixed/summary.json")
    transfer = read_json(OUT / "transfer/summary.json")
    gen_cv = pd.read_csv(OUT / "generation/cv_summary.csv")
    transfer_table = pd.read_csv(OUT / "transfer/transfer_summary.csv")
    default = transfer_table[transfer_table.is_default_policy].sort_values("source_bee
r")
    interval = fixed["paired_cluster_bootstrap"]
    fixed_metrics = pd.DataFrame(fixed["pooled_repeat_metrics"]).set_index("method")
    search_metrics = pd.DataFrame(search["pooled_repeat_metrics"]).set_index("method")
    source_holdout = pd.DataFrame(fixed["pils_holdout_metrics"]).set_index("model")
    profile_model = joblib.load(OLD / "sensory/sensory_model.joblib")
    stale_model = joblib.load(OUT / "stale_fixed/transfer_stale_model.joblib")
    profiles, profile_rows = [], []
    for style in STYLES:
        generated = pd.read_csv(OUT / f"generation/generated_{style.replace('/', '-')}
.csv")
        pred = profile_model["pipeline"].predict(generated[x.columns])
        pred[:, y.columns.get_loc("stale_hops")] = stale_model["pipeline"].predict(gen
erated[x.columns])
        profile = pd.DataFrame(pred, columns=y.columns)
        profile.insert(0, "target_style", style)
        profile.insert(0, "sample_id", generated.sample_id)
        profiles.append(profile)
        for j, target in enumerate(y.columns):
            profile_rows.append({"style": style, "target": target,
                                 "predicted_mean": float(pred[:, j].mean()),
                                 "predicted_p05": float(np.quantile(pred[:, j], .05)),
                                 "predicted_p95": float(np.quantile(pred[:, j], .95)),
                                 "real_observed_mean": float(y.loc[meta.tasting_catego
ry_fine.eq(style), target].mean()),
                                 "scale": "attachment_4_standardized",
```

src/14_q3_optimization_report.py（续）

```
                                    "prediction_model": "fixed_stale_forest_239_non_pils"
 if target == "stale_hops"
                                                        else "original_45_output_forest_2
50",
                                    "interval_meaning": "generated_spectrum_spread_not_co
nfidence_interval"})
    pd.concat(profiles, ignore_index=True).to_csv(OUT / "generated_sensory_profiles.cs
v", index=False, lineterminator="\n")
    new_profiles = pd.DataFrame(profile_rows)
    new_profiles.to_csv(OUT / "generated_sensory_summary.csv", index=False, linetermin
ator="\n")
    old_profiles = pd.read_csv(OLD / "generated_sensory_summary.csv")

    # Aggregate chemical rechecks independently of the proxy model counts.
    old_grid = pd.read_csv(OUT / "transfer/archived_threshold_recheck.csv")
    new_grid = pd.read_csv(OUT / "transfer/threshold_sensitivity.csv")
    scenario_rows = []
    for scenario in transfer["threshold_scenarios"]:
        name = scenario["scenario"]
        archived = old_grid[old_grid.scenario.eq(name)]
        current = new_grid[new_grid.scenario.eq(name)].groupby("transfer_id").passed.a
ll()
        current_checks = default.transfer_id.map(current).fillna(False)
        scenario_rows.append({"scenario": name, "statistic": scenario["statistic"], "f
actor": scenario["factor"],
                              "old_model_checks_and_chemical": int((archived.old_model
_checks_passed & archived.chemical_passed).sum()),
                              "new_chemical_only": int(current_checks.sum()),
                              "new_model_checks_and_chemical": int((current_checks & d
efault.all_model_checks_passed).sum()),
                              "cross_proxy_checks_and_chemical": int((current_checks &
 default.cross_proxy_checks_passed).sum())})
    scenario_counts = pd.DataFrame(scenario_rows)
    scenario_counts.to_csv(OUT / "transfer/threshold_pass_counts.csv", index=False, li
neterminator="\n")

    plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 10,
                         "axes.spines.top": False, "axes.spines.right": False,
                         "figure.facecolor": "white", "savefig.facecolor": "white"})
    fig, axes = plt.subplots(1, 2, figsize=(12, 5.4))
    for offset, method, label, color in [(-.24, "legacy_copula", "Old copula", "#9CA1A
8"),
                                         (0, "nested_selected", "Optimized selection",
"#2877A6"),
                                         (.24, "style_bootstrap", "Style bootstrap", "#
C68B2B")]:
        part = gen_cv[gen_cv.method.eq(method)].set_index("style").loc[["Kriek", "Lamb
ic"]]
        axes[0].bar(np.arange(2) + offset, part.energy_pca5_mean, .22, color=color,
                    yerr=part.energy_pca5_std, capsize=3, label=label,
                    error_kw={"elinewidth": 1, "ecolor": "#555B62"})
    axes[0].set_xticks([0, 1], ["Kriek", "Lambic"])
    axes[0].set_ylabel("Energy discrepancy (V-statistic)")
    axes[0].set_title("Style generation on matched outer splits", loc="left", pad=34)
    axes[0].legend(frameon=False, fontsize=8, ncol=3, loc="lower center", bbox_to_anch
or=(.5, 1.005))
    method_specs = [("mean", "Mean", "#C4C7CC"),
                    ("old_multioutput_extra_trees", "Old 45-output", "#9CA1A8"),
                    ("fixed_single_extra_trees", "Dedicated tree", "#2877A6")]
    for offset, (method, label, color) in zip([-.24, 0, .24], method_specs):
        values = [search_metrics.loc[method, "rmse"], fixed_metrics.loc[method, "rmse"
```

src/14_q3_optimization_report.py（续）

```
    ]]
        axes[1].bar(np.arange(2) + offset, values, .22, color=color, label=label)
        for i, value in enumerate(values):
            axes[1].text(i + offset, value + .001, f"{value:.4f}", ha="center", va="bottom", fontsize=8)
    axes[1].set_xticks([0, 1], ["Search-stage splits", "Follow-up splits"])
    axes[1].set_ylabel("Stale hops pooled repeated OOF RMSE")
    axes[1].set_ylim(0, .52)
    axes[1].set_title("Prediction gains remain uncertain", loc="left", pad=34)
    axes[1].legend(frameon=False, fontsize=8, ncol=3, loc="lower center", bbox_to_anchor=(.5, 1.005))
    for ax in axes:
        ax.grid(axis="y", color="#E5E8EC", linewidth=.7)
        ax.set_axisbelow(True)
    fig.text(.08, .025, "Generation: 2 x 3 outer folds, training-only method selection; bars show mean and descriptive fold SD.\n"
             "Prediction: two repeated five-fold splits per stage. Dedicated-tree advantage has a paired interval crossing zero.",
             fontsize=9, color="#555B62")
    fig.subplots_adjust(left=.08, right=.98, top=.79, bottom=.23, wspace=.35)
    fig.savefig(figures / "generation_and_prediction.png", dpi=300)
    fig.savefig(figures / "generation_and_prediction.svg")
    plt.close(fig)

    fig, ax = plt.subplots(figsize=(11.4, 5.6))
    positions = np.arange(len(scenario_counts))
    for field, label, color, marker in [
        ("old_model_checks_and_chemical", "Archived selection + original proxy", "#969DA5", "o"),
        ("new_model_checks_and_chemical", "Robust chemical + new proxy checks", "#2877A6", "s"),
        ("cross_proxy_checks_and_chemical", "Robust chemical + both proxy committees", "#C68B2B", "D")]:
        ax.plot(positions, scenario_counts[field], marker=marker, color=color, linewidth=1.6, label=label)
    ax.set_xticks(positions, [f"{r.statistic}\nx{r.factor:g}" for r in scenario_counts.itertuples()])
    ax.set_ylim(-.4, 11.5)
    ax.set_yticks(range(0, 12))
    ax.set_ylabel("Sources passing the specified checks / 11")
    ax.set_title("One selected spectrum per source, checked across ten threshold scenarios", loc="left", fontsize=13, pad=17)
    ax.axvline(4.5, color="#D8DDE3", linewidth=1)
    ax.legend(frameon=False, fontsize=9, loc="upper right")
    ax.grid(axis="y", color="#E5E8EC", linewidth=.7)
    ax.set_axisbelow(True)
    fig.text(.09, .025, "Same 2004-candidate budget; the robust method enforces the intersection of all chemical scenarios before selection.\n"
             "Proxy checks are uncalibrated diagnostics; weaker cross-style prediction has not been resolved. No audit-based reselection.",
             fontsize=9, color="#555B62")
    fig.subplots_adjust(left=.09, right=.98, top=.85, bottom=.24)
    fig.savefig(figures / "threshold_stability.png", dpi=300)
    fig.savefig(figures / "threshold_stability.svg")
    plt.close(fig)

    attributes = ["A_malt_all", "F_malt_all", "A_hops_all", "F_hops_all", "A_esters_all",
                  "F_esters_all", "acidity", "bitternes", "sweetness"]
    values, labels = [], []
    for style in ["Kriek", "Lambic"]:
```

src/14_q3_optimization_report.py（续）

```
        old_part = old_profiles[old_profiles["style"].eq(style)].set_index("target").loc[attributes]
        new_part = new_profiles[new_profiles["style"].eq(style)].set_index("target").loc[attributes]
        values.extend([old_part.predicted_mean, new_part.predicted_mean, new_part.real_observed_mean])
        labels.extend([f"{style}: old", f"{style}: optimized", f"{style}: observed"])
    values = np.asarray(values)
    bound = max(np.abs(values).max(), .5)
    fig, ax = plt.subplots(figsize=(11.8, 5.2))
    im = ax.imshow(values, cmap="RdBu_r", vmin=-bound, vmax=bound, aspect="auto")
    ax.set_yticks(np.arange(len(labels)), labels)
    ax.set_xticks(np.arange(len(attributes)), ["Malt\naroma", "Malt\nflavour", "Hops\naroma", "Hops\nflavour",
                                               "Esters\naroma", "Esters\nflavour", "Acidity", "Bitterness", "Sweetness"])
    for (i, j), value in np.ndenumerate(values):
        ax.text(j, i, f"{value:.2f}", ha="center", va="center", fontsize=9,
                color="white" if abs(value) > bound * .65 else "#25282D")
    ax.set_title("Generated sensory fingerprints: old and optimized", loc="left", fontsize=14, pad=15)
    fig.colorbar(im, ax=ax, shrink=.85, label="Attachment-standardized value")
    fig.text(.19, .025, "300 spectra per style. Old and optimized rows are model predictions; observed rows summarize existing panel scores.\n"
             "This descriptive fingerprint is not independent sensory validation and is not the original 0-6 scale.",
             fontsize=9, color="#555B62")
    fig.subplots_adjust(left=.19, right=.96, top=.85, bottom=.24)
    fig.savefig(figures / "sensory_fingerprint_comparison.png", dpi=300)
    fig.savefig(figures / "sensory_fingerprint_comparison.svg")
    plt.close(fig)

    generation_rows = []
    gains = {}
    for style in STYLES:
        table = gen_cv[gen_cv["style"].eq(style)].set_index("method")
        old, new, bootstrap = table.loc["legacy_copula"], table.loc["nested_selected"], table.loc["style_bootstrap"]
        gain = 1 - new.energy_pca5_mean / old.energy_pca5_mean
        gains[style] = float(gain)
        generation_rows.append(f"| {style} | {old.energy_pca5_mean:.4f} | {new.energy_pca5_mean:.4f} | "
                               f"{gain * 100:+.1f}% | {bootstrap.energy_pca5_mean:.4f} | "
                               f"{old.marginal_w1_iqr_median_mean:.4f} → {new.marginal_w1_iqr_median_mean:.4f} |")
    stale_rows = []
    for label, table in [("模型搜索阶段", search_metrics), ("固定方案后续复核", fixed_metrics)]:
        for method, name in [("old_multioutput_extra_trees", "旧45目标森林"),
                             ("fixed_single_extra_trees", "固定单目标森林")]:
            row = table.loc[method]
            stale_rows.append(f"| {label} | {name} | {row.rmse:.4f} | {row.r2:.4f} | "
                              f"{row.spearman:.4f} | {row.prediction_std_ratio:.3f} |")
    status_names = {"model_checks_passed": "新代理审计通过", "resampling_audit_failed": "新代理审计未通过",
                    "no_candidate_found": "未找到候选", "proxy_not_qualified": "代理训练内资格不足",
                    "proxy_uninformative": "代理对候选缺少响应", "source_defect_unverifiable": "源缺陷不可核验"}
```

src/14_q3_optimization_report.py（续）

```
    transfer_rows = []
    for row in default.itertuples():
        improvement = f"{100 * row.target_improvement:.1f}%" if pd.notna(row.candidate_index) else "—"
        transfer_rows.append(f"| {row.source_beer} | {status_names[row.status]} | {improvement} | "
                             f"{row.audit_q95:+.4f} | {'是' if row.cross_proxy_checks_passed else '否'} |")
    counts = transfer["default_policy"]
    ci = interval["rmse_difference_percentile95"]
    delta = interval["point_rmse_difference"]
    report = f"""# 第三问优化：风格保真性、缺陷预测与阈值稳健迁移

日期：2026-10-23。原始第三问65个结果文件已冻结并逐文件校验；优化结果另存 `../outputs/q3_optimized/`。

## 1. 本轮结论

- Kriek、Lambic的嵌套选择生成算法，在匹配外层划分和共同评价坐标下，能量差异指标较旧模型分别降低 **{100 * gains['Kriek']:.1f}%**、**{100 * gains['Lambic']:.1f}%**。两者仍略逊于重采样，不能称所有质量指标全面胜出。
- Stale hops专用树在后续重复切分中，RMSE由 **{fixed_metrics.loc['old_multioutput_extra_trees', 'rmse']:.4f}** 降为 **{fixed_metrics.loc['fixed_single_extra_trees', 'rmse']:.4f}**，下降 **{fixed['rmse_reduction_vs_old_multioutput'] * 100:.2f}%**。按beer配对重采样的差异区间跨零，且Pils整风格留出更差，**尚未证实稳定的预测提升**。
- 默认迁移在10个预定阈值情景下同时约束化学保护，11款源均找到候选，**{counts['model_checks_passed_all_thresholds']}**款通过新代理检查，**{counts['cross_proxy_checks_passed_all_thresholds']}**款同时通过旧冻结代理检查。旧固定谱在同一完整阈值网格下通过全部模型与化学检查的数量为 **{transfer['archived_baseline']['model_checks_passed_all_thresholds_same_spectra']}**。

模型检查仅定义本次候选筛查的通过范围，不证明实际品评结果。阈值稳健性与感官预测准确性分别报告。

## 2. 生成模型：保留风格局部边缘

旧模型采用全局经验边缘并向全局均值收缩，会削弱小样本风格的独特分布。本轮预声明三个候选：风格局部边缘copula、风格中心平滑混合、旧copula。局部模型保留目标风格边缘支持；平滑混合使用固定0.45噪声强度。内层使用新颖性门槛，避免以近似复制训练行制造误差下降。

外层为2×3折风格分层切分，种子20261123、20262123；每个训练集内2折选择方法，所有候选统一在该训练折旧copula的高斯秩空间及5维PCA参考坐标下评价。每折每法300条、2个模拟种子。内层选择不读取外层目标数据；这些新划分仍复用同一250款酒，属于重复内部验证。

| 风格 | 旧模型能量差异 | 嵌套选择能量差异 | 相对下降 | 重采样对照 | W1跨特征中位数：旧→新 |
|---|---:|---:|---:|---:|---:|
{chr(10).join(generation_rows)}

能量指标为未开平方的经验V统计量，越小越好。平均W1、相关差异、零频率、多样性和复制检查也完整保存；Kriek平均W1仍被极端化合物支配，不能只看某一项指标。外层折与重复并不独立，折间标准差不是置信区间。

全250款重新训练后的风格选择为 `{json.dumps(generation['full_selected_methods'], ensure_ascii=False)}`。四风格各生成300条谱，原始完整行及忽略训练缺失格后的观测行复制率均为0。

![生成与预测对照](../outputs/q3_optimized/figures/generation_and_prediction.png)

## 3. Stale hops：比较失败与有限改善均保留

18个配置包括稳健线性、PLS、SVR和树模型；特征限于226个化学指标及附件aging元数据确定的子集，不使用实际其他感官评分或评论特征。处理、筛选和调参均在训练折内部。内层预测方差比至少0.05才进入合格配置，再按RMSE选择。
```

```
231
232 搜索算法本身的嵌套结果为RMSE **{search_metrics.loc['nested_selected', 'rmse']:.4f}**，劣于同划分旧森林的 **{search_metrics.loc['old_multioutput_extra_trees', 'rmse']:.4f}**。因此没有把外层表现最好的家族直接替代为“无偏最优模型”。
233
234 固定单目标森林保留原基线的300树、min_leaf=3、max_features=0.7参数，不再调参；在额外种子20263123、20264123的5折切分复核。这个后续算法选择由前一轮结果触发，因此仍是探索性复核，并非全新独立测试。
235
236 | 阶段 | 模型 | 重复OOF RMSE | R² | Spearman | 预测SD/实际SD |
237 |---|---|---:|---:|---:|---:|
238 {chr(10).join(stale_rows)}
239
240 固定方案两次切分一好一坏。先按beer平均两次切分平方误差，再对250款酒做2000次配对重采样：`新RMSE−旧RMSE` 点差 **{delta:+.5f}**，95%分位区间 **[{ci[0]:+.5f}, {ci[1]:+.5f}]**，包含零。它只描述已保存OOF误差的抽样波动，不包含模型选择过程，也没有重拟合模型。
241
242 Pils整风格留出11款时，专用树RMSE为 **{source_holdout.loc['fixed_single_extra_trees', 'rmse']:.4f}**，劣于均值 **{source_holdout.loc['non_pils_mean', 'rmse']:.4f}** 和旧代理 **{source_holdout.loc['old_multioutput_transfer_proxy', 'rmse']:.4f}**。这说明内部小幅改善未转化为跨风格预测提升。迁移代理资格门槛仅来自239款非Pils训练内验证，不等于已经通过源风格外推验证。
243
244 ## 4. 同时保护10种阈值情景
245
246 定义情景集合为中位数/均值两种口径分别乘以0.8、0.9、1.0、1.1、1.2。对每个缺陷要求所有情景的超阈程度均不增加：
247
248 `c'_d ≤ min_s max(T[d,s], c_source[d]) = max(min_s T[d,s], c_source[d])`。
249
250 相应最严格阈值为DMS **13.8 μg/L**、Diacetyl **0.072 mg/L**、4VG **0.16 mg/L**。trans-2-nonenal没有附件阈值，继续采用浓度不得增加。保护依据为源的观测校准值，不以插补值替代。
251
252 迁移仍排除全部11款Pils训练数据，使用冻结的原239款训练几何和完整226维最近邻支持规则；目标距离至少改善10%、改动RMS至少0.05的条件保持不变。新旧生成器各500条、4个插值比例加4个共同目标中位谱，匹配预算均为2004候选；合并池4004候选单独报告。
253
254 239款重新训练的Tripel生成方法仍为旧copula，相同种子的新旧基础池完全一致：**{transfer['base_draws_equal']}**。因此本轮Tripel迁移改善不能归因于生成器更换，主要改变来自同时阈值约束和专用代理筛选；Kriek、Lambic的生成改善是独立结果。
255
256 专用代理有60个固定结构bootstrap模型，前40个筛选，后20个仅审计已选谱；方差和候选差异门槛用于识别近常数代理。旧30/10委员会仅对已选谱做冻结架构一致性检查，不参与重选。所有模型均排除全部11款源；不同模型仍共享239款真实训练数据，分位数不是95%真实风味保证。
257
258 | 源beer | 默认方案状态 | 目标距离改善 | 新代理审计Q95 | 同时通过旧代理 |
259 |---|---|---:|---:|---|
260 {chr(10).join(transfer_rows)}
261
262 默认有 **{counts['model_checks_passed_all_thresholds']}/11** 通过新代理检查、**{counts['cross_proxy_checks_passed_all_thresholds']}/11** 通过新旧代理共同检查，其化学保护在全部10个阈值情景中保持通过。这里的稳定性来自预先施加的情景交集约束，不能外推为任意未知阈值下都稳定。
263
264 ![阈值稳定性](../outputs/q3_optimized/figures/threshold_stability.png)
265
266 ## 5. 画像与结果文件
267
268 ![风格画像对照](../outputs/q3_optimized/figures/sensory_fingerprint_comparison.png)
269
270 四种风格各300条谱及1200×45画像已保存。其他44维使用原全量感官预测器，Stale hops由本轮239款非Pils专用树输出；没有把它们描述为全45维新训练模型。感官尺度仍为附件标准化数值，不能当作恢复的原
```

始0–6分。

271
272 - 生成谱与完整分布验证：`../outputs/q3_optimized/generation/`。
273 - 18配置嵌套搜索：`../outputs/q3_optimized/stale/`；固定专用树复核：`../outputs/q3_optimized/stale_fixed/`。
274 - 逐项情景保护、候选筛选、审计及三种λ结果：`../outputs/q3_optimized/transfer/`。
275 - 画像：`../outputs/q3_optimized/generated_sensory_profiles.csv` 和 `generated_sensory_summary.csv`。
276 - 实现：`11_q3_generation_optimized.py`、`12_q3_stale_optimized.py`、`12b_q3_stale_fixed.py`、`13_q3_transfer_optimized.py`。
277 - 一键复现：仓库根目录执行 `.venv/bin/python solution_a/run_q3_optimized.py`。原第三问对照已冻结，此入口不重写旧结果。
278
279 方法与审查详见 `q3_generation_optimized.md`、`q3_stale_optimized.md`、`q3_optimization_review.md`。数值校验见 `../outputs/q3_optimized/verification.json`。当前结果不包含真实酿造或迁移后的品评实验，Stale hops预测可靠性仍是主要瓶颈。
280 """
281   (SOLUTION / "docs/q3_optimization_results.md").write_text(report, encoding="utf-8", newline="\n")
282   summary = {"generation_energy_reduction_vs_matched_legacy": gains,
283   "fixed_stale_rmse_reduction_vs_matched_old": fixed["rmse_reduction_vs_old_multioutput"],
284   "paired_rmse_difference_interval": ci,
285   "stale_improvement_stably_established": False,
286   "source_style_prediction_improved": fixed["pils_holdout_improves_old"],
287   "default_transfer": counts, "base_draws_equal": transfer["base_draws_equal"]}
288   (OUT / "results_summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8", newline="\n")
289   print(json.dumps(summary, ensure_ascii=False, indent=2), flush=True)
290
291
292 if __name__ == "__main__":
293   main()

src/15_q3_optimized_verify.py

SHA-256: 237c77ca0ffc4b5d22df880087508302ffae20567b106bfd84844b8e977ff17c

```python
"""Verify preserved baselines, nested selection and robust transfer outputs."""
from __future__ import annotations

import hashlib
import importlib
import json
from time import perf_counter

import joblib
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist
from scipy.stats import spearmanr
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import KFold, StratifiedKFold
from threadpoolctl import threadpool_limits

from common import SOLUTION, data_dir

BASE = importlib.import_module("10_q3_verify")
OUT = SOLUTION / "outputs/q3_optimized"
TOL = 1e-10


def score(truth, prediction):
    truth, prediction = np.asarray(truth, float), np.asarray(prediction, float)
    tv, pv = np.var(truth), np.var(prediction)
    tr = np.quantile(truth, .95) - np.quantile(truth, .05)
    pr = np.quantile(prediction, .95) - np.quantile(prediction, .05)
    return {"mae": mean_absolute_error(truth, prediction),
            "rmse": np.sqrt(mean_squared_error(truth, prediction)),
            "r2": r2_score(truth, prediction),
            "spearman": spearmanr(truth, prediction).statistic if pv > 1e-20 else np.n
an,
            "prediction_variance": pv, "target_variance": tv,
            "prediction_variance_ratio": pv / tv,
            "prediction_std_ratio": np.sqrt(pv / tv),
            "prediction_range90": pr, "target_range90": tr,
            "prediction_range90_ratio": pr / tr, "mean_bias": np.mean(prediction - tru
th)}


def check_baseline():
    manifest = BASE.read_json(OUT / "baseline_hashes.json")
    baseline = SOLUTION / "outputs/q3"
    expected = {row["path"] for row in manifest["files"]}
    actual = {path.relative_to(baseline).as_posix() for path in baseline.rglob("*") if
 path.is_file()}
    assert actual == expected and len(actual) == manifest["file_count"]
    for row in manifest["files"]:
        raw = (baseline / row["path"]).read_bytes()
        assert len(raw) == row["bytes"] and hashlib.sha256(raw).hexdigest() == row["sh
a256"], row["path"]
    return {"unchanged_files": len(actual), "hash_algorithm": "sha256"}


def winner(rows):
    qualified = rows[rows.nonconstant_qualified]
    accepted = qualified if len(qualified) else rows
```

src/15_q3_optimized_verify.py（续）

```
    return accepted.sort_values(["rmse", "candidate"]).iloc[0]


def check_pipeline_training(pipeline, training, calibrated):
    chemical = pipeline.named_steps["chemical"]
    names = chemical.selected_features_
    values = training[names].to_numpy().copy()
    scales = np.ones(len(names))
    for j, name in enumerate(names):
        if name in calibrated:
            positives = values[:, j][np.isfinite(values[:, j]) & (values[:, j] > 0)]
            scales[j] = np.median(positives) if len(positives) else 1.0
            values[:, j] = np.log1p(values[:, j] / scales[j])
    BASE.close(chemical.positive_scales_, scales, "Training-only positive scales")
    bounds = np.nanquantile(values, [.01, .99], axis=0)
    BASE.close(chemical.bounds_, bounds, "Training-only clipping bounds")
    clipped = np.clip(values, bounds[0], bounds[1])
    medians = np.nanmedian(clipped, axis=0)
    medians = np.nan_to_num(medians, nan=0.0)
    imputer = pipeline.named_steps["imputer"]
    BASE.close(imputer.statistics_, medians, "Training-only imputation medians")
    indicators = np.flatnonzero(np.isnan(clipped).any(axis=0))
    np.testing.assert_array_equal(imputer.indicator_.features_, indicators)
    filled = np.where(np.isnan(clipped), medians, clipped)
    filled = np.column_stack([filled, np.isnan(clipped)[:, indicators]])
    BASE.close(pipeline.named_steps["scale"].mean_, filled.mean(axis=0), "Training-onl
y scaling means")
    BASE.close(pipeline.named_steps["scale"].var_, filled.var(axis=0), "Training-only
scaling variances")


def check_stale(context):
    folder = OUT / "stale"
    meta, x, y45 = context["meta"], context["x"], context["y"]
    y = y45.stale_hops
    manifest = BASE.read_json(folder / "run_manifest.json")
    summary = BASE.read_json(folder / "summary.json")
    oof = pd.read_csv(folder / "oof_predictions.csv")
    stored_metrics = pd.read_csv(folder / "metrics.csv")
    stored_folds = pd.read_csv(folder / "metrics_by_fold.csv")
    choices = pd.read_csv(folder / "nested_selected_configs.csv")
    inner = pd.read_csv(folder / "nested_inner_cv_results.csv")
    specs = {spec["name"]: spec for spec in manifest["candidate_specs"]}
    assert manifest["features"] == x.columns.tolist() and len(specs) == summary["candi
date_count"]
    assert not oof.duplicated(["repeat", "method", "beer"]).any()
    methods = sorted(oof.method.unique())
    metrics, folds = [], []
    for repeat, seed in enumerate(manifest["outer_seeds"], 1):
        expected_fold = np.zeros(len(x), int)
        for fold, (_, test) in enumerate(KFold(manifest["outer_folds"], shuffle=True,
random_state=seed).split(x), 1):
            expected_fold[test] = fold
        for method in methods:
            group = oof[oof.repeat.eq(repeat) & oof.method.eq(method)].set_index("beer
").loc[meta.beer]
            assert len(group) == len(x) and group.outer_seed.eq(seed).all()
            np.testing.assert_array_equal(group.fold, expected_fold)
            BASE.close(group.observed, y, "Repeated OOF observed labels")
            assert np.isfinite(group.prediction).all()
            metrics.append({"scope": "per_seed", "repeat": repeat, "method": method,
```

src/15_q3_optimized_verify.py（续）

```
                                **score(group.observed, group.prediction)})
            for fold in range(1, manifest["outer_folds"] + 1):
                test = expected_fold == fold
                folds.append({"repeat": repeat, "fold": fold, "method": method,
                              **score(y.loc[test], group.prediction.to_numpy()[test])}
)
                if method == "mean":
                    BASE.close(group.prediction.to_numpy()[test], y.loc[~test].mean(),
 "Fold-mean stale baseline")
    for method in methods:
        group = oof[oof.method.eq(method)]
        metrics.append({"scope": "pooled_repeated_oof", "repeat": 0, "method": method,
                        **score(group.observed, group.prediction)})
    columns = list(score(y, y))
    BASE.same_table(stored_metrics, pd.DataFrame(metrics), ["scope", "repeat", "method
"], columns,
                    "Repeated nested stale metrics")
    BASE.same_table(stored_folds, pd.DataFrame(folds), ["repeat", "fold", "method"], c
olumns,
                    "Nested stale fold metrics")
    gate = manifest["nonconstant_variance_ratio_min"]
    assert inner.nonconstant_qualified.equals(inner.prediction_variance_ratio.ge(gate)
)
    assert set(inner.candidate) == set(specs)
    for row in choices.itertuples(index=False):
        candidates = inner[inner.repeat.eq(row.repeat) & inner.outer_fold.eq(row.outer
_fold)]
        if row.method.startswith("nested_family_"):
            candidates = candidates[candidates.family.eq(row.method.removeprefix("nest
ed_family_"))]
        selected = winner(candidates)
        assert row.selected_candidate == selected.candidate
        assert row.nonconstant_qualified == selected.nonconstant_qualified
        BASE.close(row.inner_oof_rmse, selected.rmse, "Training-only nested candidate
choice")
        assert row.outer_seed in manifest["outer_seeds"]
    excluded = np.flatnonzero(meta.tasting_category_fine.eq("Pils/Lager"))
    training = np.flatnonzero(~meta.tasting_category_fine.eq("Pils/Lager"))
    artifact = joblib.load(folder / "transfer_stale_model.joblib")
    ensemble = joblib.load(folder / "bootstrap_ensemble.joblib")
    transfer_inner = pd.read_csv(folder / "transfer_inner_cv_results.csv")
    selected = winner(transfer_inner)
    assert artifact["selected_config"] == specs[selected.candidate] == ensemble["selec
ted_config"]
    inner_oof = pd.read_csv(folder / "transfer_selected_inner_oof.csv")
    np.testing.assert_array_equal(inner_oof.beer, meta.beer.iloc[training])
    BASE.close(inner_oof.observed, y.iloc[training], "Transfer inner observed labels")
    selected_score = score(inner_oof.observed, inner_oof.selected_inner_oof_prediction
)
    BASE.close([selected[name] for name in columns], [selected_score[name] for name in
 columns],
               "Transfer selected inner OOF metrics")
    expected_mean = np.empty(len(training))
    for train, test in KFold(manifest["inner_folds"], shuffle=True, random_state=manif
est["transfer_cv_seed"]).split(training):
        expected_mean[test] = y.iloc[training[train]].mean()
    BASE.close(inner_oof.mean_inner_oof_prediction, expected_mean, "Transfer inner fol
d-mean baseline")
    mean_rmse = score(y.iloc[training], expected_mean)["rmse"]
    qualified = bool(selected.prediction_variance_ratio >= gate and selected.rmse < me
an_rmse)
```

src/15_q3_optimized_verify.py（续）

```
    assert qualified == artifact["qualified_for_proxy"] == ensemble["qualified_for_proxy"]
    assert qualified == summary["transfer_qualified_for_proxy"]
    for saved in [artifact, ensemble]:
        np.testing.assert_array_equal(saved["train_global_indices"], training)
        np.testing.assert_array_equal(saved["excluded_global_indices"], excluded)
        assert saved["features"] == x.columns.tolist()
    check_pipeline_training(artifact["pipeline"], x.iloc[training], context["calibrated_features"])
    indices = np.asarray(ensemble["bootstrap_indices"])
    assert indices.shape == (60, len(training)) and not np.isin(indices, excluded).any()
    assert np.isin(indices, training).all()
    assert ensemble["selection_model_indices"] == list(range(40))
    assert ensemble["audit_model_indices"] == list(range(40, 60))
    assert ensemble["hyperparameter_uncertainty_included"] is False
    with np.load(folder / "bootstrap_indices.npz") as saved:
        np.testing.assert_array_equal(saved["indices"], indices)
        np.testing.assert_array_equal(saved["bootstrap_seeds"], ensemble["bootstrap_seeds"])
        np.testing.assert_array_equal(saved["estimator_seeds"], ensemble["estimator_seeds"])
    for i, (model, index, seed) in enumerate(zip(ensemble["models"], indices, ensemble["bootstrap_seeds"])):
        expected_seed = manifest["selection_bootstrap_seed"] + i if i < 40 else manifest["audit_bootstrap_seed"] + i - 40
        assert seed == expected_seed
        expected_index = np.random.default_rng(seed).choice(training, size=len(training), replace=True)
        np.testing.assert_array_equal(index, expected_index)
        check_pipeline_training(model, x.iloc[index], context["calibrated_features"])
    heldout = pd.read_csv(folder / "pils_style_holdout_predictions.csv")
    np.testing.assert_array_equal(heldout.beer, meta.beer.iloc[excluded])
    BASE.close(heldout.observed, y.iloc[excluded], "Pils holdout observed labels")
    BASE.close(heldout.prediction, np.asarray(artifact["pipeline"].predict(x.iloc[excluded])).reshape(-1),
               "Pils holdout frozen model predictions")
    holdout_metrics = pd.read_csv(folder / "pils_style_holdout_metrics.csv").set_index("model")
    for method, pred in [("optimized_single_target", heldout.prediction),
                         ("non_pils_mean", heldout.mean_baseline_prediction)]:
        BASE.close(holdout_metrics.loc[method, columns], list(score(heldout.observed, pred).values()),
                   "Pils source holdout metrics")
    context.update({"optimized_stale": artifact, "optimized_ensemble": ensemble,
                    "optimized_train": training, "optimized_sources": excluded})
    main = stored_metrics[stored_metrics.scope.eq("pooled_repeated_oof")].set_index("method")
    return {"outer_seeds": manifest["outer_seeds"], "outer_folds": manifest["outer_folds"],
            "candidate_count": len(specs), "nested_choices_recomputed": len(choices),
            "transfer_config": artifact["selected_config"], "qualified_for_proxy": qualified,
            "training_rows": len(training), "excluded_sources": len(excluded),
            "bootstrap_shape": list(indices.shape), "preprocessing_training_checks": 61,
            "nested_selected_rmse": float(main.loc["nested_selected", "rmse"]),
            "old_multioutput_same_split_rmse": float(main.loc["old_multioutput_extra_trees", "rmse"]),
            "fixed_single_tree_rmse": float(main.loc["fixed_single_extra_trees", "rmse"]),
```

src/15_q3_optimized_verify.py（续）

```
            "pils_holdout_rmse": float(holdout_metrics.loc["optimized_single_target", "rmse"]),
            "pils_holdout_mean_rmse": float(holdout_metrics.loc["non_pils_mean", "rmse"])}


def check_generation(context):
    folder = OUT / "generation"
    meta, x = context["meta"], context["x"]
    summary = BASE.read_json(folder / "summary.json")
    assignments = pd.read_csv(folder / "cv_assignments.csv")
    metrics = pd.read_csv(folder / "cv_metrics.csv")
    inner = pd.read_csv(folder / "inner_scores_outer.csv")
    choices = pd.read_csv(folder / "inner_selection_outer.csv")
    assert not assignments.duplicated(["repeat", "row_index"]).any()
    assert len(assignments) == len(meta) * len(summary["outer_seeds"])
    assert not metrics.duplicated(["repeat", "fold", "style", "method", "mc"]).any()
    expected_n = len(summary["outer_seeds"]) * summary["outer_folds"] * 4 * 5 * summary["mc_repeats"]
    assert len(metrics) == expected_n == summary["cv_records"]
    for repeat, seed in enumerate(summary["outer_seeds"], 1):
        for fold, (train, test) in enumerate(StratifiedKFold(3, shuffle=True, random_state=seed).split(x, meta.tasting_category_fine), 1):
            assigned = assignments[assignments.repeat.eq(repeat) & assignments.fold.eq(fold)]
            assert set(assigned.row_index) == set(test)
            assert assigned.beer.tolist() == assigned.row_index.map(meta.beer).tolist()
            subset = metrics[metrics.repeat.eq(repeat) & metrics.fold.eq(fold)]
            assert subset.train_n.eq(len(train)).all()
            assert subset.generated_n.eq(summary["generated_n_each_draw"]).all()
            assert set(subset.mc) == set(range(summary["mc_repeats"]))
            for style in BASE.GENERATION_STYLES:
                assert subset.loc[subset["style"].eq(style), "holdout_style_n"].eq(
                    meta.tasting_category_fine.iloc[test].eq(style).sum()).all()
    gates = summary["novelty_gates"]
    expected_eligible = (inner.copy_rate.eq(0) & inner.observed_training_copy_rate.eq(0)
                         & inner.nn_ratio.ge(gates["minimum_inner_nn_ratio"])
                         & inner.nearest_equal_feature_fraction.le(gates["maximum_inner_nearest_equal_fraction"]))
    assert inner.eligible.equals(expected_eligible)
    selected_lookup = {}
    for (repeat, fold, style), group in inner.groupby(["outer_repeat", "outer_fold", "style"]):
        assert len(group) == 2 * len(summary["candidate_methods"])
        grouped = group.groupby("method").agg(energy_pca5=("energy_pca5", "mean"),
                                              eligible=("eligible", "all"))
        acceptable = grouped[grouped.eligible]
        selected = acceptable.energy_pca5.idxmin() if len(acceptable) else "legacy_copula"
        actual = choices[choices.repeat.eq(repeat) & choices.fold.eq(fold) & choices["style"].eq(style)]
        assert len(actual) == len(summary["candidate_methods"]) and actual.selected.sum() == 1
        assert actual.loc[actual.selected, "method"].iloc[0] == selected
        BASE.close(actual.set_index("method").loc[grouped.index, "energy_pca5"], grouped.energy_pca5,
                   "Generation training-only inner energy")
        selected_lookup[(repeat, fold, style)] = selected
    folds = pd.read_csv(folder / "cv_fold_means.csv")
```

src/15_q3_optimized_verify.py（续）

```
    keys = ["repeat", "fold", "style", "method"]
    columns = [column for column in folds if column not in keys]
    for row in metrics[metrics.method.eq("nested_selected")].itertuples(index=False):
        method = selected_lookup[(row.repeat, row.fold, row.style)]
        assert row.selected_method == method
        match = metrics[metrics.repeat.eq(row.repeat) & metrics.fold.eq(row.fold)
                        & metrics["style"].eq(row.style) & metrics.mc.eq(row.mc) & met
rics.method.eq(method)]
        assert len(match) == 1
        BASE.close([getattr(row, column) for column in columns], match[columns].iloc[0
],
                   "Nested outer outcome must equal its training-selected candidate")
    expected_fold = metrics.groupby(keys, as_index=False)[columns].mean()
    BASE.same_table(folds, expected_fold, keys, columns, "Optimized generation MC fold
 means")
    expected_summary = expected_fold.groupby(["style", "method"])[columns].agg(["mean"
, "std"])
    expected_summary.columns = [f"{metric}_{stat}" for metric, stat in expected_summar
y.columns]
    expected_summary = expected_summary.reset_index()
    BASE.same_table(pd.read_csv(folder / "cv_summary.csv"), expected_summary, ["style"
, "method"],
                    [column for column in expected_summary if column not in {"style",
"method"}],
                    "Optimized generation summary")
    model = joblib.load(folder / "fullmodel.joblib")
    assert model.columns_ == x.columns.tolist()
    reference = model.fitted_candidates_["legacy_copula"]
    BASE.close(reference.median_, np.nanmedian(x, axis=0), "Full optimized reference m
edians")
    generated_tables, copy_count = [], 0
    for i, style in enumerate(BASE.GENERATION_STYLES):
        generated = pd.read_csv(folder / f"generated_{style.replace('/', '-')}.csv")
        assert len(generated) == summary["generated_n_each_draw"] and generated.sample
_id.is_unique
        assert generated.target_style.eq(style).all()
        fake = generated[x.columns].to_numpy()
        assert np.isfinite(fake).all() and generated[context["calibrated_features"]].g
e(0).all().all()
        selected = model.selected_methods_[style]
        assert selected == summary["full_selected_methods"][style]
        candidate = model.fitted_candidates_[selected]
        support = candidate.empirical_values_ if selected == "legacy_copula" else cand
idate.tables_[style][0]
        for j in range(x.shape[1]):
            assert BASE.supported(fake[:, j], support[j]).all(), f"Unsupported optimiz
ed value: {style}/{x.columns[j]}"
        BASE.close(fake, model.sample(style, len(generated), summary["outer_seeds"][0]
 + 900000 + i),
                   "Saved optimized generation replay")
        raw = x.to_numpy()
        copied = np.any(np.all(np.isclose(fake[:, None, :], raw[None, :, :], rtol=1e-1
0, atol=1e-12)
                               | np.isnan(raw)[None, :, :], axis=2), axis=1)
        copy_count += int(copied.sum())
        assert not copied.any(), f"Observed-field training-row copy generated: {style}
"
        generated_tables.append(generated)
    assert copy_count == 0 and summary["full_generation_observed_copy_rate"] == 0
    context["optimized_generated"] = pd.concat(generated_tables, ignore_index=True)
    return {"cv_records": len(metrics), "inner_records": len(inner),
```

src/15_q3_optimized_verify.py（续）

```
            "inner_choices_recomputed": len(selected_lookup), "generated_rows": sum(map(len, generated_tables)),
            "observed_training_row_copies": copy_count, "full_selected_methods": summary["full_selected_methods"],
            "training_empirical_support": True, "nested_outcomes_match_selected_methods": True}


def check_fixed_stale(context):
    folder = OUT / "stale_fixed"
    meta, x, y = context["meta"], context["x"], context["y"].stale_hops
    manifest = BASE.read_json(folder / "run_manifest.json")
    summary = BASE.read_json(folder / "summary.json")
    assert manifest["parameter_tuning"] is False and manifest["not_independent_external_confirmation"] is True
    oof = pd.read_csv(folder / "oof_predictions.csv")
    rows, fold_rows = [], []
    columns = list(score(y, y))
    for repeat, seed in enumerate(manifest["outer_seeds"], 1):
        fold_ids = np.zeros(len(x), int)
        for fold, (_, test) in enumerate(KFold(5, shuffle=True, random_state=seed).split(x), 1):
            fold_ids[test] = fold
        for method in oof.method.unique():
            group = oof[oof.repeat.eq(repeat) & oof.method.eq(method)].set_index("beer").loc[meta.beer]
            assert len(group) == len(x)
            np.testing.assert_array_equal(group.fold, fold_ids)
            BASE.close(group.observed, y, "Fixed forest OOF labels")
            rows.append({"scope": "per_seed", "repeat": repeat, "method": method,
                         **score(group.observed, group.prediction)})
            for fold in range(1, 6):
                test = fold_ids == fold
                fold_rows.append({"repeat": repeat, "fold": fold, "method": method,
                                  **score(y.loc[test], group.prediction.to_numpy()[test])})
                if method == "mean":
                    BASE.close(group.prediction.to_numpy()[test], y.loc[~test].mean(), "Fixed study fold mean")
    for method in oof.method.unique():
        group = oof[oof.method.eq(method)]
        rows.append({"scope": "pooled_repeated_oof", "repeat": 0, "method": method,
                     **score(group.observed, group.prediction)})
    metrics = pd.read_csv(folder / "metrics.csv")
    BASE.same_table(metrics, pd.DataFrame(rows), ["scope", "repeat", "method"], columns, "Fixed forest OOF metrics")
    BASE.same_table(pd.read_csv(folder / "metrics_by_fold.csv"), pd.DataFrame(fold_rows),
                    ["repeat", "fold", "method"], columns, "Fixed forest fold metrics")
    artifact = joblib.load(folder / "transfer_stale_model.joblib")
    ensemble = joblib.load(folder / "bootstrap_ensemble.joblib")
    train = np.flatnonzero(~meta.tasting_category_fine.eq("Pils/Lager"))
    excluded = np.flatnonzero(meta.tasting_category_fine.eq("Pils/Lager"))
    for saved in [artifact, ensemble]:
        np.testing.assert_array_equal(saved["train_global_indices"], train)
        np.testing.assert_array_equal(saved["excluded_global_indices"], excluded)
        assert saved["features"] == x.columns.tolist() and saved["parameter_tuning"] is False
    inner = pd.read_csv(folder / "transfer_inner_oof.csv")
    np.testing.assert_array_equal(inner.beer, meta.beer.iloc[train])
```

src/15_q3_optimized_verify.py（续）

```
    BASE.close(inner.observed, y.iloc[train], "Fixed transfer inner observed")
    inner_scores = score(inner.observed, inner.fixed_inner_oof_prediction)
    mean = np.empty(len(train))
    for tr, te in KFold(3, shuffle=True, random_state=manifest["transfer_cv_seed"]).split(train):
        mean[te] = y.iloc[train[tr]].mean()
    BASE.close(inner.mean_inner_oof_prediction, mean, "Fixed transfer fold means")
    qualified = bool(inner_scores["prediction_variance_ratio"] >= .05
                     and inner_scores["rmse"] < score(y.iloc[train], mean)["rmse"])
    assert qualified == artifact["qualified_for_proxy"] == ensemble["qualified_for_proxy"]
    indices = np.asarray(ensemble["bootstrap_indices"])
    assert indices.shape == (60, len(train)) and not np.isin(indices, excluded).any()
    assert np.isin(indices, train).all()
    assert ensemble["selection_model_indices"] == list(range(40))
    assert ensemble["audit_model_indices"] == list(range(40, 60))
    assert artifact["pipeline"].named_steps["model"].n_estimators == 300
    all_models = [(artifact["pipeline"], train)] + list(zip(ensemble["models"], indices))
    for i, (model, training_indices) in enumerate(all_models):
        imputer = model.named_steps["imputer"]
        BASE.close(imputer.statistics_, np.nanmedian(x.iloc[training_indices], axis=0), "Fixed model source-excluded medians")
        np.testing.assert_array_equal(imputer.indicator_.features_, np.flatnonzero(x.iloc[training_indices].isna().any(axis=0)))
        tree = model.named_steps["model"]
        assert tree.min_samples_leaf == 3 and tree.max_features == .7
        assert tree.n_estimators == (300 if i == 0 else 120)
    with np.load(folder / "bootstrap_indices.npz") as saved:
        np.testing.assert_array_equal(saved["indices"], indices)
        np.testing.assert_array_equal(saved["bootstrap_seeds"], ensemble["bootstrap_seeds"])
    for i, seed in enumerate(ensemble["bootstrap_seeds"]):
        expected_seed = manifest["selection_bootstrap_seed"] + i if i < 40 else manifest["audit_bootstrap_seed"] + i - 40
        assert seed == expected_seed
        np.testing.assert_array_equal(indices[i], np.random.default_rng(seed).choice(train, len(train), replace=True))
    heldout = pd.read_csv(folder / "pils_style_holdout_predictions.csv")
    BASE.close(heldout.prediction, np.asarray(artifact["pipeline"].predict(x.iloc[excluded])).reshape(-1), "Fixed source holdout")
    holdout = pd.read_csv(folder / "pils_style_holdout_metrics.csv").set_index("model")
    for name, column in [("fixed_single_extra_trees", "prediction"), ("non_pils_mean", "mean_baseline_prediction"),
                         ("old_multioutput_transfer_proxy", "old_multioutput_prediction")]:
        BASE.close(holdout.loc[name, columns], list(score(heldout.observed, heldout[column]).values()), "Fixed holdout metrics")
    context.update({"fixed_stale": artifact, "fixed_ensemble": ensemble,
                    "optimized_train": train, "optimized_sources": excluded})
    pooled = metrics[metrics.scope.eq("pooled_repeated_oof")].set_index("method")
    cluster = BASE.read_json(folder / "paired_cluster_bootstrap.json")
    errors = oof.assign(error_squared=(oof.prediction - oof.observed) ** 2)
    per_beer = errors.pivot_table(index="beer", columns="method", values="error_squared", aggfunc="mean").sort_index()
    fixed_errors = per_beer.fixed_single_extra_trees.to_numpy()
    old_errors = per_beer.old_multioutput_extra_trees.to_numpy()
    draws = np.random.default_rng(cluster["seed"]).integers(0, len(per_beer), (cluster["repeats"], len(per_beer)))
    fixed_rmse = np.sqrt(fixed_errors[draws].mean(axis=1))
```

src/15_q3_optimized_verify.py（续）

```
    old_rmse = np.sqrt(old_errors[draws].mean(axis=1))
    differences, reductions = fixed_rmse - old_rmse, 1 - fixed_rmse / old_rmse
    BASE.close(cluster["rmse_difference_percentile95"], np.quantile(differences, [.025, .975]), "Beer-cluster RMSE difference interval")
    BASE.close(cluster["relative_rmse_reduction_percentile95"], np.quantile(reductions, [.025, .975]), "Beer-cluster relative change interval")
    BASE.close(cluster["bootstrap_fraction_rmse_difference_below_zero"], np.mean(differences < 0), "Beer-cluster difference fraction")
    BASE.close(cluster["point_rmse_difference"], np.sqrt(fixed_errors.mean()) - np.sqrt(old_errors.mean()), "Beer-cluster point difference")
    return {"extra_internal_seeds": manifest["outer_seeds"], "parameter_tuning": False,
            "fixed_single_oof_rmse": float(pooled.loc["fixed_single_extra_trees", "rmse"]),
            "same_split_old_multioutput_rmse": float(pooled.loc["old_multioutput_extra_trees", "rmse"]),
            "same_split_mean_rmse": float(pooled.loc["mean", "rmse"]),
            "transfer_qualified": qualified, "bootstrap_shape": list(indices.shape),
            "beer_cluster_bootstrap_repeats": cluster["repeats"],
            "rmse_difference_interval": cluster["rmse_difference_percentile95"],
            "pils_holdout_rmse": float(holdout.loc["fixed_single_extra_trees", "rmse"]),
            "pils_holdout_mean_rmse": float(holdout.loc["non_pils_mean", "rmse"])}


def check_transfer(context):
    folder = OUT / "transfer"
    meta, x = context["meta"], context["x"]
    train, sources = context["optimized_train"], context["optimized_sources"]
    summary = BASE.read_json(folder / "summary.json")
    scenarios = BASE.read_json(folder / "threshold_scenarios.json")
    thresholds = pd.read_excel(data_dir() / "8.xlsx", sheet_name="Sensory thresholds").set_index("field_name_R")
    assert len(scenarios) == 10
    expected_ids = {f"{statistic}_x{factor:.1f}" for statistic in ["median", "mean"] for factor in [.8, .9, 1, 1.1, 1.2]}
    assert {scenario["scenario"] for scenario in scenarios} == expected_ids
    for scenario in scenarios:
        for field in BASE.DEFECT_FIELDS:
            raw = thresholds.at[field, "threshold_" + scenario["statistic"]]
            actual = scenario["thresholds"][field]
            if pd.isna(raw):
                assert actual is None and field == "X.E..2.nonenal"
            else:
                BASE.close(actual, raw * scenario["factor"], "Threshold stress scenario")
    assert scenarios == summary["threshold_scenarios"]
    geometry = joblib.load(SOLUTION / "outputs/q3/transfer/transfer_copula.joblib")
    reference = geometry["copula"]
    np.testing.assert_array_equal(geometry["training_rows"], train)
    generator_artifact = joblib.load(folder / "transfer_generator.joblib")
    np.testing.assert_array_equal(generator_artifact["training_rows"], train)
    np.testing.assert_array_equal(generator_artifact["source_rows"], sources)
    generator = generator_artifact["generator"]
    BASE.close(generator.fitted_candidates_["legacy_copula"].median_, np.nanmedian(x.iloc[train], axis=0), "Source-excluded new generator medians")
    old_draws = pd.read_csv(folder / "old_base_draws.csv")
    new_draws = pd.read_csv(folder / "optimized_base_draws.csv")
    assert old_draws.columns.tolist() == new_draws.columns.tolist() == x.columns.tolist()
    BASE.close(old_draws, reference.sample("Tripel", len(old_draws), 20261023 + 301),
```

src/15_q3_optimized_verify.py（续）

```
    "Frozen old generator base draws")
440     BASE.close(new_draws, generator.sample("Tripel", len(new_draws), 20261023 + 301),
    "Source-excluded new generator base draws")
441     assert bool(np.array_equal(old_draws.to_numpy(), new_draws.to_numpy())) == summary
    ["base_draws_equal"]
442     report = pd.read_csv(folder / "transfer_summary.csv")
443     diagnostics = pd.read_csv(folder / "candidate_diagnostics.csv")
444     spectra = pd.read_csv(folder / "transferred_spectra.csv").set_index("transfer_id")
445     paired = pd.read_csv(folder / "stale_paired_deltas.csv")
446     sensitivity = pd.read_csv(folder / "threshold_sensitivity.csv")
447     profiles = pd.read_csv(folder / "sensory_before_after.csv")
448     policies = pd.read_csv(folder / "policy_summary.csv")
449     ensemble = context["fixed_ensemble"]
450     old_ensemble = joblib.load(SOLUTION / "outputs/q3/sensory/stale_bootstrap.joblib")
451     profile_model = joblib.load(SOLUTION / "outputs/q3/sensory/transfer_proxy.joblib")
452     target = x.loc[meta.tasting_category_fine.eq("Tripel")]
453     tz = reference.transform(target)
454     dimension = x.shape[1]
455     neighbors = cdist(tz, tz) / np.sqrt(dimension)
456     np.fill_diagonal(neighbors, np.inf)
457     support_limit = np.quantile(neighbors.min(axis=1), .95)
458     BASE.close(support_limit, summary["support_nn_q95"], "Unchanged support limit")
459     assert summary["target_improvement_minimum"] == .1 and summary["latent_rms_change_
    minimum"] == .05
460     assert summary["selection_models"] == 40 and summary["audit_models"] == 20
461     medians = x.iloc[train].median()
462     minimum, maximum = x.iloc[train].min().to_numpy(), x.iloc[train].max().to_numpy()
463     target_median = target.median().to_numpy()
464     p_selected = report[report.candidate_index.notna()]
465     assert set(p_selected.transfer_id) == set(spectra.index) and spectra.index.is_uniq
    ue
466     selected_predictions = np.vstack([model.predict(spectra[x.columns]) for model in e
    nsemble["models"]])
467     old_predictions = np.vstack([model.predict(spectra[x.columns]) for model in old_en
    semble["models"]])
468     selected_profiles = profile_model["pipeline"].predict(spectra[x.columns])
469     max_quantile_error, argmin_checks, scenario_checks = 0.0, 0, 0
470     for source_index in sources:
471         beer = int(meta.beer.iloc[source_index])
472         raw_source = x.iloc[[source_index]]
473         source = raw_source.fillna(medians)
474         caps = {}
475         for field in BASE.DEFECT_FIELDS:
476             levels = [scenario["thresholds"][field] for scenario in scenarios if scena
    rio["thresholds"][field] is not None]
477             value = float(raw_source[field].iloc[0])
478             caps[field] = max(value, min(levels)) if levels else value
479             if levels:
480                 BASE.close(caps[field], min(max(value, level) for level in levels), "R
    obust minimum-of-scenario cap")
481         arrays, families, gammas, draws = [], [], [], []
482         for family, base in [("old_copula", old_draws), ("optimized", new_draws)]:
483             for gamma in [.25, .5, .75, 1.]:
484                 arrays.append((1-gamma) * source.to_numpy()[0] + gamma * base.to_numpy
    ())
485                 families.extend([family] * len(base)); gammas.extend([gamma] * len(bas
    e)); draws.extend(range(len(base)))
486         for gamma in [.25, .5, .75, 1.]:
487             arrays.append(((1-gamma) * source.to_numpy()[0] + gamma * target_median)[N
    one])
488             families.append("shared_target_median"); gammas.append(gamma); draws.appen
```

```
    d(-1)
489         values = pd.DataFrame(np.vstack(arrays), columns=x.columns)
490         for field, cap in caps.items():
491             values[field] = values[field].clip(lower=0, upper=cap)
492         diagnostic = diagnostics[diagnostics.source_beer.eq(beer)].sort_values("candid
    ate_index")
493         assert len(diagnostic) == len(values)
494         np.testing.assert_array_equal(diagnostic.candidate_index, np.arange(len(values
    )))
495         assert diagnostic.family.tolist() == families
496         BASE.close(diagnostic.gamma, gammas, "Reconstructed interpolation fractions")
497         np.testing.assert_array_equal(diagnostic.draw, draws)
498         z, sz = reference.transform(values), reference.transform(source)
499         distance, before = reference.distance(values, "Tripel"), float(reference.dista
    nce(source, "Tripel")[0])
500         nn = cdist(z, tz).min(axis=1) / np.sqrt(dimension)
501         change = np.linalg.norm(z - sz, axis=1) / np.sqrt(dimension)
502         improvement = 1 - distance / max(before, 1e-8)
503         support = (nn <= support_limit + TOL) & (values.to_numpy() >= minimum-TOL).all
    (axis=1) & (values.to_numpy() <= maximum+TOL).all(axis=1)
504         chemical = BASE.chemical_flags(values, caps)
505         source_pred = np.array([float(np.asarray(model.predict(source)).reshape(-1)[0]
    ) for model in ensemble["models"]])
506         delta = np.column_stack([np.asarray(model.predict(values)).reshape(-1)-source_
    pred[b] for b, model in enumerate(ensemble["models"][:40])])
507         q95 = np.quantile(delta, .95, axis=1)
508         nonzero = np.any(np.abs(delta) > 1e-8, axis=1)
509         feasible = support & chemical & (change >= .05) & (improvement >= .1) & (q95 <
    = TOL) & nonzero
510         BASE.close(diagnostic[["target_distance", "target_improvement", "target_nn_rms
    ", "change_latent_rms", "selection_q95"]],
511                    np.column_stack([distance, improvement, nn, change, q95]), "Indepen
    dently rebuilt candidate geometry and screening quantiles")
512         np.testing.assert_array_equal(diagnostic.selection_feasible, feasible)
513         np.testing.assert_array_equal(diagnostic.support_ok, support)
514         np.testing.assert_array_equal(diagnostic.all_thresholds_chemical_ok, chemical)
515         np.testing.assert_array_equal(diagnostic.nonzero_paired_signal, nonzero)
516         center_scale = max(float(np.linalg.norm(sz[0] - tz.mean(axis=0)) / np.sqrt(dim
    ension)), 1e-8)
517         masks = {"old_generator": np.isin(families, ["old_copula", "shared_target_medi
    an"]),
518                  "optimized_generator": np.isin(families, ["optimized", "shared_target
    _median"]),
519                  "combined_pool": np.ones(len(values), bool)}
520         old_source_pred = np.array([float(np.asarray(model.predict(source)).reshape(-1
    )[0]) for model in old_ensemble["models"]])
521         for _, row in report[report.source_beer.eq(beer)].iterrows():
522             mask = masks[row.pool]
523             allowed = feasible & mask
524             assert row.n_candidates == mask.sum() and row.n_selection_feasible == allo
    wed.sum()
525             variation = np.std(delta[mask].mean(axis=1))
526             zero_fraction = np.mean(np.all(np.abs(delta[mask]) <= 1e-8, axis=1))
527             BASE.close([row.candidate_mean_delta_sd, row.candidate_zero_paired_fractio
    n], [variation, zero_fraction], "Proxy responsiveness diagnostics")
528             if not allowed.any():
529                 assert pd.isna(row.candidate_index) and row.status == "no_candidate_fo
    und"
530                 continue
531             objective = distance / max(before, 1e-8) + row["lambda"] * change / center
    _scale
```

```
            chosen = int(np.flatnonzero(allowed)[np.argmin(objective[allowed])])
            assert chosen == int(row.candidate_index), "Candidate differs from fixed s
election-only argmin"
            argmin_checks += 1
            final = spectra.loc[[row.transfer_id], x.columns]
            BASE.close(final, values.iloc[[chosen]], "Selected spectrum reconstructed
from base draws")
            saved_position = spectra.index.get_loc(row.transfer_id)
            all_delta = selected_predictions[:, saved_position]-source_pred
            old_delta = old_predictions[:, saved_position]-old_source_pred
            quantities = [np.quantile(all_delta[:40], .95), np.quantile(all_delta[40:]
, .95),
                          np.quantile(old_delta[:30], .95), np.quantile(old_delta[30:]
, .95)]
            actual = [row.selection_q95, row.audit_q95, row.old_selection_q95, row.old
_audit_q95]
            BASE.close(actual, quantities, "Selected and frozen-architecture audit qua
ntiles")
            max_quantile_error = max(max_quantile_error, float(np.max(np.abs(np.asarra
y(actual)-quantities))))
            informative = bool(variation > 1e-6 and nonzero[chosen])
            qualified = bool(context["fixed_stale"]["qualified_for_proxy"])
            passed = bool(qualified and informative and quantities[1] <= TOL)
            old_consensus = bool(quantities[2] <= TOL and quantities[3] <= TOL)
            assert passed == row.all_model_checks_passed and old_consensus == row.old_
proxy_consensus
            assert bool(passed and old_consensus) == row.cross_proxy_checks_passed
            expected_status = "proxy_not_qualified" if not qualified else "proxy_uninf
ormative" if not informative else "model_checks_passed" if quantities[1] <= TOL else "
resampling_audit_failed"
            assert row.status == expected_status
            for label, differences in [("selection", all_delta[:40]), ("audit", all_de
lta[40:]),
                                       ("old_selection", old_delta[:30]), ("old_audit"
, old_delta[30:])]:
                recorded = paired[paired.transfer_id.eq(row.transfer_id) & paired.ense
mble.eq(label)].sort_values("model_index")
                np.testing.assert_array_equal(recorded.model_index, np.arange(len(diff
erences)))
                BASE.close(recorded.paired_delta, differences, "Per-model screening/au
dit paired differences")
            check_rows = sensitivity[sensitivity.transfer_id.eq(row.transfer_id)]
            assert len(check_rows) == len(scenarios) * len(BASE.DEFECT_FIELDS)
            for check in check_rows.itertuples(index=False):
                scenario = next(item for item in scenarios if item["scenario"] == chec
k.scenario)
                threshold = scenario["thresholds"][check.feature]
                original, transferred = float(raw_source[check.feature].iloc[0]), floa
t(final[check.feature].iloc[0])
                cap = max(original, threshold) if threshold is not None else original
                BASE.close([check.source, check.transferred, check.upper_bound, check.
robust_upper_bound],
                           [original, transferred, cap, caps[check.feature]], "All-sce
nario concentration audit")
                assert bool(0 <= transferred <= cap + TOL) == check.passed
                scenario_checks += 1
            source_profile = profile_model["pipeline"].predict(source)[0]
            after_profile = selected_profiles[saved_position].copy()
            position = context["y"].columns.get_loc("stale_hops")
            source_profile[position] = float(np.asarray(context["fixed_stale"]["pipeli
ne"].predict(source)).reshape(-1)[0])
```

src/15_q3_optimized_verify.py（续）

```
            after_profile[position] = float(np.asarray(context["fixed_stale"]["pipelin
e"].predict(final)).reshape(-1)[0])
            for phase, profile in [("source", source_profile), ("transferred", after_p
rofile)]:
                recorded = profiles[profiles.transfer_id.eq(row.transfer_id) & profile
s.phase.eq(phase)]
                assert len(recorded) == 1
                BASE.close(recorded[context["y"].columns], profile[None], "Mixed-model
 sensory profile")
        print(f"Verified robust candidates: source {beer}", flush=True)
    expected_policies = []
    for (pool, weight), group in report.groupby(["pool", "lambda"]):
        expected_policies.append({"pool": pool, "lambda": weight, "sources": len(group
),
                                  "selected": int(group.candidate_index.notna().sum()),
                                  "model_checks_passed": int(group.all_model_checks_pas
sed.sum()),
                                  "cross_proxy_checks_passed": int(group.cross_proxy_ch
ecks_passed.sum()),
                                  "audit_failed": int(group.status.eq("resampling_audit
_failed").sum()),
                                  "no_candidate": int(group.status.eq("no_candidate_fou
nd").sum())})
    BASE.same_table(policies, pd.DataFrame(expected_policies), ["pool", "lambda"],
                    ["sources", "selected", "model_checks_passed", "cross_proxy_checks
_passed", "audit_failed", "no_candidate"], "Robust policy summary")
    default = report[report.is_default_policy]
    assert default.pool.eq(summary["default_pool"]).all() and default["lambda"].eq(sum
mary["default_lambda"]).all()
    counts = {"sources": len(default), "selected": int(default.candidate_index.notna()
.sum()),
              "model_checks_passed_all_thresholds": int(default.all_model_checks_passe
d.sum()),
              "cross_proxy_checks_passed_all_thresholds": int(default.cross_proxy_chec
ks_passed.sum()),
              "audit_failed": int(default.status.eq("resampling_audit_failed").sum()),
              "no_candidate": int(default.status.eq("no_candidate_found").sum())}
    assert counts == summary["default_policy"]
    archived = pd.read_csv(folder / "archived_threshold_recheck.csv")
    old_spectra = pd.read_csv(SOLUTION / "outputs/q3/transfer/transferred_spectra.csv"
).set_index("transfer_id")
    for row in archived.itertuples(index=False):
        source = x.loc[meta.beer.eq(row.source_beer)]
        selected = old_spectra.loc[[row.transfer_id], x.columns]
        scenario = next(item for item in scenarios if item["scenario"] == row.scenario
)
        caps = {field: max(float(source[field].iloc[0]), level) if level is not None e
lse float(source[field].iloc[0])
                for field, level in scenario["thresholds"].items()}
        assert bool(BASE.chemical_flags(selected, caps)[0]) == row.chemical_passed
    original_passed = archived[archived.old_model_checks_passed].groupby("source_beer"
).chemical_passed.all()
    assert int(original_passed.sum()) == summary["archived_baseline"]["model_checks_pa
ssed_all_thresholds_same_spectra"]
    return {"scenario_count": len(scenarios), "sources": len(sources), "reconstructed_
candidates": len(diagnostics),
            "fixed_selection_argmin_checks": argmin_checks, "concentration_scenario_ch
ecks": scenario_checks,
            "maximum_paired_quantile_error": max_quantile_error,
            "base_draws_equal": summary["base_draws_equal"], "default_policy": counts}

```

src/15_q3_optimized_verify.py（续）

```
613
614 def check_report(context):
615     generated = context["optimized_generated"]
616     meta, x, y = context["meta"], context["x"], context["y"]
617     profiles = pd.read_csv(OUT / "generated_sensory_profiles.csv")
618     assert len(profiles) == len(generated) == 1200 and profiles.sample_id.is_unique
619     assert set(profiles.sample_id) == set(generated.sample_id)
620     old = joblib.load(SOLUTION / "outputs/q3/sensory/sensory_model.joblib")
621     expected = old["pipeline"].predict(generated[x.columns])
622     expected[:, y.columns.get_loc("stale_hops")] = np.asarray(
623         context["fixed_stale"]["pipeline"].predict(generated[x.columns])).reshape(-1)
624     aligned = profiles.set_index("sample_id").loc[generated.sample_id]
625     assert aligned.target_style.tolist() == generated.target_style.tolist()
626     BASE.close(aligned[y.columns], expected, "Optimized generated 44+1 sensory profile
    s")
627     summary_rows = []
628     for style in BASE.GENERATION_STYLES:
629         positions = generated.target_style.eq(style).to_numpy()
630         for j, target in enumerate(y):
631             values = expected[positions, j]
632             summary_rows.append({"style": style, "target": target,
633                                  "predicted_mean": values.mean(), "predicted_p05": np.
    quantile(values, .05),
634                                  "predicted_p95": np.quantile(values, .95),
635                                  "real_observed_mean": y.loc[meta.tasting_category_fin
    e.eq(style), target].mean()})
636     sensory_summary = pd.read_csv(OUT / "generated_sensory_summary.csv")
637     BASE.same_table(sensory_summary, pd.DataFrame(summary_rows), ["style", "target"],
638                     ["predicted_mean", "predicted_p05", "predicted_p95", "real_observe
    d_mean"],
639                     "Optimized generated sensory summary")
640     assert sensory_summary.scale.eq("attachment_4_standardized").all()
641     assert sensory_summary.interval_meaning.eq("generated_spectrum_spread_not_confiden
    ce_interval").all()
642     folder = OUT / "transfer"
643     report = pd.read_csv(folder / "transfer_summary.csv")
644     default = report[report.is_default_policy]
645     current = pd.read_csv(folder / "threshold_sensitivity.csv")
646     archived = pd.read_csv(folder / "archived_threshold_recheck.csv")
647     expected_counts = []
648     for scenario in BASE.read_json(folder / "threshold_scenarios.json"):
649         name = scenario["scenario"]
650         old_rows = archived[archived.scenario.eq(name)]
651         passing = current[current.scenario.eq(name)].groupby("transfer_id").passed.all
    ()
652         chemical = default.transfer_id.map(passing).fillna(False)
653         expected_counts.append({"scenario": name, "statistic": scenario["statistic"],
    "factor": scenario["factor"],
654                                 "old_model_checks_and_chemical": int((old_rows.old_mod
    el_checks_passed & old_rows.chemical_passed).sum()),
655                                 "new_chemical_only": int(chemical.sum()),
656                                 "new_model_checks_and_chemical": int((chemical & defau
    lt.all_model_checks_passed).sum()),
657                                 "cross_proxy_checks_and_chemical": int((chemical & def
    ault.cross_proxy_checks_passed).sum())})
658     BASE.same_table(pd.read_csv(folder / "threshold_pass_counts.csv"), pd.DataFrame(ex
    pected_counts), ["scenario"],
659                     ["factor", "old_model_checks_and_chemical", "new_chemical_only", "
    new_model_checks_and_chemical", "cross_proxy_checks_and_chemical"],
660                     "All-scenario pass-count report")
661     results = BASE.read_json(OUT / "results_summary.json")
```

src/15_q3_optimized_verify.py（续）

```
    cv = pd.read_csv(OUT / "generation/cv_summary.csv")
    gains = {}
    for style in BASE.GENERATION_STYLES:
        rows = cv[cv["style"].eq(style)].set_index("method")
        gains[style] = float(1 - rows.loc["nested_selected", "energy_pca5_mean"] / rows.loc["legacy_copula", "energy_pca5_mean"])
        BASE.close(results["generation_energy_reduction_vs_matched_legacy"][style], gains[style], "Generation report gains")
    fixed = BASE.read_json(OUT / "stale_fixed/summary.json")
    transfer = BASE.read_json(folder / "summary.json")
    assert results["default_transfer"] == transfer["default_policy"]
    assert results["base_draws_equal"] == transfer["base_draws_equal"]
    assert results["source_style_prediction_improved"] == fixed["pils_holdout_improves_old"]
    BASE.close(results["fixed_stale_rmse_reduction_vs_matched_old"], fixed["rmse_reduction_vs_old_multioutput"], "Fixed-proxy report change")
    BASE.close(results["paired_rmse_difference_interval"], fixed["paired_cluster_bootstrap"]["rmse_difference_percentile95"], "Reported paired uncertainty interval")
    text = (SOLUTION / "docs/q3_optimization_results.md").read_text(encoding="utf-8")
    assert "尚未证实稳定的预测提升" in text
    assert f"**{transfer['default_policy']['model_checks_passed_all_thresholds']}/11**" in text
    assert f"**{transfer['default_policy']['cross_proxy_checks_passed_all_thresholds']}/11**" in text
    for style in ["Kriek", "Lambic"]:
        assert f"{100*gains[style]:.1f}%" in text
    figures = ["generation_and_prediction", "threshold_stability", "sensory_fingerprint_comparison"]
    for name in figures:
        for suffix in ["png", "svg"]:
            path = OUT / "figures" / f"{name}.{suffix}"
            assert path.is_file() and path.stat().st_size > 100
    return {"generated_sensory_rows": len(profiles), "sensory_targets": len(y.columns),
            "profile_model": "44 original full-data outputs plus source-excluded fixed stale target",
            "scenario_count_rows": len(expected_counts), "summary_values_recomputed": True,
            "figures_with_png_and_svg": len(figures), "results_report_exists": True}


def main():
    start = perf_counter()
    result = {"status": "running", "checks": [],
              "scope": "Frozen baseline integrity, repeated internal validation and robust candidate model checks"}
    active = "preserved_baseline_before"
    try:
        result["checks"].append({"name": active, "details": check_baseline()})
        active = "raw_sources_and_hybrid_contract"
        provenance = BASE.check_hashes()
        context, contract = BASE.load_contract()
        result["checks"].append({"name": active, "details": {"sources": provenance, "contract": contract}})
        for active, operation in [("optimized_stale_nested_validation", check_stale),
                                  ("optimized_generation_nested_validation", check_generation),
                                  ("fixed_single_target_followup", check_fixed_stale),
                                  ("robust_threshold_transfer", check_transfer),
                                  ("report_profiles_and_threshold_counts", check_report)]:
```

src/15_q3_optimized_verify.py（续）

```
            details = operation(context)
            result["checks"].append({"name": active, "details": details})
            print(f"Verified: {active}", flush=True)
        active = "preserved_baseline_after"
        result["checks"].append({"name": active, "details": check_baseline()})
        result["status"] = "passed"
        result["limitations"] = [
            "Repeated and nested validation reuse the same real samples; new seeds are not fresh independent data.",
            "Bootstrap audit shares real training data and does not provide calibrated post-search coverage.",
            "Numerical and chemical checks do not establish actual sensory improvement or brewing feasibility.",
        ]
    except Exception as error:
        result.update({"status": "failed", "failed_check": active,
                       "error": f"{type(error).__name__}: {error}"})
        raise
    finally:
        result["elapsed_seconds"] = perf_counter() - start
        (OUT / "verification.json").write_text(json.dumps(result, ensure_ascii=False, indent=2,
                                                          allow_nan=False) + "\n", encoding="utf-8", newline="\n")
        print(json.dumps(result, ensure_ascii=False, indent=2, allow_nan=False), flush=True)


if __name__ == "__main__":
    with threadpool_limits(limits=1):
        main()
```

src/16_q3_stale_domain.py

SHA-256: 65bb6832fc558142c609129009b595681fd6a074a94f327b579cac0e85c92122

```
"""Non-Pils leave-one-style-out selection and repeatedly viewed Pils diagnosis."""
from __future__ import annotations

import json
import platform
import warnings
from time import perf_counter

import joblib
import numpy as np
import pandas as pd
import sklearn
from scipy.stats import ConstantInputWarning
from scipy.spatial.distance import cdist
from sklearn.model_selection import LeaveOneGroupOut
from threadpoolctl import threadpool_limits

from common import SOLUTION, data_dir
from q3_data import load_q3_data
from q3_stale_optimized import predict_1d, scores as regression_scores
from q3_stale_domain import (
    AUDIT_BOOTSTRAP_SEED, BIAS_PENALTY, FINAL_SEED, LOSO_SEED,
    SELECTION_BOOTSTRAP_SEED, VARIANCE_RATIO_MIN, domain_candidate_specs,
    make_domain_pipeline, training_fit_params,
)

OUT = SOLUTION / "outputs" / "q3_domain" / "stale"
TARGET = "stale_hops"


def scores(y, pred):
    # A few tiny styles have constant labels: correlation is undefined, not 0.
    with warnings.catch_warnings():
        warnings.simplefilter("ignore", ConstantInputWarning)
        result = regression_scores(y, pred)
    if result["spearman"] is not None and not np.isfinite(result["spearman"]):
        result["spearman"] = None
    return result


def write_json(path, value):
    path.write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False),
encoding="utf-8", newline="\n")


def predict_target(model, x, spec, target_position):
    if spec["family"] == "old_multioutput":
        return np.asarray(model.predict(x))[:, target_position]
    return predict_1d(model, x)


def write_local_neighborhood_diagnostics(final_model, models, indices, train_indices,
                                         source_indices, meta, x):
    """Feature-only bootstrap neighbourhood checks, never used for selection."""
    xs = x.iloc[source_indices]
    final_regressor = final_model.named_steps["model"]
    k = final_regressor.n_neighbors
    final_distances = cdist(final_model[:-1].transform(xs), final_regressor.x_)
    final_neighbors = [set(train_indices[np.argpartition(row, k-1)[:k]]) for row in fi
```

src/16_q3_stale_domain.py（续）

```
nal_distances]
    rows = []
    for b, (model, index) in enumerate(zip(models, indices)):
        regressor = model.named_steps["model"]
        distances = cdist(model[:-1].transform(xs), regressor.x_)
        prediction = predict_1d(model, xs)
        for j, (source_index, row) in enumerate(zip(source_indices, distances)):
            near = np.argpartition(row, k-1)[:k]
            neighbors = set(index[near])
            weights = regressor.sample_weight_[near] / np.maximum(row[near], 1e-12)
            # Repeated resample positions are not independent source beers.
            # Keep the original position-level Kish count and also aggregate
            # all weights belonging to the same original beer before Kish.
            _, beer_inverse = np.unique(index[near], return_inverse=True)
            weights_by_beer = np.bincount(beer_inverse, weights=weights)
            effective_positions = float(weights.sum()**2/np.sum(weights**2))
            effective_beers = float(weights_by_beer.sum()**2/np.sum(weights_by_beer**2
))
            if effective_beers > len(neighbors)+1e-10:
                raise AssertionError("Beer-level effective count exceeds unique neighb
ours")
            rows.append({"beer": int(meta.iloc[source_index].beer), "model_index": b,
                "role": "selection" if b < 40 else "audit", "prediction": prediction[j
],
                "nearest_unique_beers": len(neighbors),
                "neighbor_jaccard_vs_final": len(neighbors & final_neighbors[j])/len(n
eighbors | final_neighbors[j]),
                "effective_resampled_position_count": effective_positions,
                "effective_unique_beer_count": effective_beers,
                "imputer_indicator_count": len(model.named_steps["imputer"].indicator_
.features_)})
    frame = pd.DataFrame(rows)
    frame.to_csv(OUT / "neighborhood_bootstrap_diagnostics.csv", index=False, lineterm
inator="\n")
    frame.groupby("beer").agg(
        bootstrap_prediction_sd=("prediction", "std"),
        median_neighbor_jaccard=("neighbor_jaccard_vs_final", "median"),
        mean_unique_neighbors=("nearest_unique_beers", "mean"),
        median_effective_resampled_position_count=("effective_resampled_position_count
", "median"),
        median_effective_unique_beer_count=("effective_unique_beer_count", "median"),
    ).to_csv(OUT / "neighborhood_bootstrap_summary.csv", lineterminator="\n")


def main():
    started = perf_counter()
    OUT.mkdir(parents=True, exist_ok=True)
    meta, x, y45, manifest = load_q3_data()
    y, target_position = y45[TARGET], y45.columns.get_loc(TARGET)
    source_indices = np.flatnonzero(meta.tasting_category_fine.eq("Pils/Lager"))
    train_indices = np.flatnonzero(~meta.tasting_category_fine.eq("Pils/Lager"))
    assert len(source_indices) == 11 and len(train_indices) == 239
    xtr, ytr = x.iloc[train_indices], y.iloc[train_indices]
    groups = meta.iloc[train_indices].tasting_category_fine.to_numpy()
    features = x.columns.tolist()
    calibrated = manifest.loc[manifest.representation.eq("calibrated"), "feature"].tol
ist()
    threshold_metadata = pd.read_excel(data_dir() / "8.xlsx", sheet_name="Sensory thre
sholds")
    aging_set = set(threshold_metadata.loc[threshold_metadata.aging.notna(), "field_na
me_R"])
```

src/16_q3_stale_domain.py（续）

```
    aging = [feature for feature in features if feature in aging_set]
    specs = domain_candidate_specs()
    spec_map = {spec["name"]: spec for spec in specs}
    run_manifest = {
        "candidate_specs": specs, "target": TARGET, "features": features, "aging_features": aging,
        "population": "239 non-Pils samples", "heldout_groups": sorted(np.unique(groups).tolist()),
        "validation": "complete leave-one-style-out; 21 folds; every preprocessing operation fitted per training fold",
        "selection_rule": "minimum macro(style RMSE) + 0.25*macro(abs(style mean prediction error)) among selectable nonconstant candidates",
        "bias_penalty": BIAS_PENALTY, "variance_ratio_min": VARIANCE_RATIO_MIN,
        "qualification_rule": "LOSO prediction variance ratio >=0.05 and macro RMSE < better of global/style-balanced mean baselines",
        "fallback": "if every selectable candidate fails nonconstant gate, retain minimum score but mark unqualified",
        "loso_seed": LOSO_SEED, "final_seed": FINAL_SEED,
        "selection_bootstrap_seed": SELECTION_BOOTSTRAP_SEED, "audit_bootstrap_seed": AUDIT_BOOTSTRAP_SEED,
        "pils_labels_used_in_training_selection_or_intercept": False,
        "pils_status": "previously repeatedly viewed diagnostic set; not a new test set",
        "forbidden_prediction_features": ["style", "identity", "other observed sensory targets", "attachment 7 comments"],
    }
    # Freeze all candidates/rules before starting the new grouped evaluation.
    write_json(OUT / "run_manifest.json", run_manifest)
    pd.DataFrame({"feature": aging}).to_csv(OUT / "predeclared_aging_features.csv", index=False, lineterminator="\n")
    predictions = {spec["name"]: np.full(len(train_indices), np.nan) for spec in specs}
    fold_ids = np.zeros(len(train_indices), dtype=int)
    rows_by_style, fold_audit = [], []
    for fold, (train_local, test_local) in enumerate(LeaveOneGroupOut().split(xtr, ytr, groups), 1):
        heldout_style = str(groups[test_local[0]])
        train_global, test_global = train_indices[train_local], train_indices[test_local]
        assert not set(train_global) & set(source_indices)
        assert not set(groups[train_local]) & {heldout_style}
        fold_seed = LOSO_SEED + fold
        for spec in specs:
            name = spec["name"]
            if spec["family"] == "ensemble":
                values = np.mean([predictions[member["name"]][test_local] for member in spec["members"]], axis=0)
            else:
                model = make_domain_pipeline(spec, features, calibrated, aging, fold_seed)
                target = y45.iloc[train_global] if spec["family"] == "old_multioutput" else y.iloc[train_global]
                model.fit(x.iloc[train_global], target,
                          **training_fit_params(spec, meta.iloc[train_global].tasting_category_fine))
                values = predict_target(model, x.iloc[test_global], spec, target_position)
            predictions[name][test_local] = values
            rows_by_style.append({"method": name, "heldout_style": heldout_style,
                                  "fold": fold, "n": len(test_local),
                                  **scores(y.iloc[test_global], values),
```

src/16_q3_stale_domain.py（续）

```
                                    "bias": float(np.mean(values-y.iloc[test_global].to_numpy()))})
        fold_ids[test_local] = fold
        fold_audit.extend({"fold": fold, "heldout_style": heldout_style, "role": role,
                           "global_index": int(index), "beer": int(meta.iloc[index].beer)}
                          for role, indices in [("train", train_global), ("test", test_global)] for index in indices)
        pd.DataFrame(rows_by_style).to_csv(OUT / "loso_metrics_by_style.csv", index=False, lineterminator="\n")
        print(f"Stale domain LOSO {fold}/21: {heldout_style} ({len(test_local)} samples)", flush=True)
    loso_by_style = pd.DataFrame(rows_by_style)
    audit = pd.DataFrame(fold_audit)
    audit.to_csv(OUT / "loso_split_audit.csv", index=False, lineterminator="\n")
    overall_rows, prediction_rows = [], []
    for spec in specs:
        name, pred = spec["name"], predictions[spec["name"]]
        assert np.isfinite(pred).all()
        per_style = loso_by_style[loso_by_style.method.eq(name)]
        metric = scores(ytr, pred)
        macro_rmse = float(per_style.rmse.mean())
        macro_abs_bias = float(per_style.bias.abs().mean())
        within_pred_ss = within_truth_ss = 0.0
        for style in np.unique(groups):
            mask = groups == style
            within_pred_ss += float(np.sum((pred[mask]-pred[mask].mean())**2))
            within_truth_ss += float(np.sum((ytr.to_numpy()[mask]-ytr.to_numpy()[mask].mean())**2))
        overall_rows.append({"method": name, "selectable": spec["selectable"],
            "balanced": spec["balanced"], "family": spec["family"], "styles": len(per_style),
            "macro_style_rmse": macro_rmse, "macro_style_mae": float(per_style.mae.mean()),
            "macro_style_abs_bias": macro_abs_bias, "macro_style_bias": float(per_style.bias.mean()),
            "selection_score": macro_rmse+BIAS_PENALTY*macro_abs_bias,
            "nonconstant_qualified": metric["prediction_variance_ratio"] >= VARIANCE_RATIO_MIN,
            "within_style_prediction_variance_ratio": within_pred_ss/within_truth_ss,
            **metric})
        prediction_rows.append(pd.concat([meta.iloc[train_indices].reset_index(drop=True),
            pd.DataFrame({"global_index": train_indices, "method": name, "observed": ytr.to_numpy(),
                          "prediction": pred, "heldout_style": groups, "fold": fold_ids})], axis=1))
    overall = pd.DataFrame(overall_rows)
    mean_floor = float(overall.loc[overall.method.isin(["mean_global", "mean_style_balanced"]), "macro_style_rmse"].min())
    overall["improves_best_mean_macro_rmse"] = overall.macro_style_rmse < mean_floor
    overall["qualified_for_proxy"] = overall.selectable & overall.nonconstant_qualified & overall.improves_best_mean_macro_rmse
    eligible = overall[overall.selectable & overall.nonconstant_qualified]
    fallback_used = eligible.empty
    if fallback_used:
        eligible = overall[overall.selectable]
    selected_row = eligible.sort_values(["selection_score", "method"]).iloc[0]
    selected_method = str(selected_row.method)
    selected_config = spec_map[selected_method]
    qualified = bool(selected_row.qualified_for_proxy)
```

src/16_q3_stale_domain.py（续）

```
    loso_predictions = pd.concat(prediction_rows, ignore_index=True)
    loso_predictions.to_csv(OUT / "loso_predictions.csv", index=False, lineterminator="\n")
    overall.to_csv(OUT / "loso_metrics_overall.csv", index=False, lineterminator="\n")
    selection_result = {"selected_method": selected_method, "config": selected_config,
        "selected_loso_metrics": selected_row.to_dict(), "qualified_for_proxy": qualified,
        "fallback_used": fallback_used, "best_mean_macro_rmse": mean_floor,
        "selection_complete_before_pils_diagnostic": True,
        "qualification_scope": "training-only LOSO diagnostic; not an unbiased estimate after selection, not causal validation"}
    write_json(OUT / "selection_result.json", selection_result)
    print(f"Stale domain selected without Pils labels: {selected_method}, qualified={qualified}", flush=True)

    # Fit all fixed candidates only after the selection decision is frozen.
    # All Pils labels are used solely for the diagnostic tables below.
    all_fitted = {}
    for spec in specs:
        model = make_domain_pipeline(spec, features, calibrated, aging, FINAL_SEED)
        target = y45.iloc[train_indices] if spec["family"] == "old_multioutput" else ytr
        model.fit(xtr, target, **training_fit_params(spec, groups))
        all_fitted[spec["name"]] = model
    common_metadata = {"features": features, "target": TARGET,
        "target_scale": "attachment_4_standardized", "config": selected_config,
        "selected_config": selected_config, "selected_method": selected_method,
        "qualified_for_proxy": qualified, "qualification_scope": selection_result["qualification_scope"],
        "inner_oof_variance_ratio": float(selected_row.prediction_variance_ratio),
        "inner_oof_rmse": float(selected_row.rmse),
        "loso_macro_style_rmse": float(selected_row.macro_style_rmse),
        "loso_macro_style_abs_bias": float(selected_row.macro_style_abs_bias),
        "loso_selection_score": float(selected_row.selection_score),
        "train_global_indices": train_indices, "excluded_global_indices": source_indices,
        "excluded_style": "Pils/Lager", "selection_protocol": run_manifest["selection_rule"],
        "pils_diagnostic_used_for_selection": False}
    final_model = all_fitted[selected_method]
    joblib.dump({"pipeline": final_model, **common_metadata}, OUT / "transfer_stale_model.joblib", compress=3)

    diagnostic_rows, diagnostic_metrics = [], []
    for spec in specs:
        name = spec["name"]
        values = predict_target(all_fitted[name], x.iloc[source_indices], spec, target_position)
        diagnostic_rows.append(pd.concat([meta.iloc[source_indices].reset_index(drop=True),
            pd.DataFrame({"global_index": source_indices, "method": name,
                          "observed": y.iloc[source_indices].to_numpy(), "prediction": values,
                          "selected_without_pils_labels": name == selected_method})], axis=1))
        diagnostic_metrics.append({"method": name, "selected_without_pils_labels": name == selected_method,
                                   "n": len(source_indices), **scores(y.iloc[source_indices], values)})
    pd.concat(diagnostic_rows, ignore_index=True).to_csv(OUT / "pils_diagnostic_predictions.csv", index=False, lineterminator="\n")
```

src/16_q3_stale_domain.py（续）

```
    pd.DataFrame(diagnostic_metrics).to_csv(OUT / "pils_diagnostic_metrics.csv", index=False, lineterminator="\n")

    models, indices, bootstrap_seeds, estimator_seeds = [], [], [], []
    for b in range(60):
        seed = SELECTION_BOOTSTRAP_SEED+b if b < 40 else AUDIT_BOOTSTRAP_SEED+b-40
        index = np.random.default_rng(seed).choice(train_indices, size=len(train_indices), replace=True)
        estimator_seed = seed+10000
        model = make_domain_pipeline(selected_config, features, calibrated, aging, estimator_seed)
        model.fit(x.iloc[index], y.iloc[index],
                  **training_fit_params(selected_config, meta.iloc[index].tasting_category_fine))
        models.append(model)
        indices.append(index)
        bootstrap_seeds.append(seed)
        estimator_seeds.append(estimator_seed)
        if (b+1) % 10 == 0:
            print(f"Stale domain bootstrap {b+1}/60", flush=True)
    ensemble = {"models": models, "bootstrap_indices": indices, **common_metadata,
        "selection_model_indices": list(range(40)), "audit_model_indices": list(range(40, 60)),
        "bootstrap_seeds": bootstrap_seeds, "estimator_seeds": estimator_seeds,
        "pairing": "same_model_candidate_minus_source", "paired_quantile": 0.95,
        "hyperparameter_uncertainty_included": False,
        "resampling": "iid rows from 239 non-Pils; style-balanced weights re-estimated per draw where used",
        "audit_protocol": "first 40 screening; final 20 once-only audit; never reselect after audit",
        "interpretation": "fixed-configuration model-proxy bootstrap; not a calibrated confidence interval"}
    joblib.dump(ensemble, OUT / "bootstrap_ensemble.joblib", compress=3)
    np.savez_compressed(OUT / "bootstrap_indices.npz", indices=np.asarray(indices),
                        bootstrap_seeds=np.asarray(bootstrap_seeds), estimator_seeds=np.asarray(estimator_seeds))
    assert len(np.unique(groups)) == 21
    assert len(loso_predictions) == len(train_indices)*len(specs)
    assert np.asarray(indices).shape == (60, 239)
    assert all(not set(index) & set(source_indices) for index in indices)
    assert len(set(bootstrap_seeds)) == 60
    assert np.isfinite(predict_1d(final_model, x)).all()
    if selected_config["family"] == "local":
        write_local_neighborhood_diagnostics(final_model, models, indices, train_indices,
                                             source_indices, meta, x)
    selected_pils = next(row for row in diagnostic_metrics if row["method"] == selected_method)
    summary = {
        "target": TARGET, "samples": len(x), "training_samples": len(train_indices),
        "excluded_pils_sources": len(source_indices), "features": len(features), "aging_features": len(aging),
        "loso_styles": len(np.unique(groups)), "candidate_count": len(specs),
        "mean_baseline_method": "mean_global",
        "style_balanced_mean_baseline_method": "mean_style_balanced",
        "old_multioutput_baseline_method": "old_multioutput_tree",
        "old_single_tree_baseline_method": "fixed_single_tree",
        **selection_result, "selected_pils_diagnostic_metrics": selected_pils,
        "all_pils_diagnostic_metrics": diagnostic_metrics,
        "pils_status": "previously repeatedly viewed diagnostic set; not a new test set",
```

src/16_q3_stale_domain.py（续）

```
        "pils_used_for_training_selection_or_intercept": False,
        "bootstrap": {"models": 60, "selection": 40, "audit": 20,
                      "indices_shape": [60, 239], "fixed_selected_configuration": True,
                      "style_balanced_weights_refitted": bool(selected_config["balanced"]),
                      "algorithm_selection_uncertainty_included": False},
        "limitations": [
            "Complete LOSO uses only the 239 non-Pils samples; style labels determine groups and training weights, never prediction covariates.",
            "LOSO macro metrics treat the 21 observed styles equally; styles are observational groupings, not independent randomized domains.",
            "The selected score is the minimum among fixed candidate comparisons and is optimistic after selection; no outer grouped evaluation of the selector is claimed.",
            "Pils labels have been repeatedly inspected in prior work; current Pils metrics are diagnostics, not a fresh test or basis for selection/intercept correction.",
            "All candidate Pils results are retained and never change the previously frozen selected method.",
            "The variance gate is not a guarantee of response to each candidate chemical change.",
            "Row bootstrap conditions on the observed style pool and fixed chosen configuration; it omits algorithm-selection uncertainty and new-domain uncertainty.",
            "Screening and audit model draws share the same data; audit quantiles are neither independent experiments nor calibrated confidence bounds.",
            "Chemical-only prediction may miss oxidation, storage and process causes; passing model constraints does not establish actual stale-hops taste.",
            "Targets retain the supplied standardized scale; original 0-6 sensory scores cannot be recovered.",
        ],
        "verification": {"source_exclusion": True, "loso_styles": 21, "loso_prediction_rows": len(loso_predictions),
                         "bootstrap_indices_shape": [60, 239], "bootstrap_seeds_unique": True,
                         "prediction_finite": True},
        "versions": {"python": platform.python_version(), "numpy": np.__version__,
                     "pandas": pd.__version__, "sklearn": sklearn.__version__},
        "elapsed_seconds": perf_counter()-started,
    }
    write_json(OUT / "summary.json", summary)
    print(overall[["method", "macro_style_rmse", "macro_style_abs_bias", "selection_score",
                   "prediction_variance_ratio", "qualified_for_proxy"]].to_string(index=False), flush=True)
    print("Pils selected diagnostic:", json.dumps(selected_pils, ensure_ascii=False), flush=True)


if __name__ == "__main__":
    with threadpool_limits(limits=1):
        main()
```

src/17_q3_transfer_domain.py

SHA-256: 037118a4c4ef024736fd03c754110118fa8c67f0035d6c907e4dfcf6ef21b642

```
"""Joint committee screening with fixed candidate budgets and fresh audit draws."""
from __future__ import annotations

import json
from time import perf_counter

import joblib
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist
from threadpoolctl import threadpool_limits

from common import SOLUTION
from q3_data import load_q3_data, read_defect_thresholds
from q3_stale_optimized import predict_1d, raw_tree_pipeline

OUT = SOLUTION / "outputs/q3_domain/transfer"
OLD = SOLUTION / "outputs/q3"
PREVIOUS = SOLUTION / "outputs/q3_optimized"
DOMAIN = SOLUTION / "outputs/q3_domain/stale"
ADDITIONAL_DRAW_SEED = 20271223
FRESH_AUDIT_SEED = 20280123
GAMMAS = [.25, .5, .75, 1.0]
POOLS = ["matched_2004", "expanded_6004"]
POLICIES = ["single_geometry", "joint_geometry", "joint_margin"]
DEFAULT_POOL, DEFAULT_POLICY = "matched_2004", "joint_margin"
WEIGHT, TOL = .2, 1e-10


def write_json(path, value):
    path.write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False), encoding="utf-8", newline="\n")


def paired(models, frame, source):
    return np.column_stack([predict_1d(model, frame) - predict_1d(model, source)[0]
                            for model in models])


def chemical_ok(frame, source, scenario):
    good = np.ones(len(frame), dtype=bool)
    for field, threshold in scenario["thresholds"].items():
        observed = float(source[field].iloc[0])
        cap = max(observed, threshold) if threshold is not None else observed
        values = frame[field].to_numpy(float)
        good &= np.isfinite(values) & (values >= 0) & (values <= cap + TOL)
    return good


def build_candidates(source, draws, target_median, caps):
    blocks, descriptions = [], []
    for gamma in GAMMAS:
        block = (1-gamma) * source.to_numpy()[0] + gamma * draws.to_numpy()
        blocks.append(block)
        descriptions.extend({"gamma": gamma, "draw": j, "matched_budget": j < 500}
                            for j in range(len(draws)))
    for gamma in GAMMAS:
        blocks.append(((1-gamma)*source.to_numpy()[0]+gamma*target_median)[None])
        descriptions.append({"gamma": gamma, "draw": -1, "matched_budget": True})
```

src/17_q3_transfer_domain.py（续）

```
    values = pd.DataFrame(np.vstack(blocks), columns=source.columns)
    for field, cap in caps.items():
        values[field] = values[field].clip(lower=0, upper=cap)
    return values, pd.DataFrame(descriptions)


def policy_counts(part):
    return {"sources": len(part), "selected": int(part.candidate_index.notna().sum()),
            "new_model_checks_passed": int(part.all_model_checks_passed.sum()),
            "historical_cross_proxy_passed": int(part.cross_proxy_checks_passed.sum())
,
            "fresh_cross_proxy_passed": int(part.fresh_cross_proxy_checks_passed.sum()
),
            "all_committees_passed": int(part.all_committees_passed.sum()),
            "no_candidate": int(part.status.eq("no_candidate_found").sum())}


def main(output_directory=OUT, proxy_directory=DOMAIN, experiment="domain_loso",
         fresh_domain_audit_seed=None, fresh_original_audit_seed=FRESH_AUDIT_SEED):
    started = perf_counter()
    OUT, DOMAIN = output_directory, proxy_directory
    FRESH_AUDIT_SEED = fresh_original_audit_seed
    OUT.mkdir(parents=True, exist_ok=True)
    meta, x, y, _ = load_q3_data()
    train = np.flatnonzero(~meta.tasting_category_fine.eq("Pils/Lager"))
    sources = np.flatnonzero(meta.tasting_category_fine.eq("Pils/Lager"))
    target_rows = np.flatnonzero(meta.tasting_category_fine.eq("Tripel"))
    features = x.columns.tolist()
    defects = read_defect_thresholds()
    scenarios = json.loads((PREVIOUS / "transfer/threshold_scenarios.json").read_text(
encoding="utf-8"))
    # This protocol is persisted before committee predictions or audit fitting.
    manifest = {"experiment":experiment,"proxy_directory":DOMAIN.relative_to(SOLUTION)
.as_posix(),
        "default_pool": DEFAULT_POOL, "default_policy": DEFAULT_POLICY,
        "pools": {"matched_2004": 2004, "expanded_6004": 6004}, "policies": POLICIES,
        "selection": "40 current-proxy models; first 30 frozen original models; paired
 Q95 <= 0",
        "joint_margin_order": "minimize max(domain selection Q95, original selection Q
95), then geometry objective",
        "geometry_objective": "target_distance/source_distance + .2*change_rms/source_
center_rms",
        "domain_audit": "last 20 current-proxy models, selected spectra only",
        "fresh_domain_audit_seed":fresh_domain_audit_seed,
        "historical_diagnostic": "last 10 original models, adaptively reused across pr
evious experiments",
        "fresh_audit": "20 fresh-seed draws of original 120-tree single-target protoco
l, selected spectra only",
        "fresh_audit_seed": FRESH_AUDIT_SEED, "additional_draw_seed": ADDITIONAL_DRAW_
SEED,
        "gammas": GAMMAS, "lambda": WEIGHT, "minimum_target_improvement": .10,
        "minimum_change_rms": .05, "threshold_scenarios": scenarios,
        "audit_reselection": False, "default_selected_from_audit": False,
        "qualification_required": True,
        "limitations": ["All audits reuse the same 239 observed training beers.",
                        "Bootstrap quantiles do not have calibrated post-search or cau
sal coverage.",
                        "A safety-margin objective can require larger chemical changes
."]}
    write_json(OUT / "run_manifest.json", manifest)
    geometry = joblib.load(OLD / "transfer/transfer_copula.joblib")
```

src/17_q3_transfer_domain.py（续）

```
    np.testing.assert_array_equal(geometry["training_rows"], train)
    reference = geometry["copula"]
    previous_draws = pd.read_csv(PREVIOUS / "transfer/optimized_base_draws.csv")
    draws = pd.concat([previous_draws, reference.sample("Tripel", 1000, seed=ADDITIONAL_DRAW_SEED)], ignore_index=True)
    draws.to_csv(OUT / "base_draws.csv", index=False, lineterminator="\n")
    target_z = reference.transform(x.iloc[target_rows])
    dimension = x.shape[1]
    pairwise = cdist(target_z, target_z)/np.sqrt(dimension)
    np.fill_diagonal(pairwise, np.inf)
    support_limit = float(np.quantile(pairwise.min(axis=1), .95))
    target_center = target_z.mean(axis=0)
    medians = x.iloc[train].median()
    minimum, maximum = x.iloc[train].min().to_numpy(), x.iloc[train].max().to_numpy()
    target_median = x.iloc[target_rows].median().to_numpy()
    domain_model = joblib.load(DOMAIN / "transfer_stale_model.joblib")
    domain_ensemble = joblib.load(DOMAIN / "bootstrap_ensemble.joblib")
    if fresh_domain_audit_seed is not None:
        # Follow-up experiments keep the frozen 40 screening models, but do
        # not reuse their already observed audit predictions as fresh evidence.
        assert domain_model["selected_config"]["family"]=="extra_trees"
        replacement, replacement_indices, replacement_seeds = [], [], []
        for b in range(20):
            seed=fresh_domain_audit_seed+b
            index=np.random.default_rng(seed).choice(train,size=len(train),replace=True)
            replacement.append(raw_tree_pipeline(seed+10000,n_estimators=120).fit(x.iloc[index],y.stale_hops.iloc[index]))
            replacement_indices.append(index)
            replacement_seeds.append(seed)
        domain_ensemble={**domain_ensemble,
            "models":domain_ensemble["models"][:40]+replacement,
            "bootstrap_indices":list(domain_ensemble["bootstrap_indices"][:40])+replacement_indices,
            "bootstrap_seeds":list(domain_ensemble["bootstrap_seeds"][:40])+replacement_seeds,
            "estimator_seeds":list(domain_ensemble["estimator_seeds"][:40])+[s+10000 for s in replacement_seeds],
            "audit_protocol":"frozen 40 screening models; 20 new-seed audit models for a declared exploratory follow-up"}
        joblib.dump(domain_ensemble,OUT/"screening_and_audit.joblib",compress=3)
    original = joblib.load(OLD / "sensory/stale_bootstrap.joblib")
    old_center = joblib.load(OLD / "sensory/transfer_proxy.joblib")
    for artifact in [domain_model, domain_ensemble, original]:
        assert artifact["features"] == features
        np.testing.assert_array_equal(artifact["train_global_indices"], train)
    for index in domain_ensemble["bootstrap_indices"]:
        assert not np.isin(index, sources).any()
    new_select, new_audit = domain_ensemble["models"][:40], domain_ensemble["models"][40:]
    old_select, old_historical = original["models"][:30], original["models"][30:]
    assert len(new_select) == 40 and len(new_audit) == 20
    qualified = bool(domain_model["qualified_for_proxy"])
    fresh_models, fresh_indices = [], []
    for b in range(20):
        seed = FRESH_AUDIT_SEED+b
        index = np.random.default_rng(seed).choice(train, size=len(train), replace=True)
        fresh_models.append(raw_tree_pipeline(seed+10000, n_estimators=120).fit(x.iloc[index], y.stale_hops.iloc[index]))
        fresh_indices.append(index)
```

src/17_q3_transfer_domain.py（续）

```
    joblib.dump({"models": fresh_models, "bootstrap_indices": fresh_indices,
                 "bootstrap_seeds": list(range(FRESH_AUDIT_SEED, FRESH_AUDIT_SEED+20))
,
                 "features": features, "train_global_indices": train,
                 "excluded_global_indices": sources, "target": "stale_hops",
                 "protocol": "original single-target ExtraTrees 120/minleaf3/maxfeatur
es.7"},
                OUT / "fresh_original_protocol_audit.joblib", compress=3)
    summaries, diagnostics, spectra, deltas, thresholds, previous_checks = [], [], [],
 [], [], []
    previous_summary = pd.read_csv(PREVIOUS / "transfer/transfer_summary.csv")
    previous_summary = previous_summary[previous_summary.is_default_policy].set_index(
"source_beer")
    previous_spectra = pd.read_csv(PREVIOUS / "transfer/transferred_spectra.csv").set_
index("transfer_id")
    stale_position = y.columns.get_loc("stale_hops")
    for count, index in enumerate(sources, 1):
        beer = int(meta.beer.iloc[index])
        raw_source = x.iloc[[index]]
        assert all(np.isfinite(raw_source[rule["feature"]].iloc[0]) and raw_source[rul
e["feature"]].iloc[0]>=0
                   for rule in defects.values())
        source = raw_source.fillna(medians)
        caps = {}
        for rule in defects.values():
            field = rule["feature"]
            finite = [scenario["thresholds"][field] for scenario in scenarios if scena
rio["thresholds"][field] is not None]
            caps[field] = max(float(raw_source[field].iloc[0]), min(finite)) if finite
 else float(raw_source[field].iloc[0])
        values, descriptions = build_candidates(source, draws, target_median, caps)
        latent, source_z = reference.transform(values), reference.transform(source)
        distance = reference.distance(values, "Tripel")
        before = float(reference.distance(source, "Tripel")[0])
        nn = cdist(latent, target_z).min(axis=1)/np.sqrt(dimension)
        change = np.linalg.norm(latent-source_z, axis=1)/np.sqrt(dimension)
        center_scale = max(float(np.linalg.norm(source_z[0]-target_center)/np.sqrt(dim
ension)), 1e-8)
        improvement = 1-distance/max(before, 1e-8)
        support = (nn <= support_limit+TOL) & ((values.to_numpy()>=minimum-TOL)&(value
s.to_numpy()<=maximum+TOL)).all(axis=1)
        chemical = np.column_stack([chemical_ok(values, raw_source, s) for s in scenar
ios]).all(axis=1)
        unique, inverse = np.unique(values.to_numpy(), axis=0, return_inverse=True)
        unique_frame = pd.DataFrame(unique, columns=features)
        new_delta = paired(new_select, unique_frame, source)[inverse]
        old_delta = paired(old_select, unique_frame, source)[inverse]
        new_q95, old_q95 = np.quantile(new_delta, .95, axis=1), np.quantile(old_delta,
 .95, axis=1)
        worst = np.maximum(new_q95, old_q95)
        new_signal = np.any(np.abs(new_delta)>1e-8, axis=1)
        old_signal = np.any(np.abs(old_delta)>1e-8, axis=1)
        base_ok = support & chemical & (improvement>=.1) & (change>=.05)
        single_ok = base_ok & (new_q95<=TOL) & new_signal
        joint_ok = single_ok & (old_q95<=TOL) & old_signal
        objective = distance/max(before, 1e-8)+WEIGHT*change/center_scale
        diagnostics.append(pd.concat([pd.DataFrame({"source_beer":beer, "candidate_ind
ex":np.arange(len(values)),
            "target_distance":distance,"target_improvement":improvement,"target_nn_rms
":nn,
            "change_latent_rms":change,"support_ok":support,"all_thresholds_chemical_o
```

src/17_q3_transfer_domain.py（续）

```
k":chemical,
206             "selection_new_q95":new_q95,"selection_old_q95":old_q95,"worst_selection_q
95":worst,
207             "new_signal":new_signal,"old_signal":old_signal,"single_feasible":single_o
k,
208             "joint_feasible":joint_ok,"geometry_objective":objective}), descriptions],
axis=1))
209         before_stale = float(predict_1d(domain_model["pipeline"], source)[0])
210         before_old = float(old_center["pipeline"].predict(source)[0,stale_position])
211         audit_cache = {}
212         for pool in POOLS:
213             pool_mask = descriptions.matched_budget.to_numpy() if pool=="matched_2004"
 else np.ones(len(values),bool)
214             new_variation = float(np.std(new_delta[pool_mask].mean(axis=1)))
215             old_variation = float(np.std(old_delta[pool_mask].mean(axis=1)))
216             for policy in POLICIES:
217                 eligible = (single_ok if policy=="single_geometry" else joint_ok) & po
ol_mask
218                 ids = np.flatnonzero(eligible)
219                 transfer_id = f"beer_{beer}_{pool}_{policy}"
220                 common = {"transfer_id":transfer_id,"source_beer":beer,"pool":pool,"po
licy":policy,
221                     "is_default_policy":pool==DEFAULT_POOL and policy==DEFAULT_POLICY,
"n_candidates":int(pool_mask.sum()),
222                     "n_selection_feasible":len(ids),"target_distance_before":before,"s
upport_nn_limit":support_limit,
223                     "proxy_qualified":qualified,"new_candidate_delta_sd":new_variation
,"old_candidate_delta_sd":old_variation,
224                     "all_model_checks_passed":False,"cross_proxy_checks_passed":False,
225                     "fresh_cross_proxy_checks_passed":False,"all_committees_passed":Fa
lse}
226                 if not len(ids):
227                     summaries.append({**common,"candidate_index":np.nan,"status":"no_c
andidate_found"})
228                     continue
229                 chosen = int(ids[np.lexsort((objective[ids],worst[ids]))[0]]) if polic
y=="joint_margin" else int(ids[np.argmin(objective[ids])])
230                 selected = values.iloc[[chosen]]
231                 key = selected.to_numpy().tobytes()
232                 if key not in audit_cache:
233                     audit_cache[key] = {"new_audit":paired(new_audit,selected,source)[
0],
234                         "old_historical":paired(old_historical,selected,source)[0],
235                         "fresh_original_audit":paired(fresh_models,selected,source)[0]
,
236                         "stale_after":float(predict_1d(domain_model["pipeline"],select
ed)[0]),
237                         "old_center_after":float(old_center["pipeline"].predict(select
ed)[0,stale_position])}
238                 cached = audit_cache[key]
239                 audit_q95 = {group:float(np.quantile(cached[group],.95)) for group in
["new_audit","old_historical","fresh_original_audit"]}
240                 informative = new_variation>1e-6 and bool(new_signal[chosen])
241                 old_informative = old_variation>1e-6 and bool(old_signal[chosen])
242                 new_pass = bool(qualified and informative and audit_q95["new_audit"]<=
TOL)
243                 cross = bool(new_pass and old_informative and old_q95[chosen]<=TOL and
 audit_q95["old_historical"]<=TOL)
244                 fresh_cross = bool(new_pass and old_informative and old_q95[chosen]<=T
OL
245                                    and audit_q95["fresh_original_audit"]<=TOL)
```

src/17_q3_transfer_domain.py（续）

```
                all_committees = bool(cross and fresh_cross)
                status = ("proxy_not_qualified" if not qualified else "proxy_uninforma
tive" if not informative
                          else "new_audit_failed" if not new_pass else "fresh_original
_audit_failed" if not fresh_cross
                          else "all_checks_passed" if all_committees else "fresh_check
s_passed_historical_disagrees")
                summaries.append({**common,"candidate_index":chosen,"status":status,"o
bjective":float(objective[chosen]),
                    "target_distance_after":float(distance[chosen]),"target_improvemen
t":float(improvement[chosen]),
                    "target_nn_after":float(nn[chosen]),"change_latent_rms":float(chan
ge[chosen]),
                    "selection_new_q95":float(new_q95[chosen]),"selection_old_q95":flo
at(old_q95[chosen]),
                    "worst_selection_q95":float(worst[chosen]),"new_audit_q95":audit_q
95["new_audit"],
                    "old_historical_audit_q95":audit_q95["old_historical"],"fresh_orig
inal_audit_q95":audit_q95["fresh_original_audit"],
                    "all_thresholds_chemical_ok":True,"all_model_checks_passed":new_pa
ss,
                    "cross_proxy_checks_passed":cross,"fresh_cross_proxy_checks_passed
":fresh_cross,
                    "all_committees_passed":all_committees,
                    "stale_before":before_stale,"stale_after":cached["stale_after"],
                    "old_center_before":before_old,"old_center_after":cached["old_cent
er_after"]})
                spectra.append({"transfer_id":transfer_id,"source_beer":beer,"pool":po
ol,"policy":policy,**selected.iloc[0].to_dict()})
                groups = {"new_selection":new_delta[chosen],"old_selection":old_delta[
chosen],
                          **{group:cached[group] for group in audit_q95}}
                for group, delta in groups.items():
                    deltas.extend({"transfer_id":transfer_id,"ensemble":group,"model_i
ndex":k,"paired_delta":float(value)}
                                  for k,value in enumerate(delta))
                for scenario in scenarios:
                    for label,rule in defects.items():
                        field=rule["feature"]
                        threshold=scenario["thresholds"][field]
                        observed=float(raw_source[field].iloc[0])
                        cap=max(observed,threshold) if threshold is not None else obse
rved
                        value=float(selected[field].iloc[0])
                        thresholds.append({"transfer_id":transfer_id,"source_beer":bee
r,"pool":pool,"policy":policy,
                            "scenario":scenario["scenario"],"defect":label,"feature":f
ield,"unit":rule["unit"],
                            "threshold":threshold,"source":observed,"transferred":valu
e,"upper_bound":cap,"passed":value<=cap+TOL})
        # Fixed previous selections are evaluated only after this round's choices.
        previous_row=previous_summary.loc[beer]
        if pd.notna(previous_row.candidate_index):
            frame=previous_spectra.loc[[previous_row.transfer_id],features]
            previous_z=reference.transform(frame)
            previous_nn=float(cdist(previous_z,target_z).min()/np.sqrt(dimension))
            previous_change=float(np.linalg.norm(previous_z-source_z)/np.sqrt(dimensio
n))
            previous_improvement=1-float(reference.distance(frame,"Tripel")[0])/max(be
fore,1e-8)
            geometry_ok=bool(previous_nn<=support_limit+TOL and previous_change>=.05 a
```

src/17_q3_transfer_domain.py（续）

```
    nd previous_improvement>=.1
286                                 and ((frame.to_numpy()>=minimum-TOL)&(frame.to_numpy()<=m
    aximum+TOL)).all())
287             new_previous_delta=paired(new_select,frame,source)
288             old_previous_delta=paired(old_select,frame,source)
289             nq=float(np.quantile(new_previous_delta,.95))
290             oq=float(np.quantile(old_previous_delta,.95))
291             na=float(np.quantile(paired(new_audit,frame,source),.95))
292             oh=float(np.quantile(paired(old_historical,frame,source),.95))
293             fa=float(np.quantile(paired(fresh_models,frame,source),.95))
294             chem=all(chemical_ok(frame,raw_source,s)[0] for s in scenarios)
295             matched=descriptions.matched_budget.to_numpy()
296             new_ok=bool(qualified and chem and geometry_ok and nq<=TOL and na<=TOL
297                         and np.any(np.abs(new_previous_delta)>1e-8) and np.std(new_del
    ta[matched].mean(axis=1))>1e-6)
298             old_screen_ok=bool(oq<=TOL and np.any(np.abs(old_previous_delta)>1e-8)
299                                and np.std(old_delta[matched].mean(axis=1))>1e-6)
300             old_ok=bool(old_screen_ok and oh<=TOL)
301             previous_checks.append({"source_beer":beer,"previous_transfer_id":previous
    _row.transfer_id,
302                 "all_thresholds_chemical_ok":chem,"geometry_ok":geometry_ok,"selection
    _new_q95":nq,"selection_old_q95":oq,
303                 "new_audit_q95":na,"old_historical_audit_q95":oh,"fresh_original_audit
    _q95":fa,
304                 "all_model_checks_passed":new_ok,"cross_proxy_checks_passed":new_ok an
    d old_ok,
305                 "fresh_cross_proxy_checks_passed":new_ok and old_screen_ok and fa<=TOL
    ,
306                 "all_committees_passed":new_ok and old_ok and fa<=TOL,
307                 "previous_target_improvement":float(previous_row.target_improvement)})
308         print(f"Domain transfer {count}/11 beer={beer}: single={single_ok.sum()}, join
    t={joint_ok.sum()}",flush=True)
309     result=pd.DataFrame(summaries)
310     result.to_csv(OUT/"transfer_summary.csv",index=False, lineterminator="\n")
311     pd.concat(diagnostics,ignore_index=True).to_csv(OUT/"candidate_diagnostics.csv",in
    dex=False, lineterminator="\n")
312     pd.DataFrame(spectra).to_csv(OUT/"transferred_spectra.csv",index=False, linetermin
    ator="\n")
313     pd.DataFrame(deltas).to_csv(OUT/"stale_paired_deltas.csv",index=False, linetermina
    tor="\n")
314     pd.DataFrame(thresholds).to_csv(OUT/"threshold_sensitivity.csv",index=False, linet
    erminator="\n")
315     previous_frame=pd.DataFrame(previous_checks)
316     previous_frame.to_csv(OUT/"previous_same_spectra_recheck.csv",index=False, lineter
    minator="\n")
317     policies=[{"pool":pool,"policy":policy,**policy_counts(part)} for (pool,policy),pa
    rt in result.groupby(["pool","policy"])]
318     pd.DataFrame(policies).to_csv(OUT/"policy_summary.csv",index=False, lineterminator
    ="\n")
319     default=result[result.is_default_policy]
320     summary={"experiment":experiment,"protocol":manifest,"training_n":len(train),"sour
    ces":len(sources),"features":len(features),
321         "support_nn_limit":support_limit,"proxy_qualified":qualified,"selected_domain_
    config":domain_model.get("selected_config",domain_model.get("config")),
322         "default_policy":policy_counts(default),"policies":policies,
323         "previous_original_rules":json.loads((PREVIOUS/"transfer/summary.json").read_t
    ext(encoding="utf-8"))["default_policy"],
324         "previous_same_spectra_current_rules":{"sources":len(previous_frame),
325             "new_model_checks_passed":int(previous_frame.all_model_checks_passed.sum()
    ),
326             "historical_cross_proxy_passed":int(previous_frame.cross_proxy_checks_pass
```

src/17_q3_transfer_domain.py（续）

```
    ed.sum()),
327             "fresh_cross_proxy_passed":int(previous_frame.fresh_cross_proxy_checks_pas
    sed.sum()),
328             "all_committees_passed":int(previous_frame.all_committees_passed.sum())},
329         "original_committee_protocol":"single-target 120-tree ExtraTrees; central orig
    inal predictor alone was 45-output",
330         "elapsed_seconds":perf_counter()-started}
331     write_json(OUT/"summary.json",summary)
332     print(json.dumps(summary["default_policy"],ensure_ascii=False,indent=2),flush=True
    )
333
334
335 if __name__=="__main__":
336     with threadpool_limits(limits=1):
337         main()
```

src/18_q3_domain_report.py

SHA-256: 60ba9b79a52af7cdd74e0dfe12175209d329267cc011bc8792afa969d8927712

```
"""Build the domain-transfer report from saved prediction and audit evidence."""
from __future__ import annotations

import hashlib
import json
from pathlib import Path
import textwrap

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
import numpy as np
import pandas as pd

from common import SOLUTION


OUT = SOLUTION / "outputs/q3_domain"
STALE = OUT / "stale"
TRANSFER = OUT / "transfer"
PREVIOUS = SOLUTION / "outputs/q3_optimized"
TABLES = OUT / "report_tables"
FIGURES = OUT / "figures"
SOURCES = []
COLORS = {"current": "#277DA8", "previous": "#8C959E",
          "historical": "#BD8733", "fresh": "#667C42", "text": "#30383F"}
CHECKS = {"new": "all_model_checks_passed", "historical": "cross_proxy_checks_passed",
          "fresh": "fresh_cross_proxy_checks_passed", "all": "all_committees_passed"}


def record_source(path, rows=None):
    SOURCES.append({"path": path.relative_to(SOLUTION).as_posix(), "rows": rows,
                    "sha256": hashlib.sha256(path.read_bytes()).hexdigest()})


def read_csv(path):
    frame = pd.read_csv(path)
    record_source(path, len(frame))
    return frame


def read_json(path):
    value = json.loads(path.read_text(encoding="utf-8"))
    record_source(path)
    return value


def boolean(series):
    if pd.api.types.is_bool_dtype(series.dtype):
        return series.fillna(False)
    converted = series.map({True: True, False: False, 1: True, 0: False,
                            "True": True, "False": False, "true": True, "false": False
})
    if (converted.isna() & series.notna()).any():
        raise ValueError(f"Unexpected boolean values: {series.unique()}")
    return converted.fillna(False).astype(bool)


```

src/18_q3_domain_report.py（续）

```
def normalize_checks(frame):
    result = frame.copy()
    for column in [*CHECKS.values(), "is_default_policy", "all_thresholds_chemical_ok"]:
        if column in result:
            result[column] = boolean(result[column])
    if "all_committees_passed" not in result and all(c in result for c in
            ["cross_proxy_checks_passed", "fresh_cross_proxy_checks_passed"]):
        result["all_committees_passed"] = result.cross_proxy_checks_passed & result.fresh_cross_proxy_checks_passed
    return result


def method_name(method):
    aliases = {"mean": "Training mean", "non_pils_mean": "Training mean", "mean_global": "Training mean",
               "mean_style_balanced": "Style-balanced mean", "old_multioutput_tree": "Old 45-output tree",
               "fixed_single_tree": "Dedicated 300-tree",
               "mean_baseline": "Training mean", "old_multioutput_extra_trees": "Old 45-output tree",
               "old_multioutput": "Old 45-output tree", "fixed_single_extra_trees": "Dedicated 300-tree",
               "old_single_tree": "Dedicated 300-tree"}
    return aliases.get(str(method), str(method).replace("_", " "))


def prediction_tables(overall, by_style, pils_metrics, pils_predictions, stale_summary):
    selected = stale_summary["selected_method"]
    if selected not in set(overall.method) or selected not in set(pils_metrics.method):
        raise ValueError("Selected method is absent from prediction evidence")
    merged = overall.merge(pils_metrics[["method", "rmse", "mae", "r2"]].rename(
        columns={"rmse": "pils_rmse", "mae": "pils_mae", "r2": "pils_r2"}), on="method", validate="one_to_one")
    merged["is_selected"] = merged.method.eq(selected)
    merged.to_csv(TABLES / "prediction_method_comparison.csv", index=False, lineterminator="\n")
    selected_predictions = pils_predictions.loc[pils_predictions.method.eq(selected)].copy()
    if not selected_predictions.beer.is_unique:
        raise ValueError("Expected one saved Pils diagnostic prediction per selected-method beer")
    paired = selected_predictions[["beer", "observed", "prediction"]].rename(
        columns={"prediction": "domain_prediction"})
    baseline_ids = {"matched_single_tree": stale_summary["old_single_tree_baseline_method"],
                    "old_multioutput": stale_summary["old_multioutput_baseline_method"],
                    "training_mean": stale_summary["mean_baseline_method"]}
    for label, method in baseline_ids.items():
        part = pils_predictions.loc[pils_predictions.method.eq(method), ["beer", "observed", "prediction"]]
        paired = paired.merge(part.rename(columns={"observed": "check_observed",
                                "prediction": label + "_prediction"}), on="beer", validate="one_to_one")
        if len(paired) != len(selected_predictions) or not np.allclose(paired.observed, paired.check_observed):
            raise ValueError("Matched diagnostic predictions cover different populations or targets")
```

src/18_q3_domain_report.py（续）

```
        paired = paired.drop(columns="check_observed")
    for label, field in [("matched_single_tree", "matched_single_tree_prediction"),
                         ("old_multioutput", "old_multioutput_prediction"),
                         ("training_mean", "training_mean_prediction"),
                         ("domain_selected", "domain_prediction")]:
        paired[f"{label}_residual"] = paired[field] - paired.observed
        squared = paired[f"{label}_residual"] ** 2
        paired[f"{label}_sse_share"] = squared / squared.sum()
    paired.sort_values("beer").to_csv(TABLES / "pils_source_error_influence.csv", index=False, lineterminator="\n")
    style_table = by_style.loc[by_style.method.eq(selected)].copy()
    style_table.to_csv(TABLES / "selected_loso_by_style.csv", index=False, lineterminator="\n")
    return merged, paired, selected


def transfer_tables(transfers, thresholds, previous, scenarios, experiment):
    rows = []
    for (pool, policy), group in transfers.groupby(["pool", "policy"], sort=False):
        if not group.source_beer.is_unique:
            raise ValueError("Duplicate source within transfer policy")
        selected = group.candidate_index.notna()
        rows.append({"pool": pool, "policy": policy, "sources": len(group),
            "selected": int(selected.sum()), **{f"{key}_passed": int(group[field].sum())
                                                for key, field in CHECKS.items()},
            "median_target_improvement_selected": float(group.loc[selected, "target_improvement"].median()),
            "median_change_rms_selected": float(group.loc[selected, "change_latent_rms"].median()),
            "median_target_improvement_fresh_pass": float(group.loc[group[CHECKS["fresh"]], "target_improvement"].median())})
    policies = pd.DataFrame(rows)
    policies.to_csv(TABLES / f"{experiment}_policy_comparison.csv", index=False, lineterminator="\n")
    default = transfers.loc[transfers.is_default_policy].sort_values("source_beer")
    if not default.source_beer.is_unique:
        raise ValueError("Default policy must contain one row per source")
    if set(default.source_beer) != set(previous.source_beer):
        raise ValueError("Current and rechecked historical spectra cover different sources")
    current_chemical = thresholds.groupby(["transfer_id", "scenario"]).passed.all()
    scenario_rows = []
    for scenario in scenarios:
        name = scenario["scenario"]
        mask = np.array([bool(current_chemical.get((t, name), False)) for t in default.transfer_id])
        scenario_rows.append({"scenario": name, "statistic": scenario.get("statistic"),
            "factor": scenario.get("factor"), "sources": len(default),
            "chemical_only": int(mask.sum()),
            **{f"{key}_passed": int(np.sum(mask & default[field].to_numpy(bool)))
               for key, field in CHECKS.items()}})
    scenario_table = pd.DataFrame(scenario_rows)
    scenario_table.to_csv(TABLES / f"{experiment}_default_threshold_pass_counts.csv", index=False, lineterminator="\n")
    source = default.merge(previous.rename(columns={field: "previous_" + field for field in CHECKS.values()}),
                           on="source_beer", suffixes=("", "_previous"), validate="one_to_one")
    source.to_csv(TABLES / f"{experiment}_default_source_comparison.csv", index=False,
```

src/18_q3_domain_report.py（续）

```
     lineterminator="\n")
    return policies, default, scenario_table, source


def configure_figures():
    plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 10,
        "axes.spines.top": False, "axes.spines.right": False,
        "axes.labelcolor": COLORS["text"], "text.color": COLORS["text"],
        "figure.facecolor": "white", "savefig.facecolor": "white",
        "svg.fonttype": "none"})


def save_figure(fig, name):
    fig.savefig(FIGURES / f"{name}.png", dpi=300)
    fig.savefig(FIGURES / f"{name}.svg")
    plt.close(fig)


def prediction_figure(comparison, selected, n_styles, n_pils):
    positions = np.arange(len(comparison))
    labels = [textwrap.fill(method_name(m), 30) + (" *" if m == selected else "") for m in comparison.method]
    colors = [COLORS["current"] if m == selected else COLORS["previous"] for m in comparison.method]
    fig, axes = plt.subplots(1, 2, figsize=(12.8, max(7.0, len(comparison) * .45)), sharey=True)
    for axis, field, title in [(axes[0], "macro_style_rmse", f"Selection evidence: LOSO macro RMSE ({n_styles} styles)"),
                               (axes[1], "pils_rmse", f"Repeatedly viewed Pils diagnostic RMSE (n={n_pils})")]:
        values = comparison[field].to_numpy(float)
        axis.barh(positions, values, height=.6, color=colors)
        axis.set_xlim(0, float(values.max()) * 1.19)
        for y, value in zip(positions, values):
            axis.text(value + values.max() * .015, y, f"{value:.3f}", va="center", fontsize=9)
        axis.set_title(title, loc="left", fontsize=11, pad=15)
        axis.set_xlabel("Stale hops RMSE (attachment-standardized units)")
        axis.grid(axis="x", color="#E5E8EC", linewidth=.7)
        axis.set_axisbelow(True)
    axes[0].set_yticks(positions, labels, fontsize=9)
    axes[0].invert_yaxis()
    fig.text(.24, .025, "* Selected by non-Pils LOSO score, not by the Pils column.\n"
             "Candidate LOSO scores were used for selection; they are not unbiased post-selection test estimates.",
             fontsize=9, color="#525C65")
    fig.subplots_adjust(left=.24, right=.98, top=.9, bottom=.14, wspace=.23)
    save_figure(fig, "prediction_domain_validation")


def transfer_figure(experiments):
    rows, matrix_rows, matrix_labels = [], [], []
    for name, evidence in experiments.items():
        for kind, part in [("Previous spectra\nrechecked", evidence["previous"]),
                           ("Fixed default", evidence["default"])]:
            rows.append({"label": f"{evidence['label']}\n{kind}", "sources": len(part),
                         **{f"{key}_passed": int(part[field].sum()) for key, field in CHECKS.items()}})
        for field, label in [("chemical_only", "chemical"), ("fresh_passed", "fresh checks")]:
```

src/18_q3_domain_report.py（续）

```
            matrix_rows.append(evidence["scenario_table"][field].to_numpy())
            matrix_labels.append(evidence["label"] + ": " + label)
    table = pd.DataFrame(rows)
    n = int(table.sources.max())
    fig, axes = plt.subplots(1, 2, figsize=(14.6, 6.2), gridspec_kw={"width_ratios": [1.0, 1.45]})
    y = np.arange(len(table))
    specs = [(-.27, "new", "Primary committee checks", COLORS["current"]),
             (-.09, "historical", "+ original / historical", COLORS["historical"]),
             (.09, "fresh", "+ original / fresh", COLORS["fresh"]),
             (.27, "all", "Historical and fresh intersection", COLORS["text"])]
    for offset, key, label, color in specs:
        values = table[f"{key}_passed"]
        axes[0].barh(y + offset, values, .16, label=label, color=color)
        for yy, value, denominator in zip(y + offset, values, table.sources):
            axes[0].text(value + .08, yy, f"{value}/{denominator}", va="center", fontsize=8)
    axes[0].set_yticks(y, table.label, fontsize=9)
    axes[0].invert_yaxis()
    axes[0].set_xlim(0, n + 1.3)
    axes[0].set_xlabel("Sources passing the stated checks")
    axes[0].set_title("Each default and its matched historical recheck", loc="left", fontsize=11, pad=14)
    axes[0].grid(axis="x", color="#E5E8EC", linewidth=.7)
    axes[0].set_axisbelow(True)
    axes[0].legend(frameon=False, fontsize=8, loc="upper left", bbox_to_anchor=(0, -.12), ncol=2)
    matrix = np.asarray(matrix_rows)
    scenario_table = next(iter(experiments.values()))["scenario_table"]
    axes[1].imshow(matrix, vmin=0, vmax=n, cmap="Blues", aspect="auto")
    axes[1].set_yticks(np.arange(len(matrix)), matrix_labels, fontsize=9)
    scenario_labels = [f"{r.statistic}\nx{r.factor:g}" if pd.notna(r.factor) else r.scenario
                       for r in scenario_table.itertuples()]
    axes[1].set_xticks(np.arange(len(scenario_table)), scenario_labels, fontsize=8)
    for (row, column), value in np.ndenumerate(matrix):
        axes[1].text(column, row, f"{value}/{n}", ha="center", va="center", fontsize=8,
                     color="white" if value > n * .6 else COLORS["text"])
    axes[1].set_title("Fixed spectra across the saved threshold scenarios", loc="left", fontsize=11, pad=14)
    axes[1].spines[["left", "bottom"]].set_visible(False)
    axes[1].tick_params(length=0)
    fig.text(.16, .025, "Domain and follow-up use different primary committees; each historical recheck uses its own matching audit draws.\n"
             "Fresh checks do not require the reused historical 10 models. All groups still share the same observed training population.",
             fontsize=9, color="#525C65")
    fig.subplots_adjust(left=.16, right=.99, top=.89, bottom=.25, wspace=.56)
    save_figure(fig, "transfer_policy_and_thresholds")


def target_figure(experiments, archived):
    frame = archived[["source_beer", "target_improvement", "change_latent_rms"]].rename(
        columns={"target_improvement": "previous_target_improvement", "change_latent_rms": "previous_change_rms"})
    specifications = [("domain", "Domain default", experiments["domain"]["default"], COLORS["current"], "D", -.06),
                      ("consensus", "Consensus matched default", experiments["consensus"]["default"], COLORS["fresh"], "s", .07)]
```

src/18_q3_domain_report.py（续）

```
    consensus = experiments["consensus"]
    expanded_pool = max(consensus["manifest"]["pools"], key=consensus["manifest"]["pools"].get)
    expanded = consensus["transfers"].loc[consensus["transfers"].pool.eq(expanded_pool)
                & consensus["transfers"].policy.eq(consensus["manifest"]["default_policy"])]
    specifications.append(("expanded", f"Consensus expanded ({consensus['manifest']['pools'][expanded_pool]})",
                           expanded, COLORS["historical"], "^", .2))
    for name, label, part, color, marker, offset in specifications:
        keep = part[["source_beer", "target_improvement", "change_latent_rms", "fresh_cross_proxy_checks_passed"]]
        frame = frame.merge(keep.rename(columns={c: f"{name}_{c}" for c in keep if c != "source_beer"}),
                            on="source_beer", validate="one_to_one")
    frame = frame.sort_values("source_beer")
    frame.to_csv(TABLES / "all_default_target_comparison.csv", index=False, lineterminator="\n")
    fig, axes = plt.subplots(1, 2, figsize=(14.5, 7.0), sharey=True)
    positions = np.arange(len(frame))
    for ax, field, label, scale in [(axes[0], "target_improvement", "Reduction in target-style distance (%)", 100),
                                    (axes[1], "change_latent_rms", "Chemical change (Gaussian-space RMS)", 1)]:
        previous_field = "previous_target_improvement" if field == "target_improvement" else "previous_change_rms"
        ax.scatter(frame[previous_field] * scale, positions - .2, s=35,
                   color=COLORS["previous"], marker="o", zorder=2)
        for name, _, part, color, marker, offset in specifications:
            for position, (_, row) in zip(positions, frame.iterrows()):
                value = row[f"{name}_{field}"]
                if pd.isna(value):
                    continue
                passed = bool(row[f"{name}_fresh_cross_proxy_checks_passed"])
                ax.scatter(value * scale, position + offset, s=45, marker=marker,
                           facecolor=color if passed else "white", edgecolor=color, linewidth=1.15, zorder=3)
        ax.set_xlabel(label)
        ax.grid(axis="x", color="#E5E8EC", linewidth=.7)
        ax.set_axisbelow(True)
    axes[0].set_yticks(positions, [str(int(v)) for v in frame.source_beer])
    axes[0].invert_yaxis()
    axes[0].set_ylabel("Pils/Lager source beer")
    axes[0].set_xlim(0, 103)
    axes[0].axvline(10, color="#747E87", linewidth=1, linestyle="--")
    axes[0].set_title("Target-style proximity", loc="left", fontsize=12, pad=15)
    axes[1].set_xlim(left=0)
    axes[1].set_title("Amount of chemical change", loc="left", fontsize=12, pad=15)
    handles = [Line2D([], [], color=COLORS["previous"], marker="o", linestyle="None", label="Historical fixed spectra (reference)"),
               *[Line2D([], [], color=color, marker=marker, linestyle="None", label=label)
                 for _, label, _, color, marker, _ in specifications],
               Line2D([], [], color=COLORS["text"], marker="o", linestyle="None", label="Filled: respective fresh checks pass"),
               Line2D([], [], color=COLORS["text"], marker="o", markerfacecolor="white", linestyle="None", label="Open: at least one fresh check fails")]
    fig.legend(handles=handles, frameon=False, fontsize=9, loc="lower center", bbox_to_anchor=(.52, .085), ncol=3)
    fig.text(.09, .025, "Same source beers and frozen geometry. Missing markers mean n
```

src/18_q3_domain_report.py（续）

```
    o candidate; all missing cases stay in the 11-source denominator.\n"
293             "Expanded search is a separate budget. Fresh audit draws differ between d
    omain and consensus experiments.",
294             fontsize=9, color="#525C65")
295     fig.subplots_adjust(left=.09, right=.98, top=.9, bottom=.25, wspace=.23)
296     save_figure(fig, "target_distance_and_consistency")
297
298
299 def write_report(comparison, influence, selected, by_style, stale_summary, stale_manif
    est,
300                  experiments, neighborhood):
301     selected_row = comparison.set_index("method").loc[selected]
302     matched_tree = comparison.set_index("method").loc[stale_summary["old_single_tree_b
    aseline_method"]]
303     matched_multi = comparison.set_index("method").loc[stale_summary["old_multioutput_
    baseline_method"]]
304     matched_mean = comparison.set_index("method").loc[stale_summary["mean_baseline_met
    hod"]]
305     domain, consensus = experiments["domain"], experiments["consensus"]
306     n_sources, n_styles = len(domain["default"]), by_style.heldout_style.nunique()
307     n_train = int(stale_summary["training_samples"])
308     selected_pils = float(selected_row.pils_rmse)
309     prediction_failed = selected_pils >= min(matched_tree.pils_rmse, matched_multi.pil
    s_rmse, matched_mean.pils_rmse)
310     heading = "Pils预测未改善，联合筛查只有有限收益" if prediction_failed else "Pils诊断
    与联合筛查的探索性比较"
311     d_default, c_default = domain["default"], consensus["default"]
312     c_previous = consensus["previous"]
313     expanded_pool = max(consensus["manifest"]["pools"], key=consensus["manifest"]["poo
    ls"].get)
314     expanded_budget = consensus["manifest"]["pools"][expanded_pool]
315     matched_budget = consensus["manifest"]["pools"][consensus["manifest"]["default_poo
    l"]]
316     expanded = consensus["transfers"].loc[consensus["transfers"].pool.eq(expanded_pool
    )
317                   & consensus["transfers"].policy.eq(consensus["manifest"]["default_poli
    cy"])]
318     fresh_d = int(d_default.fresh_cross_proxy_checks_passed.sum())
319     fresh_c = int(c_default.fresh_cross_proxy_checks_passed.sum())
320     fresh_c_previous = int(c_previous.fresh_cross_proxy_checks_passed.sum())
321     fresh_expanded = int(expanded.fresh_cross_proxy_checks_passed.sum())
322     influence_row = influence.loc[influence.matched_single_tree_sse_share.idxmax()]
323     original_rules = domain["summary"]["previous_original_rules"]
324     lines = [f"# 第三问：{heading}", "",
325         f"在 {n_train} 款非 Pils 啤酒的 {n_styles} 风格留一验证中，保存的主模型为 `{select
    ed}`。"
326         f"其宏 RMSE 为 **{selected_row.macro_style_rmse:.4f}**，较优均值基线为 "
327         f"**{stale_summary['best_mean_macro_rmse']:.4f}**；但 Pils 的诊断 RMSE 为 "
328         f"**{selected_pils:.4f}**，同轮旧多输出结构重训为 **{matched_multi.pils_rmse:.4f
    }**，"
329         f"固定专用树为 **{matched_tree.pils_rmse:.4f}**。**没有实现 Pils 跨风格预测改善。
    **", "",
330         f"局部域模型的固定默认迁移仅选出 **{d_default.candidate_index.notna().sum()}/{n_
    sources}** 款，"
331         f"其中 **{fresh_d}/{n_sources}** 通过 fresh 一致检查。随后沿旧固定树协议做联合筛查
    消融："
332         f"同 **{matched_budget}** 候选预算默认策略通过 **{fresh_c}/{n_sources}**，历史固
    定谱在"
333         f"同一追加实验规则下重检通过 **{fresh_c_previous}/{n_sources}**；"
334         f"预算扩为 **{expanded_budget}** 后为 **{fresh_expanded}/{n_sources}**。"
335         "追加实验由前一轮失败触发，属于探索性策略比较，没有据此替换默认或证明预测精度提高。",
```

src/18_q3_domain_report.py（续）

```
    "",
336         "## 风格留出分数与Pils诊断", "",
337         f"完整 LOSO 每次排除一种非 Pils 风格，在 {n_styles} 折分别拟合预处理和预测器。"
338         "选择量为风格宏 RMSE 加上预声明的风格绝对偏差惩罚；风格宏平均给每种风格相同权重，"
339         "与按啤酒数量加权的 pooled RMSE 不同。",
340         f"本轮惩罚系数为 {stale_manifest['bias_penalty']:g}，选中方法的宏绝对偏差为 "
341         f"{selected_row.macro_style_abs_bias:.4f}，选择分数为 {selected_row.selection_score:.4f}，"
342         f"样本加权 LOSO RMSE 为 {selected_row.rmse:.4f}。", "",
343         "LOSO 得分参与了方法选择，最低分存在选择乐观；未另设外层来检验整个选择器。"
344         "Pils 标签此前已反复查看，本轮明确视为诊断集。配置选择在保存 LOSO 结果后完成，"
345         "没有把 Pils 列中最低误差的配置替换为主方法，也没有按 Pils 标签校正截距。", "",
346         "|方法|LOSO宏RMSE|宏绝对偏差|选择分数|Pils诊断RMSE|主方法|",
347         "|---|---:|---:|---:|---:|---|"]
348     for _, row in comparison.iterrows():
349         lines.append(f"|`{row['method']}`|{row.macro_style_rmse:.4f}|{row.macro_style_abs_bias:.4f}|"
350                      f"{row.selection_score:.4f}|{row.pils_rmse:.4f}|{'是' if row.is_selected else ''}|")
351     lines.extend(["", "![跨风格预测比较](../outputs/q3_domain/figures/prediction_domain_validation.png)", "",
352         f"图1：全部 {len(comparison)} 个预声明配置的非 Pils 风格宏 RMSE 与全 {n_sources} 款 Pils 诊断 RMSE。"
353         "蓝色只标记按 LOSO 规则选中的配置，未构造独立置信区间。", "",
354         f"同轮固定专用树的 Pils 平方误差中，**{influence_row.matched_single_tree_sse_share:.1%}** 来自 beer "
355         f"**{int(influence_row.beer)}**，其真实值为 {influence_row.observed:.4f}。"
356         f"主局部模型对同一源预测 {influence_row.domain_prediction:.4f}，该源贡献主模型平方误差的 "
357         f"**{influence_row.domain_selected_sse_share:.1%}**。所有主指标保留全部源，"
358         "没有删除影响最大的样本或以该源单独调整方法。", "",
359         f"局部模型的 bootstrap 邻域诊断中，每次名义上的 {stale_summary['config']['n_neighbors']} 个近邻"
360         f"平均仅含 {neighborhood.nearest_unique_beers.mean():.2f} 款不同啤酒；"
361         f"相对全量拟合邻域的 Jaccard 中位数为 {neighborhood.neighbor_jaccard_vs_final.median():.2f}。"
362         "有放回抽样同时改变邻居、重复权重和拟合空间，说明训练内方差门槛不足以保证迁移配对差稳定。"
363         "域模型因此保留为失败探索，不据其训练分数替代原有预测基准。", "",
364         "## 两项迁移实验与检查口径", "",
365         "第一项使用 LOSO 选出的局部域模型；第二项在看到其候选覆盖不足后，沿此前固定专用树协议追加联合筛查消融。"
366         "追加实验重新固定默认策略和审计种子，仍然复用同一批源和训练总体，不能视为独立确认实验。", "",
367         "每项实验都保存三种策略与两种预算：`single_geometry` 只用主筛选委员会并按几何目标排序；"
368         "`joint_geometry` 加原筛选委员会约束，仍按几何目标排序；`joint_margin` 优先最小化两组筛选 Q95 "
369         "的较差值，再按几何目标排序。默认都是保存协议中的 matched 预算和 joint_margin；"
370         "扩池结果单列，未依据审计通过数重选默认。", "",
371         "检查定义如下：", "",
372         "- **主代理**：主筛选委员会与该实验的主审计检查通过。",
373         "- **历史一致**：再要求原筛选委员会及已在先前实验中使用的原后10模型通过。",
374         "- **fresh一致（主口径）**：主代理加原筛选委员会、fresh原协议20模型；**不要求历史10模型通过**。",
375         "- **全部交集**：历史一致和fresh一致同时成立。", "",
376         "原中心预测器是45输出模型，但原40个bootstrap模型本身是单目标120树协议。"
377         "追加实验的两组新审计也沿单目标120树协议，只是种子不同，不能称为独立架构验证。"
378         "所有组共享同一真实训练总体；historical与fresh采用并列定义，非逐层包含。", "",
379         "|实验|预算|策略|源数|选出|主代理|历史一致|fresh一致|全部交集|选出谱改善中位数|选出谱改动RMS中位数|",
```

src/18_q3_domain_report.py（续）

```
380         "|---|---|---|---:|---:|---:|---:|---:|---:|---:|---:|"])
381     for name, evidence in experiments.items():
382         for row in evidence["policies"].itertuples():
383             lines.append(f"|{evidence['label_zh']}|{row.pool}|{row.policy}|{row.source
    s}|{row.selected}|"
384                 f"{row.new_passed}|{row.historical_passed}|{row.fresh_passed}|{row.all
    _passed}|"
385                 f"{row.median_target_improvement_selected:.1%}|{row.median_change_rms_
    selected:.3f}|")
386     lines.extend(["", "改善与改动中位数只在各行选出谱中计算；选出数量不同，不可忽略分母直接
    比较。"
387         "未找到候选的源计为未通过，保留在全部源分母。", "",
388         f"历史报告在原规则下为 {original_rules['model_checks_passed_all_thresholds']}/{n
    _sources} 主代理通过、"
389         f"{original_rules['cross_proxy_checks_passed_all_thresholds']}/{n_sources} 新
    旧一致通过。"
390         "这些旧口径不含本轮主代理和fresh审计，不能和本轮数字直接计算提升。"
391         "每项实验都对同一历史固定谱按该实验相同模型和规则重新计算：", "",
392         "|重检所用实验|历史固定谱主代理|历史一致|fresh一致|全部交集|", "|---|---:|---:|---
    :|---:|"])
393     for evidence in experiments.values():
394         previous = evidence["previous"]
395         lines.append(f"|{evidence['label_zh']}|{previous.all_model_checks_passed.sum()
    }/{len(previous)}|"
396                      f"{previous.cross_proxy_checks_passed.sum()}/{len(previous)}|"
397                      f"{previous.fresh_cross_proxy_checks_passed.sum()}/{len(previous)
    }|"
398                      f"{previous.all_committees_passed.sum()}/{len(previous)}|")
399     lines.extend(["", "![策略及阈值比较](../outputs/q3_domain/figures/transfer_policy_a
    nd_thresholds.png)", "",
400         "图2：每项固定默认策略及其历史固定谱重检；右侧为默认固定谱跨全部阈值情景的化学保护及f
    resh一致通过数。"
401         "两项实验的主代理和审计抽样不同，跨实验数字是探索性对照；同项重检才使用完全匹配规则。
    ", "",
402         "## 阈值情景、风格接近与改动代价", "",
403         "化学投影同时保护保存的中位数/均值及各乘数情景；每项缺陷超阈程度均不得增加。"
404         "trans-2-nonenal 无附件阈值时采用浓度不得增加。所有情景检查同一个已选谱，没有逐情景
    重新挑候选。"
405         "下表化学保护计数也要求已选出谱，未选出的源没有可检查迁移谱，计为未通过而非伪造浓度。
    ", "",
406         "|阈值情景|域默认化学保护|域默认fresh|固定树默认化学保护|固定树默认fresh|",
407         "|---|---:|---:|---:|---:|"])
408     ds, cs = domain["scenario_table"].set_index("scenario"), consensus["scenario_table
    "].set_index("scenario")
409     for scenario in domain["scenario_table"].scenario:
410         dr, cr = ds.loc[scenario], cs.loc[scenario]
411         lines.append(f"|{scenario}|{dr.chemical_only}/{dr.sources}|{dr.fresh_passed}/{
    dr.sources}|"
412                      f"{cr.chemical_only}/{cr.sources}|{cr.fresh_passed}/{cr.sources}|
    ")
413     lines.extend(["", "![目标距离与改动代价](../outputs/q3_domain/figures/target_distan
    ce_and_consistency.png)", "",
414         "图3：全部源的历史固定谱、域模型默认、固定树matched默认及同策略expanded谱。左为冻结
    旧几何下的目标距离改善，"
415         "右为同一高斯空间的改动RMS；历史灰点只作几何参考。其他实心点通过各自fresh检查，空心点
    失败，"
416         "缺点表示无候选。expanded使用更多搜索预算，未替换matched默认。", "",
417         f"追加实验 expanded joint_margin 的 {fresh_expanded}/{n_sources} fresh通过伴随目
    标接近度代价。"
418         f"历史固定谱的目标改善中位数为 {consensus['previous'].previous_target_improvemen
    t.median():.1%}，"
```

src/18_q3_domain_report.py（续）

```
        f"expanded选出谱为 {expanded.target_improvement.median():.1%}；改动RMS的对应差别与全部策略见上表。"
        "边际安全余量与几何最优并不一致，不能只用较高通过数量概括结果。", "",
        "|expanded joint_margin源|状态|目标改善|改动RMS|较差筛选Q95|主audit Q95|fresh原协议Q95|fresh通过|",
        "|---|---|---:|---:|---:|---:|---:|---|"])
    def number(value):
        return f"{value:+.4f}" if pd.notna(value) else "—"
    for row in expanded.sort_values("source_beer").itertuples():
        improvement = f"{row.target_improvement:.1%}" if pd.notna(row.target_improvement) else "—"
        change = f"{row.change_latent_rms:.3f}" if pd.notna(row.change_latent_rms) else "—"
        lines.append(f"|{row.source_beer}|{row.status}|{improvement}|{change}|"
            f"{number(row.worst_selection_q95)}|{number(row.new_audit_q95)}|{number(row.fresh_original_audit_q95)}|"
            f"{'是' if row.fresh_cross_proxy_checks_passed else '否'}|")
    passing = expanded.loc[expanded.fresh_cross_proxy_checks_passed]
    if len(passing):
        thin = passing.loc[passing.worst_selection_q95.idxmax()]
        lines.extend(["", f"通过谱中最接近筛选边界的是源 {int(thin.source_beer)}，"
            f"较差筛选Q95为 {thin.worst_selection_q95:+.4f}。这类临界结果应保留连续数值，"
            "不把微小负值解释为具有品评置信保证。"])
    lines.extend(["", "## 范围与复现", "",
        "本轮仍未证明跨风格Stale hops准确预测改善。LOSO模型选择、候选搜索及多次追加实验复用现有数据，"
        "fresh只指新重采样模型，未带来新的独立啤酒或真实品评。固定结构bootstrap也未包含算法选择、"
        "测量误差和生成谱分布偏移；配对差Q95不是覆盖率校准的置信上界。", "",
        "所有感官值保持附件标准化量表，未恢复0–6分。当前输出是化学目标谱和模型内筛查，"
        "需要新增真实风格样本及迁移后的理化、品评实验，才能判断真实Stale hops是否不增加。", "",
        "- LOSO与Pils诊断：`../outputs/q3_domain/stale/`。",
        "- 域模型迁移失败探索：`../outputs/q3_domain/transfer/`。",
        "- 固定树联合筛查追加实验：`../outputs/q3_domain/consensus/`。",
        "- 本报告计算表：`../outputs/q3_domain/report_tables/`。",
        "- 三张图的PNG/SVG：`../outputs/q3_domain/figures/`。",
        "- 输入逐文件哈希与行数：`../outputs/q3_domain/report_source_manifest.json`。", "",
        "```bash", ".venv/bin/python solution_a/src/18_q3_domain_report.py", "```", ""])
    (SOLUTION / "docs/q3_domain_results.md").write_text("\n".join(lines), encoding="utf-8", newline="\n")
    result = {"selected_method": selected, "non_pils_training_n": n_train, "loso_styles": n_styles,
        "pils_diagnostic_n": n_sources, "selected_loso_macro_rmse": float(selected_row.macro_style_rmse),
        "selected_pils_diagnostic_rmse": selected_pils, "matched_fixed_tree_pils_rmse": float(matched_tree.pils_rmse),
        "matched_old_multioutput_pils_rmse": float(matched_multi.pils_rmse), "pils_prediction_improved": not prediction_failed,
        "experiments": {name: {"default_pool": evidence["manifest"]["default_pool"],
            "default_policy": evidence["manifest"]["default_policy"],
            "default": {"sources": len(evidence["default"]), "selected": int(evidence["default"].candidate_index.notna().sum()),
                        **{key: int(evidence["default"][field].sum()) for key, field in CHECKS.items()}},
            "previous_same_spectra": {"sources": len(evidence["previous"]),
                        **{key: int(evidence["previous"][field].sum()) for key, field in CHECKS.items()}}}
```

src/18_q3_domain_report.py（续）

```
            for name, evidence in experiments.items()},
        "consensus_expanded_same_policy": {"budget": expanded_budget, "sources": len(expanded),
            "selected": int(expanded.candidate_index.notna().sum()), "fresh": fresh_expanded,
            "all": int(expanded.all_committees_passed.sum()),
            "median_target_improvement_selected": float(expanded.target_improvement.median()),
            "median_change_rms_selected": float(expanded.change_latent_rms.median())},
        "threshold_scenarios": len(ds), "scientific_figure_count": 3,
        "scope": "Failed Pils prediction exploration and limited consensus-search gains, no sensory guarantee"}
    (OUT / "report_summary.json").write_text(json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8", newline="\n")
    return result


def main():
    TABLES.mkdir(parents=True, exist_ok=True)
    FIGURES.mkdir(parents=True, exist_ok=True)
    stale_summary = read_json(STALE / "summary.json")
    stale_manifest = read_json(STALE / "run_manifest.json")
    overall = read_csv(STALE / "loso_metrics_overall.csv")
    by_style = read_csv(STALE / "loso_metrics_by_style.csv")
    pils_metrics = read_csv(STALE / "pils_diagnostic_metrics.csv")
    pils_predictions = read_csv(STALE / "pils_diagnostic_predictions.csv")
    neighborhood = read_csv(STALE / "neighborhood_bootstrap_diagnostics.csv")
    comparison, influence, selected = prediction_tables(overall, by_style, pils_metrics, pils_predictions, stale_summary)
    experiments = {}
    for name, directory, label, label_zh in [("domain", TRANSFER, "Domain", "局部域模型"),
            ("consensus", OUT / "consensus", "Consensus follow-up", "固定树追加实验")]:
        summary = read_json(directory / "summary.json")
        manifest = read_json(directory / "run_manifest.json")
        transfers = normalize_checks(read_csv(directory / "transfer_summary.csv"))
        thresholds = read_csv(directory / "threshold_sensitivity.csv")
        thresholds["passed"] = boolean(thresholds.passed)
        previous = normalize_checks(read_csv(directory / "previous_same_spectra_recheck.csv"))
        policies, default, scenario_table, source = transfer_tables(transfers, thresholds, previous,
            manifest["threshold_scenarios"], name)
        experiments[name] = {"summary": summary, "manifest": manifest, "transfers": transfers,
            "policies": policies, "default": default, "scenario_table": scenario_table,
            "source": source, "previous": previous, "label": label, "label_zh": label_zh}
    if experiments["domain"]["scenario_table"].scenario.tolist() != experiments["consensus"]["scenario_table"].scenario.tolist():
        raise ValueError("Domain and follow-up threshold scenarios differ")
    archived = normalize_checks(read_csv(PREVIOUS / "transfer/transfer_summary.csv"))
    archived = archived.loc[archived.is_default_policy]
    configure_figures()
    prediction_figure(comparison, selected, by_style.heldout_style.nunique(), len(experiments["domain"]["default"]))
    transfer_figure(experiments)
    target_figure(experiments, archived)
    result = write_report(comparison, influence, selected, by_style, stale_summary, stale_manifest,
```

src/18_q3_domain_report.py（续）

```
                            experiments, neighborhood)
    (OUT / "report_source_manifest.json").write_text(json.dumps(SOURCES, ensure_ascii=
False, indent=2), encoding="utf-8", newline="\n")
    print(json.dumps(result, ensure_ascii=False, indent=2), flush=True)


if __name__ == "__main__":
    main()
```

src/19_q3_domain_verify.py

SHA-256: 47b27fb2c62595bf9fbf11563176d4af2e70047402846433426d14830b7aef92

```
"""Verify style-held-out prediction and predeclared committee search."""
from __future__ import annotations

import hashlib
import importlib
import json
from time import perf_counter

import joblib
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist
from scipy.stats import spearmanr
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from threadpoolctl import threadpool_limits

from common import SOLUTION

BASE = importlib.import_module("10_q3_verify")
PREVIOUS_VERIFY = importlib.import_module("15_q3_optimized_verify")
OUT = SOLUTION / "outputs/q3_domain"
TOL = 1e-10


def score(y, prediction):
    y, prediction = np.asarray(y, float), np.asarray(prediction, float)
    actual_var, predicted_var = np.var(y), np.var(prediction)
    actual_range = np.quantile(y, .95)-np.quantile(y, .05)
    predicted_range = np.quantile(prediction, .95)-np.quantile(prediction, .05)
    return {"mae": mean_absolute_error(y, prediction), "rmse": np.sqrt(mean_squared_error(y, prediction)),
            "r2": r2_score(y, prediction),
            "spearman": spearmanr(y, prediction).statistic if actual_var > 1e-20 and predicted_var > 1e-20 else np.nan,
            "prediction_variance": predicted_var, "target_variance": actual_var,
            "prediction_variance_ratio": predicted_var/actual_var if actual_var else np.nan,
            "prediction_std_ratio": np.sqrt(predicted_var/actual_var) if actual_var else np.nan,
            "prediction_range90": predicted_range, "target_range90": actual_range,
            "prediction_range90_ratio": predicted_range/actual_range if actual_range else np.nan,
            "mean_bias": np.mean(prediction-y)}


def check_frozen():
    details = []
    for folder, manifest_path in [(SOLUTION / "outputs/q3_optimized", OUT / "previous_hashes.json"),
                                  (SOLUTION / "outputs/q3", SOLUTION / "outputs/q3_optimized/baseline_hashes.json")]:
        manifest = BASE.read_json(manifest_path)
        paths = {path.relative_to(folder).as_posix() for path in folder.rglob("*") if path.is_file()}
        assert paths == {row["path"] for row in manifest["files"]}
        for row in manifest["files"]:
            raw = (folder/row["path"]).read_bytes()
            assert len(raw) == row["bytes"] and hashlib.sha256(raw).hexdigest() == row["sha256"], row["path"]
```

src/19_q3_domain_verify.py（续）

```
        details.append({"directory": folder.relative_to(SOLUTION).as_posix(), "unchanged_files": len(paths)})
    return details


def style_weights(styles):
    labels = np.asarray(styles)
    unique, inverse, counts = np.unique(labels, return_inverse=True, return_counts=True)
    return len(labels)/(len(unique)*counts[inverse])


def check_local_training(model, training, target, styles, calibrated):
    if "chemical" in model.named_steps:
        PREVIOUS_VERIFY.check_pipeline_training(model, training, calibrated)
    elif "imputer" in model.named_steps:
        BASE.close(model.named_steps["imputer"].statistics_, np.nanmedian(training, axis=0), "Source-excluded raw medians")
    estimator = model.named_steps["model"]
    if hasattr(estimator, "models_"):
        for member in estimator.models_:
            check_local_training(member, training, target, styles, calibrated)
    if hasattr(estimator, "x_"):
        BASE.close(estimator.x_, model[:-1].transform(training), "Local predictor training coordinates")
        BASE.close(estimator.y_, target, "Local predictor training labels")
        BASE.close(estimator.sample_weight_, style_weights(styles), "Equal-style local training weights")
    if hasattr(estimator, "target_mean_"):
        BASE.close(estimator.target_mean_, np.average(target, weights=style_weights(styles)), "Centered weighted target mean")


def check_domain(context):
    from q3_stale_domain import make_domain_pipeline, training_fit_params
    folder = OUT / "stale"
    meta, x, y45 = context["meta"], context["x"], context["y"]
    y = y45.stale_hops
    train = np.flatnonzero(~meta.tasting_category_fine.eq("Pils/Lager"))
    sources = np.flatnonzero(meta.tasting_category_fine.eq("Pils/Lager"))
    manifest = BASE.read_json(folder/"run_manifest.json")
    summary = BASE.read_json(folder/"summary.json")
    selection = BASE.read_json(folder/"selection_result.json")
    assert manifest["pils_labels_used_in_training_selection_or_intercept"] is False
    assert manifest["features"] == x.columns.tolist()
    specs = {spec["name"]: spec for spec in manifest["candidate_specs"]}
    predictions = pd.read_csv(folder/"loso_predictions.csv")
    audit = pd.read_csv(folder/"loso_split_audit.csv")
    by_style = pd.read_csv(folder/"loso_metrics_by_style.csv")
    overall = pd.read_csv(folder/"loso_metrics_overall.csv").set_index("method")
    assert len(predictions) == len(train)*len(specs)
    assert not predictions.duplicated(["method", "global_index"]).any()
    assert set(predictions.global_index) == set(train)
    assert set(predictions.method) == set(specs)
    groups = meta.tasting_category_fine.iloc[train]
    assert set(groups.unique()) == set(manifest["heldout_groups"])
    assert len(groups.unique()) == summary["loso_styles"]
    calculated_styles = []
    selected_spec = selection["config"]
    for fold, fold_rows in audit.groupby("fold"):
        heldout = fold_rows.heldout_style.unique()
```

src/19_q3_domain_verify.py（续）

```
        assert len(heldout) == 1
        style = heldout[0]
        expected_test = train[groups.to_numpy() == style]
        expected_train = train[groups.to_numpy() != style]
        assert set(fold_rows.loc[fold_rows.role.eq("test"), "global_index"]) == set(expected_test)
        assert set(fold_rows.loc[fold_rows.role.eq("train"), "global_index"]) == set(expected_train)
        assert len(fold_rows) == len(train) and not fold_rows.global_index.duplicated().any()
        assert fold_rows.beer.tolist() == fold_rows.global_index.map(meta.beer).tolist()
        model = make_domain_pipeline(selected_spec, x.columns.tolist(), context["calibrated_features"],
                                     manifest["aging_features"], manifest["loso_seed"]+int(fold))
        model.fit(x.iloc[expected_train], y.iloc[expected_train],
                  **training_fit_params(selected_spec, meta.tasting_category_fine.iloc[expected_train]))
        selected_rows = predictions[predictions.method.eq(selection["selected_method"])
                                    & predictions.fold.eq(fold)].set_index("global_index").loc[expected_test]
        BASE.close(selected_rows.prediction, np.asarray(model.predict(x.iloc[expected_test])).reshape(-1),
                   "Selected LOSO prediction refitted without the held-out style")
        for method in specs:
            rows = predictions[predictions.method.eq(method) & predictions.fold.eq(fold)].set_index("global_index").loc[expected_test]
            assert rows.heldout_style.eq(style).all()
            BASE.close(rows.observed, y.iloc[expected_test], "LOSO true labels")
            calculated_styles.append({"method": method, "heldout_style": style, "fold": fold,
                                      **score(rows.observed, rows.prediction),
                                      "bias": np.mean(rows.prediction.to_numpy()-rows.observed.to_numpy())})
            if method == "mean_global":
                BASE.close(rows.prediction, y.iloc[expected_train].mean(), "LOSO global training mean")
            if method == "mean_style_balanced":
                means = y.iloc[expected_train].groupby(meta.tasting_category_fine.iloc[expected_train]).mean()
                BASE.close(rows.prediction, means.mean(), "LOSO equal-style training mean")
    metric_names = list(score(y, y))
    BASE.same_table(by_style, pd.DataFrame(calculated_styles), ["method", "heldout_style", "fold"],
                    metric_names+["bias"], "LOSO per-style metrics")
    calculated = []
    for method, spec in specs.items():
        rows = predictions[predictions.method.eq(method)].set_index("global_index").loc[train]
        styles = by_style[by_style.method.eq(method)]
        metrics = score(rows.observed, rows.prediction)
        within_prediction = within_truth = 0.
        for style in groups.unique():
            mask = groups.to_numpy() == style
            pred, truth = rows.prediction.to_numpy()[mask], rows.observed.to_numpy()[mask]
            within_prediction += np.sum((pred-pred.mean())**2)
            within_truth += np.sum((truth-truth.mean())**2)
```

src/19_q3_domain_verify.py（续）

```
        macro_rmse, macro_bias = styles.rmse.mean(), styles.bias.abs().mean()
        calculated.append({"method": method, **metrics,
                           "macro_style_rmse": macro_rmse, "macro_style_mae": styles.mae.mean(),
                           "macro_style_abs_bias": macro_bias, "macro_style_bias": styles.bias.mean(),
                           "selection_score": macro_rmse+manifest["bias_penalty"]*macro_bias,
                           "within_style_prediction_variance_ratio": within_prediction/within_truth})
        assert bool(metrics["prediction_variance_ratio"] >= manifest["variance_ratio_min"]) == overall.at[method,"nonconstant_qualified"]
        assert spec["selectable"] == overall.at[method,"selectable"]
    BASE.same_table(overall.reset_index(), pd.DataFrame(calculated), ["method"],
                    [key for key in calculated[0] if key != "method"], "LOSO macro and pooled metrics")
    best_mean = overall.loc[["mean_global", "mean_style_balanced"], "macro_style_rmse"].min()
    eligible = overall[overall.selectable & overall.nonconstant_qualified]
    fallback = eligible.empty
    if fallback:
        eligible = overall[overall.selectable]
    winner = eligible.reset_index().sort_values(["selection_score", "method"]).iloc[0]
    assert winner.method == selection["selected_method"] == summary["selected_method"]
    assert fallback == selection["fallback_used"]
    qualified = bool(winner.nonconstant_qualified and winner.macro_style_rmse < best_mean)
    assert qualified == selection["qualified_for_proxy"] == summary["qualified_for_proxy"]
    artifact = joblib.load(folder/"transfer_stale_model.joblib")
    ensemble = joblib.load(folder/"bootstrap_ensemble.joblib")
    for saved in [artifact, ensemble]:
        np.testing.assert_array_equal(saved["train_global_indices"], train)
        np.testing.assert_array_equal(saved["excluded_global_indices"], sources)
        assert saved["selected_config"] == selected_spec and saved["qualified_for_proxy"] == qualified
        assert saved["pils_diagnostic_used_for_selection"] is False
    check_local_training(artifact["pipeline"], x.iloc[train], y.iloc[train], groups, context["calibrated_features"])
    indices = np.asarray(ensemble["bootstrap_indices"])
    assert indices.shape == (60,len(train)) and not np.isin(indices,sources).any()
    assert ensemble["selection_model_indices"] == list(range(40)) and ensemble["audit_model_indices"] == list(range(40,60))
    with np.load(folder/"bootstrap_indices.npz") as saved:
        np.testing.assert_array_equal(saved["indices"],indices)
        np.testing.assert_array_equal(saved["bootstrap_seeds"],ensemble["bootstrap_seeds"])
    for b,(model,index,seed) in enumerate(zip(ensemble["models"],indices,ensemble["bootstrap_seeds"])):
        expected_seed = manifest["selection_bootstrap_seed"]+b if b<40 else manifest["audit_bootstrap_seed"]+b-40
        assert seed == expected_seed
        np.testing.assert_array_equal(index,np.random.default_rng(seed).choice(train,len(train),replace=True))
        check_local_training(model,x.iloc[index],y.iloc[index],meta.tasting_category_fine.iloc[index],context["calibrated_features"])
    diagnostic = pd.read_csv(folder/"pils_diagnostic_predictions.csv")
    diagnostic_metrics = pd.read_csv(folder/"pils_diagnostic_metrics.csv").set_index("method")
    assert set(diagnostic.method) == set(specs) and len(diagnostic)==len(sources)*len(specs)
```

src/19_q3_domain_verify.py（续）

```
    for method in specs:
        rows=diagnostic[diagnostic.method.eq(method)].set_index("global_index").loc[so
urces]
        BASE.close(rows.observed,y.iloc[sources],"Pils diagnostic labels")
        BASE.close(diagnostic_metrics.loc[method,metric_names],list(score(rows.observe
d,rows.prediction).values()),"Pils diagnostic metrics")
    rows=diagnostic[diagnostic.method.eq(selection["selected_method"])].set_index("glo
bal_index").loc[sources]
    BASE.close(rows.prediction,np.asarray(artifact["pipeline"].predict(x.iloc[sources]
)).reshape(-1),"Frozen selected Pils diagnostic")
    if selected_spec["family"]=="local":
        local_rows=[]
        final=artifact["pipeline"];final_estimator=final.named_steps["model"]
        k=final_estimator.n_neighbors
        final_distance=cdist(final[:-1].transform(x.iloc[sources]),final_estimator.x_)
        final_neighbors=[set(train[np.argpartition(distance,k-1)[:k]]) for distance in
 final_distance]
        for b,(model,index) in enumerate(zip(ensemble["models"],indices)):
            estimator=model.named_steps["model"]
            distances=cdist(model[:-1].transform(x.iloc[sources]),estimator.x_)
            predicted=np.asarray(model.predict(x.iloc[sources])).reshape(-1)
            for j,(source_index,distance) in enumerate(zip(sources,distances)):
                near=np.argpartition(distance,k-1)[:k]
                neighbors=set(index[near]);weights=estimator.sample_weight_[near]/np.m
aximum(distance[near],1e-12)
                _,inverse=np.unique(index[near],return_inverse=True)
                by_beer=np.bincount(inverse,weights=weights)
                unique_effective=by_beer.sum()**2/np.sum(by_beer**2)
                assert unique_effective<=len(neighbors)+1e-10
                local_rows.append({"beer":int(meta.beer.iloc[source_index]),"model_ind
ex":b,
                    "prediction":predicted[j],"nearest_unique_beers":len(neighbors),
                    "neighbor_jaccard_vs_final":len(neighbors&final_neighbors[j])/len(
neighbors|final_neighbors[j]),
                    "effective_resampled_position_count":weights.sum()**2/np.sum(weigh
ts**2),
                    "effective_unique_beer_count":unique_effective,
                    "imputer_indicator_count":len(model.named_steps["imputer"].indicat
or_.features_)})
        expected_neighborhood=pd.DataFrame(local_rows)
        BASE.same_table(pd.read_csv(folder/"neighborhood_bootstrap_diagnostics.csv"),e
xpected_neighborhood,
                        ["beer","model_index"],[column for column in expected_neighbor
hood if column not in {"beer","model_index"}],
                        "Bootstrap neighborhood and unique-beer influence diagnostics"
)
        expected_summary=expected_neighborhood.groupby("beer").agg(
            bootstrap_prediction_sd=("prediction","std"),median_neighbor_jaccard=("nei
ghbor_jaccard_vs_final","median"),
            mean_unique_neighbors=("nearest_unique_beers","mean"),
            median_effective_resampled_position_count=("effective_resampled_position_c
ount","median"),
            median_effective_unique_beer_count=("effective_unique_beer_count","median"
)).reset_index()
        BASE.same_table(pd.read_csv(folder/"neighborhood_bootstrap_summary.csv"),expec
ted_summary,["beer"],
                        [column for column in expected_summary if column!="beer"],"Nei
ghborhood diagnostic summaries")
    context.update({"domain_model":artifact,"domain_ensemble":ensemble,"train":train,"
sources":sources})
    return {"non_source_rows":len(train),"heldout_styles":len(groups.unique()),"candid
```

src/19_q3_domain_verify.py（续）

```
    ate_count":len(specs),
232             "prediction_rows":len(predictions),"selected_method":selection["selected_m
    ethod"],
233             "selected_loso_predictions_refitted":len(groups.unique()),"qualified":qual
    ified,
234             "selected_macro_rmse":float(winner.macro_style_rmse),"best_mean_macro_rmse
    ":float(best_mean),
235             "selected_pils_diagnostic_rmse":float(diagnostic_metrics.at[selection["sel
    ected_method"],"rmse"]),
236             "bootstrap_indices_shape":list(indices.shape),"pils_is_fresh_test":False}
237
238
239 def counts(frame):
240     return {"sources":len(frame),"selected":int(frame.candidate_index.notna().sum()),
241             "new_model_checks_passed":int(frame.all_model_checks_passed.sum()),
242             "historical_cross_proxy_passed":int(frame.cross_proxy_checks_passed.sum())
    ,
243             "fresh_cross_proxy_passed":int(frame.fresh_cross_proxy_checks_passed.sum()
    ),
244             "all_committees_passed":int(frame.all_committees_passed.sum()),
245             "no_candidate":int(frame.status.eq("no_candidate_found").sum())}
246
247
248 def check_transfer(context,experiment="transfer"):
249     folder=OUT/experiment
250     meta,x,y=context["meta"],context["x"],context["y"]
251     train,sources=context["train"],context["sources"]
252     manifest=BASE.read_json(folder/"run_manifest.json")
253     summary=BASE.read_json(folder/"summary.json")
254     proxy_folder=SOLUTION/manifest.get("proxy_directory","outputs/q3_domain/stale")
255     current_model=joblib.load(proxy_folder/"transfer_stale_model.joblib")
256     frozen_ensemble=joblib.load(proxy_folder/"bootstrap_ensemble.joblib")
257     current_ensemble=joblib.load(folder/"screening_and_audit.joblib") if (folder/"scre
    ening_and_audit.joblib").exists() else frozen_ensemble
258     for artifact in [current_model,current_ensemble]:
259         np.testing.assert_array_equal(artifact["train_global_indices"],train)
260         np.testing.assert_array_equal(artifact["excluded_global_indices"],sources)
261     if manifest.get("fresh_domain_audit_seed") is not None:
262         for b in range(40):
263             np.testing.assert_array_equal(current_ensemble["bootstrap_indices"][b],fro
    zen_ensemble["bootstrap_indices"][b])
264             BASE.close(current_ensemble["models"][b].predict(x.iloc[train]),frozen_ens
    emble["models"][b].predict(x.iloc[train]),"Frozen follow-up screening models")
265         for b in range(20):
266             seed=manifest["fresh_domain_audit_seed"]+b
267             index=current_ensemble["bootstrap_indices"][40+b]
268             assert current_ensemble["bootstrap_seeds"][40+b]==seed
269             np.testing.assert_array_equal(index,np.random.default_rng(seed).choice(tra
    in,len(train),replace=True))
270             tree=current_ensemble["models"][40+b].named_steps["model"]
271             assert tree.n_outputs_==1 and tree.n_estimators==120 and tree.min_samples_
    leaf==3 and tree.max_features==.7
272             BASE.close(current_ensemble["models"][40+b].named_steps["imputer"].statist
    ics_,np.nanmedian(x.iloc[index],axis=0),"Fresh current-protocol audit medians")
273     assert manifest["default_pool"]=="matched_2004" and manifest["default_policy"]=="j
    oint_margin"
274     assert manifest["audit_reselection"] is False and manifest["default_selected_from_
    audit"] is False
275     scenarios=manifest["threshold_scenarios"]
276     assert scenarios==BASE.read_json(SOLUTION/"outputs/q3_optimized/transfer/threshold
    _scenarios.json")
```

src/19_q3_domain_verify.py（续）

```
    geometry=joblib.load(SOLUTION/"outputs/q3/transfer/transfer_copula.joblib")
    reference=geometry["copula"]
    np.testing.assert_array_equal(geometry["training_rows"],train)
    draws=pd.read_csv(folder/"base_draws.csv")
    previous_draws=pd.read_csv(SOLUTION/"outputs/q3_optimized/transfer/optimized_base_draws.csv")
    BASE.close(draws.iloc[:500],previous_draws,"Preserved matched base draws")
    BASE.close(draws.iloc[500:],reference.sample("Tripel",1000,manifest["additional_draw_seed"]),"Additional frozen-seed draws")
    assert len(draws)==1500 and draws.columns.tolist()==x.columns.tolist()
    original=joblib.load(SOLUTION/"outputs/q3/sensory/stale_bootstrap.joblib")
    old_center=joblib.load(SOLUTION/"outputs/q3/sensory/transfer_proxy.joblib")
    fresh=joblib.load(folder/"fresh_original_protocol_audit.joblib")
    for saved in [original,fresh]:
        np.testing.assert_array_equal(saved["train_global_indices"],train)
        assert saved["features"]==x.columns.tolist()
    assert old_center["pipeline"].named_steps["model"].n_outputs_==45
    assert len(fresh["models"])==20
    for saved in [original,fresh]:
        for model,index in zip(saved["models"],saved["bootstrap_indices"]):
            tree=model.named_steps["model"]
            assert tree.n_outputs_==1 and tree.n_estimators==120 and tree.min_samples_leaf==3 and tree.max_features==.7
            assert not np.isin(index,sources).any()
            BASE.close(model.named_steps["imputer"].statistics_,np.nanmedian(x.iloc[index],axis=0),"Original-protocol audit training medians")
    for b,(index,seed) in enumerate(zip(fresh["bootstrap_indices"],fresh["bootstrap_seeds"])):
        assert seed==manifest["fresh_audit_seed"]+b
        np.testing.assert_array_equal(index,np.random.default_rng(seed).choice(train,len(train),replace=True))
    target=x.loc[meta.tasting_category_fine.eq("Tripel")]
    tz=reference.transform(target); dimension=x.shape[1]
    pairwise=cdist(tz,tz)/np.sqrt(dimension);np.fill_diagonal(pairwise,np.inf)
    support_limit=np.quantile(pairwise.min(axis=1),.95)
    BASE.close(support_limit,summary["support_nn_limit"],"Frozen full-dimensional support")
    minimum,maximum=x.iloc[train].min().to_numpy(),x.iloc[train].max().to_numpy()
    medians=x.iloc[train].median();target_median=target.median().to_numpy()
    report=pd.read_csv(folder/"transfer_summary.csv")
    diagnostics=pd.read_csv(folder/"candidate_diagnostics.csv")
    spectra=pd.read_csv(folder/"transferred_spectra.csv").set_index("transfer_id")
    deltas_csv=pd.read_csv(folder/"stale_paired_deltas.csv")
    thresholds=pd.read_csv(folder/"threshold_sensitivity.csv")
    previous_checks=pd.read_csv(folder/"previous_same_spectra_recheck.csv").set_index("source_beer")
    previous_summary=pd.read_csv(SOLUTION/"outputs/q3_optimized/transfer/transfer_summary.csv")
    previous_summary=previous_summary[previous_summary.is_default_policy].set_index("source_beer")
    previous_spectra=pd.read_csv(SOLUTION/"outputs/q3_optimized/transfer/transferred_spectra.csv").set_index("transfer_id")
    models=current_ensemble["models"]
    selected_predictions=np.vstack([np.asarray(model.predict(spectra[x.columns])).reshape(-1) for model in models])
    original_predictions=np.vstack([np.asarray(model.predict(spectra[x.columns])).reshape(-1) for model in original["models"]])
    fresh_predictions=np.vstack([np.asarray(model.predict(spectra[x.columns])).reshape(-1) for model in fresh["models"]])
    assert len(report)==len(sources)*len(manifest["pools"])*len(manifest["policies"])
    assert set(report[report.candidate_index.notna()].transfer_id)==set(spectra.index)
```

```
    selections=scenario_count=0; maximum_quantile_error=0.
    qualified=bool(current_model["qualified_for_proxy"])
    for source_index in sources:
        beer=int(meta.beer.iloc[source_index]);raw=x.iloc[[source_index]];source=raw.fillna(medians)
        caps={}
        for field in BASE.DEFECT_FIELDS:
            levels=[scenario["thresholds"][field] for scenario in scenarios if scenario["thresholds"][field] is not None]
            caps[field]=max(float(raw[field].iloc[0]),min(levels)) if levels else float(raw[field].iloc[0])
        arrays=[];gammas=[];draw_ids=[];matched=[]
        for gamma in manifest["gammas"]:
            arrays.append((1-gamma)*source.to_numpy()[0]+gamma*draws.to_numpy())
            gammas.extend([gamma]*len(draws));draw_ids.extend(range(len(draws)));matched.extend(np.arange(len(draws))<500)
        for gamma in manifest["gammas"]:
            arrays.append(((1-gamma)*source.to_numpy()[0]+gamma*target_median)[None])
            gammas.append(gamma);draw_ids.append(-1);matched.append(True)
        values=pd.DataFrame(np.vstack(arrays),columns=x.columns)
        for field,cap in caps.items(): values[field]=values[field].clip(lower=0,upper=cap)
        z,sz=reference.transform(values),reference.transform(source)
        distance=reference.distance(values,"Tripel");before=float(reference.distance(source,"Tripel")[0])
        nn=cdist(z,tz).min(axis=1)/np.sqrt(dimension)
        change=np.linalg.norm(z-sz,axis=1)/np.sqrt(dimension);improvement=1-distance/max(before,1e-8)
        objective=distance/max(before,1e-8)+manifest["lambda"]*change/max(float(np.linalg.norm(sz[0]-tz.mean(axis=0))/np.sqrt(dimension)),1e-8)
        support=(nn<=support_limit+TOL)&(values.to_numpy()>=minimum-TOL).all(axis=1)&(values.to_numpy()<=maximum+TOL).all(axis=1)
        chemical=BASE.chemical_flags(values,caps)
        source_new=np.array([float(np.asarray(model.predict(source)).reshape(-1)[0]) for model in models])
        source_old=np.array([float(np.asarray(model.predict(source)).reshape(-1)[0]) for model in original["models"]])
        source_fresh=np.array([float(np.asarray(model.predict(source)).reshape(-1)[0]) for model in fresh["models"]])
        unique,inverse=np.unique(values.to_numpy(),axis=0,return_inverse=True);unique=pd.DataFrame(unique,columns=x.columns)
        new_delta=np.column_stack([np.asarray(model.predict(unique)).reshape(-1)-source_new[b] for b,model in enumerate(models[:40])])[inverse]
        old_delta=np.column_stack([np.asarray(model.predict(unique)).reshape(-1)-source_old[b] for b,model in enumerate(original["models"][:30])])[inverse]
        nq,oq=np.quantile(new_delta,.95,axis=1),np.quantile(old_delta,.95,axis=1);worst=np.maximum(nq,oq)
        new_signal=np.any(np.abs(new_delta)>1e-8,axis=1);old_signal=np.any(np.abs(old_delta)>1e-8,axis=1)
        base_ok=support&chemical&(improvement>=.1)&(change>=.05)
        single=base_ok&(nq<=TOL)&new_signal;joint=single&(oq<=TOL)&old_signal
        diag=diagnostics[diagnostics.source_beer.eq(beer)].sort_values("candidate_index")
        assert len(diag)==len(values)==manifest["pools"]["expanded_6004"]
        np.testing.assert_array_equal(diag.candidate_index,np.arange(len(values)))
        np.testing.assert_array_equal(diag.draw,draw_ids);BASE.close(diag.gamma,gammas,"Interpolation gamma")
        np.testing.assert_array_equal(diag.matched_budget,matched)
        BASE.close(diag[["target_distance","target_improvement","target_nn_rms","change_latent_rms","selection_new_q95","selection_old_q95","worst_selection_q95","geometry_objective"]],
```

src/19_q3_domain_verify.py（续）

```
                    np.column_stack([distance,improvement,nn,change,nq,oq,worst,objecti
ve]),"Reconstructed screening-only candidate statistics")
        for column,expected in [("support_ok",support),("all_thresholds_chemical_ok",c
hemical),("new_signal",new_signal),
                                ("old_signal",old_signal),("single_feasible",single),(
"joint_feasible",joint)]:
            np.testing.assert_array_equal(diag[column],expected)
        for _,row in report[report.source_beer.eq(beer)].iterrows():
            mask=np.asarray(matched) if row.pool=="matched_2004" else np.ones(len(valu
es),bool)
            eligible=(single if row.policy=="single_geometry" else joint)&mask;ids=np.
flatnonzero(eligible)
            assert row.n_candidates==mask.sum() and row.n_selection_feasible==len(ids)
            nv,ov=np.std(new_delta[mask].mean(axis=1)),np.std(old_delta[mask].mean(axi
s=1))
            BASE.close([row.new_candidate_delta_sd,row.old_candidate_delta_sd],[nv,ov]
,"Per-pool proxy response")
            if not len(ids): assert pd.isna(row.candidate_index) and row.status=="no_c
andidate_found";continue
            chosen=int(ids[np.lexsort((objective[ids],worst[ids]))[0]]) if row.policy=
="joint_margin" else int(ids[np.argmin(objective[ids])])
            assert chosen==int(row.candidate_index),"Audit entered candidate selection
"
            selections+=1;final=spectra.loc[[row.transfer_id],x.columns];BASE.close(fi
nal,values.iloc[[chosen]],"Rebuilt selected spectrum")
            position=spectra.index.get_loc(row.transfer_id)
            nd=selected_predictions[:,position]-source_new;od=original_predictions[:,p
osition]-source_old;fd=fresh_predictions[:,position]-source_fresh
            groups_delta={"new_selection":nd[:40],"old_selection":od[:30],"new_audit":
nd[40:],"old_historical":od[30:],"fresh_original_audit":fd}
            actual=[row.selection_new_q95,row.selection_old_q95,row.new_audit_q95,row.
old_historical_audit_q95,row.fresh_original_audit_q95]
            expected=[np.quantile(difference,.95) for difference in groups_delta.value
s()]
            BASE.close(actual,expected,"Selected paired screening and audit quantiles"
)
            maximum_quantile_error=max(maximum_quantile_error,float(np.max(np.abs(np.a
sarray(actual)-expected))))
            new_pass=bool(qualified and nv>1e-6 and new_signal[chosen] and expected[2]
<=TOL)
            historical=bool(new_pass and ov>1e-6 and old_signal[chosen] and expected[1
]<=TOL and expected[3]<=TOL)
            fresh_pass=bool(new_pass and ov>1e-6 and old_signal[chosen] and expected[1
]<=TOL and expected[4]<=TOL)
            assert [new_pass,historical,fresh_pass,historical and fresh_pass]==[row.al
l_model_checks_passed,row.cross_proxy_checks_passed,row.fresh_cross_proxy_checks_passe
d,row.all_committees_passed]
            for label,difference in groups_delta.items():
                stored=deltas_csv[deltas_csv.transfer_id.eq(row.transfer_id)&deltas_cs
v.ensemble.eq(label)].sort_values("model_index")
                np.testing.assert_array_equal(stored.model_index,np.arange(len(differe
nce)));BASE.close(stored.paired_delta,difference,"Per-model paired difference")
            checks=thresholds[thresholds.transfer_id.eq(row.transfer_id)]
            assert len(checks)==len(scenarios)*len(BASE.DEFECT_FIELDS)
            for check in checks.itertuples(index=False):
                scenario=next(s for s in scenarios if s["scenario"]==check.scenario);t
hreshold=scenario["thresholds"][check.feature]
                observed=float(raw[check.feature].iloc[0]);cap=max(observed,threshold)
 if threshold is not None else observed
                value=float(final[check.feature].iloc[0]);BASE.close([check.source,che
ck.transferred,check.upper_bound],[observed,value,cap],"Scenario concentration")
```

src/19_q3_domain_verify.py（续）

```
                assert bool(0<=value<=cap+TOL)==check.passed;scenario_count+=1
        previous=previous_spectra.loc[[previous_summary.at[beer,"transfer_id"]],x.columns]
        nd=np.array([float(np.asarray(model.predict(previous)).reshape(-1)[0])-source_new[b] for b,model in enumerate(models)])
        od=np.array([float(np.asarray(model.predict(previous)).reshape(-1)[0])-source_old[b] for b,model in enumerate(original["models"])])
        fd=np.array([float(np.asarray(model.predict(previous)).reshape(-1)[0])-source_fresh[b] for b,model in enumerate(fresh["models"])])
        pq=[np.quantile(nd[:40],.95),np.quantile(od[:30],.95),np.quantile(nd[40:],.95),np.quantile(od[30:],.95),np.quantile(fd,.95)]
        recorded=previous_checks.loc[beer]
        BASE.close(recorded[["selection_new_q95","selection_old_q95","new_audit_q95","old_historical_audit_q95","fresh_original_audit_q95"]],pq,"Previous frozen-spectrum current-rule quantiles")
        pz=reference.transform(previous);pi=1-float(reference.distance(previous,"Tripel")[0])/max(before,1e-8)
        pg=bool(pi>=.1 and np.linalg.norm(pz-sz)/np.sqrt(dimension)>=.05 and cdist(pz,tz).min()/np.sqrt(dimension)<=support_limit+TOL
                and (previous.to_numpy()>=minimum-TOL).all() and (previous.to_numpy()<=maximum+TOL).all())
        pc=bool(BASE.chemical_flags(previous,caps)[0]);assert pc==recorded.all_thresholds_chemical_ok and pg==recorded.geometry_ok
        matched_mask=np.asarray(matched)
        pn=bool(qualified and pg and pc and pq[0]<=TOL and pq[2]<=TOL and np.any(np.abs(nd[:40])>1e-8) and np.std(new_delta[matched_mask].mean(axis=1))>1e-6)
        po=bool(pq[1]<=TOL and np.any(np.abs(od[:30])>1e-8) and np.std(old_delta[matched_mask].mean(axis=1))>1e-6)
        assert [pn,pn and po and pq[3]<=TOL,pn and po and pq[4]<=TOL,pn and po and pq[3]<=TOL and pq[4]<=TOL]==list(recorded[["all_model_checks_passed","cross_proxy_checks_passed","fresh_cross_proxy_checks_passed","all_committees_passed"]])
        print(f"Verified domain candidates: source {beer}",flush=True)
    policies=[]
    for (pool,policy),group in report.groupby(["pool","policy"]):policies.append({"pool":pool,"policy":policy,**counts(group)})
    BASE.same_table(pd.read_csv(folder/"policy_summary.csv"),pd.DataFrame(policies),["pool","policy"],list(counts(report)),"Domain policy summaries")
    default=report[report.is_default_policy];assert default.pool.eq(manifest["default_pool"]).all() and default.policy.eq(manifest["default_policy"]).all()
    assert counts(default)==summary["default_policy"]
    for name,column in [("new_model_checks_passed","all_model_checks_passed"),("historical_cross_proxy_passed","cross_proxy_checks_passed"),
                        ("fresh_cross_proxy_passed","fresh_cross_proxy_checks_passed"),("all_committees_passed","all_committees_passed")]:
        assert int(previous_checks[column].sum())==summary["previous_same_spectra_current_rules"][name]
    context.setdefault("transfer_summaries",{})[experiment]=summary
    return {"experiment":experiment,"reconstructed_candidates":len(diagnostics),"selection_only_argmin_checks":selections,"scenario_concentration_checks":scenario_count,
            "maximum_paired_quantile_error":maximum_quantile_error,"default_policy":counts(default),
            "original_committee_outputs":1,"original_center_outputs":45,"fresh_original_protocol_models":20}


def check_reports(context):
    tables=OUT/"report_tables"
    provenance=BASE.read_json(OUT/"report_source_manifest.json")
    for source in provenance:
        path=SOLUTION/source["path"]
        assert hashlib.sha256(path.read_bytes()).hexdigest()==source["sha256"],source[
```

src/19_q3_domain_verify.py（续）

```
    "path"]
435         if source.get("rows") is not None:assert len(pd.read_csv(path))==source["rows"
    ]
436     overall=pd.read_csv(OUT/"stale/loso_metrics_overall.csv")
437     pils=pd.read_csv(OUT/"stale/pils_diagnostic_metrics.csv")
438     selected=context["domain_model"]["selected_method"]
439     expected=overall.merge(pils[["method","rmse","mae","r2"]].rename(
440         columns={"rmse":"pils_rmse","mae":"pils_mae","r2":"pils_r2"}),on="method",vali
    date="one_to_one")
441     comparison=pd.read_csv(tables/"prediction_method_comparison.csv")
442     BASE.same_table(comparison,expected,["method"],["macro_style_rmse","pils_rmse","pi
    ls_mae","pils_r2"],"Report prediction comparison")
443     assert comparison.is_selected.equals(comparison.method.eq(selected))
444     by_style=pd.read_csv(OUT/"stale/loso_metrics_by_style.csv")
445     BASE.same_table(pd.read_csv(tables/"selected_loso_by_style.csv"),by_style[by_style
    .method.eq(selected)],
446                     ["heldout_style"],["rmse","mae","bias"],"Report selected LOSO styl
    es")
447     predictions=pd.read_csv(OUT/"stale/pils_diagnostic_predictions.csv")
448     influence=pd.read_csv(tables/"pils_source_error_influence.csv").set_index("beer")
449     assert set(influence.index)==set(context["meta"].beer.iloc[context["sources"]])
450     method_map={"domain_prediction":selected,"matched_single_tree_prediction":"fixed_s
    ingle_tree",
451                 "previous_single_tree_prediction":"fixed_single_tree","old_multioutput
    _prediction":"old_multioutput_tree",
452                 "training_mean_prediction":"mean_global"}
453     for column,method in method_map.items():
454         if column not in influence:continue
455         rows=predictions[predictions.method.eq(method)].set_index("beer").loc[influenc
    e.index]
456         BASE.close(influence[column],rows.prediction,"Report source prediction influen
    ce")
457         BASE.close(influence.observed,rows.observed,"Report source observed values")
458     for column in influence:
459         if column.endswith("_sse_share"):
460             prefix=column.removesuffix("_sse_share")
461             prediction_column="domain_prediction" if prefix=="domain_selected" else pr
    efix+"_prediction"
462             if prediction_column in influence:
463                 squared=(influence[prediction_column]-influence.observed)**2
464                 BASE.close(influence[column],squared/squared.sum(),"Report per-source
    squared-error shares")
465     check_names={"new":"all_model_checks_passed","historical":"cross_proxy_checks_pass
    ed",
466                  "fresh":"fresh_cross_proxy_checks_passed","all":"all_committees_passe
    d"}
467     for experiment,folder_name in [("domain","transfer"),("consensus","consensus")]:
468         folder=OUT/folder_name
469         transfer=pd.read_csv(folder/"transfer_summary.csv")
470         manifest=BASE.read_json(folder/"run_manifest.json")
471         previous=pd.read_csv(folder/"previous_same_spectra_recheck.csv")
472         thresholds=pd.read_csv(folder/"threshold_sensitivity.csv")
473         expected_policies=[]
474         for (pool,policy),group in transfer.groupby(["pool","policy"]):
475             valid=group.candidate_index.notna()
476             expected_policies.append({"pool":pool,"policy":policy,"sources":len(group)
    ,"selected":int(valid.sum()),
477                 **{name+"_passed":int(group[column].sum()) for name,column in check_na
    mes.items()},
478                 "median_target_improvement_selected":group.loc[valid,"target_improveme
    nt"].median(),
```

src/19_q3_domain_verify.py（续）

```
                "median_change_rms_selected":group.loc[valid,"change_latent_rms"].median(),
                "median_target_improvement_fresh_pass":group.loc[group.fresh_cross_proxy_checks_passed,"target_improvement"].median()})
        expected_policies=pd.DataFrame(expected_policies)
        BASE.same_table(pd.read_csv(tables/f"{experiment}_policy_comparison.csv"),expected_policies,["pool","policy"],
                        [column for column in expected_policies if column not in {"pool","policy"}],"Report policy counts and tradeoffs")
        default=transfer[transfer.is_default_policy]
        passed=thresholds.groupby(["transfer_id","scenario"]).passed.all()
        expected_scenarios=[]
        for scenario in manifest["threshold_scenarios"]:
            chemical=np.array([bool(passed.get((transfer_id,scenario["scenario"]),False)) for transfer_id in default.transfer_id])
            expected_scenarios.append({"scenario":scenario["scenario"],"sources":len(default),"chemical_only":int(chemical.sum()),
                                       **{name+"_passed":int((chemical&default[column].to_numpy(bool)).sum()) for name,column in check_names.items()}})
        expected_scenarios=pd.DataFrame(expected_scenarios)
        BASE.same_table(pd.read_csv(tables/f"{experiment}_default_threshold_pass_counts.csv"),expected_scenarios,["scenario"],
                        [column for column in expected_scenarios if column!="scenario"],"Report default threshold counts")
        source_comparison=pd.read_csv(tables/f"{experiment}_default_source_comparison.csv").set_index("source_beer")
        BASE.close(source_comparison.loc[default.source_beer,"target_improvement"],default.target_improvement,"Report default source geometry")
        for column in check_names.values():
            np.testing.assert_array_equal(source_comparison.loc[default.source_beer,column],default[column])
            np.testing.assert_array_equal(source_comparison.loc[previous.source_beer,"previous_"+column],previous[column])
    target_comparison=pd.read_csv(tables/"all_default_target_comparison.csv").set_index("source_beer")
    for prefix,folder_name,pool in [("domain","transfer","matched_2004"),("consensus","consensus","matched_2004"),
                                    ("expanded","consensus","expanded_6004")]:
        rows=pd.read_csv(OUT/folder_name/"transfer_summary.csv")
        rows=rows[rows.pool.eq(pool)&rows.policy.eq("joint_margin")]
        BASE.close(target_comparison.loc[rows.source_beer,prefix+"_target_improvement"],rows.target_improvement,"Report target comparison")
        BASE.close(target_comparison.loc[rows.source_beer,prefix+"_change_latent_rms"],rows.change_latent_rms,"Report chemical-change comparison")
    report_path=SOLUTION/"docs/q3_domain_results.md"
    text=report_path.read_text(encoding="utf-8")
    assert len(text)>100 and selected in text
    assert f"{pils.set_index('method').at[selected,'rmse']:.4f}" in text
    assert "不是" in text or "不能" in text
    figures=["prediction_domain_validation","transfer_policy_and_thresholds","target_distance_and_consistency"]
    for name in figures:
        for extension in ["png","svg"]:
            path=OUT/"figures"/f"{name}.{extension}"
            assert path.is_file() and path.stat().st_size>100
    report_summary=BASE.read_json(OUT/"report_summary.json")
    assert report_summary["selected_method"]==selected
    BASE.close(report_summary["selected_pils_diagnostic_rmse"],pils.set_index("method").at[selected,"rmse"],"Report diagnostic summary")
    comparison_by_method=comparison.set_index("method")
    baseline_min=comparison_by_method.loc[["fixed_single_tree","old_multioutput_tree",
```

src/19_q3_domain_verify.py（续）

```
    "mean_global"],"pils_rmse"].min()
521     assert report_summary["pils_prediction_improved"]==bool(comparison_by_method.at[se
    lected,"pils_rmse"]<baseline_min)
522     for experiment,folder_name in [("domain","transfer"),("consensus","consensus")]:
523         transfer=pd.read_csv(OUT/folder_name/"transfer_summary.csv")
524         default=transfer[transfer.is_default_policy]
525         previous=pd.read_csv(OUT/folder_name/"previous_same_spectra_recheck.csv")
526         expected_default={"sources":len(default),"selected":int(default.candidate_inde
    x.notna().sum()),
527                           **{name:int(default[column].sum()) for name,column in check_
    names.items()}}
528         expected_previous={"sources":len(previous),**{name:int(previous[column].sum())
     for name,column in check_names.items()}}
529         assert report_summary["experiments"][experiment]["default"]==expected_default
530         assert report_summary["experiments"][experiment]["previous_same_spectra"]==exp
    ected_previous
531     consensus=pd.read_csv(OUT/"consensus/transfer_summary.csv")
532     expanded=consensus[consensus.pool.eq("expanded_6004")&consensus.policy.eq("joint_m
    argin")]
533     expanded_summary=report_summary["consensus_expanded_same_policy"]
534     assert expanded_summary["budget"]==6004 and expanded_summary["sources"]==len(expan
    ded)
535     assert expanded_summary["selected"]==expanded.candidate_index.notna().sum()
536     assert expanded_summary["fresh"]==expanded.fresh_cross_proxy_checks_passed.sum()
537     BASE.close([expanded_summary["median_target_improvement_selected"],expanded_summar
    y["median_change_rms_selected"]],
538                [expanded.target_improvement.median(),expanded.change_latent_rms.median
    ()],"Report expanded-budget tradeoffs")
539     return {"hashed_report_sources":len(provenance),"prediction_comparisons":len(compa
    rison),
540             "experiments_with_policy_and_scenario_tables":2,"source_diagnostics":len(i
    nfluence),
541             "figure_pairs":len(figures),"results_report_exists":True}
542
543
544 def main():
545     start=perf_counter();result={"status":"running","checks":[]};active="frozen_previo
    us_results_before"
546     try:
547         result["checks"].append({"name":active,"details":check_frozen()})
548         context,contract=BASE.load_contract();provenance=BASE.check_hashes()
549         result["checks"].append({"name":"original_inputs","details":{"contract":contra
    ct,"provenance":provenance}})
550         for active,operation in [("non_source_loso_selection",check_domain),("predecla
    red_transfer_search",check_transfer),
551                                  ("fixed_protocol_consensus_followup",lambda ctx:check
    _transfer(ctx,"consensus")),("final_artifacts",check_reports)]:
552             details=operation(context);result["checks"].append({"name":active,"details
    ":details});print(f"Verified: {active}",flush=True)
553         active="frozen_previous_results_after";result["checks"].append({"name":active,
    "details":check_frozen()})
554         result["status"]="passed";result["limitations"]=["Pils results are repeatedly
    viewed diagnostics, not fresh external tests.",
555             "LOSO-selected scores are optimistic after candidate selection; bootstrap
    fixes the chosen configuration.",
556             "Committee consistency and audit quantiles do not establish true taste or
    causal defect preservation."]
557     except Exception as error:
558         result.update({"status":"failed","failed_check":active,"error":f"{type(error).
    __name__}: {error}"});raise
559     finally:
```

src/19_q3_domain_verify.py（续）

```
560         result["elapsed_seconds"]=perf_counter()-start
561         (OUT/"verification.json").write_text(json.dumps(result,ensure_ascii=False,inde
    nt=2,allow_nan=False)+"\n",encoding="utf-8", newline="\n")
562         print(json.dumps(result,ensure_ascii=False,indent=2,allow_nan=False),flush=Tru
    e)
563
564
565 if __name__=="__main__":
566     with threadpool_limits(limits=1):main()
```

src/20_q3_transfer_consensus.py

SHA-256: dda1e13b939733bb8c569a3d452e8e36c3b170a98035caf726be04bdc6c31fcf

```
"""Fixed-tree consensus follow-up after unsuccessful LOSO domain transfer.

The domain experiment remains an unsuccessful recorded experiment. This
follow-up isolates joint screening at the previous fixed-tree protocol;
its defaults are fixed before any of its new audit draws are evaluated.
"""
from importlib import import_module

from threadpoolctl import threadpool_limits

from common import SOLUTION


if __name__=="__main__":
    transfer=import_module("17_q3_transfer_domain")
    with threadpool_limits(limits=1):
        transfer.main(output_directory=SOLUTION/"outputs/q3_domain/consensus",
            proxy_directory=SOLUTION/"outputs/q3_optimized/stale_fixed",
            experiment="fixed_tree_consensus_followup",
            fresh_domain_audit_seed=20290123,
            fresh_original_audit_seed=20291123)
```

src/21_q3_beer81_audit.py

SHA-256: 20907975c2045de3afabc81913c3e7a98d49be21194eb90a9147d7ac00ab5e3e

```
"""Trace beer 81 identity, chemistry, sensory evidence and prediction influence."""
from __future__ import annotations

import hashlib
import json
from pathlib import Path
import re

import joblib
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist
from scipy.stats import spearmanr
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from threadpoolctl import threadpool_limits

from common import KEYS, ROOT, SOLUTION, data_dir, resolve_recorded_path
from q3_data import load_q3_data
from q3_stale_optimized import FoldChemicalTransform


OUT = SOLUTION / "outputs/q3_refined/beer81"
TARGET_BEER = 81
SOURCE_FILES = []


def source(path):
    SOURCE_FILES.append({"path": path.relative_to(ROOT).as_posix(), "sha256": hashlib.sha256(path.read_bytes()).hexdigest()})
    return path


def percentile(values, value):
    finite = np.asarray(values, float)
    finite = finite[np.isfinite(finite)]
    return float((np.sum(finite < value) + .5 * np.sum(finite == value)) / len(finite))


def rounded_id_agrees(reference, printed):
    text = str(printed)
    if "e" not in text.lower():
        return str(reference) == text
    mantissa, exponent = re.split("[eE]", text)
    digits = len(mantissa.split(".")[1]) if "." in mantissa else 0
    resolution = 10.0 ** (int(exponent) - digits)
    return abs(float(reference) - float(text)) <= resolution / 2


def identity_audit(raw, sensory, comments, calibrated):
    frames = {"1.csv": raw, "4.csv": sensory, "7.csv": comments, "8.xlsx/calibrated": calibrated}
    reference = raw.loc[raw.beer.eq(TARGET_BEER)].iloc[0]
    rows = []
    for name, frame in frames.items():
        selected = frame.loc[frame.beer.eq(TARGET_BEER)]
        row = selected.iloc[0]
        rows.append({"source": name, "rows": len(frame), "beer_unique": frame.beer.is_
```

src/21_q3_beer81_audit.py（续）

```
    unique,
57              "beer81_matches": len(selected), "style": row.tasting_category_fine,
58              "style_matches_1csv": row.tasting_category_fine == reference.tasting_categ
    ory_fine,
59              "beer_id_text": str(row.beer_id), "exact_id_matches_1csv": str(row.beer_id
    ) == str(reference.beer_id),
60              "id_matches_printed_precision": rounded_id_agrees(reference.beer_id, row.b
    eer_id),
61              "beer_id_unique_count": frame.beer_id.nunique(),
62              "rows_after_exact_beer_join": len(raw[["beer"]].merge(frame, on="beer", va
    lidate="one_to_one"))})
63      table = pd.DataFrame(rows)
64      if not (table.beer_unique.all() and table.beer81_matches.eq(1).all()
65              and table.style_matches_1csv.all() and table.id_matches_printed_precision.
    all()):
66          raise AssertionError("Identity audit found an unresolved discrepancy")
67      table.to_csv(OUT / "identity_join_audit.csv", index=False, lineterminator="\n")
68      return table
69
70
71  def chemical_audit(meta, x, manifest, raw, calibrated, thresholds, train, target):
72      chemistry = thresholds.drop_duplicates("field_name_R").set_index("field_name_R")
73      rows = []
74      other_pils = meta.tasting_category_fine.eq("Pils/Lager") & meta.beer.ne(TARGET_BEE
    R)
75      raw_row = raw.set_index("beer").loc[TARGET_BEER]
76      cal_row = calibrated.set_index("beer").loc[TARGET_BEER]
77      for entry in manifest.to_dict("records"):
78          field, value = entry["feature"], float(x.iloc[target][entry["feature"]])
79          tr = x.loc[train, field].dropna()
80          peers = x.loc[other_pils, field].dropna()
81          details = chemistry.loc[field] if field in chemistry.index else pd.Series(dtyp
    e=object)
82          threshold = details.get("threshold_median", np.nan)
83          rows.append({"feature": field, "representation": entry["representation"], "uni
    t": entry["unit"],
84              "value": value, "original_1csv_value": raw_row[field],
85              "calibrated_value": cal_row.get(field, np.nan),
86              "nonpils_n_observed": len(tr), "nonpils_min": tr.min(), "nonpils_q01": tr.
    quantile(.01),
87              "nonpils_median": tr.median(), "nonpils_q99": tr.quantile(.99), "nonpils_m
    ax": tr.max(),
88              "nonpils_percentile": percentile(tr, value), "other_pils_percentile": perc
    entile(peers, value),
89              "outside_nonpils_range": bool(value < tr.min() or value > tr.max()),
90              "at_nonpils_minimum": bool(value == tr.min()), "at_nonpils_maximum": bool(
    value == tr.max()),
91              "zero": value == 0, "aging": details.get("aging"), "origin": details.get("
    origin_paper"),
92              "chemical_name": details.get("chemical_name"), "off_flavors": details.get(
    "off_flavors"),
93              "sensory_threshold_median": threshold,
94              "ratio_to_sensory_threshold": value / threshold if entry["representation"]
     == "calibrated"
95              and pd.notna(threshold) and threshold > 0 else np.nan})
96      table = pd.DataFrame(rows)
97      table.to_csv(OUT / "chemical_profile_beer81.csv", index=False, lineterminator="\n"
    )
98      relative = manifest.loc[manifest.representation.eq("relative"), "feature"].tolist(
    )
99      floor_rows = []
```

src/21_q3_beer81_audit.py（续）

```
    for i in range(len(raw)):
        counts = raw.loc[i, relative].value_counts(dropna=True)
        floor_rows.append({"beer": int(meta.beer.iloc[i]), "style": meta.tasting_categ
ory_fine.iloc[i],
            "most_repeated_relative_value": float(counts.index[0]), "most_repeated_rel
ative_count": int(counts.iloc[0])})
    floor_table = pd.DataFrame(floor_rows)
    floor_table.to_csv(OUT / "relative_value_repeat_patterns.csv", index=False, linete
rminator="\n")
    numeric_fields = [c for c in raw if c not in KEYS]
    repeated = raw_row[numeric_fields].value_counts()
    repeated_rows = [{"value": float(v), "count": int(n),
        "features": ";".join(c for c in numeric_fields if raw_row[c] == v),
        "interpretation": "repeated supplied value; attachment does not identify an an
alytical detection limit"}
        for v, n in repeated.items() if n > 1]
    pd.DataFrame(repeated_rows).to_csv(OUT / "beer81_repeated_original_values.csv", in
dex=False, lineterminator="\n")
    calibrated_fields = manifest.loc[manifest.representation.eq("calibrated"), "featur
e"].tolist()
    zero_fields = table.loc[table.zero, "feature"].tolist()
    profile = {"raw_numeric_missing": int(raw_row[numeric_fields].isna().sum()),
        "hybrid_missing": int(x.iloc[target].isna().sum()),
        "calibrated_negative_count": int((pd.to_numeric(cal_row[calibrated_fields]) <
0).sum()),
        "raw_negative_count": int((pd.to_numeric(raw_row[numeric_fields]) < 0).sum()),
        "hybrid_negative_relative_count": int((x.iloc[target][relative] < 0).sum()),
        "hybrid_zero_count": len(zero_fields), "hybrid_zero_features": zero_fields,
        "exact_hybrid_duplicate_other_rows": int(np.sum(np.all(np.isclose(x.to_numpy(f
loat), x.iloc[target].to_numpy(float),
            rtol=0, atol=0, equal_nan=True), axis=1)) - 1),
        "outside_nonpils_feature_ranges": int(table.outside_nonpils_range.sum()),
        "repeated_original_values": repeated_rows,
        "analytical_detection_limits_available": False,
        "detection_limit_note": "Sensory thresholds are perceptual thresholds, not ana
lytical LOD/LOQ"}
    return table, floor_table, profile


def sensory_and_comments(meta, sensory, comments, train, target):
    sensory = sensory.set_index("beer").loc[meta.beer].reset_index()
    comments = comments.set_index("beer").loc[meta.beer].copy().reset_index()
    stale = sensory.stale_hops
    profiles, associations = [], []
    for field in [c for c in sensory if c not in KEYS]:
        value = float(sensory[field].iloc[target])
        correlation = spearmanr(sensory.loc[train, field], stale.loc[train]).statistic
        profiles.append({"sensory": field, "value": value, "all_percentile": percentil
e(sensory[field], value),
            "nonpils_percentile": percentile(sensory.loc[train, field], value),
            "nonpils_spearman_with_stale": float(correlation)})
        if field != "stale_hops":
            associations.append({"sensory": field, "nonpils_spearman_with_stale": floa
t(correlation)})
    pd.DataFrame(profiles).to_csv(OUT / "sensory_profile_beer81.csv", index=False, lin
eterminator="\n")
    pd.DataFrame(associations).to_csv(OUT / "sensory_covariation_nonpils.csv", index=F
alse, lineterminator="\n")
    tail = meta.copy()
    tail["stale_hops"] = stale
    tail["descending_rank"] = stale.rank(method="min", ascending=False).astype(int)
```

src/21_q3_beer81_audit.py（续）

```
    tail["nonpils_training"] = train
    tail.sort_values("stale_hops", ascending=False).to_csv(OUT / "stale_target_distribution.csv", index=False, lineterminator="\n")
    terms = []
    for field in [c for c in comments if c not in KEYS]:
        value = comments[field].iloc[target]
        relevant = any(field.endswith("_" + term) for term in
            ["stale", "cardboard", "papery", "metallic", "barnyard", "cheesy", "cabbage", "acetaldehyde"])
        terms.append({"term": field, "beer81_value": value,
            "all_percentile": percentile(comments[field], value) if pd.notna(value) else np.nan,
            "nonpils_percentile": percentile(comments.loc[train, field], value) if pd.notna(value) else np.nan,
            "predeclared_quality_term": relevant, "source": "7.csv numeric forum descriptor table"})
    terms = pd.DataFrame(terms)
    terms.to_csv(OUT / "forum_all_terms_beer81.csv", index=False, lineterminator="\n")
    terms.loc[terms.predeclared_quality_term].to_csv(OUT / "forum_quality_term_evidence.csv", index=False, lineterminator="\n")
    return pd.DataFrame(profiles), tail, terms


def distance_audit(meta, x, manifest, train, target, y):
    train_indices, pils = np.flatnonzero(train), np.flatnonzero(~train)
    calibrated = manifest.loc[manifest.representation.eq("calibrated"), "feature"].tolist()
    aging_table = pd.read_csv(source(SOLUTION / "outputs/q3_domain/stale/predeclared_aging_features.csv"))
    aging = aging_table.feature.tolist()
    values = x.to_numpy(float).copy()
    for j, field in enumerate(x):
        if field in calibrated:
            positive = values[train_indices, j]
            positive = positive[np.isfinite(positive) & (positive > 0)]
            scale = float(np.median(positive)) if len(positive) else 1.0
            values[:, j] = np.log1p(values[:, j] / scale)
    imp = SimpleImputer(strategy="median", keep_empty_features=True).fit(values[train_indices])
    imputed = imp.transform(values)
    scaler = StandardScaler().fit(imputed[train_indices])
    standard = scaler.transform(imputed)
    aging_indices = [x.columns.get_loc(c) for c in aging]
    spaces = {"unclipped_log_scaled_226": standard, "unclipped_log_scaled_aging24": standard[:, aging_indices]}
    for label, subset in [("clipped_log_scaled_226", None), ("clipped_log_scaled_aging24", aging)]:
        transform = FoldChemicalTransform(x.columns.tolist(), calibrated, subset).fit(x.iloc[train_indices])
        v = transform.transform(x)
        imputer = SimpleImputer(strategy="median", keep_empty_features=True).fit(v[train_indices])
        v = imputer.transform(v)
        fitted_scale = StandardScaler().fit(v[train_indices])
        spaces[label] = fitted_scale.transform(v)
    geometry = joblib.load(source(SOLUTION / "outputs/q3/transfer/transfer_copula.joblib"))
    np.testing.assert_array_equal(geometry["training_rows"], train_indices)
    latent = geometry["copula"].transform(x)
    spaces["old_copula_226"] = latent
    spaces["old_copula_aging24"] = latent[:, aging_indices]
```

src/21_q3_beer81_audit.py（续）

```
    local = joblib.load(source(SOLUTION / "outputs/q3_domain/stale/transfer_stale_model.joblib"))
    np.testing.assert_array_equal(local["train_global_indices"], train_indices)
    spaces["frozen_domain_aging_pca8"] = local["pipeline"][:-1].transform(x)
    ood, neighbor_rows, reference_rows, all81, other_pils_rows = [], [], [], [], []
    for name, space in spaces.items():
        d_train = cdist(space[train_indices], space[train_indices]) / np.sqrt(space.shape[1])
        np.fill_diagonal(d_train, np.inf)
        loo = d_train.min(axis=1)
        reference_rows.extend({"space": name, "beer": int(meta.beer.iloc[i]), "loo_nn_distance": float(d)}
                              for i, d in zip(train_indices, loo))
        d_query = cdist(space[pils], space[train_indices]) / np.sqrt(space.shape[1])
        d_pils = cdist(space[pils], space[pils]) / np.sqrt(space.shape[1])
        np.fill_diagonal(d_pils, np.inf)
        target_position = int(np.flatnonzero(pils == target)[0])
        for rank, position in enumerate(np.argsort(d_pils[target_position]), start=1):
            i = pils[position]
            if i == target:
                continue
            other_pils_rows.append({"space": name, "query_beer": TARGET_BEER, "neighbor_rank": rank,
                "neighbor_beer": int(meta.beer.iloc[i]), "distance": float(d_pils[target_position, position]),
                "neighbor_stale": float(y.stale_hops.iloc[i])})
        for row, query in enumerate(pils):
            order = np.argsort(d_query[row])
            nearest = float(d_query[row, order[0]])
            ood.append({"space": name, "query_beer": int(meta.beer.iloc[query]),
                "dimensions": space.shape[1], "nearest_nonpils_distance": nearest,
                "nonpils_loo_nn_percentile": float(np.mean(loo <= nearest)),
                "nearest_other_pils_distance": float(d_pils[row].min()),
                "training_loo_nn_q95": float(np.quantile(loo, .95))})
            for rank, position in enumerate(order, start=1):
                i = train_indices[position]
                record = {"space": name, "query_beer": int(meta.beer.iloc[query]), "neighbor_rank": rank,
                    "neighbor_beer": int(meta.beer.iloc[i]), "neighbor_style": meta.tasting_category_fine.iloc[i],
                    "distance": float(d_query[row, position]), "neighbor_stale": float(y.stale_hops.iloc[i])}
                if rank <= 25:
                    neighbor_rows.append(record)
                if query == target:
                    all81.append(record)
    ood, neighbors, reference, all81 = map(pd.DataFrame, [ood, neighbor_rows, reference_rows, all81])
    ood["nearest_nonpils_distance_rank_within_pils_desc"] = ood.groupby("space").nearest_nonpils_distance.rank(
        ascending=False, method="min").astype(int)
    ood.to_csv(OUT / "chemical_ood_all_pils.csv", index=False, lineterminator="\n")
    neighbors.to_csv(OUT / "nearest_training_neighbors.csv", index=False, lineterminator="\n")
    all81.to_csv(OUT / "beer81_all_training_neighbor_ranks.csv", index=False, lineterminator="\n")
    reference.to_csv(OUT / "training_loo_distance_reference.csv", index=False, lineterminator="\n")
    pd.DataFrame(other_pils_rows).to_csv(OUT / "beer81_other_pils_neighbors.csv", index=False, lineterminator="\n")
    neighbor_summaries = []
```

src/21_q3_beer81_audit.py（续）

```
    for (name, query), group in neighbors.groupby(["space", "query_beer"]):
        for k in [5, 10, 25]:
            part = group.loc[group.neighbor_rank <= k]
            neighbor_summaries.append({"space": name, "query_beer": query, "k": k,
                "stale_min": part.neighbor_stale.min(), "stale_median": part.neighbor_stale.median(),
                "stale_mean": part.neighbor_stale.mean(), "stale_max": part.neighbor_stale.max(),
                "stale_at_least_beer81": int(part.neighbor_stale.ge(y.stale_hops.iloc[target]).sum())})
    pd.DataFrame(neighbor_summaries).to_csv(OUT / "neighbor_target_summaries.csv", index=False, lineterminator="\n")
    tail_ranks = all81.loc[all81.neighbor_stale.ge(y.stale_hops.iloc[target])]
    tail_ranks.to_csv(OUT / "high_stale_training_neighbor_ranks.csv", index=False, lineterminator="\n")
    return ood, all81, tail_ranks


def influence_audit(sensory):
    saved = pd.read_csv(source(SOLUTION / "outputs/q3_domain/stale/pils_diagnostic_predictions.csv"))
    saved["stage"] = "domain_same_seed_comparison"
    parts = [saved[["stage", "method", "beer", "observed", "prediction"]]]
    prior = pd.read_csv(source(SOLUTION / "outputs/q3_optimized/stale_fixed/pils_style_holdout_predictions.csv"))
    for method, field in [("fixed_single_tree", "prediction"), ("old_multioutput", "old_multioutput_prediction"),
                          ("mean", "mean_baseline_prediction")]:
        part = prior[["beer", "observed", field]].rename(columns={field: "prediction"})
        part["stage"], part["method"] = "previous_fixed_seed_comparison", method
        parts.append(part)
    frozen = pd.read_csv(source(SOLUTION / "outputs/q3/sensory/transfer_pils_heldout_predictions.csv"))
    frozen = frozen[["beer", "stale_hops"]].rename(columns={"stale_hops": "prediction"})
    frozen["observed"] = frozen.beer.map(sensory.set_index("beer").stale_hops)
    frozen["stage"], frozen["method"] = "original_frozen_proxy", "multioutput_center"
    parts.append(frozen)
    predictions = pd.concat(parts, ignore_index=True)
    predictions["residual"] = predictions.prediction - predictions.observed
    predictions["squared_error"] = predictions.residual ** 2
    predictions["sse_share"] = predictions.squared_error / predictions.groupby(["stage", "method"]).squared_error.transform("sum")
    predictions.to_csv(OUT / "all_saved_pils_prediction_errors.csv", index=False, lineterminator="\n")
    rows, deletion = [], []
    for (stage, method), group in predictions.groupby(["stage", "method"]):
        source81 = group.loc[group.beer.eq(TARGET_BEER)].iloc[0]
        full_rmse = float(np.sqrt(group.squared_error.mean()))
        rows.append({"stage": stage, "method": method, "n_sources": len(group), "full_rmse": full_rmse,
            "beer81_prediction": source81.prediction, "beer81_residual": source81.residual,
            "beer81_sse_share": source81.sse_share})
        for beer in group.beer:
            reduced = group.loc[group.beer.ne(beer)]
            deletion.append({"stage": stage, "method": method, "omitted_beer": beer,
                "full_n": len(group), "remaining_n": len(reduced), "full_rmse": full_rmse,
                "fixed_prediction_rmse_without_one_source": float(np.sqrt(reduced.squa
```

src/21_q3_beer81_audit.py（续）

```
    red_error.mean())),
287                 "interpretation": "influence only; same fixed predictions, no retraini
    ng, never substitutes the all-source metric"})
288     influence = pd.DataFrame(rows)
289     influence.to_csv(OUT / "beer81_sse_contribution.csv", index=False, lineterminator=
    "\n")
290     pd.DataFrame(deletion).to_csv(OUT / "leave_one_source_out_influence.csv", index=Fa
    lse, lineterminator="\n")
291     return influence
292
293
294 def main():
295     OUT.mkdir(parents=True, exist_ok=True)
296     root = data_dir()
297     raw, sensory, comments = [pd.read_csv(source(root / filename), dtype={"beer_id": s
    tr})
298                               for filename in ["1.csv", "4.csv", "7.csv"]]
299     source(root / "8.xlsx")
300     calibrated = pd.read_excel(root / "8.xlsx", sheet_name="Calibrated chemical datase
    t", dtype={"beer_id": str})
301     thresholds = pd.read_excel(root / "8.xlsx", sheet_name="Sensory thresholds")
302     sensory_names = pd.read_excel(root / "8.xlsx", sheet_name="Sensory names")
303     target_metadata = sensory_names.loc[sensory_names.R_name.isin(["stale_hops", "barn
    yard", "t2n", "dms"])]
304     target_metadata.to_csv(OUT / "sensory_target_metadata.csv", index=False, linetermi
    nator="\n")
305     meta, x, y, manifest = load_q3_data()
306     target = int(np.flatnonzero(meta.beer.eq(TARGET_BEER))[0])
307     train = meta.tasting_category_fine.ne("Pils/Lager")
308     identity = identity_audit(raw, sensory, comments, calibrated)
309     chemicals, floors, profile = chemical_audit(meta, x, manifest, raw, calibrated, th
    resholds, train, target)
310     sensation, tail, terms = sensory_and_comments(meta, sensory, comments, train, targ
    et)
311     ood, neighbors, tail_ranks = distance_audit(meta, x, manifest, train, target, y)
312     influence = influence_audit(sensory)
313     local_neighbors = neighbors.loc[neighbors.space.eq("frozen_domain_aging_pca8") & n
    eighbors.neighbor_rank.le(25)]
314     key_terms = terms.loc[terms.term.isin(["aroma_stale", "taste_stale", "taste_cardbo
    ard", "taste_papery"])]
315     high_tail = tail.loc[tail.nonpils_training & tail.stale_hops.ge(y.stale_hops.iloc[
    target])]
316     selected = json.loads((SOLUTION / "outputs/q3_domain/stale/summary.json").read_tex
    t(encoding="utf-8"))["selected_method"]
317     source(SOLUTION / "outputs/q3_domain/stale/summary.json")
318     selected_influence = influence.loc[influence.stage.eq("domain_same_seed_comparison
    ") & influence.method.eq(selected)].iloc[0]
319     own_floor = floors.loc[floors.beer.eq(TARGET_BEER)].iloc[0]
320     barnyard = sensation.loc[sensation.sensory.eq("barnyard")].iloc[0]
321     summary = {"source_beer": TARGET_BEER, "exact_beer_join_passed": True,
322         "barcode_consistent_at_supplied_precision": True, "barcode_exact_1csv_8xlsx":
    True,
323         "sample_profile": profile, "stale_observed": float(y.stale_hops.iloc[target]),
324         "stale_descending_rank_all250": int(tail.loc[tail.beer.eq(TARGET_BEER), "desce
    nding_rank"].iloc[0]),
325         "nonpils_training_rows": int(train.sum()), "nonpils_stale_at_least_beer81": le
    n(high_tail),
326         "nonpils_high_tail_beers": high_tail.beer.tolist(),
327         "frozen_local_top25_stale_min": float(local_neighbors.neighbor_stale.min()),
328         "frozen_local_top25_stale_max": float(local_neighbors.neighbor_stale.max()),
329         "beer81_outside_local_top25_target_convex_hull": bool(y.stale_hops.iloc[target
```

src/21_q3_beer81_audit.py（续）

```
    ] > local_neighbors.neighbor_stale.max()),
330         "selected_domain_model": selected, "selected_domain_beer81_prediction": float(
    selected_influence.beer81_prediction),
331         "selected_domain_beer81_sse_share": float(selected_influence.beer81_sse_share)
    ,
332         "relative_repeated_floor_count": int(own_floor.most_repeated_relative_count),
333         "relative_floor_count_all250_percentile": float(np.mean(floors.most_repeated_r
    elative_count <= own_floor.most_repeated_relative_count)),
334         "stale_target_datatype": target_metadata.loc[target_metadata.R_name.eq("stale_
    hops"), "DataType"].iloc[0],
335         "barnyard_target_datatype": target_metadata.loc[target_metadata.R_name.eq("bar
    nyard"), "DataType"].iloc[0],
336         "nonpils_stale_barnyard_spearman": float(barnyard.nonpils_spearman_with_stale)
    ,
337         "ood_beer81": ood.loc[ood.query_beer.eq(TARGET_BEER)].to_dict("records"),
338         "forum_quality_evidence": key_terms.to_dict("records"),
339         "raw_forum_text_available": False, "analytical_lod_loq_available": False,
340         "label_error_established": False,
341         "interpretation": "Rare high sensory target lacks matching high-target chemica
    l neighbours; identity and descriptor evidence do not establish a wrong label. Chemica
    l predictability remains unresolved."}
342     (OUT / "summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2
    ), encoding="utf-8", newline="\n")
343     verification = {"identity_join_four_sources_passed": bool(identity.beer_unique.all
    ()),
344         "features": len(chemicals), "forum_fields": len(terms), "ood_query_space_rows"
    : len(ood),
345         "training_neighbor_rank_rows": len(neighbors),
346         "other_pils_neighbor_rows": len(pd.read_csv(OUT / "beer81_other_pils_neighbors
    .csv")),
347         "local_target_convex_hull_gap_verified": summary["beer81_outside_local_top25_t
    arget_convex_hull"],
348         "leave_one_source_out_only_fixed_prediction_influence": True,
349         "source_hashes_unchanged_after_analysis": all(hashlib.sha256(resolve_recorded_
    path(row["path"]).read_bytes()).hexdigest()
350                                                       == row["sha256"] for row in SOU
    RCE_FILES)}
351     assert verification["source_hashes_unchanged_after_analysis"]
352     (OUT / "verification.json").write_text(json.dumps(verification, ensure_ascii=False
    , indent=2), encoding="utf-8", newline="\n")
353     (OUT / "source_manifest.json").write_text(json.dumps(SOURCE_FILES, ensure_ascii=Fa
    lse, indent=2), encoding="utf-8", newline="\n")
354     write_note(summary, chemicals, sensation, terms, tail_ranks, influence)
355     print(json.dumps(summary, ensure_ascii=False, indent=2), flush=True)
356
357
358 def write_note(summary, chemicals, sensation, terms, tail_ranks, influence):
359     lines = ["# Beer 81误差来源诊断", "",
360         "beer81的高残差目前不能归因于标签错误。附件键连接、风格和可核验条码一致；"
361         "无缺失、负校准值或重复化学行解释该误差。陈旧描述在论坛术语表中也处高位。"
362         "直接观察到的困难是高stale尾部样本很少，且未落入现有化学近邻空间。", "",
363         "## 连接与测量检查", "",
364         "1.csv与8.xlsx校准表的完整条码为8715608222557，风格均Pils/Lager。4.csv与7.csv提供
    "
365         "的8.71561E+12与其打印精度一致；它们是舍入后的身份信息，实际按唯一beer键连接，"
366         "未采用舍入条码连接。", "",
367         f"原231化学列缺失数为{summary['sample_profile']['raw_numeric_missing']}，226混合
    输入缺失数为"
368         f"{summary['sample_profile']['hybrid_missing']}，负校准值为{summary['sample_pro
    file']['calibrated_negative_count']}，"
369         f"其他样本中的整行化学重复为{summary['sample_profile']['exact_hybrid_duplicate_o
```

```
    ther_rows']}。"
370         "原始表示中的负值不能直接视为负浓度；混合输入的未校准相对字段仍保留其供应表示。", "",
371         "原表有多个字段重复同一低值，完整清单见beer81_repeated_original_values.csv。"
372         "附件未提供分析LOD/LOQ，不能把共同低值直接命名为检出限；8.xlsx给的是感官阈值，"
373         "也不等于仪器检测上下限。经验最小值、最大值与0值仅作为观测支持诊断保存。", "",
374         f"81有{summary['relative_repeated_floor_count']}个相对字段重复同一低值，重复数量仅处250款的"
375         f"{summary['relative_floor_count_all250_percentile']:.1%}位置，并非该现象最严重的样本。", "",
376         "## 化学异常与支持域", "",
377         "所有尺度仅拟合239款非Pils：校准字段先按训练正数中位数做log1p尺度化，训练中位数补缺，"
378         "再训练标准化；另比较训练1%/99%截断、冻结旧copula秩空间以及16实际冻结aging/PCA8空间。"
379         "没有用11款源学习尺度或边界。", "",
380         "|空间|81最近非Pils距离|相对训练留一NN距离百分位|81最近其他Pils距离|", "|---|---:|---:|---:|"]
381     for row in summary["ood_beer81"]:
382         lines.append(f"|{row['space']}|{row['nearest_nonpils_distance']:.4f}|{row['nonpils_loo_nn_percentile']:.1%}|"
383                      f"{row['nearest_other_pils_distance']:.4f}|")
384     lines.extend(["", "不同空间有不同含义：全化学谱显示一定风格偏移，但aging及其PCA空间接近训练邻域。"
385         "参考距离只排除训练行自身，未逐行重拟合预处理；这些百分位只是相同训练表示下的描述性比较，"
386         "不是交叉拟合或校准OOD概率，也不证明所有有效化学信息已被表示。", "",
387         "校准字段中，训练百分位低于2.5%或高于97.5%的项目完整列在下表；这是多变量描述性筛查，非显著性检验。", "",
388         "|字段|81校准值|单位|非Pils百分位|附件aging标记|", "|---|---:|---|---:|---|"])
389     extreme = chemicals.loc[chemicals.representation.eq("calibrated") &
390                             ((chemicals.nonpils_percentile < .025) | (chemicals.nonpils_percentile > .975))]
391     for row in extreme.itertuples():
392         lines.append(f"|{row.feature}|{row.value:.6g}|{row.unit}|{row.nonpils_percentile:.1%}|{row.aging if pd.notna(row.aging) else '无'}|")
393     lines.extend(["", "SO2高百分位、部分酒花来源指标低百分位均未形成附件内已验证的stale机制。"
394         "不能从这些异常单独推导储存条件、氧化时间或标签错误。所有226字段的值、训练边界、百分位、"
395         "来源、aging与阈值信息保存在chemical_profile_beer81.csv。", "",
396         "题目四种化学缺陷的核对如下。感官阈值与stale_hops不是同一目标，不可把某一化学项超阈直接当作其误差原因。", "",
397         "|字段|81校准值|单位|非Pils百分位|附件感官阈值中位数|", "|---|---:|---|---:|---:|"])
398     for row in chemicals.loc[chemicals.feature.isin(["DMS", "X2.3.butanedione", "X4VG", "X.E..2.nonenal"])].itertuples():
399         threshold = f"{row.sensory_threshold_median:.6g}" if pd.notna(row.sensory_threshold_median) else "未提供"
400         lines.append(f"|{row.feature}|{row.value:.6g}|{row.unit}|{row.nonpils_percentile:.1%}|{threshold}|")
401     outside = chemicals.loc[chemicals.outside_nonpils_range]
402     if len(outside):
403         labels = "; ".join(f"{r.feature}={r.value:.6g}（训练范围{r.nonpils_min:.6g}至{r.nonpils_max:.6g}）"
404                            for r in outside.itertuples())
405         lines.extend(["", f"全226字段超出训练观测范围的完整清单为：{labels}。"
406             "该字段仍是未校准相对表示，不能擅自换算成浓度或视为仪器越限。"])
407     lines.extend(["",
408         "## 尾部稀缺与邻域错位", "",
409         f"81的stale={summary['stale_observed']:.6f}，在250款中第{summary['stale_descending_rank_all250']}高；"
```

src/21_q3_beer81_audit.py（续）

```
        f"239款训练中仅{summary['nonpils_stale_at_least_beer81']}款不低于它。"
        f"16实际局部模型的最近25训练邻居stale最高只有{summary['frozen_local_top25_stale_max']:.6f}，"
        f"低于81实测值；其预测为{summary['selected_domain_beer81_prediction']:.6f}。"
        "局部加权均值受邻居目标凸包限制，因此现有邻域结构无法给出81的高标签。", "",
        "|高stale训练beer|风格|实测stale|aging/PCA8近邻排名|", "|---|---|---:|---:|"])
    for row in tail_ranks.loc[tail_ranks.space.eq("frozen_domain_aging_pca8")].sort_values("neighbor_rank").itertuples():
        lines.append(f"|{row.neighbor_beer}|{row.neighbor_style}|{row.neighbor_stale:.6f}|{row.neighbor_rank}|")
    lines.extend(["", "全226旧copula的最近邻同样多为低stale样本。完整近邻及其真实目标保存，而非只展示支持结论的邻居。"
        "这说明现有化学距离不足以组织该尾部目标，不能证明不存在其他可预测化学组合；"
        "当前附件也无法核验储存、包装、批次或品评个体差异。", "",
        "## 其他感官与论坛描述", "",
        "其他实测感官仅用于解释。所有50项值与239款非Pils中的stale相关均保存，"
        "它们不会作为源样本的chemical-only预测输入。论坛7.csv是数值描述符表，未提供原文、"
        "评论人数、时间或逐批次对应信息；字段数值不能解释为词出现次数。", "",
        f"8.xlsx明确stale_hops与barnyard均为numeric，不能把它们当作categorical父类继承值。"
        f"两者在239款非Pils中的Spearman相关为{summary['nonpils_stale_barnyard_spearman']:.4f}；"
        "这是很强的共同变化，但没有原始品评记录来区分共享感官因素或数据加工来源，不能据此判错。", "",
        "numeric说明数据类型，不代表恢复了0–6原始评分；本诊断始终保留4.csv供应的标准化表示。", "",
        "|论坛字段|81供应数值|250款百分位|", "|---|---:|---:|"])
    for row in terms.loc[terms.term.isin(["aroma_stale", "taste_stale", "taste_cardboard", "taste_papery", "aroma_papery"])].itertuples():
        lines.append(f"|{row.term}|{row.beer81_value:.4f}|{row.all_percentile:.1%}|")
    lines.extend(["", "这些陈旧、纸板、纸张相关描述构成另一供应数据通道的相符证据，支持谨慎保留高stale标签；"
        "但其采样是否与品评完全独立、是否对应同一批次均不明，不能称为独立真值认证。", "",
        "论坛barnyard描述并不高，两个通道并非所有感官属性逐项相符；完整186项记录均已保存，"
        "没有仅保留支持高stale解释的字段。", "",
        "## 误差影响与可采取的解释", "",
        "|保存阶段|模型|全11源RMSE|81预测|81平方误差占比|", "|---|---|---:|---:|---:|"])
    preferred = influence.loc[(influence.stage.eq("original_frozen_proxy")) |
        (influence.stage.eq("domain_same_seed_comparison") & influence.method.isin(
            [summary["selected_domain_model"], "old_multioutput_tree", "fixed_single_tree", "mean_global"]))]
    for row in preferred.itertuples():
        lines.append(f"|{row.stage}|{row.method}|{row.full_rmse:.4f}|{row.beer81_prediction:.4f}|{row.beer81_sse_share:.1%}|")
    lines.extend(["", "leave_one_source_out_influence.csv只从已保存预测中临时去掉一个源计算影响，"
        "没有重训，也不构成新的交叉验证；任何去掉81后的误差都不能替代全11源主指标。", "",
        "目前可确认的是尾部样本稀缺、化学近邻与高stale错位，以及已有模型的强回归均值倾向。"
        "不能确认是测量、储存、批次、未观测成分、感官个体差异或标签问题所致。"
        "后续仅可在非Pils训练内验证尾部权重、共享感官表示及其他化学模型；论坛描述和81标签不得进入选型。", "",
        "```bash", ".venv/bin/python solution_a/src/21_q3_beer81_audit.py", "```", ""])
    (SOLUTION / "docs/q3_beer81_audit.md").write_text("\n".join(lines), encoding="utf-8", newline="\n")


if __name__ == "__main__":
    with threadpool_limits(limits=1):
        main()
```

src/22_q3_stale_refined.py

SHA-256: d9a9f67965d6270217dfb145ebbfd429c4928f349085cd266bb7b47aa19236ef

```
"""Grouped selection, auxiliary supervision and stability-first stale proxy."""
from __future__ import annotations

import json
import platform
import warnings
from time import perf_counter

import joblib
import numpy as np
import pandas as pd
import sklearn
from scipy.spatial.distance import cdist
from scipy.stats import ConstantInputWarning
from sklearn.model_selection import GroupKFold, LeaveOneGroupOut
from threadpoolctl import threadpool_limits

from common import SOLUTION, data_dir
from q3_data import load_q3_data
from q3_stale_optimized import scores as previous_scores
from q3_stale_refined import (
    AUDIT_BOOTSTRAP_SEED, BIAS_PENALTY, FINAL_SEED, FixedChemicalTransform,
    NESTED_SEED, RefinedStaleModel, SELECTION_BOOTSTRAP_SEED,
    STABILITY_DRAWS, STABILITY_SEED, TARGET, VARIANCE_RATIO_MIN, refined_specs,
)

OUT = SOLUTION / "outputs" / "q3_refined" / "stale"


def write_json(path, value):
    path.write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False), encoding="utf-8", newline="\n")


def scores(y, pred):
    with warnings.catch_warnings():
        warnings.simplefilter("ignore", ConstantInputWarning)
        result = previous_scores(y, pred)
    if result["spearman"] is not None and not np.isfinite(result["spearman"]):
        result["spearman"] = None
    return result


def model(spec, context, seed):
    return RefinedStaleModel(context["features"], context["calibrated"],
                             context["aging"], context["target_names"], spec, seed)


def uses_auxiliary(spec):
    return spec["family"] in ("aux", "aux_kernel") or (
        spec["family"] == "ensemble" and any(uses_auxiliary(member) for member in spec["members"]))


def save_candidate_models(fitted_candidates, specs, context, final_table,
                          decision, stability, train, source):
    """Keep every selectable candidate, including failed development paths."""
    destination = OUT/"candidate_models"
    destination.mkdir(parents=True, exist_ok=True)
```

src/22_q3_stale_refined.py（续）

```
    table = final_table.set_index("method")
    stability_table = stability.set_index("method")
    manifest_rows = []
    for spec in specs:
        if not spec["selectable"]:
            continue
        name = spec["name"]
        row = table.loc[name]
        candidate_stability = stability_table.loc[name].to_dict() if name in stability_table.index else None
        internal_qualification = None if candidate_stability is None else bool(
            row.eligible_for_one_se and candidate_stability["pair_delta_signal_rms"] > 1e-4
            and candidate_stability["response_noise_ratio"] <= 1)
        selected = name == decision["selected_method"]
        auxiliary = uses_auxiliary(spec)
        artifact = {"pipeline": fitted_candidates[name], "features": context["features"],
            "target": TARGET, "target_scale": "attachment_4_standardized", "config": spec,
            "selected_config": spec, "train_global_indices": train, "excluded_global_indices": source,
            "excluded_style": "Pils/Lager", "not_selected": not selected,
            "selection_status": "selected_reference_copy" if selected else "not_selected",
            "development_diagnostic_only": True, "selected_in_frozen_protocol": selected,
            "eligible_for_one_se": bool(row.eligible_for_one_se), "within_one_se": bool(row.within_one_se),
            "loso_metrics": row.to_dict(), "candidate_stability": candidate_stability,
            "candidate_internal_qualification": internal_qualification,
            "qualified_for_proxy": bool(selected and decision["qualified_for_proxy"]),
            "qualification_scope": "candidate archive; unselected artifacts are development diagnostics, not deployment models",
            "uses_auxiliary_supervision": auxiliary,
            "training_auxiliary_targets": [name for name in context["target_names"] if name != TARGET] if auxiliary else [],
            "pils_status": "repeatedly viewed development diagnostic", "pils_used_for_selection": False,
            "final_seed": FINAL_SEED}
        joblib.dump(artifact, destination/f"{name}.joblib", compress=3)
        manifest_rows.append({"method": name, "file": f"candidate_models/{name}.joblib",
            "selected_in_frozen_protocol": selected, "eligible_for_one_se": bool(row.eligible_for_one_se),
            "within_one_se": bool(row.within_one_se), "development_diagnostic_only": True,
            "macro_style_rmse": float(row.macro_style_rmse), "pooled_rmse": float(row.rmse)})
    pd.DataFrame(manifest_rows).to_csv(OUT/"candidate_model_manifest.csv", index=False, lineterminator="\n")


def evaluate_loso(indices, specs, context, seed, scope):
    meta, x, y45 = context["meta"], context["x"], context["y45"]
    groups = meta.iloc[indices].tasting_category_fine.to_numpy()
    truth = y45.iloc[indices][TARGET].to_numpy()
    predictions = {spec["name"]: np.full(len(indices), np.nan) for spec in specs}
    folds, thresholds = np.zeros(len(indices), dtype=int), np.full(len(indices), np.nan)
    style_rows, audit_rows = [], []
```

src/22_q3_stale_refined.py（续）

```
    for fold, (tr, te) in enumerate(LeaveOneGroupOut().split(indices, truth, groups), 1):
        train_global, test_global = indices[tr], indices[te]
        style = str(groups[te[0]])
        assert not meta.iloc[train_global].tasting_category_fine.eq(style).any()
        assert not meta.iloc[train_global].tasting_category_fine.eq("Pils/Lager").any()
        for spec in specs:
            if spec["family"] == "ensemble":
                values = np.mean([predictions[member["name"]][te] for member in spec["members"]], axis=0)
            else:
                fitted = model(spec, context, seed+fold).fit(x.iloc[train_global], y45.iloc[train_global],
                    meta.iloc[train_global].tasting_category_fine)
                values = fitted.predict(x.iloc[test_global])
            predictions[spec["name"]][te] = values
            style_rows.append({"scope": scope, "method": spec["name"], "heldout_style": style,
                               "fold": fold, "n": len(te), **scores(truth[te], values),
                               "bias": float(np.mean(values-truth[te]))})
        folds[te] = fold
        thresholds[te] = np.quantile(y45.iloc[train_global][TARGET], .9)
        audit_rows.extend({"scope": scope, "fold": fold, "heldout_style": style,
                           "role": role, "global_index": int(index)}
                          for role, sample in [("train", train_global), ("test", test_global)] for index in sample)
    per_style = pd.DataFrame(style_rows)
    summaries, long_rows = [], []
    tail = truth >= thresholds
    for spec in specs:
        pred = predictions[spec["name"]]
        assert np.isfinite(pred).all()
        subset = per_style[per_style.method.eq(spec["name"])]
        losses = subset.rmse.to_numpy()+BIAS_PENALTY*subset.bias.abs().to_numpy()
        metric = scores(truth, pred)
        summaries.append({"scope": scope, "method": spec["name"], "selectable": spec["selectable"],
            "priority": spec["priority"], "styles": len(subset),
            "macro_style_rmse": float(subset.rmse.mean()),
            "macro_style_abs_bias": float(subset.bias.abs().mean()),
            "selection_score": float(losses.mean()),
            "style_score_se": float(losses.std(ddof=1)/np.sqrt(len(losses))),
            "tail_n": int(tail.sum()), "tail_rmse": float(np.sqrt(np.mean((pred[tail]-truth[tail])**2))),
            **metric})
        long_rows.append(pd.concat([meta.iloc[indices].reset_index(drop=True), pd.DataFrame({
            "global_index": indices, "scope": scope, "method": spec["name"], "observed": truth,
            "prediction": pred, "heldout_style": groups, "fold": folds,
            "tail_threshold_training_q90": thresholds, "is_tail_eval": tail})], axis=1))
    return pd.DataFrame(summaries), per_style, pd.concat(long_rows, ignore_index=True), pd.DataFrame(audit_rows)


def feature_pairs(indices, context):
    transform = FixedChemicalTransform(context["features"], context["calibrated"], context["aging"]).fit(context["x"].iloc[indices])
```

src/22_q3_stale_refined.py（续）

```
    z = transform.transform(context["x"].iloc[indices])
    distances = cdist(z, z)
    np.fill_diagonal(distances, np.inf)
    # Third-nearest distinct training row; the fixed rule never sees target y.
    nearest_local = np.argsort(distances, axis=1)[:, 2]
    return indices[nearest_local]


def stability_compare(indices, candidates, context, seed, scope):
    pairs = feature_pairs(indices, context)
    records, pair_rows = [], []
    for spec in candidates:
        delta = []
        for b in range(STABILITY_DRAWS):
            index = np.random.default_rng(seed+b).choice(indices, size=len(indices), r
eplace=True)
            fitted = model(spec, context, seed+10000+b).fit(context["x"].iloc[index],
context["y45"].iloc[index],
                context["meta"].iloc[index].tasting_category_fine)
            delta.append(fitted.predict(context["x"].iloc[indices])-fitted.predict(con
text["x"].iloc[pairs]))
        delta = np.asarray(delta)
        noise = float(np.sqrt(np.mean(np.var(delta, axis=0, ddof=1))))
        signal = float(np.sqrt(np.mean(delta.mean(axis=0)**2)))
        ratio = noise/max(signal, 1e-8)
        records.append({"scope": scope, "method": spec["name"], "stability_draws": STA
BILITY_DRAWS,
                        "pair_delta_noise_rms": noise, "pair_delta_signal_rms": signal
,
                        "response_noise_ratio": ratio, "priority": spec["priority"]})
    pair_rows = [{"scope": scope, "global_index": int(index), "paired_global_index": i
nt(pair)}
                 for index, pair in zip(indices, pairs)]
    return pd.DataFrame(records), pd.DataFrame(pair_rows)


def select_stable(indices, summary, specs, context, seed, scope):
    table = summary.copy()
    mean = table[table.method.eq("mean_global")].iloc[0]
    eligible = table.selectable & (table.prediction_variance_ratio >= VARIANCE_RATIO_M
IN)
    eligible &= (table.macro_style_rmse < mean.macro_style_rmse) & (table.rmse < mean.
rmse)
    table["eligible_for_one_se"] = eligible
    candidates = table[eligible]
    fallback = candidates.empty
    if fallback:
        best = table[table.selectable].sort_values(["selection_score", "method"]).iloc
[0]
        one_se = table[table.method.eq(best.method)]
    else:
        best = candidates.sort_values(["selection_score", "method"]).iloc[0]
        one_se = candidates[candidates.selection_score <= best.selection_score+best.st
yle_score_se]
    table["within_one_se"] = table.method.isin(one_se.method)
    spec_map = {spec["name"]: spec for spec in specs}
    stability, pairs = stability_compare(indices, [spec_map[name] for name in one_se.m
ethod], context, seed, scope)
    chosen = stability.sort_values(["response_noise_ratio", "priority", "method"]).ilo
c[0]
    selected_method = str(chosen.method)
```

src/22_q3_stale_refined.py（续）

```
    chosen_metrics = table[table.method.eq(selected_method)].iloc[0]
    qualified = bool(not fallback and chosen.pair_delta_signal_rms > 1e-4 and chosen.response_noise_ratio <= 1.0)
    decision = {"selected_method": selected_method, "config": spec_map[selected_method],
        "qualified_for_proxy": qualified, "eligibility_fallback": bool(fallback),
        "lowest_score_method": str(best.method), "lowest_selection_score": float(best.selection_score),
        "one_se_width": float(best.style_score_se),
        "one_se_threshold": float(best.selection_score+best.style_score_se),
        "selected_loso_metrics": chosen_metrics.to_dict(), "selected_stability": chosen.to_dict(),
        "qualification_scope": "training-only grouped selection and response stability; not Pils or causal validation"}
    return decision, table, stability, pairs


def main():
    started = perf_counter()
    OUT.mkdir(parents=True, exist_ok=True)
    meta, x, y45, manifest = load_q3_data()
    features = x.columns.tolist()
    calibrated = manifest.loc[manifest.representation.eq("calibrated"), "feature"].tolist()
    source = np.flatnonzero(meta.tasting_category_fine.eq("Pils/Lager"))
    train = np.flatnonzero(~meta.tasting_category_fine.eq("Pils/Lager"))
    thresholds = pd.read_excel(data_dir()/"8.xlsx", sheet_name="Sensory thresholds")
    aging_set = set(thresholds.loc[thresholds.aging.notna(), "field_name_R"])
    aging = [feature for feature in features if feature in aging_set]
    context = {"meta": meta, "x": x, "y45": y45, "features": features,
               "calibrated": calibrated, "aging": aging, "target_names": y45.columns.tolist()}
    specs = refined_specs()
    write_json(OUT/"run_manifest.json", {
        "candidate_specs": specs, "features": features, "aging_features": aging,
        "training_indices": train.tolist(), "excluded_indices": source.tolist(),
        "auxiliary_target_names": [name for name in y45 if name != TARGET],
        "auxiliary_representation": "weighted Ridge alpha100 to 44 standardized auxiliary outcomes; right singular directions rank4/8; stale excluded",
        "fixed_missing_representation": "always append all selected missing flags at weight0.2; never variance-scale flags",
        "tail_training_weight": "multiply top20% training-only stale outcomes by2, then normalize mean weight1",
        "score": "macro(style RMSE)+0.25*macro(abs(style mean error))",
        "one_se_eligibility": "selectable and OOF variance ratio>=0.05 and macro/pooled RMSE<global mean baseline",
        "one_se_rule": "score<=best score+SE across held-out styles; choose lowest response-noise/signal ratio from8 common row bootstraps; complexity priority breaks exact ties",
        "qualification": "eligibility and response signal RMS>1e-4 and response noise/signal<=1",
        "chemical_pair_rule": "third-nearest distinct non-Pils training row in fixed aging transform",
        "final_validation": "complete non-Pils21 LOSO", "nested_validation": "4 outer shuffled GroupKFold; complete inner LOSO; selector stability computed on outer training only",
        "stability_seed": STABILITY_SEED, "stability_draws": STABILITY_DRAWS,
        "nested_seed": NESTED_SEED, "final_seed": FINAL_SEED,
        "selection_bootstrap_seed": SELECTION_BOOTSTRAP_SEED, "audit_bootstrap_seed": AUDIT_BOOTSTRAP_SEED,
        "pils_labels_used_for_selection_or_intercept": False,
```

src/22_q3_stale_refined.py（续）

```
        "pils_status": "repeatedly viewed development diagnostic, not a new test",
        "all_candidates_retained": True, "no_beer81_exclusion": True,
    })
    # The full grouped candidate comparison is fixed before reading Pils errors.
    full, by_style, pred, audit = evaluate_loso(train, specs, context, FINAL_SEED, "fu
ll_loso")
    full.to_csv(OUT/"loso_metrics_overall_raw.csv", index=False, lineterminator="\n")
    by_style.to_csv(OUT/"loso_metrics_by_style.csv", index=False, lineterminator="\n")
    pred.to_csv(OUT/"loso_predictions.csv", index=False, lineterminator="\n")
    audit.to_csv(OUT/"loso_split_audit.csv", index=False, lineterminator="\n")
    decision, final_table, stability, pairs = select_stable(train, full, specs, contex
t, STABILITY_SEED, "full_loso")
    final_table.to_csv(OUT/"loso_metrics_overall.csv", index=False, lineterminator="\n
")
    stability.to_csv(OUT/"selection_stability.csv", index=False, lineterminator="\n")
    pairs.to_csv(OUT/"selection_chemical_pairs.csv", index=False, lineterminator="\n")
    write_json(OUT/"selection_result.json", decision)
    print("Refined full LOSO selected:", decision["selected_method"], "qualified", dec
ision["qualified_for_proxy"], flush=True)

    # Evaluate the entire predeclared selector in an additional grouped layer.
    outer_rows, nested_choices, nested_inner_rows, nested_stability_rows = [], [], [],
 []
    nested_inner_audits, nested_outer_audits = [], []
    selector_specs = [spec for spec in specs if spec["family"] not in ("old_tree", "si
ngle_tree")]
    groups = meta.iloc[train].tasting_category_fine.to_numpy()
    for fold, (tr, te) in enumerate(GroupKFold(4, shuffle=True, random_state=NESTED_SE
ED).split(train, groups=groups), 1):
        tr_global, te_global = train[tr], train[te]
        summary, _, _, inner_audit = evaluate_loso(tr_global, selector_specs, context,
 NESTED_SEED+fold*1000, f"nested_{fold}")
        inner_audit["outer_fold"] = fold
        nested_inner_audits.append(inner_audit)
        nested_outer_audits.extend({"outer_fold":fold,"role":role,"global_index":int(i
ndex),
            "style":str(meta.iloc[index].tasting_category_fine)}
            for role,sample in [("train",tr_global),("test",te_global)] for index in s
ample)
        selected, inner_table, stab, _ = select_stable(tr_global, summary, selector_sp
ecs, context,
                                                       STABILITY_SEED+fold*1000, f"neste
d_{fold}")
        inner_table["outer_fold"] = fold
        stab["outer_fold"] = fold
        nested_inner_rows.append(inner_table)
        nested_stability_rows.append(stab)
        nested_choices.append({"outer_fold": fold, "selected_method": selected["select
ed_method"],
                               "qualified_for_proxy": selected["qualified_for_proxy"],
                               "heldout_styles": ";".join(sorted(meta.iloc[te_global].
tasting_category_fine.unique())),
                               "train_n": len(tr_global), "test_n": len(te_global)})
        for method_name in ("nested_stability_rule", "mean_global", "old_multioutput_t
ree", "fixed_single_tree"):
            spec = selected["config"] if method_name == "nested_stability_rule" else n
ext(spec for spec in specs if spec["name"] == method_name)
            fitted = model(spec, context, NESTED_SEED+fold).fit(x.iloc[tr_global], y45
.iloc[tr_global], meta.iloc[tr_global].tasting_category_fine)
            values = fitted.predict(x.iloc[te_global])
            outer_rows.append(pd.concat([meta.iloc[te_global].reset_index(drop=True),
```

src/22_q3_stale_refined.py（续）

```
    pd.DataFrame({
285                 "global_index": te_global, "outer_fold": fold, "method": method_name,
286                 "observed": y45.iloc[te_global][TARGET].to_numpy(), "prediction": valu
    es,
287                 "selected_method": selected["selected_method"] if method_name == "nest
    ed_stability_rule" else method_name,
288                 "tail_threshold_training_q90": np.quantile(y45.iloc[tr_global][TARGET]
    , .9)})], axis=1))
289         print(f"Refined nested group fold {fold}/4: {selected['selected_method']}", fl
    ush=True)
290     nested = pd.concat(outer_rows, ignore_index=True)
291     nested.to_csv(OUT/"nested_predictions.csv", index=False, lineterminator="\n")
292     pd.DataFrame(nested_choices).to_csv(OUT/"nested_choices.csv", index=False, lineter
    minator="\n")
293     pd.concat(nested_inner_rows, ignore_index=True).to_csv(OUT/"nested_inner_metrics.c
    sv", index=False, lineterminator="\n")
294     pd.concat(nested_stability_rows, ignore_index=True).to_csv(OUT/"nested_selection_s
    tability.csv", index=False, lineterminator="\n")
295     pd.concat(nested_inner_audits, ignore_index=True).to_csv(OUT/"nested_inner_split_a
    udit.csv", index=False, lineterminator="\n")
296     pd.DataFrame(nested_outer_audits).to_csv(OUT/"nested_outer_split_audit.csv", index
    =False, lineterminator="\n")
297     nested_metrics, nested_style_rows = [], []
298     for name, group in nested.groupby("method"):
299         style_metrics = []
300         for style, z in group.groupby("tasting_category_fine"):
301             value = {"method": name, "heldout_style": style, "n": len(z), **scores(z.o
    bserved, z.prediction)}
302             nested_style_rows.append(value)
303             style_metrics.append(value)
304         tail = group.observed >= group.tail_threshold_training_q90
305         nested_metrics.append({"method": name, "macro_style_rmse": float(np.mean([z["r
    mse"] for z in style_metrics])),
306             "macro_style_abs_bias": float(np.mean([abs(z["mean_bias"]) for z in style_
    metrics])),
307             "tail_n": int(tail.sum()), "tail_rmse": float(np.sqrt(np.mean((group.loc[t
    ail,"prediction"]-group.loc[tail,"observed"])**2))),
308             **scores(group.observed, group.prediction)})
309     pd.DataFrame(nested_metrics).to_csv(OUT/"nested_metrics.csv", index=False, lineter
    minator="\n")
310     pd.DataFrame(nested_style_rows).to_csv(OUT/"nested_metrics_by_style.csv", index=Fa
    lse, lineterminator="\n")
311
312     selected_spec = decision["config"]
313     selected_metrics = decision["selected_loso_metrics"]
314     selected_uses_auxiliary = uses_auxiliary(selected_spec)
315     metadata = {"features": features, "target": TARGET, "target_scale": "attachment_4_
    standardized",
316         "config": selected_spec, "selected_config": selected_spec, "selected_method":
    decision["selected_method"],
317         "qualified_for_proxy": decision["qualified_for_proxy"], "qualification_scope":
     decision["qualification_scope"],
318         "inner_oof_variance_ratio": selected_metrics["prediction_variance_ratio"],
319         "inner_oof_rmse": selected_metrics["rmse"], "loso_macro_style_rmse": selected_
    metrics["macro_style_rmse"],
320         "loso_macro_style_abs_bias": selected_metrics["macro_style_abs_bias"],
321         "train_global_indices": train, "excluded_global_indices": source, "excluded_st
    yle": "Pils/Lager",
322         "uses_auxiliary_supervision": selected_uses_auxiliary,
323         "training_auxiliary_targets": [name for name in y45 if name != TARGET] if sele
    cted_uses_auxiliary else [],
```

src/22_q3_stale_refined.py（续）

```
        "candidate_protocol_auxiliary_targets": [name for name in y45 if name != TARGET],
        "prediction_inputs": "chemical features only", "pils_used_for_selection": False}
    fitted_candidates = {spec["name"]: model(spec, context, FINAL_SEED).fit(x.iloc[train], y45.iloc[train], meta.iloc[train].tasting_category_fine) for spec in specs}
    save_candidate_models(fitted_candidates, specs, context, final_table, decision, stability, train, source)
    final = fitted_candidates[decision["selected_method"]]
    joblib.dump({"pipeline": final, **metadata}, OUT/"transfer_stale_model.joblib", compress=3)
    # Development diagnostics only: every model is already fixed above.
    diagnostic_rows, diagnostic_metrics = [], []
    for spec in specs:
        values = fitted_candidates[spec["name"]].predict(x.iloc[source])
        diagnostic_rows.append(pd.concat([meta.iloc[source].reset_index(drop=True), pd.DataFrame({
            "global_index": source, "method": spec["name"], "observed": y45.iloc[source][TARGET].to_numpy(),
            "prediction": values, "selected_without_pils_labels": spec["name"] == decision["selected_method"]})], axis=1))
        diagnostic_metrics.append({"method": spec["name"], "selected_without_pils_labels": spec["name"] == decision["selected_method"],
                                   "n": len(source), **scores(y45.iloc[source][TARGET], values)})
    pd.concat(diagnostic_rows, ignore_index=True).to_csv(OUT/"pils_diagnostic_predictions.csv", index=False, lineterminator="\n")
    pd.DataFrame(diagnostic_metrics).to_csv(OUT/"pils_diagnostic_metrics.csv", index=False, lineterminator="\n")

    models, indices, seeds = [], [], []
    for b in range(60):
        seed = SELECTION_BOOTSTRAP_SEED+b if b < 40 else AUDIT_BOOTSTRAP_SEED+b-40
        index = np.random.default_rng(seed).choice(train, size=len(train), replace=True)
        fitted = model(selected_spec, context, seed+10000).fit(x.iloc[index], y45.iloc[index], meta.iloc[index].tasting_category_fine)
        models.append(fitted)
        indices.append(index)
        seeds.append(seed)
    ensemble = {"models": models, "bootstrap_indices": indices, **metadata,
        "selection_model_indices": list(range(40)), "audit_model_indices": list(range(40,60)),
        "bootstrap_seeds": seeds, "estimator_seeds": [seed+10000 for seed in seeds],
        "pairing": "same_model_candidate_minus_source", "paired_quantile": .95,
        "hyperparameter_uncertainty_included": False,
        "audit_protocol": "40 selection;20 once-only audit;no reselection", "fixed_indicator_dimensions": True}
    joblib.dump(ensemble, OUT/"bootstrap_ensemble.joblib", compress=3)
    np.savez_compressed(OUT/"bootstrap_indices.npz", indices=np.asarray(indices), bootstrap_seeds=np.asarray(seeds))

    # Same chemical pairs compare stability with the prior local architecture.
    chemical_pairs = feature_pairs(train, context)
    comparisons = []
    for label, committee in [("refined_selected", models), ("domain_local_previous", joblib.load(SOLUTION/"outputs/q3_domain/stale/bootstrap_ensemble.joblib")["models"])]:
        delta = np.asarray([m.predict(x.iloc[train])-m.predict(x.iloc[chemical_pairs]) for m in committee])
        noise = float(np.sqrt(np.mean(np.var(delta, axis=0, ddof=1))))
        signal = float(np.sqrt(np.mean(delta.mean(axis=0)**2)))
```

src/22_q3_stale_refined.py（续）

```
        comparisons.append({"method": label, "models": len(committee), "pair_delta_noise_rms": noise,
                            "pair_delta_signal_rms": signal, "response_noise_ratio": noise/max(signal,1e-8),
                            "status": "descriptive fixed-pair architecture comparison; not used for selection"})
    pd.DataFrame(comparisons).to_csv(OUT/"bootstrap_response_comparison.csv", index=False, lineterminator="\n")
    assert np.asarray(indices).shape == (60,239)
    assert all(not set(index)&set(source) for index in indices)
    assert all(np.isfinite(m.predict(x)).all() for m in models)
    summary = {
        "selected_method": decision["selected_method"], "config": selected_spec,
        "qualified_for_proxy": decision["qualified_for_proxy"], "selection_result": decision,
        "training_samples": len(train), "pils_sources": len(source), "loso_styles":21,"candidate_count":len(specs),
        "mean_baseline_method":"mean_global","old_multioutput_baseline_method":"old_multioutput_tree","old_single_tree_baseline_method":"fixed_single_tree",
        "selected_loso_metrics": selected_metrics, "nested_metrics":nested_metrics,
        "all_pils_diagnostic_metrics":diagnostic_metrics,
        "selected_pils_diagnostic_metrics":next(row for row in diagnostic_metrics if row["method"]==decision["selected_method"]),
        "pils_status":"repeatedly viewed development diagnostic; never used to select or correct intercept",
        "bootstrap_response_comparison":comparisons,
        "bootstrap":{"models":60,"selection":40,"audit":20,"source_excluded":True,"configuration_fixed":True},
        "limitations":[
            "Auxiliary sensory values are training outcomes only; neither stale nor any measured sensory value is an inference feature.",
            "Every auxiliary fit, fixed indicator transform, tail threshold, style weight and stability draw excludes its held-out group.",
            "Full LOSO is used to fit the final selection rule; selected scores are optimistic. Four outer grouped folds assess that fixed rule internally.",
            "The same dataset has guided prior exploration; grouped resampling is not new external validation and Pils diagnostics are repeatedly viewed.",
            "Tail weighting uses only training quantiles and never the beer81 outcome, but can increase average bias.",
            "The one-SE rule uses a descriptive across-style SE; observed styles are not independent randomized domains.",
            "Model response stability does not establish accuracy on synthetic interventions or genuine stale-hops taste.",
            "60 bootstrap models fix the chosen configuration and omit algorithm-selection and new-domain uncertainty.",
            "The 20 audit models cannot be used to reselect a proposal; model quantiles are not calibrated confidence limits.",
            "The provided standardized target is retained; no original 0-6 score is inferred."],
        "verification":{"source_exclusion":True,"bootstrap_indices_shape":[60,239],"all_model_predictions_finite":True},
        "versions":{"python":platform.python_version(),"numpy":np.__version__,"pandas":pd.__version__,"sklearn":sklearn.__version__},
        "elapsed_seconds":perf_counter()-started}
    write_json(OUT/"summary.json",summary)
    print(final_table[["method","macro_style_rmse","selection_score","prediction_variance_ratio","within_one_se"]].to_string(index=False),flush=True)
    print("Pils selected development diagnostic:",json.dumps(summary["selected_pils_diagnostic_metrics"],ensure_ascii=False),flush=True)
    print("Bootstrap response:",comparisons,flush=True)

```

src/22_q3_stale_refined.py（续）

```
403
404 if __name__ == "__main__":
405     with threadpool_limits(limits=1):
406         main()
```

src/23_q3_transfer_balanced.py

SHA-256: 953bf8882498756a1dbcf8675f41144e55967be5d2475f35868fb31efa3a935d

```
"""Candidate contraction and predeclared proximity/committee trade-offs."""
from __future__ import annotations

from importlib import import_module
import json
from time import perf_counter

import joblib
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist
from threadpoolctl import threadpool_limits

from common import SOLUTION
from q3_data import load_q3_data, read_defect_thresholds
from q3_stale_optimized import predict_1d, raw_tree_pipeline

CORE = import_module("17_q3_transfer_domain")
OUT = SOLUTION / "outputs/q3_refined/transfer"
PRIOR = SOLUTION / "outputs/q3_domain"
ORIGINAL = SOLUTION / "outputs/q3"
OPTIMIZED = SOLUTION / "outputs/q3_optimized"
REFINED = SOLUTION / "outputs/q3_refined/stale"
POOLS = ("legacy_2004", "legacy_6004", "contracted_6004")
BOUNDS = {"absolute10":(.10,None), "absolute30":(.30,None),
          "absolute50":(.50,None), "absolute60":(.60,None),
          "retain80":(.10,.80), "retain100":(.10,1.0)}
RANKINGS = ("geometry", "margin")
CONTRACTIONS = (.2,.4,.6)
GAMMAS = (.25,.5,.75,1.0)
DEFAULT = {"arm":"refined", "pool":"contracted_6004", "bound":"retain80", "ranking":"m
argin"}
OLD_AUDIT_SEED = 202104123
FIXED_AUDIT_SEED = 202105123
TOL = 1e-10


def write_json(path, value):
    path.write_text(json.dumps(value,ensure_ascii=False,indent=2,allow_nan=False)+"\n"
,encoding="utf-8", newline="\n")


def target_draws(reference, legacy):
    base = reference.transform(legacy.iloc[:500])
    center = reference.style_center_["Tripel"]
    latent = np.vstack([center+c*(base-center) for c in CONTRACTIONS])
    return reference.inverse_transform(latent), latent


def candidate_values(source, legacy, contracted, median, caps):
    blocks, rows = [], []
    for family, draws in [("legacy",legacy),("contracted",contracted)]:
        for gamma in GAMMAS:
            blocks.append((1-gamma)*source.to_numpy()[0]+gamma*draws.to_numpy())
            rows.extend({"family":family,"gamma":gamma,"draw":j,
                         "contraction":CONTRACTIONS[j//500] if family=="contracted" el
se np.nan}
                        for j in range(len(draws)))
    for gamma in GAMMAS:
```

src/23_q3_transfer_balanced.py（续）

```
        blocks.append(((1-gamma)*source.to_numpy()[0]+gamma*median)[None])
        rows.append({"family":"median","gamma":gamma,"draw":-1,"contraction":np.nan})
    frame = pd.DataFrame(np.vstack(blocks),columns=source.columns)
    for field,cap in caps.items():
        frame[field] = frame[field].clip(lower=0,upper=cap)
    return frame,pd.DataFrame(rows)


def pool_masks(descriptions):
    median = descriptions.family.eq("median").to_numpy()
    legacy = descriptions.family.eq("legacy").to_numpy()
    return {"legacy_2004":median|(legacy&descriptions.draw.lt(500).to_numpy()),
            "legacy_6004":median|legacy,
            "contracted_6004":median|descriptions.family.eq("contracted").to_numpy()}


def fresh_trees(x, y, train, seed):
    models, indices = [], []
    for b in range(20):
        current = seed+b
        index = np.random.default_rng(current).choice(train,len(train),replace=True)
        models.append(raw_tree_pipeline(current+10000,n_estimators=120).fit(x.iloc[ind
ex],y.iloc[index]))
        indices.append(index)
    return {"models":models,"bootstrap_indices":indices,"bootstrap_seeds":list(range(s
eed,seed+20)),
            "train_global_indices":train,"features":x.columns.tolist(),
            "protocol":"single-target ExtraTrees 120/minleaf3/max_features.7"}


def counts(part):
    selected = part.candidate_index.notna()
    passing = part.fresh_consistency_passed
    return {"sources":len(part),"selected":int(selected.sum()),"new_passed":int(part.n
ew_model_passed.sum()),
        "fresh_consistency_passed":int(passing.sum()),"historical_consistency_passed":
int(part.historical_consistency_passed.sum()),
        "no_candidate":int((~selected).sum()),
        "median_improvement_selected":float(part.loc[selected,"target_improvement"].me
dian()) if selected.any() else None,
        "median_improvement_passed":float(part.loc[passing,"target_improvement"].media
n()) if passing.any() else None,
        "median_retention_selected":float(part.loc[selected,"retained_anchor_gain"].me
dian()) if selected.any() else None,
        "median_change_rms_selected":float(part.loc[selected,"change_latent_rms"].medi
an()) if selected.any() else None}


def main():
    started = perf_counter()
    OUT.mkdir(parents=True,exist_ok=True)
    meta,x,y45,_ = load_q3_data()
    y = y45.stale_hops
    train = np.flatnonzero(~meta.tasting_category_fine.eq("Pils/Lager"))
    sources = np.flatnonzero(meta.tasting_category_fine.eq("Pils/Lager"))
    target = np.flatnonzero(meta.tasting_category_fine.eq("Tripel"))
    features = x.columns.tolist()
    defects = read_defect_thresholds()
    scenarios = json.loads((OPTIMIZED/"transfer/threshold_scenarios.json").read_text(e
ncoding="utf-8"))
    manifest = {"default":DEFAULT,"pools":dict(zip(POOLS,(2004,6004,6004))),
```

src/23_q3_transfer_balanced.py（续）

```
        "bounds":{key:{"absolute_minimum":value[0],"anchor_fraction":value[1]} for key
,value in BOUNDS.items()},
        "rankings":RANKINGS,"contractions":CONTRACTIONS,"gammas":GAMMAS,
        "contracted_latents":"transform frozen legacy first500 chemical rows, then mu+
c*(z-mu); empirical inverse CDF",
        "anchor":"frozen q3_optimized default spectrum per source; same original geome
try",
        "retention_floor":"max(.10, anchor_fraction*anchor_gain)",
        "screens":"new40 and original30 paired Q95<=0; both current centers delta<=0;
nonzero paired signals",
        "fresh_audit":"new20 and new-seed original single-target120tree20; no reselect
ion",
        "historical_audit":"original last10 reused diagnostic only, excluded from prim
ary fresh gate",
        "old_audit_seed":OLD_AUDIT_SEED,"fixed_audit_seed":FIXED_AUDIT_SEED,
        "threshold_scenarios":scenarios,"minimum_change_rms":.05,"geometry_lambda":.2,
        "default_chosen_from_audit":False,"audit_reselection":False,
        "interpretation":"finite empirical trade-off grid; not a global Pareto optimum
 or tasting validation"}
    write_json(OUT/"run_manifest.json",manifest)
    geometry = joblib.load(ORIGINAL/"transfer/transfer_copula.joblib")
    np.testing.assert_array_equal(geometry["training_rows"],train)
    reference = geometry["copula"]
    legacy = pd.read_csv(PRIOR/"consensus/base_draws.csv")
    contracted,contracted_latents = target_draws(reference,legacy)
    legacy.to_csv(OUT/"legacy_base_draws.csv",index=False, lineterminator="\n")
    contracted.to_csv(OUT/"contracted_base_draws.csv",index=False, lineterminator="\n"
)
    np.save(OUT/"contracted_latents.npy",contracted_latents)
    pd.DataFrame({"draw":np.arange(1500),"base_draw":np.tile(np.arange(500),3),
        "contraction":np.repeat(CONTRACTIONS,500)}).to_csv(OUT/"contracted_draw_metada
ta.csv",index=False, lineterminator="\n")
    tz = reference.transform(x.iloc[target])
    dim = x.shape[1]
    pairwise = cdist(tz,tz)/np.sqrt(dim)
    np.fill_diagonal(pairwise,np.inf)
    support_limit = float(np.quantile(pairwise.min(axis=1),.95))
    center = tz.mean(axis=0)
    medians = x.iloc[train].median()
    minimum,maximum = x.iloc[train].min().to_numpy(),x.iloc[train].max().to_numpy()
    target_median = x.iloc[target].median().to_numpy()
    old = joblib.load(ORIGINAL/"sensory/stale_bootstrap.joblib")
    old_center = joblib.load(ORIGINAL/"sensory/transfer_proxy.joblib")
    refined_center = joblib.load(REFINED/"transfer_stale_model.joblib")
    refined_ensemble = joblib.load(REFINED/"bootstrap_ensemble.joblib")
    fixed_center = joblib.load(OPTIMIZED/"stale_fixed/transfer_stale_model.joblib")
    fixed_ensemble = joblib.load(OPTIMIZED/"stale_fixed/bootstrap_ensemble.joblib")
    old_audit = fresh_trees(x,y,train,OLD_AUDIT_SEED)
    fixed_audit = fresh_trees(x,y,train,FIXED_AUDIT_SEED)
    old_audit["excluded_global_indices"] = sources
    fixed_audit["excluded_global_indices"] = sources
    joblib.dump(old_audit,OUT/"fresh_original_audit.joblib",compress=3)
    joblib.dump(fixed_audit,OUT/"fresh_fixed_audit.joblib",compress=3)
    arms = {"refined":{"center":refined_center,"selection":refined_ensemble["models"][
:40],
                       "audit":refined_ensemble["models"][40:]},
            "fixed":{"center":fixed_center,"selection":fixed_ensemble["models"][:40],"
audit":fixed_audit["models"]}}
    for artifact in [old,old_center,refined_center,refined_ensemble,fixed_center,fixed
_ensemble]:
        np.testing.assert_array_equal(artifact["train_global_indices"],train)
```

src/23_q3_transfer_balanced.py（续）

```
        assert artifact["features"]==features
    assert len(refined_ensemble["models"])==60
    anchor_summary = pd.read_csv(OPTIMIZED/"transfer/transfer_summary.csv")
    anchor_summary = anchor_summary[anchor_summary.is_default_policy].set_index("sourc
e_beer")
    anchor_spectra = pd.read_csv(OPTIMIZED/"transfer/transferred_spectra.csv").set_ind
ex("transfer_id")
    prior_summary = pd.read_csv(PRIOR/"consensus/transfer_summary.csv")
    prior_spectra = pd.read_csv(PRIOR/"consensus/transferred_spectra.csv").set_index("
transfer_id")
    records,diags,geometry_rows,selected_rows,delta_rows,threshold_rows,rechecks,uniqu
eness = [],[],[],[],[],[],[],[]
    stale_position = y45.columns.get_loc("stale_hops")
    for number,index in enumerate(sources,1):
        beer = int(meta.beer.iloc[index])
        raw = x.iloc[[index]]
        source = raw.fillna(medians)
        assert all(np.isfinite(raw[r["feature"]].iloc[0]) and raw[r["feature"]].iloc[0
]>=0 for r in defects.values())
        caps = {}
        for rule in defects.values():
            field = rule["feature"]
            values = [s["thresholds"][field] for s in scenarios if s["thresholds"][fie
ld] is not None]
            caps[field] = max(float(raw[field].iloc[0]),min(values)) if values else fl
oat(raw[field].iloc[0])
        candidates,descriptions = candidate_values(source,legacy,contracted,target_med
ian,caps)
        masks = pool_masks(descriptions)
        unique,inverse = np.unique(candidates.to_numpy(),axis=0,return_inverse=True)
        unique_frame = pd.DataFrame(unique,columns=features)
        z,sz = reference.transform(candidates),reference.transform(source)
        distance = reference.distance(candidates,"Tripel")
        before = float(reference.distance(source,"Tripel")[0])
        improvement = 1-distance/max(before,1e-8)
        change = np.linalg.norm(z-sz,axis=1)/np.sqrt(dim)
        nn = cdist(z,tz).min(axis=1)/np.sqrt(dim)
        support = (nn<=support_limit+TOL)&((candidates.to_numpy()>=minimum-TOL)&(candi
dates.to_numpy()<=maximum+TOL)).all(axis=1)
        chemical = np.column_stack([CORE.chemical_ok(candidates,raw,s) for s in scenar
ios]).all(axis=1)
        geometric = support&chemical&(change>=.05)
        scale = max(float(np.linalg.norm(sz[0]-center)/np.sqrt(dim)),1e-8)
        objective = distance/max(before,1e-8)+.2*change/scale
        anchor_row = anchor_summary.loc[beer]
        anchor_frame = anchor_spectra.loc[[anchor_row.transfer_id],features]
        anchor_gain = 1-float(reference.distance(anchor_frame,"Tripel")[0])/max(before
,1e-8)
        assert anchor_gain>0
        geometry_rows.append(pd.concat([pd.DataFrame({"source_beer":beer,"candidate_in
dex":np.arange(len(candidates)),
            "target_distance":distance,"target_improvement":improvement,"change_latent
_rms":change,"target_nn_rms":nn,
            "support_ok":support,"chemical_ok":chemical,"geometry_objective":objective
}),descriptions],axis=1))
        for pool,mask in masks.items():
            unique_n = len(np.unique(candidates.to_numpy()[mask],axis=0))
            uniqueness.append({"source_beer":beer,"pool":pool,"nominal_candidates":int
(mask.sum()),"unique_candidates":unique_n,
                "duplicate_fraction":1-unique_n/int(mask.sum()),"anchor_gain":anchor_g
ain})
```

src/23_q3_transfer_balanced.py（续）

```
        old_delta = CORE.paired(old["models"][:30],unique_frame,source)[inverse]
        old_q = np.quantile(old_delta,.95,axis=1)
        old_signal = np.any(np.abs(old_delta)>1e-8,axis=1)
        center_old_source = float(old_center["pipeline"].predict(source)[0,stale_posit
ion])
        center_old_delta = old_center["pipeline"].predict(unique_frame)[:,stale_positi
on][inverse]-center_old_source
        for arm,models in arms.items():
            current = models["center"]
            qualified = bool(current["qualified_for_proxy"])
            new_delta = CORE.paired(models["selection"],unique_frame,source)[inverse]
            new_q = np.quantile(new_delta,.95,axis=1)
            new_signal = np.any(np.abs(new_delta)>1e-8,axis=1)
            center_new_source = float(predict_1d(current["pipeline"],source)[0])
            center_new_delta = predict_1d(current["pipeline"],unique_frame)[inverse]-c
enter_new_source
            worst = np.maximum(new_q,old_q)
            center_ok = (center_new_delta<=TOL)&(center_old_delta<=TOL)
            joint = geometric&(new_q<=TOL)&(old_q<=TOL)&new_signal&old_signal&center_o
k
            diags.append(pd.DataFrame({"source_beer":beer,"arm":arm,"candidate_index":
np.arange(len(candidates)),
                "new_selection_q95":new_q,"old_selection_q95":old_q,"worst_selection_q
95":worst,
                "center_new_delta":center_new_delta,"center_old_delta":center_old_delt
a,
                "new_signal":new_signal,"old_signal":old_signal,"joint_feasible":joint
}))
            cache = {}
            def assess(frame):
                key=frame.to_numpy().tobytes()
                if key not in cache:
                    cache[key]={"new_audit":CORE.paired(models["audit"],frame,source)[
0],
                        "old_fresh_audit":CORE.paired(old_audit["models"],frame,source
)[0],
                        "old_historical":CORE.paired(old["models"][30:],frame,source)[
0]}
                return cache[key]
            for pool,mask in masks.items():
                new_variation=float(np.std(new_delta[mask].mean(axis=1)))
                old_variation=float(np.std(old_delta[mask].mean(axis=1)))
                informative=new_variation>1e-6 and old_variation>1e-6
                for bound,(absolute,retention) in BOUNDS.items():
                    floor=max(absolute,retention*anchor_gain) if retention is not None
 else absolute
                    ids=np.flatnonzero(joint&mask&(improvement>=floor))
                    for ranking in RANKINGS:
                        uid=f"{arm}_beer{beer}_{pool}_{bound}_{ranking}"
                        common={"transfer_id":uid,"source_beer":beer,"arm":arm,"pool":
pool,"bound":bound,"ranking":ranking,
                            "is_default_policy":arm==DEFAULT["arm"] and pool==DEFAULT[
"pool"] and bound==DEFAULT["bound"] and ranking==DEFAULT["ranking"],
                            "is_arm_default":pool==DEFAULT["pool"] and bound==DEFAULT[
"bound"] and ranking==DEFAULT["ranking"],
                            "anchor_gain":anchor_gain,"required_gain":floor,"qualified
":qualified,"informative":informative,
                            "candidate_count":int(mask.sum()),"feasible_count":len(ids
),"new_delta_sd":new_variation,"old_delta_sd":old_variation,
                            "candidate_index":np.nan,"new_model_passed":False,"fresh_c
onsistency_passed":False,"historical_consistency_passed":False}
```

src/23_q3_transfer_balanced.py（续）

```
                        if not len(ids):
                            records.append({**common,"status":"no_candidate"});continu
e
                        chosen=int(ids[np.lexsort((objective[ids],worst[ids]))[0]]) if
 ranking=="margin" else int(ids[np.argmin(objective[ids])])
                        frame=candidates.iloc[[chosen]]
                        audited=assess(frame)
                        aq={name:float(np.quantile(delta,.95)) for name,delta in audit
ed.items()}
                        new_pass=bool(qualified and informative and aq["new_audit"]<=T
OL)
                        fresh_pass=bool(new_pass and aq["old_fresh_audit"]<=TOL)
                        history_pass=bool(new_pass and aq["old_historical"]<=TOL)
                        status="unqualified" if not qualified else "uninformative" if
not informative else "new_audit_failed" if not new_pass else "old_fresh_audit_failed"
if not fresh_pass else "passed"
                        records.append({**common,"candidate_index":chosen,"status":sta
tus,"target_distance_before":before,
                            "target_distance_after":float(distance[chosen]),"target_im
provement":float(improvement[chosen]),
                            "retained_anchor_gain":float(improvement[chosen]/anchor_ga
in),"change_latent_rms":float(change[chosen]),
                            "target_nn_rms":float(nn[chosen]),"support_nn_limit":suppo
rt_limit,"geometry_objective":float(objective[chosen]),
                            "new_selection_q95":float(new_q[chosen]),"old_selection_q9
5":float(old_q[chosen]),"worst_selection_q95":float(worst[chosen]),
                            "center_new_delta":float(center_new_delta[chosen]),"center
_old_delta":float(center_old_delta[chosen]),
                            "new_audit_q95":aq["new_audit"],"old_fresh_audit_q95":aq["
old_fresh_audit"],"old_historical_q95":aq["old_historical"],
                            "new_model_passed":new_pass,"fresh_consistency_passed":fre
sh_pass,"historical_consistency_passed":history_pass})
                        selected_rows.append({"transfer_id":uid,**frame.iloc[0].to_dic
t()})
                        for name,delta in {"new_selection":new_delta[chosen],"old_sele
ction":old_delta[chosen],**audited}.items():
                            delta_rows.extend({"transfer_id":uid,"ensemble":name,"mode
l_index":j,"delta":float(value)} for j,value in enumerate(delta))
                        for scenario in scenarios:
                            for label,rule in defects.items():
                                field=rule["feature"];threshold=scenario["thresholds"]
[field]
                                source_value=float(raw[field].iloc[0]);after=float(fra
me[field].iloc[0])
                                cap=max(source_value,threshold) if threshold is not No
ne else source_value
                                threshold_rows.append({"transfer_id":uid,"scenario":sc
enario["scenario"],"defect":label,"feature":field,
                                    "source":source_value,"transferred":after,"cap":ca
p,"passed":after<=cap+TOL})
            references=[("original_anchor",anchor_frame)]
            for pool in ("matched_2004","expanded_6004"):
                prior_row=prior_summary[prior_summary.source_beer.eq(beer)&prior_summa
ry.pool.eq(pool)&prior_summary.policy.eq("joint_margin")].iloc[0]
                if pd.notna(prior_row.candidate_index):
                    references.append((f"prior_{pool}",prior_spectra.loc[[prior_row.tr
ansfer_id],features]))
            for label,frame in references:
                pz=reference.transform(frame);gain=1-float(reference.distance(frame,"T
ripel")[0])/max(before,1e-8)
                shift=float(np.linalg.norm(pz-sz)/np.sqrt(dim))
```

src/23_q3_transfer_balanced.py（续）

```
                support_ok=bool(cdist(pz,tz).min()/np.sqrt(dim)<=support_limit+TOL and
                    ((frame.to_numpy()>=minimum-TOL)&(frame.to_numpy()<=maximum+TOL)).all())
                chemical_ok=bool(all(CORE.chemical_ok(frame,raw,s)[0] for s in scenarios))
                nd=CORE.paired(models["selection"],frame,source)[0];od=CORE.paired(old["models"][:30],frame,source)[0]
                nq,oq=float(np.quantile(nd,.95)),float(np.quantile(od,.95))
                cdn=float(predict_1d(current["pipeline"],frame)[0]-center_new_source)
                cdo=float(old_center["pipeline"].predict(frame)[0,stale_position]-center_old_source)
                assessed=assess(frame);aq={key:float(np.quantile(value,.95)) for key,value in assessed.items()}
                evaluation_mask=masks[DEFAULT["pool"]]
                base_pass=bool(qualified and np.std(new_delta[evaluation_mask].mean(axis=1))>1e-6
                    and np.std(old_delta[evaluation_mask].mean(axis=1))>1e-6
                    and support_ok and chemical_ok and shift>=.05 and nq<=TOL and oq<=TOL and cdn<=TOL and cdo<=TOL
                    and np.any(np.abs(nd)>1e-8) and np.any(np.abs(od)>1e-8))
                for bound,(absolute,retention) in BOUNDS.items():
                    floor=max(absolute,retention*anchor_gain) if retention is not None else absolute
                    passed=bool(base_pass and gain>=floor and aq["new_audit"]<=TOL and aq["old_fresh_audit"]<=TOL)
                    rechecks.append({"source_beer":beer,"arm":arm,"reference":label,"evaluation_pool":DEFAULT["pool"],"bound":bound,"required_gain":floor,
                        "target_improvement":gain,"retained_anchor_gain":gain/anchor_gain,"change_latent_rms":shift,
                        "support_ok":support_ok,"chemical_ok":chemical_ok,"new_selection_q95":nq,"old_selection_q95":oq,
                        "center_new_delta":cdn,"center_old_delta":cdo,"new_audit_q95":aq["new_audit"],
                        "old_fresh_audit_q95":aq["old_fresh_audit"],"fresh_consistency_passed":passed})
            print(f"Balanced transfer source {number}/11 beer={beer}, arm={arm}, joint={joint.sum()}",flush=True)
    result=pd.DataFrame(records)
    result.to_csv(OUT/"transfer_summary.csv",index=False, lineterminator="\n")
    pd.concat(geometry_rows,ignore_index=True).to_csv(OUT/"candidate_geometry.csv.gz",index=False,compression="gzip", lineterminator="\n")
    pd.concat(diags,ignore_index=True).to_csv(OUT/"candidate_model_checks.csv.gz",index=False,compression="gzip", lineterminator="\n")
    pd.DataFrame(selected_rows).to_csv(OUT/"selected_spectra.csv",index=False, lineterminator="\n")
    pd.DataFrame(delta_rows).to_csv(OUT/"paired_deltas.csv",index=False, lineterminator="\n")
    pd.DataFrame(threshold_rows).to_csv(OUT/"threshold_checks.csv",index=False, lineterminator="\n")
    pd.DataFrame(rechecks).to_csv(OUT/"fixed_spectra_rechecks.csv",index=False, lineterminator="\n")
    pd.DataFrame(uniqueness).to_csv(OUT/"pool_uniqueness.csv",index=False, lineterminator="\n")
    frontier=[{"arm":arm,"pool":pool,"bound":bound,"ranking":ranking,**counts(part)}
        for (arm,pool,bound,ranking),part in result.groupby(["arm","pool","bound","ranking"])]
    pd.DataFrame(frontier).to_csv(OUT/"frontier.csv",index=False, lineterminator="\n")
    arm_defaults={arm:counts(result[result.arm.eq(arm)&result.is_arm_default]) for arm in arms}
    summary={"default":DEFAULT,"default_results":counts(result[result.is_default_policy]),"arm_defaults":arm_defaults,
```

src/23_q3_transfer_balanced.py（续）

```
        "model_qualified":{arm:bool(models["center"]["qualified_for_proxy"]) for arm,models in arms.items()},
        "nominal_candidates_per_source":12004,"sources":len(sources),"training_n":len(train),"support_limit":support_limit,
        "contracted_targets_unique":len(np.unique(contracted.to_numpy(),axis=0)),"contracted_targets_nominal":len(contracted),
        "frontier_rows":len(frontier),"policies":len(result),"selected":len(selected_rows),"threshold_rows":len(threshold_rows),
        "elapsed_seconds":perf_counter()-started}
    write_json(OUT/"summary.json",summary)
    print(json.dumps(summary,ensure_ascii=False,indent=2),flush=True)


if __name__=="__main__":
    with threadpool_limits(limits=1):main()
```

src/24_q3_refinement_report.py

SHA-256: ddcd0c5097ddbfa230a6544cdc717bfa08eec80d964cb1a59fc4999162c3f87e

```
"""Report beer81 diagnosis, grouped prediction, and finite transfer trade-offs."""
import hashlib
import json

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from common import SOLUTION

OUT=SOLUTION/"outputs/q3_refined"
SOURCES=[]
COLORS={"legacy_2004":"#929AA3","legacy_6004":"#B98632","contracted_6004":"#267DA7"}


def read(path):
    raw=path.read_bytes()
    value=json.loads(raw) if path.suffix==".json" else pd.read_csv(path)
    SOURCES.append({"path":path.relative_to(SOLUTION).as_posix(),"sha256":hashlib.sha256(raw).hexdigest(),
                    "rows":len(value) if isinstance(value,pd.DataFrame) else None})
    return value


def number(value, percent=False):
    return "—" if pd.isna(value) else (f"{value:.1%}" if percent else f"{value:.4f}")


def save(fig,name):
    fig.savefig(OUT/f"figures/{name}.png",dpi=300,bbox_inches="tight")
    fig.savefig(OUT/f"figures/{name}.svg",bbox_inches="tight")
    plt.close(fig)


def main():
    (OUT/"figures").mkdir(parents=True,exist_ok=True)
    (OUT/"report_tables").mkdir(parents=True,exist_ok=True)
    audit=read(OUT/"beer81/summary.json")
    prediction=read(OUT/"stale/summary.json")
    loso=read(OUT/"stale/loso_metrics_overall.csv")
    pils=read(OUT/"stale/pils_diagnostic_metrics.csv")
    pils_rows=read(OUT/"stale/pils_diagnostic_predictions.csv")
    nested=read(OUT/"stale/nested_metrics.csv")
    response=read(OUT/"stale/bootstrap_response_comparison.csv")
    transfer=read(OUT/"transfer/summary.json")
    manifest=read(OUT/"transfer/run_manifest.json")
    frontier=read(OUT/"transfer/frontier.csv")
    outcomes=read(OUT/"transfer/transfer_summary.csv")
    references=read(OUT/"transfer/fixed_spectra_rechecks.csv")
    unique=read(OUT/"transfer/pool_uniqueness.csv")
    comparison=loso.merge(pils[["method","rmse","mean_bias"]].rename(columns={"rmse":"pils_rmse","mean_bias":"pils_bias"}),on="method",validate="one_to_one")
    comparison.to_csv(OUT/"report_tables/prediction_comparison.csv",index=False, lineterminator="\n")
    selected=prediction["selected_method"]
    selected_row=comparison.set_index("method").loc[selected]
    tail=comparison.set_index("method").loc["tail_ridge_aging100"]
```

src/24_q3_refinement_report.py（续）

```
    tail_r2=float(pils.set_index("method").loc["tail_ridge_aging100","r2"])
    old=comparison.set_index("method").loc["old_multioutput_tree"]
    nested_values=nested.set_index("method")
    selected81=float(pils_rows.loc[pils_rows.method.eq(selected)&pils_rows.beer.eq(81),"prediction"].iloc[0])
    tail81=float(pils_rows.loc[pils_rows.method.eq("tail_ridge_aging100")&pils_rows.beer.eq(81),"prediction"].iloc[0])
    retained=outcomes[outcomes.bound.eq("retain80")]
    retained.to_csv(OUT/"report_tables/retain80_all_policies.csv",index=False, lineterminator="\n")
    ref_counts=references.groupby(["arm","reference","bound"],as_index=False).agg(
        available=("source_beer","size"),passed=("fresh_consistency_passed","sum"),
        median_gain=("target_improvement","median"))
    ref_counts["sources_denominator"]=11
    ref_counts.to_csv(OUT/"report_tables/reference_counts.csv",index=False, lineterminator="\n")
    plt.rcParams.update({"font.family":"DejaVu Sans","font.size":10,"axes.spines.top":False,"axes.spines.right":False,"svg.fonttype":"none"})
    fig,axes=plt.subplots(1,2,figsize=(12,5.4),gridspec_kw={"width_ratios":[1.1,1]})
    for row in comparison.itertuples():
        color="#267DA7" if row.method==selected else "#B98632" if row.method=="tail_ridge_aging100" else "#929AA3"
        axes[0].scatter(row.macro_style_rmse,row.pils_rmse,color=color,s=50)
    for method,label,offset in [(selected,"Selected stable ridge",(10,8)),("tail_ridge_aging100","Tail ridge (not eligible)",(-125,12)),("old_multioutput_tree","45-output reference",(8,-12))]:
        row=comparison.set_index("method").loc[method]
        axes[0].annotate(label,(row.macro_style_rmse,row.pils_rmse),xytext=offset,textcoords="offset points",fontsize=8,
                         arrowprops={"arrowstyle":"-","color":"#68717C","lw":.6})
    axes[0].set(xlabel="Non-Pils LOSO macro RMSE (selection data)",ylabel="Pils RMSE (repeatedly viewed diagnostic)",title="Prediction trade-off; lower is better",xlim=(.36,.422),ylim=(.509,.567))
    beer=pils_rows[pils_rows.beer.eq(81)].set_index("method")
    methods=["old_multioutput_tree",selected,"tail_ridge_aging100","aux4_tail_ridge10"]
    values=beer.loc[methods,"prediction"].to_numpy()
    axes[1].barh(np.arange(4),values,color=["#929AA3","#267DA7","#B98632","#C5AC79"])
    axes[1].set_yticks(np.arange(4),["45-output reference","Selected stable ridge","Tail ridge (not selected)","Auxiliary tail (not selected)"],fontsize=8)
    axes[1].axvline(audit["stale_observed"],color="#333D46",label="Observed beer81",linestyle="--")
    axes[1].axvline(audit["frozen_local_top25_stale_max"],color="#929AA3",label="Previous top25 maximum",linestyle=":")
    axes[1].set(xlabel="Standardized stale_hops",title="Beer81 underprediction",xlim=(-.3,1.85))
    axes[1].legend(frameon=False,fontsize=8,loc="lower right")
    for ax in axes:ax.grid(alpha=.2);ax.set_axisbelow(True)
    fig.subplots_adjust(wspace=.75,bottom=.2)
    fig.text(.1,.025,"Pils labels did not select the main model. The tail configuration improves this diagnostic point estimate\nbut fails training-only eligibility; no candidate is promoted using this diagnostic.",fontsize=9,color="#5C6470")
    save(fig,"beer81_and_prediction")
    fig,axes=plt.subplots(1,2,figsize=(12.5,5.1),sharey=True)
    for ax,arm in zip(axes,["refined","fixed"]):
        part=frontier[frontier.arm.eq(arm)]
        for pool,color in COLORS.items():
            for ranking,marker in [("margin","o"),("geometry","s")]:
                rows=part[part.pool.eq(pool)&part.ranking.eq(ranking)&part.fresh_consistency_passed.gt(0)]
                ax.scatter(100*rows.median_improvement_passed,rows.fresh_consistency_p
```

src/24_q3_refinement_report.py（续）

```
    assed,
 99                     marker=marker,facecolors=color if ranking=="margin" else "none",ed
    gecolors=color,s=50,label=f"{pool} / {ranking}")
100         point=part[part.pool.eq(manifest["default"]["pool"])&part.bound.eq("retain80")
    &part.ranking.eq("margin")].iloc[0]
101         ax.scatter(100*point.median_improvement_passed,point.fresh_consistency_passed,
    marker="*",s=250,color="#27323C",zorder=5)
102         ax.set(title=f"{arm}: fixed policy grid",xlabel="Median Tripel distance reduct
    ion among passing spectra (%)",ylim=(-.3,11.5),xlim=(0,100),yticks=range(0,12))
103         ax.grid(alpha=.2)
104     axes[0].set_ylabel("Sources passing fresh checks (out of 11)")
105     handles,labels=axes[0].get_legend_handles_labels()
106     fig.legend(handles,labels,loc="lower center",ncol=3,frameon=False,fontsize=8,bbox_
    to_anchor=(.5,.035))
107     fig.subplots_adjust(bottom=.29,wspace=.15)
108     fig.text(.12,.005,"Stars: predeclared contracted / retain80 / margin. No audit-dri
    ven default switch; finite observed grid, not a global Pareto proof.",fontsize=8,color
    ="#5C6470")
109     save(fig,"proximity_consistency_grid")
110     lines=["# 第三问：beer 81误差诊断与迁移接近度保留","",
111         "## 本轮结果","",
112         f"beer 81身份连接正确，陈旧感目标为 **{audit['stale_observed']:.4f}**（全250款第{
    audit['stale_descending_rank_all250']}高）。"
113         "论坛术语表也有较高的陈旧/纸板/纸质描述；没有证据据此改标签或删除该样本。",
114         f"稳定性选型保留 `{selected}`，Pils诊断RMSE **{selected_row.pils_rmse:.4f}**，仍
    未优于本轮旧45输出对照 **{old.pils_rmse:.4f}**。"
115         f"预声明尾部加权备选的Pils点误差为 **{tail.pils_rmse:.4f}**，但其非Pils留出误差变
    差，未获准替换主模型。",
116         f"主预测模型迁移臂通过 **{transfer['arm_defaults']['refined']['fresh_consistency
    _passed']}/11**；"
117         f"固定树方法对照的中央候选、retain80、margin通过 **{transfer['arm_defaults']['fix
    ed']['fresh_consistency_passed']}/11**，"
118         f"所选谱目标距离改善中位数 **{transfer['arm_defaults']['fixed']['median_improvem
    ent_selected']:.1%}**。"
119         "两臂分别报告，没有依据审计把较高通过数的一臂改名为默认。","",
120         "## beer 81为什么难预测","",
121         f"1.csv与8.xlsx精确条码一致，4.csv/7.csv条码只是给定精度较低；始终以beer精确连接。"
122         f"该行混合化学输入没有缺失或非法校准负值，只有{audit['sample_profile']['outside_no
    npils_feature_ranges']}项略超非Pils训练范围。"
123         "相对量中重复负值是附件给定表示，不能擅自当作浓度或检测下限。",
124         f"旧局部模型的25近邻目标范围为 [{audit['frozen_local_top25_stale_min']:.4f}, {au
    dit['frozen_local_top25_stale_max']:.4f}]，"
125         f"实际值{audit['stale_observed']:.4f}超出该凸包，非负加权均值结构无法达到它。训练2
    39款中仅"
126         f"{audit['nonpils_stale_at_least_beer81']}款同等高尾，且均不在这25个近邻中。"
127         "这说明高缺陷尾部稀疏、化学相似性与该感官目标不一致，不能证明所有化学模型都不可能预测
    。",
128         f"新主模型对81预测 **{selected81:.4f}**，尾部加权备选为 **{tail81:.4f}**，仍明显
    低估。"
129         "缺陷可能涉及未记录的储存、批次或品评差异，当前附件无法识别真实原因。7.csv是数值术语
    表而非原始评论；"
130         "它仅用于语义佐证，未用作生成谱推理特征。所有Pils主指标保留11源。","",
131         "## 预测的内部改善与外推代价","",
132         "固定缺失指示维数、平滑岭回归、辅助感官训练监督及训练内尾部权重均为本轮预声明比较。"
133         "辅助44感官只在训练阶段学习化学方向，推理只接受226维化学输入。"
134         "主规则先应用完整风格留出精度与非退化门槛，再在一标准误差容忍带内按训练内响应稳定性选
    择。"
135         "该容忍带是选择启发式，不是置信区间。",
136         f"4外层分组、内层完整风格留出评价整个规则：pooled RMSE **{nested_values.loc['nest
    ed_stability_rule','rmse']:.4f}**，"
137         f"同折旧45输出树 **{nested_values.loc['old_multioutput_tree','rmse']:.4f}**、固
```

```
    定单目标树 **{nested_values.loc['fixed_single_tree','rmse']:.4f}**。"
138         "这是复用本批非Pils数据的嵌套内部点比较，不是新增外部测试。",
139         "|配置|LOSO宏RMSE|LOSO pooled RMSE|Pils诊断RMSE|训练内准入|主方法|","|---|---:|---:|---:|---|---|"]
140     for row in comparison.itertuples():
141         lines.append(f"|{row.method}|{row.macro_style_rmse:.4f}|{row.rmse:.4f}|{row.pils_rmse:.4f}|{'是' if row.eligible_for_one_se else '否'}|{'是' if row.method==selected else ''}|")
142     lines.extend(["",f"尾部备选的LOSO宏RMSE为{tail.macro_style_rmse:.4f}、pooled为{tail.rmse:.4f}，Pils R²仍为{tail_r2:.4f}，因此不能因Pils较小就后验换主方法。"
143         "全部可选配置工件均保存于 `../outputs/q3_refined/stale/candidate_models/`，未选配置明确标作开发诊断。",
144         "", "|60次bootstrap响应|噪声RMS|信号RMS|噪声/信号|","|---|---:|---:|---:|"])
145     for row in response.itertuples():lines.append(f"|{row.method}|{row.pair_delta_noise_rms:.4f}|{row.pair_delta_signal_rms:.4f}|{row.response_noise_ratio:.4f}|")
146     lines.extend(["","响应对由非Pils真实化学邻居固定产生；其中包含拟合内点，只是训练内稳定性诊断，不能证明迁移后的真实感官因果效应。",
147         "","![81与预测误差](../outputs/q3_refined/figures/beer81_and_prediction.png)","",
148         "## 同时保护一致性与Tripel接近幅度","",
149         "新增中央候选把冻结500个目标谱经原高斯秩映射后，按0.2/0.4/0.6向Tripel中心收缩，再经经验逆CDF还原。"
150         "它是搜索proposal，不当作更高质量生成模型的证据；每源投影后重新检查原完整226维几何、NN支持和十阈值保护。",
151         f"1500个中央目标谱中有{transfer['contracted_targets_unique']}个唯一向量。逐源投影后去重数和重复率另存pool_uniqueness.csv；"
152         "6004表示名义预算，不等于6004个独立新样本。",
153         "retain80要求每源保留最早冻结默认谱至少80%的目标距离改善，并保持最少10%改善。"
154         "另完整保存retain100及绝对10%/30%/50%/60%下限、两种排序、三个池的经验权衡网格。"
155         "默认在审计前固定为refined/contracted_6004/retain80/margin，没有根据审计改选。",
156         "筛选同时要求新旧委员会配对Q95不增加及两个中心预测不增加。旧中心确为45输出模型，但旧bootstrap仍是单目标120树；"
157         "45输出中心只作单点一致检查，不声称已做该架构的bootstrap审计。新20与新种子旧协议20只审计已选谱，失败不重选。",
158         "","|臂|候选池|retain80排序|选出/11|fresh通过/11|通过谱改善中位数|所选谱改善中位数|保留率中位数|",
159         "|---|---|---|---:|---:|---:|---:|---:|"])
160     for row in frontier[frontier.bound.eq("retain80")].itertuples():
161         lines.append(f"|{row.arm}|{row.pool}|{row.ranking}|{row.selected}|{row.fresh_consistency_passed}|{number(row.median_improvement_passed,True)}|{number(row.median_improvement_selected,True)}|{number(row.median_retention_selected,True)}|")
162     lines.extend(["", "同一臂、同一审计及下限才可直接比较。geometry是预声明几何优先对照，margin优先预测余量；"
163         "两者的观察结果均保留，不按通过数宣布一个新默认。没有找到谱的源始终计入11分母。",
164         "","|同本轮规则重检历史固定谱（retain80）|可用谱|fresh通过/11|原改善中位数|","|---|---:|---:|---:|"])
165     for row in ref_counts[ref_counts.bound.eq("retain80")].itertuples():
166         lines.append(f"|{row.arm}/{row.reference}|{row.available}|{row.passed}|{row.median_gain:.1%}|")
167     lines.extend(["", "上一轮保守谱的8/11和19.9%使用旧规则，不能直接当成本轮统计提升率；这里按新下限和同一新审计重检。",
168         "","![接近度和一致性网格](../outputs/q3_refined/figures/proximity_consistency_grid.png)","",
169         "完整72个网格点见 `../outputs/q3_refined/transfer/frontier.csv`，792条策略记录、选谱、逐情景浓度及配对差均保留。"
170         "这是有限候选的经验权衡，不证明连续空间的全局Pareto最优，也没有真实品评覆盖率。","",
171         "## 复现与边界","",
172         "Pils/81目标已经多轮查看，属于开发诊断。尾部候选的点改善、嵌套内部改善和模型检查通过均不能替代新批次外测。"
173         "全部目标仍为附件标准化尺度，未恢复原0–6分。",
174         "诊断详见 `q3_beer81_audit.md`，预测方法见 `q3_stale_refined.md`，独立审查见 `q3_
```

src/24_q3_refinement_report.py（续）

```
    refinement_review.md`。",
175         "","```bash",".venv/bin/python solution_a/run_q3_refined.py","```",""])
176     (SOLUTION/"docs/q3_refinement_results.md").write_text("\n".join(lines),encoding="u
    tf-8", newline="\n")
177     summary={"selected_method":selected,"selected_pils_rmse":float(selected_row.pils_r
    mse),
178         "tail_candidate_pils_rmse":float(tail.pils_rmse),"tail_candidate_eligible":boo
    l(tail.eligible_for_one_se),
179         "matched_old_pils_rmse":float(old.pils_rmse),"beer81_observed":audit["stale_ob
    served"],"beer81_selected_prediction":selected81,
180         "nested_rule_rmse":float(nested_values.loc["nested_stability_rule","rmse"]),
181         "default_results":transfer["default_results"],"arm_defaults":transfer["arm_def
    aults"],
182         "candidate_rows":len(comparison),"frontier_rows":len(frontier),"source_denomin
    ator":11,"figure_pairs":2}
183     (OUT/"report_summary.json").write_text(json.dumps(summary,ensure_ascii=False,inden
    t=2)+"\n",encoding="utf-8", newline="\n")
184     (OUT/"report_sources.json").write_text(json.dumps(SOURCES,ensure_ascii=False,inden
    t=2)+"\n",encoding="utf-8", newline="\n")
185     print(json.dumps(summary,ensure_ascii=False,indent=2),flush=True)
186
187
188 if __name__=="__main__":main()
```

src/25_q3_refined_verify.py

SHA-256: 7db48f83f62efdf7ed8240cbff3404c1788d88a3f18a578b64f1a4b3fd1f9491

```
"""Recompute source diagnosis, grouped proxy selection and balanced transfer."""
from __future__ import annotations

import hashlib
import importlib
import json
from decimal import Decimal
from pathlib import Path
from time import perf_counter

import joblib
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist
from sklearn.impute import SimpleImputer
from sklearn.linear_model import Ridge
from sklearn.model_selection import GroupKFold
from sklearn.preprocessing import StandardScaler
from threadpoolctl import threadpool_limits

from common import SOLUTION, data_dir, resolve_recorded_path

BASE = importlib.import_module("10_q3_verify")
DOMAIN = importlib.import_module("19_q3_domain_verify")
OUT = SOLUTION / "outputs/q3_refined"
TOL = 1e-10


def check_frozen():
    details = DOMAIN.check_frozen()
    folder = SOLUTION / "outputs/q3_domain"
    manifest = BASE.read_json(OUT / "previous_hashes.json")
    actual = {p.relative_to(folder).as_posix() for p in folder.rglob("*") if p.is_file()}
    assert actual == {r["path"] for r in manifest["files"]}
    for row in manifest["files"]:
        raw = (folder / row["path"]).read_bytes()
        assert len(raw) == row["bytes"] and hashlib.sha256(raw).hexdigest() == row["sha256"], row["path"]
    details.append({"directory": "outputs/q3_domain", "unchanged_files": len(actual)})
    return details


def check_beer81(context):
    from q3_stale_optimized import FoldChemicalTransform
    folder = OUT / "beer81"
    meta, x, y45 = context["meta"], context["x"], context["y"]
    y = y45.stale_hops
    train = np.flatnonzero(~meta.tasting_category_fine.eq("Pils/Lager"))
    sources = np.flatnonzero(meta.tasting_category_fine.eq("Pils/Lager"))
    index = np.flatnonzero(meta.beer.eq(81)).item()
    summary = BASE.read_json(folder / "summary.json")
    for row in BASE.read_json(folder / "source_manifest.json"):
        assert hashlib.sha256(resolve_recorded_path(row["path"]).read_bytes()).hexdigest() == row["sha256"]
    identity = pd.read_csv(folder / "identity_join_audit.csv")
    assert len(identity) == 4 and identity.beer81_matches.eq(1).all()
    assert identity.beer_unique.all() and identity.style_matches_1csv.all() and identity.id_matches_printed_precision.all()
```

src/25_q3_refined_verify.py（续）

```
    assert identity.rows_after_exact_beer_join.eq(250).all()
    originals = {f:pd.read_csv(data_dir()/f,dtype={"beer_id":str}) for f in ["1.csv","4.csv","7.csv"]}
    originals["8.xlsx/calibrated"] = pd.read_excel(data_dir()/"8.xlsx",sheet_name="Calibrated chemical dataset",dtype={"beer_id":str})
    reference_id = Decimal(originals["1.csv"].set_index("beer").at[81,"beer_id"])
    for name, frame in originals.items():
        assert frame.beer.is_unique and set(frame.beer)==set(meta.beer)
        row = frame.set_index("beer").loc[81]
        assert row.tasting_category_fine=="Pils/Lager"
        printed = Decimal(row.beer_id)
        resolution = Decimal(10)**printed.as_tuple().exponent
        assert abs(reference_id-printed)<=resolution/2
        actual = identity.set_index("source").loc[name]
        assert actual.beer_id_unique_count==frame.beer_id.nunique()
        assert bool(actual.exact_id_matches_1csv)==(reference_id==printed)
    names = pd.read_excel(data_dir()/"8.xlsx", sheet_name="Sensory names")
    saved_names = pd.read_csv(folder/"sensory_target_metadata.csv")
    assert set(saved_names.R_name) == {"stale_hops", "barnyard", "t2n", "dms"}
    assert names.set_index("R_name").loc[["stale_hops", "barnyard"], "DataType"].eq("numeric").all()
    profile = pd.read_csv(folder/"chemical_profile_beer81.csv").set_index("feature").loc[x.columns]
    BASE.close(profile.value, x.iloc[index], "Beer81 hybrid measurements")
    BASE.close(profile[["nonpils_min","nonpils_q01","nonpils_median","nonpils_q99","nonpils_max"]],
               np.column_stack([x.iloc[train].min(),x.iloc[train].quantile(.01),x.iloc[train].median(),x.iloc[train].quantile(.99),x.iloc[train].max()]))
    outside = (x.iloc[index] < x.iloc[train].min())|(x.iloc[index] > x.iloc[train].max())
    np.testing.assert_array_equal(profile.outside_nonpils_range,outside)
    assert summary["sample_profile"]["outside_nonpils_feature_ranges"]==outside.sum()
    assert not x.iloc[index].isna().any() and x.iloc[index][context["calibrated_features"]].ge(0).all()
    assert summary["label_error_established"] is False and summary["analytical_lod_loq_available"] is False
    BASE.close(summary["stale_observed"], y.iloc[index])
    assert summary["stale_descending_rank_all250"] == int(y.rank(ascending=False, method="min").iloc[index])
    high = train[y.iloc[train].to_numpy() >= y.iloc[index]]
    assert summary["nonpils_stale_at_least_beer81"] == len(high)
    assert set(summary["nonpils_high_tail_beers"]) == set(meta.beer.iloc[high])
    raw = x.to_numpy(float).copy()
    for j, field in enumerate(x):
        if field in context["calibrated_features"]:
            positive = raw[train, j]
            positive = positive[np.isfinite(positive) & (positive > 0)]
            raw[:, j] = np.log1p(raw[:, j] / (np.median(positive) if len(positive) else 1.))
    imp = SimpleImputer(strategy="median", keep_empty_features=True).fit(raw[train])
    filled = imp.transform(raw)
    standard = StandardScaler().fit(filled[train]).transform(filled)
    aging = BASE.read_json(OUT/"stale/run_manifest.json")["aging_features"]
    positions = [x.columns.get_loc(c) for c in aging]
    spaces = {"unclipped_log_scaled_226": standard, "unclipped_log_scaled_aging24": standard[:, positions]}
    for name, subset in [("clipped_log_scaled_226", None), ("clipped_log_scaled_aging24", aging)]:
        z = FoldChemicalTransform(x.columns.tolist(), context["calibrated_features"], subset).fit(x.iloc[train]).transform(x)
        imputer = SimpleImputer(strategy="median", keep_empty_features=True).fit(z[tra
```

src/25_q3_refined_verify.py（续）

```
in])
        z = imputer.transform(z)
        spaces[name] = StandardScaler().fit(z[train]).transform(z)
    reference = joblib.load(SOLUTION/"outputs/q3/transfer/transfer_copula.joblib")["co
pula"]
    spaces["old_copula_226"] = reference.transform(x)
    spaces["old_copula_aging24"] = spaces["old_copula_226"][:, positions]
    local = joblib.load(SOLUTION/"outputs/q3_domain/stale/transfer_stale_model.joblib"
)["pipeline"]
    spaces["frozen_domain_aging_pca8"] = local[:-1].transform(x)
    saved = pd.read_csv(folder/"chemical_ood_all_pils.csv").set_index(["space", "query
_beer"])
    neighbors = pd.read_csv(folder/"nearest_training_neighbors.csv")
    full81 = pd.read_csv(folder/"beer81_all_training_neighbor_ranks.csv")
    assert len(saved) == 7*len(sources) and len(neighbors) == 7*len(sources)*25 and le
n(full81) == 7*len(train)
    for name, z in spaces.items():
        training_d = cdist(z[train], z[train])/np.sqrt(z.shape[1])
        np.fill_diagonal(training_d, np.inf)
        loo = training_d.min(axis=1)
        query_d = cdist(z[sources], z[train])/np.sqrt(z.shape[1])
        for query, distances in zip(sources, query_d):
            beer = int(meta.beer.iloc[query]); row = saved.loc[(name, beer)]
            BASE.close(row[["nearest_nonpils_distance", "nonpils_loo_nn_percentile", "
training_loo_nn_q95"]],
                       [distances.min(), np.mean(loo <= distances.min()), np.quantile(
loo, .95)])
            order = np.argsort(distances)
            part = neighbors[neighbors.space.eq(name)&neighbors.query_beer.eq(beer)].s
ort_values("neighbor_rank")
            np.testing.assert_array_equal(part.neighbor_beer, meta.beer.iloc[train[ord
er[:25]]])
            BASE.close(part.distance, distances[order[:25]])
            BASE.close(part.neighbor_stale, y.iloc[train[order[:25]]])
            if beer == 81:
                part = full81[full81.space.eq(name)].sort_values("neighbor_rank")
                np.testing.assert_array_equal(part.neighbor_beer, meta.beer.iloc[train
[order]])
                BASE.close(part.distance, distances[order])
    local25 = neighbors[neighbors.space.eq("frozen_domain_aging_pca8")&neighbors.query
_beer.eq(81)]
    assert summary["beer81_outside_local_top25_target_convex_hull"] == bool(y.iloc[ind
ex] > local25.neighbor_stale.max())
    errors = pd.read_csv(folder/"all_saved_pils_prediction_errors.csv")
    influence = pd.read_csv(folder/"beer81_sse_contribution.csv").set_index(["stage",
"method"])
    deletion = pd.read_csv(folder/"leave_one_source_out_influence.csv").set_index(["st
age", "method", "omitted_beer"])
    for keys, part in errors.groupby(["stage", "method"]):
        assert len(part) == len(sources) and set(part.beer) == set(meta.beer.iloc[sour
ces])
        BASE.close(part.observed, part.beer.map(pd.Series(y.to_numpy(), index=meta.bee
r)))
        residual = part.prediction.to_numpy()-part.observed.to_numpy()
        BASE.close(part.residual, residual); BASE.close(part.squared_error, residual**
2)
        BASE.close(part.sse_share, residual**2/np.sum(residual**2))
        r81 = part[part.beer.eq(81)].iloc[0]
        BASE.close(influence.loc[keys, ["full_rmse", "beer81_residual", "beer81_sse_sh
are"]],
                   [np.sqrt(np.mean(residual**2)), r81.residual, r81.sse_share])
```

src/25_q3_refined_verify.py（续）

```
        for row in part.itertuples():
            BASE.close(deletion.loc[(*keys, row.beer), "fixed_prediction_rmse_without_one_source"],
                       np.sqrt(part.loc[part.beer.ne(row.beer), "squared_error"].mean()))
    forum = pd.read_csv(data_dir()/"7.csv").set_index("beer")
    terms = pd.read_csv(folder/"forum_all_terms_beer81.csv")
    assert len(terms) == 186
    for row in terms.itertuples():
        BASE.close(row.beer81_value, forum.at[81, row.term])
        values = forum[row.term].to_numpy(float)
        values = values[np.isfinite(values)]
        value = forum.at[81, row.term]
        BASE.close(row.all_percentile, (np.sum(values < value)+.5*np.sum(values==value))/len(values) if pd.notna(value) else np.nan)
    return {"source": 81, "chemical_features": len(profile), "ood_spaces": len(spaces),
            "retained_sources": len(sources), "stale_rank": summary["stale_descending_rank_all250"],
            "high_tail_training": len(high), "label_error_established": False,
            "fixed_prediction_influence_groups": len(influence)}


def check_fixed_transform(transform, training, calibrated):
    selected = transform.selected_features_
    values = training[selected].to_numpy(float).copy()
    missing = np.isnan(values)
    scales = np.ones(len(selected))
    for j, field in enumerate(selected):
        if field in calibrated:
            positive = values[np.isfinite(values[:, j])&(values[:, j] > 0), j]
            scales[j] = np.median(positive) if len(positive) else 1.
            values[:, j] = np.log1p(values[:, j]/scales[j])
    bounds = np.nanquantile(values, [.01, .99], axis=0)
    values = np.clip(values, *bounds)
    medians = np.nan_to_num(np.nanmedian(values, axis=0), nan=0.)
    filled = np.where(np.isnan(values), medians, values)
    means, std = filled.mean(axis=0), filled.std(axis=0)
    std[std < 1e-12] = 1.
    for actual, expected in [(transform.positive_scales_, scales), (transform.bounds_, bounds),
                             (transform.medians_, medians), (transform.means_, means), (transform.scales_, std)]:
        BASE.close(actual, expected, "Fold-local fixed-width chemical transform")
    assert transform.indicator_weight == .2 and transform.n_output_features_ == 2*len(selected)
    result = np.column_stack([(filled-means)/std, .2*missing])
    BASE.close(transform.transform(training), result, "Fixed unscaled missing flags")
    return result


def check_model_training(model, training, y45, styles, calibrated):
    assert model.n_features_in_ == 226 and model.n_outputs_ == 1
    if hasattr(model, "members_"):
        for member in model.members_:
            check_model_training(member, training, y45, styles, calibrated)
        return
    if hasattr(model, "chemical_"):
        z = check_fixed_transform(model.chemical_, training, calibrated)
        target = y45.stale_hops.to_numpy()
        BASE.close(model.tail_training_threshold_, np.quantile(target, .8))
```

src/25_q3_refined_verify.py（续）

```
        if hasattr(model, "auxiliary_target_indices_"):
            assert model.auxiliary_target_names_ == [c for c in y45 if c != "stale_hop
s"]
            assert len(model.auxiliary_target_indices_) == 44 and model.target_positio
n_ not in model.auxiliary_target_indices_
            w = DOMAIN.style_weights(styles)
            aux = y45[model.auxiliary_target_names_].to_numpy()
            BASE.close(model.auxiliary_mean_, np.average(aux, axis=0, weights=w))
        if model.config["family"] == "ridge":
            weights = DOMAIN.style_weights(styles)
            if model.config.get("tail", False):
                weights *= np.where(target >= np.quantile(target, .8), 2., 1.)
                weights *= len(weights)/weights.sum()
            fitted = Ridge(alpha=model.config["alpha"], solver="svd").fit(z, target, s
ample_weight=weights)
            BASE.close(model.model_.coef_, fitted.coef_, "Training-only weighted ridge
 coefficients")
            BASE.close(model.model_.intercept_, fitted.intercept_)


def select_from_tables(table, stability):
    baseline = table[table.method.eq("mean_global")].iloc[0]
    eligible = table.selectable & (table.prediction_variance_ratio >= .05)
    eligible &= (table.macro_style_rmse < baseline.macro_style_rmse)&(table.rmse < bas
eline.rmse)
    np.testing.assert_array_equal(eligible, table.eligible_for_one_se)
    candidates = table[eligible]
    fallback = candidates.empty
    best = (table[table.selectable] if fallback else candidates).sort_values(["selecti
on_score", "method"]).iloc[0]
    one_se = [best.method] if fallback else candidates.loc[candidates.selection_score
<= best.selection_score+best.style_score_se, "method"].tolist()
    assert set(table.loc[table.within_one_se, "method"]) == set(one_se) == set(stabili
ty.method)
    selected = stability.sort_values(["response_noise_ratio", "priority", "method"]).i
loc[0]
    qualified = bool(not fallback and selected.pair_delta_signal_rms > 1e-4 and select
ed.response_noise_ratio <= 1.)
    return selected.method, qualified, best


def check_stale(context):
    from q3_stale_refined import RefinedStaleModel, FixedChemicalTransform, refined_sp
ecs
    folder = OUT/"stale"
    meta, x, y45 = context["meta"], context["x"], context["y"]
    y = y45.stale_hops
    train = np.flatnonzero(~meta.tasting_category_fine.eq("Pils/Lager"))
    sources = np.flatnonzero(meta.tasting_category_fine.eq("Pils/Lager"))
    manifest, decision, summary = [BASE.read_json(folder/f) for f in ("run_manifest.js
on", "selection_result.json", "summary.json")]
    specs = {s["name"]: s for s in manifest["candidate_specs"]}
    assert list(specs.values()) == refined_specs() and len(specs) == 12
    assert manifest["pils_labels_used_for_selection_or_intercept"] is False and manife
st["no_beer81_exclusion"]
    assert manifest["features"] == x.columns.tolist()
    assert manifest["auxiliary_target_names"] == [c for c in y45 if c != "stale_hops"]
    np.testing.assert_array_equal(manifest["training_indices"], train)
    np.testing.assert_array_equal(manifest["excluded_indices"], sources)
    def fit(spec, rows, seed):
        return RefinedStaleModel(x.columns.tolist(), context["calibrated_features"], m
```

```
    anifest["aging_features"],
247                                   y45.columns.tolist(), spec, seed).fit(x.iloc[rows], y
    45.iloc[rows], meta.tasting_category_fine.iloc[rows])
248     predictions = pd.read_csv(folder/"loso_predictions.csv")
249     audit = pd.read_csv(folder/"loso_split_audit.csv")
250     by_style = pd.read_csv(folder/"loso_metrics_by_style.csv")
251     overall = pd.read_csv(folder/"loso_metrics_overall.csv")
252     assert len(predictions) == len(train)*len(specs) and not predictions.duplicated(["
    method", "global_index"]).any()
253     calculated_styles = []
254     for fold, part in audit.groupby("fold"):
255         heldout = part.heldout_style.unique().item()
256         te = train[meta.tasting_category_fine.iloc[train].eq(heldout)]
257         tr = train[~meta.tasting_category_fine.iloc[train].eq(heldout)]
258         assert set(part.loc[part.role.eq("train"), "global_index"]) == set(tr)
259         assert set(part.loc[part.role.eq("test"), "global_index"]) == set(te)
260         assert len(part) == len(train) and not part.global_index.duplicated().any()
261         fitted = fit(decision["config"], tr, manifest["final_seed"]+int(fold))
262         check_model_training(fitted, x.iloc[tr], y45.iloc[tr], meta.tasting_category_f
    ine.iloc[tr], context["calibrated_features"])
263         for method in specs:
264             rows = predictions[predictions.method.eq(method)&predictions.fold.eq(fold)
    ].set_index("global_index").loc[te]
265             assert rows.heldout_style.eq(heldout).all()
266             BASE.close(rows.observed, y.iloc[te])
267             BASE.close(rows.tail_threshold_training_q90, np.quantile(y.iloc[tr], .9))
268             np.testing.assert_array_equal(rows.is_tail_eval, rows.observed >= rows.tai
    l_threshold_training_q90)
269             if method == decision["selected_method"]:
270                 BASE.close(rows.prediction, fitted.predict(x.iloc[te]), "Selected LOSO
     refit excluding held-out style")
271             if method == "mean_global": BASE.close(rows.prediction, y.iloc[tr].mean())
272             calculated_styles.append({"method": method, "heldout_style": heldout, "fol
    d": fold,
273                                       **DOMAIN.score(rows.observed, rows.prediction),
    "bias": (rows.prediction-rows.observed).mean()})
274     metrics = list(DOMAIN.score(y, y))
275     BASE.same_table(by_style, pd.DataFrame(calculated_styles), ["method", "heldout_sty
    le", "fold"], metrics+["bias"], "LOSO style statistics")
276     calculated = []
277     for method in specs:
278         rows = predictions[predictions.method.eq(method)].set_index("global_index").lo
    c[train]
279         bs = by_style[by_style.method.eq(method)]
280         losses = bs.rmse+.25*bs.bias.abs()
281         tail = rows.is_tail_eval
282         calculated.append({"method": method, **DOMAIN.score(rows.observed, rows.predic
    tion),
283                            "macro_style_rmse": bs.rmse.mean(), "macro_style_abs_bias":
     bs.bias.abs().mean(),
284                            "selection_score": losses.mean(), "style_score_se": losses.
    std(ddof=1)/np.sqrt(len(losses)),
285                            "tail_n": tail.sum(), "tail_rmse": np.sqrt(np.mean((rows.lo
    c[tail, "prediction"]-rows.loc[tail, "observed"])**2))})
286     BASE.same_table(overall, pd.DataFrame(calculated), ["method"], list(calculated[0])
    [1:], "LOSO pooled and selection statistics")
287     stability = pd.read_csv(folder/"selection_stability.csv")
288     selected, qualified, best = select_from_tables(overall, stability)
289     assert selected == decision["selected_method"] == summary["selected_method"] and q
    ualified == decision["qualified_for_proxy"]
290     BASE.close([decision["lowest_selection_score"], decision["one_se_width"], decision
```

```
    ["one_se_threshold"]],
291                 [best.selection_score, best.style_score_se, best.selection_score+best.s
    tyle_score_se])
292     pairs_transform = FixedChemicalTransform(x.columns.tolist(), context["calibrated_f
    eatures"], manifest["aging_features"]).fit(x.iloc[train])
293     z = check_fixed_transform(pairs_transform, x.iloc[train], context["calibrated_feat
    ures"])
294     d = cdist(z, z); np.fill_diagonal(d, np.inf)
295     pairs = train[np.argsort(d, axis=1)[:, 2]]
296     pair_table = pd.read_csv(folder/"selection_chemical_pairs.csv").set_index("global_
    index").loc[train]
297     np.testing.assert_array_equal(pair_table.paired_global_index, pairs)
298     for row in stability.itertuples():
299         delta = []
300         for b in range(manifest["stability_draws"]):
301             seed = manifest["stability_seed"]+b
302             resample = np.random.default_rng(seed).choice(train, len(train), replace=T
    rue)
303             fitted = fit(specs[row.method], resample, seed+10000)
304             delta.append(fitted.predict(x.iloc[train])-fitted.predict(x.iloc[pairs]))
305         delta = np.asarray(delta)
306         noise, signal = np.sqrt(np.mean(np.var(delta, axis=0, ddof=1))), np.sqrt(np.me
    an(delta.mean(axis=0)**2))
307         BASE.close([row.pair_delta_noise_rms, row.pair_delta_signal_rms, row.response_
    noise_ratio], [noise, signal, noise/max(signal, 1e-8)])
308     artifact = joblib.load(folder/"transfer_stale_model.joblib")
309     ensemble = joblib.load(folder/"bootstrap_ensemble.joblib")
310     for saved in [artifact, ensemble]:
311         np.testing.assert_array_equal(saved["train_global_indices"], train)
312         np.testing.assert_array_equal(saved["excluded_global_indices"], sources)
313         assert saved["pils_used_for_selection"] is False and saved["selected_config"]
    == specs[selected]
314         assert saved["qualified_for_proxy"] == qualified and saved["features"] == x.co
    lumns.tolist()
315     check_model_training(artifact["pipeline"], x.iloc[train], y45.iloc[train], meta.ta
    sting_category_fine.iloc[train], context["calibrated_features"])
316     indices = np.asarray(ensemble["bootstrap_indices"])
317     assert indices.shape == (60, len(train)) and not np.isin(indices, sources).any()
318     assert ensemble["selection_model_indices"] == list(range(40)) and ensemble["audit_
    model_indices"] == list(range(40,60))
319     with np.load(folder/"bootstrap_indices.npz") as saved:
320         np.testing.assert_array_equal(saved["indices"], indices)
321         np.testing.assert_array_equal(saved["bootstrap_seeds"], ensemble["bootstrap_se
    eds"])
322     for b, (model, rows, seed) in enumerate(zip(ensemble["models"], indices, ensemble[
    "bootstrap_seeds"])):
323         assert seed == (manifest["selection_bootstrap_seed"]+b if b < 40 else manifest
    ["audit_bootstrap_seed"]+b-40)
324         np.testing.assert_array_equal(rows, np.random.default_rng(seed).choice(train,
    len(train), replace=True))
325         assert model.seed == seed+10000
326         check_model_training(model, x.iloc[rows], y45.iloc[rows], meta.tasting_categor
    y_fine.iloc[rows], context["calibrated_features"])
327     archive = pd.read_csv(folder/"candidate_model_manifest.csv")
328     assert set(archive.method) == {name for name, spec in specs.items() if spec["selec
    table"]}
329     for row in archive.itertuples():
330         saved = joblib.load(folder/row.file)
331         np.testing.assert_array_equal(saved["train_global_indices"], train)
332         assert saved["pils_used_for_selection"] is False and saved["selected_config"]
    == specs[row.method]
```

```
        check_model_training(saved["pipeline"], x.iloc[train], y45.iloc[train], meta.tasting_category_fine.iloc[train], context["calibrated_features"])
    diagnostic = pd.read_csv(folder/"pils_diagnostic_predictions.csv")
    dm = pd.read_csv(folder/"pils_diagnostic_metrics.csv")
    assert len(diagnostic) == len(specs)*len(sources)
    for method, part in diagnostic.groupby("method"):
        part = part.set_index("global_index").loc[sources]
        BASE.close(part.observed, y.iloc[sources])
        BASE.close(dm.set_index("method").loc[method, metrics], list(DOMAIN.score(part.observed, part.prediction).values()))
        if method == selected: BASE.close(part.prediction, artifact["pipeline"].predict(x.iloc[sources]))
        if specs[method]["selectable"]:
            saved = joblib.load(folder/f"candidate_models/{method}.joblib")
            BASE.close(part.prediction, saved["pipeline"].predict(x.iloc[sources]), "All archived candidate Pils diagnostics")
            refitted = fit(specs[method], train, manifest["final_seed"])
            BASE.close(saved["pipeline"].predict(x), refitted.predict(x), "Candidate refit on non-Pils training only")
    comparisons = pd.read_csv(folder/"bootstrap_response_comparison.csv").set_index("method")
    old_models = joblib.load(SOLUTION/"outputs/q3_domain/stale/bootstrap_ensemble.joblib")["models"]
    for label, models in [("refined_selected", ensemble["models"]), ("domain_local_previous", old_models)]:
        delta = np.asarray([m.predict(x.iloc[train])-m.predict(x.iloc[pairs]) for m in models])
        noise, signal = np.sqrt(np.mean(np.var(delta, axis=0, ddof=1))), np.sqrt(np.mean(delta.mean(axis=0)**2))
        BASE.close(comparisons.loc[label, ["pair_delta_noise_rms", "pair_delta_signal_rms", "response_noise_ratio"]], [noise, signal, noise/max(signal, 1e-8)])
    nested = pd.read_csv(folder/"nested_predictions.csv")
    choices = pd.read_csv(folder/"nested_choices.csv").set_index("outer_fold")
    outer_audit = pd.read_csv(folder/"nested_outer_split_audit.csv")
    inner_audit = pd.read_csv(folder/"nested_inner_split_audit.csv")
    inner_metrics = pd.read_csv(folder/"nested_inner_metrics.csv")
    inner_stability = pd.read_csv(folder/"nested_selection_stability.csv")
    assert len(nested) == len(train)*4 and not nested.duplicated(["method", "global_index"]).any()
    groups = meta.tasting_category_fine.iloc[train].to_numpy()
    for fold, (tr, te) in enumerate(GroupKFold(4, shuffle=True, random_state=manifest["nested_seed"]).split(train, groups=groups), 1):
        tr, te = train[tr], train[te]
        for role, rows in [("train", tr), ("test", te)]:
            actual = outer_audit[outer_audit.outer_fold.eq(fold)&outer_audit.role.eq(role)]
            np.testing.assert_array_equal(actual.global_index, rows)
        assert not set(meta.tasting_category_fine.iloc[tr])&set(meta.tasting_category_fine.iloc[te])
        for _, part in inner_audit[inner_audit.outer_fold.eq(fold)].groupby("fold"):
            style = part.heldout_style.unique().item()
            assert set(part.global_index) == set(tr)
            assert set(part.loc[part.role.eq("test"), "global_index"]) == set(tr[meta.tasting_category_fine.iloc[tr].eq(style)])
            assert not set(part.global_index)&set(te)
        method, ok, _ = select_from_tables(inner_metrics[inner_metrics.outer_fold.eq(fold)], inner_stability[inner_stability.outer_fold.eq(fold)])
        assert method == choices.at[fold, "selected_method"] and ok == choices.at[fold, "qualified_for_proxy"]
        rows = nested[nested.outer_fold.eq(fold)&nested.method.eq("nested_stability_rule")].set_index("global_index").loc[te]
```

src/25_q3_refined_verify.py（续）

```
        fitted = fit(specs[method], tr, manifest["nested_seed"]+fold)
        BASE.close(rows.prediction, fitted.predict(x.iloc[te]), "Outer grouped selected predictor")
        BASE.close(rows.observed, y.iloc[te]); BASE.close(rows.tail_threshold_training_q90, np.quantile(y.iloc[tr], .9))
    nm = pd.read_csv(folder/"nested_metrics.csv").set_index("method")
    for method, rows in nested.groupby("method"):
        BASE.close(nm.loc[method, metrics], list(DOMAIN.score(rows.observed, rows.prediction).values()))
    context.update(refined_artifact=artifact, refined_ensemble=ensemble)
    return {"candidate_models": len(specs), "archived_selectable_models": len(archive), "loso_styles": len(audit.fold.unique()),
            "loso_predictions": len(predictions), "nested_outer_folds": len(choices), "bootstrap_models": len(indices),
            "source_excluded": True, "selected_method": selected, "qualified_internal": qualified,
            "pils_rmse": float(dm.set_index("method").at[selected, "rmse"]),
            "pooled_loso_rmse": float(overall.set_index("method").at[selected, "rmse"]),
            "nested_selected_rmse": float(nm.at["nested_stability_rule", "rmse"])}


def paired(models, candidate, source):
    return np.column_stack([np.asarray(m.predict(candidate)).reshape(-1)-np.asarray(m.predict(source)).reshape(-1)[0] for m in models])


def check_transfer(context):
    from q3_data import read_defect_thresholds
    folder = OUT/"transfer"
    meta, x, y45 = context["meta"], context["x"], context["y"]
    features = x.columns.tolist(); dim = len(features)
    train = np.flatnonzero(~meta.tasting_category_fine.eq("Pils/Lager"))
    sources = np.flatnonzero(meta.tasting_category_fine.eq("Pils/Lager"))
    target = np.flatnonzero(meta.tasting_category_fine.eq("Tripel"))
    manifest, summary = [BASE.read_json(folder/f) for f in ("run_manifest.json", "summary.json")]
    default = {"arm":"refined", "pool":"contracted_6004", "bound":"retain80", "ranking":"margin"}
    assert manifest["default"] == summary["default"] == default
    assert manifest["default_chosen_from_audit"] is False and manifest["audit_reselection"] is False
    bounds = {"absolute10":(.1,None), "absolute30":(.3,None), "absolute50":(.5,None),
              "absolute60":(.6,None), "retain80":(.1,.8), "retain100":(.1,1.)}
    assert manifest["bounds"] == {k:{"absolute_minimum":v[0],"anchor_fraction":v[1]} for k,v in bounds.items()}
    assert manifest["pools"] == {"legacy_2004":2004,"legacy_6004":6004,"contracted_6004":6004}
    geometry = joblib.load(SOLUTION/"outputs/q3/transfer/transfer_copula.joblib")
    np.testing.assert_array_equal(geometry["training_rows"], train)
    reference = geometry["copula"]
    tz = reference.transform(x.iloc[target]); center = tz.mean(axis=0)
    d = cdist(tz,tz)/np.sqrt(dim); np.fill_diagonal(d,np.inf)
    limit = np.quantile(d.min(axis=1),.95)
    BASE.close(summary["support_limit"],limit)
    legacy = pd.read_csv(folder/"legacy_base_draws.csv")
    BASE.close(legacy,pd.read_csv(SOLUTION/"outputs/q3_domain/consensus/base_draws.csv"))
    assert legacy.shape == (1500,226)
    base = reference.transform(legacy.iloc[:500])
    expected_latent = np.vstack([reference.style_center_["Tripel"]+c*(base-reference.s
```

src/25_q3_refined_verify.py（续）

```
    tyle_center_["Tripel"]) for c in [.2,.4,.6]])
422     BASE.close(np.load(folder/"contracted_latents.npy"),expected_latent)
423     contracted = pd.read_csv(folder/"contracted_base_draws.csv")
424     BASE.close(contracted,reference.inverse_transform(expected_latent),"Contracted lat
    ent inverse")
425     drawmeta = pd.read_csv(folder/"contracted_draw_metadata.csv")
426     BASE.close(drawmeta, np.column_stack([np.arange(1500),np.tile(np.arange(500),3),np
    .repeat([.2,.4,.6],500)]))
427     assert summary["contracted_targets_unique"] == len(np.unique(contracted,axis=0))
428     scenarios = BASE.read_json(SOLUTION/"outputs/q3_optimized/transfer/threshold_scena
    rios.json")
429     assert manifest["threshold_scenarios"] == scenarios and len(scenarios) == 10
430     rules = read_defect_thresholds()
431     original = joblib.load(SOLUTION/"outputs/q3/sensory/stale_bootstrap.joblib")
432     center_old = joblib.load(SOLUTION/"outputs/q3/sensory/transfer_proxy.joblib")["pip
    eline"]
433     fixed = joblib.load(SOLUTION/"outputs/q3_optimized/stale_fixed/transfer_stale_mode
    l.joblib")
434     fixed_ensemble = joblib.load(SOLUTION/"outputs/q3_optimized/stale_fixed/bootstrap_
    ensemble.joblib")
435     fresh = {}
436     for label,filename,seed in [("old","fresh_original_audit.joblib",202104123),("fixe
    d","fresh_fixed_audit.joblib",202105123)]:
437         saved = joblib.load(folder/filename)
438         np.testing.assert_array_equal(saved["train_global_indices"],train)
439         np.testing.assert_array_equal(saved["excluded_global_indices"],sources)
440         assert saved["features"] == features and len(saved["models"]) == 20
441         for b,(model,rows,current) in enumerate(zip(saved["models"],saved["bootstrap_i
    ndices"],saved["bootstrap_seeds"])):
442             assert current == seed+b
443             np.testing.assert_array_equal(rows,np.random.default_rng(current).choice(t
    rain,len(train),replace=True))
444             BASE.close(model.named_steps["imputer"].statistics_,x.iloc[rows].median())
445             tree = model.named_steps["model"]
446             assert tree.n_outputs_ == 1 and tree.n_estimators == 120 and tree.min_samp
    les_leaf == 3 and tree.max_features == .7
447             assert tree.random_state == current+10000
448         fresh[label] = saved["models"]
449     assert center_old.named_steps["model"].n_outputs_ == 45
450     assert all(m.named_steps["model"].n_outputs_ == 1 for m in original["models"])
451     arms = {"refined":(context["refined_artifact"],context["refined_ensemble"]["models
    "][:40],context["refined_ensemble"]["models"][40:]),
452             "fixed":(fixed,fixed_ensemble["models"][:40],fresh["fixed"])}
453     result = pd.read_csv(folder/"transfer_summary.csv")
454     geom = pd.read_csv(folder/"candidate_geometry.csv.gz")
455     model_checks = pd.read_csv(folder/"candidate_model_checks.csv.gz")
456     spectra = pd.read_csv(folder/"selected_spectra.csv").set_index("transfer_id")
457     deltas = pd.read_csv(folder/"paired_deltas.csv")
458     thresholds = pd.read_csv(folder/"threshold_checks.csv")
459     rechecks = pd.read_csv(folder/"fixed_spectra_rechecks.csv")
460     assert rechecks.evaluation_pool.eq(default["pool"]).all()
461     uniqueness = pd.read_csv(folder/"pool_uniqueness.csv").set_index(["source_beer","p
    ool"])
462     anchors = pd.read_csv(SOLUTION/"outputs/q3_optimized/transfer/transfer_summary.csv
    ")
463     anchors = anchors[anchors.is_default_policy].set_index("source_beer")
464     anchor_spectra = pd.read_csv(SOLUTION/"outputs/q3_optimized/transfer/transferred_s
    pectra.csv").set_index("transfer_id")
465     prior = pd.read_csv(SOLUTION/"outputs/q3_domain/consensus/transfer_summary.csv")
466     prior_spectra = pd.read_csv(SOLUTION/"outputs/q3_domain/consensus/transferred_spec
    tra.csv").set_index("transfer_id")
```

```
    assert len(result) == len(arms)*3*6*2*len(sources) and not result.transfer_id.dupl
icated().any()
    assert len(geom) == 12004*len(sources) and len(model_checks) == len(arms)*len(geom
)
    expected_deltas, expected_thresholds, expected_rechecks = [],[],[]
    medians = x.iloc[train].median(); target_median = x.iloc[target].median().to_numpy
()
    minimum,maximum = x.iloc[train].min().to_numpy(),x.iloc[train].max().to_numpy()
    stale_position = y45.columns.get_loc("stale_hops")
    for index in sources:
        beer = int(meta.beer.iloc[index]); raw = x.iloc[[index]]; source = raw.fillna(
medians)
        descriptions,blocks = [],[]
        for family,draws in [("legacy",legacy),("contracted",contracted)]:
            for gamma in [.25,.5,.75,1.]:
                blocks.append((1-gamma)*source.to_numpy()[0]+gamma*draws.to_numpy())
                descriptions.extend((family,gamma,j) for j in range(len(draws)))
        for gamma in [.25,.5,.75,1.]:
            blocks.append(((1-gamma)*source.to_numpy()[0]+gamma*target_median)[None])
            descriptions.append(("median",gamma,-1))
        candidates = pd.DataFrame(np.vstack(blocks),columns=features)
        desc = pd.DataFrame(descriptions,columns=["family","gamma","draw"])
        chemical = np.ones(len(candidates),bool)
        for rule in rules.values():
            field = rule["feature"]; c0 = raw[field].iloc[0]
            assert np.isfinite(c0) and c0 >= 0
            finite_t = [s["thresholds"][field] for s in scenarios if s["thresholds"][f
ield] is not None]
            cap = max(c0,min(finite_t)) if finite_t else c0
            candidates[field] = candidates[field].clip(lower=0,upper=cap)
            for scenario in scenarios:
                threshold = scenario["thresholds"][field]
                chemical &= candidates[field].to_numpy() <= (max(c0,threshold) if thre
shold is not None else c0)+TOL
        assert np.isfinite(candidates.to_numpy()).all() and candidates[context["calibr
ated_features"]].ge(0).all().all()
        masks = {"legacy_2004":desc.family.eq("median")|(desc.family.eq("legacy")&desc
.draw.lt(500)),
                 "legacy_6004":desc.family.isin(["legacy","median"]),"contracted_6004"
:desc.family.isin(["contracted","median"])}
        z,sz = reference.transform(candidates),reference.transform(source)
        distance = reference.distance(candidates,"Tripel"); before = reference.distanc
e(source,"Tripel")[0]
        improvement = 1-distance/max(before,1e-8)
        change = np.linalg.norm(z-sz,axis=1)/np.sqrt(dim)
        nn = cdist(z,tz).min(axis=1)/np.sqrt(dim)
        support = (nn<=limit+TOL)&((candidates.to_numpy()>=minimum-TOL)&(candidates.to
_numpy()<=maximum+TOL)).all(axis=1)
        objective = distance/max(before,1e-8)+.2*change/max(np.linalg.norm(sz[0]-cente
r)/np.sqrt(dim),1e-8)
        geometric = support&chemical&(change>=.05)
        actual_geom = geom[geom.source_beer.eq(beer)].sort_values("candidate_index")
        BASE.close(actual_geom[["target_distance","target_improvement","change_latent_
rms","target_nn_rms","support_ok","chemical_ok","geometry_objective"]],
                   np.column_stack([distance,improvement,change,nn,support,chemical,ob
jective]),"All post-projection fixed geometry")
        assert actual_geom.family.tolist() == desc.family.tolist()
        BASE.close(actual_geom[["gamma","draw"]],desc[["gamma","draw"]])
        anchor_frame = anchor_spectra.loc[[anchors.at[beer,"transfer_id"]],features]
        anchor_gain = 1-reference.distance(anchor_frame,"Tripel")[0]/max(before,1e-8)
        unique,inverse = np.unique(candidates.to_numpy(),axis=0,return_inverse=True)
```

```
        unique = pd.DataFrame(unique,columns=features)
        old_delta = paired(original["models"][:30],unique,source)[inverse]
        old_q = np.quantile(old_delta,.95,axis=1); old_signal = np.any(np.abs(old_delt
a)>1e-8,axis=1)
        old_center_source = center_old.predict(source)[0,stale_position]
        old_center_delta = center_old.predict(unique)[:,stale_position][inverse]-old_c
enter_source
        for pool,mask in masks.items():
            actual_unique = len(np.unique(candidates.to_numpy()[mask],axis=0)); nomina
l = int(mask.sum())
            BASE.close(uniqueness.loc[(beer,pool),["nominal_candidates","unique_candid
ates","duplicate_fraction","anchor_gain"]],
                       [nominal,actual_unique,1-actual_unique/nominal,anchor_gain])
        for arm,(artifact,selection,audit) in arms.items():
            new_delta = paired(selection,unique,source)[inverse]
            new_q = np.quantile(new_delta,.95,axis=1); new_signal = np.any(np.abs(new_
delta)>1e-8,axis=1)
            current = artifact["pipeline"]
            center_source = np.asarray(current.predict(source)).reshape(-1)[0]
            center_delta = np.asarray(current.predict(unique)).reshape(-1)[inverse]-ce
nter_source
            worst = np.maximum(new_q,old_q)
            joint = geometric&(new_q<=TOL)&(old_q<=TOL)&new_signal&old_signal&(center_
delta<=TOL)&(old_center_delta<=TOL)
            checks = model_checks[model_checks.source_beer.eq(beer)&model_checks.arm.e
q(arm)].sort_values("candidate_index")
            BASE.close(checks[["new_selection_q95","old_selection_q95","worst_selectio
n_q95","center_new_delta","center_old_delta","new_signal","old_signal","joint_feasible
"]],
                       np.column_stack([new_q,old_q,worst,center_delta,old_center_delt
a,new_signal,old_signal,joint]),"Complete selection diagnostics without audit")
            qualified = artifact["qualified_for_proxy"]
            rows = result[result.source_beer.eq(beer)&result.arm.eq(arm)]
            cache = {}
            def audit_frame(frame):
                key = frame.to_numpy().tobytes()
                if key not in cache:
                    cache[key] = {"new_audit":paired(audit,frame,source)[0],"old_fresh
_audit":paired(fresh["old"],frame,source)[0],
                                  "old_historical":paired(original["models"][30:],fram
e,source)[0]}
                return cache[key]
            for row in rows.itertuples():
                absolute,retention = bounds[row.bound]
                floor = max(absolute,retention*anchor_gain) if retention is not None e
lse absolute
                mask = masks[row.pool].to_numpy()
                nsd,osd = np.std(new_delta[mask].mean(axis=1)),np.std(old_delta[mask].
mean(axis=1))
                informative = bool(nsd>1e-6 and osd>1e-6)
                ids = np.flatnonzero(joint&mask&(improvement>=floor))
                BASE.close([row.anchor_gain,row.required_gain,row.candidate_count,row.
feasible_count,row.new_delta_sd,row.old_delta_sd],
                           [anchor_gain,floor,mask.sum(),len(ids),nsd,osd])
                assert row.qualified == qualified and row.informative == informative
                arm_default = row.pool==default["pool"] and row.bound==default["bound"
] and row.ranking==default["ranking"]
                assert row.is_arm_default == arm_default and row.is_default_policy ==
(arm==default["arm"] and arm_default)
                if not len(ids):
                    assert pd.isna(row.candidate_index) and row.status=="no_candidate"
```

src/25_q3_refined_verify.py（续）

```
     and not row.fresh_consistency_passed
557                      continue
558                  chosen = ids[np.lexsort((objective[ids],worst[ids]))[0]] if row.rankin
    g=="margin" else ids[np.argmin(objective[ids])]
559                  assert int(row.candidate_index)==chosen, "Frozen eligible argmin canno
    t use audit predictions"
560                  frame = candidates.iloc[[chosen]]
561                  BASE.close(spectra.loc[row.transfer_id,features],frame.iloc[0],"Select
    ed spectrum reconstruction")
562                  assessed = audit_frame(frame); aq = {k:np.quantile(v,.95) for k,v in a
    ssessed.items()}
563                  newpass = bool(qualified and informative and aq["new_audit"]<=TOL)
564                  freshpass = bool(newpass and aq["old_fresh_audit"]<=TOL)
565                  historypass = bool(newpass and aq["old_historical"]<=TOL)
566                  status = "unqualified" if not qualified else "uninformative" if not in
    formative else "new_audit_failed" if not newpass else "old_fresh_audit_failed" if not
    freshpass else "passed"
567                  assert row.status==status and row.new_model_passed==newpass and row.fr
    esh_consistency_passed==freshpass and row.historical_consistency_passed==historypass
568                  BASE.close([row.target_distance_before,row.target_distance_after,row.t
    arget_improvement,row.retained_anchor_gain,row.change_latent_rms,
569                              row.target_nn_rms,row.support_nn_limit,row.geometry_object
    ive,row.new_selection_q95,row.old_selection_q95,row.worst_selection_q95,
570                              row.center_new_delta,row.center_old_delta,row.new_audit_q9
    5,row.old_fresh_audit_q95,row.old_historical_q95],
571                             [before,distance[chosen],improvement[chosen],improvement[ch
    osen]/anchor_gain,change[chosen],nn[chosen],limit,objective[chosen],
572                              new_q[chosen],old_q[chosen],worst[chosen],center_delta[cho
    sen],old_center_delta[chosen],aq["new_audit"],aq["old_fresh_audit"],aq["old_historical
    "]])
573                  for label,delta in {"new_selection":new_delta[chosen],"old_selection":
    old_delta[chosen],**assessed}.items():
574                      expected_deltas.extend({"transfer_id":row.transfer_id,"ensemble":l
    abel,"model_index":i,"delta":value} for i,value in enumerate(delta))
575                  for scenario in scenarios:
576                      for label,rule in rules.items():
577                          field = rule["feature"]; threshold = scenario["thresholds"][fi
    eld]; c0 = raw[field].iloc[0]
578                          cap = max(c0,threshold) if threshold is not None else c0
579                          expected_thresholds.append({"transfer_id":row.transfer_id,"sce
    nario":scenario["scenario"],"defect":label,
580                              "source":c0,"transferred":frame[field].iloc[0],"cap":cap,"
    passed":frame[field].iloc[0]<=cap+TOL})
581              references = [("original_anchor",anchor_frame)]
582              for pool in ("matched_2004","expanded_6004"):
583                  pr = prior[prior.source_beer.eq(beer)&prior.pool.eq(pool)&prior.policy
    .eq("joint_margin")].iloc[0]
584                  if pd.notna(pr.candidate_index): references.append((f"prior_{pool}",pr
    ior_spectra.loc[[pr.transfer_id],features]))
585              evaluation = masks[default["pool"]].to_numpy()
586              informative = np.std(new_delta[evaluation].mean(axis=1))>1e-6 and np.std(o
    ld_delta[evaluation].mean(axis=1))>1e-6
587              for label,frame in references:
588                  pz = reference.transform(frame); gain = 1-reference.distance(frame,"Tr
    ipel")[0]/max(before,1e-8)
589                  shift = np.linalg.norm(pz-sz)/np.sqrt(dim)
590                  supported = bool(cdist(pz,tz).min()/np.sqrt(dim)<=limit+TOL and ((fram
    e.to_numpy()>=minimum-TOL)&(frame.to_numpy()<=maximum+TOL)).all())
591                  chem = all(frame[r["feature"]].iloc[0] <= (max(raw[r["feature"]].iloc[
    0],s["thresholds"][r["feature"]]) if s["thresholds"][r["feature"]] is not None else ra
    w[r["feature"]].iloc[0])+TOL for s in scenarios for r in rules.values())
```

src/25_q3_refined_verify.py（续）

```
                nd,od = paired(selection,frame,source)[0],paired(original["models"][:30],frame,source)[0]
                nq,oq = np.quantile(nd,.95),np.quantile(od,.95)
                cdn = np.asarray(current.predict(frame)).reshape(-1)[0]-center_source
                cdo = center_old.predict(frame)[0,stale_position]-old_center_source
                assessed = audit_frame(frame); aq = {k:np.quantile(v,.95) for k,v in assessed.items()}
                basepass = bool(qualified and informative and supported and chem and shift>=.05 and nq<=TOL and oq<=TOL and cdn<=TOL and cdo<=TOL and np.any(np.abs(nd)>1e-8) and np.any(np.abs(od)>1e-8))
                for bound,(absolute,retention) in bounds.items():
                    floor = max(absolute,retention*anchor_gain) if retention is not None else absolute
                    expected_rechecks.append({"source_beer":beer,"arm":arm,"reference":label,"bound":bound,"required_gain":floor,
                        "target_improvement":gain,"retained_anchor_gain":gain/anchor_gain,"change_latent_rms":shift,"support_ok":supported,"chemical_ok":chem,
                        "new_selection_q95":nq,"old_selection_q95":oq,"center_new_delta":cdn,"center_old_delta":cdo,"new_audit_q95":aq["new_audit"],
                        "old_fresh_audit_q95":aq["old_fresh_audit"],"fresh_consistency_passed":bool(basepass and gain>=floor and aq["new_audit"]<=TOL and aq["old_fresh_audit"]<=TOL)})
        print(f"Verified transfer source beer={beer}",flush=True)
    BASE.same_table(deltas,pd.DataFrame(expected_deltas),["transfer_id","ensemble","model_index"],["delta"],"Every selected paired model delta")
    BASE.same_table(thresholds,pd.DataFrame(expected_thresholds),["transfer_id","scenario","defect"],["source","transferred","cap","passed"],"All selected threshold scenarios")
    keys = ["source_beer","arm","reference","bound"]
    BASE.same_table(rechecks,pd.DataFrame(expected_rechecks),keys,[k for k in expected_rechecks[0] if k not in keys],"Same-rule frozen-spectrum rechecks")
    assert len(spectra)==result.candidate_index.notna().sum()==summary["selected"]
    assert len(thresholds)==len(spectra)*len(scenarios)*len(rules)==summary["threshold_rows"]
    frontier = pd.read_csv(folder/"frontier.csv")
    expected_frontier = []
    def counts(part):
        selected = part.candidate_index.notna(); passing = part.fresh_consistency_passed
        return {"sources":len(part),"selected":selected.sum(),"new_passed":part.new_model_passed.sum(),"fresh_consistency_passed":passing.sum(),
            "historical_consistency_passed":part.historical_consistency_passed.sum(),"no_candidate":(~selected).sum(),
            "median_improvement_selected":part.loc[selected,"target_improvement"].median(),"median_improvement_passed":part.loc[passing,"target_improvement"].median(),
            "median_retention_selected":part.loc[selected,"retained_anchor_gain"].median(),"median_change_rms_selected":part.loc[selected,"change_latent_rms"].median()}
    for (arm,pool,bound,ranking),part in result.groupby(["arm","pool","bound","ranking"]):
        expected_frontier.append({"arm":arm,"pool":pool,"bound":bound,"ranking":ranking,**counts(part)})
    BASE.same_table(frontier,pd.DataFrame(expected_frontier),["arm","pool","bound","ranking"],list(counts(result)),"Complete trade-off frontier")
    assert len(frontier)==summary["frontier_rows"]==72 and len(result)==summary["policies"]
    for key,value in counts(result[result.is_default_policy]).items():
        BASE.close(summary["default_results"][key],value)
    for arm in arms:
        for key,value in counts(result[result.arm.eq(arm)&result.is_arm_default]).items(): BASE.close(summary["arm_defaults"][arm][key],value)
    return {"candidate_geometry_rows":len(geom),"candidate_screening_rows":len(model_c
```

src/25_q3_refined_verify.py（续）

```
hecks),"policies":len(result),
628             "selected_spectra":len(spectra),"threshold_checks":len(thresholds),"frontier_rows":len(frontier),
629             "same_rule_rechecks":len(rechecks),"default":default,"default_counts":summary["default_results"],
630             "fixed_control_counts":summary["arm_defaults"]["fixed"],"all_argmins_reconstructed_without_audit":True}
631
632
633 def check_reports(context):
634     import xml.etree.ElementTree as ET
635     from PIL import Image
636     report = SOLUTION/"docs/q3_refinement_results.md"
637     assert report.is_file() and report.stat().st_size > 500
638     text = report.read_text(encoding="utf-8")
639     provenance = BASE.read_json(OUT/"report_sources.json")
640     assert len(provenance)==13 and len({r["path"] for r in provenance})==len(provenance)
641     for row in provenance:
642         path = SOLUTION/row["path"]
643         assert hashlib.sha256(path.read_bytes()).hexdigest()==row["sha256"],row["path"]
644         if row["rows"] is not None: assert len(pd.read_csv(path))==row["rows"]
645     comparison = pd.read_csv(OUT/"report_tables/prediction_comparison.csv")
646     loso = pd.read_csv(OUT/"stale/loso_metrics_overall.csv")
647     pils = pd.read_csv(OUT/"stale/pils_diagnostic_metrics.csv")
648     expected = loso.merge(pils[["method","rmse","mean_bias"]].rename(columns={"rmse":"pils_rmse","mean_bias":"pils_bias"}),on="method",validate="one_to_one")
649     columns = expected.select_dtypes(include=[np.number,bool]).columns.tolist()
650     BASE.same_table(comparison,expected,["method"],columns,"Report prediction table")
651     transfer = pd.read_csv(OUT/"transfer/transfer_summary.csv")
652     retained = pd.read_csv(OUT/"report_tables/retain80_all_policies.csv")
653     expected = transfer[transfer.bound.eq("retain80")]
654     columns = expected.select_dtypes(include=[np.number,bool]).columns.tolist()
655     BASE.same_table(retained,expected,["transfer_id"],columns,"Report retained-gain policy table")
656     rechecks = pd.read_csv(OUT/"transfer/fixed_spectra_rechecks.csv")
657     expected = rechecks.groupby(["arm","reference","bound"],as_index=False).agg(available=("source_beer","size"),passed=("fresh_consistency_passed","sum"),median_gain=("target_improvement","median"))
658     expected["sources_denominator"]=len(context["meta"][context["meta"].tasting_category_fine.eq("Pils/Lager")])
659     BASE.same_table(pd.read_csv(OUT/"report_tables/reference_counts.csv"),expected,["arm","reference","bound"],
660                     ["available","passed","median_gain","sources_denominator"],"Report same-rule historical comparison")
661     summary = BASE.read_json(OUT/"report_summary.json")
662     selection = BASE.read_json(OUT/"stale/selection_result.json")
663     ts = BASE.read_json(OUT/"transfer/summary.json")
664     audit = BASE.read_json(OUT/"beer81/summary.json")
665     nested = pd.read_csv(OUT/"stale/nested_metrics.csv").set_index("method")
666     metric = pils.set_index("method")
667     assert summary["selected_method"]==selection["selected_method"]
668     for key,value in {"selected_pils_rmse":metric.at[selection["selected_method"],"rmse"],
669                       "tail_candidate_pils_rmse":metric.at["tail_ridge_aging100","rmse"],
670                       "matched_old_pils_rmse":metric.at["old_multioutput_tree","rmse"],
671                       "nested_rule_rmse":nested.at["nested_stability_rule","rmse"],
672                       "beer81_observed":audit["stale_observed"]}.items():
```

src/25_q3_refined_verify.py（续）

```
        BASE.close(summary[key],value)
        assert f"{value:.4f}" in text, f"Reported number missing: {key}"
    prediction = pd.read_csv(OUT/"stale/pils_diagnostic_predictions.csv")
    selected81 = prediction[prediction.method.eq(selection["selected_method"])&predict
ion.beer.eq(81)].prediction.item()
    BASE.close(summary["beer81_selected_prediction"],selected81)
    assert summary["arm_defaults"]==ts["arm_defaults"] and summary["default_results"]=
=ts["default_results"]
    assert summary["tail_candidate_eligible"]==bool(loso.set_index("method").at["tail_
ridge_aging100","eligible_for_one_se"])
    assert summary["candidate_rows"]==len(loso) and summary["frontier_rows"]==len(pd.r
ead_csv(OUT/"transfer/frontier.csv"))
    for row in comparison.itertuples():
        assert f"|{row.method}|{row.macro_style_rmse:.4f}|{row.rmse:.4f}|{row.pils_rms
e:.4f}|" in text
    frontier = pd.read_csv(OUT/"transfer/frontier.csv")
    for row in frontier[frontier.bound.eq("retain80")].itertuples():
        assert f"|{row.arm}|{row.pool}|{row.ranking}|{row.selected}|{row.fresh_consist
ency_passed}|" in text
    figures=[]
    for name in ["beer81_and_prediction","proximity_consistency_grid"]:
        png,svg = OUT/f"figures/{name}.png",OUT/f"figures/{name}.svg"
        with Image.open(png) as im:
            im.verify()
        with Image.open(png) as im:
            assert im.width>=1000 and im.height>=600
        root = ET.parse(svg).getroot()
        assert root.tag.endswith("svg") and len(list(root.iter()))>50
        assert f"figures/{name}.png" in text
        figures.append(name)
    assert summary["figure_pairs"]==len(figures) and summary["source_denominator"]==le
n(transfer[transfer.is_default_policy])
    return {"report":report.relative_to(SOLUTION).as_posix(),"source_hashes":len(prove
nance),"candidate_rows":len(comparison),
            "retention_policy_rows":len(retained),"reference_rows":len(expected),"figu
re_pairs":len(figures),
            "report_tables_and_text_reconciled":True}


def main(output_path=None):
    started = perf_counter()
    checks = {}
    checks["raw_sources"] = BASE.check_hashes()
    checks["frozen_history"] = check_frozen()
    context,checks["data_contract"] = BASE.load_contract()
    checks["beer81"] = check_beer81(context)
    checks["stale_prediction"] = check_stale(context)
    checks["balanced_transfer"] = check_transfer(context)
    checks["final_artifacts"] = check_reports(context)
    assert check_frozen() == checks["frozen_history"]
    result = {"status":"PASS","checks":checks,"elapsed_seconds":perf_counter()-started
,
              "scope":"Saved-data provenance, training-only internal validation, model
-response constraints and predeclared finite candidate search; no external sensory val
idation."}
    if output_path is not None:
        output_path = Path(output_path)
        output_path.parent.mkdir(parents=True, exist_ok=True)
        output_path.write_text(json.dumps(result,ensure_ascii=False,indent=2,allow_nan
=False)+"\n",encoding="utf-8", newline="\n")
    print(json.dumps(result,ensure_ascii=False,indent=2),flush=True)
```

src/25_q3_refined_verify.py（续）

```
721
722
723 if __name__ == "__main__":
724     import argparse
725     parser = argparse.ArgumentParser(description=__doc__)
726     parser.add_argument("--output", type=Path, help="Optional report path; existing re
    sults are read-only by default.")
727     args = parser.parse_args()
728     with threadpool_limits(limits=1): main(args.output)
```

src/common.py

SHA-256: b1b8ef459cfe3b9461efa0310ebe49e9593b655d8e58d2a02f32e7138fc5fa0f

```
"""Shared data contract for the A-track modelling experiments."""
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin

ROOT = Path(__file__).resolve().parents[2]
SOLUTION = ROOT / "solution_a"
SEED = 20261023
KEYS = ["beer", "beer_id", "tasting_category_fine"]
EXCLUDED_TARGETS = ["alcohol", "aftertaste", "body", "co2", "overall"]


class QuantileClipper(TransformerMixin, BaseEstimator):
    """Learn clipping bounds on the training fold; preserve missing entries."""

    def __init__(self, lower=0.01, upper=0.99):
        self.lower = lower
        self.upper = upper

    def fit(self, x, y=None):
        values = np.asarray(x, dtype=float)
        self.n_features_in_ = values.shape[1]
        self.bounds_ = np.nanquantile(values, [self.lower, self.upper], axis=0)
        return self

    def transform(self, x):
        values = np.asarray(x, dtype=float)
        if values.shape[1] != self.n_features_in_:
            raise ValueError("Feature count differs from training")
        return np.clip(values, self.bounds_[0], self.bounds_[1])


def data_dir():
    candidates = list(ROOT.glob("*/2026*赛道A*/附件"))
    if len(candidates) != 1:
        raise ValueError(f"Expected one A-track attachment directory; got {candidates}
")
    return candidates[0]


def resolve_recorded_path(value):
    """Resolve repository-relative provenance and the original machine's paths."""
    recorded = str(value).replace("\\", "/")
    if recorded.startswith("solution_a/"):
        return ROOT / recorded
    if "/solution_a/" in recorded:
        return SOLUTION / recorded.split("/solution_a/", 1)[1]
    if "/附件/" in recorded:
        return data_dir() / recorded.rsplit("/", 1)[1]
    direct = Path(recorded)
    return direct if direct.is_absolute() else ROOT / direct


def load_xy():
    """Join by exact beer number, never by the rounded barcodes in 4.csv."""
    chem = pd.read_csv(data_dir() / "1.csv", dtype={"beer_id": str})
    sensory = pd.read_csv(data_dir() / "4.csv", dtype={"beer_id": str})
```

src/common.py（续）

```
    for frame in (chem, sensory):
        if frame["beer"].isna().any() or not frame["beer"].is_unique:
            raise ValueError("beer must be a complete, unique sample key")
    if set(chem.beer) != set(sensory.beer):
        raise ValueError("Chemical and sensory populations differ")
    sensory = sensory.set_index("beer").loc[chem.beer].reset_index()
    if not chem.tasting_category_fine.equals(sensory.tasting_category_fine):
        raise ValueError("Styles disagree after joining by beer")
    features = [c for c in chem if c not in KEYS]
    targets = [c for c in sensory if c not in KEYS + EXCLUDED_TARGETS]
    x = chem[features].apply(pd.to_numeric, errors="raise")
    y = sensory[targets].apply(pd.to_numeric, errors="raise")
    if y.isna().any().any():
        raise ValueError("Missing labels need target-specific training, not imputation
")
    return chem[KEYS], x, y
```

src/platform_selfcheck.py

SHA-256: 35df4e836b8fb2d968dca1e679f27f3e1ebb97dd5bb9a93b70a1fd0a4b1c702b

```
"""Check portable paths, source hashes and frozen-history tamper detection."""
import hashlib
import importlib
import io
import json
import os
from pathlib import Path, PurePosixPath, PureWindowsPath
import sys
import tempfile
import unittest
from unittest.mock import patch

from common import SOLUTION, resolve_recorded_path
import common

sys.path.insert(0, str(SOLUTION))
OPTIMIZED = importlib.import_module("run_q3_optimized")
REFINED = importlib.import_module("run_q3_refined")
RUNNER = importlib.import_module("run_all")


class PlatformChecks(unittest.TestCase):
    def test_working_directory_cannot_shadow_project_sources(self):
        with tempfile.TemporaryDirectory(prefix="mathorcup-paths-") as temporary:
            project, shadow = Path(temporary) / "project", Path(temporary) / "shadow"
            for folder, data in [(project, b"project source"), (shadow, b"unrelated so
urce")]:
                (folder / "solution_a").mkdir(parents=True)
                (folder / "solution_a/example.txt").write_bytes(data)
            previous = Path.cwd()
            try:
                os.chdir(shadow)
                with patch.object(common, "ROOT", project), patch.object(common, "SOLU
TION", project / "solution_a"):
                    expected = project / "solution_a/example.txt"
                    self.assertEqual(resolve_recorded_path("solution_a/example.txt"),
expected)
                    self.assertEqual(resolve_recorded_path("/Users/old/repo/solution_a
/example.txt"), expected)
            finally:
                os.chdir(previous)

    def test_rebuild_refuses_existing_results(self):
        with tempfile.TemporaryDirectory(prefix="mathorcup-runner-") as temporary:
            root = Path(temporary)
            (root / "outputs").mkdir()
            saved = root / "outputs/saved.txt"
            saved.write_bytes(b"preserve this result")
            with patch.object(RUNNER, "HERE", root), patch.object(RUNNER, "run") as ex
ecute:
                with patch.object(sys, "argv", ["run_all.py", "--rebuild"]), patch.obj
ect(sys, "stderr", io.StringIO()):
                    with self.assertRaises(SystemExit) as error:
                        RUNNER.main()
                self.assertEqual(error.exception.code, 2)
                execute.assert_not_called()
            self.assertEqual(saved.read_bytes(), b"preserve this result")

    def test_clean_rebuild_orders_every_phase(self):
```

src/platform_selfcheck.py（续）

```
        with tempfile.TemporaryDirectory(prefix="mathorcup-runner-") as temporary:
            with patch.object(RUNNER, "HERE", Path(temporary)), patch.object(RUNNER, "run") as execute:
                with patch.object(sys, "argv", ["run_all.py", "--rebuild"]):
                    RUNNER.main()
                phases = [call.args[0] for call in execute.call_args_list]
            self.assertEqual(phases, ["01_audit.py", "02_q1_baseline.py", "02b_q1_robust.py",
                                      "03_q2_network.py", "04_stage_summary.py", "05_verify_outputs.py",
                                      "run_q3.py", "run_q3_optimized.py", "run_q3_domain.py", "run_q3_refined.py", "run_supplement.py"])

    def test_rebuild_refuses_existing_supplement(self):
        with tempfile.TemporaryDirectory(prefix="mathorcup-supplement-") as temporary:
            root = Path(temporary)
            saved = root / "supplement/results/sentinel.csv"
            saved.parent.mkdir(parents=True)
            saved.write_bytes(b"preserve supplementary results")
            with patch.object(RUNNER, "HERE", root), patch.object(RUNNER, "run") as execute:
                with patch.object(sys, "argv", ["run_all.py", "--rebuild"]), patch.object(sys, "stderr", io.StringIO()):
                    with self.assertRaises(SystemExit):
                        RUNNER.main()
                execute.assert_not_called()
            self.assertEqual(saved.read_bytes(), b"preserve supplementary results")

    def test_portable_relative_paths(self):
        self.assertEqual(PureWindowsPath(r"stale\summary.json").as_posix(),
                         PurePosixPath("stale/summary.json").as_posix())

    def test_all_three_frozen_histories(self):
        snapshots = [SOLUTION / "outputs" / name for name in
                     ["q3_optimized/baseline_hashes.json", "q3_domain/previous_hashes.json",
                      "q3_refined/previous_hashes.json"]]
        before = [path.read_bytes() for path in snapshots]
        REFINED.preserve_refined()
        self.assertEqual(before, [path.read_bytes() for path in snapshots])

    def test_recorded_sources_resolve_and_match(self):
        manifest = SOLUTION / "outputs/q3_refined/beer81/source_manifest.json"
        for row in json.loads(manifest.read_text(encoding="utf-8")):
            path = resolve_recorded_path(row["path"])
            self.assertTrue(path.is_file(), row["path"])
            self.assertEqual(hashlib.sha256(path.read_bytes()).hexdigest(), row["sha256"])

    def test_changed_file_is_rejected(self):
        with tempfile.TemporaryDirectory(prefix="mathorcup-freeze-") as temporary:
            root = Path(temporary)
            baseline, snapshot = root / "q3", root / "snapshot.json"
            target = baseline / "stale/summary.json"
            target.parent.mkdir(parents=True)
            target.write_text('{"label":"检验"}\n', encoding="utf-8", newline="\n")
            raw = target.read_bytes()
            record = {"files": [{"path": "stale/summary.json", "bytes": len(raw),
                                 "sha256": hashlib.sha256(raw).hexdigest()}]}
            (baseline / "verification.json").write_text("{}\n", encoding="utf-8", newline="\n")
```

src/platform_selfcheck.py（续）

```
            raw_verification = (baseline / "verification.json").read_bytes()
            record["files"].append({"path": "verification.json", "bytes": len(raw_verification),
                                    "sha256": hashlib.sha256(raw_verification).hexdigest()})
            snapshot.write_text(json.dumps(record), encoding="utf-8", newline="\n")
            with patch.object(OPTIMIZED, "BASELINE", baseline), patch.object(OPTIMIZED, "SNAPSHOT", snapshot):
                OPTIMIZED.preserve_baseline()
                target.write_bytes(raw + b" ")
                with self.assertRaisesRegex(RuntimeError, "frozen baseline has changed"):
                    OPTIMIZED.preserve_baseline()


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

src/q3_copula.py

SHA-256: 1d5e21ca13654b7724a87f7590615cfc809a3061a8970c0e81475993c7763b73

```
"""Small-sample style conditional Gaussian copula in the mixed Q3 feature space."""
import numpy as np
import pandas as pd
from scipy.linalg import solve_triangular
from scipy.special import ndtr, ndtri
from sklearn.covariance import LedoitWolf


class ConditionalGaussianCopula:
    """Empirical margins, shrunken style means, and positive definite covariances.

    ``fit`` learns *all* preprocessing from its input rows. Missing entries are
    replaced by training medians. The inverse empirical CDF returns observed
    column values, so zeros and ties have explicit probability intervals rather
    than being replaced by interpolated concentrations. Their generated
    frequencies need not equal empirical frequencies after conditioning and
    the latent Gaussian approximation. Sampling creates new
    combinations, not necessarily new individual measurement values.

    ``sample(None, ...)`` is the unconditional copula comparison. ``distance``
    is sqrt(Mahalanobis squared / number of features); its fitted training
    reference must be used to interpret it, not a chi-square calibration.
    """

    def __init__(self, mean_strength=5.0, cov_strength=20.0):
        if mean_strength < 0 or cov_strength < 0:
            raise ValueError("Shrinkage strengths must be nonnegative")
        self.mean_strength = float(mean_strength)
        self.cov_strength = float(cov_strength)

    def _array(self, x):
        if isinstance(x, pd.DataFrame):
            missing = set(self.columns_) - set(x.columns)
            extra = set(x.columns) - set(self.columns_)
            if missing or extra:
                raise ValueError(f"Feature mismatch: missing={missing}, extra={extra}"
)
            values = x.loc[:, self.columns_].to_numpy(dtype=float)
        else:
            values = np.asarray(x, dtype=float)
        if values.ndim != 2 or values.shape[1] != len(self.columns_):
            raise ValueError("Expected a matrix with the fitted feature count")
        if np.isinf(values).any():
            raise ValueError("Infinite chemical measurements are unsupported")
        return values

    def impute(self, x):
        values = self._array(x)
        return np.where(np.isnan(values), self.median_, values)

    @staticmethod
    def _regularized_covariance(z, assume_centered=False):
        if len(z) > 1:
            covariance = LedoitWolf(assume_centered=assume_centered).fit(z).covariance
_
        else:
            covariance = np.eye(z.shape[1])
        # A tiny explicit ridge also handles completely constant columns.
        ridge = 1e-6 * max(float(np.trace(covariance) / len(covariance)), 1.0)
```

src/q3_copula.py（续）

```
        return (covariance + covariance.T) / 2 + ridge * np.eye(z.shape[1])

    def fit(self, x, styles):
        if not isinstance(x, pd.DataFrame) or not x.columns.is_unique:
            raise ValueError("fit requires a DataFrame with unique feature names")
        self.columns_ = x.columns.tolist()
        values = self._array(x)
        labels = np.asarray(styles, dtype=str)
        if labels.ndim != 1 or len(labels) != len(values) or len(values) < 2:
            raise ValueError("One style per row and at least two training rows require
d")
        if np.isnan(values).all(axis=0).any():
            raise ValueError("A feature is completely missing in this training fold")
        self.median_ = np.nanmedian(values, axis=0)
        values = np.where(np.isnan(values), self.median_, values)
        self.n_features_in_ = values.shape[1]
        self.n_samples_ = len(values)
        self.empirical_values_ = []
        self.empirical_midprob_ = []
        self.empirical_cumprob_ = []
        for column in values.T:
            unique, counts = np.unique(column, return_counts=True)
            cumulative = counts.cumsum() / len(column)
            midpoint = (counts.cumsum() - counts / 2) / len(column)
            self.empirical_values_.append(unique)
            self.empirical_midprob_.append(midpoint)
            self.empirical_cumprob_.append(cumulative)
        z = self.transform(x)
        self.global_center_ = z.mean(axis=0)
        self.global_cov_ = self._regularized_covariance(z)
        self.global_cholesky_ = np.linalg.cholesky(self.global_cov_)
        self.style_counts_ = {s: int(np.sum(labels == s)) for s in np.unique(labels)}
        raw_centers = {s: z[labels == s].mean(axis=0) for s in self.style_counts_}
        residuals = np.vstack([z[labels == s] - raw_centers[s]
                               for s in self.style_counts_])
        self.pooled_within_cov_ = self._regularized_covariance(
            residuals, assume_centered=True)
        self.style_center_ = {}
        self.style_cov_ = {}
        self.style_cholesky_ = {}
        self.fit_diagnostics_ = []
        for style, count in self.style_counts_.items():
            a = count / (count + self.mean_strength)
            b = count / (count + self.cov_strength)
            center = a * raw_centers[style] + (1 - a) * self.global_center_
            local_cov = self._regularized_covariance(z[labels == style])
            covariance = b * local_cov + (1 - b) * self.pooled_within_cov_
            self.style_center_[style] = center
            self.style_cov_[style] = covariance
            self.style_cholesky_[style] = np.linalg.cholesky(covariance)
            self.fit_diagnostics_.append({
                "style": style, "n": count, "mean_local_weight": a,
                "cov_local_weight": b,
                "minimum_cov_eigenvalue": float(np.linalg.eigvalsh(covariance)[0]),
            })
        return self

    def transform(self, x):
        values = self.impute(x)
        z = np.empty_like(values)
        # Mid-CDF is exact at observed ties. Linear interpolation is used only
```

src/q3_copula.py（续）

```
        # to score intermediate inputs (e.g. projected migration candidates).
        tail = 0.5 / self.n_samples_
        for j in range(values.shape[1]):
            probability = np.interp(values[:, j], self.empirical_values_[j],
                                    self.empirical_midprob_[j])
            z[:, j] = ndtri(np.clip(probability, tail, 1 - tail))
        return z

    def inverse_transform(self, z):
        latent = np.asarray(z, dtype=float)
        if latent.ndim != 2 or latent.shape[1] != self.n_features_in_:
            raise ValueError("Latent feature count differs from training")
        probabilities = ndtr(latent)
        values = np.empty_like(latent)
        for j in range(latent.shape[1]):
            indices = np.searchsorted(self.empirical_cumprob_[j],
                                      probabilities[:, j], side="left")
            indices = np.minimum(indices, len(self.empirical_values_[j]) - 1)
            values[:, j] = self.empirical_values_[j][indices]
        return pd.DataFrame(values, columns=self.columns_)

    def sample(self, style, n=300, seed=20261023):
        if n < 1:
            raise ValueError("At least one sample is required")
        center = self.global_center_ if style is None else self.style_center_[style]
        chol = self.global_cholesky_ if style is None else self.style_cholesky_[style]
        z = np.random.default_rng(seed).normal(size=(n, self.n_features_in_))
        return self.inverse_transform(center + z @ chol.T)

    def distance(self, x, style=None):
        center = self.global_center_ if style is None else self.style_center_[style]
        chol = self.global_cholesky_ if style is None else self.style_cholesky_[style]
        residual = (self.transform(x) - center).T
        whitened = solve_triangular(chol, residual, lower=True, check_finite=False)
        return np.sqrt(np.mean(whitened ** 2, axis=0))
```

src/q3_data.py

SHA-256: 00f645b5502651a2f4f57998c9286d38ef35ac8e6320c99455afd3a6a300a7f7

```python
"""Explicit hybrid chemical representation shared by the question-3 models.

This module reads attachments without modifying them or fitting preprocessing.
"""
from __future__ import annotations

import numpy as np
import pandas as pd

from common import KEYS, data_dir, load_xy

DERIVED_FEATURES = (
    "kcalperc", "esters_sum", "aroma_hops_sum", "acids_sum", "sulfur_sum"
)
DEFECT_FEATURES = ("DMS", "X2.3.butanedione", "X4VG", "X.E..2.nonenal")
DEFECT_FIELD_MAP = {
    "DMS": "DMS",
    "Diacetyl": "X2.3.butanedione",
    "4VG": "X4VG",
    "trans_2_nonenal": "X.E..2.nonenal",
}


def read_defect_thresholds() -> dict:
    """Read exact field matches and threshold units from attachment 8.

    Missing thresholds are Python None, deliberately not inferred or imported
    from external sources. Values are in the calibrated representation only.
    """
    table = pd.read_excel(data_dir() / "8.xlsx", sheet_name="Sensory thresholds")
    if table["field_name_R"].duplicated().any():
        raise ValueError("Ambiguous sensory-threshold field names")
    table = table.set_index("field_name_R")
    result = {}
    for label, feature in DEFECT_FIELD_MAP.items():
        row = table.loc[feature]
        result[label] = {
            "feature": feature,
            "chemical_name": str(row["chemical_name"]),
            "unit": str(row["unit"]) if pd.notna(row["unit"]) else None,
            "threshold_median": float(row["threshold_median"])
            if pd.notna(row["threshold_median"]) else None,
            "threshold_mean": float(row["threshold_mean"])
            if pd.notna(row["threshold_mean"]) else None,
            "threshold_source": "8.xlsx / Sensory thresholds / exact field_name_R",
        }
    return result


# Exported immutable-by-convention attachment metadata for downstream scripts.
DEFECTS = read_defect_thresholds()


def load_q3_data() -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    """Return identity metadata, 226 raw hybrid features, 45 targets, manifest.

    Rows and features preserve load_xy()/1.csv order. The 69 overlapping,
    non-derived calibrated indicators replace their 1.csv representations by
    exact beer key. The remaining 157 variables keep their supplied 1.csv
```

src/q3_data.py（续）

```
    representations. Calibrated negatives become missing; relative negatives
    are retained because their transformation/zero point is not identified.
    No imputation, standardization or fitted data transformation occurs here.
    """
    meta, original, y = load_xy()
    workbook = data_dir() / "8.xlsx"
    calibrated = pd.read_excel(workbook, sheet_name="Calibrated chemical dataset")
    thresholds = pd.read_excel(workbook, sheet_name="Sensory thresholds")
    if calibrated["beer"].isna().any() or not calibrated["beer"].is_unique:
        raise ValueError("Calibrated beer keys must be complete and unique")
    if set(calibrated["beer"]) != set(meta["beer"]):
        raise ValueError("Original and calibrated sample populations differ")
    calibrated = calibrated.set_index("beer").loc[meta["beer"]].reset_index()
    if not calibrated["tasting_category_fine"].equals(meta["tasting_category_fine"]):
        raise ValueError("Calibrated styles disagree after exact beer join")
    if thresholds["field_name_R"].duplicated().any():
        raise ValueError("Unit metadata must have unique exact field names")
    unit_map = thresholds.set_index("field_name_R")["unit"].to_dict()
    features = [feature for feature in original if feature not in DERIVED_FEATURES]
    calibrated_features = [feature for feature in calibrated if feature not in KEYS
                           and feature not in DERIVED_FEATURES]
    if not set(calibrated_features).issubset(features):
        raise ValueError("Calibrated indicators are absent from 1.csv")
    if len(features) != 226 or len(calibrated_features) != 69:
        raise ValueError("Attachment shape differs from verified 226/69 contract")
    x = original.loc[:, features].copy()
    records = []
    for position, feature in enumerate(features):
        is_calibrated = feature in calibrated_features
        values = pd.to_numeric(calibrated[feature] if is_calibrated
                               else original[feature], errors="raise").copy()
        original_missing = int(values.isna().sum())
        if np.isinf(values.to_numpy(dtype=float)).any():
            raise ValueError(f"Infinite attachment feature: {feature}")
        negative = values.lt(0) if is_calibrated else pd.Series(False, index=values.in
dex)
        values.loc[negative] = np.nan
        x.loc[:, feature] = values.to_numpy()
        raw_unit = unit_map.get(feature)
        unit_available = is_calibrated and pd.notna(raw_unit)
        records.append({
            "position": position,
            "feature": feature,
            "representation": "calibrated" if is_calibrated else "relative",
            "unit": str(raw_unit) if unit_available else
                    ("not_provided_in_attachment" if is_calibrated
                     else "supplied_relative_or_transformed_units"),
            "unit_source": "8.xlsx / Sensory thresholds / exact field_name_R"
                           if unit_available else "unidentified",
            "source": "8.xlsx / Calibrated chemical dataset" if is_calibrated else "1.
csv",
            "original_missing_count": original_missing,
            "original_1csv_missing_count": int(original[feature].isna().sum()),
            "negative_invalid_count": int(negative.sum()),
            "final_missing_count": int(values.isna().sum()),
            "zero_count": int(values.eq(0).sum()),
            "negative_policy": "replace_with_NaN" if is_calibrated else "retain_suppli
ed_representation",
            "is_defect_concentration": feature in DEFECT_FEATURES,
        })
    manifest = pd.DataFrame(records)
```

src/q3_data.py（续）

```
    if x.shape != (250, 226) or y.shape != (250, 45):
        raise ValueError("Unexpected question-3 input or target dimensions")
    if manifest["representation"].value_counts().to_dict() != {"relative": 157, "calibrated": 69}:
        raise AssertionError("Hybrid representation count mismatch")
    return meta.loc[:, KEYS].copy(), x, y.copy(), manifest
```

src/q3_generator_optimized.py

SHA-256: 3d7f1004577298d5ecdd2ad9b7e0bbd2f299d863eff699fc45c047d279785156

```python
"""Nested selection of small-sample generators with style-specific margins."""
import numpy as np
import pandas as pd
from scipy.special import ndtr, ndtri
from scipy.spatial.distance import cdist
from sklearn.covariance import LedoitWolf
from sklearn.decomposition import PCA
from sklearn.model_selection import StratifiedKFold

from q3_copula import ConditionalGaussianCopula


EVALUATION_STYLES = ("Tripel", "Kriek", "Lambic", "Pils/Lager")
CANDIDATE_METHODS = ("style_margin_copula", "style_smoothed_mixture", "legacy_copula")
MIN_INNER_NN_RATIO = 0.35
MAX_INNER_MATCH_FRACTION = 0.95
SMOOTHING_NOISE = 0.45


def empirical_tables(values):
    unique_values, midpoints, cumulative = [], [], []
    for column in values.T:
        unique, counts = np.unique(column, return_counts=True)
        unique_values.append(unique)
        midpoints.append((counts.cumsum() - counts / 2) / len(column))
        cumulative.append(counts.cumsum() / len(column))
    return unique_values, midpoints, cumulative


def normal_scores(values, tables):
    z = np.empty_like(values, dtype=float)
    for j, (unique, midpoints) in enumerate(zip(tables[0], tables[1])):
        z[:, j] = ndtri(np.interp(values[:, j], unique, midpoints))
    return z


def regularized_correlation(z):
    if len(z) < 2 or np.max(np.abs(z)) < 1e-12:
        return np.eye(z.shape[1])
    covariance = LedoitWolf().fit(z).covariance_
    covariance += 1e-6 * np.eye(z.shape[1])
    scale = np.sqrt(np.diag(covariance))
    correlation = covariance / scale[:, None] / scale[None, :]
    return (correlation + correlation.T) / 2


class StyleMarginCopula:
    """Use per-style inverse empirical margins and shrunken dependence.

    ``mode='copula'`` uses a zero-mean, unit-marginal Gaussian whose covariance
    mixes local and shared style-rank correlations. ``mode='smoothed'`` draws
    an empirical style row in normal-score space and adds correlated noise
    with fixed standard deviation 0.45. It smooths empirical combinations,
    rather than choosing an almost-zero perturbation to avoid exact copies.

    Zeros have probability intervals in the inverse CDF. Gaussian mode has
    standard-normal latent margins, so empirical zero rates are preserved in
    expectation; smoothed mode need not reproduce them exactly.
    """
```

src/q3_generator_optimized.py（续）

```
60
61      def __init__(self, mode="copula", cov_strength=10.0, noise=SMOOTHING_NOISE):
62          if mode not in ("copula", "smoothed"):
63              raise ValueError("Unknown style-margin generation mode")
64          self.mode = mode
65          self.cov_strength = float(cov_strength)
66          self.noise = float(noise)
67
68      def fit(self, x, styles):
69          self.columns_ = x.columns.tolist()
70          values = x.to_numpy(dtype=float)
71          labels = np.asarray(styles, dtype=str)
72          self.global_median_ = np.nanmedian(values, axis=0)
73          if np.isnan(self.global_median_).any() or np.isinf(values).any():
74              raise ValueError("A feature is entirely missing or infinite")
75          self.tables_, self.style_scores_, self.style_median_ = {}, {}, {}
76          self.style_counts_ = {}
77          for style in np.unique(labels):
78              local = values[labels == style]
79              median = np.array([np.median(c[np.isfinite(c)]) if np.isfinite(c).any()
80                                 else self.global_median_[j] for j, c in enumerate(local
    .T)])
81              local = np.where(np.isnan(local), median, local)
82              self.style_median_[style] = median
83              self.tables_[style] = empirical_tables(local)
84              self.style_scores_[style] = normal_scores(local, self.tables_[style])
85              self.style_counts_[style] = len(local)
86          pooled = np.vstack(list(self.style_scores_.values()))
87          self.shared_correlation_ = regularized_correlation(pooled)
88          self.style_correlation_, self.style_cholesky_ = {}, {}
89          for style, scores in self.style_scores_.items():
90              weight = len(scores) / (len(scores) + self.cov_strength)
91              correlation = (weight * regularized_correlation(scores)
92                             + (1 - weight) * self.shared_correlation_)
93              self.style_correlation_[style] = correlation
94              self.style_cholesky_[style] = np.linalg.cholesky(correlation)
95          return self
96
97      def sample(self, style, n=300, seed=20261123):
98          if n < 1:
99              raise ValueError("At least one sample is required")
100         rng = np.random.default_rng(seed)
101         z = rng.normal(size=(n, len(self.columns_))) @ self.style_cholesky_[style].T
102         if self.mode == "smoothed":
103             centers = self.style_scores_[style]
104             indices = rng.integers(0, len(centers), size=n)
105             z = centers[indices] + self.noise * z
106         probability = ndtr(z)
107         values = np.empty_like(z)
108         unique, _, cumulative = self.tables_[style]
109         for j in range(values.shape[1]):
110             indices = np.searchsorted(cumulative[j], probability[:, j], side="left")
111             values[:, j] = unique[j][np.minimum(indices, len(unique[j]) - 1)]
112         return pd.DataFrame(values, columns=self.columns_)
113
114
115 def fit_candidates(x, labels):
116     legacy = ConditionalGaussianCopula(5.0, 20.0).fit(x, labels)
117     copula = StyleMarginCopula(mode="copula").fit(x, labels)
118     # Both families have identical fitted margins and covariance; reuse fitted
119     # arrays and change only the declared sampling kernel.
```

src/q3_generator_optimized.py（续）

```
    mixture = StyleMarginCopula(mode="smoothed")
    mixture.__dict__.update(copula.__dict__)
    mixture.mode = "smoothed"
    return {"style_margin_copula": copula, "style_smoothed_mixture": mixture,
            "legacy_copula": legacy}


def selection_metrics(reference_model, pca, pc_scale, style_train, heldout, generated)
:
    train_z = reference_model.transform(style_train)
    real_z = reference_model.transform(heldout)
    fake_z = reference_model.transform(generated)
    real_pc, fake_pc = pca.transform(real_z) / pc_scale, pca.transform(fake_z) / pc_sc
ale
    energy = float(2 * cdist(real_pc, fake_pc).mean()
                   - cdist(real_pc, real_pc).mean() - cdist(fake_pc, fake_pc).mean())
    distances = cdist(fake_z, train_z) / np.sqrt(train_z.shape[1])
    nn = distances.min(axis=1)
    real_nn = cdist(real_z, train_z).min(axis=1) / np.sqrt(train_z.shape[1])
    nearest = distances.argmin(axis=1)
    fake = reference_model.impute(generated)
    training = reference_model.impute(style_train)[nearest]
    equal_fraction = np.isclose(fake, training, rtol=1e-10, atol=1e-12).mean(axis=1)
    observed = style_train.to_numpy(dtype=float)
    copied_observed = (np.isclose(fake[:, None, :], observed[None, :, :],
                                  rtol=1e-10, atol=1e-12)
                       | np.isnan(observed)[None, :, :]).all(axis=2).any(axis=1)
    return {"energy_pca5": energy,
            "nn_ratio": float(np.median(nn) / max(np.median(real_nn), 1e-12)),
            "copy_rate": float(np.mean(nn < 1e-12)),
            "observed_training_copy_rate": float(copied_observed.mean()),
            "nearest_equal_feature_fraction": float(np.median(equal_fraction))}


class OptimizedStyleGenerator:
    """Choose one of three fixed generators using training-only inner folds.

    Inner validation uses two folds stratified across every training style.
    The generator for each of the four evaluation styles minimizes inner
    five-dimensional energy distance, subject to no exact copies, NN ratio
    >= 0.35, and median matching fraction <= 0.95 in every inner fold.
    Other styles default to the legacy generator. Bootstrap is evaluated as a
    benchmark but is excluded from selection because it is a copying model.
    """

    def __init__(self, selection_seed=20261123, inner_n_generated=300):
        self.selection_seed = int(selection_seed)
        self.inner_n_generated = int(inner_n_generated)

    def fit(self, x, styles):
        self.columns_ = x.columns.tolist()
        labels = pd.Series(np.asarray(styles, dtype=str), index=x.index)
        if labels.value_counts().min() < 2:
            raise ValueError("Inner stratified two-fold requires two rows per training
 style")
        selected_styles = [style for style in EVALUATION_STYLES if style in set(labels
)]
        records = []
        splitter = StratifiedKFold(n_splits=2, shuffle=True, random_state=self.selecti
on_seed)
        for fold, (train, validation) in enumerate(splitter.split(x, labels), start=1)
```

src/q3_generator_optimized.py（续）

```
    :
176             inner_train = x.iloc[train]
177             inner_labels = labels.iloc[train]
178             fitted = fit_candidates(inner_train, inner_labels)
179             reference = fitted["legacy_copula"]
180             train_z = reference.transform(inner_train)
181             pca = PCA(n_components=min(5, len(train_z) - 1), svd_solver="full").fit(tr
    ain_z)
182             pc_scale = np.maximum(np.sqrt(pca.explained_variance_), 1e-8)
183             for style_index, style in enumerate(selected_styles):
184                 training_style = inner_train.loc[inner_labels == style]
185                 heldout = x.iloc[validation].loc[labels.iloc[validation] == style]
186                 for method, candidate in fitted.items():
187                     seed = self.selection_seed + fold * 1000 + style_index * 10
188                     generated = candidate.sample(style, self.inner_n_generated, seed)
189                     metrics = selection_metrics(reference, pca, pc_scale, training_sty
    le, heldout, generated)
190                     eligible = (metrics["copy_rate"] == 0
191                                 and metrics["observed_training_copy_rate"] == 0
192                                 and metrics["nn_ratio"] >= MIN_INNER_NN_RATIO
193                                 and metrics["nearest_equal_feature_fraction"] <= MAX_I
    NNER_MATCH_FRACTION)
194                     records.append({"inner_fold": fold, "style": style, "method": meth
    od,
195                                     "seed": seed, "train_n_style": len(training_style)
    ,
196                                     "validation_n_style": len(heldout), "eligible": el
    igible, **metrics})
197         self.inner_scores_ = pd.DataFrame(records)
198         self.selected_methods_ = {style: "legacy_copula" for style in labels.unique()}
199         self.selection_table_ = []
200         for style in selected_styles:
201             subset = self.inner_scores_.loc[self.inner_scores_["style"] == style]
202             grouped = subset.groupby("method").agg(energy_pca5=("energy_pca5", "mean")
    ,
203                 eligible=("eligible", "all"), worst_nn_ratio=("nn_ratio", "min"),
204                 largest_matching_fraction=("nearest_equal_feature_fraction", "max"))
205             accepted = grouped.loc[grouped.eligible]
206             if len(accepted):
207                 selected = accepted.energy_pca5.idxmin()
208                 status = "selected_by_training_only_inner_energy_with_novelty_gates"
209             else:
210                 selected = "legacy_copula"
211                 status = "no_candidate_passed_inner_gates_legacy_fallback_requires_dia
    gnostics"
212             self.selected_methods_[style] = selected
213             for method, row in grouped.iterrows():
214                 self.selection_table_.append({"style": style, "method": method,
215                     "selected": method == selected, "selection_status": status, **row.
    to_dict()})
216         self.fitted_candidates_ = fit_candidates(x, labels)
217         return self
218
219     def sample(self, style, n=300, seed=20261123):
220         if style is None:
221             return self.fitted_candidates_["legacy_copula"].sample(None, n, seed)
222         return self.fitted_candidates_[self.selected_methods_[style]].sample(style, n,
     seed)
```

src/q3_stale_domain.py

SHA-256: 6d0be013ea3249f3ff95cd212e7011414796569d32eebd6fbdf26b412830e1c9

```
"""Style-held-out stale-hops candidates with chemical-only prediction inputs."""
from __future__ import annotations

import numpy as np
from scipy.spatial.distance import cdist
from sklearn.base import BaseEstimator, RegressorMixin
from sklearn.decomposition import PCA
from sklearn.dummy import DummyRegressor
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.impute import SimpleImputer
from sklearn.kernel_ridge import KernelRidge
from sklearn.linear_model import Ridge
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR

from q3_stale_optimized import FoldChemicalTransform

BIAS_PENALTY = 0.25
VARIANCE_RATIO_MIN = 0.05
LOSO_SEED = 20270123
FINAL_SEED = 20271123
SELECTION_BOOTSTRAP_SEED = 20272123
AUDIT_BOOTSTRAP_SEED = 20273123


def style_balanced_weights(styles):
    """Each observed training style receives equal total regression weight."""
    values = np.asarray(styles)
    _, inverse, counts = np.unique(values, return_inverse=True, return_counts=True)
    return len(values) / (len(counts) * counts[inverse])


class CenteredKernelRidge(RegressorMixin, BaseEstimator):
    """RBF ridge with a training-only weighted target mean for far points."""
    def __init__(self, alpha=1.0, gamma_factor=0.1):
        self.alpha = alpha
        self.gamma_factor = gamma_factor

    def fit(self, x, y, sample_weight=None):
        values = np.asarray(x, dtype=float)
        target = np.asarray(y, dtype=float).reshape(-1)
        self.target_mean_ = float(np.average(target, weights=sample_weight))
        self.n_features_in_ = values.shape[1]
        self.model_ = KernelRidge(alpha=self.alpha, kernel="rbf",
                                  gamma=self.gamma_factor / self.n_features_in_)
        self.model_.fit(values, target - self.target_mean_, sample_weight=sample_weigh
t)
        return self

    def predict(self, x):
        return self.target_mean_ + self.model_.predict(np.asarray(x, dtype=float))


class StyleWeightedLocalMean(RegressorMixin, BaseEstimator):
    """Local chemical neighbours, inverse distance times training-style weight."""
    def __init__(self, n_neighbors=25):
        self.n_neighbors = n_neighbors

```

src/q3_stale_domain.py（续）

```
    def fit(self, x, y, sample_weight=None):
        self.x_ = np.asarray(x, dtype=float)
        self.y_ = np.asarray(y, dtype=float).reshape(-1)
        self.sample_weight_ = np.ones(len(self.y_)) if sample_weight is None else np.a
sarray(sample_weight)
        self.n_features_in_ = self.x_.shape[1]
        return self

    def predict(self, x):
        values = np.asarray(x, dtype=float)
        result = np.empty(len(values))
        k = min(self.n_neighbors, len(self.y_))
        for start in range(0, len(values), 2048):
            distances = cdist(values[start:start+2048], self.x_)
            for local, row in enumerate(distances):
                exact = row <= 1e-12
                if exact.any():
                    result[start + local] = np.average(self.y_[exact], weights=self.sa
mple_weight_[exact])
                else:
                    indices = np.argpartition(row, k-1)[:k]
                    weights = self.sample_weight_[indices] / np.maximum(row[indices],
1e-12)
                    result[start + local] = np.average(self.y_[indices], weights=weigh
ts)
        return result


class EqualDomainEnsemble(RegressorMixin, BaseEstimator):
    """A fixed equal-weight average; no stacking or fitted mixture weights."""
    def __init__(self, features, calibrated_features, aging_features, member_specs, se
ed):
        self.features = features
        self.calibrated_features = calibrated_features
        self.aging_features = aging_features
        self.member_specs = member_specs
        self.seed = seed

    def fit(self, x, y, sample_weight=None):
        self.models_ = []
        for spec in self.member_specs:
            model = make_domain_pipeline(spec, self.features, self.calibrated_features
,
                                         self.aging_features, self.seed)
            params = {"model__sample_weight": sample_weight} if sample_weight is not N
one else {}
            model.fit(x, y, **params)
            self.models_.append(model)
        self.n_features_in_ = len(self.features)
        return self

    def predict(self, x):
        return np.mean([np.asarray(model.predict(x)).reshape(-1) for model in self.mod
els_], axis=0)


def domain_candidate_specs():
    """Freeze 14 comparisons before any new LOSO or Pils diagnostic results."""
    specs = [
        {"name": "mean_global", "family": "mean", "balanced": False, "selectable": Fal
se},
```

src/q3_stale_domain.py（续）

```
        {"name": "mean_style_balanced", "family": "mean", "balanced": True, "selectable": False},
        {"name": "old_multioutput_tree", "family": "old_multioutput", "balanced": False, "selectable": False},
        {"name": "fixed_single_tree", "family": "tree", "balanced": False, "selectable": True},
        {"name": "style_balanced_tree", "family": "tree", "balanced": True, "selectable": True},
        {"name": "ridge_all_balanced_100", "family": "ridge", "feature_set": "all226", "alpha": 100.0, "balanced": True, "selectable": True},
        {"name": "ridge_all_balanced_1000", "family": "ridge", "feature_set": "all226", "alpha": 1000.0, "balanced": True, "selectable": True},
        {"name": "ridge_aging_balanced_10", "family": "ridge", "feature_set": "aging", "alpha": 10.0, "balanced": True, "selectable": True},
        {"name": "ridge_aging_balanced_100", "family": "ridge", "feature_set": "aging", "alpha": 100.0, "balanced": True, "selectable": True},
        {"name": "krr_aging_balanced_g01", "family": "krr", "feature_set": "aging", "alpha": 1.0, "gamma_factor": 0.1, "balanced": True, "selectable": True},
        {"name": "krr_aging_balanced_g1", "family": "krr", "feature_set": "aging", "alpha": 1.0, "gamma_factor": 1.0, "balanced": True, "selectable": True},
        {"name": "svr_aging_balanced_C1", "family": "svr", "feature_set": "aging", "C": 1.0, "epsilon": 0.05, "balanced": True, "selectable": True},
        {"name": "local_aging_pca8_k25", "family": "local", "feature_set": "aging", "n_components": 8, "n_neighbors": 25, "balanced": True, "selectable": True},
    ]
    ensemble_names = ["style_balanced_tree", "ridge_aging_balanced_100", "krr_aging_balanced_g01"]
    specs.append({"name": "ensemble_equal3", "family": "ensemble", "balanced": True,
                  "selectable": True, "weights": [1/3, 1/3, 1/3],
                  "members": [dict(spec) for spec in specs if spec["name"] in ensemble_names]})
    return specs


def make_domain_pipeline(spec, features, calibrated, aging, seed):
    family = spec["family"]
    if family == "ensemble":
        return Pipeline([("model", EqualDomainEnsemble(features, calibrated, aging, spec["members"], seed))])
    imputer = SimpleImputer(strategy="median", add_indicator=True, keep_empty_features=True)
    if family in ("tree", "mean", "old_multioutput"):
        regressor = DummyRegressor(strategy="mean") if family == "mean" else ExtraTreesRegressor(
            n_estimators=300, min_samples_leaf=3, max_features=0.7, random_state=seed, n_jobs=1)
        return Pipeline([("imputer", imputer), ("model", regressor)])
    selected = aging if spec["feature_set"] == "aging" else None
    steps = [("chemical", FoldChemicalTransform(features, calibrated, selected)),
             ("imputer", imputer), ("scale", StandardScaler())]
    if family == "ridge":
        regressor = Ridge(alpha=spec["alpha"], solver="svd")
    elif family == "krr":
        regressor = CenteredKernelRidge(alpha=spec["alpha"], gamma_factor=spec["gamma_factor"])
    elif family == "svr":
        regressor = SVR(kernel="rbf", C=spec["C"], epsilon=spec["epsilon"], gamma="scale")
    elif family == "local":
        steps.append(("pca", PCA(n_components=spec["n_components"], svd_solver="full")))
```

src/q3_stale_domain.py（续）

```
        regressor = StyleWeightedLocalMean(n_neighbors=spec["n_neighbors"])
    else:
        raise ValueError(f"Unknown domain candidate: {family}")
    return Pipeline([*steps, ("model", regressor)])


def training_fit_params(spec, styles):
    return {"model__sample_weight": style_balanced_weights(styles)} if spec["balanced"
] else {}
```

src/q3_stale_optimized.py

SHA-256: 1720024ac5855f1b905de115a751cbdcfc9f97e06249091e27c24549bd34dc4a

```
"""Fold-local chemical preprocessing and single-target stale-hops models."""
from __future__ import annotations

import numpy as np
from scipy.stats import spearmanr
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.cross_decomposition import PLSRegression
from sklearn.ensemble import ExtraTreesRegressor, GradientBoostingRegressor, RandomForestRegressor
from sklearn.impute import SimpleImputer
from sklearn.linear_model import ElasticNet, Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import KFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR

VARIANCE_RATIO_MIN = 0.05
OUTER_SEEDS = (20261123, 20262123)
TRANSFER_CV_SEED = 20263123
SELECTION_BOOTSTRAP_SEED = 20264123
AUDIT_BOOTSTRAP_SEED = 20265123


class FoldChemicalTransform(TransformerMixin, BaseEstimator):
    """Select predeclared chemical fields, log calibrated values, then clip.

    Positive-median scaling, 1%/99% clipping bounds and all downstream
    imputation/scaling are learned anew in each training subset. Relative
    representations are not exponentiated or treated as concentrations.
    """

    def __init__(self, features, calibrated_features, selected_features=None):
        self.features = features
        self.calibrated_features = calibrated_features
        self.selected_features = selected_features

    def _values(self, x):
        raw = x.loc[:, self.features].to_numpy(dtype=float) if hasattr(x, "loc") else np.asarray(x, dtype=float)
        if raw.ndim != 2 or raw.shape[1] != len(self.features):
            raise ValueError("Expected the declared 226-feature hybrid representation")
        return raw[:, self.positions_].copy()

    def _log(self, values):
        for position in self.calibrated_positions_:
            column = values[:, position]
            if np.any(column[np.isfinite(column)] < 0):
                raise ValueError("Calibrated concentrations/indicators must be nonnegative")
            values[:, position] = np.log1p(column / self.positive_scales_[position])
        return values

    def fit(self, x, y=None):
        selected = self.features if self.selected_features is None else self.selected_features
        self.positions_ = np.asarray([self.features.index(feature) for feature in selected])
```

src/q3_stale_optimized.py（续）

```
        self.selected_features_ = list(selected)
        self.calibrated_positions_ = [i for i, feature in enumerate(selected)
                                      if feature in self.calibrated_features]
        values = self._values(x)
        self.positive_scales_ = np.ones(values.shape[1])
        for position in self.calibrated_positions_:
            positive = values[:, position][np.isfinite(values[:, position]) & (values[:, position] > 0)]
            self.positive_scales_[position] = float(np.median(positive)) if len(positive) else 1.0
        values = self._log(values)
        self.bounds_ = np.nanquantile(values, [0.01, 0.99], axis=0)
        self.n_features_in_ = len(self.features)
        return self

    def transform(self, x):
        return np.clip(self._log(self._values(x)), self.bounds_[0], self.bounds_[1])


def candidate_specs() -> list[dict]:
    """Small grid declared before inspecting any new outer-fold outcomes."""
    specs = []
    def add(family, label, **params):
        specs.append({"name": f"{family}_{label}", "family": family,
                      "feature_set": "all226", "params": params})
    for alpha in (10.0, 100.0, 1000.0):
        add("ridge", f"alpha{alpha:g}", alpha=alpha)
    for alpha in (0.01, 0.1):
        add("elastic_net", f"alpha{alpha:g}", alpha=alpha, l1_ratio=0.5)
    for n_components in (3, 8):
        add("pls", f"n{n_components}", n_components=n_components)
    for c in (0.3, 1.0, 3.0):
        add("svr", f"C{c:g}", C=c, epsilon=0.05, gamma="scale")
    for leaf in (3, 8):
        add("extra_trees", f"leaf{leaf}", n_estimators=160,
            min_samples_leaf=leaf, max_features=0.7)
        add("random_forest", f"leaf{leaf}", n_estimators=160,
            min_samples_leaf=leaf, max_features=0.7)
    for loss in ("squared_error", "huber"):
        add("gradient_boosting", loss, loss=loss, n_estimators=150,
            learning_rate=0.03, max_depth=2, min_samples_leaf=5)
    specs.append({"name": "ridge_aging_alpha10", "family": "ridge", "feature_set": "aging",
                  "params": {"alpha": 10.0}})
    specs.append({"name": "svr_aging_C1", "family": "svr", "feature_set": "aging",
                  "params": {"C": 1.0, "epsilon": 0.05, "gamma": "scale"}})
    return specs


def make_pipeline(spec, features, calibrated_features, aging_features, seed):
    selected = aging_features if spec["feature_set"] == "aging" else None
    family, params = spec["family"], dict(spec["params"])
    if family == "ridge":
        estimator = Ridge(solver="svd", **params)
    elif family == "elastic_net":
        estimator = ElasticNet(max_iter=20000, tol=1e-5, random_state=seed, **params)
    elif family == "pls":
        estimator = PLSRegression(scale=False, max_iter=1000, **params)
    elif family == "svr":
        estimator = SVR(**params)
    elif family == "extra_trees":
```

src/q3_stale_optimized.py（续）

```
        estimator = ExtraTreesRegressor(random_state=seed, n_jobs=1, **params)
    elif family == "random_forest":
        estimator = RandomForestRegressor(random_state=seed, n_jobs=1, **params)
    elif family == "gradient_boosting":
        estimator = GradientBoostingRegressor(random_state=seed, **params)
    else:
        raise ValueError(f"Unsupported family: {family}")
    return Pipeline([
        ("chemical", FoldChemicalTransform(features, calibrated_features, selected)),
        ("imputer", SimpleImputer(strategy="median", add_indicator=True, keep_empty_features=True)),
        ("scale", StandardScaler()),
        ("model", estimator),
    ])


def raw_tree_pipeline(seed, n_estimators=300):
    return Pipeline([
        ("imputer", SimpleImputer(strategy="median", add_indicator=True, keep_empty_features=True)),
        ("model", ExtraTreesRegressor(n_estimators=n_estimators, min_samples_leaf=3,
                                      max_features=0.7, random_state=seed, n_jobs=1)),
    ])


def predict_1d(model, x):
    return np.asarray(model.predict(x), dtype=float).reshape(-1)


def scores(y, pred):
    truth, prediction = np.asarray(y, dtype=float).reshape(-1), np.asarray(pred, dtype=float).reshape(-1)
    truth_var, pred_var = float(np.var(truth)), float(np.var(prediction))
    correlation = float(spearmanr(truth, prediction).statistic) if pred_var > 1e-20 else None
    target_range = float(np.quantile(truth, .95) - np.quantile(truth, .05))
    prediction_range = float(np.quantile(prediction, .95) - np.quantile(prediction, .05))
    return {"mae": float(mean_absolute_error(truth, prediction)),
            "rmse": float(np.sqrt(mean_squared_error(truth, prediction))),
            "r2": float(r2_score(truth, prediction)), "spearman": correlation,
            "prediction_variance": pred_var, "target_variance": truth_var,
            "prediction_variance_ratio": pred_var / truth_var if truth_var > 0 else None,
            "prediction_std_ratio": float(np.sqrt(pred_var / truth_var)) if truth_var > 0 else None,
            "prediction_range90": prediction_range, "target_range90": target_range,
            "prediction_range90_ratio": prediction_range / target_range if target_range > 0 else None,
            "mean_bias": float(np.mean(prediction - truth))}


def inner_compare(x, y, specs, features, calibrated, aging, seed):
    """Training-only OOF comparison, with a prespecified nonconstant gate."""
    rows, predictions = [], {}
    splits = list(KFold(3, shuffle=True, random_state=seed).split(x))
    for spec in specs:
        pred = np.full(len(x), np.nan)
        for fold, (train, test) in enumerate(splits, 1):
            model = make_pipeline(spec, features, calibrated, aging, seed + fold)
            model.fit(x.iloc[train], y.iloc[train])
```

src/q3_stale_optimized.py（续）

```
            pred[test] = predict_1d(model, x.iloc[test])
        if not np.isfinite(pred).all():
            raise AssertionError(f"Nonfinite inner predictions: {spec['name']}")
        metrics = scores(y, pred)
        rows.append({"candidate": spec["name"], "family": spec["family"],
                     "feature_set": spec["feature_set"], **metrics,
                     "nonconstant_qualified": metrics["prediction_variance_ratio"] >= VARIANCE_RATIO_MIN})
        predictions[spec["name"]] = pred
    return rows, predictions


def choose_candidate(rows, family=None):
    choices = [row for row in rows if family is None or row["family"] == family]
    eligible = [row for row in choices if row["nonconstant_qualified"]]
    # A failed gate never disappears: callers persist nonconstant_qualified.
    return min(eligible or choices, key=lambda row: (row["rmse"], row["candidate"]))
```

src/q3_stale_refined.py

SHA-256: 11730edf74f74a438072a3ad3bc9e3218fbbf0fe1a9a602a2b936e37eec4b076

```
"""Smooth chemical-only stale prediction with training-only auxiliary targets."""
from __future__ import annotations

import numpy as np
from sklearn.base import BaseEstimator, RegressorMixin, TransformerMixin
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.impute import SimpleImputer
from sklearn.kernel_ridge import KernelRidge
from sklearn.linear_model import Ridge

TARGET = "stale_hops"
BIAS_PENALTY = 0.25
VARIANCE_RATIO_MIN = 0.05
STABILITY_DRAWS = 8
STABILITY_SEED = 20280123
NESTED_SEED = 20281123
FINAL_SEED = 20282123
SELECTION_BOOTSTRAP_SEED = 20283123
AUDIT_BOOTSTRAP_SEED = 20284123


def style_weights(styles):
    _, inverse, counts = np.unique(np.asarray(styles), return_inverse=True, return_cou
nts=True)
    return len(inverse)/(len(counts)*counts[inverse])


class FixedChemicalTransform(TransformerMixin, BaseEstimator):
    """Fold-local log/clip/median/scale plus a fixed-width missing block.

    Missing indicators are always present and weighted by 0.2; unlike rare
    Bernoulli scaling, a single missing value cannot make a huge new axis.
    """
    def __init__(self, features, calibrated_features, selected_features=None,
                 indicator_weight=0.2):
        self.features = features
        self.calibrated_features = calibrated_features
        self.selected_features = selected_features
        self.indicator_weight = indicator_weight

    def _raw(self, x):
        values = x.loc[:, self.features].to_numpy(dtype=float) if hasattr(x, "loc") el
se np.asarray(x, dtype=float)
        if values.shape[1] != len(self.features):
            raise ValueError("Expected declared hybrid feature order")
        return values[:, self.positions_].copy()

    def _log(self, values):
        for j in self.calibrated_positions_:
            if np.any(values[np.isfinite(values[:, j]), j] < 0):
                raise ValueError("Negative calibrated input")
            values[:, j] = np.log1p(values[:, j]/self.positive_scales_[j])
        return values

    def fit(self, x, y=None):
        selected = self.features if self.selected_features is None else self.selected_
features
        self.selected_features_ = list(selected)
        self.positions_ = np.asarray([self.features.index(feature) for feature in sele
```

src/q3_stale_refined.py（续）

```
cted])
        self.calibrated_positions_ = [j for j, feature in enumerate(selected)
                                      if feature in self.calibrated_features]
        values = self._raw(x)
        self.positive_scales_ = np.ones(values.shape[1])
        for j in self.calibrated_positions_:
            positive = values[np.isfinite(values[:, j]) & (values[:, j] > 0), j]
            self.positive_scales_[j] = np.median(positive) if len(positive) else 1.0
        values = self._log(values)
        self.bounds_ = np.nanquantile(values, [.01, .99], axis=0)
        values = np.clip(values, self.bounds_[0], self.bounds_[1])
        self.medians_ = np.nanmedian(values, axis=0)
        self.medians_ = np.nan_to_num(self.medians_, nan=0.0)
        filled = np.where(np.isnan(values), self.medians_, values)
        self.means_ = filled.mean(axis=0)
        self.scales_ = filled.std(axis=0)
        self.scales_[self.scales_ < 1e-12] = 1.0
        self.n_features_in_ = len(self.features)
        self.n_output_features_ = 2*len(selected)
        return self

    def transform(self, x):
        values = self._raw(x)
        missing = np.isnan(values)
        values = np.clip(self._log(values), self.bounds_[0], self.bounds_[1])
        filled = np.where(np.isnan(values), self.medians_, values)
        return np.column_stack([(filled-self.means_)/self.scales_,
                                self.indicator_weight*missing.astype(float)])


def refined_specs():
    specs = [
        {"name": "mean_global", "family": "mean", "selectable": False, "priority": 99}
,
        {"name": "old_multioutput_tree", "family": "old_tree", "selectable": False, "p
riority": 99},
        {"name": "fixed_single_tree", "family": "single_tree", "selectable": False, "p
riority": 99},
        {"name": "stable_ridge_aging100", "family": "ridge", "features": "aging", "alp
ha": 100.0, "priority": 1, "selectable": True},
        {"name": "stable_ridge_all1000", "family": "ridge", "features": "all226", "alp
ha": 1000.0, "priority": 2, "selectable": True},
        {"name": "tail_ridge_aging100", "family": "ridge", "features": "aging", "alpha
": 100.0, "tail": True, "priority": 3, "selectable": True},
        {"name": "aux4_ridge10", "family": "aux", "features": "all226", "rank": 4, "al
pha": 10.0, "priority": 4, "selectable": True},
        {"name": "aux8_ridge10", "family": "aux", "features": "all226", "rank": 8, "al
pha": 10.0, "priority": 5, "selectable": True},
        {"name": "aux4_tail_ridge10", "family": "aux", "features": "all226", "rank": 4
, "alpha": 10.0, "tail": True, "priority": 6, "selectable": True},
        {"name": "aux4_smooth_kernel", "family": "aux_kernel", "features": "all226", "
rank": 4, "alpha": 1.0, "gamma_factor": .25, "priority": 7, "selectable": True},
        {"name": "style_center_within_ridge", "family": "within", "features": "all226"
, "center_alpha": 1000.0, "within_alpha": 100.0, "style_shrinkage": 5.0, "priority": 8
, "selectable": True},
    ]
    members = ["stable_ridge_all1000", "aux4_ridge10", "tail_ridge_aging100"]
    specs.append({"name": "equal3_supervised_ensemble", "family": "ensemble", "priorit
y": 9,
                  "selectable": True, "members": [dict(spec) for spec in specs if spec
["name"] in members],
```

src/q3_stale_refined.py（续）

```
                  "weights": [1/3, 1/3, 1/3]})
    return specs


class RefinedStaleModel(RegressorMixin, BaseEstimator):
    """All sensory values are training targets; inference uses chemistry only."""
    def __init__(self, features, calibrated_features, aging_features, target_names,
                 config, seed):
        self.features = features
        self.calibrated_features = calibrated_features
        self.aging_features = aging_features
        self.target_names = target_names
        self.config = config
        self.seed = seed

    def fit(self, x, y, styles):
        all_targets = y.loc[:, self.target_names].to_numpy(dtype=float) if hasattr(y,
"loc") else np.asarray(y, dtype=float)
        if all_targets.ndim != 2 or all_targets.shape[1] != len(self.target_names):
            raise ValueError("Training requires the declared sensory target matrix")
        self.target_position_ = self.target_names.index(TARGET)
        target = all_targets[:, self.target_position_]
        self.n_features_in_ = len(self.features)
        self.n_outputs_ = 1
        family = self.config["family"]
        if family == "mean":
            self.target_mean_ = float(target.mean())
            return self
        if family in ("old_tree", "single_tree"):
            self.imputer_ = SimpleImputer(strategy="median", add_indicator=True,
                                          keep_empty_features=True).fit(x)
            self.model_ = ExtraTreesRegressor(n_estimators=300, min_samples_leaf=3,
                max_features=.7, random_state=self.seed, n_jobs=1)
            self.model_.fit(self.imputer_.transform(x), all_targets if family == "old_
tree" else target)
            return self
        if family == "ensemble":
            self.members_ = [RefinedStaleModel(self.features, self.calibrated_features
,
                self.aging_features, self.target_names, member, self.seed).fit(x, y, s
tyles)
                for member in self.config["members"]]
            return self
        selected = self.aging_features if self.config["features"] == "aging" else None
        self.chemical_ = FixedChemicalTransform(self.features, self.calibrated_feature
s, selected).fit(x)
        z = self.chemical_.transform(x)
        base_weights = style_weights(styles)
        weights = base_weights.copy()
        self.tail_training_threshold_ = float(np.quantile(target, .8))
        if self.config.get("tail", False):
            weights *= np.where(target >= self.tail_training_threshold_, 2.0, 1.0)
            weights *= len(weights)/weights.sum()
        if family == "within":
            center = np.empty(len(target))
            values = np.asarray(styles)
            global_mean = np.average(target, weights=base_weights)
            for style in np.unique(values):
                mask = values == style
                shrinkage = mask.sum()/(mask.sum()+self.config["style_shrinkage"])
                center[mask] = global_mean+shrinkage*(target[mask].mean()-global_mean)
```

src/q3_stale_refined.py（续）

```
            self.center_ = Ridge(alpha=self.config["center_alpha"], solver="svd").fit(z, center, sample_weight=base_weights)
            self.within_ = Ridge(alpha=self.config["within_alpha"], solver="svd").fit(z, target-center, sample_weight=base_weights)
            return self
        if family in ("aux", "aux_kernel"):
            # Stale is explicitly excluded from auxiliary supervision.
            self.auxiliary_target_indices_ = [j for j in range(len(self.target_names)) if j != self.target_position_]
            self.auxiliary_target_names_ = [self.target_names[j] for j in self.auxiliary_target_indices_]
            if TARGET in self.auxiliary_target_names_:
                raise AssertionError("Primary target cannot supervise its auxiliary directions")
            auxiliary = all_targets[:, self.auxiliary_target_indices_]
            self.auxiliary_mean_ = np.average(auxiliary, axis=0, weights=base_weights)
            self.auxiliary_scale_ = np.sqrt(np.average((auxiliary-self.auxiliary_mean_)**2, axis=0, weights=base_weights))
            self.auxiliary_scale_[self.auxiliary_scale_ < 1e-12] = 1.0
            standardized_auxiliary = (auxiliary-self.auxiliary_mean_)/self.auxiliary_scale_
            self.auxiliary_ridge_ = Ridge(alpha=100.0, solver="svd").fit(z, standardized_auxiliary, sample_weight=base_weights)
            _, self.auxiliary_singular_values_, vt = np.linalg.svd(self.auxiliary_ridge_.coef_, full_matrices=False)
            self.directions_ = vt[:self.config["rank"]].T
            latent = z@self.directions_
            self.latent_mean_ = np.average(latent, axis=0, weights=base_weights)
            self.latent_scale_ = np.sqrt(np.average((latent-self.latent_mean_)**2, axis=0, weights=base_weights))
            self.latent_scale_[self.latent_scale_ < 1e-12] = 1.0
            z = (latent-self.latent_mean_)/self.latent_scale_
        if family == "aux_kernel":
            self.target_mean_ = float(np.average(target, weights=weights))
            self.model_ = KernelRidge(alpha=self.config["alpha"], kernel="rbf",
                                      gamma=self.config["gamma_factor"]/z.shape[1])
            self.model_.fit(z, target-self.target_mean_, sample_weight=weights)
        else:
            self.model_ = Ridge(alpha=self.config["alpha"], solver="svd").fit(z, target, sample_weight=weights)
        return self

    def predict(self, x):
        family = self.config["family"]
        if family == "mean":
            return np.full(len(x), self.target_mean_)
        if family in ("old_tree", "single_tree"):
            prediction = self.model_.predict(self.imputer_.transform(x))
            return prediction[:, self.target_position_] if family == "old_tree" else prediction
        if family == "ensemble":
            return np.mean([member.predict(x) for member in self.members_], axis=0)
        z = self.chemical_.transform(x)
        if family == "within":
            return self.center_.predict(z)+self.within_.predict(z)
        if family in ("aux", "aux_kernel"):
            z = (z@self.directions_-self.latent_mean_)/self.latent_scale_
        prediction = self.model_.predict(z)
        return prediction+self.target_mean_ if family == "aux_kernel" else prediction
```

supplement/q1_additional_checks.py

SHA-256: fef1d87fbcf839edc7270cf9fbcb25ed71fe5d27328e7c652d780b648cac4c93

```
"""Supplement the exploratory comparison with balanced groups and a NaN probe.

The shuffled group split had markedly unequal sample counts. This balanced
split was added after noticing that issue; both sets of results are retained.
"""
import json

import numpy as np
import pandas as pd
import sklearn
from sklearn.ensemble import ExtraTreesRegressor, GradientBoostingRegressor, HistGradientBoostingRegressor
from sklearn.model_selection import GroupKFold
from threadpoolctl import threadpool_limits

import q1_comparison as c


def main():
    x_small = np.array([[1., 2.], [2., np.nan], [np.nan, 4.], [4., 5.],
                        [5., 6.], [6., 7.], [7., 8.], [8., 9.]])
    y_small = np.arange(1., 9.)
    checks = {}
    for model in [GradientBoostingRegressor(n_estimators=3, random_state=1),
                  HistGradientBoostingRegressor(max_iter=3, min_samples_leaf=2,
                                                early_stopping=False, random_state=1),
                  ExtraTreesRegressor(n_estimators=3, random_state=1)]:
        name = type(model).__name__
        try:
            model.fit(x_small, y_small)
            pred = model.predict(np.array([[np.nan, 3.], [3., np.nan]]))
            checks[name] = {"fit_success": True, "predict_nan_success": bool(np.isfinite(pred).all()),
                            "predictions": pred.tolist()}
        except Exception as error:
            checks[name] = {"fit_success": False, "exception_type": type(error).__name__,
                            "message": str(error)}
    (c.OUT / "nan_support_check.json").write_text(json.dumps(
        {"sklearn": sklearn.__version__, "sample_n": 8, "checks": checks},
        ensure_ascii=False, indent=2), encoding="utf-8")
    meta, x, y = c.load_xy()
    rows, per_target = [], []
    predictions = {name: np.full(y.shape, np.nan) for name in ["mean", "extra_trees"]}
    fold_ids = np.zeros(len(x), dtype=int)
    for fold, (train, test) in enumerate(GroupKFold(3, shuffle=False).split(
            x, groups=meta.tasting_category_fine), 1):
        fold_ids[test] = fold
        assert not set(meta.iloc[train].tasting_category_fine) & set(meta.iloc[test].tasting_category_fine)
        for name in predictions:
            model = c.estimator(name, c.SEED + fold).fit(x.iloc[train], y.iloc[train])
            pred = model.predict(x.iloc[test])
            predictions[name][test] = pred
            rows.append({"protocol": "unseen_style_balanced_3fold", "model": name, "fold": fold,
                         "train_n": len(train), "test_n": len(test), **c.scores(y.iloc[test], pred)})
    overall = []
```

supplement/q1_additional_checks.py（续）

```
    for name, pred in predictions.items():
        overall.append({"protocol": "unseen_style_balanced_3fold", "model": name, **c.scores(y, pred)})
        for j, target in enumerate(y.columns):
            per_target.append({"model": name, "target": target, **c.scores(y.iloc[:, j], pred[:, j])})
        pd.concat([meta, pd.DataFrame({"fold": fold_ids}), pd.DataFrame(pred, columns=y.columns)], axis=1).to_csv(
            c.OUT / f"oof_unseen_style_balanced_3fold_{name}.csv", index=False, lineterminator="\n")
    for name, values in [("by_fold", rows), ("overall", overall), ("by_target", per_target)]:
        pd.DataFrame(values).to_csv(c.OUT / f"balanced_group_metrics_{name}.csv", index=False, lineterminator="\n")
    print(pd.DataFrame(overall).to_string(index=False))


if __name__ == "__main__":
    with threadpool_limits(limits=1):
        main()
```

supplement/q1_comparison.py

SHA-256: 02febdf363a347d68891531d88abcfa1396db9679155d55238a3b650ba091ec3

```
"""Exploratory Q1 checks prompted by the supplied suggestions.

This script writes the supplementary Q1 comparisons. It does not
replace the frozen experiment, choose a model for submission, or recover
original 0-6 sensory labels. Parameters are fixed before this script is run.
"""
import hashlib
import json
import platform
import sys
import time
from pathlib import Path

HERE = Path(__file__).resolve().parent
SOLUTION = HERE.parent
sys.path.insert(0, str(SOLUTION / "src"))

import numpy as np
import pandas as pd
import sklearn
from sklearn.dummy import DummyRegressor
from sklearn.ensemble import ExtraTreesRegressor, GradientBoostingRegressor, HistGradientBoostingRegressor
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import GroupKFold, KFold
from sklearn.multioutput import MultiOutputRegressor
from sklearn.pipeline import Pipeline
from threadpoolctl import threadpool_limits

from common import data_dir, load_xy

SEED = 20261024
OUT = HERE / "results/q1"


def scores(actual, predicted):
    return {"mae": float(mean_absolute_error(actual, predicted)),
            "rmse": float(np.sqrt(mean_squared_error(actual, predicted))),
            "macro_r2": float(r2_score(actual, predicted, multioutput="uniform_average"))}


def estimator(name, seed):
    if name == "mean":
        return DummyRegressor(strategy="mean")
    if name == "extra_trees":
        model = ExtraTreesRegressor(n_estimators=300, min_samples_leaf=3,
                                    max_features=.7, random_state=seed, n_jobs=1)
    elif name == "gbr_per_target":
        model = MultiOutputRegressor(GradientBoostingRegressor(
            n_estimators=100, learning_rate=.05, max_depth=2, min_samples_leaf=5,
            loss="squared_error", random_state=seed), n_jobs=1)
    elif name == "hgb_per_target":
        model = MultiOutputRegressor(HistGradientBoostingRegressor(
            max_iter=100, learning_rate=.05, max_leaf_nodes=7, min_samples_leaf=10,
            l2_regularization=1., early_stopping=False, random_state=seed), n_jobs=1)
    else:
        raise ValueError(name)
```

supplement/q1_comparison.py（续）

```
    return Pipeline([("imputer", SimpleImputer(strategy="median", add_indicator=True,
                                               keep_empty_features=True)), ("model", model)])


def main():
    OUT.mkdir(parents=True, exist_ok=True)
    meta, x, y = load_xy()
    y_np = y.to_numpy()
    random_splits = list(KFold(3, shuffle=True, random_state=SEED).split(x))
    group_splits = list(GroupKFold(3, shuffle=True, random_state=SEED).split(
        x, groups=meta.tasting_category_fine))
    summary_rows, fold_rows, target_rows, masks = [], [], [], []
    started = time.perf_counter()
    for protocol, splits, names in [
        ("random_3fold", random_splits, ["mean", "extra_trees", "gbr_per_target", "hgb_per_target"]),
        ("unseen_style_3fold", group_splits, ["mean", "extra_trees"]),
    ]:
        fold_ids = np.zeros(len(x), dtype=int)
        predictions = {name: np.full(y.shape, np.nan) for name in names}
        stressed = {rate: np.full((20, *y.shape), np.nan) for rate in (.1, .2)} if protocol == "random_3fold" else {}
        timings = {name: 0. for name in names}
        for fold, (train, test) in enumerate(splits, 1):
            fold_ids[test] = fold
            if protocol == "unseen_style_3fold":
                assert not set(meta.iloc[train].tasting_category_fine) & set(meta.iloc[test].tasting_category_fine)
            for name in names:
                start = time.perf_counter()
                model = estimator(name, SEED + fold)
                model.fit(x.iloc[train], y.iloc[train])
                pred = model.predict(x.iloc[test])
                timings[name] += time.perf_counter() - start
                predictions[name][test] = pred
                if name != "mean":
                    np.testing.assert_allclose(model.named_steps["imputer"].statistics_,
                                               np.nanmedian(x.iloc[train], axis=0))
                fold_rows.append({"protocol": protocol, "model": name, "fold": fold,
                                  "train_n": len(train), "test_n": len(test),
                                  "test_styles": "|".join(sorted(meta.iloc[test].tasting_category_fine.unique())),
                                  **scores(y.iloc[test], pred)})
                if protocol == "random_3fold" and name == "extra_trees":
                    for repeat in range(20):
                        uniforms = np.random.default_rng(SEED + 10000 + 100 * repeat + fold).random((len(test), x.shape[1]))
                        for rate in (.1, .2):
                            mask = (uniforms < rate) & x.iloc[test].notna().to_numpy()
                            stressed[rate][repeat, test] = model.predict(x.iloc[test].mask(mask))
                            masks.append({"repeat": repeat + 1, "fold": fold, "rate": rate,
                                          "additional_hidden": int(mask.sum())})
                print(f"{protocol} fold {fold}/3 {name}: RMSE={scores(y.iloc[test], pred)['rmse']:.6f}", flush=True)
        for name, pred in predictions.items():
            assert np.isfinite(pred).all()
            summary_rows.append({"protocol": protocol, "model": name,
```

supplement/q1_comparison.py（续）

```
                                    "fit_and_predict_seconds": timings[name], **scores(y, pred)})
            for j, target in enumerate(y.columns):
                target_rows.append({"protocol": protocol, "model": name,
                                    "target": target, **scores(y_np[:, j], pred[:, j])})
            pd.concat([meta, pd.DataFrame({"fold": fold_ids}), pd.DataFrame(pred, columns=y.columns)], axis=1).to_csv(
                OUT / f"oof_{protocol}_{name}.csv", index=False, lineterminator="\n")
        pd.concat([meta, pd.DataFrame({"fold": fold_ids})], axis=1).to_csv(
            OUT / f"fold_assignment_{protocol}.csv", index=False, lineterminator="\n")
        if stressed:
            np.savez_compressed(OUT / "test_mask_predictions.npz",
                                rate10=stressed[.1], rate20=stressed[.2])
        for rate, values in stressed.items():
            for repeat in range(20):
                summary_rows.append({"protocol": "random_3fold_test_mask", "model": "extra_trees",
                                     "rate": rate, "repeat": repeat + 1, **scores(y, values[repeat])})
    for filename, rows in [("metrics_overall.csv", summary_rows), ("metrics_by_fold.csv", fold_rows),
                           ("metrics_by_target.csv", target_rows), ("mask_counts.csv", masks)]:
        pd.DataFrame(rows).to_csv(OUT / filename, index=False, lineterminator="\n")
    manifest = {"purpose": "Supplementary post-hoc comparison; primary experiment retained", "seed": SEED,
                "samples": len(x), "features": x.shape[1], "targets": list(y.columns),
                "target_scale": "Attachment 4 standardized continuous scale, not original 0-6 scores",
                "total_seconds": time.perf_counter() - started,
                "protocols": {"random_3fold": "Same 3 shuffled folds and training-only imputation for all four models",
                              "unseen_style_3fold": "Whole fine styles excluded from training; Mean and ExtraTrees only",
                              "stress": "20 test-mask draws per 10%/20% rate; masks are paired and nested; models remain unchanged"},
                "models": {name: repr(estimator(name, SEED + 1)) for name in ["mean", "extra_trees", "gbr_per_target", "hgb_per_target"]},
                "input_sha256": {filename: hashlib.sha256((data_dir()/filename).read_bytes()).hexdigest() for filename in ["1.csv", "4.csv"]},
                "versions": {"python": platform.python_version(), "numpy": np.__version__,
                             "pandas": pd.__version__, "sklearn": sklearn.__version__},
                "limitations": ["Same 250 beers as development; a new split is not an independent data set.",
                                "One fixed parameter set per model; not a tuned-family superiority claim.",
                                "45-output ExtraTrees shares splits; boosting models fit each target separately.",
                                "Random and style-group protocols answer different deployment questions.",
                                "MCAR test masking does not model detection limits, training-set missingness, or instrument failures.",
                                "Attachment creators' precomputed global transformations cannot be undone."]}
    (OUT / "manifest.json").write_text(json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8")
    print(pd.DataFrame(summary_rows).head(6).to_string(index=False), flush=True)
    print(f"Completed in {manifest['total_seconds']:.2f} seconds", flush=True)
```

supplement/q1_comparison.py（续）

```
147
148
149 if __name__ == "__main__":
150     with threadpool_limits(limits=1):
151         main()
```

supplement/q2_permutation.py

SHA-256: 603de849964bae027c68f5bd6735070bb2dfc742fb5586e6f20b0301a9373a77

```
"""Held-out permutation dependence for all 45 sensory targets.

The original five splits and tree parameters are retained. Permutations act
on validation inputs only, including each feature's missing indicator.
Correlated modules are learned separately from each training fold.
"""
import hashlib
import importlib.util
import json
from pathlib import Path
import sys
import time

import numpy as np
import pandas as pd
from sklearn.model_selection import KFold
from threadpoolctl import threadpool_limits

HERE = Path(__file__).resolve().parent
ROOT = HERE.parent
sys.path.insert(0, str(ROOT / "src"))
from common import SEED, data_dir, load_xy

OUT = HERE / "results/q2_permutation"
REPEATS = 5


def load_module(name, filename):
    spec = importlib.util.spec_from_file_location(name, ROOT / "src" / filename)
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)
    return module


def components(training):
    correlation = training.corr(method="spearman").fillna(0).to_numpy()
    adjacency = np.abs(correlation) >= .7
    remaining = set(range(training.shape[1]))
    groups = []
    while remaining:
        current = [min(remaining)]
        remaining.remove(current[0])
        group = []
        while current:
            j = current.pop()
            group.append(j)
            neighbors = sorted(remaining.intersection(np.flatnonzero(adjacency[j])))
            remaining.difference_update(neighbors)
            current.extend(neighbors)
        groups.append(sorted(group))
    assert sorted(j for group in groups for j in group) == list(range(training.shape[1
]))
    return groups


def predict_permutations(tree, base, groups, columns, permutation, truth, baseline):
    result = np.empty((len(groups), truth.shape[1]))
    for start in range(0, len(groups), 32):
        batch = groups[start:start + 32]
```

supplement/q2_permutation.py（续）

```
        stacked = np.tile(base, (len(batch), 1))
        n = len(base)
        for k, group in enumerate(batch):
            selected = [col for j in group for col in columns[j]]
            stacked[k*n:(k+1)*n, selected] = base[np.ix_(permutation, selected)]
        prediction = tree.predict(stacked).reshape(len(batch), n, truth.shape[1])
        result[start:start + len(batch)] = ((prediction - truth) ** 2).mean(axis=1) - baseline
    return result


def main():
    started = time.perf_counter()
    OUT.mkdir(parents=True, exist_ok=True)
    q1 = load_module("q1_permutation_baseline", "02_q1_baseline.py")
    q2 = load_module("q2_permutation_candidates", "03_q2_network.py")
    meta, x, y = load_xy()
    features, targets = list(x), list(y)
    candidates = [j for j, feature in enumerate(features) if feature not in q2.EXCLUDE_X]
    assert len(candidates) == 218
    y_values = y.to_numpy()
    delta = np.empty((5, REPEATS, len(features), len(targets)))
    oof = np.full(y.shape, np.nan)
    fold_ids = np.zeros(len(x), dtype=int)
    fold_sizes = []
    module_rows, membership = [], []
    check_features = ["ethyl_acetate", "X4VG", "ethanol..v.v."]
    check_predictions = np.empty((5, REPEATS, len(check_features), 50, len(targets)))
    check_rows = []
    original = pd.read_csv(ROOT / "outputs/q1/oof_extra_trees.csv")
    assert original.beer.tolist() == meta.beer.tolist()
    for fold, (train, test) in enumerate(KFold(5, shuffle=True, random_state=SEED).split(x)):
        fold_ids[test] = fold + 1
        fold_sizes.append(len(test))
        assert original.iloc[test].fold.eq(fold + 1).all()
        model = q1.tree_pipeline(SEED + fold + 1).fit(x.iloc[train], y.iloc[train])
        imputer, tree = model.named_steps["imputer"], model.named_steps["model"]
        np.testing.assert_allclose(imputer.statistics_, np.nanmedian(x.iloc[train], axis=0))
        base = imputer.transform(x.iloc[test])
        oof[test] = tree.predict(base)
        truth = y_values[test]
        baseline = ((truth - oof[test]) ** 2).mean(axis=0)
        columns = [[j] for j in range(len(features))]
        for k, j in enumerate(imputer.indicator_.features_):
            columns[j].append(len(features) + k)
        groups = components(x.iloc[train])
        group_delta = np.empty((REPEATS, len(groups), len(targets)))
        for g, group in enumerate(groups):
            membership.append({"fold": fold + 1, "module": g + 1, "size": len(group),
                               "members": " | ".join(features[j] for j in group)})
        for repeat in range(REPEATS):
            permutation = np.random.default_rng(SEED + 50000 + 100*fold + repeat).permutation(len(test))
            delta[fold, repeat] = predict_permutations(
                tree, base, [[j] for j in range(len(features))], columns, permutation, truth, baseline)
            group_delta[repeat] = predict_permutations(
                tree, base, groups, columns, permutation, truth, baseline)
```

supplement/q2_permutation.py（续）

```
            for k, feature in enumerate(check_features):
                j = features.index(feature)
                raw = x.iloc[test].copy()
                raw.loc[:, feature] = x.iloc[test].iloc[permutation][feature].to_numpy()
                predicted = model.predict(raw)
                check_predictions[fold, repeat, k] = predicted
                independent = ((predicted - truth) ** 2).mean(axis=0) - baseline
                np.testing.assert_allclose(independent, delta[fold, repeat, j], rtol=1e-10, atol=1e-12)
        for g, group in enumerate(groups):
            for k, target in enumerate(targets):
                module_rows.append({"fold": fold + 1, "module": g + 1, "size": len(group),
                                    "target": target, "delta_mse": group_delta[:, g, k].mean(),
                                    "repeat_sd": group_delta[:, g, k].std(ddof=1)})
        check_rows.append(test)
        print(f"Permutation fold {fold + 1}/5: {len(groups)} training modules; held-out rows={len(test)}", flush=True)
    assert np.isfinite(delta).all() and np.isfinite(oof).all()
    by_fold = delta.mean(axis=1)
    mean = np.average(by_fold, axis=0, weights=fold_sizes)
    sd = by_fold.std(axis=0, ddof=1)
    positive = (by_fold > 0).sum(axis=0)
    overall, fold_rows, top_rows, comparisons = [], [], [], []
    original_top = pd.read_csv(ROOT / "outputs/q2/top3_compounds.csv")
    for k, target in enumerate(targets):
        ordered = sorted(candidates, key=lambda j: (-mean[j, k], j))
        ranks = {j: rank for rank, j in enumerate(ordered, 1)}
        chosen = ordered[:3]
        existing = original_top.loc[original_top.sensory_target.eq(target)].sort_values("rank").compound.tolist()
        chosen_names = [features[j] for j in chosen]
        common = set(chosen_names) & set(existing)
        comparisons.append({"sensory_target": target, "spearman_top3": " | ".join(existing),
                            "permutation_top3": " | ".join(chosen_names), "overlap": len(common),
                            "jaccard": len(common) / len(set(chosen_names) | set(existing)),
                            "all_three_positive_mean": bool((mean[chosen, k] > 0).all())})
        for j, feature in enumerate(features):
            row = {"sensory_target": target, "feature": feature, "chemical_candidate": j in ranks,
                   "candidate_rank": ranks.get(j), "delta_mse": mean[j, k],
                   "fold_sd": sd[j, k], "positive_folds": int(positive[j, k])}
            overall.append(row)
            if j in chosen:
                top_rows.append(row)
            for fold in range(5):
                fold_rows.append({"fold": fold + 1, "sensory_target": target, "feature": feature,
                                  "delta_mse": by_fold[fold, j, k],
                                  "repeat_sd": delta[fold, :, j, k].std(ddof=1)})
    for name, rows in [("importance", overall), ("importance_by_fold", fold_rows),
                       ("top3", top_rows), ("top3_comparison", comparisons),
                       ("module_importance_by_fold", module_rows), ("module_membership", membership)]:
        pd.DataFrame(rows).to_csv(OUT / f"{name}.csv", index=False, lineterminator="\n
```

supplement/q2_permutation.py（续）

```
    ", float_format="%.12g")
162     pd.concat([meta, pd.DataFrame({"fold": fold_ids}), pd.DataFrame(oof, columns=targe
    ts)], axis=1).to_csv(
163         OUT / "oof.csv", index=False, lineterminator="\n")
164     np.savez_compressed(OUT / "permutation_draws.npz", delta_mse=delta,
165                         fold_ids=fold_ids, features=np.array(features), targets=np.arr
    ay(targets),
166                         check_predictions=check_predictions, check_features=np.array(c
    heck_features),
167                         check_test_rows=np.array(check_rows))
168     summary = {"scope": "Supplementary post-hoc model-dependence analysis; not causal
    effects or a new test dataset.",
169                "seed": SEED, "folds": 5, "repeats": REPEATS, "features": len(features)
    , "targets": len(targets),
170                "chemical_candidates": len(candidates), "rmse": q1.scores(y_values, oof
    )["rmse"],
171                "frozen_oof_max_abs_difference": float(np.max(np.abs(oof-original[targe
    ts].to_numpy()))),
172                "frozen_oof_rmse": q1.scores(y_values, original[targets].to_numpy())["r
    mse"],
173                "modules_per_fold": [sum(r["fold"] == f for r in membership) for f in r
    ange(1, 6)],
174                "mean_jaccard_with_spearman": float(np.mean([r["jaccard"] for r in comp
    arisons])),
175                "targets_with_no_top3_overlap": sum(r["overlap"] == 0 for r in comparis
    ons),
176                "targets_with_all_positive_top3_means": sum(r["all_three_positive_mean"
    ] for r in comparisons),
177                "input_sha256": {name: hashlib.sha256((data_dir()/name).read_bytes()).h
    exdigest() for name in ["1.csv", "4.csv"]},
178                "elapsed_seconds": time.perf_counter() - started,
179                "limitations": ["Same development beers; follow-up designed after viewi
    ng the main results.",
180                                "Correlated and derived inputs can substitute for each
    other; individual permutation may break chemical consistency.",
181                                "Training-fold correlation modules differ across folds
    and are reported separately, not pooled as identical objects.",
182                                "Positive folds and fold SD describe stability, not ind
    ependent significance tests or causal contributions."]}
183     (OUT / "summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2
    ) + "\n", encoding="utf-8")
184     print(json.dumps(summary, ensure_ascii=False, indent=2), flush=True)
185
186
187 if __name__ == "__main__":
188     with threadpool_limits(limits=1):
189         main()
```

supplement/q2_stratified_bootstrap.py

SHA-256: af7ab7e04241b297b306c35215c15472b1ba69330ff096afb2d19fc8d65b5c50

```
"""Exploratory Q2 bootstrap with the observed style counts held fixed.

Writes supplementary results in results/q2_stratified. Existing Q2 rankings and frozen outputs remain
the reference. Run with the solution's requirements-lock.txt environment.
"""
from __future__ import annotations

import hashlib
import importlib.metadata
import importlib.util
import json
from pathlib import Path
import platform
import time

import numpy as np
import pandas as pd


HERE = Path(__file__).resolve().parent
SOLUTION = HERE.parent
OUT = HERE / "results/q2_stratified"
REPOSITORY = SOLUTION.parent
SEED = 20261023
BOOTSTRAPS = 200


def sha256(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()


def csv(frame, name):
    frame.to_csv(OUT / name, index=False, encoding="utf-8-sig",
                 float_format="%.10g", lineterminator="\n")


def set_score(a, b):
    a, b = set(a), set(b)
    return len(a & b) / len(a | b)


def main():
    OUT.mkdir(parents=True, exist_ok=True)
    started = time.perf_counter()
    helper_path = SOLUTION / "src/03_q2_network.py"
    spec = importlib.util.spec_from_file_location("q2_existing_helper", helper_path)
    helper = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(helper)
    files = [helper_path, helper.DATA / "1.csv", helper.DATA / "4.csv",
             SOLUTION / "outputs/q2/all_candidate_stability.csv",
             SOLUTION / "outputs/q2/top3_compounds.csv",
             SOLUTION / "outputs/q2/summary.json"]
    before_hashes = {p.relative_to(REPOSITORY).as_posix(): sha256(p) for p in files}
    chemical = pd.read_csv(helper.DATA / "1.csv")
    sensory = pd.read_csv(helper.DATA / "4.csv")
    candidates = [c for c in chemical.columns[3:] if c not in helper.EXCLUDE_X]
    targets = [c for c in sensory.columns[3:] if c not in helper.EXCLUDE_Y]
    merged = chemical.merge(sensory[["beer", *targets]], on="beer",
```

supplement/q2_stratified_bootstrap.py（续）

```
                              validate="one_to_one", how="inner")
    assert merged.shape[0] == 250 and len(candidates) == 218 and len(targets) == 45
    x = merged[candidates].to_numpy(float)
    y = merged[targets].to_numpy(float)
    rho = helper.cross_correlation(x, y, "spearman")
    initial_top = helper.top3(rho)
    original = pd.read_csv(SOLUTION / "outputs/q2/all_candidate_stability.csv")
    original_top = pd.read_csv(SOLUTION / "outputs/q2/top3_compounds.csv")
    for t, target in enumerate(targets):
        expected = original_top[original_top.sensory_target.eq(target)].sort_values("r
ank")
        assert expected.compound.tolist() == [candidates[j] for j in initial_top[:, t]
]
        for j in range(len(candidates)):
            existing = original[(original.sensory_target == target)
                                & (original.compound == candidates[j])]
            assert len(existing) == 1
            assert abs(float(existing.spearman_rho.iloc[0]) - rho[j, t]) < 1e-9

    labels = merged.tasting_category_fine.astype(str).to_numpy()
    # First appearance order is explicit, rather than relying on groupby sorting.
    styles = list(pd.unique(labels))
    style_indices = [np.flatnonzero(labels == style) for style in styles]
    style_sizes = [len(indices) for indices in style_indices]
    counts = np.zeros((len(candidates), len(targets)), dtype=np.int64)
    sampled_indices = np.empty((BOOTSTRAPS, len(merged)), dtype=np.int16)
    winners = np.empty((BOOTSTRAPS, 3, len(targets)), dtype=np.int16)
    rng = np.random.default_rng(SEED)
    for b in range(BOOTSTRAPS):
        rows = np.concatenate([rng.choice(indices, size=len(indices), replace=True)
                               for indices in style_indices])
        assert all(int(np.sum(labels[rows] == style)) == size
                   for style, size in zip(styles, style_sizes))
        selected = helper.top3(helper.cross_correlation(x[rows], y[rows], "spearman"))
        counts[selected, np.arange(len(targets))] += 1
        sampled_indices[b] = rows
        winners[b] = selected
    assert np.all(counts.sum(axis=0) == 3 * BOOTSTRAPS)
    frequency = counts / BOOTSTRAPS
    extra = []
    for t, target in enumerate(targets):
        for j, candidate in enumerate(candidates):
            extra.append({"sensory_target": target, "compound": candidate,
                          "stratified_top3_count": int(counts[j, t]),
                          "stratified_top3_frequency": float(frequency[j, t])})
    comparison = original.merge(pd.DataFrame(extra), on=["sensory_target", "compound"]
,
                                validate="one_to_one")
    comparison["frequency_difference_stratified_minus_global"] = (
        comparison.stratified_top3_frequency - comparison.bootstrap_top3_frequency)
    csv(comparison, "all_candidates_comparison.csv")
    top = comparison[comparison["rank"].le(3)].copy()
    csv(top, "original_top3_comparison.csv")
    set_rows = []
    for t, target in enumerate(targets):
        base_set = [candidates[j] for j in initial_top[:, t]]
        global_freq = original[original.sensory_target.eq(target)].set_index("compound
")
        global_order = global_freq.reindex(candidates).bootstrap_top3_frequency.to_num
py()
        global_set = [candidates[j] for j in np.argsort(-global_order, kind="stable")[
```

supplement/q2_stratified_bootstrap.py（续）

```
    :3]]
115         stratified_set = [candidates[j] for j in np.argsort(-frequency[:, t], kind="st
    able")[:3]]
116         per_draw = [helper.jaccard(initial_top[:, t], winners[b, :, t])
117                     for b in range(BOOTSTRAPS)]
118         set_rows.append({
119             "sensory_target": target,
120             "original_full_sample_spearman_top3": " | ".join(base_set),
121             "global_frequency_ranked_top3": " | ".join(global_set),
122             "stratified_frequency_ranked_top3": " | ".join(stratified_set),
123             "jaccard_original_vs_stratified_frequency": set_score(base_set, stratified
    _set),
124             "jaccard_global_frequency_vs_stratified_frequency": set_score(global_set,
    stratified_set),
125             "original_full_sample_set_changed": False,
126             "frequency_ranked_set_changed_global_vs_stratified": set(global_set) != se
    t(stratified_set),
127             "stratified_draw_mean_jaccard_with_original": float(np.mean(per_draw)),
128             "stratified_draw_exact_original_top3_fraction": float(np.mean(np.array(per
    _draw) == 1)),
129             "original_top3_stratified_frequency_ge_0p8": int(np.sum(frequency[initial_
    top[:, t], t] >= .8)),
130         })
131     sets = pd.DataFrame(set_rows)
132     csv(sets, "target_set_comparison.csv")
133     cases = comparison[((comparison.sensory_target == "X4vg") & (comparison.compound =
    = "X4VG"))
134                        | ((comparison.sensory_target == "F_esters_ethac") & (compariso
    n.compound == "ethyl_acetate"))
135                        | ((comparison.sensory_target == "F_malt_all") & (comparison.co
    mpound == "ethanol..v.v."))].copy()
136     assert len(cases) == 3
137     csv(cases, "case_studies.csv")
138     np.savez_compressed(OUT / "draw_indices_and_selections.npz", row_indices=sampled_i
    ndices,
139                         selected_feature_indices=winners,
140                         candidate_order=np.asarray(candidates), target_order=np.asarra
    y(targets),
141                         beer_order=merged.beer.to_numpy(), style_order=np.asarray(styl
    es))
142     after_hashes = {p.relative_to(REPOSITORY).as_posix(): sha256(p) for p in files}
143     assert before_hashes == after_hashes, "An input or frozen Q2 file changed during t
    he run."
144     summary = {
145         "scope": "Exploratory follow-up after the original Q2 results were examined; n
    o original ranking or frozen result changed.",
146         "seed": SEED, "bootstraps": BOOTSTRAPS,
147         "resampling": "Independently resample each style with replacement, retaining i
    ts observed count; concatenate styles in their first-appearance order.",
148         "n_beers": 250, "n_candidate_compounds": 218, "n_targets": 45,
149         "style_counts": dict(zip(styles, style_sizes)),
150         "n_styles": len(styles), "minimum_style_n": min(style_sizes),
151         "singleton_styles": [style for style, size in zip(styles, style_sizes) if size
     == 1],
152         "same_helper": "solution_a/src/03_q2_network.py::cross_correlation, top3, jacc
    ard",
153         "original_full_sample_top3_exactly_matched": True,
154         "original_full_sample_ranking_changed": False,
155         "original_top3_positions": len(top),
156         "original_top3_global_frequency_ge_0p8": int(top.bootstrap_top3_frequency.ge(.
    8).sum()),
```

supplement/q2_stratified_bootstrap.py（续）

```
        "original_top3_stratified_frequency_ge_0p8": int(top.stratified_top3_frequency.ge(.8).sum()),
        "all_candidates_stratified_frequency_ge_0p8": int(comparison.stratified_top3_frequency.ge(.8).sum()),
        "original_top3_mean_frequency_global": float(top.bootstrap_top3_frequency.mean()),
        "original_top3_mean_frequency_stratified": float(top.stratified_top3_frequency.mean()),
        "original_top3_median_frequency_stratified": float(top.stratified_top3_frequency.median()),
        "original_top3_mean_abs_frequency_change": float(top.frequency_difference_stratified_minus_global.abs().mean()),
        "original_top3_max_abs_frequency_change": float(top.frequency_difference_stratified_minus_global.abs().max()),
        "frequency_ranked_top3_sets_changed": int(sets.frequency_ranked_set_changed_global_vs_stratified.sum()),
        "frequency_ranked_top3_mean_jaccard_global_vs_stratified": float(sets.jaccard_global_frequency_vs_stratified_frequency.mean()),
        "targets_whose_stratified_frequency_ranked_set_differs_from_original": int(sets.jaccard_original_vs_stratified_frequency.lt(1).sum()),
        "original_vs_stratified_frequency_ranked_mean_jaccard": float(sets.jaccard_original_vs_stratified_frequency.mean()),
        "stratified_draw_mean_jaccard_with_original": float(sets.stratified_draw_mean_jaccard_with_original.mean()),
        "max_single_frequency_monte_carlo_standard_error": float(np.sqrt(.25 / BOOTSTRAPS)),
        "frequency_set_definition": "Top three ranked by bootstrap inclusion frequency, input-column-order tie break. This sensitivity summary is not a replacement for the original full-sample Spearman Top-3.",
        "cases": cases[["sensory_target", "compound", "rank", "spearman_rho", "bootstrap_top3_frequency", "stratified_top3_frequency", "frequency_difference_stratified_minus_global", "within_style_spearman_rho"]].to_dict("records"),
        "input_sha256_before": before_hashes, "input_sha256_after": after_hashes,
        "inputs_and_frozen_q2_unchanged": before_hashes == after_hashes,
        "python": platform.python_version(),
        "package_versions": {p: importlib.metadata.version(p) for p in ["numpy", "pandas", "scipy", "scikit-learn"]},
        "elapsed_seconds": time.perf_counter() - started,
        "limitations": [
            "Preserving the observed style mixture does not remove between-style confounding.",
            "The real sample count remains 250; bootstrap draws are not new beers.",
            "Rare styles carry little information about within-style uncertainty; a singleton style would always resample its sole row and cannot estimate within-style variation. There are no singleton styles in these data (minimum n=3).",
            "The identical seed does not make these draws paired with the old global-row bootstrap, because their sampling algorithms differ.",
            "Differences of a few percentage points are subject to Monte Carlo variation, and no significance claim is made for a frequency difference.",
            "No causal inference, false-discovery guarantee, or held-out predictive validation is provided by this experiment.",
        ],
    }
    (OUT / "summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2) + "\n", encoding="utf-8", newline="\n")
    print(json.dumps({k: v for k, v in summary.items() if k not in ["input_sha256_before", "input_sha256_after", "style_counts"]}, ensure_ascii=False, indent=2))


if __name__ == "__main__":
    main()
```

supplement/verify_supplement.py

SHA-256: 420d2a03ba6eb1b0e2167c45f0fa39eeb6180a97f45a4258c0aa2d6a40f0484a

```
"""Read-only numerical checks for the supplementary analyses."""
import hashlib
import importlib.util
import json
from pathlib import Path
import sys

import numpy as np
import pandas as pd
from scipy.sparse.csgraph import connected_components
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import KFold

HERE = Path(__file__).resolve().parent
ROOT = HERE.parent
sys.path.insert(0, str(ROOT / "src"))
from common import SEED, data_dir, load_xy


def module(filename):
    spec = importlib.util.spec_from_file_location("supplement_check_" + filename, ROOT / "src" / filename)
    result = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(result)
    return result


def close(a, b):
    np.testing.assert_allclose(a, b, rtol=1e-8, atol=1e-10)


def scores(actual, predicted):
    return dict(mae=mean_absolute_error(actual, predicted),
                rmse=np.sqrt(mean_squared_error(actual, predicted)),
                macro_r2=r2_score(actual, predicted, multioutput="uniform_average"))


def main():
    meta, x, y = load_xy()
    targets = list(y)
    checks = []
    q1 = HERE / "results/q1"
    metrics = pd.read_csv(q1 / "metrics_overall.csv")
    balanced = pd.read_csv(q1 / "balanced_group_metrics_overall.csv")
    tested = 0
    for path in q1.glob("oof_*.csv"):
        frame = pd.read_csv(path)
        assert frame.beer.tolist() == meta.beer.tolist()
        predicted = frame[targets].to_numpy()
        assert np.isfinite(predicted).all()
        name = path.stem.removeprefix("oof_")
        if "balanced" in name:
            model = "extra_trees" if name.endswith("extra_trees") else "mean"
            row = balanced.loc[balanced.model.eq(model)].iloc[0]
        else:
            protocol = "unseen_style_3fold" if name.startswith("unseen_style") else "random_3fold"
            model = name.removeprefix(protocol + "_")
            row = metrics.loc[metrics.protocol.eq(protocol) & metrics.model.eq(model)]
```

supplement/verify_supplement.py（续）

```
    .iloc[0]
        for key, value in scores(y, predicted).items():
            close(row[key], value)
        for fold in sorted(frame.fold.unique()):
            test = frame.fold.eq(fold).to_numpy()
            if "unseen_style" in name:
                assert not set(meta.loc[test, "tasting_category_fine"]) & set(meta.loc[~test, "tasting_category_fine"])
            if model == "mean":
                close(predicted[test], np.tile(y.loc[~test].mean().to_numpy(), (test.sum(), 1)))
        tested += 1
    assert tested == 8
    draws = np.load(q1 / "test_mask_predictions.npz")
    for rate, field in [(.1, "rate10"), (.2, "rate20")]:
        assert draws[field].shape == (20, 250, 45)
        for repeat, predicted in enumerate(draws[field], 1):
            row = metrics.loc[metrics.protocol.eq("random_3fold_test_mask") & metrics.rate.eq(rate)
                              & metrics["repeat"].eq(repeat)].iloc[0]
            for key, value in scores(y, predicted).items():
                close(row[key], value)
    checks.append({"name": "q1_oof_metrics_and_group_exclusion", "oof_tables": tested, "mask_draws": 40})

    q2 = HERE / "results/q2_stratified"
    helper = module("03_q2_network.py")
    saved = np.load(q2 / "draw_indices_and_selections.npz")
    chemical = pd.read_csv(data_dir() / "1.csv")
    sensory = pd.read_csv(data_dir() / "4.csv")
    merged = chemical.merge(sensory[["beer", *targets]], on="beer", validate="one_to_one")
    candidates = saved["candidate_order"].tolist()
    candidate_targets = saved["target_order"].tolist()
    xx, yy = merged[candidates].to_numpy(), merged[candidate_targets].to_numpy()
    assert saved["beer_order"].tolist() == merged.beer.tolist()
    labels = merged.tasting_category_fine.to_numpy()
    counts = np.zeros((218, 45), dtype=int)
    for rows, winners in zip(saved["row_indices"], saved["selected_feature_indices"]):
        assert pd.Series(labels[rows]).value_counts().to_dict() == pd.Series(labels).value_counts().to_dict()
        counts[winners, np.arange(45)] += 1
    for i in [0, 99, 199]:
        rows = saved["row_indices"][i]
        np.testing.assert_array_equal(helper.top3(helper.cross_correlation(xx[rows], yy[rows], "spearman")),
                                      saved["selected_feature_indices"][i])
    table = pd.read_csv(q2 / "all_candidates_comparison.csv")
    for k, target in enumerate(candidate_targets):
        subset = table.loc[table.sensory_target.eq(target)].set_index("compound").loc[candidates]
        close(subset.stratified_top3_count, counts[:, k])
        close(subset.stratified_top3_frequency, counts[:, k] / 200)
    checks.append({"name": "q2_stratified_draws_and_frequencies", "draws": 200, "candidate_target_pairs": 9810})

    perm = HERE / "results/q2_permutation"
    arrays = np.load(perm / "permutation_draws.npz")
    delta = arrays["delta_mse"]
    assert delta.shape == (5, 5, 231, 45) and np.isfinite(delta).all()
    frame = pd.read_csv(perm / "oof.csv")
```

supplement/verify_supplement.py（续）

```
    assert frame.beer.tolist() == meta.beer.tolist()
    importance = pd.read_csv(perm / "importance.csv")
    fold_means = delta.mean(axis=1)
    for k, target in enumerate(targets):
        part = importance.loc[importance.sensory_target.eq(target)].set_index("feature
").loc[x.columns]
        close(part.delta_mse, fold_means[:, :, k].mean(axis=0))
        close(part.fold_sd, fold_means[:, :, k].std(axis=0, ddof=1))
        close(part.positive_folds, (fold_means[:, :, k] > 0).sum(axis=0))
    membership = pd.read_csv(perm / "module_membership.csv")
    for fold, (train, test) in enumerate(KFold(5, shuffle=True, random_state=SEED).spl
it(x)):
        np.testing.assert_array_equal(arrays["check_test_rows"][fold], test)
        assert frame.iloc[test].fold.eq(fold + 1).all()
        truth = y.iloc[test].to_numpy()
        baseline = ((truth - frame.iloc[test][targets].to_numpy()) ** 2).mean(axis=0)
        for k, feature in enumerate(arrays["check_features"]):
            j = list(x).index(feature)
            independent = ((arrays["check_predictions"][fold, :, k] - truth) ** 2).mea
n(axis=1) - baseline
            close(independent, delta[fold, :, j])
        corr = x.iloc[train].corr(method="spearman").fillna(0).to_numpy()
        _, labels = connected_components(np.abs(corr) >= .7, directed=False)
        independent_groups = {frozenset(x.columns[labels == label]) for label in set(l
abels)}
        recorded_groups = {frozenset(r.split(" | ")) for r in membership.loc[membershi
p.fold.eq(fold+1), "members"]}
        assert independent_groups == recorded_groups
    top = pd.read_csv(perm / "top3.csv")
    for target in targets:
        eligible = importance.loc[importance.sensory_target.eq(target) & importance.ch
emical_candidate].copy()
        order = {feature: i for i, feature in enumerate(x.columns)}
        eligible["order"] = eligible.feature.map(order)
        expected = eligible.sort_values(["delta_mse", "order"], ascending=[False, True
]).head(3).feature.tolist()
        actual = top.loc[top.sensory_target.eq(target)].sort_values("candidate_rank").
feature.tolist()
        assert actual == expected
    checks.append({"name": "held_out_permutation_metrics_and_training_modules", "featu
re_target_pairs": 10395,
                   "folds": 5, "repeats": 5, "independent_raw_feature_checks": 75})
    for source in [q1 / "manifest.json", q2 / "summary.json", perm / "summary.json"]:
        record = json.loads(source.read_text(encoding="utf-8"))
        hashes = record.get("input_sha256", record.get("input_sha256_before", {}))
        for name, expected in hashes.items():
            path = ROOT.parent / name if "/" in name else data_dir() / name
            assert hashlib.sha256(path.read_bytes()).hexdigest() == expected, name
    report = {"status": "PASS", "checks": checks,
              "scope": "Saved supplementary calculations, fold separation and source i
ntegrity; no causal or independent-population validation."}
    (HERE / "verification.json").write_text(json.dumps(report, ensure_ascii=False, ind
ent=2) + "\n", encoding="utf-8")
    print(json.dumps(report, ensure_ascii=False, indent=2))


if __name__ == "__main__":
    main()
```